The sTeX4 Package Collection*

Michael Kohlhase, Dennis Müller
FAU Erlangen-Nürnberg
http://kwarc.info/

2026-03-24

Contents

1 Introduction & Setup **8**
1.1 What is sTeX? . . . 8
1.2 The sTeX package . . . 9
1.3 Math archives and the MathHub Directory . . . 9
1.4 Setting Up the FLAMI IDE . . . 9

I Tutorial **11**

2 The Basics **12**
2.1 Text symbols . . . 15
2.1.1 Using Modules & Search in the IDE . . . 15
2.2 Symbol References . . . 17
2.3 Modules and Simple Symbol Declarations . . . 20
2.4 Documenting Symbols . . . 23
2.5 Sectioning and Reusing Document Fragments . . . 25
2.6 Building and Exporting HTML . . . 27

3 Mathematical Concepts **30**
3.1 Simple Symbol Declarations . . . 30
3.1.1 Semantic Macros and Notations . . . 30
3.1.2 Types and Variables . . . 33
3.1.3 Flexary Macros and Argument Modes . . . 35
3.1.4 Precedences . . . 37
3.1.5 Finishing Equality . . . 38
3.1.6 Variable Sequences . . . 38
3.2 Statements . . . 38
3.2.1 Definitions . . . 39
Semantic Macros in Text Mode . . . 40
Definientia . . . 41

*Version 4.0.0 (last revised 2026-03-24)

Using Symbols Without Semantic Macros and Exporting Code in Modules . . . 42
3.2.2 Assertions . . . 43
3.2.3 Proofs . . . 45
3.3 Mathematical Structures . . . 45
3.3.1 Declaring and Using Structures . . . 45
Instantiating Structures . . . 46
3.3.2 Extending Structures and Axioms . . . 48
Conservative Extensions . . . 49
3.3.3 Nesting Structures and `\this` . . . 50
3.4 Complex Inheritance and Theory Morphisms . . . 52
3.4.1 Glueing Structures Together . . . 55
3.4.2 Realizations . . . 56

4 Extensions for Education **59**
4.1 Slides and Course Notes . . . 59
4.1.1 Introduction . . . 59
4.1.2 Package Options . . . 60
4.1.3 Notes and Slides . . . 60
4.1.4 Managing and Customizing Themes . . . 61
4.1.5 Frame Images . . . 62
4.1.6 Ending Documents Prematurely . . . 63
4.1.7 Global Document Variables . . . 63
4.1.8 Excursions . . . 63
4.2 Problems and Exercises . . . 64
4.2.1 Background . . . 64
4.2.2 The Package, Options, and Configuration . . . 65
4.2.3 (Open) Problems . . . 66
4.2.4 Solutions and Answer Classes . . . 67
4.2.5 Grading Support . . . 68
4.2.6 Structured Problems . . . 69
4.2.7 Single/Multiple Choice Blocks . . . 70
4.2.8 Filling-In Concrete Solutions . . . 73
4.2.9 Including Problems . . . 75
4.2.10 Testing and Spacing . . . 75
4.3 Homework Assignments and Exams . . . 76
4.3.1 Introduction . . . 76
4.3.2 Package Options . . . 76
4.3.3 Assignments . . . 76
4.3.4 Including Assignments . . . 76
4.3.5 Typesetting Exams . . . 77

II User Manual **78**

5 Basics **79**
5.1 Package and Class Options . . . 79
5.2 Math Archives and the `MathHub` Directory . . . 80
5.2.1 The Structure of Math Archives . . . 81
5.2.2 MANIFEST.MF-Files . . . 81
5.3 The `lib`-Directory . . . 82
5.4 Basic Macros . . . 83

6 Document Features **84**
6.1 Document Fragments . . . 84
6.2 Using and Referencing Document Fragments . . . 85
6.3 Cross-Document References . . . 85

7 Modules and Symbols **88**
7.1 Modules . . . 88
7.1.1 Signature Modules, Languages, and Multilinguality . . . 89
7.2 Symbol Declarations . . . 89
7.2.1 Returns . . . 90
7.3 Referencing Symbols . . . 90
7.4 Notations and Semantic Macros . . . 92
7.4.1 Precedences and Bracketing . . . 93
7.4.2 Notations for Argument Sequences . . . 94
7.4.3 Semantic Macros . . . 94
7.5 Simple Inheritance . . . 95
7.6 Variables and Sequences . . . 97
7.7 Structures . . . 98
7.7.1 Semantic Macros for Structures . . . 99

8 Statements **101**
8.1 More on Definitions . . . 102
8.2 More on Assertions . . . 103

9 Customizing Typesetting **104**
9.1 Highlighting Symbol References . . . 104
9.2 Styling Environments and Macros . . . 105
9.3 Custom CSS for Environments . . . 107

10 Additional Packages **108**
10.1 NotesSlides Manual . . . 108
10.1.1 Introduction . . . 108
10.1.2 Package Options . . . 108
10.1.3 Notes and Slides . . . 109
10.1.4 Customizing Header and Footer Lines . . . 110
10.1.5 Frame Images . . . 110
10.1.6 Ending Documents Prematurely . . . 111
10.1.7 Global Document Variables . . . 111
10.1.8 Excursions . . . 112
10.2 Problem Manual . . . 113
10.2.1 Introduction . . . 113
10.2.2 Problems and Solutions . . . 113

10.2.3 Markup for Added-Value Services . . . 115
Multiple Choice Blocks . . . 115
Filling-In Concrete Solutions . . . 116
10.2.4 Including Problems . . . 117
10.2.5 Testing and Spacing . . . 118
10.3 HWExam Manual . . . 118
10.3.1 Introduction . . . 118
10.3.2 Package Options . . . 118
10.3.3 Assignments . . . 119
10.3.4 Including Assignments . . . 119
10.3.5 Typesetting Exams . . . 119
10.4 Tikzinput Manual . . . 120

III Documentation 122

11 Utilities 123
11.1 `kpsewhich` and Environment Variables . . . 123
11.2 Logging . . . 124
11.3 File Paths . . . 124
11.4 Group-like Behaviours . . . 125
11.5 Key Handling . . . 126
11.6 Languages . . . 127
11.7 Styling . . . 127
11.8 Persisting Dependencies . . . 127

12 HTML Output 128

13 Math Archives 132

14 URIs 134
14.1 Document URIs . . . 134
14.2 Module URIs . . . 135
14.3 Symbol URIs . . . 136

15 Documents 138
15.1 Sectioning . . . 139
15.2 Inter-Document References . . . 140
15.3 Inputting From MathHub Resources . . . 140

16 Modules 142
16.1 SMS Mode . . . 143
16.2 Inheritance and Morphisms . . . 145
16.3 Theory Morphisms . . . 145

17 Symbols **147**
17.1 Declarations . . . 147
17.2 Retrieval . . . 148
17.3 Variables . . . 148
17.3.1 Variable Sequences . . . 149
17.4 Expressions . . . 149
17.4.1 Term Annotations . . . 150
17.4.2 Symbol Arguments . . . 151
17.4.3 Notation components . . . 151
17.4.4 Symbol References . . . 152
17.5 Checking . . . 152

18 Notations **153**
18.1 Automated Bracketing . . . 155

19 Structures **156**

20 Statements **157**

21 Proofs **158**

22 Metatheory **160**

23 Others **161**

24 Additional Packages **162**
24.1 NotesSlides Documentation . . . 162
24.2 Problem Documentation . . . 162
24.3 HWExam Documentation . . . 162
24.4 Tikzinput Documentation . . . 162

IV Implementation **163**

25 Setting up **164**

26 Utilities **166**
26.1 `kpsewhich` and Environment Variables . . . 167
26.2 Logging . . . 168
26.3 File Paths . . . 169
26.4 Group-like Behaviours . . . 173
26.5 Key Handling . . . 174
26.6 Languages . . . 176
26.7 Styling . . . 177
26.8 Persisting Dependencies . . . 179
26.9 Auxiliary Methods . . . 180

27 HTML Output **182**

28 Math Archives **187**

29 URIs **194**
29.1 Document URIs . . . 194
29.2 Module URIs . . . 197
29.3 Symbol URIs . . . 201

30 Documents **204**
30.1 Sectioning . . . 206
30.2 Inter-Document References . . . 211
30.3 Inputting From MathHub Resources . . . 216

31 Modules **222**
31.1 SMS Mode . . . 230
31.2 Inheritance and Morphisms . . . 234
31.3 Theory Morphisms . . . 239

32 Symbols **254**
32.1 Declarations . . . 254
32.2 Retrieval . . . 262
32.3 Variables . . . 266
32.3.1 Variable Sequences . . . 269
32.4 Expressions . . . 271
32.4.1 Term Annotations . . . 291
32.4.2 Symbol Arguments . . . 293
32.4.3 Notation components . . . 298
32.4.4 Symbol References . . . 300
32.5 Checking . . . 305

33 Notations **307**
33.1 Automated Bracketing . . . 322

34 Structures **324**

35 Statements **329**

36 Proofs **336**

37 Metatheory **349**

38 Others **352**

39 Additional Packages **354**
39.1 Implementation: The notesslides Package . . . 354
39.1.1 Class and Package Options . . . 354
39.1.2 Notes and Slides . . . 358
39.1.3 Environment and Macro Patches . . . 362
39.1.4 Styling Across Notes/Slides . . . 363
39.1.5 Beamer Compatibility . . . 363
39.1.6 TODO Excursions . . . 365
39.2 Implementation: The problem Package . . . 367
39.2.1 Package Options . . . 367
39.2.2 Problems and Solutions . . . 369

39.3 Implementation: The hwexam Package 387
39.3.1 Package Options 387
39.3.2 QR Codes 388
39.3.3 Assignments 393
39.3.4 Leftovers 397
39.4 Tikzinput Implementation 398

Index **400**

sTeX is – by now – relatively stable and ready to use for the general public. However, it is also actively being developed further and subject to ongoing research. Some of the features described in here might not fully work as expected, some are still experimental, there might occasionally be cryptic error messages, and in general bugs are expected.
We welcome all kinds of issues you might encounter at `https://github.com/slatex/sTeX`.

If you have questions or problems with sTeX, you can talk to us directly at `https://matrix.to/#/#stex:fau.de`.

Chapter 1

Introduction & Setup

1.1 What is sTeX?

sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

At its core is the sTeX package for LaTeX, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over sTeX-annotated documents formats them into normal-looking PDF.

But sTeX also comes with a conversion pipeline into semantically annotated HTML (FTML), which can host semantic added-value services that make the documents *active* (i.e. interactive and user-adaptive) – essentially turning LaTeX into a document format for (mathematical) knowledge management (MKM).

The sTeX system consists of the following components:

- The sTeX package,
- the RuSTeX system for converting LaTeX documents to (semantically enriched) FTML,
- the FLAMS system; a semantically informed back-end for managing and hosting the FTML with integrated added-value services,
- a collection of math archives of sTeX document fragments and symbols for reuse, available of `mathhub.info`, and
- the FLAMS IDE: A VS Code extension that, besides the usual IDE functionalities like syntax highlighting, integrates FLAMS and allows for accessing the public math archives on `mathhub.info` to make the entire toolchain easily accessible to authors.

If PDF is all you are interested in, the sTeX package is all you *need*. Either way, however, we recommend using the sTeX IDE – it very much helps with semantically annotating your LaTeX documents.

1.2 The sTeX package

The sTeX package extends LaTeX with:

- A mechanism to declare symbols – concepts, functions, relations, variables, etc., which can be used and referenced in text or via semantic macros in mathematical formulae,
- a module system based on logical identifiers – modules bundle declarations, definitions, theorems, document snippets, and symbols for reuse, and
- an analogous organizational structure for developing documents modularly from individual fragments and sections.

The sTeX package has been designed to have minimal impact on other packages and document classes, or the actual document layout – formatting of semantic environments, symbol references and semantic macros can be fully customized.

1.3 Math archives and the MathHub Directory

To make the most of sTeX, it is strongly encouraged to follow a workflow of small document fragments and modules to maximize reuse.

One considerable weakness of LaTeX is the way source files are referenced: they need to be either in a `texmf` directory, or else be referenced via file paths relative to the main `.tex`-file being compiled. This is highly inconvenient if we want to collaboratively develop many highly interrelated document fragments.

sTeX therefore adds an organizational layer on top of LaTeX's: **math archives** stored in a fixed `MathHub` directory anywhere on your hard drive. Referencing source files and modules is then done relative to the containing math archive, and is thus *independent* of user's individual setups or the current `.tex`-file.

The drawback of this approach is that sTeX needs to know the location of your `MathHub` directory. There are multiple ways to achieve that, but the simplest and recommended approach is to set an environment variable: Simply create a new directory `<path>/MathHub` somewhere on your hard drive and set the environment variable `MATHHUB` as the path to this new directory.

Alternatively, you can let the FLAMS IDE do the work for you (see section 1.4).

For more on math archives, see Section 1 (Math Archives and the `MathHub` Directory) in the sTeX Manual.

1.4 Setting Up the FLAMS IDE

sTeX is based on LaTeX, and adds additional layers of presentational and functional markup to it. As a consequence the source files of sTeX documents look quite different from the resulting FTML and `PDF` documents. Thus the best way of interacting the sTeX document collections is via an integrated development environment (IDE). In this tutorial we will use the FLAMS plugin for the VS Code, which you should set up as a first step (this also sets up the necessary auxiliary software).

Setting up sTeX with the dedicated IDE is easy:

1. Download and install VS Code here: `https://code.visualstudio.com/download`

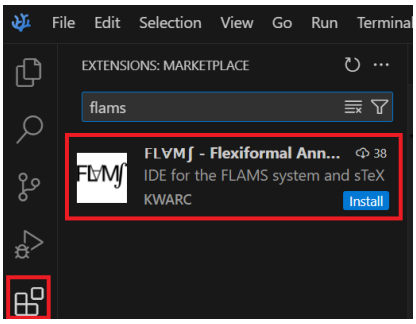

Figure 1: Installing the FL∀M∫ IDE

2. Start VS Code and navigate to the *Extensions*-tab on the left. Here you can search for Extensions in the VS Code marketplace. Look for the FL∀M∫ extension by *KWARC*, as in Figure 1.

3. Having done so, upon opening any folder in VS Code containing a `.tex`-file, you will be prompted to either download a new FL∀M∫ or point the extension to an existing FL∀M∫ executable. In the vast majority of cases, you'll want to download and choose a directory to install FL∀M∫ to.

Part I
Tutorial

The dynamic HTML version of this part can be found at
https://stexmmt.mathhub.info/:
sTeX/fullhtml?archive=sTeX/Documentation&filepath=tutorial.en.xhtml

sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

In this part, we will give a broad but shallow introduction to sTeX, and what you can get out of it. Additionally, this serves as an introduction to the FLAMI IDE, and we consequently assume that you have that one set up, as described in section 1.4.

Note that in PDFs, the specific highlighting of semantically annotated text is fully customizable (see chapter 9). In this document, we use this highlighting for notation components, this highlighting for symbol references, this highlighting for (local) variables and **this highlighting** for definienda; i.e. new concepts being introduced.

Chapter 2

The Basics

This document itself uses sTeX and serves as a direct example for the following. You can download its source files, the generated PDF files, and the generated HTML documents directly from within the IDE, by navigating to the FLAMS tab in the menu on the left and finding `sTeX/Documentation` in the list of math archives and clicking the small "Install"-button next to it, see the screenshot on the left of Figure 2.

Once downloading is finished (this may take a while since dependencies are also downloaded), you can then browse the `.tex`-files in `sTeX/Documentation` directly from the math archives panel in the sTeX tab, as you can see in the right screenshot in Figure 2.

For example, you can now navigate to the file `tutorial/intro.en` to see the sources of this very chapter.

As a first example, consider the following document fragment from section 1.1:

> sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

If you were to look at the generated HTML from this fragment, you could hover over the highlighted words (sTeX, PDF, HTML, OMDoc) and get a little popup with their definitions (Figure 3). Neat, huh?

Here, in the PDF, hovering will only show you a unique identifier (MMT-URI) for the word, and link to a definition on the web. Still useful, but not quite as neat, of course.

A plain LaTeX-version of the above document fragment, without any sTeX markup, could look like this:

Example 1
Input:

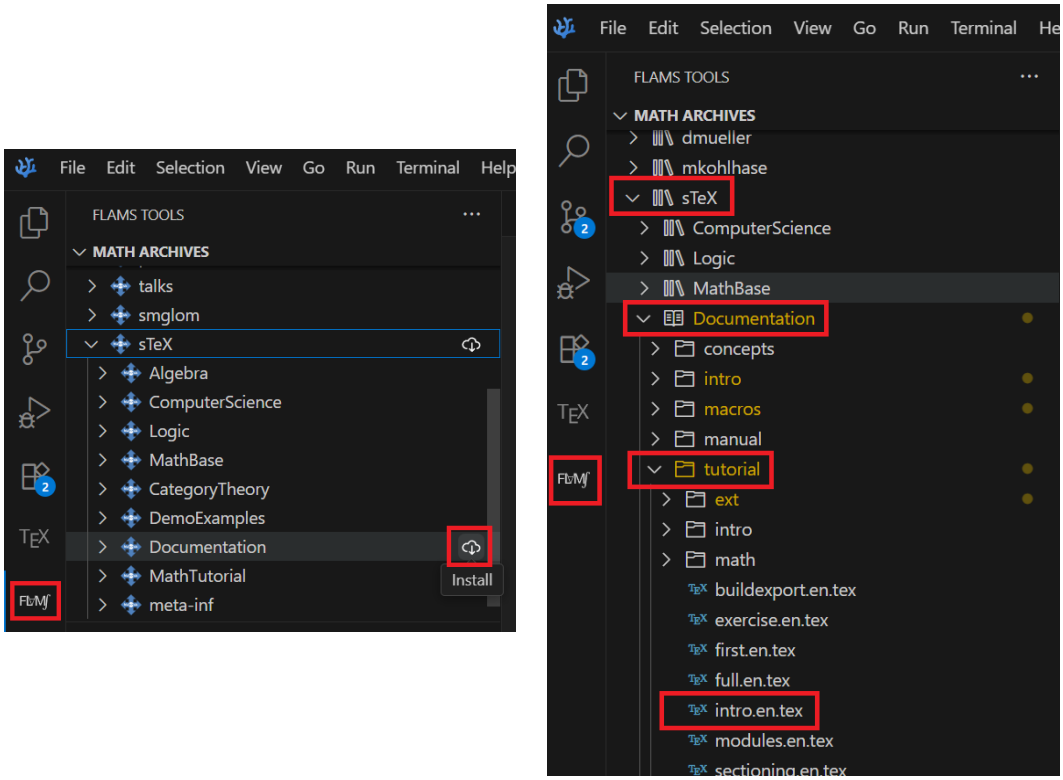

Figure 2: Installing Math Archives

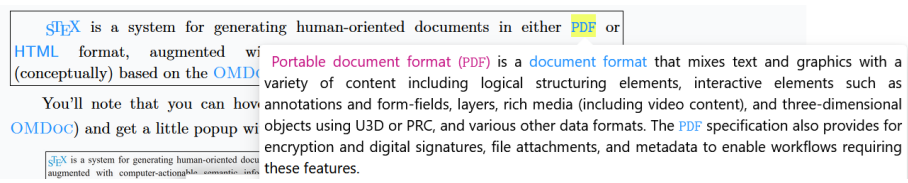

Figure 3: Definition on Hover

File `tutorial/intro/intro1plain.en.tex`

```
\documentclass{article}
\usepackage{stex-logo}
\begin{document}

  \sTeX{} is a system for generating human-oriented documents
  in either \textsf{PDF} or \textsf{HTML} format, augmented
  with computer-actionable semantic information (conceptually)
  based on the \textsc{OMDoc} format and ontology.

\end{document}
```

Output:

> sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

(Examples like the one above always show the file the source code is in, so if you have downloaded the `sTeX/Documentation` math archive you can toy around with it yourself)

If you save a file in the IDE (regardless of whether it has unsaved changes), a preview window will pop up, showing you the HTML generated from the `.tex`-file; see (Figure 4).

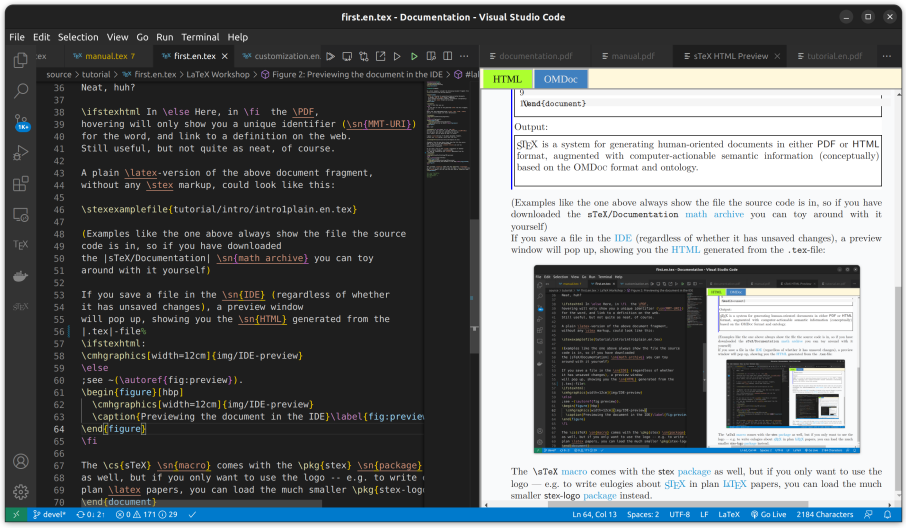

Figure 4: Previewing the document in the IDE

The `\sTeX` macro comes with the stex package as well, but if you only want to use the logo – e.g. to write eulogies about sTeX in plain LaTeX papers, you can load the much smaller stex-logo package instead.

2.1 Text symbols

The most central concept behind sTeX is that of a *symbol*:

> sTeX A **symbol** is a *named* concept that can be defined, documented and referenced. Examples for symbols are mathematical constants, functions, theorems, statements, principles – anything that has a (somewhat) precise meaning and can be referenced by name can be a symbol.

Before we explain how we can declare new symbols and associate them with definitions, notations and all that, let's assume an ideal world in which others have done that job already for us – after all, sTeX is all about *reuse*, and naturally, there are sTeX symbols for all of the above already. Let's start with the one for sTeX itself:

2.1.1 Using Modules & Search in the IDE

In the VS Code IDE, navigate to the FLAMS-tab on the left. In the search panel, select the "`Symbols`" radio button and search for "`sTeX`". The third search result should be what we're looking for (Figure 5).

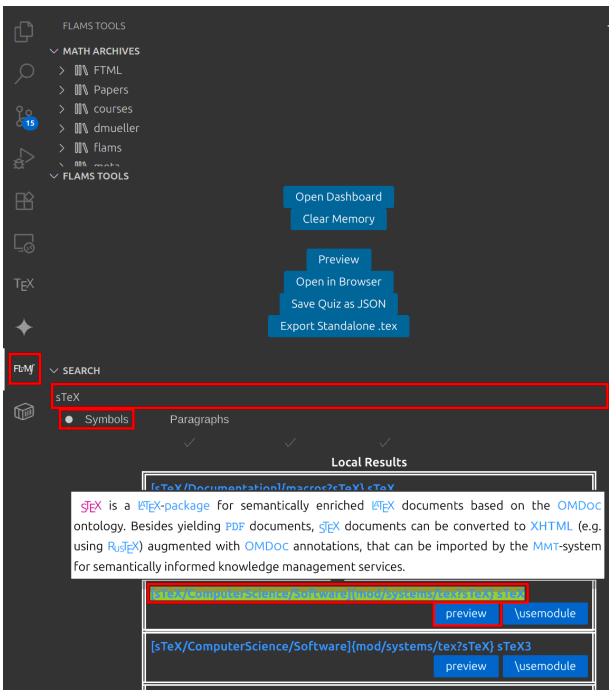

Figure 5: Search in the sTeX IDE

Search results are grouped into *local* and *remote* results. Local ones are the ones you already have in your local `MathHub` directory; remote ones you can download directly from within the IDE.

You can click the preview button to see the generated FTML for the document – the resulting window that pops up also has an OMDoc button you can click, which (among other things) shows you the semantic macros provided by the respective module: In this case, it tells us that there is a *text symbol* named "`sTeX`" with semantic macro `\stex` in the module `sTeX` in the same document. It produces the presentation "sTEX" as we want (Figure 6).

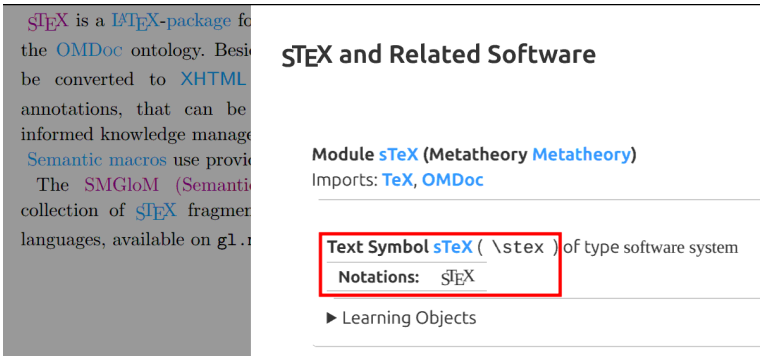

Figure 6: OMDoc Preview

> sTEX A **text symbol** is a symbol foo with an associated semantic macro `\foo`. The macro `\foo` is allowed in text or math mode and produces a predefined piece of text output annotated with foo.
> The variant `\fooname` produces the same output without annotation.

If we want to use the sTEX symbol in a document – which we have open in the IDE – we simply click on the `\usemodule` button, and the IDE will automatically insert the line
`\usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}`,
making all symbols in that module available to use – in particular, we can now use the `\stex` semantic macro instead of the plain, non-semantic `\sTeX` macro – that is, of course, after we include the `stex` package first.

> sTEX The `\usemodule` macro takes as *optional* argument the name of a math archive, and as a regular argument the path to an sTEX module (see section 7.5).

Analogously, we can also search for the PDF, HTML and OMDoc symbols, all of which are also text symbols and have the associated semantic macros `\PDF`, `\HTML` and `\omdoc`; the document should thus look like this:

Example 2
Input:

File tutorial/intro/intro1stex.en.tex

```
\documentclass{article}
\usepackage{stex}
\begin{document}
  \usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}
  \usemodule[sTeX/ComputerScience/Software]{mod/formats?PDF}
  \usemodule[sTeX/ComputerScience/Software]{mod/formats?HTML}
  \usemodule[sTeX/ComputerScience/Software]{mod/formats?OMDoc}

  \stex is a system for generating human-oriented documents
  in either \PDF or \HTML format, augmented
  with computer-actionable semantic information (conceptually)
  based on the \omdoc format and ontology.
\end{document}
```

Output:

sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

Now, our generated HTML looks a lot more interesting, with highlighting, pop-ups on hover and all that. Notably however, if we compile the file with `pdflatex`, it looks pretty much exactly as before.

That's because we haven't told sTeX what to do with semantic annotations yet – and by default, it does not do anything fancy, except for wrapping them in an `\emph`. We can customize how we want sTeX to highlight various semantic text fragments (see chapter 9). A default highlighting schema is provided in the stex-highlighting package – including that will

- highlight semantically annotated text in this color,
- show the MMT-URI of the corresponding symbol in a tooltip on hovering over the text,
- make the text link to the place the symbol is being defined in the current document (if it is), or, alternatively,
- make it link to an external resource, if one is known. In our case, they link to `stexmmt.mathhub.info/:sTeX`, where the HTML for all the symbols we use in this document are hosted.

2.2 Symbol References

Let's continue with the next paragraph of section 1.1; for now unannotated:

Example 3
Input:

File tutorial/intro/intro2plain.en.tex

```
\documentclass{article}
\usepackage{stex}
\begin{document}

  At its core is the \sTeX{} package for \LaTeX, that allows for
  semantically marking up document fragments; in particular
  concepts, formulae and mathematical statements (such as
  definitions, theorems and proofs). Running \texttt{pdflatex}
  over \sTeX-annotated documents formats them into normal-looking
  \textsf{PDF}.

\end{document}
```

Output:

At its core is the sTeX package for LaTeX, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over sTeX-annotated documents formats them into normal-looking PDF.

We already know how to annotate "sTeX" and "PDF"; and if we use the search field in the IDE again, we can also find a text symbol for "LaTeX". But if we look at the documentation, we will note that *more* is highlighted:

At its core is the sTeX package for LaTeX, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over sTeX-annotated documents formats them into normal-looking PDF.

The "`package`"-symbol can be found in the LaTeX module too, and searching for the keywords "formula" and "mathematics" will yield the symbols "`well-formed formula`" and "`mathematics`", but they are not *text symbols* and "`mathematics`" and "`package`" do not even have a semantic macro – and the one for "`well-formed formula`" would not work outside of math mode.

Text symbols are special in that way – they are intended for symbols that have a specific formatting associated (such as LaTeX, OMDoc, or HTML, which we prefer to typeset as sans serif). For those settings, it makes sense to associate that formatting with a semantic macro that does the typesetting for us.

Symbols *without* a text macro can be referenced with the `\symname` macro: `\symname{package}` prints the *name* of the "`package`"-symbol and annotates it accordingly, without any special formatting – in particular it is compatible with being in `\emph`, `\textbf` and similar macros. That takes care of *one* of the missing annotations.

More generally, the `\symref` macro can be used to annotate arbitrary text with a symbol: `\symref{mathematics}{mathematical}` associates the text `mathematical` with the symbol "`mathematics`"; thus, we get "mathematical" and similarly "formulae".

In general, any macro that expects a symbol name can be given either

1. the *name* of the symbol,
2. the name of its semantic macro,
3. or any suffix of its MMT-URI containing at least the module name.

The second option is often short – and therefore convenient to write; for example, to achieve "formulae", we can also write `\symref{wff}{formulae}`, since `\wff` is the semantic macro for "`well-formed formula`".
The third option allows for distinguishing between multiple symbols with the same name – the IDE can help in the latter case, by underlining ambiguous symbol references in yellow, and offering the `Quick Fix` functionality to let you select and autocomplete the specific symbol you want to reference.

Since `\symname` and `\symref` are a lot to type for something that should ideally be used as often as possible, the macros `\sn` and `\sr` exist as well and behave exactly the same way. We also provide some convenience abbreviations for `\sn`; namely `\Sn` (capitalizes the first letter of the symbol name), `\sns` (adds an "`s`" at the end, for the most common pluralization of a name), and `\Sns` (both).

Using all of the above, our annotated fragment now looks like this:

Example 4

Input:

File `tutorial/intro/intro2stex.en.tex`

```
\usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}
\usemodule[sTeX/Logic/General]{mod/syntax?Formula}
\usemodule[sTeX/MathBase/General]{mod?Mathematics}
\usemodule[sTeX/ComputerScience/Software]{mod/formats?PDF}

  At its core is the \stex \sn{package} for \latex, that allows for
  semantically marking up document fragments; in particular
  concepts, \sr{wff}{formulae} and \sr{mathematics}{mathematical}
  statements (such as definitions, theorems and proofs). Running
  \texttt{pdflatex} over \stex-annotated documents formats them
  into normal-looking \PDF.
```

Output:

At its core is the sTeX package for LaTeX, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over sTeX-annotated documents formats them into normal-looking `PDF`.

There's only one problem: *the document does not compile*, with an error `Undefined control sequence`. The reason being that *some* macro in the module `Formula` uses the `\text` macro. We can fix that by using the `amsfonts` package of course, but this points to a more general problem; namely that modules can make use of various LaTeX packages for typesetting symbols.

Good practice suggests putting those packages into a *prelude* per math archive, which we can then import from anywhere, using the `\libinput` macro. For more on that, see

section 5.3.

For now, suffice it to say that we can import all packages required for the module `Formula` from the math archive `sTeX/Logic/General` by adding the line

```
\libinput[sTeX/Logic/General]{preamble}
```

before the `\begin{document}`.

2.3 Modules and Simple Symbol Declarations

Consider again the first two paragraphs of section 1.1:

> sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.
>
> At its core is the sTeX package for LaTeX, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over sTeX-annotated documents formats them into normal-looking PDF.

Firstly, note that the first paragraph would be perfectly suitable to serve as a pop-up definition on hover for the sTeX symbol. Secondly, what if all the symbols used in the above *didn't* already exist?

In this section, we will describe how to make your own symbols and collect them as reusable fragments in modules and math archives from scratch.

We start by creating a new math archive. In the IDE, switch to the sTeX-tab on the left and click the "`New sTeX Archive`" button (Figure 7). You will then be asked for

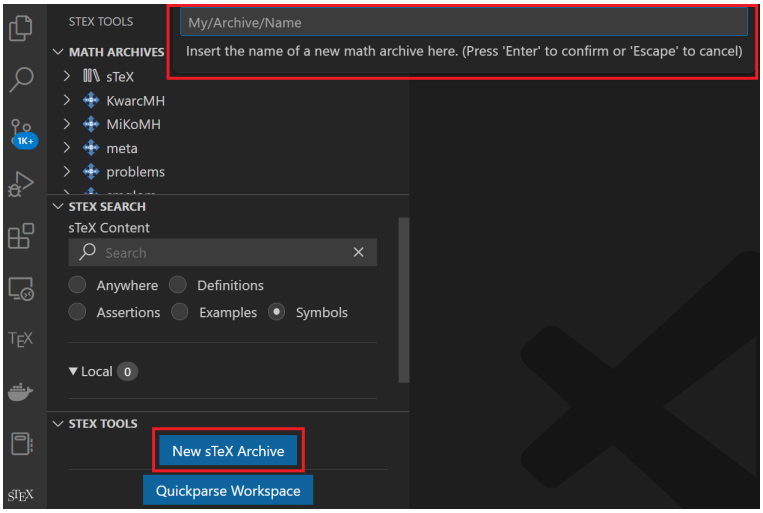

Figure 7: New Math Archive in the IDE

the name of the archive, and a url-base, where the content is supposedly going to end up

online. You can safely keep the defaults for the latter. In the following, we assume that your archive is named `my/archive`.

The IDE will then create the following files and directories in your MathHub directory:

```
- my
  - archive
    - lib
      - preamble.tex
    - META-INF
      - MANIFEST.MF
    - source
      - helloworld.tex
```

...and open the file `helloworld.tex` with the content

```
\documentclass{stex}
\libinput{preamble}
\begin{document}
% A first sTeX document
\end{document}
```

You can now reference any newly created content in you new archive using for example `\usemodule[my/archive]{...}`.

Let's start with the "LaTeX" symbol. Rename the file `helloworld.tex` to something more meaningful, for example `latex.en.tex` – the `.en` will be picked up on by sTeX to signify that the fragment will be in *english* (see subsection 7.1.1).

What we want to achieve in this file is the following:

> **TeX** is a document typesetting software developed by Donald Knuth, with a focus on mathematical formulae. It is based on a powerful and extensible **macro** expansion engine.
>
> **LaTeX** is a (nowadays) default collection of TeX macros developed by Leslie Lamport. Among other things, LaTeX introduces **environments**, a distinction between preamble and document content, **packages** to bundle and distribute macro definitions, and **document classes**: special packages that govern the global layout of a document.

In particular, in the HTML the two paragraphs above should be shown when hovering over the symbols they define (as indicated by the magenta definiendum highlighting). So we need symbols and semantic macros, for: TeX, macro, LaTeX, environment, package and document class.

Symbol declarations are only allowed within modules:

> sTeX A **module** is a *named* block that bundles symbol declarations for subsequent reuse. A module is introduced with the `smodule`-environment.

Let's name our module `LaTeX`. We then wrap the contents of our document in a `smodule` environment:

```
\begin{document}
  \begin{smodule}{LaTeX}
    ...
  \end{smodule}
\end{document}
```

FLAMS Note, that the IDE immediately picks up on this and displays the full FTML URI of our new module over the `\begin{smodule}{LaTeX}` (Figure 8) –

Figure 8: VS Code URI on Hover

From this, we can glimpse that the namespace of the module is `http://mathhub.info?a=my/archive&p=latex&m=LaTeX`. This implies, that to use the module somewhere else, we will have to type `\usemodule[my/archive]{latex?LaTeX}` – the `latex`-part pointing to the *file* and `LaTeX` referring to the actual module.

If we rename the file to `LaTeX.en.tex`, we notice that the namespace changes to `http://mathhub.info?a=my/archive&m=LaTeX`, allowing us to now use it with `\usemodule[my/archive]{LaTeX}` directly. That's because the module name `LaTeX` and the file name `LaTeX` match now (see section 7.5, Figure 9).

Figure 9: VS Code URI on Hover

Note that "`LaTeX`" and "`latex`" only differ in capitalization – if your file system is case-insensitive (as e.g. MacOS's was until quite recently), this distinction gets murky, but remains very important especially if you want to share your math archive with others!
It is therefore *highly recommended* to treat file names as case-sensitive either way.

Within the module, we can now declare new symbols using the `\symdecl`-macro. We start with those that are not text symbols:

```
\symdecl*{macro}
\symdecl*{environment}
\symdecl*{package}
\symdecl*{document class}
```

The `*` after the `\symdecl` indicates, that we do not want a semantic macro for the symbol – otherwise, it would generate one with the same name as the symbol itself and "pollute the macro space", so to speak.

The symbols TEX and LATEX, however, have a definite way of being typeset associated with them, which can be produced using the standard `\TeX` and `\LaTeX` macros. So let's make them text symbols, using the `\textsymdecl` macro:

```
\textsymdecl{tex}{\TeX}
\textsymdecl{latex}{\LaTeX}
```

The first argument being the name of the generated macro (i.e. `\tex` and `\latex`) and the second one specifying the output to produce.

2.4 Documenting Symbols

We can now use the two new macros, `\symname`/`\sn`, `\symref`/`\sr` etc. to mark up the above two paragraphs. However, the system does not yet know what to show a reader when hovering over the symbol in the HTML. We can fix that by using the `sdefinition` or `sparagraph` environments.

Ignoring the former for now, which is more useful for mathematical concepts, we can use the following to mark up the first paragraph:

```
\begin{sparagraph}[style=symdoc,for={tex,macro}]
  \tex is a document typesetting
  software developed by Donald Knuth, with a focus on
  mathematical formulae. It is based on a powerful
  and extensible \sn{macro} expansion engine.
\end{sparagraph}
```

In general, the `sparagraph` environment can be used to mark up arbitrary paragraphs semantically, but the `style=symdoc` option tells STEX to use this paragraph as a documentation for the symbols provided in the `for=` option.

We just used the semantic macro `\stex` and the `\sn` macro to mark up the fragment – but we can do better. Both concepts are being *introduced* in the above paragraph, and we can let STEX know that that is the case:

Within an `sparagraph` environment with `style=symdoc` (or an `sdefinition` environment), we can mark up *definienda*, meaning the terms *being defined*, explicitly. Analogously to `\symname` and `\symref`, we have the macros `\definame` and `\definiendum` for that purpose.

Note that the `\tex` macro induced by the text symbol above already marks up the "TEX" it produces, so wrapping it in another `\definiendum` would be redundant. However, every text symbol also generates a *second* macro with the suffix `name` that generates a non-marked-up version of the same presentation. In other words, we get the macro `\texname` for free, that produces "TEX" (of course, we could just as well use the `\TeX` macro, but that one you probably know already).

Furthermore, every `\definiendum` or `\definame` automatically adds the symbol being referenced to the internal `for=`-list of the `sparagraph` environment, obviating the need to list it explicitly.

As such, we can produce a better markup like this:

```
\begin{sparagraph}[style=symdoc]
  \definiendum{tex}{\texname} is a document typesetting
  software developed by Donald Knuth, with a focus on
  mathematical formulae. It is based on a powerful
  and extensible \definame{macro} expansion engine.
\end{sparagraph}
```

Exercise

In your archive `my/archive`, create additional files that produce the following outputs:

`Mathematics.en.tex`

To do **mathematics** is to be, at once, touched by fire and bound by reason. This is no contradiction. Logic forms a narrow channel through which intuition flows with vastly augmented force.

– Jordan Ellenberg

`PDF.en.tex`

Portable Document Format (PDF) is a document format that mixes text and graphics with a variety of content.

`HTML.en.tex`

The **HyperText Markup Language (HTML)** is a representation format for web-pages.

`OMDoc.en.tex`

OMDoc is a document format for representing mathematical documents with their flexiformal semantics.

such that the following file compiles and shows the above snippets on hover:

`sTeX.en.tex`

```
\documentclass{stex}
\libinput{preamble}
\begin{document}
\begin{smodule}{sTeX}
  \usemodule{OMDoc}
  \usemodule{PDF}
  \usemodule{HTML}
  \textsymdecl{stex}{\sTeX}
  \begin{sparagraph}[style=symdoc]
    \definiendum{stex}{\stexname} is a system for generating
    documents in either \PDF or \HTML format, augmented with
    computer-actionable semantic information (conceptually)
    based on the \OMDoc format and ontology.
  \end{sparagraph}
\end{smodule}
\end{document}
```

sTeX is a system for generating documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

The preamble of every file should only be

```
\documentclass{stex}
\libinput{preamble}
```

and the macros `\OMDoc`, `\PDF`, `\HTML` should produce `\textsc{OMDoc}`, `\textsf{PDF}` and `\textsf{HTML}`, respectively (but with semantic annotations of course).

Lösung: Can be found in `[sTeX/Documentation]source/tutorial/solution`

2.5 Sectioning and Reusing Document Fragments

We know now how to import and reuse the symbols of some module (using `\usemodule`). What about the actual document *content*?

Assume we want to write a new article that includes all of the fragments in `my/archive` we made so far, in a file `all.en.tex` in the same math archive:

```
\documentclass{article}
\usepackage{stex}
\libinput{preamble}
\begin{document}
  \author{Me}
  \title{The \texttt{my/archive} Archive}
  \maketitle
  \tableofcontents
  ...
\end{document}
```

In there, we want sections as follows:

```
- 1 Preliminaries
  (Mathematics)
  - 1.1 Document Formats
    (PDF)
    (HTML)
    (OMDoc)
- 2 \TeX and Friends
  (LaTeX)
  (sTeX)
```

We could of course do the following:

```
\section{Preliminaries}
  \input{Mathematics.en}
  \subsection{Document Formats}
    \input{PDF.en}
    \input{HTML.en}
    \input{OMDoc.en}
\section{\TeX and Friends}
  \input{LaTeX.en}
  \input{sTeX.en}
```

...but this approach has two drawbacks:

Firstly, we need to manually keep track of the section levels, by explicitly writing `\section`, `\subsection` etc. This is fine as long as we are just interested in this particular article. But what if we want to *reuse* the article's content in another document at some point? The section levels might be entirely different then – e.g. we might want the "Preliminaries" section to be a subsection instead.

Secondly, the `\input` macro considers the file name/path provided to be either *absolute* or relative to the *current* `tex` *file being compiled* – which means that the `\input{Mathematics.en}` only works for files in the same directory as `Mathematics.en.tex`.

In short: using `\section`, `\chapter` etc. explicitly, and `\input` to reuse fragments, breaks reusability.

Instead of using `\section` and `\subsection`, sTeX therefore provides the `sfragment` environment.

`\begin{sfragment}{Foo}...\end{sfragment}` inserts a sectioning header depending on the current section level and availability. These are: `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\paragraph` and `\subparagraph`. This allows us to do the following instead:

```
\begin{sfragment}{Preliminaries}
  \input{Mathematics.en}
  \begin{sfragment}{Document Formats}
    \input{PDF.en}
    \input{HTML.en}
    \input{OMDoc.en}
  \end{sfragment}
\end{sfragment}
\begin{sfragment}{\TeX and Friends}
  \input{LaTeX.en}
  \input{sTeX.en}
\end{sfragment}
```

The only problem remaining now is that if we do this, sTeX will insert a `\part` for the first `sfragment`. If we want the "top-level" sectioning level to be `\section` instead, we can insert a `\setsectionlevel{section}` in the preamble.

As a more reuse-friendly replacement of `\input`, sTeX provides the `\inputtref` macro. Using that has two advantages: Firstly, its argument is relative to some (optionally provided, or the current) math archive and is thus independent of the specific location of the file relative to the currently being compiled `.tex`-file. Secondly, when converting to HTML, it will *not* "copy" the referenced file's content in its entirety (as `\input` would), but instead dynamically insert the already existent (if so) HTML of the referenced file, avoiding content duplication and having to process the file all over again.

In general `\inputtref[some/archive]{file/path}` inputs the file `file/path.tex` in the archive `some/archive`. As the `\input`-ed files in the example above are in the same archive anyway, we can simply substitute the `\inputs` by `\inputtref`s and call it a day.

Finally, we can make two more minor changes:

1. The *title* of our document is only supposed to be there, if we compile the document directly – if we were to `\inputtref` our file into a "driver file" `all.en.tex`, the title and the table of contents should be omitted.

   We can achieve this using the `\ifinputtref` conditional: by wrapping the header in an `\ifinputtref` `\else...\fi`, it will only be processed if the file is *not* being loaded using `\inputtref`. `\ifinputtref` is a "classic" TeX conditional and is treated as such in both PDF and HTML compilation. A smarter macro to use is `\IfInputtref`, which takes two arguments for the *true* and *false* cases, respectively.

Additionally, when compiling to HTML, *both* arguments to `\IfInputref` will be processed, and the back-end will decide which of the two to present when serving a document.

2. The table of contents should also be omitted in HTML mode. To achieve that, we can use the `\ifstexhtml` conditional, which is *true* if the document is being compiled to HTML, and *false* if compiled to PDF.

> Note, that since *both* arguments of `\IfInputref` are processed, they should *not* open TEX groups or environments!

In summary, we can modify our document to do the following:

```
\IfInputref{}{
  \author{Me}
  \title{The \texttt{my/archive} Archive}
  \maketitle
  \ifstexhtml \else \tableofcontents \fi
}
```

The final `all.en.tex` can be found in `[sTeX/Documentation]tutorial/solution/all.en.tex`.

2.6 Building and Exporting HTML

So far we know how to write STEX documents, (we assume) how to build PDF files from them (via `pdflatex` of course), and on saving documents the IDE will preview the generated HTML. But if we do that with our new `all.en.tex`, we get presented with Figure 10 Where did all of our fragments go?

Figure 10: Missing Fragments in the HTML Preview

Well, they don't exist yet as HTML. The HTML Preview window in the IDE is really just that: A *preview*. But when using `\inputref`, it has to find the HTML of the `\inputref`ed fragment *somewhere*. Meaning: we have to compile all of the fragments we used to HTML first. Individually, we can compile the currently open file and all its dependencies in VS Code using the button in Figure 11.

This will queue all dependencies of the file in a new build queue and open the FLAM dashboard for us (see Figure 12). Clicking on the `run` button in the dashboard will then for every queued file:

1. Run `pdflatex` twice,

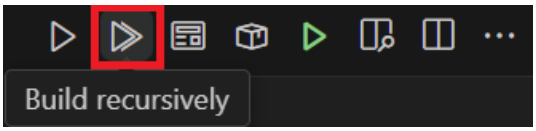

Figure 11: The `Build PDF/XHTML/OMDoc` Button

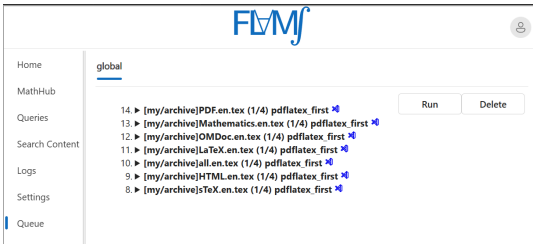

Figure 12: The FLAMS build queue

2. Convert the file to HTML,
3. Extract all the semantics, and
4. Construct a search index.

Once that's done, saving `all.en.tex` again yields the correct HTML in the preview window.

At this point, FLAMS knows about the HTML and the semantics, and we can look at the HTML in VS Code or the FLAMS dashboard – of course, features like the fancy pop-up windows require a semantically informed back-end infrastructure, in the form of the FLAMS system. However, FLAMS *can* dump a standalone and self-contained HTML version for you. Let's do that now:

With our `all.en.tex` file open and everything built as above, click the `Export Standalone HTML`-button in the IDE (see Figure 13).

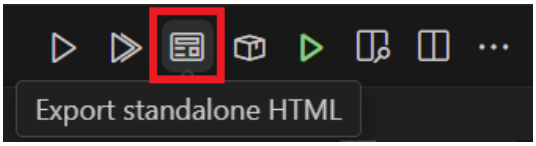

Figure 13: Exporting HTML in the IDE

In the dialog box that opens now, select an **empty** directory and FLAMS will dump a standalone version of our `all.en.tex` document there. You will still not be able to open it in the browser directly without issues, because most browsers forbid javascript modules on the `file://` protocol, but opening the file via `http` will yield the desired result, and you can now upload the directory's content to wherever you might want to use it.

If you want to test this, a quick and easy way to do so is to use VS Code: You can install the `Live Server` extension, open the directory and click the `Go Live` button on the lower right of the window, which will start a small web-server in the selected directory and open its `index.html` in the browser for you.

Chapter 3

Mathematical Concepts

So far, we have seen how to declare and reference symbols generate semantic macros for text symbols, collect them in modules and document them properly.

But where sTeX really shines is when it comes to mathematics and related subject areas: semantic macros are significantly more useful when used for generating symbolic notations in math mode, and by associating symbols with (flexi-)formal semantics, sTeX can even *check* that your content is (to some degree) formally correct, or at least well-formed.

Alos sTeX provides specialized functionality for mathematical statements: the text fragments marked as Definition, Theorem, Proof that are iconic to mathematical documents.

The example snippets in this chapter can be found in the math archive `sTeX/MathTutorial`. If you downloaded the `sTeX/Documentation` archive in the sTeX IDE, you already have that archive. If not, you can download it from within the IDE, as described in Part 1 (The Basics) in the sTeX tutorial.

3.1 Simple Symbol Declarations

We will start with symbols and semantic macros for mathematical concepts and objects and their contribution to mathematical formulae.

3.1.1 Semantic Macros and Notations

Let us start with a very fundamental concept; namely equality. As you should by now know, declaring a new symbol requires a module, so let's open a new one and use `\symdecl`:

```
\begin{smodule}{Equality}
  \symdecl{equal}
\end{smodule}
```

As mentioned in Section 1 (Modules and Simple Symbol Declarations), the starred variant `\symdecl*` does not create a semantic macro, so presumably, the variant without a `*` *does*. And indeed, we now have a macro `\equal`, which however will produce errors if we try to use it. That's because we haven't told sTeX what to do with it yet.

A **semantic macro** is a LaTeX-macro that allows for referencing a symbol itself, or – in the case of e.g. a function – the *application* of a symbol to (one or multiple) *arguments*; primarily by invoking a symbol's notation in *math mode*.
The command `\symdecl{macroname}` declares a new symbol with name `macroname` and a semantic macro `\macroname`. In the case where we want the name and the semantic macro to be distinct, the command `\symdecl{macroname}[name=some name]` declares the name of the symbol to be `some name` instead.
The starred variant `\symdecl*{name}` declares the concept with the given name, but does not generate a semantic macro.

So let's provide equality with a notation. As a first step, we should let sTeX know that "equal" takes two arguments. We might also want to shorten the semantic macro to e.g. `\eq`, without changing the name. Hence:

```
\symdecl{eq}[name=equal,args=2]
```

Next, we add an infix notation with the `\notation` macro:

```
\notation{eq}{#1 = #2}
```

That seems like a lot to write, so for the very common case where we want to declare a symbol with a semantic macro and a notation all at once, the `\symdef` macro does all three by combining the optional and mandatory argument of `\symdecl` and `\notation`:

```
\symdef{eq}[name=equal,args=2]{#1 = #2}
```

and indeed, we can now use the `\eq` macro in math mode to invoke our new notation: `$\eq{a}{b}$` now yields $a = b$ – notably without any highlighting (and hover interaction in the HTML) though. Since our semantic macro takes *arguments*, which should be differently highlighted, we need to let our notation know which parts of the notation are highlightable components.

We can do so with the `\comp` and `\maincomp` macros:

The `\comp`-macro marks components to be highlighted in a notation for a symbol taking (one or more) arguments.
This is necessary because it is (nearly) impossible for LaTeX to figure out, which parts of a notation to highlight and which not on its own – in particular, the highlighting should stop for the *arguments* of a semantic macro.
Additionally, the `\maincomp` macro can be used to mark (at most) one notation component to represent the *primary* component of the notation.
Notations that do not take arguments, as well as operator notations, are automatically wrapped in `\maincomp`.

In our case, this applies only to the "=", symbol, so:

```
\symdef{eq}[name=equal,args=2]{#1 \mathrel{\maincomp{=}} #2}
```

You may be wondering about the role of the `\mathrel` macro in the example above: TeX determines spacing/kerning in math mode by assigning a *class* to every

character. Both individual characters and whole subexpressions can be assigned one of these classes using dedicated macros. These are:

class	TEX macro	examples
ordinary (default class)	`\mathord`	$\alpha\ i\ \Diamond$
large operator	`\mathop`	$\sum \prod \int$
opening	`\mathopen`	$(\ [\ \langle$
closing	`\mathclose`	$)\ ]\ \rangle$
binary relation	`\mathrel`	$\leq\ >\ =$
binary operator	`\mathbin`	$+\ \cdot\ \circ$
punctuation	`\mathpunct`	$,\ ;$

TEX "forgets" the class of an expression if it is wrapped in a `\comp` macro. It is therefore a good idea to wrap any occurence of a `\comp` in the corresponding TEX macro for the desired class (e.g. `\mathrel{\comp{\leq}}`).

Having done so, we can now type `$\eq{a}{b}$` to get $a = b$. Thanks to using `\maincomp`, we now also have an operator notation, which we can invoke using `$\eq!$`, yielding $=$.

What if we want to add more notations? Say we want to be able to invoke equality to get the variant notation $a \equiv b$ ()without changing the intended meaning). If we want to be able to choose one of several notations, we should give the notation an *identifier*.

Let's again modify our earlier notation by adding the identifier `eq` to the optional arguments of `\symdef`, like so:

```
\symdef{eq}[name=equal,args=2,eq]{#1 \mathrel{\maincomp{=}} #2}
```

We can now invoke the specific notation provided here by writing `$\eq[eq]{a}{b}$` to the same effect. But we can also add more notations using the `\notation` macro:

```
\notation{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
```

which we can now invoke with `$\eq[equiv]{a}{b}$`, yielding $a \equiv b$.

By default, the *first* notation provided for a given symbol is considered the *default* notation, which is invoked if the semantic macro is used without an optional argument – hence, `$\eq{a}{b}$` still yields $a = b$.

If we use the starred variant of the `\notation` macro, the notation is set as the new default. Hence, had we done

```
\notation*{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
```

then `$\eq{a}{b}$` would now yield $a \equiv b$.

Any already existing notation can be set as default using the `\setnotation` macro; e.g. instead of using `\notation*`, we could also do

```
\notation{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
\setnotation{eq}{equiv}
```

Exercise

Implement the symbol "equal" as above in a new module "`Equality`" and add a documentation such that hovering over the symbol in the HTML yields the following snippet:

> Two objects a, b are considered **equal** (written $a = b$ or $a \equiv b$), if there is no property that distinguishes them.

Lösung: Can be found in `[sTeX/MathTutorial]/mod/Equality1.en.tex`

3.1.2 Types and Variables

sTeX Every symbol or variable can be assigned a **type**, signifying what "kind of object" the symbol represents, or what (primary) set it is contained in.

Types are particularly useful for *variables*:

sTeX A **variable** represents a *generic* or *unspecified* object.
Variables can be declared using the `\vardef`-macro, whose syntax is analogous to `\symdef`.
Note that variables are local to the current TEX-group (e.g. environment).

Let's leave our equality-module aside for now and turn our attention to something simpler: natural numbers. Consider the following module:

Example 5
Input:

```
\begin{smodule}{Nat}
  \symdef{Nat}[name=natural numbers]{\mathbb N}
  \begin{sparagraph}[style=symdoc]
    The \definame{Nat} $\defnotation{\Nat}$ are the numbers
    $0,1,2,...$
  \end{sparagraph}
  \symdef{plus}[name=addition,args=2]{#1 \mathbin{\maincomp{+}} #2}
  \begin{sparagraph}[style=symdoc]
    \Definame{addition} $\defnotation{\plus{a}{b}}$
    refers to the process of adding two \sn{Nat}.
  \end{sparagraph}
\end{smodule}
```

Output:

The **natural numbers** $\mathbb{N}$ are the numbers 0,1,2,...
Addition $a+b$ refers to the process of adding two natural numbers.

(like `\definame` and `\definiendum`, the `\defnotation` macro is only allowed in documenting environments like `sparagraph[style=symdoc]` or `sdefinition`, and highlights the notation components marked with `\comp` or `\maincomp` the same way as `\definame` and `\definiendum` do.)

Note, that as the `\Nat` semantic macro does not take any arguments, we do not need to wrap the notation in a `\comp` or `\maincomp`.

The above fragment uses two variables a and b. In fact, FLAMI will consider them variables even though they are not marked up as such – but since they are not marked up, we are missing out on useful functionality.

Let's change that by adding two variable definitions[1]:

Example 6

Input:

```
\begin{sparagraph}[style=symdoc]
  \vardef{va}[name=a]{a}\vardef{vb}[name=b]{b}
  \Definame{addition} $\defnotation{\plus{\va}{\vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Output:

Addition $a+b$ refers to the process of adding two natural numbers.

Okay, so now a and b are gray, but besides that, we haven't achieved much yet.

Let's change that by giving the variables the type $\mathbb{N}$:

Example 7

Input:

```
\begin{sparagraph}[style=symdoc]
  \vardef{va}[name=a,type=\Nat]{a}\vardef{vb}[name=b,type=\Nat]{b}
  \Definame{addition} $\defnotation{\plus{\va}{\vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Output:

Addition $a+b$ refers to the process of adding two natural numbers.

Now if we hover over the a and b (in the HTML), it will show us that their type is $\mathbb{N}$!

We can of course also assign types to symbols. In the IDE, find the symbol *"function space"* with semantic macro `\funspace` (in `[sTeX/MathBase/Functions]{mod?Function}`). The OMDoc preview window shows you how to use this symbol (Figure 14). This tells us that if we write `\funspace{a_1,...,a_n}{b}` (depending on which notation we use), we will get $a_1 \times ... \times a_n \to b$.

We want addition to have type $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$, hence we do:

```
\symdef{plus}[name=addition,args=2,
  type=\funspace{\Nat,\Nat}{\Nat}
]{#1 \mathbin{\maincomp{+}} #2}
```

[1] Technically, this is called a *variable reservation*, for those in the know.

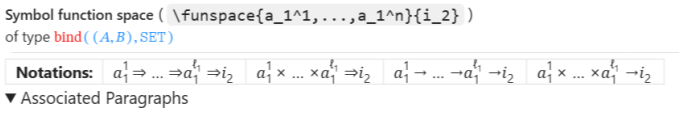

Figure 14: Syntax Preview

So far (and when using the **use** button in the IDE), we have been using the `\usemodule` macro to import content. `\usemodule` is allowed anywhere and imports the referenced module content local to the current TEX group.
Now that we use imported symbols in types (and since we are *in* a module), we need to make sure that the imported modules are also (transitively) *exported*, since our new symbols now *depend* on the imported module.
For that we use the `\importmodule` macro within the module; i.e. the file should now look something like this:

```
\begin{smodule}{Nat}
  \importmodule[sTeX/MathBase/Functions]{mod?Function}
  ...
```

Note that the HTML is aware of this now (after you save): *Clicking* on any occurence of addition now yields Figure 15.

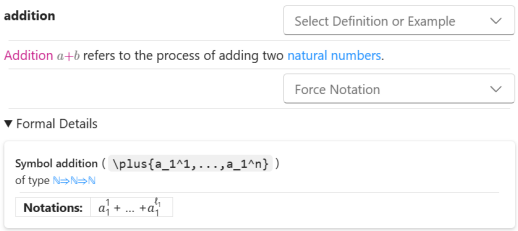

Figure 15: On-Click Popup in the HTML

By virtue of using `[sTeX/MathBase/Functions]{mod?Function}`, we also imported `[sTeX/MathBase/Sets]{mod?Set}`, which gives us the *"collection"* symbol. Let's use this as a type for the natural numbers:

```
\symdef{Nat}[name=natural numbers,type=\collection]{\mathbb N}
```

3.1.3 Flexary Macros and Argument Modes

Here is one thing you might wonder: Writing `$\plus{a}{b}$` is one thing, but what if we want to produce $a + b + c + d + e$? Do we really need to write `$\plus{a}{\plus{b}{\plus{c}{...}}}$`?

Of course not. We can declare the symbol such that the semantic macro `\plus` expects a (comma-separated) *sequence* of arguments instead of two "normal" arguments.

The optional `args`-argument of `\symdecl` expects a string of characters indicating the semantic macro's **argument modes**. There are four such modes:

- `i` a **simple argument**,
- `a` a – *(left or right) associative* – **sequence argument**, represented as a single TEX-argument `{a,b,...}`,
- `b` A **binding argument** that expects a variable that is bound by the symbol in its application, and
- `B` A **binding sequence argument** of arbitrarily many bound variables by the symbol (`{x,y,z,...}`).

If `args` is given as a number n instead, the semantic macro takes n arguments of mode `i`.

Example 8

- For `\plus{a,b,c}` yielding $a+b+c$, we do `\symdecl{plus}[args=a]`,
- for `\inset{a,b,c}{A}` yielding $a,b,c \in A$, we do `\symdecl{inset}[args=ai]`,
- in `\add{i}{1}{n}{f(i)}` yielding $\sum_{i=1}^{n} f(i)$, the variable i is bound in the expression, we hence do `\symdecl{add}[args=biii]`,
- in `\foral{x,y,z}{P(x,y,z)}` yielding $\forall x,y,z.\ P(x,y,z)$, the variables x,y,z are all bound by the $\forall$, we hence do `\symdecl{foral}[args=Bii]`.

So when we wrote `\symdecl{plus}[args=2]`, this was actually shorthand for `\symdecl{plus}[args=ii]`.

Let's revise our previous declaration and the syntax of the `\plus` macro:

```
\symdef{plus}[name=addition,args=a,
  type=\funspace{\Nat,\Nat}{\Nat}
]{#1 \mathbin{\maincomp{+}} #2}
\begin{sparagraph}[style=symdoc]
  \vardef{va}[name=a]{a}\vardef{vb}[name=b]{b}
  \Definame{addition} $\defnotation{\plus{\va,\vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Now we get new errors, that are easy to explain: Our notation `{#1 \mathbin{\maincomp{+}} #2}` refers to *two* arguments, but our semantic macro only takes *one* (albeit a sequence argument). We now need to let STEX know what to do with the sequence argument in our notation. Using the `\argsep` macro, we can tell STEX to insert the *separator* "+" between the individual elements of the argument sequence `#1`:

```
\symdef{plus}[name=addition,args=a,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{+}}}}
```

TODO[2] TODO[3]

Now we can finally write `$\plus{a,b,c,d,e}$` and get $a+b+c+d+e$ – hooray!

[2] TODO: argmap
[3] TODO: argarraymap

Exercise
Analogously to the above, implement a symbol "`multiplication`" with semantic macro `\mult`, that takes a single sequence argument and has a default notation such that `\mult{a,b,c}` produces $a \cdot b \cdot c$.

Lösung: Can be found in `[sTeX/MathTutorial]mod/Nat.en.tex`

3.1.4 Precedences

If you have done the previous exercise, you now have semantic macros `\plus` and `\mult` at your disposal. We can of course nest them to produce e.g. $a + b \cdot c$ (with `$\plus{a,\mult{b,c}}$`). If we do `$\mult{a,\plus{b,c}}$` however, we get $a{\cdot}b{+}c$. Annoying – we now have to insert parentheses: `$\mult{a,(\plus{b,c})}$`... or do we?

We do *not*. Instead, we can assign *precedences* to notations to have sTeX insert parentheses automatically.

sTeX

`\notation` (and hence `\symdef`) take an optional argument `prec=<opprec>;<argprec1>x...x<argprec n>` consisting of an **operator precedence** `<opprec>` and for each argument `k` an **argument precedence** `<argprec k>`.
All precedences are integers, e.g. 10 or -500. It is good practice to use precedences that leave enough room to smuggle values inbetween, so that we can fine-tune them later as more symbols may intervene.
The precise numbers used for precedences are arbitrary – what matters is which precedence is higher than which other precedence when used together.
By default, all precedences are 0, unless the symbol takes no arguments, in which case the operator precedence is `\neginfprec` (negative infinity).
If we only provide a single number in `prec=`, this is taken as both the operator precedence and all argument precedences.

The *lower* a precedence, the *stronger* a notation binds its arguments. In our particular case, we want multiplication to bind stronger than addition, so we can (arbitrarily) assign them precedences e.g. 10 and 20:

```
\symdef{plus}[name=addition,args=a,assoc=bin,prec=20,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{+}}}}
\symdef{mult}[name=multiplication,args=a,assoc=bin,prec=10,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{\cdot}}}}
```

And now if we type `$\mult{a,\plus{b,c}}$`, sTeX will automatically insert parentheses and yield $a \cdot (b + c)$ – and conversely, if we do `$\plus{a,\mult{b,c}}$`, sTeX will *not* insert parentheses and yield $a + b \cdot c$.

3.1.5 Finishing Equality

You might wonder if – as with addition – we can make "equal" take a sequence argument as well. Naturally, we can:

```
\symdef{eq}[name=equal,args=a,eq,
  type=\funspace{\vA,\vA}{\prop}
]{\argsep{#1}{\mathrel{\maincomp=}}}
\notation{eq}[equiv]{\argsep{#1}{\mathrel{\maincomp\equiv}}}
```

3.1.6 Variable Sequences

There is a special kind of variable in STEX for when we want to use *sequences* of variables.

We can use the `\varseq` macro to declare a new sequence variable; in the simplest case that looks something like the following:

```
\varseq{seqn}[name=n,type=\Nat]{1,\ellipses,k}{\maincomp{n}_{#1}}
```

We have just declared a new variable sequence of type $\mathbb{N}$, that ranges over indices $1, \ldots, k$, with notation n_i for some specific index i.

If we now do `\seqn{i}`, we get n_i, and if we do `\seqn!`, we get $n_1, \ldots, n_k$.

We can also do multi-dimensional sequences, e.g.

```
\varseq{seqm}[name=m,type=\Nat,args=2]
  {{1}{1},\ellipses,{\ell}{k}}
  {\maincomp{m}_{#1}^{#2}}
```

Now `\seqm{i}{j}` produces m_i^j, and `\seqm!` produces $m_1^1, \ldots, m_\ell^k$.

Of course, we can manually change the way `\seqn!` is typeset by providing an explicit operator notation using `op=`; e.g. if we do

```
\varseq{seqn}[name=n,type=\Nat,op={(n_i)_{i=1}^k}]
  {1,\ellipses,k}{\maincomp{n}_{#1}}
```

then `\seqn!` produces $(n_i)_{i=1}^k$.

So far so nice, but sequence variables get especially useful in combination with sequence arguments: Consider for example the `\plus` semantic macro for addition. This expects one sequence argument, or alternatively, a *sequence variable*: `\plus{\seqn}` now produces $n_1 + \ldots + n_k$, and `\eq{\seqm}` now produces $m_1^1 = \ldots = m_\ell^k$.

TODO[4]

3.2 Statements

Now that we have equality, natural numbers, addition and multiplication at our disposal, let's implement some *statements*. Both addition and multiplication are, for example, *associative* and *commutative*.

We could state these properties directly for the two operations, but we can also first define *associativity* and *commutativity* in general, and then assert them specifically for addition and multiplication.

[4] TODO: seqmap

3.2.1 Definitions

Let's define what it means to be *associative*. This means, of course, declaring a new symbol. Note that we don't need a semantic macro for associativity, since there is no notation to attach to it. We will also for now ignore its type. Note however, that associativity is still a property of (binary) operations, so it still makes sense to have the symbol take an *argument*; namely the operation it applies to.

We will also finally provide an actual (more or less) formal *definition* for the symbol, so where we used the `sparagraph` environment with `style=symdoc` before, we will now use the `sdefinition` environment, which also gives us `\definame`, `\definiendum`, `\defnotation` and all that.

A first variant of a corresponding module could look like this:

Example 9

Input:

File [sTeX/MathTutorial]`props/Associative1.en.tex`

```
\begin{smodule}{Associative}
  \importmodule{mod?Equality}

  \symdecl*{associative}[args=1]
  \begin{sdefinition}[for=associative]
    \vardef{vA}[name=A,type=\collection]{A}
    \vardef{vop}[name=op,type=\funspace{\vA,\vA}\vA,args=a,assoc=bin]
      {\argsep{#1}{\mathbin{\maincomp{\circ}}}}
    %
    A binary operation $\fun{\vop!}{\vA,\vA}\vA$ is called
    \definame{associative}, if
    $\eq{
      \vop{(\vop{a,b}),c},
      \vop{a,(\vop{b,c})}
    }$ for all $\inset{a,b,c}\vA$.
  \end{sdefinition}
\end{smodule}
```

Output:

Definition 3.2.1. A binary operation $\circ : A \times A \to A$ is called **associative**, if $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$.

Note, that the semantic macros `\fun` and `\inset` come from `[sTeX/MathBase/Functions]mod?Function` and `[sTeX/MathBase/Sets]mod?Set`, respectively. Also note, that the variable declaration for `\vop` makes use of all the fun features we already discussed for addition.

Note that the above is more than good enough, if you merely want to produce nice-looking, "wikified" HTML and PDF documents. The rest of this subsection will cover how to add more flexiformal semantics to the above.
If this seems laborious and/or difficult, keep in mind that this is to some degree experimental still, and you are not forced to go overboard with semantic annota-

tions!
But if you aim to create a "library of symbols" for mathematical concepts, then all of the possibilities that we discuss here will add value for the community. Generally, the higher the ratio of readers to authors the more any investment in semantization will pay off.

Semantic Macros in Text Mode

The first thing we can do to further improve this document is marking up the "for all" in the definition – after all, there naturally is a symbol for the universal quantifier, which can be found in `[sTeX/Logic/General]mod/syntax?UniversalQuantifier` and has the semantic macro `\foral` (as to not conflict with the TEX primitive macro `\forall`).

The naive approach would be to replace the "for all" by e.g. `\sr{foral}{for all}`. That would (correctly) associate and highlight the text fragment with the symbol "universal quantifier", *but* we are not just referencing the symbol here – we are actually using it, by *applying* it to the variables a, b, c and the expression $(a \circ b) \circ c = a \circ (b \circ c)$.

In *math mode*, we can just use the semantic macro `\foral` – that will take two arguments (of modes `B` and `i`) and produce the corresponding notation, so that

```
$\foral{\inset{a,b,c}{\vA}}{
  \eq{ \vop{(\vop{a,b}),c} , \vop{a,(\vop{b,c})} }
}$
```

will produce $\forall a, b, c \in A.(a \circ b) \circ c = a \circ (b \circ c)$.

In *text mode*, however, we don't have a specific notation – instead, the specific "notation" is whatever sentence we want to mark up semantically. In text mode, semantic macros therefore behave differently:

1. They take *precisely* one argument, regardless of how many arguments the macro would take in math mode or (equivalently) the `args` property of the symbol.
2. *Within* that argument, we can use `\comp` to highlight arbitrary text fragments, and
3. we can use the `\arg` macro to mark up the *actual* arguments that the symbol is supposed to be applied to.

   `\arg` takes as optional argument the index of the argument that is being marked up; if not they are used consecutively. The starred variant `\arg*` produces no output.

So we could now do

```
\foral{\comp{For all} $\arg{\inset{a,b,c}{\vA}}$, we have
  $\arg{
    \eq{ \vop{(\vop{a,b}),c} , \vop{a,(\vop{b,c})}}
  }$
}
```

which produces "For all $a, b, c \in A$, we have $(a \circ b) \circ c = a \circ (b \circ c)$".

In our case though, we want to "switch the arguments around" – first comes the equation, then the variables to be bound. Hence:

```
\foral{
  $\arg[2]{
    \eq{ \vop{(\vop{a,b}),c}, \vop{a,(\vop{b,c})} }
  }$
  \comp{for all}
  $\arg[1]{ \inset{a,b,c}{\vA} }$
}
```

which produces "$(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$".

Definientia

Now we have a fully semantically annotated expression in the definition for "associative". Can we let MMT know, that this expression really is *the* definition of the symbol?

Yes, we can. All we need to do is wrap the sentence in a `\definiens` macro (plural: *definientia*; like the word *"definiendum"* refers to "the term being defined", *"definiens"* refers to "the thing the term is being defined *as*").

The `\definiens` macro is only allowed within the `sdefinition` environment, and requires that the environment lists the symbol that gets the definiens attached explicitly in its `for=` argument. It is possible to attach definientia to multiple symbols within an `sdefinition` environment, in which case the symbol needs to be provided as an optional argument, e.g. we could do `\definiens[associative]{...}`. Since "associative" is the only symbol being defined in our definition, we can omit that argument.

Alternatively, as with types we can attach definientia to a `\symdecl` directly using the optional argument `def=...`.

At this point, you might justifiably wonder, why we even still need to declare associative with `\symdecl*` before we define it. And indeed, we don't – the `sdefinition` environment takes the same optional arguments as the `\symdecl` macro, and if we explicitly provide a `name=` (or a `macro=`), it will generate a symbol for us. We can hence get rid of the `\symdecl*` and instead do:

```
\begin{sdefinition}[name=associative,args=1]
  ...
\end{sdefinition}
```

One more problem remains: We stated that associative is to take one argument – but we haven't told STEX what it is yet. In our case, the argument is represented by the variable `\vop`. In fact, chances are that arguments to symbols in types or definientia are almost always represented by some variable.

We can use one of two ways to a variable as being an argument:

1. If the variable (e.g. `\vop` with name `op`) was already declared prior to the `sdefinition` environment, we can use the `\varbind` macro in the environment; e.g. by adding `\varbind{op}`.

2. We can move (or copy) the `\vardef` for the variable into the environment and add `bind` to its optional arguments.

In total, our fully annotated definition now looks like this:

Example 10
Input:

File [sTeX/MathTutorial]**props/Associative.en.tex**

```
\begin{sdefinition}[name=associative,args=1]
  \vardef{vA}[name=A,type=\collection]{A}
  \vardef{vop}[name=op,type=\funspace{\vA,\vA}\vA,
    args=a,assoc=bin,bind % <- argument for the symbol
  ]{\argsep{#1}{\mathbin{\maincomp{\circ}}}}
  \vardef{va}[name=a,type=\vA]{a}
  \vardef{vb}[name=b,type=\vA]{b}
  \vardef{vc}[name=c,type=\vA]{c}
  %
  A binary operation $\fun{\vop!}{\vA,\vA}\vA$ is called
  \definame{associative}, if
  \definiens{\foral{$\arg[2]{\eq{
    \vop{(\vop{\va,\vb}),\vc},
    \vop{\va,(\vop{\vb,\vc})}
  }}$ \comp{for all} $\arg[1]{\inset{\va,\vb,\vc}\vA}$}}.
\end{sdefinition}
%
```

Output:

Definition 3.2.2. A binary operation $\circ : A \times A \to A$ is called **associative**, if $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$.

And indeed, if we look at the OMDoc tab of the HTML preview, we can see that not only does MMT attach the definiens to the symbol, it has also inferred the type of "associative" from the definiens (Figure 16).

▼ Symbol `associative`

Definiens $\{A\colon \mathtt{SET}\}_I(\circ\colon A\to A\to A)\to\forall a\colon A, b\colon A, c\colon A.\ (\underbrace{(a\circ b)\circ c = a\circ(b\circ c)}_{A})$

Type $\{A\colon \mathtt{SET}\}_I(\circ\colon A\to A\to A)\to\mathtt{Prop}$

Figure 16: Type Inferred from Definiens

Using Symbols Without Semantic Macros and Exporting Code in Modules

So now we don't have a semantic macro for "associative", but it *does* take an argument. How can we ever actually *use* the symbol now?

The answer is: with the `\symuse` macro. Like `\symref` and friends, `\symuse` takes a symbol name or the name of its semantic macro as argument, but behaves otherwise like using a semantic macro directly. So for, say, addition, `\symuse{addition}` and `\symuse{plus}` behave exactly like `\plus`.

In our case, this means we can do `\symuse{associative}`. "associative" does not have a notation, but in practice, we say something like *"+ is associative"* rather than using some specific mathematical notation for the same thing.

Combining this with what we just learned, we can now say that addition is associative by doing:

```
\symuse{associative}{$\arg{\plus!}$ \comp{is associative}}
```

In fact, we would do the exact same thing every time we want to say that *some* operator is associative, so it makes sense to introduce a macro for this. In fact, such a macro is easy to define using standard LaTeX methods. This is where `\STEXexport` becomes very handy:

In a module, we can put arbitrary LaTeX code in an `\STEXexport`, and this code will be executed every time the module is imported via `\usemodule` or `\importmodule`. This is especially useful for macro definitions, and this way modules can almost act like LaTeX packages!

So we can define a new macro `\isassociative` that applies "associative" to an arbitrary operation and produces the semantically marked-up text "`#1` is associative", and wrap that macro definition in an `\STEXexport`, and whenever we use the `Associative` module, we also get the `\isassociative` macro:

```
\STEXexport{
  \def\isassociative#1{
    \symuse{associative}{\arg{#1} ~is ~\comp{associative}}
  }
}
```

And now, we can do e.g. `\isassociative{$\plus!$}` to produce "$+$ is associative".

For technical reasons, `\STEXexport` processes its content in the `expl3` category code scheme – what this means is that all spaces are ignored entirely, and the characters `_` and `:` are valid characters in macro names.
In practice, this means you will have to use the `~` character for spaces, and if you want to use a subscript `_`, you should use the macro `\c_math_subscript_token` instead.

Exercise
Analogously to all the above, implement a module for *commutativity*; i.e the property of a binary operation that $a \circ b = b \circ a$ for all a, b. Make the module export a macro `\iscommutative` analogously to `\isassociative`.

Lösung: Can be found in `[sTeX/MathTutorial]props/Commutative.en.tex`

TODO[5]

3.2.2 Assertions

Having defined associativity and commutativity, we can now assert that both properties hold for addition and multiplication.

For *assertions* (i.e. theorems, lemmata, axioms, claims,...), sTeX provides the `sassertion` environment.

In the simplest case, that can look like the following:

[5] TODO: intent?

```
\begin{sassertion}
  \isassociative{\Sn{plus}}
\end{sassertion}
```

which yields

Addition is associative

Do we want this to be typeset as a **Theorem**? For that we just add a `[style=theorem]` to the `sassertion` environment, provided we have a customization for that – (see chapter 9). We can also load the stexthm package, which uses the amsthm package to provide common typesettings for the types: `theorem`, `observation`, `corollary`, `lemma`, `axiom` and `remark`.

So far, this is not too useful – after all, we could have just as well used e.g. the `amsthm` package and gone straight for the non-sTEX variant

```
\begin{theorem}
  \isassociative{\Sn{plus}}
\end{theorem}
```

But as with `sdefinition`, we can immediately add a corresponding symbol in the `sassertion` `environment`, and have it be documented directly by the environment:

```
\begin{sassertion}[style=theorem,name=addition is associative]
  \isassociative{\Sn{plus}}
\end{sassertion}
```

And now, if we do `\sn{addition is associative}`, we get addition is associative with a corresponding hover pop-up (in the HTML).

Of course, the usefulness of this grows with more elaborate assertions. For very short assertions such as the above, we might not even want to typeset them in such a space hungry manner.

For that purpose, we provide the `\inlineass` macro (and analogously: `\inlinedef` for `sdefinition`), which takes the same optional arguments as the environment. So we could also do:

```
\inlineass[name=addition is associative]{\isassociative{\Sn{plus}}}
```

So far, MMT is blissfully unaware of the semantic contents of our assertions. We can easily remedy that by wrapping the expression representing the assertion in a `\conclusion` macro, analogously to the definiens macro in `sdefinition`s:

```
\inlineass[name=addition is associative]{
  \conclusion{\isassociative{\Sn{plus}}}
}
```

We can now see the statement in the OMDoc tab of the HTML preview (Figure 17).

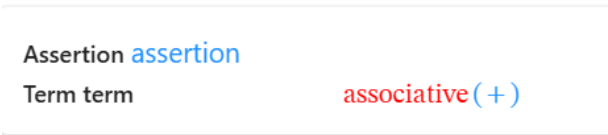

Figure 17: Assertion Statement in OMDoc

Not exactly pretty – the OMDoc tab uses notations to render content, and we did not provide any for associative.

3.2.3 Proofs

> sTeX provides the `sproof` environment for marking up *proofs*.
> The markup mechanism for `sproof` is still highly experimental and likely subject to change in the near future. As such, we omit a closer explanation of its usage until the syntax and functionality have sufficiently stabilized.

3.3 Mathematical Structures

A common concept in mathematics is that of a mathematical structure – a *tuple* of interdependent components. For example: A *monoid* is a structure $\langle M, \circ, e\rangle$ such that certain axioms hold; where M is a set, $\circ$ is a binary operation, and $e \in M$.

From a representational perspective, this is particularly interesting: M, $\circ$ and e in the above are not symbols in the same way that the previous symbols we considered were – they don't represent definite objects. Instead, they are *components* of some other object, namely a monoid; where a *particular* monoid could either be a fixed object (such as $\langle \mathbb{Z}, +, 0\rangle$) or an *indefinite* monoid; i.e. a variable. We call the components of a mathematical structure **fields**.

In this section, we will discuss how to declare and use mathematical structures in sTeX, build them up modularly, and connect them among each other to avoid duplication.

We will do so by considering *lattices* both algebraically and order-theoretically, and identify the two perspectives.

3.3.1 Declaring and Using Structures

The simplest kinds of structures are *magmas* and *(directed) graphs*, so we might as well start there:

> **Definition 3.3.1.** A **magma** is a structure $\langle U, \circ\rangle$, where U is a collection and $\circ$ a binary operation $U \times U \to U$.

The obvious start is to create a new module `Magma`. Within this module, we import the `Functions` module so we can later assign a type to the operation. We can then use the `mathstructure` environment, that creates a new symbol "magma":

```
\begin{smodule}{Magma}
  \importmodule[sTeX/MathBase/Functions]{mod?Function}
  \begin{mathstructure}{magma}
    ...
  \end{mathstructure}
\end{smodule}
```

mathstructure behaves very similarly as smodule – within the environment, we can declare new symbols, notations and all that.

So within the mathstructure, we can add symbols for the two fields U and $\circ$:

```
\symdef{univ}[name=universe,type=\collection]{U}
\symdef{op}[name=operation,args=a,assoc=bin,
  type=\funspace{\univ,\univ}\univ
]{\argsep{#1}{\mathbin{\maincomp{\circ}}}}
```

Once we close the environment (with \end{mathstructure}), the symbols are "gone". However, we now have a new symbol "magma" with semantic macro \magma. It's usage is somewhat more complicated than "normal" semantic macros, but one thing we *can* do with it now is $\magma!$, which will produce $\langle U, \circ \rangle$.

Notably however, the \magma macro is already available *within* the mathstructure environment as well.

This allows us to provide an sdefinition using the semantic macros declared in the structure:

Example 11

Input:

File [sTeX/MathTutorial]algebra/Magma.en.tex

```
\begin{mathstructure}{magma}
  \symdef{univ}[name=universe,type=\collection]{U}
  \symdef{op}[name=operation,args=a,assoc=bin,
    type=\funspace{\univ,\univ}\univ]
    {\argsep{#1}{\mathbin{\maincomp\circ}}}

  \begin{sdefinition}[for={magma,univ,op}]
    A \definame{magma} is a \sr{mathstruct}{structure} $\magma!$,
    where $\univ$ is a \sn{collection} and $\op!$
    a binary operation $\funspace{\univ,\univ}\univ$.
  \end{sdefinition}
\end{mathstructure}
```

Output:

Definition 3.3.2. A **magma** is a structure $\langle U, \circ \rangle$, where U is a collection and $\circ$ a binary operation $U \times U \to U$.

Instantiating Structures

More importantly however, we can now declare a variable magma, using the optional return= argument. For example, we can now do

```
\vardef{vM}[name=M,return=\magma]{M}
```

and we get the semantic macro \vM with which we can do the following:

Syntax	Result
`$\vM!$`	M
`$\vM{}$`	$\langle U_M, \circ_M \rangle$
`$\vM{univ}$`	U_M
`$\vM{op}!$`	$\circ_M$
`$\vM{op}{a,b,c}$`	$a \circ_M b \circ_M c$

In other words: Given a symbol or variable with semantic macro `\foo` and `return=\struct`, then `\foo{<fn>}` behaves like the semantic macro `\fn` *within* the `mathstructure` environment for `struct` – but instantiated for the specific instance `foo`.

By default, sTEX attaches the symbol's (or variable's) operator notation as a subscript suffix to the notation component marked with `\maincomp` – e.g., since the "`\circ`" in the notation for `op` is marked with `\maincomp`, doing `$\vM{op}{a,b}$` ultimately outputs `a \circ_{\vM!} b`. Hence, we get $a \circ_M b$.

We can change the way the `\maincomp` notation component is modified, by using the optional argument `copm=` in the semantic macro for the mathematical structure. For example, to not change it at all, we can do:

```
\vardef{vM}[name=M,return={\magma[comp=##1]}]{M}
```

...or to suffix it with a `'`, we can do

```
\vardef{vMp}[name=Mp,return={\magma[comp=##1']}]{M'}
```

This allows us to do things like:

```
Let $\vM!:=\vM{}$ and $\vMp!:=\vMp{}$ \sns{magma}. Then...
```

yielding

> Let $M := \langle U, \circ \rangle$ and $M' := \langle U', \circ' \rangle$ magmas. Then...

We can also *assign* fields to (arbitrary) expressions, by doing `name=<tex>` in square brackets. For example we can do the following:

```
\vardef{vA}[type=\collection]{A}
\vardef{vM}[name=M,return={\magma[comp=##1][univ=\vA]}]{M}
\vardef{vMp}[name=Mp,return={\magma[comp={{##1}'}][univ=\vA]}]{M'}

Let $\vM!:=\vM{}$ and $\vMp!:=\vMp{}$ \sns{magma} on $\vA$....
```

> Let $M := \langle A, \circ \rangle$ and $M' := \langle A, \circ' \rangle$ magmas.

Of course, we can also use `return=` with variable sequences – for example:

```
\varseq{vMs}[name=M,return={\magma[comp={##1}_{#1}]},op=(M_i)_1^n]
  {1,\ellipses,n}{\maincomp{M}_{#1}}
Let $\vMs! := \vMs{i}{}_1^n$ a sequence of \sns{magma}...
```

> Let $(M_i)_1^n := \langle U_i, \circ_i \rangle_1^n$ a sequence of magmas...

Note that in the above, it seems that using `#1` in the `return` argument is allowed. Indeed, it is - the `return` statement takes the same arguments as the semantic macro itself does and is appropriately instantiated. Since the first (and only) argument to the

sequence `\vMs` is the index, when doing `\vMs{i}...` the `#1` in the `return`-statement will be replaced by `i`.

Also, note that if we want to produce M_i – i.e. the magma at index i in the sequence, we can do `\vMs{i}!`.

> Think of the `!` as a "stop sign" - if the expression up to the `!` has an associated presentation, the `!` tells sTeX to "stop eating arguments" and present whatever it has until now.

3.3.2 Extending Structures and Axioms

It is extremely common to "build up" structures in a hierarchical manner by adding new fields or axioms: A *semigroup* is an associative magma. A *band* is an idempotent semigroup. A *monoid* is a semigroup with a unit. A *partial order* is an antisymmetric preorder.

We alluded to the fact earlier, that the `mathstructure` environment behaves like an `smodule` – that is literally true: Every `mathstructure foo` in a module `FooMod` is in fact also a module `?FooMod/foo-module`. We can therefore easily extend structures using `\importmodule{...?FooMod/foo-module}` – but extending structures is so common, and using `\importmodule` tiring, that there is a shortcut: the `extstructure` environment. It takes as second argument a comma-separated list of structure names. That allows us to easily define semigroups:

Example 12

Input:

File [sTeX/MathTutorial]`algebra/Semigroup.en.tex`

```
\begin{extstructure}{semigroup}{magma}
  \begin{sdefinition}
    A \definame{semigroup} is a \sn{magma} $\semigroup!$,
    where \inlineass[name=associative axiom]{
      \conclusion{\isassociative{$\op!$}}.
    }
  \end{sdefinition}
\end{extstructure}
```

Output:

> **Definition 3.3.3.** A **semigroup** is a magma $\langle U, \circ \rangle$, where $\circ$ is associative.

Note our usage of `\inlineass` to generate a new symbol for the associative axiom.

If we look at the OMDoc tab in the HTML preview window, we can see the output in Figure 18.

So MMT has decided that our statement is an *axiom*.

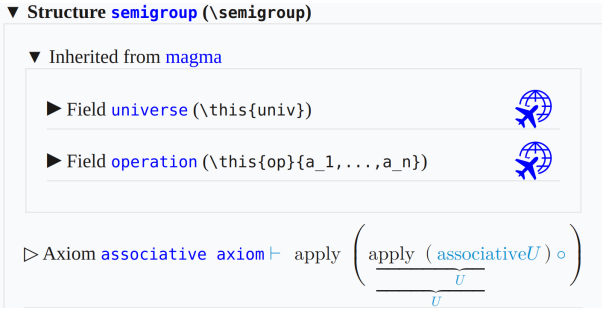

Figure 18: Axioms in OMDoc

Conservative Extensions

For structures, there is a *critical* distinction between *defined* and *undefined* symbols; and analogously between *theorems* and *axioms*.

Remember that structures are more like *templates* that are *instantiated* by particular objects. An *undefined* field in a structure, in that sense, is like an *obligation*: If something is supposed to be a semigroup, it *has to* have a universe, an operation and the operation needs to satisfy the associative axiom.

Defined fields on the other hand have a *definiens* on the basis of the remaining fields – they don't need to be explicitly provided for something to instantiate the structure; if all the *undefined* fields are provided, the *defined* ones we get *"for free"*.

The same holds for *theorems*: If a statement is *provable* from the axioms, then we don't need to explicitly prove it to hold for some particular instance – we have a proof already, provided the axioms hold.

> sTeX
> The relation between axioms and theorems is not just analogous to that between undefined and defined symbols: It is the very same. Remember the judgments as types paradigm?
> For a proposition P, an assertion in sTeX induces a symbol of type $\vdash P$. Without a proof, this symbol is *undefined* – and hence an *axiom*. A *proof* for P is a specific term of type $\vdash P$ – i.e. a potential *definiens*. To prove an assertion turns it into a *theorem*, which is to say that the symbol can be *defined*.

One consequence of this is: Extending a structure only by *defined* fields does not actually (conceptually) introduce a *new* structure – every instance of the old one *should* also be an instance of the new one. The new fields are basically just "syntactic sugar".

There is a name for extending a structure only by defined fields (or theorems): A *conservative extension*.

sTeX provides the `extstructure*` environment for that purpose. Unlike `extstructure`, it does *not* take a name (technically, sTeX generates one internally). Instead, conceptually `extstructure*` modifies the extended structure directly, rather than generating a new structure. The caveat however is, that every symbol introduced in an `extstructure*` **must** be defined.

Consider the following conservative extension:

Example 13

Input:

File [sTeX/MathTutorial]algebra/MagmaSquare.en.tex

```
\begin{extstructure*}{magma}
  \begin{sdefinition}[macro=sq,args=1]
    \notation{sq}[op=\cdot^2]{{#1}^{\comp 2}}
    \vardef{va}[name=a,type=\univ,bind]{a}
    Let $\inset{\va}{\univ}$. We define
    $\defnotation{\sq{\va}} := \definiens{\op{\va,\va}}$.
  \end{sdefinition}
\end{extstructure*}
```

Output:

Definition 3.3.4. Let $a \in U$. We define $a^2 := a \circ a$.

Via `\definiens`, the new symbol sq is now *defined* (note the `macro=` argument, taht generates a semantic macro as well). Whenever we import the containing module, we now have an additional field `sq` in (any extension of) magma – e.g., as semigroup extends magma, the following is now valid:

```
\usemodule[sTeX/MathTutorial]{algebra?MagmaSquare}
\vardef{vsg}[name=S,return=\semigroup]{S}
$\vsg{sq}{a}$
```

...producing a^2.

3.3.3 Nesting Structures and `\this`

A prehaps not too surprising, but a notable aspect of structures is that fields themselves can be instances. This is important for example for implementing *vector spaces*, but can also be used to bundle things that are not normally thought of as structures, such as objects with certain defining properties.

Take as an example, the notion of a (magma) homomorphism:

Definition 3.3.5. Let $M_1 = \langle U_1, \circ_1 \rangle$ and $M_2 = \langle U_2, \circ_2 \rangle$ magmas. A **magma homomorphism** is a function $F : U_1 \to U_2$ such that $F(a \circ_1 b) = F(a) \circ_2 F(b)$ for all $a, b \in U_1$.

So a homomorphism is a function with certain properties. And structures can be used to "bundle" the function itself with both the magmas on whose universes the function operates, as well as the *axiom* that *makes* it a homomorphism. After all, considered as a mere function, $F : U_1 \to U_2$ contains no information about the operation with respect to which it is homomorphic.

The first thing to note is that we can provide `mathstructure` with an optional argument for a *name* distict from the name of its semantic macro. We then add two fields that `return` magmas. So far, so unexciting:

```
\begin{mathstructure}{magmahom}[magma homomorphism]
  \symdef{dom}[name=domain,return={\magma[comp={##1}_1]}]{M_1}
  \symdef{cod}[name=codomain,return={\magma[comp={##1}_2]}]{M_2}
```

For the function itself, we know how to give it a maningful type, already:

```
\symdef{f}[type=\funspace{\dom{univ}}{\cod{univ}},args=1]{???}
```

...but what should its notation be? Ideally we would want it to just be the notation of whatever particular instance it is – in informal mathematics, we rarely distinguish notationally between a homomorphism and its underlying function (to the point where it's not clear, whether *informally* the distinction is even meaningful). Similarly, we rarely distinguish e.g. between a magma (or semigroup, monoid, group, ring, vector space,...) and its underlying universe.

This is where `\this` comes into play (pun intended). Within an `mathstructure` or `exstructure`, or in the context of a particular instance of one, `\this` represents "the" instance.

We can set it in the context of `mathstructure` as a further optional argument; e.g.

```
\begin{mathstructure}{magmahom}[magma homomorphism,this=F]
```

and then use `\this` in the notation for the function. We can further provide the homomorphism condition as an axiom using `\inlineass`:

Example 14

Input:

File [sTeX/MathTutorial]algebra/Homomorphism.en.tex

```
\begin{mathstructure}{magmahom}[magma homomorphism,this=F]
  \symdef{dom}[name=domain,return={\magma[comp={##1}_1]}]{M_1}
  \symdef{cod}[name=codomain,return={\magma[comp={##1}_2]}]{M_2}
  \symdef{f}[op=\this,args=1,
    type=\funspace{\dom{univ}}{\cod{univ}}
  ]{\this \dobrackets{#1}}

  \begin{sdefinition}[for={magmahom,dom,cod,f}]
    \vardef{va}[name=a,type=\dom{univ}]{a}
    \vardef{vb}[name=b,type=\dom{univ}]{b}
    Let $\dom!=\dom{}$ and $\cod!=\cod{}$ \sns{magma}.
    A \definame{magmahom} is a function
    $\fun{\f!}{\dom{univ}}{\cod{univ}}$ such that
    \inlineass[name=homomorphism condition]{\conclusion{\foral{
      $\arg[2]{\eq{
        \f{\dom{op}{\va,\vb}}, \cod{op}{\f{\va},\f{\vb}}
      }}$ \comp{for all} $\arg[1]{\inset{\va,\vb}{\dom{univ}}}$.
    }}}
  \end{sddefinition}
\end{mathstructure}
```

Output:

Definition 3.3.6. Let $M_1 = \langle U_1, \circ_1 \rangle$ and $M_2 = \langle U_2, \circ_2 \rangle$ magmas. A **magma homomorphism** is a function $F : U_1 \to U_2$ such that $F(a \circ_1 b) = F(a) \circ_2 F(b)$ for all $a, b \in U_1$.

Now if we instantiate our magma homomorphism:

```
\vardef{vh}[name=H,return={\magmahom[this=H]}]{H}
```

Here is a list of what we can do now:

Syntax	Result
`$\vh!$`	H
`$\vh{}$`	$\langle M_1, M_2, H\rangle$
`$\vh{f}!$`	H
`$\vh{f}{a}$`	$H(a)$
`$\vh{dom}!$`	M_1
`$\vh{cod}{}$`	$\langle U_2, \circ_2\rangle$
`$\vh{cod}{univ}$`	U_2
`$\vh{dom}{op}!$`	$\circ_1$
`$\vh{cod}{op}{a,b,c}$`	$a \circ_2 b \circ_2 c$

Note how – as one would expect – we can treat `\vh{dom}` and `\vh{cod}` like any other instance of magma.

Note that some of the outputs in the above table are probably not quite what we want. Determining the precise typesetting of an expression involving *nested paths* of fields is difficult, to say the least (e.g., what exactly should `\this` refer to in a deeply nested sequence of fields?).
Using instances within structures is still very useful; at the very least when defining structures. When subsequently *using* structures, however, accessing fields of fields (of fields (of ...)) of an instance should be avoided.
Luckily, there is rarely a need for doing so – in practice, those fields we might want to access in such a way, we usually also want to provide specific notations and talk about independently of the "containing" instance, such that introducing a new variable (or symbol), and assigning the corresponding field to that variable, makes considerably more sense. And subsequently using the variable is easier than concatenating `{...}`, too.

3.4 Complex Inheritance and Theory Morphisms

We are starting to approach seriously experimental territory.
While the theory behind all the following is relatively well understood, and their implementation in MMT is mature, the same can not be said out the implementation in sTEX.
There are still kinks to be ironed out, but feel free to experiment.

We now have all the tools available to progress towards something more interesting. Here is a list of documents with respective modules and symbols we will build on in the following:

`[sTeX/MathTutorial]props/Idempotent.en.tex`

Definition 3.4.1. Let $e \in A$ and $\circ : A \times A \to A$. e is called **idempotent** with respect to $\circ$, if $e \circ e = e$.

Definition 3.4.2. The operation $\circ : A \times A \to A$ is called **idempotent**, if every element $a \in A$ is idempotent with respect to $\circ$.

[sTeX/MathTutorial]props/Distributive.en.tex

Definition 3.4.3. Let $\odot : B \times A \to A$ and $\oplus : A \times A \to A$. We say $\odot$ **distributes over** $\oplus$, if $b \odot (a_1 \oplus a_2) = (b \odot a_1) \oplus (b \odot a_2)$ for all $a_1, a_2 \in A$ and $b \in B$.

[sTeX/MathTutorial]props/Absorption.en.tex

Definition 3.4.4. Let $\odot : A \times B \to A$ and $\oplus : A \times B \to B$. We say $\odot$ **absorbs** $\oplus$, if $a_1 \odot (a_1 \oplus b) = a_1$ for all $a_1 \in A$ and $b \in B$.

[sTeX/MathTutorial]algebra/Band.en.tex

Definition 3.4.5. A **band** is an idempotent semigroup.

[sTeX/MathTutorial]algebra/Semilattice.en.tex

Definition 3.4.6. A **semilattice** is a commutative band.

[sTeX/MathTutorial]props/Reflexive.en.tex

Definition 3.4.7. A binary relation R on A is called **reflexive**, if $R(a, a)$ for all $a \in A$.

[sTeX/MathTutorial]props/Symmetric.en.tex

Definition 3.4.8. A binary relation R on A is called **symmetric**, if $R(a, b)$ implies $R(b, a)$ for all $a, b \in A$.

[sTeX/MathTutorial]props/Transitive.en.tex

Definition 3.4.9. A binary relation R on A is called **transitive**, if $R(a, b)$ and $R(b, c)$ implies $R(a, c)$ for all $a, b, c \in A$.

[sTeX/MathTutorial]props/Antisymmetric.en.tex

Definition 3.4.10. A binary relation R on A is called **antisymmetric**, if $R(a,b)$ and $R(b,a)$ implies $a = b$ for all $a, b \in A$.

[sTeX/MathTutorial]orders/Graph.en.tex

Definition 3.4.11. A **directed graph** is a structure $\langle U, R\rangle$, where U is a collection and R a binary relation on U.

Definition 3.4.12. An **(undirected) graph** is a directed graph $\langle U, R\rangle$, where R is symmetric.

[sTeX/MathTutorial]orders/Preorder.en.tex

Definition 3.4.13. A structure $\langle U, \leq\rangle$ is called a **preorder** (or **quasiorder**, or **preordered set**; in short **proset**), if $\leq$ is reflexive and transitive.

[sTeX/MathTutorial]orders/Poset.en.tex

Definition 3.4.14. A preorder $\langle U, R\rangle$ is called a **partial order** (or **poset**), if R is antisymmetric.

[sTeX/MathTutorial]orders/InfSup.en.tex

Definition 3.4.15. Let $\langle U, R\rangle$ a poset. An element $a \in U$ is called an **infimum** or **greatest lower bound** of x_1 and x_2, if $a \mathrel{R} x_1$, $a \mathrel{R} x_2$, and for any x with $x \mathrel{R} x_1$ and $x \mathrel{R} x_2$, we have $x \mathrel{R} a$.

Definition 3.4.16. Let $\langle U, R\rangle$ a poset. An element $a \in U$ is called a **supremum** or **least upper bound** of x_1 and x_2, if $x_1 \mathrel{R} a$, $x_2 \mathrel{R} a$, and for any x with $x_1 \mathrel{R} x$ and $x_2 \mathrel{R} x$, we have $a \mathrel{R} x$.

Note that infima and suprema are more generally defined on *sets* of elements. Doing so in sTeX is significantly more complicated *for now*, and will require some amount of research to make convenient – especially if we want to subsequently define *operators* on pairs of elements, as below. We therefore opt for the simpler version where it is defined as binary from the get go.

[sTeX/MathTutorial]orders/MeetJoinSemilattice.en.tex

Definition 3.4.17. A poset $\langle U, R\rangle$ is called a **meet semilattice** if for every two elements a, b the infimum $a \wedge b$ exists.

Definition 3.4.18. A poset $\langle U, R\rangle$ is called a **join semilattice** if for every two elements a, b the supremum $a \vee b$ exists.

Definition 3.4.19. An **(order) semilattice** is a meet and join semilattice.

Exercise
Try to implement all of the above yourself!

3.4.1 Glueing Structures Together

We now want to progress towards lattices, i.e. the following:

Definition 3.4.20. A lattice is a structure $\langle U, \wedge, \vee\rangle$ such that $\langle U, \wedge\rangle$ and $\langle U, \vee\rangle$ are semilattices, and $\vee$ absorbs $\wedge$ and vice versa; i.e. $a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$.

The operations $\wedge$ and $\vee$ are called **meet** and **join**, respectively.

So we make a new module, open an `extstructure` environment and... realize two problems:

1. We can't just extend semilattice: We need *two* copies of semilattice that share a universe, and importing semilattice twice is of course redundant.
2. We also want to *rename* the operations of the two semilattices to be subsequently called `join` and `meet`.

What we need is a way to *inherit* from semilattice while a) *modifying* the symbols therein, and b) not be idempotent – i.e. two imports from the same structure or module should not be identified. We can do that with the `\copymod` macro, which takes three arguments:

1. A *name* for the copy,
2. the structure or module to copy, and
3. a comma-separated list of renamings and redefinitions of the symbol. ⟨*symbol*⟩=⟨*def*⟩ redefines ⟨*symbol*⟩, ⟨*symbol*⟩@⟨*newname*⟩ renames it, ⟨*symbol*⟩=⟨*def*⟩@⟨*newname*⟩ (or ⟨*symbol*⟩@⟨*newname*⟩=⟨*def*⟩) does both.

In our case, we want two copies of semilattice, which we will call `meetsl` and `joinsl`. In the first copy, we only want to rename `op` to `meet`. In the second, we want to rename `op` to `join`, and *also* redefine the universe to be the one from `meetsl`:

```
\copymod{meetsl}{semilattice}{
  op @ meet
}
\copymod{joinsl}{semilattice}{
  univ = \univ,
  op @ join
}
```

You might have already noticed some problem with that – which of the two universes does `\univ` refer to now? (They are *defined* as equal, but LaTeX does not know that!) Or which of the two `commutative axioms` does "`commutative axiom`" refer to now? Everything is ambiguous now!

Not really - if you have wondered why the `\copymod` takes a *name* as argument: The name is prefixed to every symbol name. Hence, the `universe` in `joinsl` is now called `joinsl/universe`, and the one in `meetsl` is called `meetsl/universe`. Furthermore, `\copymod` by default generates no semantic macros for any of the imported symbols – except for those renamed with `@`. In fact, what the `@` syntax actually does, is to generated a semantic macro by that name. If we want to change the *name* (that is shown when using `\symname` et al), we add that new name in square brackets. Hence, what we really want to do is:

```
\copymod{meetsl}{semilattice}{
  univ @ univ,
  op @ [meet]meet
}
\copymod{joinsl}{semilattice}{
  univ = \univ,
  op @ [join]join
}
```

This now gives us two copies of semilattice, generates semantic macros `\univ` for `meetsl/universe`, `\meet` for `meetsl/op` and `\join` for `joinsl/op`, and renames `meetsl/op` to `meet` and `joinsl/op` to `join`.

That allows us to then add the absorption axioms, an `sdefinition` for lattice and subsequently `$\lattice!$` produces $\langle U, \wedge, \vee\rangle$, with all axioms inherited (see `[sTeX/MathTutorial]algebra/Lattice.en.tex`).

3.4.2 Realizations

A very common situation we find in connection with mathematical structures is that "every *this* is a *that*" (or the conrete case "*this* is a *that*").

With what we did so far, we are in this situation regarding the algebraic definition of semilattices and the order-theoretic one (exemplary meet semilattice).

In MMT parlance, this corresponds to a total (implicit) theory morphism from "that" to "this".

In sTeX words, we want to inherit from "that" by assigning all the symbols in "that" to concrete terms. In our case:

`[sTeX/MathTutorial]algebra/SemiLatticeOrder.en.tex`

Definition 3.4.21. Let $\langle U, \circ\rangle$ a semilattice. We let $a \; R \; b$ iff $a \circ b = a$.

Satz 3.4.22. $\langle U, R\rangle$ *is a meet semilattice.*

Proof:
We need to prove the following

1. reflexivity ($a\ R\ a$):
2. antisymmetry ($a\ R\ b$ and $b\ R\ a$ implies $a = b$):

   Assume $a \circ b = a$ and $b \circ a = b = a \circ b$ (by the commutative axiom). Hence, $a = b$
3. transitivity (If $a\ R\ b$ and $b\ R\ c$, then $a\ R\ c$.):

   Assume $a \circ b = a$ and $b \circ c = b$ Then $a \circ c = (a \circ b) \circ c = a \circ (b \circ c) = a \circ b = a$. Hence, $a\ R\ c$.
4. $a \circ b$ is the infimum of $\{a, b\}$:

   By definition (and the commutative axiom), $a \circ b\ R\ a$ and $a \circ b\ R\ b$. We need to show, that if $x\ R\ a$ and $x\ R\ b$, then $x\ R\ a \circ b$. Assume $x \circ a = x$ and $x \circ b = x$. Then $x \circ (a \circ b) = (x \circ a) \circ b = x \circ b = x$. Hence $x\ R\ a \circ b$

□

So to be precise, we want to provide *definientia* for all undefined symbols in meet semilattice (i.e. the relation and meet) and *proofs* for all *axioms* (reflexive axiom, antisymmetric axiom, transitive axiom, and infimum axiom), and by so obtain the fact that every semilattice is a meet semilattice.

For that purpose, we have the `\realize` macro. It behaves like `\copymod`, but does not take a name, and additionally requires that all undefined fields get assigned. So we could do the following:

Example 15
Input:

File [sTeX/MathTutorial]`algebra/SemiLatticeOrder1.en.tex`

```
\begin{extstructure*}{semilattice}
  \interpretmod{meetsl}{meetsl}{
    univ = \univ,
    meet @ [meet]meet = \op!,
    rel @ [order]order = \map{a,b}{\eq{\op{a,b},a}},
    reflexive axiom = trivial,
    transitive axiom = trivial,
    antisymmetric axiom = trivial,
    infimum axiom = trivial
  }
\end{extstructure*}

\vardef{mysl}[return=\semilattice]{S}
$\mysl{order}{a,b} \qquad \mysl{}[univ,op,order]$
```

Output:

$a \leq_S b \qquad \langle U_S, \circ_S, \leq_S \rangle$

As we can see, we can now access the field `order`, which is renamed from `relation` in

meet semilattice and also has the desired definiens in MMT. But of course this approach is very "declarative": We do all the assigning in one macro, rather than narratively as what they *should* be: definitions and proofs.

If we want to achieve the more narrative version at the beginning of the subsection, we can use the `realization` environment instead. It behaves like the `\realize` macro, but allows us to do the assignments and renamings individually somewhere in the body of the environment, interleaved with arbitrary text. Additionally, within the environment, all sTeX features that introduce *definientia* (like the `\definiens` macro) induce assignments instead.

To declaratively rename or assign fields, we can then use the `\assign` and `\renamedecl` macros instead. That allows us to do the following instead:

```
\begin{realization}{meetsl}
  \assign{univ}{\univ}
  \assign{meet}{\op!}
  \renamedecl{rel}[order]{order}
  ...
```

...and then use text to do the remaining assignments. For example, we can use the `sdefinition` environment to assign `rel` to the desired definiens:

```
\usestructure{meetsl}
\begin{sdefinition}[for=order]
  \varbind{va,vb}
  Let $\semilattice![univ,op]$ a \sn{semilattice}.
  We let $\rel{\va,\vb}$
  iff $\definiens{\eq{\op{\va,\vb},\va}}$.
\end{sdefinition}
```

And now sTeX will use the `\definiens` to assign $a, b \mapsto a \circ b = a$ to the relation of meet semilattice.

Analogously, we can use the `sproof` and `subproof` environments to produce "definientia" (i.e. proofs) for the axioms (see `[sTeX/MathTutorial]algebra/SemiLatticeOrder.en.tex`)

Chapter 4

Extensions for Education

The last two chapters have shown generic markup and semantization facilities in sTeX. As said before, investments in semantic markup pay off, iff the impact of a document is high, e.g. if there are many more readers than authors or if the semantic services afforded by the semantic markup can help reduce the help readers need to understand the material.

Educational documents constitute one category of high-impact documents which are supported by the sTeX ecosystem, we will cover them here. In fact, educational documents have been one of the initial document categories sTeX has been developed for. The idea is that if we can mark up the meaning and didactic role of learning objects, we can base learning support services on that and embed them into the documents.

Another reason educational documents are particularly interesting is that in a sense all academic communication is educational, as all documents try to "teach" the reader new concepts and results.

Concretely, we cover a document class for combining slides and course notes (section 4.1) and functionality for marking up problems and excercises (section 4.2) and for marking up homework assignments and exams (section 4.3).

4.1 Slides and Course Notes

4.1.1 Introduction

The notesslides document class is derived from `beamer.cls` [**beamerclass:on**], it adds a "notes version" for course notes that is more suited to printing than the one supplied by `beamer.cls`.

The notesslides class takes the notion of a slide frame from Till Tantau's excellent beamer class and adapts its notion of frames for use in sTeX. To support semantic course notes, it extends the notion of mixing frames and explanatory text, but rather than treating the frames as images (or integrating their contents into the flowing text), the notesslides package displays the slides as such in the course notes to give students a visual anchor into the slide presentation in the course (and to distinguish the different writing styles in slides and course notes).

In practice we want to generate two documents from the same source: the slides for presentation in the lecture and the course notes as a narrative document for home study.

To achieve this, the notesslides class has two modes: *slides mode* and *notes mode* which are determined by the package option.

4.1.2 Package Options

The notesslides class takes a variety of class options:

`slides` `notes` The options `slides` and `notes` switch between slides mode and notes mode (see subsection 10.1.3).

`sectocframes` `topsect` If the option `sectocframes` is given, then for the `sfragment`s, special frames with the `sfragment` title (and number) are generated. The `topsect` allows to specify the top section level (e.g. `chapter`, `section`, ...) for documents that are formally stand-alone, but intended to be chapters, sections, or ....

EdN:1 [1]

`frameimages` `fiboxed` If the option `frameimages` is set, then slide mode also shows the `\frameimage`-generated frames (see ???). If also the `fiboxed` option is given, the slides are surrounded by a box.

4.1.3 Notes and Slides

`frame` (*env.*) Slides are represented with the `frame` environment just like in the beamer class, see [**Tantau:ugbc**] for details.

`note` (*env.*) The notesslides class adds the `note` environment for encapsulating the course note fragments. Content of this environment is only shown in *notes mode*.

By interleaving the `frame` and `note` environments, we can build course notes as shown here:

```
\ifnotes\maketitle\else
\frame[noframenumbering]\maketitle\fi

\begin{note}
  We start this course with ...
\end{note}

\begin{frame}
  \frametitle{The first slide}
  ...
\end{frame}
\begin{note}
  ... and more explanatory text
\end{note}

\begin{frame}
  \frametitle{The second slide}
  ...
\end{frame}
...
```

[1] EdNote: MK: maybe this option should be used in the other sTEX classes as well. Actually I think this already is treated by stex.sty; where to document?

`\ifnotes` Note the use of the `\ifnotes` conditional, which allows different treatment between `notes` and `slides` mode – manually setting `\notestrue` or `\notesfalse` is strongly discouraged however.

> We need to give the title frame the `noframenumbering` option so that the frame numbering is kept in sync between the slides and the course notes.

> The beamer class recommends not to use the `allowframebreaks` option on frames (even though it is very convenient). This holds even more in the `notesslides` case: At least in conjunction with `\newpage`, frame numbering behaves funnily (we have tried to fix this, but who knows).

`\inputref*` If we want to transclude a the contents of a file as a note, we can use a new variant `\inputref*` of the `\inputref` macro: `\inputref*{foo}` is equivalent to `\begin{note}\inputref{foo}\end{note}`.

`nparagraph` (*env.*)
`nparagraph` (*env.*)
`ndefinition` (*env.*)
`nexample` (*env.*)
`nsproof` (*env.*)
`nassertion` (*env.*)

There are some environments that tend to occur at the top-level of `note` environments. We make convenience versions of these: e.g. the `nparagraph` environment is just an `sparagraph` inside a `note` environment (but looks nicer in the source, since it avoids one level of source indenting). Similarly, we have the `nfragment`, `ndefinition`, `nexample`, `nsproof`, and `nassertion` environments.

4.1.4 Managing and Customizing Themes

As the `notesslides` package is based on the `beamer` class, it can in principle (in slides mode) take advantage of all `beamer` themes. Themes can be specified/loaded with the normal `\usetheme`, but the `notesslides` package provides the `\libusetheme` macro:

`\libusetheme` In analogy to the `\libinput` functionality of the `stex` package, this macro allows loading `beamer` themes from the `lib` directories leading up to the current archive.

Unfortunately, there is no `beamer` functionality of using a themed `frame` environment in `article` mode. Therefore `notesslides` has to hand-craft one – for a very limited set of themes. The notesslides package and class comes with a simple default theme named sTeX that provided by the beamterthemesTeX.sty style file. It is assumed as the default theme for sTEX-based notes and slides. The result in `notes` mode (which is like the `slides` version except that the slide height is variable) is

The footer line can be customized. In particular the logos.

\setslidelogo The default logo provided by the notesslides package is the sTeX logo it can be customized using \setslidelogo{⟨*logo name*⟩}.

\setsource The default footer line of the notesslides package mentions copyright and licensing. In notesslides \source stores the author's name as the copyright holder. By default it is the author's name as defined in the \author macro in the preamble. \setsource{⟨*name*⟩} can change the writer's name.

\setlicensing For licensing, we use the Creative Commons Attribution-ShareAlike license by default to strengthen the public domain. If package hyperref is loaded, then we can attach a hyperlink to the license logo. \setlicensing[⟨*url*⟩]{⟨*logo name*⟩} is used for customization, where ⟨*url*⟩ is optional.

These macros can be used set up a institution-specific theme; for instance the one for FAU simply customizes the logo:

```
% Beamer slide theme for FAU;
\typeout{Beamer FAU theme}
\RequirePackage{beamerthemesTeX}
\setslidelogo[courses/FAU/meta-inf]{PIC/FAU_Logo_Bildmarke.png}
```

This could be placed in one of the lib folders available to the current archive and could then be activated with

```
\documentclass[slides]{notesslides}
\libusetheme{FAU}
```

in the preamble of the main noteslides file.

4.1.5 Frame Images

Sometimes, we want to integrate slides as images after all – e.g. because we already have a PowerPoint presentation, to which we want to add sTeX notes.

\frameimage
\mhframeimage In this case we can use \frameimage[⟨*opt*⟩]{⟨*path*⟩}, where ⟨*opt*⟩ are the options of \includegraphics from the graphicx package [**CarRah:tpp99**] and ⟨*path*⟩ is the file path (extension can be left off like in \includegraphics). We have added the label key that allows to give a frame label that can be referenced like a regular beamer frame.

The \mhframeimage macro is a variant of \frameimage with repository support. Instead of writing

```
\frameimage{\MathHub{fooMH/bar/source/baz/foobar}}
```

we can simply write (assuming that \MathHub is defined as above)

```
\mhframeimage[fooMH/bar]{baz/foobar}
```

Note that the \mhframeimage form is more semantic, which allows more advanced document management features in MathHub.

If baz/foobar is the "current module", i.e. if we are on the MathHub path ...MathHub/fooMH/bar..., then stating the repository in the first optional argument is redundant, so we can just use

```
\mhframeimage{baz/foobar}
```

\textwarning

The \textwarning macro generates a warning sign: ⚠

4.1.6 Ending Documents Prematurely

\prematurestop
\afterprematurestop

For prematurely stopping the formatting of a document, sTEX provides the \prematurestop macro. It can be used everywhere in a document and ignores all input after that – backing out of the sfragment environments as needed. After that – and before the implicit \end{document} it calls the internal \afterprematurestop, which can be customized to do additional cleanup or e.g. print the bibliography.

\prematurestop is useful when one has a driver file, e.g. for a course taught multiple years and wants to generate course notes up to the current point in the lecture. Instead of commenting out the remaining parts, one can just move the \prematurestop macro. This is especially useful, if we need the rest of the file for processing, e.g. to generate a theory graph of the whole course with the already-covered parts marked up as an overview over the progress; see import_graph.py from the lmhtools utilities [**lmhtools:github:on**].

Text fragments and modules can be made more re-usable by the use of global variables. For instance, the admin section of a course can be made course-independent (and therefore re-usable) by using variables (actually token registers) courseAcronym and courseTitle instead of the text itself. The variables can then be set in the sTEX preamble of the course notes file.

4.1.7 Global Document Variables

To make document fragments more reusable, we sometimes want to make the content depend on the context. We use **document variables** for that.

\setSGvar
\useSGvar

\setSGvar{⟨*vname*⟩}{⟨*text*⟩} to set the global variable ⟨*vname*⟩ to ⟨*text*⟩ and \useSGvar{⟨*vname*⟩} to reference it.

\ifSGvar

With\ifSGvar we can test for the contents of a global variable: the macro call \ifSGvar{⟨*vname*⟩}{⟨*val*⟩}{⟨*ctext*⟩} tests the content of the global variable ⟨*vname*⟩, only if (after expansion) it is equal to ⟨*val*⟩, the conditional text ⟨*ctext*⟩ is formatted.

4.1.8 Excursions

In course notes, we sometimes want to point to an "excursion" – material that is either presupposed or tangential to the course at the moment – e.g. in an appendix. The typical setup is the following:

```
\excursion{founif}{../fragments/founif.en}
  {We will cover first-order unification in}
...
\begin{appendix}\printexcursions\end{appendix}
```

It generates a paragraph that references the excursion whose source is in the file `../fragments/founif.en.tex` and automatically books the file for the `\printexcursions` command that is used here to put it into the appendix. We will look at the mechanics now.

`\excursion` The `\excursion{⟨ref⟩}{⟨path⟩}{⟨text⟩}` is syntactic sugar for

```
\begin{nparagraph}[title=Excursion]
  \activateexcursion{founif}{../ex/founif}
  We will cover first-order unification in \sref{founif}.
\end{nparagraph}
```

`\activateexcursion` `\printexcursion` `\excursionref` Here `\activateexcursion{⟨path⟩}` augments the `\printexcursions` macro by a call `\inputref{⟨path⟩}`. In this way, the `\printexcursions` macro (usually in the appendix) will collect up all excursions that are specified in the main text.

Sometimes, we want to reference – in an excursion – part of another. We can use `\excursionref{⟨label⟩}` for that.

`\excursiongroup` Finally, we usually want to put the excursions into an `sfragment` environment and add an introduction, therefore we provide the a variant of the `\printexcursions` macro: `\excursiongroup[id=⟨id⟩,intro=⟨path⟩]` is equivalent to

```
\begin{note}
\begin{sfragment}[id=<id>]{Excursions}
  \inputref{<path>}
  \printexcursions
\end{sfragment}
\end{note}
```

Contents

4.2 Problems and Exercises

4.2.1 Background

Problems/exercises are text fragments that contain a task assigned to the learner – e.g. computing the value of a specified quantity, simplifying an expression, modeling a described situation in a mathematical structure, or judging the veracity of a given statement. Problems/exercises are used in very different contexts, in

- *homework assignments and formal exams*, where they are graded and points are awarded for course credit,

- *self-study materials* that allow learners explore applications of some concepts and/or practice,
- *automated tutoring systems* to determine learner competencies to trigger computer supported learning services.

In all of these contexts, diagnosis the correctness or suitability of an answer plays a great role, e.g. for grading, the generation of feedback, or the suggestion of remedial actions that may "heal" any diagnosed competency gaps or misconceptions. To support that we might want to specify "answer classes" that classify answers received for automating/triggering feedback and grading.

There are various types of problems/exercises in practical use. They are mainly distinguished by how they support diagnosis of student answers: there are

- open problems, where expected answers are free-form, and classification into answer classes is usually a manual process; see subsection 4.2.3
- single/multiple choice questions where the answer classes and thus feedback/grading correspond to the selection pattern of items; see subsection 4.2.7.
- fill-in-the-blanks questions where we may want to specify how the answer string is interpreted; see subsection 4.2.8.

Additionally, problems and exercises often come with text fragments that serve auxiliary functions: hints, notes, and and grading specifications. Furthermore, we can specify how long solving a given problem is estimated to take and how many points will be awarded for a perfect solution – e.g. in an exam. See subsection 4.2.10 for details.

The `problem` package gives semantic markup support for all these aspects of problems/exercises with a focus on problem libraries that collect carefully formulated and tested problems/exercises, so that they can be re-used in many contexts. It also tries to support all possible stakeholders in dealing with problems/exercises:

- *learners*, who try to solve problems and may profit from feedback
- *instructors*, who pose problems in homeworks, quiz, and exams,
- *graders*, who try to assess learner's competencies from the answers given
- *authors* of learning objects and (automated) *course generators*, who may use problems to diagnose learner's competencies (e.g. as prerequisites).

4.2.2 The Package, Options, and Configuration

The problem package provides functionality for marking up problems and exercises as semantic sources from which the presentations for the various contexts can be generated. E.g. documents without solutions for paper or online exams, and the corresponding exams with master solutions for exam reviews. Similarly with/without hints, or points. Their visibility is specified in the options of the problem package, which can be used in any LaTeX class. The following is a typical preamble for a problem file:

```
\documentclass[lang=de]{article}
\usepackage[solutions,hints,pts,min]{problem}
```

Here we have specified the options `solutions` (solutions should be shown), `hints` (hints should be given), `pts` (display the points awarded for solving the problem?), `min` (display the estimated minutes for problem solving). Leaving out the options would make the corresponding functionality invisible.

EdN:2 The `problem` package generates content and labels according to the language specified in the `lang` key of the main document[2], these can be localized by in the files `ldf/problem-<lang>.ldf`[6].

The `problem` package also supplies the `test` option, which specifies that the content is intended for use in an assessment situation, where certain additional content (e.g. exam legalse) should be shown while other content (e.g. solutions) should not be.

Note that documents (or document fragments) with problems are often formatted in different versions that can be configured by these options. Therefore the following variant of the preamble prefix above can be very useful:

```
\documentclass[lang=de]{article}
\providecommand\probopts{,solutions,min}
\usepackage[hints,pts\probopts]{problem}
```

We add a level of indirection via the `\probopts` macro (`\providecommand` sets it unless it already exists). If the current file is `foo.tex`, then we can make a LaTeX file `foo-test.tex` that formats in test mode as

```
\newcommand\probopts{,test}\input{foo}
```

Alternatively, we do not need a file at all: we can just (e.g. in a Makefile or a PDF generation script) issue the following command in the terminal:

```
pdflatex "\def\probopts{,test}\input{foo}"
```

4.2.3 (Open) Problems

The main environment provided by the problem package is (surprise surprise) the `sproblem` environment. It is used to mark up problems and exercises. The following example shows the main functionality:

Example 16
Input:

[2] EdNote: MK: document the attribute somewhere

[6] The current crop of language definition files is quite limited. Please feel free to contribute the to the localization effort

File tutorial/ext/simple-problem.en.tex

```
\begin{document}
\begin{sproblem}[id=prob.elefants,pts=10,min=2,title=Fitting Elefants]
  How many Elefants can you fit into a Volkswagen beetle?
  \begin{hint}
    Think positively, this is simple!
  \end{hint}
  \begin{exnote}
    Justify your answer
  \end{exnote}
  \begin{gnote}[title=deduct 5pts if justification is missing]
    \anscls[feedback=you forget that elephants could be small]{none}
    \anscls[feedback=you forget the back seats]{2}
  \end{gnote}
  \begin{solution}
    Four, two in the front seats, and two in the back.
```

Output:

Exercise (Fitting Elefants)
How many Elefants can you fit into a Volkswagen beetle?

Lösung: Four, two in the front seats, and two in the back.

The `sproblem` environment takes an optional key/value argument with the keys `id` as an identifier that can be reference later, `pts` for the points to be gained from this exercise in homework or quiz situations, `min` for the estimated minutes needed to solve the problem, and finally `title` for an informative title of the problem.

The `sproblem` can contain several `solution` environments, which take the well-known `id`, `style`, and `title` attributes, and also the `testspace` and `answerclass`. The former

The additional functionality is specified in the `hint` `exnote` (notes in exercises), `solution`, and `gnote` (grading notes) environments. Here, the first three are shown whereas the grading notes are hidden, since the corresponding option was not given in the `\usepackage[...]{problem}`. All of these environments can occur any number of times in the `sproblem` environment. The `solution` environment takes an optional argument that is interpreted as the identifier.

4.2.4 Solutions and Answer Classes

Most problem libraries also contain (master) solutions, which show the intended way of solving a problem. The `solution` environment allows to specify solutions. It is only formatted if the `solutions` options is specified for the `problem` package.

We observe that many problems have more than one possible solution, so the `problem` package allows multiple `solution` environments; they can be distinguished by an `id` key in the optional first argument.

We also observe that in problem libraries we may want to keep track of classes of answers that have been observed e.g. in previous assessment situations, e.g. to provide

feedback. In this setting, the `solution` environment can be used to encode (typical representative of) **answer classes** (see also ??? below).

`solution` The `solution` environment takes an optional argument that allows the keys `id`, `title` and `style` keys that we have already seen above, further more

1. `testspace=`, which is equivalent to a `\testpsace{}` (see subsection 4.2.10).
2. `answerclass=`, which gives the name of an answer class, this solution corresponds to.[3]

EdN:3

4.2.5 Grading Support

When problems are graded by multiple persons (e.g. for large university exams), then consistency of grading is a major issue. Both within a cohort and between cohorts e.g. in successive instances of a course.

Therefore keeping records of how to grade a problem (which problem is worth how many "points" and where to deduct how many points in which situations) and making them accessible to graders in a "master solution" – a version of an exam formatted to have the known solutions/answer classes (see above) and the grading specifications is a good practice. And it makes sense to include grading information into a problem library.

The simplest and arguably most effective measure is to specify the points that can be reached for a problem in the problem itself via the 'pts=' key (but see subsection 4.2.9)

The `problem` package also supplies the `gnotes` environment, which is only formatted when the `gnotes=` key is given. This environment takes an optional argument that allows the `id`, `style`, and `title` keys. The latter is used to give a short version of the grading specification. More structured grading support can be expressed in `\anscls` markup which we discuss next.

We have already encountered the notion of answer classes – classes of answers that are supposed equivalent in terms of learner's competencies inscribed in them – above. These are the natural carriers of both grading feedback and points.

In the `gnote` environment we use the `\anscls` macro for specifying such information using meta-level descriptions rather than typical representatives. It takes an optional keyword argument for the grading/feedback information and a required one for the answer class description.

We distinguish two usages of `\anscls[...]{<desc>}`: in

- *answer classes* `<desc>` is a specification of a class of answers and the `pts=` key the points to be awarded to that.
- *answer traits* `<desc>` specifies the deviation from an assumed correct solution, and the value of the `pts=` key specifies changes to the points specified in the problem itself.

Consider for instance the following situation:

```
\begin{sproblem}[pts=3]
  Give an example for a foo and explain it in one sentence.
  \begin{gnote}[title=1 pt for the example and 2 for the explanation]
    \anscls[pts=0,feedback=I did not understand this]{complete gibberish}
    \anscls[pts=-2,feedback=You forgot the explanation]{no explanation}
  \end{gnote}
\end{sproblem}
```

[3] EdNote: This mechanism needs to be further worked our or deprecated; if we keep it we probably need to add pts and feedback attributes for parity with \anscls.

Here we have a problem that is worth three points, and the grading note explains how to treat deviations from a correct solutions – which is not given, since there may be hundreds of examples for foos. The `\anscls` specify this out for grading support systems[7]. The first is an answer class description for the class of answers that cannot be made sense of by the graders and specifies a default grade and feedback. The second specifies the trait of a missing explanation and specifies a deduction commensurate with the grading note title and a feedback to the learner. Note that a grading support system can distinguish answer classes from traits by the form of the value of points: absolute values vs. differences like `+5` or `-3`.

4.2.6 Structured Problems

Problems can be structured into subproblems via the `subproblem` environment. It takes the same arguments and allows the same content model as as `sproblem`.

Example 17

Input:

File `tutorial/ext/structured-problem.en.tex`

```
\begin{document}
\begin{sproblem}[id=prob.elefants-mod,title=Fitting Elefants Modularly]
  Consider the problem of fitting elephants into a Volkswagen beetle.
  \begin{subproblem}[pts=1,min=2]
    Estimate the number of elephants on the front seats.
    \begin{solution}
      Two on each side one.
    \end{solution}
  \end{subproblem}
  \begin{subproblem}[pts=1,min=2]
    Estimate the number of elephants on the back seats.
    \begin{solution}
      Three little elephants.
    \end{solution}
  \end{subproblem}
```

Output:

[7] like e.g. `https://gitos.rrze.fau.de/yn06uhoc/vollkorn-prototype`

Exercise (Fitting Elefants Modularly)
Consider the problem of fitting elephants into a Volkswagen beetle.

1. Estimate the number of elephants on the front seats.

   Lösung: Two on each side one.

2. Estimate the number of elephants on the back seats.

   Lösung: Three little elephants.

3. What is the total number?

   Lösung: A family of five; we do not want elephants in the frunk.

By default, subproblems are numbered subordinate to the "mother problem". The `problem` package does not make any assumptions about whether subproblems can be used independently from their sister problems, or on order requirements.

Note that neither `sproblem` nor `subproblem` can be nested, if you want to have further structure, you need to resort to regular LaTeX functionality. Note that you can add `solution` and `gnote` environments wherever you want, so that this need not force you to forego structure.

The `subproblem` environment is however very useful for modularizing problems: `subproblem` environments can appear outside of `sproblem` and in particular at the top-level of a file. This allows them to be shared between `sproblem` via `\inputref`.

4.2.7 Single/Multiple Choice Blocks

Multiple choice blocks can be formatted using the `mcb` environment, in which single choices are marked up with `\mcc` macro.

`\mcc`

`\mcc[`⟨*keyvals*⟩`]{`⟨*text*⟩`}` takes an optional key/value argument ⟨*keyvals*⟩ for choice metadata and a required argument ⟨*text*⟩ for the proposed answer text. The following keys are supported

- `T` for correct answers, `F` (the default value) for false ones,
- `Ttext` the verdict for correct answers, `Ftext` for false ones, and
- `feedback` for a short feedback text given to the student.

What we see when this is formatted to PDF depends on the context. In solutions mode (we start the solutions in the code fragment below) we get

Example 18

Input:

```
\startsolutions
\begin{sproblem}[title=Functions,id=functions1]
  What is the keyword to introduce a function definition in python?
  \begin{mcb}
    \mcc[T]{def} \mcc[F,feedback=that is for C and C++]{function}
    \mcc[F,feedback=that is for Standard ML]{fun}
    \mcc[F,Ftext=Nooooooooo,feedback=that is for Java]{public static void}
  \end{mcb}
\end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☑ def
- ☐ function[a]
- ☐ fun[b]
- ☐ public static void[c]

---

[a] *that is for C and C++*

[b] *that is for Standard ML*

[c] Nooooooooo *that is for Java*

In "exam mode" where disable solutions (here via `\stopsolutions`) we get the questions without solutions (that is what the students see during the exam/quiz).

Example 19

Input:

```
\stopsolutions
\begin{sproblem}[title=Functions,id=functions1]
  What is the keyword to introduce a function definition in python?
  \begin{mcb}
    \mcc[T]{def} \mcc[F,feedback=that is for C and C++]{function}
    \mcc[F,feedback=that is for Standard ML]{fun}
    \mcc[F,Ftext=Nooooooooo,feedback=that is for Java]{public static void}
  \end{mcb}
\end{sproblem}
```

Output:

Exercise (Functions)
What is the keyword to introduce a function definition in python?

- ☐ def
- ☐ function
- ☐ fun
- ☐ public static void

What we have seen is the default rendering of the multiple choice block, which is as an itemize like environment with check boxes. There are alternatives to that which we can choose with the `style` key in the optional attribute of the `mcb` environment. Currently the only alternative is `inline` style:

Example 20
Input:

```
\startsolutions
\begin{sproblem}[title=Functions,id=functions1]
 What is the keyword to introduce a function definition in python?

 \begin{mcb}[style=inline]
   \mcc[T]{def} \mcc[F,feedback=that is for C and C++]{function}
   \mcc[F,feedback=that is for Standard ML]{fun}
   \mcc[F,Ftext=Nooooooooo,feedback=that is for Java]{public static void}
 \end{mcb}
\end{sproblem}
```

Output:

Exercise (Functions)
What is the keyword to introduce a function definition in python?

[☑ def ☐ function[a] ☐ fun[b] ☐ public static void[c]]

[a] *that is for C and C++*
[b] *that is for Standard ML*
[c] Nooooooooo
that is for Java

This is the solutions mode, i.e. with solutions, otherwise, the footnotes will fully disappear.

Analogously, single choice blocks are marked up by the `scb` environment and the individual choices by the `\scc` macro. In PDF, there is little difference to the multiple choice case. In interactive formats like HTML, single choice blocks are typically realized via radio buttons to enforce single choice.[4]

EdN:4

Often we only want to ask whether a statement is true or false. This is essentially a single choice block. STEX provides the macros `\yesTnoF`, `\yesFnoT`, `\trueTfalseF`, and `\trueFfalseT` for that. They are used as in the example below:

[4] EdNote: MK: are there any differences between mcc and scc we need to discuss here?

Example 21
Input:

```
\begin{sproblem}\startsolutions
  Mark the following statements as true or false:
  \begin{enumerate}
  \item The moon is made of green cheese \trueFfalseT
  \item \ldots
  \end{enumerate}
\end{sproblem}
```

Output:

Exercise
Mark the following statements as true or false:

1. The moon is made of green cheese [◯ true ✓ false]
2. ...

4.2.8 Filling-In Concrete Solutions

The next simplest situation, where we can implement auto-grading is the case where we have fill-in-the-blanks

The `\fillinsol` macro takes a single argument, which contains a concrete solution (i.e. a number, a string, ...), which generates a fill-in-box in test mode:

Example 22
Input:

```
\stopsolutions
\begin{sproblem}[id=elefants.fillin,title=Fitting Elefants]
  How many Elefants can you fit into a Volkswagen beetle? \fillinsol{4}
\end{sproblem}
```

Output:

Exercise (Fitting Elefants)

How many Elefants can you fit into a Volkswagen beetle? ☐

and the actual solution in solutions mode:

Example 23
Input:

```
\startsolutions
\begin{sproblem}[id=elefants.fillin,title=Fitting Elefants]
 How many Elefants can you fit into a Volkswagen beetle? \fillinsol{4}
\end{sproblem}
```

Output:

Exercise (Fitting Elefants)

How many Elefants can you fit into a Volkswagen beetle? 4

If we do not want to leak information about the solution by the size of the blank we can also give `\fillinsol` an optional argument with a size: `\fillinsol[3cm]{12}` makes a box three cm wide.

Obviously, the required argument of `\fillinsol` can be used for auto-grading. For concrete data like numbers, this is immediate, for more complex data like strings "soft comparisons" might be in order. Indeed the default matching of answers against the string provided in the required argument of the `\fillinsol` is up to whitespace normalization.

The `problem` package allows to specify different behavior via three additional keys whose arguments are all triples of the form

```
{⟨target⟩}{⟨TF⟩}{⟨feedback⟩}
```

where ⟨*target*⟩ is the target specification, ⟨*TF*⟩ is the verdict `T` or `F` on the correctness of an answer that is accepted by ⟨*target*⟩, and ⟨*feedback*⟩ a feedback string. In the following (somewhat contrived) example, we see all three at work:

Example 24

Input:

```
\begin{sproblem}
 In 2019, Erlangen is a city of
 \fillinsol[testspace=3cm,
 exact={111962}T{Wow, you known your numbers, according to
   Wikipedia that is exactly correct},
 numrange={0-1000}F{No,that would be a village},
 numrange={1001-100000}F{No,that would be a town},
 numrange={100000-120000}T{Yes, it is close to 109000},
 numrange={1200001-}F{No, that is too large},
 regex={-.[0-9]+}F{What do negative Inhabitants even mean?},
 regex={.*}F{Please give a natural number}]
 {111962}
 inhabitants.
\end{sproblem}
```

Output:

Exercise

In 2019, Erlangen is a city of 111962[a] inhabitants.

[a]

type	case	verdict	feedback
exact	111962	T	
exact	111962	T	Wow, you known your numbers, according to Wikipedia that is exactly correct
numrange	0-1000	F	No,that would be a village
numrange	1001-100000	F	No,that would be a town
numrange	100000-120000	T	Yes, it is close to 109000
numrange	1200001-	F	No, that is too large
regex	-.[0-9]+	F	What do negative Inhabitants even mean?
regex	.*	F	Please give a natural number

- `exact=` gives the exact string (no whitespace normalization).
- `numrange=` specifies a number range, i.e. a comma-separated list of pairs ⟨*low*⟩-⟨*high*⟩, where ⟨*low*⟩ and ⟨*high*⟩ are number representations or empty.
- `regex=` gives a POSIX regular expression that specifies all acceptable strings.

4.2.9 Including Problems

`\includeproblem`

The `\includeproblem` macro can be used to include a problem from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one problem in the include file). The keys `title`, `min`, and `pts` specify the problem title, the estimated minutes for solving the problem and the points to be gained, and their values (if given) overwrite the ones specified in the `problem` environment in the included file.

The sum of the points and estimated minutes (that we specified in the `pts` and `min` keys to the `problem` environment or the `\includeproblem` macro) to the log file and the screen after each run. This is useful in preparing exams, where we want to make sure that the students can indeed solve the problems in an allotted time period.

The `\min` and `\pts` macros allow to specify (i.e. to print to the margin) the distribution of time and reward to parts of a problem, if the `pts` and `pts` options are set. This allows to give students hints about the estimated time and the points to be awarded.

4.2.10 Testing and Spacing

The `problem` package is often used by the `hwexam` package, which is used to create homework assignments and exams. Both of these have a "test mode" (invoked by the package option `test`), where certain information –master solutions or feedback – is not shown in the presentation.

`\testspace`
`\testsmallspace`
`\testsmallspace`
`\testsmallspace`
`\testnewpage`
`\testemptypage`

`\testspace` takes an argument that expands to a dimension, and leaves vertical space accordingly. Specific instances exist: `\testsmallspace`, `\testsmallspace`, `\testsmallspace` give small (1cm), medium (2cm), and big (3cm) vertical space.

`\testnewpage` makes a new page in `test` mode, and `\testemptypage` generates an empty page with the cautionary message that this page was intentionally left empty.

TODO[8]

4.3 Homework Assignments and Exams

4.3.1 Introduction

The hwexam package and class supplies an infrastructure that allows to format nice-looking assignment sheets by simply including problems from problem files marked up with the problem package. It is designed to be compatible with `problems.sty`, and inherits some of the functionality.

4.3.2 Package Options

`solutions`
`notes`
`hints`
`gnotes`
`pts`
`min`

The hwexam package and class take the options `solutions`, `notes`, `hints`, `gnotes`, `pts`, `min`, and `boxed` that are just passed on to the problems package (cf. its documentation for a description of the intended behavior).

`multiple`

Furthermore, the hwexam package takes the option `multiple` that allows to combine multiple assignment sheets into a compound document (the assignment sheets are treated as section, there is a table of contents, etc.).

`test`

Finally, there is the option `test` that modifies the behavior to facilitate formatting tests. Only in `test` mode, the macros `\testspace`, `\testnewpage`, and `\testemptypage` have an effect: they generate space for the students to solve the given problems. Thus they can be left in the LaTeX source.

4.3.3 Assignments

`assignment` (*env.*)
`number`
`title`
`type`
`given`
`due`

This package supplies the `assignment` environment that groups problems into assignment sheets. It takes an optional KeyVal argument with the keys `number` (for the assignment number; if none is given, 1 is assumed as the default or — in multi-assignment documents — the ordinal of the `assignment` environment), `title` (for the assignment title; this is referenced in the title of the assignment sheet), `type` (for the assignment type; e.g. "quiz", or "homework"), `given` (for the date the assignment was given), and `due` (for the date the assignment is due).

4.3.4 Including Assignments

`\includeassignment`

The `\includeassignment` macro can be used to input an assignment from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one `assignment` environment in the included file). The keys `number`, `title`, `type`, `given`, and `due` are just as for the `assignment` environment and (if given) overwrite the ones specified in the `assignment` environment in the included file.

[8] TODO: check what is still undescribed problem.sty and make examples for it.

4.3.5 Typesetting Exams

`testheading` (*env.*) The `\testheading` takes an optional keyword argument where the keys `duration` specifies a string that specifies the duration of the test, `min` specifies the equivalent in number of minutes, and `reqpts` the points that are required for a perfect grade.

`duration`
`min`
`reqpts`

```
\title{320101 General Computer Science (Fall 2010)}
\begin{testheading}[duration=one hour,min=60,reqpts=27]
  Good luck to all students!
\end{testheading}
```

Will result in

Name: Matriculation Number:

320101 General Computer Science (Fall 2010)

2026-03-24

You have one hour (sharp) for the test;
Write the solutions to the sheet.
The estimated time for solving this exam is 23 minutes, leaving you 37 minutes for revising your exam.
You can reach 23 points if you solve all problems. You will only need 27 points for a perfect score, i.e. -4 points are bonus points.

Different problems test different skills and knowledge, so do not get stuck on one problem.

	To be used for grading, do not write here								
prob.	4.1	1.1	1.2	2.1	2.2	2.3	2.4	2.5	2.6
total	0	0	0	10	0	0	0	0	0
reached									

good luck

[9]

[9] MK: The first three "problems" come from the stex examples above, how do we get rid of this?

Part II
User Manual

The dynamic HTML version of this part can be found at
https://stexmmt.mathhub.info/:
sTeX/fullhtml?archive=sTeX/Documentation&filepath=manual.xhtml

Chapter 5

Basics

5.1 Package and Class Options

- `debug=`⟨*prefixes*⟩: (see Developer Manual)
- `lang=`⟨*languages*⟩: If set, sTeX will load the `babel` package with the provided languages. Supported languages (currently) are:

`ar`	arabic
`bg`	bulgarian
`de`	german (with option `ngerman`)
`en`	english
`fi`	finnish
`fr`	french
`ro`	romanian
`ru`	russian
`tr`	turkish (with option `shorthands=:!`)

- `mathhub=`⟨*path*⟩: Uses the provided file path as `MathHub` directory (see Section 1 (Math Archives and the `MathHub` Directory) in the sTeX Manual).
- `usesms`/`writesms`: If `writesms` is set, content loaded from external math archives (i.e. modules) is persisted in the file `\jobname.sms`.

  If `usesms` is set, the content of the `.sms`-file is loaded, obviating the need to reprocess the original files.

  The options are not mutually exclusive, but care should be taken if dependencies have changed between builds.

  This offers two advantages:

  1. If a document has many (transitive) dependencies, `usesms` should significantly speed up the build process, and
  2. setting `usesms` allows for distributing the `.sms`-file to make the document *standalone*, allowing for compilation without needing imported/used modules to be present.

The options `debug`, `mathhub`, `usesms` and `writesms` can also be set by the environment variables `STEX_DEBUG`, `MATHHUB`, `STEX_USESMS` and `STEX_WRITESMS`. In fact, the `MATHHUB` environment variable is the recommended way to set the `MathHub` directory. This is the only option where the *package option* overrides the environment variable.

The environment variables for `USE/WRITESMS` are particularly useful, in that they allow for convenient compilation workflows. For example, the `Build PDF/XHTML/OMDoc`-button in the IDE does the following:

```
STEX_WRITESMS=true pdflatex <job>.tex
[bibtex|biber] <job>
STEX_USESMS=true pdflatex <job>.tex
STEX_USESMS=true pdflatex <job>.tex
```

Guaranteeing (in the first run) that all dependencies are loaded from their respective sources and persisted, and in the two subsequent runs read from the generated `.sms` file, (likely) speeding ub the subsequent runs significantly.

5.2 Math Archives and the MathHub Directory

sTeX uses math archives to organize document content modularly, without a user having to specify absolute paths, which would differ between users and machines.

All sTeX archives need to exist in the local `MathHub`-directory. sTeX knows where this folder is via one of five means:

1. If the sTeX package is loaded with the option `mathhub=/path/to/mathhub`, then sTeX will consider `/path/to/mathhub` as the local `MathHub` directory.
2. If the `mathhub` package option is *not* set, but the macro `\mathhub` exists when the sTeX-package is loaded, then this macro is assumed to point to the local `MathHub` directory; i.e. `\def\mathhub{/path/to/mathhub}\usepackage{stex}` will set the `MathHub` directory as `path/to/mathhub`.
3. Otherwise, sTeX will attempt to retrieve the system variable `MATHHUB`, assuming it will point to the local `MathHub` directory. Since this variant needs setting up only *once* and is machine-specific (rather than defined in tex code), it is compatible with collaborating and sharing tex content, and hence recommended.
4. If that too fails, sTeX will look for a file `~/.stex/mathhub.path`. If this file exists, sTeX will assume that it contains the path to the local `MathHub`-directory. This method is recommended on systems where it is difficult to set environment variables, and is used by the IDE setup.
5. Finally, if all else fails, sTeX considers `~/MathHub` to be the `MathHub` directory.

The sTeX IDE allows you to directly download math archives from `gl.mathhub.info` – currently available archives there are:

- `sTeX/*` – a group of semi-experimental documents showcasing sTeX3.3 features,
- `smglom/*` – a vast collection of multilingual modules of concepts in mathematics and computer science. The SMGloM predates sTeX3 and is thus largely "underannotated" with respect to (formal) semantics,
- `courses/*` – a vast collection of lecture slides and notes in computer science for courses held by Michael Kohlhase. They largely make use of SMGloM modules.

5.2.1 The Structure of Math Archives

An archive `group/name` is stored in the directory `<MathHub>/group/name`; e.g. assuming your local `MathHub`-directory is set as `/user/foo/MathHub`, then in order for the `sTeX/Documentation` archive to be found by the sTeX system, it needs to be in `/user/foo/MathHub/sTeX/Documentation`.

Each such archive needs two subdirectories:

- `/source` – this is where all your tex files go.
- `/META-INF` – a directory containing a single file `MANIFEST.MF`, the content of which we will consider shortly.

An additional `lib`-directory is optional, and is discussed in section 5.3.

5.2.2 MANIFEST.MF-Files

The `MANIFEST.MF` in the `META-INF` directory consists of key-value-pairs, informing sTeX (and associated software, e.g. MMT) of various properties of an archive. For example, the `MANIFEST.MF` of the `sTeX/Documentation` archive looks like this:

```
id: sTeX/Documentation
ns: http://mathhub.info/sTeX/Documentation
narration-base: http://mathhub.info/sTeX/Documentation
format: stex
title: The sTeX Documentation
teaser: The full Documentation for the sTeX system
url-base: https://mathhub.info
dependencies:sTeX/ComputerScience/Software,sTeX/MathTutorial
ignore: */code/*|*/tikz/*|*/tutorial/solution/*
```

Many of these are in fact ignored by sTeX, but some are important:

`id`: The name of the archive, including its group (e.g. `sTeX/Document`). This is used by the MMT system in favor of the directory, but TeX's limited access to the file system enforces the directory structure.

`source-base` or

`ns`: The namespace from which all symbol and module MMT-URIs in this archive are formed.

`narration-base`: The namespace from which all document MMT-URI in this repository are formed. It can safely match the `ns`-field.

`url-base`: A URL that is formed as a basis for *external references*. and hyperlinks. An MMT (or comparable system) instance should run there and host (sTeX-generated) HTML.

`dependencies`: All archives that this archive depends on. sTeX ignores this field, but MMT can pick up on them to resolve dependencies, e.g. when downloading archives in the IDE, which will also download all dependencies.

`ignore`: A regular expression of `.tex` files in the `source` directory that should be ignored; e.g. they will not be compiled when building a whole directory or archive in the IDE.

5.3 The lib-Directory

A math archive group/archive may have a lib directory primarily intended for preamble code, packages, .bib files, etc., the files in which can be referenced in various ways.

Additionally, a *group* of archives group may have an additional archive group/meta-inf. If this meta-inf archive has a /lib-subdirectory, they too will be considered by the following.

\libinput

\libinput {some/file} searches for a file some/file[.tex] in

- the lib-directory of the current archive, and
- the lib-directory of a meta-inf-archive in (any of) the archive groups containing the current archive

and \inputs all found files in reverse order; e.g. \libinput{preamble} in a .tex-file in sTeX/Documentation will *first* input .../sTeX/meta-inf/lib/preamble.tex and then ../sTeX/Documentation/lib/preamble.tex.

\libinput will throw an error if *no* candidate for some/file is found.

\libinput[some/archive]{some/file} will do the same, but starting in the lib directory of the math archive some/archive.

\libusepackage

\libusepackage [package-options]{some/file} searches for a file some/file.sty in the same way that \libinput does, but will call \usepackage[package-options]{path/to/some/file} instead of \input.

\libusepackage throws an error if not *exactly one* candidate for some/file.sty is found.

\addmhbibresource

\addmhbibresource [some/archive]{some/file} searches for a file like some/file.bib in some/archive's lib directory and calls \addbibresource to the result.

\libusetikzlibrary

\libusetikzlibrary behaves like \libusepackage but looks specifically for tikz libraries and calls \usetikzlibrary on the results.

throws an error if not *exactly one* candidate for for the library is found.

A good practice is to have individual sTeX fragments follow basically this document frame:

```
\documentclass{stex}
\libinput{preamble}
\setsectionlevel{<your preference>}
\begin{document}
    \IfInputref{}{
      ...
      \maketitle
      \ifstexhtml \else \tableofcontents \fi
    }
    ...
    \IfInputref{}{\libinput{postamble}}
\end{document}
```

Then the `preamble.tex` files can take care of loading the generally required packages, setting presentation customizations etc. (per archive or archive group or both), and a `postamble.tex` can e.g. print the bibliography, index etc.

`\libusepackage` is particularly useful in such a `preamble.tex` when we want to use custom packages that are not part of a TEX distribution, or on CTAN. In this case we commit the respective packages in one of the `lib` folders and use `\libusepackage` to load them.

5.4 Basic Macros

`\sTeX` `\stex`

The `\sTeX` macro produces this sTEX logo. It is provided by the stex-logo package, included with the stex package.

`\ifstexhtml`

The TEX conditional `\ifstexhtml` is *true* if the current compilation generates HTML, and *false* otherwise (i.e. generates PDF).

`\STEXinvisible`

`\STEXinvisible{⟨code⟩}`

Processes ⟨*code*⟩, but does not generate any output. In the HTML, ⟨*code*⟩ is exported with `display:none`.

Can be used to declare formal content and preserve its semantics in HTML without generating output.

Chapter 6

Document Features

6.1 Document Fragments

`sfragment` (*env.*) To make reusability of document fragments more feasible, sTeX provides the **`sfragment`** environment. `\begin{sfragment}[id=⟨id⟩,short=⟨short title⟩]{section title}` calls `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\paragraph` or `\subparagraph` with argument `{section title}` depending on the current *section level* and availability, and increases the level accordingly.

The ⟨*id*⟩ can be used for cross-document references (see Section 0.3 (Cross-Document References) in the sTeX Manual).

`blindfragment` (*env.*) In the case where we want to increase the section level *without* producing a corresponding section header, the **`blindfragment`** environment can be used. This allows e.g. typesetting `\section`s before the first `\chapter`.

`\skipfragment` The `\skipfragment` macro "skips an `sfragment`", i.e. it just steps the respective sectioning counter. This macro is useful, when we want to keep two documents in sync structurally, so that section numbers match up: Any section that is left out in one becomes a `\skipfragment`.

`\setsectionlevel` The `\setsectionlevel` macro sets the current section level to that provided as argument. This is particularly useful in the preamble of a document, as to be ignored in e.g. `\inputref` and make sure that sectioning proceeds as desired; e.g. `\setsectionlevel{section}` make sure that the first `sfragment` will be typeset as a `\section` (rather than e.g. a `\part`).

`\currentsectionlevel`
`\Currentsectionlevel` The `\currentsectionlevel` macro produces the literal string corresponding to the current section level – e.g. within a chapter (but outside of a section), `\currentsectionlevel` produces "chapter".

The `\Currentsectionlevel` macro does the same, but capitalizes the first letter; e.g. in the above situation, `\Currentsectionlevel` produces "Chapter".

6.2 Using and Referencing Document Fragments

\inputref `\inputref [⟨archive⟩]{⟨file⟩}`

Inputs the file ⟨*file*⟩ in ⟨*archive*⟩'s `source` directory. If `[⟨archive⟩]` is empty, the current archive's `source` directory is used. If there is no current archive, ⟨*file*⟩ is resolved relative to the current file.

The file's content is processed within a TEX group when using `pdflatex`. When converting to HTML however, the file is not processed *at all*, and instead, a reference to the file is inserted, that can be replaced by the HTML generated by the referenced file by e.g. the MMT system.

This is the recommended method to assemble documents from individual `.tex` files.

\mhinput Like `\inputref`, but actually calls `\input` in both PDF and HTML mode. Useful for small fragments or those without modules, but generally `\inputref` should be preferred.

\ifinputref **\IfInputref** `\ifinputref` is a TEX conditional for whether the current file is currently processed via `\inputref`.

`\IfInputref {⟨true code⟩}{⟨false code⟩}` behaves like `\ifinputref⟨true code⟩\else⟨false code⟩\fi` when using `pdflatex`; in HTML mode however, *both* arguments are processed and marked-up accordingly, so a hosting server (like MMT) can dynamically decide which parts to show or omit.

\mhgraphics **\cmhgraphics** *If* the graphicx package is loaded, `\mhgraphics` takes the same arguments as `\includegraphics`, with the additional optional key `archive`. It then resolves the file path in `\mhgraphics[archive=some/archive]{some/image}` relative to the `source`-folder of the `some/archive` archive. If no `archive` is provided, the file path `some/image` is resolved relative to the current archive (if existent).

`\cmhgraphics` additional wraps the image in a `center`-environment.

\lstinputmhlisting **\clstinputmhlisting** Like `\mhgraphics`, but for `\lstinputlisting` instead of `\includegraphics`. Only defined if the listings package is loaded.

6.3 Cross-Document References

If we take features like `\inputref` and `\mhinput` (and the `sfragment` environment) seriously and build large documents modularly from individually compiling documents for sections, chapters and so on, cross-referencing becomes an interesting problem.

Say, we have a document `main.tex`, which `\inputref`s a section `section1.tex`, which references a definition with label `some_definition` in `section2.tex` (subsequently also `\inputref`ed in `main.tex`). Then the numbering of the definition will depend on the *document context* in which the document fragment `section2.tex` occurs - in `section2.tex` itself (as a standalone document), it might be *Definition 1*, in `main.tex`

it might be *Definition 3.1*, and in `section1.tex`, the definition *does not even occur*, so it needs to be referenced by some other text.

What we would want in that instance is an equivalent of `\autoref`, that takes the document context into account to yield something like *Definition 1*, *Definition 3.1* or "*Definition 1 in the section on Foo*" respectively.

For that to work, we need to supply (up to) three pieces of information:

- The *label* of the reference target (e.g. `some_definition`),
- (optionally) the *file*/document containing the reference target (e.g. `section2`). This is not strictly necessary if the reference target occurs in the *same* document, but if not, we need to know where to find the label,
- (optionally) the document context, in which we want to refer to the reference target (e.g. `main`).

Additionally, the document in which we want to reference a label needs a title for external references.

\sref `\sref [archive=⟨archive1⟩,file=⟨file1⟩] {⟨label⟩}[archive=⟨archive2⟩,file=⟨file2⟩,title=⟨title⟩]`

This macro references ⟨*label*⟩ (declared in ⟨*file1*⟩ in math archive ⟨*archive1*⟩). If the object (section, figure, etc.) with that label occurs (eventually) in the same document, `\sref` will ignore the second set of optional arguments and simply defer to `\autoref` if that command exists, or `\ref` if the hyperref package is not included.

If the referenced object does *not* occur in the current document however, `\sref` will refer to it by the object's name as it occurs in the file ⟨*file2*⟩ in archive ⟨*archive2*⟩, followed by the title.

In HTML mode, the reference additionally links to the HTML of the `file1`.[10]

This works by storing labels during compilation in a file ⟨*jobname*⟩`.sref`, analogous to e.g. the `.toc`. Note that this consequently requires both `file1.tex` and `file2.tex` to have been compiled previously, to generate the `.sref` file.

For example, doing
`\sref[file=tutorial/full.en]{sec:basics}[file=tutorial.en,title=the \stex Tutorial]`
in this very document fragment (`[sTeX/Documentation]macros/sref.en.tex`) will yield Part I (The Basics) in the sTeX Tutorial if compiled itself, or if compiled as part of the sTeX manual, and will yield the `\autoref` link chapter 2 in the documentation (which includes the tutorial).

\srefsetin `\srefsetin [⟨archive2⟩]{⟨file2⟩}{⟨title⟩}`

Sets a default value for the optional arguments ⟨*archive2*⟩, ⟨*file2*⟩ and ⟨*title*⟩ of `\sref`. If the second set of optional arguments in `\sref` are omitted, these default values are used. Particularly useful to set in a preamble.

\sreflabel `\sreflabel {⟨label⟩}` sets a label analogous to `\label{⟨label⟩}`, but for use in `\sref`.

Note that for every sTeX macro or environment that takes an optional `id=⟨id⟩` argument, the ⟨*id*⟩ (if non-empty) generates an `\sreflabel` automatically.

For example, `\begin{sfragment}[id=foo]{Foo}` is equivalent to `\begin{sfragment}{Foo}\sreflabel{foo}`.

\extref

\extref [archive=⟨*archive1*⟩,file=⟨*file1*⟩]
{⟨*label*⟩}{archive=⟨*archive2*⟩,file=⟨*file2*⟩,title=⟨*title*⟩}

Like \sref, but with the third argument mandatory, \extref will *always* produce the output as if ⟨*label*⟩ would *not* occur in the current document.

Chapter 7

Modules and Symbols

7.1 Modules

A module is required to declare any new formal content such as symbols or notations (but not variables, which may be introduced anywhere).

> M An sTEX module corresponds to an MMT/OMDoc *theory*. As such M it gets assigned an MMT-URI (*universal resource identifier*) of the form T `<namespace>?<module-name>`.

`smodule` (*env.*) A new module is declared using the basic syntax

`\begin{smodule}[options]{ModuleName}...\end{smodule}`.

A module is required to declare any new formal content such as symbols or notations (but not variables, which may be introduced anywhere).

The `smodule`-environment takes several keyword arguments, all of which are optional:

`title` (⟨*token list*⟩) to display in customizations.

`style` (⟨*string*⟩∗) for use in customizations, see Part 1 (Customizing Typesetting) in the sTEX Manual

`id` (⟨*string*⟩) for cross-referencing, see `\sreflabel`.

`ns` (⟨*URI*⟩) the namespace to use. *Should not be used, unless you know precisely what you're doing.* If not explicitly set, is computed from the containing file and archive's namespace.

`lang` (⟨*language*⟩) if not set, computed from the current file name (e.g. `foo.en.tex`).

`sig` (⟨*language*⟩) see below.

`\STEXexport` `\STEXexport{`⟨*code*⟩`}` executes ⟨*code*⟩ immediately and every time the current module is being used.

For technical reasons, \STEXexport processes its content in the expl3 category code scheme – what this means is that all spaces are ignored entirely, and the characters _ and : are valid characters in macro names.
In practice, this means you will have to use the ~ character for spaces, and if you want to use a subscript _, you should use the macro \c_math_subscript_token instead.
Also, note that no *global* macro definitions should happen in \STEXexport; this can lead to unexpected behaviour if the containing module has been used previously in the current document.

7.1.1 Signature Modules, Languages, and Multilinguality

if the current file is a translation of a file with the same base name but a different language suffix, setting sig=<lang> will preload the module from that language file. This helps ensuring that the (formal) content of both modules is (almost) identical across languages and avoids duplication.

For example, we can have a file Foo.en.tex, that declares and documents a module Foo (using \begin{smodule}{Foo}). If we put a file Foo.de.tex next to it, we can do \begin{smodule}[sig=en]{Foo} to have all the content in the module Foo (as declared in Foo.en.tex) available and translate its document content to german.

The MMT back-end, when serving sTEX content as HTML, will always attempt to find documentation in the language corresponding to the context; e.g. a user's preference.

7.2 Symbol Declarations

\symdecl

\symdecl {⟨*mname*⟩}[⟨*options*⟩]

The \symdecl macro is the simplest way to introduce a new symbol. If ⟨*options*⟩ contains name=⟨*name*⟩, then ⟨*name*⟩ is the *name* of the symbol; otherwise, ⟨*mname*⟩ is used for the *name*. Additionally, a semantic macro \mname is generated.

The starred variant \symdecl* does not generate a semantic macro, in which case the name-option is superfluous.

M \symdecl introduces a new MMT/OMDoc constant in the current module (i.e. M MMT/OMDoc theory). Correspondingly, they get assigned the MMT-URI T <module-URI>?<constant-name>.

\symdecl takes the following optional arguments:

name see above,

args the arity of the symbol and its semantic macro; may be a number 0...9 or a string consisting of the characters i, a, b and B of length ≤ 9,

type the symbol's type,

def the symbol's definiens,

return the symbol's *return code* (see below), most commonly the semantic macro of a mathematical structure,

assoc how to resolve arguments of mode `a` or `B`; may be `pre`, `bin`, `binl`, `binr` or `conj`,

reorder how to reorder the arguments in OMDoc (*advanced*),

role symbols with certain roles are treated in particular ways in MMT/OMDoc (*advanced*),

argtypes TODO[11].

\textsymdecl `\textsymdecl{⟨mname⟩}[⟨options⟩]{⟨code⟩}`

Like `\symdecl`, but requires that the symbol has arity 0 (hence `\textsymdecl` does not take the `args`-option), and generates a semantic macro that takes no arguments in either text or math mode, and produces marked-up ⟨*code*⟩ as output.

Additionally, a macro `\⟨mname⟩name` is generated that produces ⟨*code*⟩ without any semantic markup.

\symdef `\symdef{⟨mname⟩}[⟨options⟩]{⟨notation⟩}`

Combines the functionalities and optional arguments of `\symdecl` and `\notation` in one.

7.2.1 Returns

Assume we have a symbol `foo` with semantic macro `\foo`, (examplary) taking two arguments, and `return=`⟨*code*⟩. If we do `\foo{a}{b}!`, the return code is simply ignored. If we do `\foo{a}{b}` *without* the `!`, here is what happens:

1. STEX will replace `#1` and `#2` in ⟨*code*⟩ by `a` and `b`, yielding ⟨*retcode*⟩.
2. STEX will insert ⟨*retcode*⟩`{\foo{a}{b}!}` in the input token stream.

This means that ⟨*code*⟩ should contain at most ⟨*arity of foo*⟩ argument markers, and eat precisely one argument appended to ⟨*code*⟩.

When in doubt, we recommend only using semantic macros for mathematical structures (with only optional arguments) and `\apply` (with only optional arguments) in `return`.

7.3 Referencing Symbols

\symref `\symref{⟨symbol⟩}{⟨text⟩}`
\sr

The `\symref` macro (and its short version `\sr`) is the most general variant to mark-up arbitrary LaTeX code ⟨*text*⟩ with the symbol.

[11] TODO: experimental

This is as good a place as any other to explain how sTeX resolves a string `symbol` to an actual symbol.
If `\symbol` is a semantic macro, then sTeX has no trouble resolving `symbolname` to the full MMT-URI of the symbol that is being invoked.
However, especially in `\symname` (or if a symbol was introduced using `\symdecl*` without generating a semantic macro), we might prefer to use the *name* of a symbol directly for readability – e.g. we would want to write `A \symname{natural number} is...` rather than `A \symname{Nat} is...`. sTeX attempts to handle this case thusly:
If `symbol` does *not* correspond to a semantic macro `\symbol` and does *not* contain a `?`, then sTeX checks all symbols currently in scope until it finds one, whose name is `symbol`. If `symbol` is of the form `pre?name`, sTeX first looks through all modules currently in scope, whose full MMT-URI ends with `pre`, and then looks for a symbol with name `name` in those. This allows for disambiguating more precisely, e.g. by saying `\symname{Integers?addition}` or `\symname{RealNumbers?addition}` in the case where several `additions` are in scope.

`\symref{⟨symbol⟩}{⟨text⟩}` in MMT/OMDoc generates the term `<OMS name="{⟨symbol URI⟩}"/>`.

`\symname` `\sn` `\Symname` `\Sn` `\sns` `\Sns`

`\symname[pre=⟨pre⟩,post=⟨post⟩]{⟨symbol⟩}`

If the symbol referenced by ⟨*symbol*⟩ has name `name`, this is a shortcut for `\symref{⟨symbol⟩}{⟨pre⟩name⟨post⟩}`.

For example, given a symbol `agroup` with name `abelian group`, we can do `\symname[pre=Non-,post=s]{agroup}` to produce Non-abelian groups.

`\sn` is a shorter variant for `\symname`; `\Symname` and `\Sn` additionally capitalize the first letter. `\sns` and `\Sns` are short for `\sn [post=s]` and `\Sn [post=s]`, respectively.

`\srefsym` `\srefsymuri`

`\srefsym{⟨symbol⟩}{⟨text⟩}`

turns ⟨*text*⟩ into a link to

- The documentation of ⟨*symbol*⟩, if it occurs in the same document, or
- the symbol's documentation online, based on the containing math archive's `url-base`.

`\srefsymuri` does the same, but expects a symbol's full MMT-URI as first argument. This is particularly useful for e.g. customizing highlighting (see chapter 9).

`\symuse`

`\symuse{⟨symbol⟩}` behaves exactly like a semantic macro for ⟨*symbol*⟩.

7.4 Notations and Semantic Macros

\notation

\notation{⟨symbol⟩}[⟨options⟩]{⟨code⟩}

introduces a new notation for the referenced symbol.

The starred variant \notation* sets this notation as the (new) default notation.

The optional arguments are:

- prec=⟨opprec⟩;⟨argprec 1⟩x...x⟨argprec n⟩: An operator precedence and one argument precedence for each argument of the semantic macro. If no argument precedences are given, all argument precedences are equal to the operator precedence. By default, all precedences are 0, unless the symbol takes no argument, in which case the operator precedence is \neginfprec (negative infinity).

  prec=nobrackets is an abbreviation for prec=\neginfprec;\infprec x...x\infprec.

- op=⟨code⟩: An operator notation. If none is given, the notation component marked with \maincomp is used. If no \maincomp occurs in the notation, the default operator notation is \symname{⟨symbol⟩}.

- variant=⟨id⟩: An id for this notation. The key variant= can be omitted; i.e. \notation[foo] is equivalent to \notation[variant=foo].

\comp
\maincomp

\comp is used to mark notation components in a \notation to be highlighted. Additionally, each notation can use \maincomp at most once to mark the *primary* notation component.

Ideally, \comp would not be necessary: Everything in a notation that is *not* an argument should be a notation component. Unfortunately, it is computationally expensive to determine where an argument begins and ends, and the argument markers #n may themselves be nested in other macro applications or TEX groups, making it ultimately almost impossible to determine them automatically while also remaining compatible with arbitrary highlighting customizations (such as tooltips, hyperlinks, colors) that users might employ, and that are ultimately invoked by \comp.

Note that it is required that

1. the argument markers #n never occur inside a \comp, and
2. no semantic macros may ever occur inside a notation.

Both criteria are not just required for technical reasons, but conceptionally meaningful:
The underlying principle is that the arguments to a semantic macro represent *arguments to the mathematical operation* represented by a symbol. For example, a semantic macro application \plus{a}{b} would represent *the actual addition of (mathematical objects) a and b*. It should therefore be impossible for a or b to be part of a notation component of \plus.
Similarly, a semantic macro can not conceptually be part of the notation of \plus,

since a symbol represents a *distinct (mathematical) concept* with *its own semantics*, and notations are syntactic representations of the very symbol to which the notation belongs.
If you want an argument to a semantic macro to be a purely syntactic parameter, then you are likely somewhat confused with respect to the distinction between the precise *syntax* and *semantics* of the symbol you are trying to declare (which happens quite often even to experienced sTeX users, like us), and might want to give those another thought - quite likely, the concept you aim to implement does not actually represent a semantically meaningful (mathematical) concept, and you will want to use `\def` and similar native LaTeX macro definitions rather than semantic macros.

`\setnotation`

The first notation provided will stay the default notation unless explicitly changed:
`\setnotation{⟨symbol⟩}{⟨id⟩}` sets the default notation of ⟨*symbol*⟩ to that with id ⟨*id*⟩.

7.4.1 Precedences and Bracketing

`\infprec`
`\neginfprec`

`\infprec` and `\neginfprec` represent *infinitely large* and *infinitely small* precedences, respectively.

sTeX decides whether to insert parentheses by comparing operator precedences to a *downward precedence* p_d with initial value `\infprec`. When encountering a semantic macro, sTeX takes the operator precedence p_{op} of the notation used and checks whether $p_{op} > p_d$. If so, sTeX inserts parentheses.
When sTeX steps into an argument of a semantic macro, it sets p_d to the respective argument precedence of the notation used.

Example 25
Consider semantic macros `\plus` and `\mult` taking two arguments, with notations $a+b$ and $a \cdot b$ respectively, and precedences 100 for `\plus` and 50 for `\mult`.
Consider `$\plus{a,\mult{b,\plus{c,d}}}$` (i.e. $a + b \cdot (c + d)$). Then:

1. sTeX starts out with $p_d =$`\infprec`.
2. sTeX encounters `\plus` with $p_{op} = 100$. Since $100 \not> $`\infprec`, it inserts no parentheses.
3. Next, sTeX encounters the two arguments for `\plus`. Both have no specifically provided argument precedence, so sTeX uses $p_d = p_{op} = 100$ for both and recurses.
4. Next, sTeX encounters `\mult{b,...}`, whose notation has $p_{op} = 50$.
5. We compare to the current downward precedence p_d set by `\plus`, arriving at $p_{op} = 50 \not> 100 = p_d$, so sTeX again inserts no parentheses.

6. Since the notation of \mult has no explicitly set argument precedences, sTeX again uses the operator precedence for the arguments of \mult, hence sets $p_d = p_{op} = 50$ and recurses.

7. Next, sTeX encounters the inner \plus{c,...} whose notation has $p_{op} = 100$. We compare to the current downward precedence p_d set by \mult, arriving at $p_{op} = 100 > 50 = p_d$ – which finally prompts sTeX to insert parentheses, and we proceed as before.

\dobracket

\dobracket{⟨*code*⟩} wraps parentheses around {⟨*code*⟩}.

\withbrackets

\withbrackets{⟨*left*⟩}{⟨*right*⟩}{⟨*code*⟩} uses the opening and closing parentheses ⟨*left*⟩ and ⟨*right*⟩ for the next pair of parentheses automatically inserted in {⟨*code*⟩}.

7.4.2 Notations for Argument Sequences

The following macros can be used in notations that take mode a or B arguments:

\argsep

\argsep{⟨*parameter token*⟩}{⟨*separator*⟩}

takes the elements of the argument sequence in position ⟨*parameter token*⟩ and separates them by ⟨*separator*⟩.

Note that the first argument *must* be a parameter token of the form #k, and the argument at position k of the notation has to have argument mode a or B.

\argmap

\argmap{⟨*parameter token*⟩}{⟨*code*⟩}{⟨*separator*⟩}

takes the elements of the argument sequence in position ⟨*parameter token*⟩, applies the code {⟨*code*⟩} to each of them (which therefore should use ##1) and separates them by ⟨*separator*⟩.

For example, the notation {\argmap{#1}{X^{##1}}{++}} applied to the argument {a,b,c} produces $X^a + +X^b + +X^c$.

\argarraymap

TODO[12]

7.4.3 Semantic Macros

Assume we have a semantic macro \smacro taking (exemplary) two arguments. The precise behaviour of \smacro depends on whether we are in text or math mode.

Math Mode \smacro! produces the default operator notation of its symbol. Without !, \smacro expects at least two arguments, and \smacro{a}{b}! produces the default notation of its symbol.

If the symbol has a return code, then \smacro{a}{b} continues with executing the return code. Otherwise, \smacro{a}{b} also simply produces the default operator notation.

The starred variants \smacro* and \smacro!* behave as in *text mode*.

Text Mode \smacro!{⟨*arg*⟩} marks up ⟨*arg*⟩ similarly to how \symref{smacro}{⟨*arg*⟩} would.

Without the !, \smacro still only takes a single argument, but it is expected, that within ⟨*arg*⟩, the arguments for the symbol are explicitly marked up. The \comp macro is allowed in ⟨*arg*⟩ to determine the components of ⟨*arg*⟩ to be highlighted.

\arg The \arg macro can be used to explicitly mark the arguments of a semantic macro in text mode.

By default, they are numbered consecutively; e.g. \smacro{...\arg{a}...\arg{b}} determines a and b to be the (first and second) arguments.

The starred variant \arg* allows for marking up the arguments, but does not produce any output. This can be used to provide arguments that are not mentioned in the text we want to mark up, because they are implicitly obvious or mentioned elsewhere.

If we want to change the order of the arguments, we can provide the precise argument number as an optional argument; e.g. \smacro{...\arg[2]{a}...\arg[1]{b}} determines b to be the first and a to be the second argument.

An argument number may be used repeatedly, if the corresponding argument mode is a or B.

M M T

Applications of semantic macros with arguments are translated to MMT/OMDoc as OMA-terms with head <OMS name="⟨*symbol*⟩"/>, or <OMBIND name="⟨*symbol*⟩"/>, depending on the absence or presence of argument mode b or B arguments.
Semantic macros with no arguments or invoked with ! correspond to OMS directly.

7.5 Simple Inheritance

There are three macros that allow for opening a module, making its contents available for use:

\usemodule \usemodule{⟨*module*⟩} is allowed anywhere and makes the module's contents available up to the current TEX group. This is the right macro to use outside of modules, or when none of its contents use any of the used module's symbols directly (e.g. in types or definientia).

\requiremodule \requiremodule{⟨*module*⟩} is only allowed in modules and makes the required module's contents available within the current module. The imported symbols can be safely used in types and definientia, but not in the return code of symbols, and the imported content is not exported further – i.e. using the current module does not also open the required module.

`\importmodule`

`\importmodule{`⟨*module*⟩`}` is only allowed in modules and makes the required module's contents available within the current module. The imported symbols can be safely used anywhere, and the imported content exported to any modules subsequently importing the current one.

In MMT, every *document* and every module induces an MMT theory. `\usemodule` induces and MMT include in the document theory, `\importmodule` and `\requiremodule` both induce an include in the module's theory.

It is worth going into some detail how exactly `\usemodule`, `\importmodule` and `\requiremodule` resolve their arguments to find the desired module – which is closely related to the *namespace* generated for a module, that is used to generate its MMT-URI.

Ideally, sTeX would allow for arbitrary MMT-URIs for modules, with no forced relationships between the *logical* namespace of a module and the *physical* location of the file declaring it – like MMT in fact allows for.
Unfortunately, TeX only provides very restricted access to the file system, so we are forced to generate namespaces systematically in such a way that they reflect the physical location of the associated files, so that sTeX can resolve them accordingly. Largely, users need not concern themselves with namespaces at all, but for completenesses sake, we describe how they are constructed:

- If `\begin{smodule}{Foo}` occurs in a file `/path/to/file/Foo[.`⟨*lang*⟩`].tex` which does not belong to an math archive, the namespace is `file://path/to/file`.
- If the same statement occurs in a file `/path/to/file/bar[.`⟨*lang*⟩`].tex`, the namespace is `file://path/to/file/bar`.

In other words: outside of math archives, the namespace corresponds to the file URI with the filename dropped iff it is equal to the module name, and ignoring the (optional) language suffix.
If the current file is in an archive, the procedure is the same except that the initial segment of the file path up to the archive's `source` directory is replaced by the archive's namespace URI.

Conversely, here is how namespaces/URIs and file paths are computed in import statements, examplary `\importmodule`:

- `\importmodule{Foo}` outside of an archive refers to module `Foo` in the current namespace. Consequently, `Foo` must have been declared earlier in the same file or, if not, in a file `Foo[.`⟨*lang*⟩`].tex` in the same directory.
- The same statement *within* an archive refers to either the module `Foo` declared earlier in the same file, or otherwise to the module `Foo` in the archive's top-level namespace. In the latter case, is has to be declared in a file `Foo[.`⟨*lang*⟩`].tex` directly in the archive's `source` directory.
- Similarly, in `\importmodule{some/path?Foo}` the path `some/path` refers to either the sub-directory and relative namespace path of the current directory

and namespace outside of an archive, or relative to the current archive's top-level namespace and `source` directory, respectively.

The module `Foo` must either be declared in the file ⟨*top-directory*⟩`/some/path/Foo[.`⟨*lang*⟩`].tex`, or in ⟨*top-directory*⟩`/some/path[.`⟨*lang*⟩`].tex` (which are checked in that order).

- Similarly, `\importmodule[Some/Archive]{some/path?Foo}` is resolved like the previous cases, but relative to the archive `Some/Archive` in the `MathHub` directory.

7.6 Variables and Sequences

\vardef `\vardef{`⟨*mname*⟩`}[`⟨*options*⟩`]{`⟨*notation*⟩`}`

Takes the same arguments as `\symdef`, but produces a variable rather than a symbol. Variables definitions are always local to the current TEX group and are allowed anywhere (i.e. outside of modules).

⟨*options*⟩ may include the additional keyword `bind`, in which case the variable will be appropriately abstracted away in statements (see also `\varbind`).

Unlike `\symdef`, there is no starred variant `\vardef*` – variables always generate a semantic macro.

The semantic macro for a variable behaves analogously to that of a symbol.

Variables induces the same MMT/OMDoc terms as symbols do, except for the head of the term being `<OMV name="..."/>` instead of `<OMS/>`.

\varnotation `\varnotation{`⟨*variable*⟩`}[`⟨*options*⟩`]{`⟨*notation*⟩`}`

Takes the exact same argumens as `\notation`, but attaches an additional notation to the *variable* ⟨*variable*⟩ rather than a symbol.

\svar `\svar[`⟨*name*⟩`]{`⟨*text*⟩`}`

Semantically marks up ⟨*text*⟩ as representing a variable ⟨*name*⟩. The variable does not need to have been defined prior. If no ⟨*name*⟩ is givenm ⟨*text*⟩ will be used as the name.

This is useful in situations like "throwaway expressions" or remarks; e.g. `$\plus{\svar{n},\svar{m}}$ means...`

\varseq \varseq{⟨*mname*⟩}[⟨*options*⟩]{⟨*range*⟩}{⟨*notation*⟩}

Declares a new variable sequence. The ⟨*options*⟩ are the same as for \vardef. If not provided, `args=1` by default (a 0-ary sequence would just be a normal variable).

A type (given as `type=`) is interpreted to be the type of every element a_i of the sequence $a_1, ..., a_n$ (not of the sequence itself). If the type is itself a sequence $A_1, ..., A_n$, the assumption is that its range is the same as the one of the new sequence, and the type of every a_i in the sequence is A_i.

⟨*range*⟩ needs to be a comma-separated sequence of either `args` many arguments, or \ellipses.

The resulting semantic macro is allowed anywhere sTEX expects an argument mode `a` or `B` argument.

\ellipses Represents ellipses in a range; produces \ellipses in math mode.

\seqmap \seqmap{⟨*code*⟩}{⟨*sequence*⟩}

Maps the function ⟨*code*⟩ (containing `#1`) over every element of the ⟨*sequence*⟩.

Is allowed anywhere sTEX expects an argument mode `a` or `B` argument.

7.7 Structures

Mathematical structure bundle interdependent symbols together.

mathstructure (*env.*) \begin{mathstructure}{⟨*mname*⟩}[⟨*name*⟩,this=⟨*code*⟩] opens a new mathematical structure with name ⟨*mname*⟩ (if provided) or ⟨*mname*⟩ (otherwise), and semantic macro \mname. It subsequently behaves like the smodule environment.

\this The optional `this=`⟨*code*⟩ option allows for setting the typesetting of the \this macro within a mathstructure. In particular, \this can be used in notations for symbols declared in the structure. \this can be though of as representing "*the*" (current) instance of this structure.

extstructure (*env.*) \begin{extstructure}{⟨*mname*⟩}[⟨*name*⟩,this=⟨*code*⟩]{⟨*structs*⟩} opens a new mathematical structure extending the structures given in ⟨*structs*⟩ (a comma-separated list of names).

extstructure* (*env.*) \begin{extstructure*}{⟨*struct*⟩} opens a new mathematical structure *conservatively* extending the (single) structure ⟨*struct*⟩. *Conservative* meaning: Every symbol newly introduced in this structure needs to have a *definiens*. The new symbols are attached as fields directly to ⟨*struct*⟩.

\usestructure The \usestructure macro behaves like \usemodule for mathematical structures, making the symbols available to use directly.

mathstructure make use of the *Theories-as-Types* paradigm (see [**MueRabKoh:tat18**]):

M M T

`\begin{mathstructure}{⟨name⟩}` creates a nested theory with name ⟨name⟩`-module`. The constant ⟨name⟩ is defined as a *dependent record type with manifest fields*, the fields of which are generated from (and correspond to) the constants in ⟨name⟩`-module`.

7.7.1 Semantic Macros for Structures

Assume we have a mathematical structure with semantic macro `\struct`:

Example 26

```
\begin{mathstructure}{struct}
  \symdef{fielda}{a}
  \symdef{fieldb}[args=2]{#1 \maincomp{b} #2}
  \symdef{fieldc}[args=2,def=\sn{fieldb}]{#1 \maincomp{c} #2}
  \inlineass[name=axiom1]{\conclusion{some axiom}}
\end{mathstructure}
\notation{struct}{StRuCt}
```

- If `\struct` has no notations, then `$\struct!$` produces $\langle a, b, c\rangle$. Otherwise, it produces the notation, i.e. *StRuCt*. In both cases, `$\struct{}{}$` produces $\langle a, b, c\rangle$ (for reasons that will become clearer in a moment).
- `$\struct{}{}$` (or `$\struct!$` in the case where no notations are around) can be modified in the following ways:
  - `$\struct{}{}[⟨fieldname⟩,...]$` lets you pick, which precise fields to show, so e.g. `$\struct{}{}[fielda,fieldb]$` produces $\langle a, b\rangle$. By default, `$\struct{}{}$` shows exactly the fields that have semantic macros (which are also used to access the fields in `$\struct{...}{fieldname}`).
  - `$\struct{}{}[⟨id⟩]$` lets you pick the notation of the "mathematical structure" symbol to use to typeset the structure; e.g. `$\struct{}{}[parens]` yields (a, b, c), and (combining both) `$\struct{}{}[fielda,fieldb][angle]` yields $\langle a, b\rangle$.
- The two arguments in `$\struct{first}{second}$` represent 1. the term that is to be treated as an instance of `\struct`, and 2. the precise field to invoke. If the first is empty, then there is no instance. If the second is empty, sTEX will present all of them (that are not assertions). Hence, `$\struct{S}{}$` yields $\langle a_S, b_S, c_S\rangle$, `$\struct{S}{fielda}$` produces a_S, `$\struct{}{fielda}$` produces a, and `$\struct{S}{fieldb}{x}{y}$` produces xb_Sy.
- For the sake of completion, `$\struct{first}!$` simply produces the given argument; e.g. `$\struct{S}!$` simply produces S.

More precisely: `$\struct{⟨code⟩}$` acts like a "type coercion" of ⟨code⟩ to be an instance of `\struct`.

Of course, it is (occasionally, but) rarely useful to use the semantic macro `\struct` by giving it two arguments *manually*; but this is what sTEX does when using `\struct` in the `return` of a symbol (or variable).

Continuing:

- `$\struct[comp=⟨code⟩]{...}{...}$` applies ⟨*code*⟩ (using `#1`) to every occurence of `\maincomp` in the notations of the fields, as a replacement for the default modification `{#1}_{\this}`. For example, `$\struct[comp={#1}^{Foo}]{S}{}$` produces $\langle a^{Foo}, b^{Foo}, c^{Foo}\rangle$, and `$\struct[comp={#1}^{\this}]{S}{fieldb}{x}{y}$` yields $xb^S y$.

- `$\struct[this=⟨code⟩]{...}{...}$` modifies the way `\this` is being typeset; i.e. the presentation of the first `{...}` argument. For example `$\struct[this=T]{S}{}$` produces $\langle a, b, c\rangle$ – since `\this` is not being used in the notations of the fields. Note that the `this=` and `comp=` variants are (as of yet) mutually incompatible.

- Finally, `$\struct[⟨fieldname⟩=⟨code⟩]{...}{...}$` *assigns* the field ⟨*fieldname*⟩ to the term ⟨*code*⟩. ⟨*code*⟩ is subsequently used when using `{...}{}`, but not in fields directly. For example, `$\struct[fielda=A]{S}{}$` produces $\langle A!, b_S, c_S\rangle$, but `$\struct[fielda=A]{S}{fielda}$` produces a_S.

  Note the insertion of `!` behind the `A` – this is to make sure that assignments to semantic macros that takes arguments don't accidentally eat more than they should.

  Also note that multiple assignments can be done in the same pair of `[]`, or chained – i.e. both `$\struct[fielda=A,fieldb=B]...$` and `$\struct[fielda=A][fieldb=B]...$` are valid and equivalent. `$\struct[fielda=A,fielda=B]...$` however is not – every field may be assigned at most once.

Chapter 8

Statements

sTEX provides four environments to semantically annotate various kinds of statements:

`sdefinition` (*env.*) The **`sdefinition`** environment represents (primarly mathematical) *definitions*; in particular for symbols. The contents of the environment will be used as *documentation* for any symbol that either orrurs as a `\definiendum` (or `\definame`) within the **`sdefinition`**, or that is listed in the optional `for=` argument of the environment.

If a `\definiens` occurs, this will be used by MMT as the formal definiens for the respective symbol.

`sassertion` (*env.*) The **`sassertion`** environment represents *assertions*, i.e. propositions such as *theorems*, *lemmata*, *axioms*, etc. If a `\conclusion` occurs within the **`sassertion`**, its argument will be used as the formal *statement* of the assertion.

`sexample` (*env.*) The **`sexample`** environment represents examples, the `for=` attribute specifies the concept this is an example for. For counterexamples use `style=counterexample`

`sparagraph` (*env.*) The **`sparagraph`** environment represents all other kinds of (logical) paragraphs, such as remarks, comments, transitions bwetween topics, recaps, reminders, etc.

All of these take the same arguments:

- `for=`⟨*csl*⟩: a comma-separated list of symbols.
- The same optional arguments as `\symdecl`, with `macro=` replacing the name of the semantic macro. All of them are only relevant, if either `name=` or `macro=` are provided.

  As with `\symdecl`, if no `name` is given, but `macro` is, then the same name is used for both the semantic macro and the symbol itself.

  If `name` is given but `macro` isn't, no semantic macro is generated. Subsequently, the newly generated symbol is added the `for`-list.
- `style`: see chapter 9.
- `title`: a title to use in various styles (see chapter 9).
- `id`: a label to use for `\sref`.

\inlineass
\inlinedef
\inlineex

The macros \inlineass, \inlinedef and \inlineex behave like the sassertion, sdefinition and sexample environments respectively, but take the text to annotate as an argument, rather than as the body of an environment, and do not break paragraphs.

The same macros available in the environments are also available in the argument of the \inline* macros.

\varbind

\varbind{⟨*cls*⟩}

retroactively attaches the bind option to every variable provided (as a comma-separated list).

8.1 More on Definitions

In sdefinition (and sparagraph with style=symdoc), the following additional macros are available:

\definiendum
\definame
\Definame
\defnotation

The \definiendum macro behaves largely like \symref, but it uses a dedicated highlighting for *definienda* and adds the referenced symbol to the for= list of the environment.

\definame is to \definiendum as \symname is to \symref. Analogously, \Definame behaves like \Symname.

\defnotation can be used in math mode to apply the \definiendum highlighting to notations.

\definiens

The \definiens macro can be used to semantically annotate the *definiens* in a sdefinition.

If the sdefinition environment has several elements in its for list, an optional argument \definiens[⟨*symbol*⟩]{...} can be used to tell sTeX which symbol's definiens this is. By default, the *first* symbol in the for list is used.

Here is how MMT will treat the fragment marked up with \definiens:
Firstly, it will attempt to translate its contents into an MMT/OMDoc term. This succeeds easily if \definiens is some semantic macro (applied to arguments).
Secondly, it will check for all variables currently in scope, that were defined with the optional argument bind. It then will check, whether a symbol is in scope, that has role=lambda. If so, it will use that lambda symbol to bind all these variables (in the order in which they were defined) in the term. If no lambda symbol is found, it will use the bind symbol that ships with sTeX.
The final term will be attached as definiens to the corresponding MMT constant, if it was declared in the same module as the \definiens occurrence.

8.2 More on Assertions

`\premise`
`\conclusion`

The `\conclusion` macro can be used to mark up the actual statement within an `sassertion`. The `\premise` macro can be used to additionally mark up *premises*.

If the `sassertion` environment has several elements in its `for` list, an optional argument `\conclusion[`⟨*symbol*⟩`]{...}` can be used to tell sTeX which symbol's statement this is. By default, the *first* symbol in the `for` list is used.

Here is how MMT will treat the fragments marked up with `\conclusion` and `\premise`:
Firstly, it will attempt to translate the contents of `\conclusion` into an MMT/OMDOC term c. This succeeds easily if the `\conclusion` is some semantic macro (applied to arguments).
Secondly, it will collect all fragments marked up with `\premise` and do the same to them $(p_1, ..., p_n)$.
It will then check, whether a symbol is in scope, that has `role=implication`. If so, it will use that `implication` symbol to attach all the premises to the conclusion, resulting in $t :=$ `imply`$(p_1, ..., p_n, c)$.
Next, it will check for all variables currently in scope, that were defined with the optional argument `bind`. It then will check, whether a symbol is in scope, that has `role=forall`. If so, it will use that `forall` symbol to bind all these variables (in the order in which they were defined) in the term t.
Finally, it will check, whether a symbol is in scope, that has `role=judgment`. If so, it will set $t :=$ `judgment`(t).
If no `forall` symbol is found, it will first apply the `judgment` symbol (if existent) and then use the bind symbol that ships with sTeX to bind the variables.
The final term will be attached as type to the corresponding MMT constant, if it was declared in the same module as the `\definiens` occurrence.

M
M
T

`sproof` (*env.*)

TODO[13]

[13] TODO: proofs

Chapter 9

Customizing Typesetting

There are two kinds of typesetting that can be customized in sTeX: symbol references (notation components, \symref, variables, etc.) are highlighted using a small set of macros that can be simply redefined by authors.

Other macros and environments usually have more complicated "typesetting rules" associated with them – often in the form of other already existing environments that should be used.

Lastly, in HTML we can provide custom CSS rules in math archives that determine the styling of certain environments, so that the actual presentation depends on the document in which the fragments are included (e.g. via \inputref), rather than the file the fragment is implemented in.

It is generally recommended to implement these customizations in a preamble in the lib directory (see Section 0.3 (The lib-Directory) in the sTeX Manual)

9.1 Highlighting Symbol References

\symrefemph
\symrefemph@uri

\symrefemph governs how references via \symref (or \symname, or theirt short variants) are highlighted;

Doing \symref{⟨*symbol*⟩}{⟨*text*⟩} ultimately expands to \symrefemph@uri{⟨*text*⟩}{⟨*symbol URI*⟩}, the default implementation of which is just \symrefemph{⟨*text*⟩}. The default implementation of \symrefemph, in turn, is just \emph{⟨*text*⟩}.

If you only want to change e.g. the color of \symrefs, you only need to redefine \symrefemph, e.g. using

```
\renewcommand\symrefemph[1]{\textcolor{red}{#1}}
```

the @uri variant is useful if you want to link somewhere, or show the URI in a tooltip. The stex-highlighting package does both, using:

```
\usepackage{pdfcomment}
\protected\def\symrefemph@uri#1#2{%
  \pdftooltip{%
    \srefsymuri{#2}{\symrefemph{#1}}%
  }{%
    URI:~\detokenize{#2}%
  }%
```

```
}
\def\symrefemph#1{%
  \ifcsname textcolor\endcsname
    \textcolor{teal}{#1}%
  \else#1\fi
}
```

`\compemph`
`\compemph@uri`

Like `\symrefemph` and `\symrefemph@uri`, but governs the highlighting of components (marked with `\comp` or `\maincomp`) in notations.

`\defemph`
`\defemph@uri`

Like `\symrefemph` and `\symrefemph@uri`, but governs the highlighting of definienda marked with `\definiendum` (or `\definame`).

`\varemph`
`\varemph@uri`

Like `\compemph` and `\compemph@uri`, but governs the highlighting of components (marked with `\comp` or `\maincomp`) in the notations of variables.

The second argument to `\varemph@uri` is the *name* of the variable.

9.2 Styling Environments and Macros

A variety of environments and macros provided by sTeX are *stylable* using the macros `\stexstyle`⟨*name*⟩`[`⟨*style*⟩`]{...}`. These stylable environments and macros bind various of their parameters to macros `\this`⟨*parameter*⟩, which can then be used in the styles.

For example, if we have a `definition` environment that we would want to use to style our `sdefinition`s, we can do (in the simplest case)

```
\stexstyledefinition{\begin{definition}}{\end{definition}}
```

This tells sTeX to insert `\begin{definition}` at the beginning of every `sdefinition` environment, and `\end{definition}` at the end.

If we have a environments `theorem` and `lemma`, we probably want the `sassertion` environment to use those for theorems and lemma. We can achieve that by doing

```
\stexstyleassertion[theorem]{\begin{theorem}}{\end{theorem}}
\stexstyleassertion[lemma]{\begin{lemma}}{\end{lemma}}
```

Now if we do `\begin{sassertion}[style=theorem]`, it will wrap the environment with `\begin{theorem}...\end{theorem}`.

Of course, many such statements might have a title, as e.g. in

> **Satz 9.2.1** (Gödel's First Incompleteness Theorem)**.** ...

In `sasssertion`, we can provide that title as optional argument using `title=...`. Before calling the styling provided, `sassertion` will store that title in the macro `\thistitle`, which we can use in the styling. For example, we might prefer to pass it on to the `theorem` environment:

```
\stexstyleassertion[theorem]{\ifx\thistitle\@empty
  \begin{theorem}\else\begin{theorem}[\thistitle]\fi}
  {\end{theorem}}
```

\stexstylemodule
\stexstylecopymodule
\stexstyleinterpretmodule
\stexstylerealization
\stexstylemathstructure
\stexstyleextstructure
\stexstyledefinition
\stexstyleassertion
\stexstyleexample
\stexstyleparagraph
\stexstyleproof
\stexstylesubproof

TODO[14]

Additionally, we can style certain macros, if we want them to produce output. For example, we might (for debuggin or documentation purposes) \symdecl to give a short summary of the symbol.

We can achieve that by doing, for example:

```
\stexstylesymdecl[debug]{
  Symbol \thisdeclname~(with arity \thisargs) of type $\thistype$.
}
```

in which case

```
\symdecl{foo}[args=2,type=\mathbb{N},style=debug]
```

will yield

> Symbol foo (with arity ii) of type $\mathbb{N}$.

\stexstyleusemodule
\stexstyleimportmodule
\stexstylerequiremodule
\stexstyleassign
\stexstylerenamedecl
\stexstyleassignMorphism
\stexstylecopymod
\stexstyleinterpretmod
\stexstylerealize
\stexstylesymdecl
\stexstyletextsymdecl
\stexstylenotation
\stexstylevarnotation
\stexstylesymdef
\stexstylevardef
\stexstylevarseq
\stexstylespfsketch
\stexstyleMMTinclude

TODO[15]

9.3 Custom CSS for Environments

Chapter 10

Additional Packages

10.1 NotesSlides Manual

10.1.1 Introduction

The notesslides document class is derived from `beamer.cls` [**beamerclass:on**], it adds a "notes version" for course notes that is more suited to printing than the one supplied by `beamer.cls`.

The notesslides class takes the notion of a slide frame from Till Tantau's excellent beamer class and adapts its notion of frames for use in the sTEX and OMDoc. To support semantic course notes, it extends the notion of mixing frames and explanatory text, but rather than treating the frames as images (or integrating their contents into the flowing text), the notesslides package displays the slides as such in the course notes to give students a visual anchor into the slide presentation in the course (and to distinguish the different writing styles in slides and course notes).

In practice we want to generate two documents from the same source: the slides for presentation in the lecture and the course notes as a narrative document for home study. To achieve this, the notesslides class has two modes: *slides mode* and *notes mode* which are determined by the package option.

10.1.2 Package Options

The notesslides class takes a variety of class options:

`slides` `notes` The options `slides` and `notes` switch between slides mode and notes mode (see subsection 4.1.3).

`sectocframes` If the option `sectocframes` is given, then for the `sfragment`s, special frames with the `sfragment` title (and number) are generated.

`frameimages` `fiboxed` If the option `frameimages` is set, then slide mode also shows the `\frameimage`-generated frames (see ???). If also the `fiboxed` option is given, the slides are surrounded by a box.

10.1.3 Notes and Slides

`frame` (*env.*) Slides are represented with the `frame` environment just like in the beamer class, see [**Tantau:ugbc**] for details.

`note` (*env.*) The notesslides class adds the `note` environment for encapsulating the course note fragments.

> Note that it is essential to start and end the `notes` environment at the start of the line – in particular, there may not be leading blanks – else LaTeX becomes confused and throws error messages that are difficult to decipher.

By interleaving the `frame` and `note` environments, we can build course notes as shown here:

```
\ifnotes\maketitle\else
\frame[noframenumbering]\maketitle\fi

\begin{note}
  We start this course with ...
\end{note}

\begin{frame}
  \frametitle{The first slide}
  ...
\end{frame}
\begin{note}
  ... and more explanatory text
\end{note}

\begin{frame}
  \frametitle{The second slide}
  ...
\end{frame}
...
```

`\ifnotes` Note the use of the `\ifnotes` conditional, which allows different treatment between `notes` and `slides` mode – manually setting `\notestrue` or `\notesfalse` is strongly discouraged however.

> We need to give the title frame the `noframenumbering` option so that the frame numbering is kept in sync between the slides and the course notes.

> The beamer class recommends not to use the `allowframebreaks` option on frames (even though it is very convenient). This holds even more in the `notesslides` case: At least in conjunction with `\newpage`, frame numbering behaves funnily (we have tried to fix this, but who knows).

`\inputref*` If we want to transclude a the contents of a file as a note, we can use a new variant `\inputref*` of the `\inputref` macro: `\inputref*{foo}` is equivalent to `\begin{note}\inputref{foo}\end{note}`.

`nparagraph` (*env.*)
`nparagraph` (*env.*)
`ndefinition` (*env.*)
`nexample` (*env.*)
`nsproof` (*env.*)
`nassertion` (*env.*)

There are some environments that tend to occur at the top-level of `note` environments. We make convenience versions of these: e.g. the `nparagraph` environment is just an `sparagraph` inside a `note` environment (but looks nicer in the source, since it avoids one level of source indenting). Similarly, we have the `nfragment`, `ndefinition`, `nexample`, `nsproof`, and `nassertion` environments.

10.1.4 Customizing Header and Footer Lines

The notesslides package and class comes with a simple default theme named sTeX that provided by the beamterthemesTeX. It is assumed as the default theme for sTeX-based notes and slides. The result in `notes` mode (which is like the `slides` version except that the slide hight is variable) is

The footer line can be customized. In particular the logos.

`\setslidelogo` The default logo provided by the notesslides package is the sTeX logo it can be customized using `\setslidelogo{`⟨*logo name*⟩`}`.

`\setsource` The default footer line of the notesslides package mentions copyright and licensing. In notesslides `\source` stores the author's name as the copyright holder. By default it is the author's name as defined in the `\author` macro in the preamble. `\setsource{`⟨*name*⟩`}` can change the writer's name.

`\setlicensing` For licensing, we use the Creative Commons Attribuition-ShareAlike license by default to strengthen the public domain. If package `hyperref` is loaded, then we can attach a hyperlink to the license logo. `\setlicensing[`⟨*url*⟩`]{`⟨*logo name*⟩`}` is used for customization, where ⟨*url*⟩ is optional.

10.1.5 Frame Images

Sometimes, we want to integrate slides as images after all – e.g. because we already have a PowerPoint presentation, to which we want to add sTeX notes.

\frameimage
\mhframeimage

In this case we can use \frameimage[⟨opt⟩]{⟨path⟩}, where ⟨opt⟩ are the options of \includegraphics from the graphicx package [**CarRah:tpp99**] and ⟨path⟩ is the file path (extension can be left off like in \includegraphics). We have added the label key that allows to give a frame label that can be referenced like a regular beamer frame.

The \mhframeimage macro is a variant of \frameimage with repository support. Instead of writing

```
\frameimage{\MathHub{fooMH/bar/source/baz/foobar}}
```

we can simply write (assuming that \MathHub is defined as above)

```
\mhframeimage[fooMH/bar]{baz/foobar}
```

Note that the \mhframeimage form is more semantic, which allows more advanced document management features in MathHub.

If baz/foobar is the "current module", i.e. if we are on the MathHub path ...MathHub/fooMH/bar..., then stating the repository in the first optional argument is redundant, so we can just use

```
\mhframeimage{baz/foobar}
```

\textwarning

The \textwarning macro generates a warning sign: ⚠

10.1.6 Ending Documents Prematurely

\prematurestop
\afterprematurestop

For prematurely stopping the formatting of a document, sTEX provides the \prematurestop macro. It can be used everywhere in a document and ignores all input after that – backing out of the sfragment environments as needed. After that – and before the implicit \end{document} it calls the internal \afterprematurestop, which can be customized to do additional cleanup or e.g. print the bibliography.

\prematurestop is useful when one has a driver file, e.g. for a course taught multiple years and wants to generate course notes up to the current point in the lecture. Instead of commenting out the remaining parts, one can just move the \prematurestop macro. This is especially useful, if we need the rest of the file for processing, e.g. to generate a theory graph of the whole course with the already-covered parts marked up as an overview over the progress; see import_graph.py from the lmhtools utilities [**lmhtools:github:on**].

Text fragments and modules can be made more re-usable by the use of global variables. For instance, the admin section of a course can be made course-independent (and therefore re-usable) by using variables (actually token registers) courseAcronym and courseTitle instead of the text itself. The variables can then be set in the sTEX preamble of the course notes file.

10.1.7 Global Document Variables

To make document fragments more reusable, we sometimes want to make the content depend on the context. We use **document variables** for that.

\setSGvar
\useSGvar

\setSGvar{⟨vname⟩}{⟨text⟩} to set the global variable ⟨vname⟩ to ⟨text⟩ and \useSGvar{⟨vname⟩} to reference it.

\ifSGvar

With\ifSGvar we can test for the contents of a global variable: the macro call \ifSGvar{⟨vname⟩}{⟨val⟩}{⟨ctext⟩} tests the content of the global variable ⟨vname⟩, only if (after expansion) it is equal to ⟨val⟩, the conditional text ⟨ctext⟩ is formatted.

10.1.8 Excursions

In course notes, we sometimes want to point to an "excursion" – material that is either presupposed or tangential to the course at the moment – e.g. in an appendix. The typical setup is the following:

```
\excursion{founif}{../fragments/founif.en}
  {We will cover first-order unification in}
...
\begin{appendix}\printexcursions\end{appendix}
```

It generates a paragraph that references the excursion whose source is in the file ../fragments/founif.en.tex and automatically books the file for the \printexcursions command that is used here to put it into the appendix. We will look at the mechanics now.

\excursion

The \excursion{⟨ref⟩}{⟨path⟩}{⟨text⟩} is syntactic sugar for

```
\begin{nparagraph}[title=Excursion]
  \activateexcursion{founif}{../ex/founif}
  We will cover first-order unification in \sref{founif}.
\end{nparagraph}
```

\activateexcursion
\printexcursion
\excursionref

Here \activateexcursion{⟨path⟩} augments the \printexcursions macro by a call \inputref{⟨path⟩}. In this way, the \printexcursions macro (usually in the appendix) will collect up all excursions that are specified in the main text.

Sometimes, we want to reference – in an excursion – part of another. We can use \excursionref{⟨label⟩} for that.

\excursiongroup

Finally, we usually want to put the excursions into an sfragment environment and add an introduction, therefore we provide the a variant of the \printexcursions macro: \excursiongroup[id=⟨id⟩,intro=⟨path⟩] is equivalent to

```
\begin{note}
\begin{sfragment}[id=<id>]{Excursions}
  \inputref{<path>}
  \printexcursions
\end{sfragment}
\end{note}
```

When option `book` which uses `\pagestyle{headings}` is given and semantic macros are given in the `sfragment` titles, then they sometimes are not defined by the time the heading is formatted. Need to look into how the headings are made. This is a problem of the underlying document-structure package.

Local Variables: mode: latex TeX-master: `../stex-manual` End:

10.2 Problem Manual

10.2.1 Introduction

The problem package supplies an infrastructure that allows specify problem. Problems are text fragments that come with auxiliary functions: hints, notes, and solutions. Furthermore, we can specify how long the solution to a given problem is estimated to take and how many points will be awarded for a perfect solution.

Finally, the problem package facilitates the management of problems in small files, so that problems can be re-used in multiple environment.

10.2.2 Problems and Solutions

`solutions`
`notes`
`hints`
`gnotes`
`pts`
`min`
`boxed`
`test`

The problem package takes the options `solutions` (should solutions be output?), `notes` (should the problem notes be presented?), `hints` (do we give the hints?), `gnotes` (do we show grading notes?), `pts` (do we display the points awarded for solving the problem?), `min` (do we display the estimated minutes for problem soling). If theses are specified, then the corresponding auxiliary parts of the problems are output, otherwise, they remain invisible.

The `boxed` option specifies that problems should be formatted in framed boxes so that they are more visible in the text. Finally, the `test` option signifies that we are in a test situation, so this option does not show the solutions (of course), but leaves space for the students to solve them.

`problem` (*env.*) The main environment provided by the problempackage is (surprise surprise) the `problem` environment. It is used to mark up problems and exercises. The environment takes an optional KeyVal argument with the keys `id` as an identifier that can be reference later, `pts` for the points to be gained from this exercise in homework or quiz situations, `min` for the estimated minutes needed to solve the problem, and finally `title` for an informative title of the problem.

Example 27
Input:

```
\documentclass{article}
\usepackage[solutions,hints,pts,min]{problem}
\begin{document}
  \begin{sproblem}[id=elefants,pts=10,min=2,title=Fitting Elefants]
    How many Elefants can you fit into a Volkswagen beetle?
    \begin{hint}
      Think positively, this is simple!
    \end{hint}
    \begin{exnote}
      Justify your answer
    \end{exnote}
\begin{solution}
  Four, two in the front seats, and two in the back.
  \begin{gnote}
    if they do not give the justification deduct 5 pts
  \end{gnote}
\end{solution}
\end{sproblem}
\end{document}
```

Output:

Exercise (Fitting Elefants)
How many Elefants can you fit into a Volkswagen beetle?

Lösung: Four, two in the front seats, and two in the back.

`solution` (*env.*) The `solution` environment can be to specify a solution to a problem. If the package option `solutions` is set or `\solutionstrue` is set in the text, then the solution will be presented in the output. The `solution` environment takes an optional KeyVal argument with the keys `id` for an identifier that can be reference `for` to specify which problem this is a solution for, and `height` that allows to specify the amount of space to be left in test situations (i.e. if the `test` option is set in the `\usepackage` statement).

`hint` (*env.*)
`exnote` (*env.*)
`gnote` (*env.*)
The `hint` and `exnote` environments can be used in a `problem` environment to give hints and to make notes that elaborate certain aspects of the problem. The `gnote` (grading notes) environment can be used to document situations that may arise in grading.

`\startsolutions`
`\stopsolutions`
Sometimes we would like to locally override the `solutions` option we have given to the package. To turn on solutions we use the `\startsolutions`, to turn them off, `\stopsolutions`. These two can be used at any point in the documents.

`\ifsolutions`
Also, sometimes, we want content (e.g. in an exam with master solutions) conditional on whether solutions are shown. This can be done with the `\ifsolutions` conditional.

10.2.3 Markup for Added-Value Services

The `problem` package is all about specifying the meaning of the various moving parts of practice/exam problems. The motivation for the additional markup is that we can base added-value services from these, for instance auto-grading and immediate feedback.

The simplest example of this are multiple-choice problems, where the `problem` package allows to annotate answer options with the intended values and possibly feedback that can be delivered to the users in an interactive setting. In this section we will give some infrastructure for these, we expect that this will grow over time.

Multiple Choice Blocks

`mcb` (*env.*) Multiple choice blocks can be formatted using the `mcb` environment, in which single choices are marked up with `\mcc` macro.

`\mcc` `\mcc[`⟨*keyvals*⟩`]{`⟨*text*⟩`}` takes an optional key/value argument ⟨*keyvals*⟩ for choice metadata and a required argument ⟨*text*⟩ for the proposed answer text. The following keys are supported

- `T` for true answers, `F` for false ones,
- `Ttext` the verdict for true answers, `Ftext` for false ones, and
- `feedback` for a short feedback text given to the student.

What we see when this is formatted to PDF depends on the context. In solutions mode (we start the solutions in the code fragment below) we get

Example 28

Input:

```
 \startsolutions
\begin{sproblem}[title=Functions,name=functions1]
 What is the keyword to introduce a function definition in python?
 \begin{mcb}
   \mcc[T]{def}
   \mcc[F,feedback=that is for C and C++]{function}
   \mcc[F,feedback=that is for Standard ML]{fun}
   \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
 \end{mcb}
\end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☑ def
- ☐ function[a]
- ☐ fun[b]
- ☐ public static void[c]

---

[a] *that is for C and C++*

[b] *that is for Standard ML*

[c] Nooooooooo *that is for Java*

In "exam mode" where disable solutions (here via `\stopsolutions`)

Example 29

Input:

```
 \stopsolutions
\begin{sproblem}[title=Functions,name=functions1]
 What is the keyword to introduce a function definition in python?
 \begin{mcb}
   \mcc[T]{def}
   \mcc[F,feedback=that is for C and C++]{function}
   \mcc[F,feedback=that is for Standard ML]{fun}
   \mcc[F,Ftext=Nooooooooo,feedback=that is for Java]{public static void}
 \end{mcb}
\end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☐ def
- ☐ function
- ☐ fun
- ☐ public static void

we get the questions without solutions (that is what the students see during the exam/quiz).

Filling-In Concrete Solutions

The next simplest situation, where we can implement auto-grading is the case where we have fill-in-the-blanks

`\fillinsol` The `\fillinsol` macro takes a single argument, which contains a concrete solution (i.e. a number, a string, ...), which generates a fill-in-box in test mode:

Example 30

Input:

```
\stopsolutions
\begin{sproblem}[id=elefants.fillin,title=Fitting Elefants]
How many Elefants can you fit into a Volkswagen beetle? \fillinsol{4}
\end{sproblem}
```

Output:

Exercise (Fitting Elefants)

How many Elefants can you fit into a Volkswagen beetle? ☐

and the actual solution in solutions mode:

Example 31

Input:

```
\begin{sproblem}[id=elefants.fillin,title=Fitting Elefants]
How many Elefants can you fit into a Volkswagen beetle? \fillinsol{4}
\end{sproblem}
```

Output:

Exercise (Fitting Elefants)

How many Elefants can you fit into a Volkswagen beetle? ☐

If we do not want to leak information about the solution by the size of the blank we can also give `\fillinsol` an optional argument with a size: `\fillinsol[3cm]{12}` makes a box three cm wide.

Obviously, the required argument of `\fillinsol` can be used for auto-grading. For concrete data like numbers, this is immediate, for more complex data like strings "soft comparisons" might be in order. [16]

10.2.4 Including Problems

`\includeproblem` The `\includeproblem` macro can be used to include a problem from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one problem in the include file). The keys `title`, `min`, and `pts` specify the problem title, the estimated minutes for solving the problem and the points to be gained, and their values (if given) overwrite the ones specified in the `problem` environment in the included file.

[16] For the moment we only assume a single concrete value as correct. In the future we will almost certainly want to extend the functionality to multiple answer classes that allow different feedback like im MCQ. This still needs a bit of design. Also we want to make the formatting of the answer in solutions/test mode configurable.

The sum of the points and estimated minutes (that we specified in the `pts` and `min` keys to the `problem` environment or the `\includeproblem` macro) to the log file and the screen after each run. This is useful in preparing exams, where we want to make sure that the students can indeed solve the problems in an allotted time period.

The `\min` and `\pts` macros allow to specify (i.e. to print to the margin) the distribution of time and reward to parts of a problem, if the `pts` and `pts` options are set. This allows to give students hints about the estimated time and the points to be awarded.

10.2.5 Testing and Spacing

The `problem` package is often used by the `hwexam` package, which is used to create homework assignments and exams. Both of these have a "test mode" (invoked by the package option `test`), where certain information –master solutions or feedback – is not shown in the presentation.

`\testspace`
`\testsmallspace`
`\testsmallspace`
`\testsmallspace`
`\testnewpage`
`\testemptypage`

`\testspace` takes an argument that expands to a dimension, and leaves vertical space accordingly. Specific instances exist: `\testsmallspace`, `\testsmallspace`, `\testsmallspace` give small (1cm), medium (2cm), and big (3cm) vertical space.

`\testnewpage` makes a new page in `test` mode, and `\testemptypage` generates an empty page with the cautionary message that this page was intentionally left empty.

Local Variables: mode: latex TeX-master: `../stex-manual` End:

LocalWords: solutions,notes,hints,gnotes,pts,min,boxed,test gnotes elefants,pts gnote LocalWords: 2,title exnote hint,exnote,gnote ifsolutions mcb keyvals Ttext Ftext LocalWords: Functions,name F,feedback Noooooooooo,feedback 2,title includeproblem LocalWords: elefants.fillin,title

10.3 HWExam Manual

10.3.1 Introduction

The hwexam package and class supplies an infrastructure that allows to format nice-looking assignment sheets by simply including problems from problem files marked up with the problem package. It is designed to be compatible with `problems.sty`, and inherits some of the functionality.

10.3.2 Package Options

`solutions`
`notes`
`hints`
`gnotes`
`pts`
`min`

The hwexam package and class take the options `solutions`, `notes`, `hints`, `gnotes`, `pts`, `min`, and `boxed` that are just passed on to the problems package (cf. its documentation for a description of the intended behavior).

`multiple`

Furthermore, the hwexam package takes the option `multiple` that allows to combine multiple assignment sheets into a compound document (the assignment sheets are treated as section, there is a table of contents, etc.).

`test`

Finally, there is the option `test` that modifies the behavior to facilitate formatting tests. Only in `test` mode, the macros `\testspace`, `\testnewpage`, and `\testemptypage`

have an effect: they generate space for the students to solve the given problems. Thus they can be left in the LaTeX source.

10.3.3 Assignments

`assignment` (*env.*)
`number`
`title`
`type`
`given`
`due`

This package supplies the `assignment` environment that groups problems into assignment sheets. It takes an optional KeyVal argument with the keys `number` (for the assignment number; if none is given, 1 is assumed as the default or — in multi-assignment documents — the ordinal of the `assignment` environment), `title` (for the assignment title; this is referenced in the title of the assignment sheet), `type` (for the assignment type; e.g. "quiz", or "homework"), `given` (for the date the assignment was given), and `due` (for the date the assignment is due).

10.3.4 Including Assignments

`\inputassignment`

The `\inputassignment` macro can be used to input an assignment from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one `assignment` environment in the included file). The keys `number`, `title`, `type`, `given`, and `due` are just as for the `assignment` environment and (if given) overwrite the ones specified in the `assignment` environment in the included file.

10.3.5 Typesetting Exams

`testheading` (*env.*)
`duration`
`min`
`reqpts`

The `\testheading` takes an optional keyword argument where the keys `duration` specifies a string that specifies the duration of the test, `min` specifies the equivalent in number of minutes, and `reqpts` the points that are required for a perfect grade.

```
\title{320101 General Computer Science (Fall 2010)}
\begin{testheading}[duration=one hour,min=60,reqpts=27]
  Good luck to all students!
\end{testheading}
```

Will result in

Name: Matriculation Number:

320101 General Computer Science (Fall 2010)

2026-03-24

You have one hour (sharp) for the test;
Write the solutions to the sheet.
The estimated time for solving this exam is 23 minutes, leaving you 37 minutes for revising your exam.
You can reach 23 points if you solve all problems. You will only need 27 points for a perfect score, i.e. -4 points are bonus points.

Different problems test different skills and knowledge, so do not get stuck on one problem.

	To be used for grading, do not write here								
prob.	4.1	1.1	1.2	2.1	2.2	2.3	2.4	2.5	2.6
total	0	0	0	10	0	0	0	0	0
reached									

good luck

[17]

Local Variables: mode: latex TeX-master: `../stex-manual` End:
LocalWords: hwexam solutions,notes,hints,gnotes,pts,min gnotes testemptypage reqpts LocalWords: inputassignment reqpts hour,min 60,reqpts

10.4 Tikzinput Manual

`image` The behavior of the ikzinput package is determined by whether the `image` option is given. If it is not, then the tikz package is loaded, all other options are passed on to it and `\tikzinput{`⟨*file*⟩`}` inputs the TIKZ file ⟨*file*⟩`.tex`; if not, only the graphicx package is loaded and `\tikzinput{`⟨*file*⟩`}` loads an image file ⟨*file*⟩.⟨*ext*⟩ generated from ⟨*file*⟩`.tex`.

The selective input functionality of the tikzinput package assumes that the TIKZ pictures are externalized into a standalone picture file, such as the following one

[17] MK: The first three "problems" come from the stex examples above, how do we get rid of this?

```
\documentclass{standalone}
\usepackage{tikz}
\usetikzpackage{...}
\begin{document}
  \begin{tikzpicture}
     ...
  \end{tikzpicture}
\end{document}
```

The standalone class is a minimal LaTeX class that when loaded in a document that uses the standalone package: the preamble and the `documenat` environment are disregarded during loading, so they do not pose any problems. In effect, an `\input` of the file above only sees the `tikzpicture` environment, but the file itself is standalone in the sense that we can run LaTeX over it separately, e.g. for generating an image file from it.

`\tikzinput`
`\ctikzinput`

This is exactly where the tikzinput package comes in: it supplies the `\tikzinput` macro, which – depending on the `image` option – either directly inputs the TIKZ picture (source) or tries to load an image file generated from it.

Concretely, if the `image` option is not set for the tikzinput package, then `\tikzinput[`⟨*opt*⟩`]{`⟨*file*⟩`}` disregards the optional argument ⟨*opt*⟩ and inputs ⟨*file*⟩`.tex` via `\input` and resizes it to as specified in the `width` and `height` keys. If it is, `\tikzinput[`⟨*opt*⟩`]{`⟨*file*⟩`}` expands to `\includegraphics[`⟨*opt*⟩`]{`⟨*file*⟩`}`.

`\ctizkinput` is a version of `\tikzinput` that is centered.

`\mhtikzinput`
`\cmhtikzinput`

`\mhtizkinput` is a variant of `\tikzinput` that treats its file path argument as a relative path in a math archive in analogy to `\inputtref`. To give the archive path, we use the `mhrepos=` key. Again, `\cmhtizkinput` is a version of `\mhtikzinput` that is centered.

`\libusetikzlibrary`

Sometimes, we want to supply archive-specific TIKZ libraries in the `lib` folder of the archive or the `meta-inf/lib` of the archive group. Then we need an analogon to `\libinput` for `\usetikzlibrary`. The stex-tikzinput package provides the `libusetikzlibrary` for this purpose.

Local Variables: mode: latex TeX-master: `../stex-manual` End:

Part III
Documentation

Chapter 11

Utilities

Various utility methods

`\stex_ignore_spaces_and_pars:`

Ignores space characters and empty lines (which would close the current paragraph)

`\stex_undefine:N`
`\stex_undefine:c`

Locally undefines a macro

`\stex_str_if_starts_with_p:nn` ⋆ `\stex_str_if_starts_with:nn {⟨str1⟩} {⟨str2⟩}`
`\stex_str_if_starts_with:nnTF` ⋆

Tests if ⟨*str1*⟩ starts with ⟨*str2*⟩.

`\stex_str_if_ends_with_p:nn` ⋆ `\stex_str_if_ends_with:nn {⟨str1⟩} {⟨str2⟩}`
`\stex_str_if_ends_with:nnTF` ⋆

Tests if ⟨*str1*⟩ ends with ⟨*str2*⟩.

`\stex_deactivate_macro:Nn` `\stex_deactivate_macro:Nn \macro {⟨scope⟩}`
`\stex_reactivate_macro:N`

redefines `\macro` to throw an error, that the `\macro` is only allowed in ⟨*scope*⟩. This allows for more informative error messages than `undefined control sequence`.

`\stex_reactivate_macro:N` makes the macro valid again.

11.1 kpsewhich and Environment Variables

`\stex_kpsewhich:Nn` `\stex_kpsewhich:Nn \macro {⟨args⟩}`

Calls "`kpsewhich` ⟨*args*⟩" and stores the result in `\macro`,

TEXhackers note: (Usually) does not require `shell-escape`, since by default, `kpsewhich` is in the list of "allowed" commands.

`\stex_get_env:Nn` `\stex_get_env:Nn \macro {⟨envvar⟩}`

Stores the value of the environment variable ⟨*envvar*⟩ in `\macro`.

11.2 Logging

`\stex_debug:nn` `\stex_debug:nn {⟨prefix⟩} {⟨msg⟩}`

Logs the debug message `{⟨msg⟩}` under the prefix `{⟨prefix⟩}`. A message is shown if its prefix is in a list of prefixes given either via the package option `debug=⟨prefixes⟩` or the environment variable `STEX_DEBUG=⟨prefixes⟩`, where the latter overrides the former.

11.3 File Paths

Since sTeX needs to handle files from various *math archives* in a user-file-system-independent manner in arbitrary *MathHub* directories, we primarily use *absolute* file paths, for operations such as `\inputref`, `\usemodule`, `\importmodule`, etc.

We therefore provide macros to explicitly handle file paths independent of the underlying operating system, resolving and canonicalizing `..` segments and relative paths, etc.

`\stex_file_set:Nn` `\stex_file_set:(No|Ne)` `\stex_file_set:Nn \macro {⟨string⟩}`

represents an already canonicalized file path string as a LaTeX3 sequence and stores it in `\macro`.

`\stex_file_resolve:Nn` `\stex_file_resolve:(No|Ne)` `\stex_file_resolve:Nn \macro {⟨string⟩}`

resolves and canonicalizes the file path string ⟨*string*⟩ and stores the result in `\macro`.

`\stex_file_use:N ⋆` expands to a string representation of the given file path (using `/` as separator, regardless of file system).

`\stex_file_split_off_ext:NN` `\stex_file_split_off_ext:NN \target \source`

splits off the file extension of `\source` and stores the resulting file path in `\target`.

`\stex_file_split_off_lang:NN` `\stex_file_split_off_lang:NN \target \source`

splits off *both* the file extension *and* language component of `\source` and stores the resulting file path in `\target`; e.g. if `\source` is `foo.en.tex`, `\target` will be `foo`. Explicitly checks that the segment before the file extension is a valid language identifier; otherwise, it will not be stripped.

`\c_stex_pwd_file` `\c_stex_main_file` represent the *parent working directory* and absolute path to the top-level `.tex` file currently being processed (as absolute, canonicalized paths)

`\c_stex_home_file` represents the absolute path of the user's home directory (as absolute, canonicalized path)

`\g_stex_current_file` the current file (as absolute, canonicalized path)

`\stex_input_with_hooks:Nn` `\stex_input_with_hooks:Ne`

`\stex_input_with_hooks:Nn \document-URI {⟨file string⟩}`

Inputs the given file (as a string) by passing it on to `\input`, setting `\g_stex_current_file`, `\l_stex_current_language_str` and `\l_stex_current_document_uri`, and reverting them again afterwards

`\stex_with_file_hooks:Nnn`

`\stex_with_file_hooks:Nnn \document-URI {⟨file string⟩} {⟨code⟩}`

sets `\g_stex_current_file`, `\l_stex_current_language_str` and `\l_stex_current_document_uri`, executes ⟨*code*⟩, and reverts them again afterwards

`\stex_if_file_absolute_p:N` ⋆
`\stex_if_file_absolute:NTF` ⋆

tests whether the given file path (represented as a canonicalized LaTeX3 sequence) is absolute.

`\stex_if_file_starts_with:NNTF` `\stex_if_file_starts_with:NN \child \parent`

tests whether the file path `\child` is a child of `\parent`.

11.4 Group-like Behaviours

Macros for mimicking the behaviour of TeX groups without actually opening a TeX group. This is relevant for two purposes:

- Temporarily redefining a macro, doing a bunch of stuff, and then reverting it to its previous definition; that is what a "*pseudogroup*" does.
- Defining macros at a specific group level further up. That is what a "*metagroup*" does – code wrapped in `\stex_metagroup_do_in:n` will be repeatedly executed in `\aftergroup` calls, up to the group level of the innermost metagroup. That allows for e.g. declaring new symbols in `sdefinition` paragraphs that are valid in the entire `mathstructure` or `module`.

`\stex_pseudogroup:nn` `\stex_pseudogroup:nn {⟨code⟩} {⟨reset code⟩}`

will *expand* ⟨*reset code*⟩ first, then process ⟨*code*⟩, then process the expanded ⟨*reset code*⟩.

`\stex_pseudogroup_restore:N` ⋆ `\stex_pseudogroup_restore:N \macro`

will expand to code that will redefine `\macro` as its current definition. Only works for *token lists* and similar macros that do not take arguments.

In combination, that allows for things like

Example 32

```
% \stex_pseudogroup:nn{
% \def \foo {..}
% ...
% }{\stex_pseudogroup_restore:N \foo}
%
```

`\stex_pseudogroup_with:nn` `\stex_pseudogroup_with:nn {⟨macro-list⟩} {⟨code⟩}`

will store the definitions of the macros in ⟨*macro-list*⟩, execute ⟨*code*⟩, and then revert the macros to their previous definitions without opening a TEX group. TODO: this might be unnecessary? Check/profile

`\stex_metagroup_new:` opens a new *metagroup*. All code executed in `\stex_metagroup_do_in:n` will survive up to the current group level. Metagroups can be nested.

`\stex_metagroup_do_in:n`
`\stex_metagroup_do_in:e` Executes the given code in the current metagroup. If there is no metagroup currently, the code will simply be executed once, so there is little danger in calling this spuriously.

TEXhackers note: This repeatedly stores the provided code in a macro and uses `\aftergroup` until the group level of the current metagroup is reached.

11.5 Key Handling

Key handling mechanisms on top of l3keys for inheriting key sets and initializing the associated macros

`\stex_keys_define:nnnn` `\stex_keys_define:nnnn {⟨id⟩} {⟨init⟩} {⟨spec⟩} {⟨exts⟩}`

defined the key set ⟨*id*⟩ with the l3keys-style key-parsing code `{`⟨*spec*⟩`}`; ⟨*init*⟩ is called every time before parsing and is intended to e.g. clear the relevant macros. ⟨*exts*⟩ may be a comma-separated list of key set ids from which this key set inherits.

`\stex_keys_set:nn` Like `\keys_set:nn`, but additionally calling the relevant initialization code.

11.6 Languages

\c_stex_languages_prop
\c_stex_language_abbrevs_prop

Two inverse property lists; \c_stex_languages_prop maps language abbreviations (e.g. en, de) to their (babel) names (e.g. english, ngerman), \c_stex_language_abbrevs_prop does the inverse.

\l_stex_current_language_str

The language of the current file; may change, if a file with a different language is imported.

11.7 Styling

\stex_new_stylable_cmd:nnnn
\stex_style_apply:

\stex_new_stylable_cmd:nnnn {⟨*macroname*⟩} {⟨*argument spec*⟩} {⟨*definition*⟩} {⟨*default*⟩}

Defines a new stylable command \macroname with ⟨*argument spec*⟩ (passed on to \NewDocumentCommand) with ⟨*definition*⟩ as its macro body and default typesetting style ⟨*default*⟩.

⟨*default*⟩ may be empty. ⟨*definition*⟩ should at some point use \stex_style_apply: to do the typesetting according to the chosen style for this macro, which by default will be defined as ⟨*default*⟩.

Also generates the macro \stexstyle⟨*macroname*⟩[⟨*name*⟩]{⟨*code*⟩} to define a new typesetting style for \macroname.

\stex_new_stylable_env:nnnnnnn

\stex_new_stylable_env:nnnnnnn {⟨*environment name*⟩} {⟨*argument spec*⟩} {⟨*begin code*⟩} {⟨*end code*⟩} {⟨*default begin*⟩} {⟨*default end*⟩} {⟨*prefix*⟩}

Defines a new stylable environment ⟨*prefix*⟩⟨*environment name*⟩ with ⟨*argument spec*⟩ (passed on to \NewDocumentEnironment) with ⟨*begin code*⟩ and ⟨*end code*⟩ as its macro body and default typesetting style dictated by ⟨*default begin*⟩ and ⟨*default end*⟩.

⟨*begin code*⟩ and ⟨*end code*⟩ should at some point use \stex_style_apply: to do the typesetting according to the chosen style for this environment.

Also generates the macro \stexstyle⟨*environmentname*⟩[⟨*name*⟩]{⟨*begin*⟩}{⟨*end*⟩} to define a new typesetting style for environment name.

A prefix s is e.g. used to generate the environment sparagraph and the macro \stexstyleparagraph.

11.8 Persisting Dependencies

\stex_persist:n
\stex_persist:e

Writes the given code in the .sms-file (if smsmode is on)

Chapter 12

HTML Output

Macros for dealing with HTML output.

`\stex@backend` Which engine is running; possible values are `pdflatex`, `rustex`, `tex4ht` (TODO) and `latexml` (TODO).

`\stex_if_html_backend_p:` ⋆
`\stex_if_html_backend:TF` ⋆
LaTeX3 conditional testing whether the engine running compiles to HTML.

`\ifstexhtml` ⋆ LaTeX2 conditional testing whether the engine running compiles to HTML.

`\stex_if_do_html_p:` ⋆
`\stex_if_do_html:TF` ⋆
Whether to (locally) produce HTML output. May be turned off locally even if the backend produces HTML; e.g. when parsing dependencies in `\importmodule` or `\usemodule`.

`\stex_suppress_html:n` temporarily suppresses HTML output when processing the provided code.

`\stex_annotate:nn` `\stex_annotate:nn {⟨keyvals⟩} code`

attaches the provided comma-separated key-value-pairs (as `key=val`) as HTML attributes to the HTML node(s) generated from ⟨*code*⟩. If ⟨*code*⟩ results in multiple nodes, this needs to insert a new `<div>`, `<span>` or `<mrow>` node for the attributes. Multiple nested calls to `\stex_annotate:nn` may be merged into a single node.

`stex_annotate_env` (*env.*) like `\stex_annotate:nn`, but as an environment; i.e. `\begin{stex\_annotate\_env}{⟨keyvals⟩}{⟨` is equivalent to `\stex_annotate:nn{⟨keyvals⟩}{⟨code⟩}`

`\stex_html_node:nnn` `\stex_html_node:nnn {⟨tag⟩} {⟨keyvals⟩} code`

like `\stex_annotate:nn`, but makes sure to wrap ⟨*code*⟩ in a new `<tag>` node.

`stex_env_node` (*env.*) like `\stex_html_node:nnn`, but as an environment; i.e. `\begin{stex\_env\_node}{⟨tag⟩}{⟨keyva`

is equivalent to `\stex_html_node:nnn{⟨tag⟩}{⟨keyvals⟩}{⟨code⟩}`

`\stex_annotate_invisible:nn` `\stex_annotate_invisible:n`

`\stex_annotate_invisible:nn {⟨keyvals⟩} code`

like `\stex_annotate:nn`, but additionally adds the key values `data-ftml-invisible="true"` and `style="display:none;"`. `\stex_annotate_invisible:n` does not take additional attribute-value-pairs.

`\STEXinvisible`

public wrapper for `\stex_annotate_invisible:n`.

`\stex_css_link:n`

`\stex_css_link:n {⟨url⟩}`

inserts a `<link rel="stylesheet" href="⟨url⟩">` node in the header of the generated HTML.

`\stex_css_literal:n`

`\stex_css_literal:n {⟨text⟩}`

inserts a `<style>⟨text⟩</style>` node in the header of the generated HTML.

`\_stex_annotate_force_break:n`

should guarantee that the HTML nodes generated by the provided code are not merged with any outer nodes with respect to attached attributes.

`\mmlintent` `\mmlarg`

`\mmlintent {⟨value⟩} {⟨code⟩}`

annotates ⟨*code*⟩ with the provided MathML *intent* attribute (or *intent* `arg` attribute, respectively).

and style patching:

```
⟨@@=stex_styles⟩
\stex_if_html_backend:TF{
  \cs_new_protected:Nn \stex_style_apply: {}
  \cs_new_protected:Nn \__stex_styles_patch:nnnn {}
  \stex_keys_define:nnnn{ csspatch }{
    \str_clear:N \l_stex_keys_cls_id
    \str_clear:N \l_stex_keys_counter_id
    \str_clear:N \l_stex_keys_counter_parent
  }{
    counter    .str_set_x:N  = \l_stex_keys_counter_id,
    parent     .str_set_x:N  = \l_stex_keys_counter_parent,
    unknown    .code:n       = {
      \exp_args:NNo \str_set:Nn \l_stex_keys_cls_id {\l_keys_key_tl}
    }
  }{}
  \cs_new_protected:Npn \__stex_styles_css_patch:nnnn #1 #2 {
    \stex_keys_set:nn{ csspatch }{ #2 }
    \group_begin:
    \catcode`\ =12\relax
    \catcode`^^J=12\relax
    \__stex_styles_patch_i:nnn {#1}
  }
```

```
\seq_new:N \g__stex_styles_counters_seq

\cs_new:Nn \__stex_styles_patch_html_c: {
  \stex_html_literal:n{
    <span~data-ftml-counter="\l_stex_keys_counter_id"~style="display:none;"~
      data-ftml-counter-parent="\l_stex_keys_counter_parent"
    ></span>
  }
}

\cs_new:Nn \__stex_styles_patch_html: {
  \stex_html_literal:n{
    <span~data-ftml-style="\l_stex_keys_cls_id"~style="display:none;"~
      data-ftml-counter="\l_stex_keys_counter_id"
    ></span>
  }
}

\cs_new_protected:Nn \__stex_styles_patch_i:nnn {
  \str_if_empty:NTF \l_stex_keys_cls_id {
    \str_set:Nn \l_stex_keys_cls_id { #1 }
  }{
    \str_set:Ne \l_stex_keys_cls_id { #1-\l_stex_keys_cls_id }
  }
  \exp_args:No \str_case:nnF \l_stex_keys_counter_parent {
    {}{}
    {part}{\str_set:Nn \l_stex_keys_counter_parent { 0 }}
    {chapter}{\str_set:Nn \l_stex_keys_counter_parent { 1 }}
    {section}{\str_set:Nn \l_stex_keys_counter_parent { 2 }}
    {subsection}{\str_set:Nn \l_stex_keys_counter_parent { 3 }}
    {subsubsection}{\str_set:Nn \l_stex_keys_counter_parent { 4 }}
    {paragraph}{\str_set:Nn \l_stex_keys_counter_parent { 5 }}
    {subparagraph}{\str_set:Nn \l_stex_keys_counter_parent { 6 }}
  }{
    \msg_error:nnx{stex}{error/notasection}\l_stex_keys_counter_parent
  }
  \str_set:Ne \l__stex_styles_first_str { &~.ftml-title-paragraph~ { display:inline-block
  \str_if_empty:NF \l_stex_keys_counter_id {
    %\str_put_left:Ne \l__stex_styles_first_str { counter-increment:~ ftml-\l_stex_keys_c
    \exp_args:NNo \seq_if_in:NnF \g__stex_styles_counters_seq \l_stex_keys_counter_id {
      \seq_gpush:No \g__stex_styles_counters_seq \l_stex_keys_counter_id
      \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
        \__stex_styles_patch_html_c:
        % in body
      }{
        \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html_c:
        % in preamble
      }
    }
    \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
        \__stex_styles_patch_html:
        % in body
    }{
```

```
        \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html:
        % in preamble
      }
    }
    \exp_args:Ne \stex_css_literal:n { .ftml-\l_stex_keys_cls_id { \l__stex_styles_first_st

    % TODO export counter reset somehow

    \group_end:
  }
}{
  \cs_new_protected:Nn \__stex_styles_patch:nnnn {
    \str_if_empty:nTF {#2}{
      \tl_set:cn{__stex_styles_style_#1_start:}{#3}
      \tl_set:cn{__stex_styles_style_#1_end:}{#4}
    }{
      \tl_set:cn{__stex_styles_style_#1_#2_start:}{#3}
      \tl_set:cn{__stex_styles_style_#1_#2_end:}{#4}
    }
  }
  \cs_new_protected:Nn \__stex_styles_css_patch:nnnn {}
}
```

Chapter 13

Math Archives

`\c_stex_mathhub_files`
`\mathhub`

represents the comma-separated, absolute path *strings* of the user's MathHub directories. If the `\mathhub` macro is set when the `stex` package is loaded, this one is used for finding archives. Otherwise it is determined from the `MATHHUB` environment variable, if set, or from the path in `<HOME>/.stex/mathhub.path`, if existent, or set to `<HOME>/MathHub` if all else fails. In all cases, `\mathhub` will be defined.

MathHub directories are scanned in order; i.e. earlier paths are prioritized over later ones.

A *math archive* is represented as a triple `{<archive id>}{<base uri>}{<file path string>}`, where an invariant is that ⟨*file path string*⟩ is of the form `<mh>/<archive id>` for some directory `<mh>` in `\c_stex_mathhub_files`.

Such a triple is stored in the macro `\c_stex_mathhub_<id>_archive` when the archives metadata is first loaded.

`\stex_archive_id:N` ⋆

expands to the id of the given archive (a macro defined as a triple as described above)

`\stex_archive_base:N` ⋆
`\stex_archive_base:n` ⋆

expands to the base URI of the given archive; either given as a macro defined as a triple as described above, or as the archive's id. The latter requires the archive to be loaded beforehand!

`\stex_archive_path:N` ⋆
`\stex_archive_path:n` ⋆

expands to the file path *as a string* of the given archive; either given as a macro defined as a triple as described above, or as the archive's id. The latter requires the archive to be loaded beforehand!

`\stex_in_archive:nn`

`\stex_in_archive:nn` {⟨*archive id*⟩} {⟨*code*⟩}

Temporarily sets the current archive `\l_stex_current_archive` to ⟨*archive id*⟩, executes ⟨*code*⟩ and reverts back to the previous archive. Passes the id of the *now current* archive to ⟨*code*⟩ as `#1`: If the first argument is empty, this will be the current archive's id without changing it. Otherwise, the archive with the desired id will be loaded; throws an error, if it does not exist.

`\c_stex_no_archive_str`
`\c_stex_no_archive`

The archive ID `\c_stex_no_archive_str`=`no/archive` is used for documents and other content that is not contained in a *math archive*. `\c_stex_no_archive` is the corresponding archive triple.

`\c_stex_main_archive`
`\l_stex_current_archive`

Point to the main file's archive and the *current* archive, respectively, if existent; undefined otherwise.

`\stex_source_path:n` ⋆
`\stex_source_path:nn` ⋆

`\stex_source_path:n {`⟨*relative path*⟩`}`

expands to the absolute file path (string) of the given ⟨*relative path*⟩ relative to the `source` directory of the current archive – or relative to the current file, if we are not in an archive. In the latter case, the file path will *not* be canonicalized as to remain expandable.

The variant `\stex_source_path:nn` takes the ID of an archive as first argument (instead of the current archive).

Chapter 14

URIs

There are kinds of URIs used in iMmt/OMDoc:

- *Base URIs*, i.e. namespaces,
- *Archive URIs* of the form `<base uri>?a=<archive id>`,
- *Document URIs* of the form `<archive uri>[&p=<path>]&d=<name>&l=<language>`,
- *Document element URIs* of the form `<document uri>&e=<name>`,
- *Module URIs* of the form `<archive uri>[&p=<path>]&m=<name>`, and
- *Declaration URIs* of the form `<module uri>&s=<name>`.

We do not need all of these in sTeX itself: the *base uri* of any URI is uniquely determined by the archive ID, and we conceptually merge document URIs and document element URIs.

We therefore represent document (element) URIs as 5-tuples `{<archive id>}{<path>}{<name>}{<language>}{<element name>}`, where `<path>` and `<element name>` may be empty; module URIs as triples `{<archive id>}{<path>}{<name>}`, and declaration URIs as 4-tuples `{<archive id>}{<path>}{<module name>}{<name>}`.

`\stex_use_archive_uri:n` ⋆ expands to the string representation of the archive URI of the archive with the given ID. Requires, that the archive has been loaded first

14.1 Document URIs

`\stex_use_document_uri:N` ⋆
`\stex_use_document_uri:n` ⋆
expands to the string representation of the document uri (represented as a macro containing a 5-tuple as above). The variant `\stex_document_uri:n` expects a 5-tuple directly.

`\stex_document_uri_archive:N` ⋆

expands to the archive ID of the document URI

`\stex_document_uri_path:N ⋆`

expands to the path of the document URI (possibly empty)

`\stex_document_uri_name:N ⋆`

expands to the document name of the document URI

`\stex_document_uri_language:N ⋆`

expands to the language of the document URI

`\stex_document_uri_element:N ⋆`

expands to the element name of the document (element) URI (possibly empty)

`\stex_document_uri_with_language:Nn ⋆`

expands to the document URI with the given language

`\stex_new_document_element_uri:n ⋆`

expands to a new document element URI in the current document

`\stex_document_uri_from_archive_file:Nn` `\stex_uri_from_archive_file:Nn \macro {⟨relative path⟩}`

Constructs the document URI of the file ⟨*relative path*⟩ in the current archive and stores it in `\macro`

`\c_stex_main_document_uri`
`\l_stex_current_document_uri`

store the document URIs of the top-level file being processed and the *current* file, respectively.

14.2 Module URIs

`\stex_use_module_uri:N ⋆`
`\stex_use_module_uri:n ⋆`

expands to the string representation of the module uri (represented as a macro containing a triple as above). The variant `\stex_module_uri:n` expects a triple directly.

`\stex_module_uri_archive:N ⋆`

expands to the archive ID of the module URI

`\stex_module_uri_path:N ⋆`
`\stex_module_uri_path:n ⋆`

expands to the path of the module URI (possibly empty)

`\stex_module_uri_name:N` ⋆
`\stex_module_uri_name:n` ⋆

expands to the name of the module URI

`\stex_module_uri_split_name:NNN`
`\stex_module_uri_split_name:NNn`

sets `#1` as the parent module of the module URI and `#2` as the last name

`\stex_module_uri_as_qm:n` ⋆

`\stex_new_module_uri:n` ⋆

`\stex_new_module_uri:n {`⟨*name*⟩`}`

expands to a new module URI triple with the given ⟨*name*⟩ based on the current document URI or module URI, if existent

`\stex_uri_from_pair:Nnn`

`\stex_uri_from_pair:Nnn {`⟨*archive id*⟩`} {`⟨*module path*⟩`}`

resolves the pair `[`⟨*archive id*⟩`]{`⟨*module path*⟩`}` to a module URI and stores the result in `#1`

`\stex_uri_from_pair:Nn`

like `\stex_uri_from_pair:Nnn`, but takes a token list as argument that may or may not be of the form `[archive]module`

14.3 Symbol URIs

`\stex_use_symbol_uri:N` ⋆
`\stex_use_symbol_uri:n` ⋆

expands to the string representation of the module uri (represented as a macro containing a 4-tuple as above). The variant `\stex_symbol_uri:n` expects a 4-tuple directly.

`\stex_new_symbol_uri:n` ⋆

`\stex_new_symbol_uri:n {`⟨*name*⟩`}`

expands to a new symbol URI tuple with the given ⟨*name*⟩ based on the current module URI, which is assumed to exist(!).

`\stex_new_symbol_uri:nn` ⋆

expands to a new symbol URI tuple with the given ⟨*name*⟩ based on the *given* module URI (either as macro or as tuple), which is assumed to exist(!).

`\stex_symbol_uri_archive:N` ⋆

expands to the archive ID of the symbol URI

`\stex_symbol_uri_path:N` ⋆

expands to the path of the symbol URI (possibly empty)

`\stex_symbol_uri_module:N ⋆`
`\stex_symbol_uri_module:n ⋆`

expands to the URI of the containing module of the symbol URI

`\stex_symbol_uri_name:N ⋆`
`\stex_symbol_uri_name:n ⋆`

expands to the name of the symbol URI

Chapter 15

Documents

\STEXftmllink
\STEXlinkftml
\STEXsetlinkftml

\STEXlinkftml inserts the following link/disclaimer:

> *The FTML version of this document can be found at*
> *http://unknown.source?a=no/archive&p=/home/jazzpirate/work/Software/sTeX/sTeX/doc&d=stex-doc&l=en*

The precise text can be set using \STEXsetlinkftml{⟨*text*⟩}; The link on its own can be inserted using \STEXftmllink.

\stexdoctitle

\stexdoctitle {⟨*title*⟩}

Sets ⟨*title*⟩ to be the title of the document. Only the first call of this command does something.

15.1 Sectioning

`\l_stex_current_section_level_int`
`\l_stex_current_section_level_str`

integer keeping track of the current sectioning level:

0 part

1 chapter

2 section

3 subsection

4 subsubsection

5 paragraph

>5 subparagraph

`\setsectionlevel` sets `\l_stex_current_section_level_int` to the corresponding integer value. `\l_stex_current_section_level_str` stores a string representation of the same value, but will initially contain `document`.

`\currentsectionlevel`
`\Currentsectionlevel`

will leave the current section level as text in the token input. The variant `\Currentsectionlevel` will produce the capitalized version.

If the `xspace` package is loaded, this will insert an `\xspace` afterwards.

In HTML mode, this will insert an annotation, so it can be adapted dynamically.

`sfragment` (*env.*)

A section at the current section level; preferred over the primitives `\section`, `\subsection`, etc., because a) environments are better suited for such structuring, and b) it allows for adapting the inserted section header based on document context, rather than it being hardcoded – e.g. what's a subsection in one document may be a section in another.

`titlefragment` (*env.*)

like `sfragment`, but will use `\maketitle` instead of (one of) the sectioning macros, if no title has been used yet.

`blindfragment` (*env.*)

skips one sectioning level in its body so that e.g. subsection 0.1 can be used before the first section

`\skipfragment`

Skips one counter at the current sectioning level; e.g. increase the subsection counter to 2 if called immediately after `\section` (or, rather, after `\begin{sfragment}`).

`\setsectionlevel`

`\setsectionlevel {`⟨*name*⟩`}`

Sets the current section level to the one specified (e.g. `\setsectionlevel{subsection}`). Should ideally be called at most once in the preamble of a document.

15.2 Inter-Document References

`[id=foo]` sets the current fragment's (e.g. `Definition 3.1 (Foo)`) name to `foo`, resulting in a DocumentElementURI `..&e=foo`. It then delegates to `\label{...&e=foo}` (which defines `\r@...&e=foo`), stores `:::&e=foo` in `\g_stex_sref_label_foo`, and writes `\STeXInternalSRefLabel{foo}{..&e=foo}{definition.3.1}{Foo}` to the sref file.

15.3 Inputting From MathHub Resources

\inputref `\inputref [⟨archive id⟩] {⟨filepath⟩}`

Inputs the referenced file in the referenced archive (or current archive, if empty) in a tex group, while setting all the relevant macros for the current archive, document URI, etc.

In HTML, will instead just insert an annotation, so that the HTML of the referenced document can be dynamically inserted instead.

\mhinput `\inputref [⟨archive id⟩] {⟨filepath⟩}`

Like `\inputref`, but *always* actually inputs the referenced file (also in HTML mode!) and without opening a tex group.

\ifinputref LaTeX2 conditional; is true, if we currently are in an `\inputref` or `\mhinput`.

\IfInputref `\IfInputref {⟨true⟩} {⟨false⟩}`

Executes ⟨*true*⟩ if we currently are in an `\inputref` or `\mhinput`, otherwise executes ⟨*false*⟩.

In HTML, *both* branches are executed(!) and inserted in the HTML with corresponding attributes, so that the relevant parts can be shown or hidden.

\libinput `\libinput [⟨archive id⟩] {⟨file name⟩}`

inputs all files ⟨*file name*⟩`.tex` in the `lib` directories of the current archive. Will throw an error if we are not in an archive or no suitable file is found.

\libusepackage `\libusepackage [⟨package options⟩] {⟨package name⟩}`

inputs all packages ⟨*package name*⟩`.sty` in the `lib` directories of the current archive. Will throw an error if we are not in an archive, or there is not *exactly one* file ⟨*package name*⟩`.sty` somewhere in the `lib` directories.

Note that unlike `\libinput`, the optional argument does *not* refer to an archive ID, since this would conflict with package options.

Will give a spurious warning that package `mathhub/directory/foo` was requested, but package `foo` was found. This is a side-effect of using absolute file paths, which is necessary to make the package findable in the first place.

\libusetikzlibrary `\libusetikzlibrary [⟨archive id⟩] {⟨name⟩}`

like `\libusepackage`, but for `\usetikzlibrary`.

`\addmhbibresource` `\addmhbibresource [⟨archive id⟩] {⟨file name⟩}`

Calls `\addbibresource` on all ⟨*file name*⟩`.bib` files in the `lib` directories of the current archive. Will throw an error if we are not in an archive or no suitable file is found.

`\mhgraphics`
`\cmhgraphics`

`\mhgraphics` adds the `archive` key to the optional arguments of `\includegraphics` to allow for using images in math archives (relative to its source directory). `\cmhgraphics` additionally wraps the image in a `\begin{center}`.

Only defined if the graphicx package is loaded (but may be loaded after the stex package).

`\lstinputmhlisting`
`\clstinputmhlisting`

like `\mhgraphics`, but for `\lstinputlisting`.

Only defined if the listings package is loaded (but may be loaded after the stex package).

`\mhtikzinput`
`\cmhtikzinput`

like `\mhgraphics`, but for `\tikzinput`.

Only defined if the tikzinput package is loaded (but may be loaded after the stex package).

Chapter 16

Modules

`\l_stex_current_module_uri` stores the URI of the current module, if existent

`\l_stex_all_modules_seq` stores the URIs of all modules *currently in scope*.

`smodule` (*env.*) Parses the optional arguments, (if relevant) inserts HTML annotations, and then delegates to `\stex_module_setup:n`

`\stex_module_setup:n` Sets up a new module with the given name by checking whether it is a nested module, and if not, has a signature etc. Loads signature and metatheory, if necessary TODO: deprecate in favor of every document being a module

`\stex_module_setup_top_nosig:n`

Sets up a new top-level module with the given name and no signature. Does not load a meta theory. TODO: deprecate in favor of every document being a module

`\stex_close_module:` closes the current module.

`\stex_every_module:n` adds code to be executed every time a new module is opened (e.g. activating certain macros).

`\stex_do_up_to_module:n`
`\stex_do_up_to_module:e` Execute code in the current module (i.e. as if the `\begin{`⟨*smodule*⟩`}` was the current tex group)

`\stex_execute_in_module:n`
`\stex_execute_in_module:e` Execute code in the current module (i.e. as if the `\begin{`⟨*smodule*⟩`}` was the current tex group) and also adds it to the initialization code of the module; i.e. exports all macros defined.

`\stex_if_in_module_p:` ⋆
`\stex_if_in_module:TF` ⋆

conditional whether we currently are in a module. TODO: deprecate in favor of every document being a module

`\stex_if_module_exists_p:N` ⋆
`\stex_if_module_exists:NTF` ⋆
`\stex_if_module_exists_p:n` ⋆
`\stex_if_module_exists:nTF` ⋆

conditional whether a module with the given URI exists

`\stex_activate_module:N`
`\stex_activate_module:n`

`\setmetatheory` `\setmetatheory [⟨archive id⟩] {⟨module⟩}`

sets the metatheory of all subsequent modules to the specified one

`\STEXexport` `\STEXexport {⟨code⟩}`

executes the provided code every time the current module is opened (including immediately). Processes its content in the `\ExplSyntaxOn` category code scheme.

`\stex_structural_feature_module:nn`
`\stex_structural_feature_module_end:`

16.1 SMS Mode

sTEX has to extract formal content (i.e. modules and their symbols) from LaTeX-files, that may otherwise contain arbitrary code, including macros that may not be defined unless the file is fully processed by TeX. Those modules and symbols also may depend on other modules that have not yet been loaded. The naive way to achieve this, which would be to just suppress output (e.g. by storing it in a box register) and then `\input` the required file, does not work thanks to TeX's limited *file stack*, which would overflow quickly for modules that have a deeply nested list of dependencies.

To solve those problems, sTEX reads dependencies in what we call *sms mode*, which can be summarized thusly:

- In a first pass, we parse the file token by token, ignoring everything other than a select list of macros and environments that introduce dependencies (such as `\importmodule` and `\begin{smodule}[sig=...]`). Instead of loading those, we remember them for later.
- After the file as been fully parsed thusly, the dependencies found are loaded, again in sms-mode.
- In a second pass, we parse the file *again* in the same way, but this time execute all macros that are explicitly allowed in sms mode, such as `\importmodule`, `\symdecl`, `\notation`, `\symdef`, etc.

- all this parsing happens additionally in a `\setbox\throwaway\vbox{...}`-block to suppress any accidental output.

This means that TEX's input stack never grows by more than +1, but still behaves *as if* the dependencies were loaded recursively, at the detriment of being somewhat slow.

`\stex_sms_allow:N` registers the provided macro to be allowed in sms mode.

This only works, if the macro takes no arguments and/or does not touch the subsequent tokens.

`\stex_sms_allow_escape:N` registers the provided macro to be allowed in sms mode.

If the macro is subsequently encountered in sms-mode, parsing is halted and it can process arguments as desired. It then needs to continue parsing manually though, by calling `\stex_smsmode_do:` as (usually) its last token.

`\stex_sms_allow_env:n` registers the provided environment to be allowed in sms mode.

As with `\stex_sms_allow_escape:N`, the `\begin{`⟨*envname*⟩`}` is escaped, hence the begin-code of the environment needs to call `\stex_smsmode_do:`. Since `\end{`⟨*envname*⟩`}` never takes arguments, it does not need to be escaped.

`\stex_sms_allow_import:Nn` `\stex_sms_allow_import:Nn \macro {`⟨*code*⟩`}`

registers the provided macro to be allowed in the *first pass* in sms mode. The provided ⟨*code*⟩ is executed at the start of the first pass, to define macros accordingly.

`\stex_sms_allow_import_env:nn` `\stex_sms_allow_import_env:nn \envname {`⟨*code*⟩`}`

registers the provided environment to be allowed in the *first pass* in sms mode. The provided ⟨*code*⟩ is executed at the start of the first pass, to define macros accordingly.

`\g_stex_sms_import_code` inserted between the first and second pass; macros/environments allowed via `\stex_sms_allow_import_*` should store their "results" in here.

`\stex_if_smsmode_p:` ⋆
`\stex_if_smsmode:`*TF* ⋆
tests for whether we are currently in sms-mode.

`\stex_file_in_smsmode:Nn` `\stex_file_in_smsmode:Nn \document-URI {`⟨*file string*⟩`}`

Loads `{`⟨*file string*⟩`}` in sms mode, setting `\l_stex_current_document_uri` etc. accordingly and reverting them afterwards.

`\stex_smsmode_do:` should be called in escaped macros or environments allowed in sms mode; switches back to parsing file contents ignoring everything not explicitly allowed in sms mode.

Does nothing outside of sms mode, so can be called safely.

16.2 Inheritance and Morphisms

`\stex_require_module:N`
`\stex_require_module_noerr:N`
`\stex_require_module_noerr:n`

makes sure that the module with the given URI is in scope. If not, it will attempt to load the module from disk. `\stex_require_module:N` throws an error if the module can't be found; `\stex_require_module_noerr:N` silently fails.

The `\*:n`-variant takes the URI tuple directly rather than a macro.

`\usemodule` `\usemodule [⟨archive ID⟩] {⟨module⟩}`

loads the specified module without exporting it further.

`\importmodule` `\importmodule [⟨archive ID⟩] {⟨module⟩}`

loads the specified module and exports it further.

`\requiremodule` `\requiremodule [⟨archive ID⟩] {⟨module⟩}`

loads the specified module without exporting it further, but generates FTML equivalent to `\importmodule`.

`\stex_add_morphism:nnnn` `\stex_add_morphism:nnnn {⟨name⟩} {⟨module URI⟩} {⟨kind⟩} {⟨contents⟩}`

adds a morphism to the current module

16.3 Theory Morphisms

`\stex_structural_feature_morphism:nnnnn`
`\stex_structural_feature_morphism_with_macros:nnnnn`
`\stex_structural_feature_morphism_end:`

`#1` :	name
`#2` :	kind
archive ID	
`#3` :	module spec
`#4` :	additional attributes

`\stex_iterate_morphisms:nn`

`\stex_get_in_morphism:n`

`copymodule` (*env.*) (and `\copymod`)

`interpretmodule` (*env.*) (and `\interpretmod`)

\assign

\renamedecl

\assignMorphism

Chapter 17

Symbols

17.1 Declarations

\symdecl `\symdecl[*] {⟨iname⟩} [⟨options⟩]`

takes care of the optional arguments, styling, generates the symbol, defines the relevant macros and adds them to the current module.

\symdef `\symdef {⟨iname⟩} [⟨options⟩] {⟨notation⟩}`

\textsymdecl

\stex_symdecl_do: does the actual work. Requires all the `\l_stex_key_*` macros are set.

_stex_symdecl_html: not namespaced so it can be reused in e.g. `sdefinition`s that generate new symbols. Requires that the relevant `\l_stex_key_*` macros and `\l_stex_macroname_str` are set.

\stex_add_symbol:nnnnnnN

#1 : {⟨*Macro name*⟩}
#2 : {⟨*Name*⟩}
#3 : {⟨*arity*⟩}
#4 : {({⟨*Arg num*⟩}{⟨*Arg str*⟩})*}
#5 : Definiens
#6 : type
#7 : Return
#8 : Command

adds a new symbol to the current module.

\stex_has_definiens_p:N ⋆
\stex_has_definiens:N*TF* ⋆
\stex_has_definiens_p:n ⋆
\stex_has_definiens:n*TF* ⋆

17.2 Retrieval

`\stex_iterate_symbols:n`
`\stex_iterate_break:`
`\stex_iterate_break:n`

iterates over all symbols currently in scope. The provided code will receive nine arguments: The URI of the containing module and the eight parameters as in `\stex_add_symbol:nnnnnnnN`.

iteration is stopped in the code using `\stex_iterate_break:` or `\stex_iterate_break:n`.

`\stex_iterate_symbols:nn`

`\stex_iterate_symbols:nn {⟨modules⟩} {⟨code⟩}`

iterates over all symbols in the comma-separated list of module URIs in ⟨*modules*⟩. The provided code will receive nine arguments: The URI of the containing module and the eight parameters as in `\stex_add_symbol:nnnnnnnN`.

iteration is stopped in the code using `\stex_iterate_break:` or `\stex_iterate_break:n`.

`\stex_get_symbol:n`
`\stex_get_symbol:nF`

attempts fo find the symbol with the given name/id/URI suffix. If not successful, will execute the `F` code or throw an error. Otherwise, defines the following macros:

- `\l_stex_get_symbol_uri`,
- `\l_stex_get_symbol_arity_int`,
- `\l_stex_get_symbol_args_tl`,
- `\l_stex_get_symbol_def_tl`,
- `\l_stex_get_symbol_type_tl`,
- `\l_stex_get_symbol_return_tl`,
- `\l_stex_get_symbol_invoke_cs`

17.3 Variables

`\l_stex_variables_prop`

contains all variables currently in scope.

`\vardef`

`\stex_get_var:n`

17.3.1 Variable Sequences

`\varseq`

17.4 Expressions

`\symuse` retrieves a symbol by name/id and invokes it as if using a semantic macro.

`\_stex_next_symbol:n` code to be executed the next time a symbol is invoked.

`\_stex_invoke_symbol:NnnnnnN`
`\_stex_invoke_symbol:NeoooN`
`\_stex_invoke_symbol:nnnnnnN`

invokes the symbol. Takes as arguments the following data, and defines the corresponding macros accordingly:

- `\l_stex_current_symbol_uri`,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- definiens (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invokation macro (is called after setup).

Also defines `\l_stex_current_full_tl` and `\l_stex_current_display_tl`

For a "normal" symbol defined via `\symdecl` or `\symdef`, `\l_stex_current_symbol_invoke_cs` is defined as `\stex_invoke_symbol:`.

Opens a new TEX group that needs to be closed by `\l_stex_current_symbol_invoke_cs`!

`\stex_invoke_symbol:` Invokes `\l_stex_current_symbol_uri`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_text_symbol:` Invokes `\l_stex_current_symbol_uri` as a symbol declared via `\textsymdecl`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_sequence:` Invokes `\l_stex_current_symbol_uri` as a sequence variable declared via `\varseq`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_structure:` Invokes a mathstructure symbol; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\_stex_invoke_variable:nnnnnn`

invokes the variable. Takes as arguments the following data, and defines the corresponding macros accordingly:

- variable name,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- definiens (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invokation macro (is called after setup).

`\svar`

17.4.1 Term Annotations

`\_stex_eat_exclamation_point:`

removes spurious `!` characters

`\_stex_term_oms:nn` `\_stex_term_oms:nn {⟨notation id⟩} {⟨code⟩}`

annotes ⟨*code*⟩ with `OMID="\l_stex_current_symbol_uri"`

`\_stex_term_omv:nn` `\_stex_term_omv:nn {⟨notation id⟩} {⟨code⟩}`

annotes ⟨*code*⟩ with `OMV="\l_stex_current_symbol_uri"`

TODO: This shouldn't use `\l_stex_current_symbol_uri`!

`\_stex_term_oma:nn` `\_stex_term_oma:nn {⟨notation id⟩} {⟨code⟩}`

annotes ⟨*code*⟩ with `OMA="\l_stex_current_symbol_uri"`

`\_stex_term_omb:nn` `\_stex_term_omb:nn {⟨notation id⟩} {⟨code⟩}`

annotes ⟨*code*⟩ with `OMBIND="\l_stex_current_symbol_uri"`

17.4.2 Symbol Arguments

`\_stex_term_arg:nnnnn` marks an argument of a symbol application, sets the precedence accordingly, sets `\l_stex_allow_semantic_bool` etc.

`#1` : argument number
`#2` : argument mode
`#3` : precedence
`#4` : argument name (WiP)
`#5` : code / body

`\_stex_term_arg:nnn` `\_stex_term_arg:nnn {⟨mode⟩} {⟨num⟩} {⟨code⟩}`

inserts the HTML annotations for the argument

`\_stex_term_arg_aB:nnnnn` takes care of sequence arguments

17.4.3 Notation components

`\comp`
`\compemph@uri`
`\compemph`

`\varemph@uri`
`\varemph`

`\defemph@uri`
`\defemph`

`\symrefemph@uri`
`\symrefemph`

The following commands do the actual attributes:

`\_do_comp:nnNn` `\_do_comp:nNn {⟨key-suffix⟩} {⟨html-value⟩} \comp-macro {⟨code⟩}`

annotated ⟨*code*⟩ with `data-ftml-`⟨*key-suffix*⟩ using the highlighting dictated by `\comp-macro`.

17.4.4 Symbol References

`\symref`
`\sr`

`\symname`
`\sn`
`\sns`
`\Symname`
`\Sn\Sns`

`\varref`
`\varname`
`\Varname`

`\definiendum`
`\definame`
`\Definame`
`\defnotation`

`\foldexpr`

17.5 Checking

`\stex_if_check_terms_p:` ⋆
`\stex_if_check_terms:`*TF* ⋆

conditional whether terms (types and definientia of symbols) should get syntactically checked. In HTML mode, this is always false, since they get written to the HTML anyway.

`\stex_check_term:nn`

typesets the given expression in a `\tiny\marginpar`, checking its syntactic validity. Does nothing, if term checking is not explicitly activated.

`\_stex_check_terms:`

checks the type and definiens components of a symbol declaration (i.e. `\stex_key_type_tl`, `\stex_key_def_tl` and `\stex_key_return_tl!`).

Chapter 18

Notations

`\notation`

`\varnotation`

`\stex_notation_parse:n` parses the body of a notations and saves it in `\l_stex_notation_macrocode_cs`

`\_stex_notation_add:` adds the fully parsed notation to the current module

`\stex_add_notation:nnnnn`
`\stex_add_notation:ooeoo`

#1 : URI
#2 : variant
#3 : arity
#4 : macro body
#5 : op
adds the notation to the current module

`\_stex_map_args:N`
`\_stex_map_notation_args:N`

`\_stex_var_notation_macro:`

`\setnotation`
`\_stex_notation_set_default:n`

`\stex_iterate_notations:nn` `\stex_iterate_notations:nn {⟨modules⟩} {⟨code⟩}`

iterates over all notations in the comma-separated list of module URIs and executes {⟨*code*⟩} with #1,#2,#3,#4,#5 as the notation parameters.

`\stex_use_notation:nnTF`
`\stex_use_op_notation:nnTF`

`\stex_use_notation:nn {⟨uri⟩} {⟨id⟩} {⟨exists code⟩} {⟨missing code⟩}`

sets `\l_stex_notation_cs`

`\_stex_notation_check:`

`\_stex_notation_do_html:n`

`\_stex_notation_make_args:`

`\stex_do_default_notation:`
`\stex_do_default_notation_op:`

`\STEXInternalNotation`

#1 : notation id
#2 : operator precedence
#3 : intent (WiP)
#4 : arguments
#5 : notation code
#6 : continuation

`\argsep`

`\seqmap`

`\argmap`

`\argarraymap`

18.1 Automated Bracketing

`\infprec`
`\neginfprec`

infinite and negative infinite precedences.

`\_stex_maybe_brackets:nn`

`\_stex_maybe_brackets:nn {⟨prec⟩} {⟨body⟩}`

inserts parentheses around ⟨*body*⟩ iff precedences and context call for it.

`\dobrackets`

actually inserts parentheses around its argument.

`\withbrackets`

`\withbrackets {⟨left⟩} {⟨right⟩} {⟨body⟩}`

sets ⟨*left*⟩ and ⟨*right*⟩ as the parentheses to use in ⟨*body*⟩.

`\dowithbrackets`

`\dowithbrackets {⟨left⟩} {⟨right⟩} {⟨body⟩}`

combines `\withbrackets` and `\dobrackets`.

Chapter 19

Structures

`mathstructure` (*env.*)

`extstructure` (*env.*)

`\this`

`\stex_get_mathstructure:n` sets `\l_stex_get_structure_module_uri` or throws an error if no structure by the given name/id exists.

`\usestructure`

Chapter 20

Statements

`\_stex_do_for_list:` resolves each id in the `for={...}` comma-separated list of ids `\l_stex_key_for_clist`. Stores the full resolved URIs in `\l_stex_fors_seq`.

`\stex_new_statement:nnn` `\stex_new_statement:nnn {⟨env name⟩} {⟨macro name⟩} {⟨code⟩}`

defines a new statement environment `s⟨env name⟩` and the optional macro-variant `\inline⟨macro name⟩` (may be empty). `{⟨code⟩}` does arbitrary additional setup and is called in the `\begin` part of the environment and the macro after optional arguments have been parsed. e.g. `\stex_new_statement:nnn{definition}{def}{...}` defines the `sdefinition` environment and the `\inlinedef` macro.

`sdefinition` (*env.*) (and `\inlinedef`)

`sassertion` (*env.*) (and `\inlineass`)

`sexample` (*env.*) (and `\inlineex`)

`sparagraph` (*env.*)

`definiens`

`\_stex_add_definiens:nn` marks `#2` as the definiens of the symbol with uri `#1`. Also marks the symbol as being defined, iff the symbol is declared in the current module.

`\varbind`

`\conclusion`

`\premise`

Chapter 21

Proofs

`sproof` (*env.*)

`subproof` (*env.*)

`spfsketchenv` (*env.*)

`\spfsketch`

`spfstepenv` (*env.*)

`spfblock` (*env.*)

`\spfstep`

`\assumption`

`\conclude`

`\eqstep`

`\yield`

`\spfjust`

`\spfby`

\spfarg

\sproofend

\stexcommentfont

Chapter 22

Metatheory

TODO

Chapter 23

Others

Chapter 24

Additional Packages

24.1 NotesSlides Documentation

TODO

24.2 Problem Documentation

TODO

24.3 HWExam Documentation

TODO

24.4 Tikzinput Documentation

TODO

Part IV
Implementation

Chapter 25

Setting up

Setup code for the document class

```
⟨*cls⟩
%%%%%%%%%%%%%   stex.dtx   %%%%%%%%%%%%%

\RequirePackage{expl3,l3keys2e}
\ProvidesExplClass{stex}{2025/11/11}{4.0.0}{sTeX document class}
\IfFileExists{stex-expl-compat.sty}{
  \usepackage{stex-expl-compat}
}{}

\DeclareOption*{\PassOptionsToPackage{\CurrentOption}{stex}}
\ProcessOptions

\RequirePackage{stex}
\str_set:Nn \g_stex_document_kind_str{fragment}

\LoadClass{article}
⟨/cls⟩
```

Setup code for the package

```
⟨*package⟩
\RequirePackage{expl3,l3keys2e,ltxcmds}
\ProvidesExplPackage{stex}{2025/11/11}{4.0.0}{sTeX package}
\IfFileExists{stex-expl-compat.sty}{
  \usepackage{stex-expl-compat}
}{}
\RequirePackage{stex-logo} % externalized for backwards-compatibility reasons
\RequirePackage{standalone}

\bool_new:N \c_stex_use_sref_bool
\message{^^J*~This~is~sTeX~version~4.0.0~*^^J}
```

Package options:

```
\keys_define:nn { stex / package } {
  debug      .str_set_x:N  = \c_stex_debug_clist ,
  lang       .clist_set:N  = \c_stex_languages_clist ,
  mathhub    .tl_set_x:N   = \mathhub ,
  usesms     .bool_set:N   = \c_stex_persist_mode_bool ,
```

```
  writesms   .bool_set:N   = \c_stex_persist_write_mode_bool ,
  forcenousesms .bool_set:N   = \c_stex_persist_force_bool ,
  checkterms .bool_set:N   = \c_stex_check_terms_bool ,
  metadata   .bool_set:N   = \c_stex_metadata_bool ,
  image      .bool_set:N   = \c_tikzinput_image_bool,
  nofrontmatter .bool_set:N = \c_stex_no_frontmatter_bool,
  unknown    .code:n       = {}
}
\exp_args:NNo \clist_set:Nn \c_stex_debug_clist \c_stex_debug_clist
\ProcessKeysOptions { stex / package }
```

Error messages:

```
\input{stex-en.ldf}
```

Chapter 26

Utilities

\stex_ignore_spaces_and_pars:

```
\cs_new_protected:Nn \stex_ignore_spaces_and_pars:{
  \begingroup\catcode13=10\relax
  \@ifnextchar\par{
    \endgroup\expandafter\stex_ignore_spaces_and_pars:\@gobble
  }{
    \endgroup
  }
}
```

(*End of definition for* `\stex_ignore_spaces_and_pars:`. *This function is documented on page 123.*)

sfunction

`\stex_undefine:N`
`\stex_undefine:c`

```
\cs_new_protected:Nn \stex_undefine:N {
  \cs_set_eq:NN #1 \tex_undefined:D
}
\cs_generate_variant:Nn \stex_undefine:N {c}
```

(*End of definition for* `\stex_undefine:N`. *This function is documented on page 123.*)

sfunction

\stex_str_if_starts_with_p:nn
`\stex_str_if_starts_with:nnTF`

```
\prg_new_conditional:Nnn \stex_str_if_starts_with:nn {p,T,F,TF} {
  \exp_args:Ne \str_if_eq:nnTF {
    \str_range:nnn{#1}{1}{\str_count:n{#2}}
  }{#2}\prg_return_true: \prg_return_false:
}
```

(*End of definition for* `\stex_str_if_starts_with:nnTF`. *This function is documented on page 123.*)

sfunction

`\stex_str_if_ends_with_p:nn`
`\stex_str_if_ends_with:nnTF`

```
\prg_new_conditional:Nnn \stex_str_if_ends_with:nn {p,T,F,TF} {
  \exp_args:Ne \str_if_eq:nnTF {
    \str_range:nnn{#1}{- \str_count:n{#2}}{-1}
  }{#2}\prg_return_true: \prg_return_false:
}
```

(*End of definition for* `\stex_str_if_ends_with:nnTF`. *This function is documented on page 123.*)

sfunction

`\stex_deactivate_macro:Nn`
`\stex_reactivate_macro:N`

```
\cs_new_protected:Nn \stex_deactivate_macro:Nn {
  \tl_set_eq:cN{\tl_to_str:n{#1}~-~orig}#1
  \cs_set_protected:Npn#1{
    \msg_error:nnnn{stex}{error/deactivated-macro}{#1}{#2}
  }
}
\cs_new_protected:Nn \stex_reactivate_macro:N {
  \cs_set_eq:Nc #1{\tl_to_str:n{#1}~-~orig}
}
```

(*End of definition for* `\stex_deactivate_macro:Nn` *and* `\stex_reactivate_macro:N`. *These functions are documented on page 123.*)

sfunction

26.1 kpsewhich and Environment Variables

```
⟨@@=stex_kpse⟩
```

`\stex_kpsewhich:Nn`

```
\cs_new_protected:Nn \stex_kpsewhich:Nn {
  \group_begin:
    \catcode`\ =12
    \sys_get_shell:nnN { kpsewhich ~ #2 } { } \l_tmpa_tl
    \tl_gset_eq:NN \l_tmpa_tl \l_tmpa_tl
  \group_end:
  \exp_args:NNo\str_set:Nn #1 \l_tmpa_tl
  \tl_trim_spaces:N #1
}
```

(*End of definition for* `\stex_kpsewhich:Nn`. *This function is documented on page 123.*)

sfunction

`\stex_get_env:Nn`

```
\sys_if_platform_windows:TF{
  \cs_new_protected:Nn \stex_get_env:Nn {\group_begin:
    \escapechar=-1\catcode`\\=12
    \exp_args:NNe \stex_kpsewhich:Nn #1 {-expand-var~\c_percent_str#2\c_percent_str}
    \exp_args:NNx \str_if_eq:VnT #1 {\c_percent_str #2 \c_percent_str}{
      \str_clear:N #1
    }
    \exp_args:NNx\use:nn\group_end:{
      \str_set:Nn \exp_not:N #1 { #1 }
    }
  }
}{
  \cs_new_protected:Nn \stex_get_env:Nn {
    \stex_kpsewhich:Nn #1 {-var-value~#2}
  }
}
```

(*End of definition for* `\stex_get_env:Nn`. *This function is documented on page 124.*)

sfunction

26.2 Logging

```
⟨@@=stex_debug⟩
```

`\stex_debug:nn`

```
\cs_new_protected:Nn \stex_debug:nn {
  \exp_args:NNo \clist_if_in:NnTF \c_stex_debug_clist { \tl_to_str:n{all} }{
    \__stex_debug_:nn{#1}{#2}
  }{
    \exp_args:NNo \clist_if_in:NnT \c_stex_debug_clist { \tl_to_str:n{#1} }{
      \__stex_debug_:nn{#1}{#2}
    }
  }
}

\cs_new_protected:Nn \__stex_debug_:nn {
  \msg_set:nnn{stex}{debug / #1}{
    \\Debug~#1:~#2\\
  }
  \msg_none:nn{stex}{debug / #1}
}
```

We check an environment variable for debugging and set things up:

```
\stex_get_env:Nn\__stex_debug_env_str{STEX_DEBUG}
\str_if_empty:NTF\__stex_debug_env_str {
  \clist_set_eq:NN \l__stex_debug_cl \c_stex_debug_clist
}{
  \clist_set:No \l__stex_debug_cl {\__stex_debug_env_str}
}
\clist_clear:N \c_stex_debug_clist
\clist_map_inline:Nn \l__stex_debug_cl {
  \exp_args:NNo \clist_put_right:Nn \c_stex_debug_clist
  { \tl_to_str:n{#1} }
}

\exp_args:NNo \clist_if_in:NnTF \c_stex_debug_clist {\tl_to_str:n{all}} {
    \msg_redirect_module:nnn{ stex }{ none }{ warning }
    \stex_debug:nn{all}{Logging~everything!}
}{
  \clist_map_inline:Nn \c_stex_debug_clist {
    \msg_redirect_name:nnn{ stex }{ debug / #1 }{ warning }
    \stex_debug:nn{#1}{Logging~#1}
  }
}
```

(*End of definition for* `\stex_debug:nn`. *This function is documented on page 124.*)

sfunction

26.3 File Paths

```
⟨@@=stex_path⟩
```

\stex_file_set:Nn
\stex_file_set:No
\stex_file_set:Ne

```
\cs_new_protected:Nn \stex_file_set:Nn {
  \str_if_empty:nTF {#2} { \seq_clear:N #1 }{
    \exp_args:NNno \seq_set_split:Nnn #1 / { \tl_to_str:n{#2} }
  }
}
\cs_generate_variant:Nn \stex_file_set:Nn {No, Ne}
```

(*End of definition for* `\stex_file_set:Nn`. *This function is documented on page 124.*)

sfunction

\stex_file_resolve:Nn
\stex_file_resolve:No
\stex_file_resolve:Ne

```
\sys_if_platform_windows:TF{
  \cs_new_protected:Npn \__stex_path_win_take:w #1#2#3 \__stex_path_: {
    \uppercase{ \str_set:Nn \l__stex_path_str{#1}}
    \str_set:Ne \l__stex_path_win_drive {\l__stex_path_str #2}
    \str_set:Nn \l__stex_path_str{#3}
  }
  \cs_new_protected:Nn \stex_file_resolve:Nn {
    \str_set:Nn \l__stex_path_str {#2}
    \str_clear:N \l__stex_path_win_drive
    \exp_args:NNo \str_replace_all:Nnn \l__stex_path_str \c_backslash_str /
    \exp_args:Ne \str_if_eq:nnT {\str_item:Nn \l__stex_path_str 2} : {
      \exp_after:wN \__stex_path_win_take:w \l__stex_path_str \__stex_path_:
    }
    \stex_file_set:No #1 \l__stex_path_str
    \__stex_path_canonicalize:N #1
    \str_if_empty:NF \l__stex_path_win_drive {
      \seq_pop_left:NN #1 \l__stex_path_str
      \seq_put_left:No #1 \l__stex_path_win_drive
    }
    %\stex_debug:nn{files}{Set~\tl_to_str:n{#1}~to~\stex_file_use:N #1}
  }
}{
  \cs_new_protected:Nn \stex_file_resolve:Nn {
    \str_set:Nn \l__stex_path_str {#2}
    \stex_file_set:No #1 \l__stex_path_str
    \__stex_path_canonicalize:N #1
    % \stex_debug:nn{files}{Set~\tl_to_str:n{#1}~to~\stex_file_use:N #1}
  }
}
\cs_generate_variant:Nn \stex_file_resolve:Nn {No, Ne}
```

Auxiliary methods:

```
\cs_new_protected:Nn \__stex_path_canonicalize:N {
  \seq_if_empty:NF #1 {
    \seq_pop:NN #1 \l__stex_path_can_str
    \seq_clear:N \l__stex_path_can_seq
    \str_if_empty:NTF \l__stex_path_can_str {
      \seq_map_function:NN #1 \__stex_path_dodots:n
      \seq_put_left:Nn \l__stex_path_can_seq {}
```

```
    }{
      \seq_push:No #1 \l__stex_path_can_str
      \seq_map_function:NN #1 \__stex_path_dodots:n
    }
    \seq_set_eq:NN #1 \l__stex_path_can_seq
  }
}

\cs_new_protected:Nn \__stex_path_dodots:n {
  \str_if_empty:nF{#1}{
    \str_if_eq:nnF {#1} {.} {
      \str_if_eq:nnTF {#1} {..} {
        \seq_if_empty:NF \l__stex_path_can_seq {
          \seq_pop_right:NN \l__stex_path_can_seq \l__stex_path_can_str
        }
      }{
        \seq_put_right:Nn \l__stex_path_can_seq {#1}
      }
    }
  }
}
```

(*End of definition for* `\stex_file_resolve:Nn`*. This function is documented on page 124.*)

sfunction

`\stex_file_use:N`

```
\cs_new:Nn \stex_file_use:N {
  \seq_use:Nn #1 /
}
```

(*End of definition for* `\stex_file_use:N`*. This function is documented on page 124.*)

sfunction

`\stex_file_split_off_ext:NN`

```
\cs_new_protected:Nn \stex_file_split_off_ext:NN {
  \seq_set_eq:NN #1 #2
  \seq_pop_right:NN #1 \l__stex_path_str
  \seq_set_split:NnV \l__stex_path_seq . \l__stex_path_str
  \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
  \seq_put_right:Ne #1 {\seq_use:Nn \l__stex_path_seq .}
}
```

(*End of definition for* `\stex_file_split_off_ext:NN`*. This function is documented on page 124.*)

sfunction

`\stex_file_split_off_lang:NN`

```
\cs_new_protected:Nn \stex_file_split_off_lang:NN {
  \seq_set_eq:NN #1 #2
  \seq_pop_right:NN #1 \l__stex_path_str
  \seq_set_split:NnV \l__stex_path_seq . \l__stex_path_str
  \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str

  \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
  \exp_args:NNo \prop_if_in:NnF \c_stex_languages_prop \l__stex_path_str {
```

```
    \seq_put_right:No \l__stex_path_seq \l__stex_path_str
  }

  \seq_put_right:Ne #1 {\seq_use:Nn \l__stex_path_seq .}
}
```

(*End of definition for* `\stex_file_split_off_lang:NN`*. This function is documented on page 124.*)

sfunction

`\c_stex_pwd_file`
`\c_stex_main_file`

We determine the pwd

```
\sys_if_platform_windows:TF{
  \stex_get_env:Nn\l__stex_path_str{CD}
}{
  \stex_get_env:Nn\l__stex_path_str{PWD}
}
\stex_file_resolve:No \c_stex_pwd_file \l__stex_path_str
\seq_set_eq:NN \c_stex_main_file \c_stex_pwd_file
\seq_put_right:Ne \c_stex_main_file {\jobname\tl_to_str:n{.tex}}

\stex_debug:nn {files} {PWD:~\stex_file_use:N \c_stex_pwd_file}
```

(*End of definition for* `\c_stex_pwd_file` *and* `\c_stex_main_file`*. These variables are documented on page 124.*)

`\c_stex_home_file`

```
\sys_if_platform_windows:TF{
  \stex_get_env:Nn \l__stex_path_str {homedrive\c_percent_str\c_percent_str homepath}
}{
  \stex_get_env:Nn \l__stex_path_str {HOME}
}
\stex_file_resolve:No \c_stex_home_file \l__stex_path_str
```

(*End of definition for* `\c_stex_home_file`*. This variable is documented on page 125.*)

`\g_stex_current_file`

```
\seq_gset_eq:NN \g_stex_current_file \c_stex_main_file
```

(*End of definition for* `\g_stex_current_file`*. This variable is documented on page 125.*)

`\stex_input_with_hooks:Nn`
`\stex_input_with_hooks:Ne`

```
\cs_new_protected:Nn \stex_input_with_hooks:Nn {
  \stex_with_file_hooks:Nnn #1 {#2} {\input{#2}}
}
\cs_generate_variant:Nn \stex_input_with_hooks:Nn {Ne}
```

(*End of definition for* `\stex_input_with_hooks:Nn`*. This function is documented on page 125.*)

sfunction

`\stex_with_file_hooks:Nnn`

```
\cs_new_protected:Nn \stex_with_file_hooks:Nnn {
  \stex_pseudogroup:nn {
    \stex_file_resolve:Nn \l__stex_path_seq {#2}
    \seq_gset_eq:NN \g_stex_current_file \l__stex_path_seq
    \tl_set_eq:NN \l_stex_current_document_uri #1
    \str_set:Ne \l_stex_current_language_str { \stex_document_uri_language:N \l_stex_curren
```

```
    #3
  }{
    \tl_gset:Nn \exp_not:N \g_stex_current_file { \exp_args:No \exp_not:n \g_stex_current_f
    \stex_pseudogroup_restore:N \l_stex_current_language_str
    \stex_pseudogroup_restore:N \l_stex_current_document_uri
  }
}
```

(*End of definition for* `\stex_with_file_hooks:Nnn`. *This function is documented on page 125.*)

sfunction

`\stex_if_file_absolute_p:N`
`\stex_if_file_absolute:N`*TF*

```
\sys_if_platform_windows:TF {
  \prg_new_conditional:Nnn \stex_if_file_absolute:N {p, T, F, TF} {
    \seq_if_empty:NTF #1 \prg_return_false: {
      \exp_args:Ne \tl_if_empty:nTF {\seq_item:Nn #1 1} \prg_return_true: {
        \exp_args:Ne \str_if_eq:nnTF { \exp_args:Ne \str_item:nn {\seq_item:Nn #1 1} 2} :
          \prg_return_true: \prg_return_false:
      }
    }
  }
}{
  \prg_new_conditional:Nnn \stex_if_file_absolute:N {p, T, F, TF} {
    \seq_if_empty:NTF #1 \prg_return_false: {
      \exp_args:Ne \tl_if_empty:nTF {\seq_item:Nn #1 1}
        \prg_return_true: \prg_return_false:
    }
  }
}
```

(*End of definition for* `\stex_if_file_absolute:NTF`. *This function is documented on page 125.*)

sfunction

`stex_if_file_starts_with:NN`*TF*

```
\prg_new_protected_conditional:Nnn \stex_if_file_starts_with:NN {T,F,TF} {
  \seq_set_eq:NN \l__stex_path_a_seq #1
  \seq_set_eq:NN \l__stex_path_b_seq #2
  \tl_clear:N \l__stex_path_return_tl
  \bool_while_do:nn{
    \bool_not_p:n{
      \bool_lazy_any_p:n{
        {\seq_if_empty_p:N \l__stex_path_a_seq}
        {\seq_if_empty_p:N \l__stex_path_b_seq}
        {\bool_not_p:n{\tl_if_empty_p:N \l__stex_path_return_tl}}
      }
    }
  }{
    \seq_pop_left:NN \l__stex_path_a_seq \l__stex_path_a_tl
    \seq_pop_left:NN \l__stex_path_b_seq \l__stex_path_b_tl
    \str_if_eq:NNF \l__stex_path_a_tl \l__stex_path_b_tl {
      \tl_set:Nn \l__stex_path_return_tl {\prg_return_false:}
    }
  }
  \tl_if_empty:NTF \l__stex_path_return_tl {
```

```
    \seq_if_empty:NTF \l__stex_path_b_seq \prg_return_true: \prg_return_false:
  } \l__stex_path_return_tl
}
```

(*End of definition for* `\stex_if_file_starts_with:NNTF`. *This function is documented on page 125.*)

sfunction

26.4 Group-like Behaviours

```
⟨@@=stex_groups⟩
```

`\stex_pseudogroup:nn`

```
\cs_new_protected:Npn \stex_pseudogroup:nn {
  \exp_args:Nne \use:nn
}
```

(*End of definition for* `\stex_pseudogroup:nn`. *This function is documented on page 125.*)

sfunction

`\stex_pseudogroup_restore:N`

```
\cs_new:Nn \stex_pseudogroup_restore:N {
  \tl_if_exist:NTF #1 {
    \tl_set:Nn \exp_not:N #1 { \exp_args:No \exp_not:n #1 }
  }{
    \stex_undefine:N \exp_not:N #1
  }
}
```

(*End of definition for* `\stex_pseudogroup_restore:N`. *This function is documented on page 126.*)

sfunction

`\stex_pseudogroup_with:nn`

```
\cs_new_protected:Nn \stex_pseudogroup_with:nn {
  \tl_map_inline:nn{#1}{
    \cs_set_eq:cN{__stex_groups_\tl_to_str:n{##1}}##1
  }
  #2
  \tl_map_inline:nn{#1}{
    \cs_set_eq:Nc##1{__stex_groups_\tl_to_str:n{##1}}
    \stex_undefine:c{__stex_groups_\tl_to_str:n{##1}}
  }
}
```

(*End of definition for* `\stex_pseudogroup_with:nn`. *This function is documented on page 126.*)

sfunction

`\stex_metagroup_new:` Current metagroup group level

```
\int_new:N \l__stex_groups_lvl_int
```

start a new metagroup at the current group level

```
\cs_new_protected:Nn \stex_metagroup_new: {
  \int_set_eq:NN \l__stex_groups_lvl_int \currentgrouplevel
}
```

(*End of definition for* `\stex_metagroup_new:`. *This function is documented on page 126.*)

sfunction

`\stex_metagroup_do_in:n`
`\stex_metagroup_do_in:e`

```
\cs_new_protected:Npn \stex_metagroup_do_in:n {
  \int_compare:nNnTF \l__stex_groups_lvl_int = \currentgrouplevel
    \use:n \__stex_groups_do_in:n
}
\cs_generate_variant:Nn \stex_metagroup_do_in:n {e}

\cs_new_protected:Nn \__stex_groups_do_in:n {
  \tl_if_exist:cTF{g__stex_groups_\the\currentgrouplevel _tl}{
    \exp_args:Nno \tl_gput_right:cn{g__stex_groups_\the\currentgrouplevel _tl}
  }{
    \exp_args:Nno \tl_gset:cn{g__stex_groups_\the\currentgrouplevel _tl}
  }{ #1 }
  \bool_if_exist:cF {l__stex_groups_\the\currentgrouplevel _bool} {
      \group_insert_after:N \__stex_groups_do:
      \bool_set_true:c {l__stex_groups_\the\currentgrouplevel _bool}
  }
  #1
}

\cs_new_protected:Nn \__stex_groups_do: {
  \tl_if_exist:cT{g__stex_groups_\int_eval:n{\currentgrouplevel+1}_tl}{
    \exp_args:NNo \exp_args:No \stex_metagroup_do_in:n {
      \csname g__stex_groups_\int_eval:n{\currentgrouplevel+1}_tl \endcsname
    }
    \cs_undefine:c{g__stex_groups_\int_eval:n{\currentgrouplevel+1}_tl}
  }
  \bool_if_exist:cF {l__stex_groups_\the\currentgrouplevel _bool} {
    \group_insert_after:N \__stex_groups_do:
    \bool_set_true:c {l__stex_groups_\the\currentgrouplevel _bool}
  }
}
```

(*End of definition for* `\stex_metagroup_do_in:n`. *This function is documented on page 126.*)

sfunction

26.5 Key Handling

```
⟨@@=stex_keys⟩
```

`\stex_keys_define:nnnn`

```
\cs_new_nopar:Nn \stex_keys_define:nnnn {
  \tl_gset:cn {__stex_keys_keys_#1_pre_tl}{#2}
  \tl_gset:cn {__stex_keys_keys_#1_def_tl}{#3}
  \tl_if_empty:nF{#4}{
    \clist_map_inline:nn{#4}{
      \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_##1_pre_tl}
      \tl_gput_left:co{__stex_keys_keys_#1_pre_tl} \l__stex_keys_tl
      \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_##1_def_tl}
      \tl_gput_left:cn{__stex_keys_keys_#1_def_tl} ,
```

```
      \tl_gput_left:co{__stex_keys_keys_#1_def_tl} \l__stex_keys_tl
    }
  }
  \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_#1_def_tl}
  \exp_args:Nno \keys_define:nn {stex / #1} {\l__stex_keys_tl}
}
```

(*End of definition for* `\stex_keys_define:nnnn`*. This function is documented on page 126.*)

sfunction

`\stex_keys_set:nn`

```
\cs_new_nopar:Nn \stex_keys_set:nn {
  \use:c{__stex_keys_keys_#1_pre_tl}
  \keys_set:nn {stex / #1} { #2 }
}
```

(*End of definition for* `\stex_keys_set:nn`*. This function is documented on page 126.*)

sfunction

Some ubiquitous key sets:

```
\stex_keys_define:nnnn{archive file}{
  \str_clear:N \l_stex_key_archive_str
  \str_clear:N \l_stex_key_file_str
}{
  archive .str_set_x:N = \l_stex_key_archive_str ,
  file    .str_set_x:N = \l_stex_key_file_str
}{}

\stex_keys_define:nnnn{id}{
  \str_clear:N \l_stex_key_id_str
}{
  id .str_set_x:N = \l_stex_key_id_str
}{}

\stex_keys_define:nnnn{title}{
  \tl_clear:N \l_stex_key_title_tl
}{
  title .tl_set:N = \l_stex_key_title_tl
}{}

\stex_keys_define:nnnn{style}{
  \clist_clear:N \l_stex_key_style_clist
}{
  style .clist_set:N = \l_stex_key_style_clist
}{}

\stex_keys_define:nnnn{deprecate}{
  \str_clear:N \l_stex_key_deprecate_str
}{
  deprecate     .str_set_x:N  = \l_stex_key_deprecate_str
}{}

\stex_keys_define:nnnn{uses}{}{
  uses .code:n = {
    \clist_map_inline:nn{#1}{
```

```
      \stex_str_if_starts_with:nnTF{##1}[{
        \__stex_keys_split_at_bracket:w ##1 \_stex_end:
      }{
        \usemodule{##1}
      }
    }
  }
}{}

\cs_new_protected:Npn \__stex_keys_split_at_bracket:w [ #1 ] #2 \_stex_end: {
  \usemodule[#1]{#2}
}
```

26.6 Languages

```
⟨@@=stex_lang⟩
```

\c_stex_languages_prop
\c_stex_language_abbrevs_prop

```
\exp_args:NNx \prop_const_from_keyval:Nn \c_stex_languages_prop { \tl_to_str:n {
  en = english ,
  de = ngerman ,
  ar = arabic ,
  bg = bulgarian ,
  ru = russian ,
  fi = finnish ,
  ro = romanian ,
  tr = turkish ,
  fr = french ,
  sl = slovenian
}}

\exp_args:NNx \prop_const_from_keyval:Nn \c_stex_language_abbrevs_prop { \tl_to_str:n {
  english   = en ,
  ngerman   = de ,
  arabic    = ar ,
  bulgarian = bg ,
  russian   = ru ,
  finnish   = fi ,
  romanian  = ro ,
  turkish   = tr ,
  french    = fr ,
  slovenian = sl ,
}}
% todo: chinese simplified (zhs)
%       chinese traditional (zht)
```

(*End of definition for* `\c_stex_languages_prop` *and* `\c_stex_language_abbrevs_prop`*. These variables are documented on page 127.*)

\l_stex_current_language_str

```
\str_new:N \l_stex_current_language_str
```

(*End of definition for* `\l_stex_current_language_str`*. This variable is documented on page 127.*)

Loading babel (if necessary), depending on the `lang` package option:

```
\clist_if_empty:NF \c_stex_languages_clist {
  \bool_set_false:N \l__stex_lang_turkish_bool
  \seq_clear:N \l_tmpa_seq
  \clist_map_inline:Nn \c_stex_languages_clist {
    \str_if_empty:NF \l_stex_current_language_str {
      \str_set:Nn \l_stex_current_language_str {#1}
    }
    \str_set:Ne \l_tmpa_str {#1}
    \str_if_eq:nnT {#1}{tr}{
      \bool_set_true:N \l__stex_lang_turkish_bool
    }
    \prop_get:NoNTF \c_stex_languages_prop \l_tmpa_str \l_tmpa_str {
      \tl_set_rescan:Nno \l_tmpa_str {} \l_tmpa_str
      \seq_put_right:No \l_tmpa_seq \l_tmpa_str
    } {
      \msg_error:nnx{stex}{error/unknownlanguage}{\l_tmpa_str}
    }
  }
  \stex_debug:nn{lang} {Languages:~\seq_use:Nn \l_tmpa_seq {,~} }
  \bool_if:NTF \l__stex_lang_turkish_bool {
    \exp_args:NNe \use:nn \RequirePackage
      {[main=\seq_use:Nn \l_tmpa_seq, ,shorthands=:!]}{babel}
  }{
    \exp_args:NNe \use:nn \RequirePackage
      {[main=\seq_use:Nn \l_tmpa_seq, ]}{babel}
  }
}
```

26.7 Styling

```
⟨@@=stex_styles⟩
```

\stex_new_stylable_cmd:nnnn
\stex_style_apply:

```
\cs_new_protected:Nn \stex_new_stylable_cmd:nnnn {
  \exp_after:wN \newcommand \cs:w stexstyle#1 \cs_end:[2][]{
    \__stex_styles_patch:nnn{#1}{##1}{##2}
  }
  \exp_after:wN \NewDocumentCommand\cs:w #1\cs_end:{#2}{
    \cs_set:Npn \stex_style_apply: {
      \__stex_styles_apply_patch:n{#1}
    }
    #3
  }
  \tl_set:cn {__stex_styles_style_#1:} { #4 }
}

\cs_new_protected:Nn \__stex_styles_patch:nnn {
  \str_if_empty:nTF {#2}{
    \tl_set:cn{__stex_styles_style_#1:}{#3}
  }{
    \tl_set:cn{__stex_styles_style_#1_#2:}{#3}
  }
}
```

```
\cs_new_protected:Nn \__stex_styles_apply_patch:n {
  \clist_if_empty:NTF \l_stex_key_style_clist {
    \tl_clear:N \thisstyle
    \use:c{__stex_styles_style_#1:}
  }{
    \clist_get:NN \l_stex_key_style_clist \thisstyle
    \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle :}{
      \use:c{__stex_styles_style_#1_\thisstyle :}
    }{
      \use:c{__stex_styles_style_#1:}
    }
  }
}
```

(*End of definition for* `\stex_new_stylable_cmd:nnnn` *and* `\stex_style_apply:`*. These functions are documented on page 127.*)

sfunction

`\stex_new_stylable_env:nnnnnnn`

```
\cs_new_protected:Nn \stex_new_stylable_env:nnnnnnn {
  \exp_after:wN \newcommand \cs:w stexstyle#1 \cs_end:[3][]{
    \__stex_styles_patch:nnnn{#1}{##1}{##2}{##3} % <- defined after \stex_if_html_backend
  }
  \exp_after:wN \newcommand \cs:w stexcss#1 \cs_end:[1][]{
    \__stex_styles_css_patch:nnnn{#1}{##1}
  }
  \NewDocumentEnvironment{#7#1}{#2}{
    \cs_set:Npn \stex_style_apply: {
      \_stex_apply_patch_begin:n{#1}
    }
    #3
  }{
    \stex_if_html_backend:F{
      \cs_set:Npn \stex_style_apply: {
        \_stex_apply_patch_end:n{#1}
      }
    }
    #4
  }
  \tl_set:cn {__stex_styles_style_#1_start:} { #5 }
  \tl_set:cn {__stex_styles_style_#1_end:} { #6 }
}


\cs_new_protected:Nn \_stex_apply_patch_begin:n {
  \clist_if_empty:NTF \l_stex_key_style_clist {
    \tl_clear:N \thisstyle
    \use:c{__stex_styles_style_#1_start:}
  }{
    \clist_get:NN \l_stex_key_style_clist \thisstyle
    \stex_debug:nn{styling}{dominant~style:~\thisstyle}
    \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle _start:}{
      \use:c{__stex_styles_style_#1_\thisstyle _start:}
```

```
    }{
      \use:c{__stex_styles_style_#1_start:}
    }
  }
}

\cs_new_protected:Nn \_stex_apply_patch_end:n {
  \tl_if_empty:NTF \thisstyle {
    \use:c{__stex_styles_style_#1_end:}
  }{
    \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle _end:}{
      \use:c{__stex_styles_style_#1_\thisstyle _end:}
    }{
      \use:c{__stex_styles_style_#1_end:}
    }
  }
}
```

(*End of definition for* `\stex_new_stylable_env:nnnnnnn`. *This function is documented on page 127.*)

sfunction

26.8 Persisting Dependencies

```
⟨@@=stex_persist⟩
```

We check the environment variables:

```
\stex_get_env:Nn\__stex_persist_env_str{STEX_USESMS}
\str_if_empty:NF\__stex_persist_env_str{
  \exp_args:No \str_if_eq:nnF \__stex_persist_env_str{false}{
    \bool_set_true:N \c_stex_persist_mode_bool
  }
}
\stex_get_env:Nn\__stex_persist_env_str{STEX_WRITESMS}
\str_if_empty:NF\__stex_persist_env_str{
  \exp_args:No \str_if_eq:nnF \__stex_persist_env_str{false}{
    \bool_set_true:N \c_stex_persist_write_mode_bool
  }
}

\bool_if:NT \c_stex_persist_force_bool {
  \bool_set_false:N \c_stex_persist_mode_bool
}

\iow_new:N \c__stex_persist_sms_iow
```

`\stex_persist:n`
`\stex_persist:e`

defined later; requires `\stex_if_html_backend:TF`

(*End of definition for* `\stex_persist:n`. *This function is documented on page 127.*)

sfunction

Is called at the end of the `.sty`-file:

```
\cs_new_protected:Nn \_stex_persist_read_now: {
  \bool_if:NTF \c_stex_persist_mode_bool {
    \bool_if:NTF \c_stex_persist_write_mode_bool
      \__stex_persist_read_and_write:
```

```
      {
        \__stex_persist_load_file:n{\jobname.sms}
      }
  }{
    \bool_if:NT \c_stex_persist_write_mode_bool \__stex_persist_write_only:
  }
}

\cs_new_protected:Nn \__stex_persist_load_file:n {
  \file_if_exist:nT{#1}{
    \group_begin:
    \cs_set:Npn \stex_persist:n ##1 {}
    \cs_set:Npn \stex_persist:e ##1 {}
      \stex_debug:nn{persist}{restoring~from~sms~file}
      \catcode`\ =10\relax
      \cs:w @ @ input \cs_end:#1\relax
    \group_end:
  }
}

\cs_new_protected:Nn \__stex_persist_write_only: {
  \iow_open:Nn \c__stex_persist_sms_iow {\jobname.sms}
  \AtEndDocument{ \iow_close:N \c__stex_persist_sms_iow }
}

\cs_new_protected:Nn \__stex_persist_read_and_write: {
  \file_if_exist:nTF{\jobname.sms}{
    \ior_open:Nn \g_tmpa_ior {\jobname.sms}
    \iow_open:Nn \g_tmpa_iow {\jobname.sms2}
    \ior_str_map_inline:Nn \g_tmpa_ior {
      \iow_now:Nn \g_tmpa_iow {##1}
    }
    \iow_close:N \g_tmpa_iow
    \ior_close:N \g_tmpa_ior
    \__stex_persist_write_only:
    \ior_open:Nn \g_tmpa_ior {\jobname.sms2}
    \ior_str_map_inline:Nn \g_tmpa_ior {
      \iow_now:Nn \c__stex_persist_sms_iow {##1}
    }
    \ior_close:N \g_tmpa_ior
    \__stex_persist_load_file:n{\jobname.sms2}
  }\__stex_persist_write_only:
}
```

26.9 Auxiliary Methods

```
⟨@@=stex_aux⟩
```

_stex_do_deprecation:n

```
\cs_new:Nn \_stex_do_deprecation:n {
  \str_if_empty:NF \l_stex_key_deprecate_str {
    \msg_warning:nnxx{stex}{warning/deprecated}{#1}{\l_stex_key_deprecate_str}
  }
}
```

(*End of definition for* `\_stex_do_deprecation:n`. *This function is documented on page* **??**.)

`\_stex_do_id:`

```
\cs_new_protected:Nn \_stex_do_id: {
  \stex_if_smsmode:F {
    \str_if_empty:NF \l_stex_key_id_str {
      \exp_args:No \stex_ref_new_doc_target:n \l_stex_key_id_str
    }
  }
}
```

(*End of definition for* `\_stex_do_id:`. *This function is documented on page* **??**.)

Chapter 27

HTML Output

```
⟨@@=stex_annotate⟩
```

\stex@backend We determine and \input the backend config file:

```
\ifcsname if@rustex\endcsname\else
  \expandafter\newif\csname if@rustex\endcsname
  \@rustexfalse
\fi

\stex_get_env:Nn\__stex_annotate_env_str{STEX_FORCE_PDF}
\exp_args:No \str_if_eq:nnTF \__stex_annotate_env_str {true} {
  \def\stex@backend{pdflatex}
}{
  \tl_if_exist:NF\stex@backend{
    \if@rustex
      \def\stex@backend{rustex}
    \else
      \cs_if_exist:NTF\HCode{
        \def\stex@backend{tex4ht}
      }{
        \def\stex@backend{pdflatex}
      }
    \fi
  }
}

\input{stex-backend-\stex@backend.cfg}
```

(*End of definition for* \stex@backend*. This variable is documented on page 128.*)

\stex_if_html_backend_p:
\stex_if_html_backend:*TF*

is provided by the backend config file.

(*End of definition for* \stex_if_html_backend:TF*. This function is documented on page 128.*)

sfunction

\ifstexhtml

```
\bool_new:N \l__stex_annotate_do_output_bool

\newif\ifstexhtml
\stex_if_html_backend:TF {
```

```
  \stexhtmltrue
  \bool_set_true:N \l__stex_annotate_do_output_bool
}{
  \stexhtmlfalse
  \bool_set_false:N \l__stex_annotate_do_output_bool
}
```

(*End of definition for* `\ifstexhtml`. *This function is documented on page 128.*)

sfunction

`\stex_if_do_html_p:`
`\stex_if_do_html:TF`

```
\prg_new_conditional:Nnn \stex_if_do_html: {p,T,F,TF} {
  \bool_if:NTF \l__stex_annotate_do_output_bool
    \prg_return_true: \prg_return_false:
}
```

(*End of definition for* `\stex_if_do_html:TF`. *This function is documented on page 128.*)

sfunction

`\stex_suppress_html:n`

```
\cs_new_protected:Nn \stex_suppress_html:n {
  \stex_pseudogroup:nn{
    \bool_set_false:N \l__stex_annotate_do_output_bool
    #1
  }{
    \stex_if_do_html:T {
      \bool_set_true:N \l__stex_annotate_do_output_bool
    }
  }
}
```

(*End of definition for* `\stex_suppress_html:n`. *This function is documented on page 128.*)

sfunction

`\stex_annotate:nn` is provided by the backend config file.

(*End of definition for* `\stex_annotate:nn`. *This function is documented on page 128.*)

sfunction

`stex_annotate_env` (*env.*) is provided by the backend config file.

`\stex_html_node:nnn` is provided by the backend config file.

(*End of definition for* `\stex_html_node:nnn`. *This function is documented on page 128.*)

sfunction

`stex_env_node` (*env.*) is provided by the backend config file.

`\stex_annotate_invisible:nn`
`\stex_annotate_invisible:n` is provided by the backend config file.

(*End of definition for* `\stex_annotate_invisible:nn` *and* `\stex_annotate_invisible:n`. *These functions are documented on page 129.*)

sfunction

\STEXinvisible

```
\cs_new_protected:Npn \STEXinvisible #1 {
  \stex_annotate_invisible:n { #1 }
}
```

(*End of definition for* `\STEXinvisible`*. This function is documented on page 129.*)

sfunction

\stex_css_link:n is provided by the backend config file.

(*End of definition for* `\stex_css_link:n`*. This function is documented on page 129.*)

sfunction

\stex_css_literal:n is provided by the backend config file.

(*End of definition for* `\stex_css_literal:n`*. This function is documented on page 129.*)

sfunction

_stex_annotate_force_break:n is provided by the backend config file.

(*End of definition for* `\_stex_annotate_force_break:n`*. This function is documented on page 129.*)

sfunction

\mmlintent
\mmlarg

```
\stex_if_html_backend:TF {
  \cs_new_protected:Npn \mmlintent #1 #2 {
    \stex_annotate:nn{mml:intent={#1}}{#2}
  }
  \cs_new_protected:Npn \mmlarg #1 #2 {
    \stex_annotate:nn{mml:arg={#1}}{#2}
  }
}{
  \cs_new_protected:Npn \mmlintent #1 #2 { #2 }
  \cs_new_protected:Npn \mmlarg #1 #2 { #2 }
}
```

(*End of definition for* `\mmlintent` *and* `\mmlarg`*. These functions are documented on page 129.*)

sfunction

We can now define \stex_persist:n:

```
⟨@@=stex_persist⟩
\bool_if:NTF \c_stex_persist_write_mode_bool {
  \stex_if_html_backend:TF{
    \cs_new:Npn \stex_persist:n #1 {}
    \cs_new:Npn \stex_persist:e #1 {}
  }{
    \cs_new_protected:Nn \stex_persist:n {
      \iow_now:Nn \c__stex_persist_sms_iow {#1}
    }
    \cs_generate_variant:Nn \stex_persist:n {e}
  }
}{
  \cs_new:Npn \stex_persist:n #1 {}
  \cs_new:Npn \stex_persist:e #1 {}
}
```

and style patching:

```
⟨@@=stex_styles⟩
\stex_if_html_backend:TF{
  \cs_new_protected:Nn \stex_style_apply: {}
  \cs_new_protected:Nn \__stex_styles_patch:nnnn {}
  \stex_keys_define:nnnn{ csspatch }{
    \str_clear:N \l_stex_keys_cls_id
    \str_clear:N \l_stex_keys_counter_id
    \str_clear:N \l_stex_keys_counter_parent
  }{
    counter    .str_set_x:N  = \l_stex_keys_counter_id,
    parent     .str_set_x:N  = \l_stex_keys_counter_parent,
    unknown    .code:n       = {
      \exp_args:NNo \str_set:Nn \l_stex_keys_cls_id {\l_keys_key_tl}
    }
  }{}
  \cs_new_protected:Npn \__stex_styles_css_patch:nnnn #1 #2 {
    \stex_keys_set:nn{ csspatch }{ #2 }
    \group_begin:
    \catcode`\ =12\relax
    \catcode`^^J=12\relax
    \__stex_styles_patch_i:nnn {#1}
  }

  \seq_new:N \g__stex_styles_counters_seq

  \cs_new:Nn \__stex_styles_patch_html_c: {
    \stex_html_literal:n{
      <span~data-ftml-counter="\l_stex_keys_counter_id"~style="display:none;"~
        data-ftml-counter-parent="\l_stex_keys_counter_parent"
      ></span>
    }
  }

  \cs_new:Nn \__stex_styles_patch_html: {
    \stex_html_literal:n{
      <span~data-ftml-style="\l_stex_keys_cls_id"~style="display:none;"~
        data-ftml-counter="\l_stex_keys_counter_id"
      ></span>
    }
  }

  \cs_new_protected:Nn \__stex_styles_patch_i:nnn {
    \str_if_empty:NTF \l_stex_keys_cls_id {
      \str_set:Nn \l_stex_keys_cls_id { #1 }
    }{
      \str_set:Ne \l_stex_keys_cls_id { #1-\l_stex_keys_cls_id }
    }
    \exp_args:No \str_case:nnF \l_stex_keys_counter_parent {
      {}{}
      {part}{\str_set:Nn \l_stex_keys_counter_parent { 0 }}
      {chapter}{\str_set:Nn \l_stex_keys_counter_parent { 1 }}
      {section}{\str_set:Nn \l_stex_keys_counter_parent { 2 }}
      {subsection}{\str_set:Nn \l_stex_keys_counter_parent { 3 }}
```

```
      {subsubsection}{\str_set:Nn \l_stex_keys_counter_parent { 4 }}
      {paragraph}{\str_set:Nn \l_stex_keys_counter_parent { 5 }}
      {subparagraph}{\str_set:Nn \l_stex_keys_counter_parent { 6 }}
    }{
      \msg_error:nnx{stex}{error/notasection}\l_stex_keys_counter_parent
    }
    \str_set:Ne \l__stex_styles_first_str { &~.ftml-title-paragraph~ { display:inline-block
    \str_if_empty:NF \l_stex_keys_counter_id {
      %\str_put_left:Ne \l__stex_styles_first_str { counter-increment:~ ftml-\l_stex_keys_c
      \exp_args:NNo \seq_if_in:NnF \g__stex_styles_counters_seq \l_stex_keys_counter_id {
        \seq_gpush:No \g__stex_styles_counters_seq \l_stex_keys_counter_id
        \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
          \__stex_styles_patch_html_c:
          % in body
        }{
          \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html_c:
          % in preamble
        }
      }
      \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
          \__stex_styles_patch_html:
          % in body
      }{
        \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html:
        % in preamble
      }
    }
    \exp_args:Ne \stex_css_literal:n { .ftml-\l_stex_keys_cls_id { \l__stex_styles_first_st

    % TODO export counter reset somehow

    \group_end:
  }
}{
  \cs_new_protected:Nn \__stex_styles_patch:nnnn {
    \str_if_empty:nTF {#2}{
      \tl_set:cn{__stex_styles_style_#1_start:}{#3}
      \tl_set:cn{__stex_styles_style_#1_end:}{#4}
    }{
      \tl_set:cn{__stex_styles_style_#1_#2_start:}{#3}
      \tl_set:cn{__stex_styles_style_#1_#2_end:}{#4}
    }
  }
  \cs_new_protected:Nn \__stex_styles_css_patch:nnnn {}
}
```

Chapter 28

Math Archives

```
⟨@@=stex_mathhub⟩
```

\c_stex_mathhub_files
\mathhub

```
\str_if_empty:NTF\mathhub{
  \stex_get_env:Nn \l__stex_mathhub_str {MATHHUB}
  \str_if_empty:NTF \l__stex_mathhub_str {
    \ior_open:NnTF \g_tmpa_ior{\stex_file_use:N \c_stex_home_file/.stex/mathhub.path}{
      \group_begin:
        \escapechar=-1\catcode`\\=12
        \ior_str_get:NN \g_tmpa_ior \l__stex_mathhub_str
        \str_gset_eq:NN \l__stex_mathhub_str \l__stex_mathhub_str
      \group_end:
      \ior_close:N \g_tmpa_ior
      \stex_debug:nn{mathhub}{MathHub~directory~determined~from~home~directory}
    }{
      \str_clear:N \l__stex_mathhub_str
    }
  }{
    \stex_debug:nn{mathhub}{MathHub~directory~determined~from~environment~variable}
  }
}{
  \str_set_eq:NN \l__stex_mathhub_str \mathhub
}

\str_if_empty:NTF \l__stex_mathhub_str {
  \msg_warning:nn{stex}{warning/nomathhub}
  \stex_file_set:Ne \l_tmpa_seq {\stex_file_use:N \c_stex_home_file \tl_to_str:n{/MathHub}}
  \seq_clear:N \c_stex_mathhub_files
  \seq_push:Ne \c_stex_mathhub_files {\stex_file_use:N \l_tmpa_seq}
}{
  \seq_clear:N \c_stex_mathhub_files
  \seq_set_split:NnV \l_tmpa_seq , \l__stex_mathhub_str
  \seq_map_inline:Nn \l_tmpa_seq {
    \stex_file_resolve:Nn \l__stex_mathhub_str {#1}
    \stex_if_file_absolute:NF \l__stex_mathhub_str {
      \stex_file_resolve:Ne \l__stex_mathhub_str {
        \stex_file_use:N \c_stex_pwd_file / #1
      }
```

```
    }
    \seq_put_right:Ne \c_stex_mathhub_files {
      \stex_file_use:N \l__stex_mathhub_str
    }
  }
}

\exp_args:NNe \str_set:Nn \mathhub {\seq_use:Nn \c_stex_mathhub_files ,}
\stex_debug:nn{mathhub}{MATHHUBs:~\mathhub}
```

(*End of definition for* `\c_stex_mathhub_files` *and* `\mathhub`*. These variables are documented on page 132.*)

`\stex_archive_id:N`

```
\cs_new:Npn \stex_archive_id:N {
  \exp_after:wN \use_i:nnn
}
```

(*End of definition for* `\stex_archive_id:N`*. This function is documented on page 132.*)

sfunction

`\stex_archive_base:N`
`\stex_archive_base:n`

```
\cs_new:Npn \stex_archive_base:N {
  \exp_after:wN \use_ii:nnn
}

\cs_new:Nn \stex_archive_base:n {
  \exp_args:NNe \use:nn \use_ii:nnn { \use:c {c_stex_mathhub_#1_archive} }
}
```

(*End of definition for* `\stex_archive_base:N` *and* `\stex_archive_base:n`*. These functions are documented on page 132.*)

sfunction

`\stex_archive_path:N`
`\stex_archive_path:n`

```
\cs_new:Npn \stex_archive_path:N {
  \exp_after:wN \use_iii:nnn
}

\cs_new:Nn \stex_archive_path:n {
  \exp_args:NNe \use:nn \use_iii:nnn { \use:c {c_stex_mathhub_#1_archive} }
}
```

(*End of definition for* `\stex_archive_path:N` *and* `\stex_archive_path:n`*. These functions are documented on page 132.*)

sfunction

`\stex_in_archive:nn`

```
\cs_new_protected:Nn \stex_in_archive:nn {
  \stex_pseudogroup:nn{
    \cs_set:Npn \l__stex_mathhub_cs ##1 {#2}
    \tl_if_empty:nTF{#1}{
      \tl_if_exist:NTF \l_stex_current_archive {
        \exp_args:Ne \l__stex_mathhub_cs {\stex_archive_id:N \l_stex_current_archive }
      }{
```

```
        \l__stex_mathhub_cs {}
      }
    }{
      \__stex_mathhub_set_current:n{#1}
      \l__stex_mathhub_cs {#1}
    }
  }{
    \stex_pseudogroup_restore:N \l_stex_current_archive
    \cs_set:Npn \exp_not:N \l__stex_mathhub_cs ##1 {
      \exp_args:No \exp_not:n {\l__stex_mathhub_cs {##1}}
    }
  }
}

\cs_set:Npn \l__stex_mathhub_cs #1 {}
```

Auxiliary macros for loading archives:

```
\cs_new_protected:Nn \__stex_mathhub_set_current:n {
  \__stex_mathhub_require:n { #1 }
  \stex_debug:nn{mathhub}{switching~to~archive~#1}
  \tl_set_eq:Nc \l_stex_current_archive {
    c_stex_mathhub_#1_archive
  }
}

\cs_new_protected:Nn \__stex_mathhub_require:n {
  \prop_if_exist:cF { c_stex_mathhub_#1_archive } {
    \seq_if_empty:NTF \c_stex_mathhub_files {
      \msg_fatal:nn{stex}{warning/nomathhub}
    }{
      \stex_debug:nn{mathhub}{Opening~archive:~#1}
      \__stex_mathhub_do_manifest:nn { #1 }{
        \msg_fatal:nnee{stex}{error/noarchive}
        {#1}{\seq_use:Nn \c_stex_mathhub_files ,}
      }
    }
  }
}
```

Attempts to find the archive's manifest file:

```
\cs_new_protected:Nn \__stex_mathhub_do_manifest:nn {
  \seq_map_inline:Nn \c_stex_mathhub_files {
    \__stex_mathhub_find_manifest:nn {##1}{#1}
    \str_if_empty:NF \l__stex_mathhub_manifest_str \seq_map_break:
  }
  %\exp_args:Ne \__stex_mathhub_find_manifest:n {\stex_file_use:N \c_stex_mathhub_file / #1
  \str_if_empty:NTF \l__stex_mathhub_manifest_str { #2 }{
    \__stex_mathhub_parse_manifest:n {#1}
  }
}

\cs_new_protected:Nn \__stex_mathhub_find_manifest:nn {
  \str_clear:N \l__stex_mathhub_manifest_str
  \seq_set_split:Nnn \l_tmpa_seq / { #1 }
  \seq_set_split:Nnn \l__stex_mathhub_seq / {#1 / #2}
```

```
  \bool_set_true:N \l__stex_mathhub_bool
  \bool_while_do:Nn \l__stex_mathhub_bool {
    \tl_if_eq:NNTF \l__stex_mathhub_seq \l_tmpa_seq {
      \bool_set_false:N \l__stex_mathhub_bool
    }{
      \__stex_mathhub_check_manifest:
      \bool_if:NT \l__stex_mathhub_bool {
        \seq_pop_right:NN \l__stex_mathhub_seq \l__stex_mathhub_tl
      }
    }
  }
}

\cs_new_protected:Nn \__stex_mathhub_check_manifest: {
  \__stex_mathhub_check_manifest:n {MANIFEST.MF}
  \bool_if:NT \l__stex_mathhub_bool {
    \__stex_mathhub_check_manifest:n {META-INF/MANIFEST.MF}
    \bool_if:NT \l__stex_mathhub_bool {
      \__stex_mathhub_check_manifest:n {meta-inf/MANIFEST.MF}
    }
  }
}

\cs_new_protected:Nn \__stex_mathhub_check_manifest:n {
  \stex_debug:nn{mathhub}{Checking~\stex_file_use:N \l__stex_mathhub_seq / #1}
  \file_if_exist:nT {\stex_file_use:N \l__stex_mathhub_seq / #1} {
    \bool_set_false:N \l__stex_mathhub_bool
    \str_set:Ne \l__stex_mathhub_manifest_str {\stex_file_use:N \l__stex_mathhub_seq / #1}
  }
}
```

Parses the archive's manifest file:

```
\ior_new:N \c__stex_mathhub_manifest_ior

\str_set:Nn \l_tmpa_str {https://}

\exp_after:wN \cs_new:Npn \exp_after:wN \__stex_mathhub_replace_https:w \l_tmpa_str #1 \__s
  http:// #1
}

\cs_new_protected:Nn \__stex_mathhub_parse_manifest:n {
  \ior_open:Nn \c__stex_mathhub_manifest_ior \l__stex_mathhub_manifest_str
  \str_set:Ne \l__stex_mathhub_file_str {\stex_file_use:N \l__stex_mathhub_seq}
  \str_set:Nn \l__stex_mathhub_id_str {#1}
  \str_set:Nn \l__stex_mathhub_base_str {http://mathhub.info}

  \ior_map_inline:Nn \c__stex_mathhub_manifest_ior {
    \exp_args:NNNo \exp_args:NNNx
      \seq_set_split:Nnn \l__stex_mathhub_seq \c_colon_str {\tl_to_str:n{##1}}
    \seq_pop_left:NNT \l__stex_mathhub_seq \l__stex_mathhub_key {
      \exp_args:NNo \str_set:Nn \l__stex_mathhub_key \l__stex_mathhub_key
      \str_set:Ne \l__stex_mathhub_val {\seq_use:Nn \l__stex_mathhub_seq :}
      \str_case:Nn \l__stex_mathhub_key {
        {id}              { \str_set_eq:NN \l__stex_mathhub_id_str \l__stex_mathhub_val }
        {url-base}        { \str_set_eq:NN \l__stex_mathhub_base_str \l__stex_mathhub_val }
```

```
      }
    }
  }
  \ior_close:N \c__stex_mathhub_manifest_ior
  \exp_args:No \stex_str_if_starts_with:nnT \l__stex_mathhub_base_str {https://} {
    \str_set:Ne \l__stex_mathhub_base_str { \exp_after:wN \__stex_mathhub_replace_https:w \
  }
  \tl_gset:ce { c_stex_mathhub_#1_archive } { {\l__stex_mathhub_id_str} {\l__stex_mathhub_b
  \stex_debug:nn{mathhub}{Result:~\use:c{c_stex_mathhub_#1_archive}}
  \str_if_eq:nnF {#1}{main}{
    \stex_persist:e {
      \__stex_mathhub_restore:nn{\l__stex_mathhub_id_str}{\l__stex_mathhub_base_str}
    }
  }
}
```

The `.sms` file persisting content should not store the file path of the archive, since that depends on the directory layout of the system that generated it. We therefore, when restoring archive macros, check if we find the archive on the local system, and if not, leave the directory path empty. This will throw an error iff some functionality at some point *requires* the archive to actually exist locally.

```
\cs_set_protected:Nn \__stex_mathhub_restore:nn {
  \__stex_mathhub_do_manifest:nn { #1 }{}
  \tl_if_exist:cF { c_stex_mathhub_#1_archive }{
    \tl_set:ce{ c_stex_mathhub_#1_archive }{
      {\tl_to_str:n{#1}}
      {\tl_to_str:n{#2}}
      {}
    }
  }
}

```

(*End of definition for* `\stex_in_archive:nn`. *This function is documented on page 132.*)

sfunction

`\c_stex_no_archive_str`
`\c_stex_no_archive`

```
\str_const:Nn \c_stex_no_archive_str { no/archive }

\tl_const:Ne \c_stex_no_archive_uri {
  {\c_stex_no_archive_str}
  {\tl_to_str:n{http://unknown.source}}
  {}
}

\tl_set_eq:cN {c_stex_mathhub_ no/archive _ archive}\c_stex_no_archive_uri
```

(*End of definition for* `\c_stex_no_archive_str` *and* `\c_stex_no_archive`. *These variables are documented on page 133.*)

`\c_stex_main_archive`
`\l_stex_current_archive`

```
\seq_map_inline:Nn \c_stex_mathhub_files {
  \seq_set_split:Nnn \l_tmpa_seq / {#1}
  \stex_if_file_starts_with:NNT \c_stex_pwd_file \l_tmpa_seq {
    \seq_set_eq:NN \l_tmpb_seq \c_stex_pwd_file
```

```
    \seq_map_inline:Nn \l_tmpa_seq { \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl }
    \exp_args:Nne \__stex_mathhub_find_manifest:nn {#1} { \stex_file_use:N \l_tmpb_seq }
    \str_if_empty:NTF \l__stex_mathhub_manifest_str {
      \stex_debug:nn{mathhub}{Not~currently~in~a~MathHub~archive}
    }{
      \__stex_mathhub_parse_manifest:n { main }
      \tl_set_eq:NN \c_stex_main_archive \c_stex_mathhub_main_archive
      \cs_undefine:N \c_stex_mathhub_main_archive
      \tl_set_eq:cN { c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _archive }
        \c_stex_main_archive
      \prop_set_eq:NN \l_stex_current_archive \c_stex_main_archive
      \stex_debug:nn{mathhub}{Current~archive:~
        \stex_archive_id:N \c_stex_main_archive
      }
      \stex_persist:e {
        \__stex_mathhub_restore:nn{\stex_archive_id:N \c_stex_main_archive}{\stex_archive_b
        \tl_gset_eq:Nc \exp_not:N \c_stex_main_archive {c_stex_mathhub_ \stex_archive_id:N
        \tl_gset_eq:NN \exp_not:N \l_stex_current_archive \exp_not:N \c_stex_main_archive
      }
      %\bool_if:NT \c_stex_persist_write_mode_bool {
      %  \tl_put_right:Ne \_stex_persist_read_now: {
      %    \stex_persist:n {{c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _archi
      %      \tl_gset_eq:cN{c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _manife
      %      \tl_gset_eq:NN \exp_not:N \l_stex_current_archive \exp_not:N\c_stex_main_archi
      %    }
      %  }
      %}
    }
    \seq_map_break:
  }
}
\tl_if_exist:NF \c_stex_main_archive {
  \stex_debug:nn{mathhub}{Not~currently~in~a~MathHub~directory}
}
```

(*End of definition for* `\c_stex_main_archive` *and* `\l_stex_current_archive`*. These variables are documented on page 133.*)

`\stex_source_path:n`
`\stex_source_path:nn`

```
\cs_new:Nn \stex_source_path:n {
  \tl_if_exist:NTF \l_stex_current_archive {
    \stex_archive_path:N \l_stex_current_archive / source \tl_if_empty:nF{#1}{/ #1}
  }{
    \stex_file_use:N \g_stex_current_file / .. \tl_if_empty:nF{#1}{/ #1}
  }
}
\cs_new:Nn \stex_source_path:nn {
  \tl_if_empty:nTF{#1}{
    \stex_source_path:n {#2}
  }{
    \tl_if_exist:cTF {c_stex_mathhub_ #1 _archive } {
      \stex_archive_path:n {#1} / source \tl_if_empty:nF{#2}{/ #2}
    }{
      \stex_file_use:N \g_stex_current_file / .. \tl_if_empty:nF{#2}{/ #2}
```

```
        }
    }
}
```

(*End of definition for* `\stex_source_path:n` *and* `\stex_source_path:nn`*. These functions are documented on page 133.*)

sfunction

Chapter 29

URIs

```
⟨@@=stex_uris⟩
```

\stex_use_archive_uri:n

```
\cs_new:Ne \stex_use_archive_uri:n {
  \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
}
```

(*End of definition for* \stex_use_archive_uri:n*. This function is documented on page 134.*)

sfunction

29.1 Document URIs

\stex_use_document_uri:N
\stex_use_document_uri:n

```
\cs_new:Nn \stex_use_document_uri:N {
  \exp_after:wN \__stex_uris_doc:nnnnn #1
}
\cs_new:Nn \stex_use_document_uri:n {
  \__stex_uris_doc:nnnnn #1
}
\cs_new:Ne \__stex_uris_doc:nnnnn {
  \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
  \exp_not:N \tl_if_empty:nF{#2}{
    \c_ampersand_str\tl_to_str:n{p=}#2
  }
  \c_ampersand_str\tl_to_str:n{d=}#3
  \c_ampersand_str\tl_to_str:n{l=}#4
  \exp_not:N \tl_if_empty:nF{#5}{
    \c_ampersand_str\tl_to_str:n{e=}#5
  }
}
```

(*End of definition for* \stex_use_document_uri:N *and* \stex_use_document_uri:n*. These functions are documented on page 134.*)

sfunction

\stex_document_uri_archive:N

```
\cs_new:Nn \stex_document_uri_archive:N {
  \exp_after:wN \use_i:nnnnn #1
```

```
}
```

(*End of definition for* `\stex_document_uri_archive:N`. *This function is documented on page 134.*)

sfunction

`\stex_document_uri_path:N`

```
\cs_new:Nn \stex_document_uri_path:N {
  \exp_after:wN \use_ii:nnnnn #1
}
```

(*End of definition for* `\stex_document_uri_path:N`. *This function is documented on page 135.*)

sfunction

`\stex_document_uri_name:N`

```
\cs_new:Nn \stex_document_uri_name:N {
  \exp_after:wN \use_iii:nnnnn #1
}
```

(*End of definition for* `\stex_document_uri_name:N`. *This function is documented on page 135.*)

sfunction

`\stex_document_uri_language:N`

```
\cs_new:Nn \stex_document_uri_language:N {
  \exp_after:wN \use_iv:nnnnn #1
}
```

(*End of definition for* `\stex_document_uri_language:N`. *This function is documented on page 135.*)

sfunction

`\stex_document_uri_element:N`

```
\cs_new:Nn \stex_document_uri_element:N {
  \exp_after:wN \use_v:nnnnn #1
}
```

(*End of definition for* `\stex_document_uri_element:N`. *This function is documented on page 135.*)

sfunction

`\stex_document_uri_with_language:Nn`

```
\cs_new:Npn \stex_document_uri_with_language:Nn {
  \exp_after:wN \__stex_uris_with_language:nnnnnn
}
\cs_new:Nn \__stex_uris_with_language:nnnnnn {
  {#1}{#2}{#3}{#6}{}
}
```

(*End of definition for* `\stex_document_uri_with_language:Nn`. *This function is documented on page 135.*)

sfunction

`\stex_new_document_element_uri:n`

```
\cs_new:Npn \stex_new_document_element_uri:n {
  \exp_after:wN \__stex_uris_with_elem:nnnnnn \l_stex_current_document_uri
}
\cs_new:Nn \__stex_uris_with_elem:nnnnnn {
  {#1}{#2}{#3}{#4}{ \tl_if_empty:nTF{#5}{#6}{#5/#6}}
}
```

(*End of definition for* `\stex_new_document_element_uri:n`. *This function is documented on page 135.*)

sfunction

\stex_document_uri_from_archive_file:Nn

```
\cs_new_protected:Nn \stex_document_uri_from_archive_file:Nn {
  \stex_file_set:Nn \l__stex_uris_file {#2}
  \stex_debug:nn{URI}{relative~path:~\stex_file_use:N \l__stex_uris_file}
  \__stex_uris_doc_from_archive_file:NN #1 \l__stex_uris_file
}

\cs_new_protected:Nn \__stex_uris_doc_from_archive_file:NN {
  \__stex_uris_split_file:N #2
  \prop_if_exist:NTF \l_stex_current_archive {
    \str_set:Ne \l__stex_uris_archive {
      \stex_archive_id:N \l_stex_current_archive
    }
  }{
    \str_set_eq:NN \l__stex_uris_archive \c_stex_no_archive_str
  }
  \tl_set:Ne #1 {
    { \l__stex_uris_archive }
    { \stex_file_use:N \l__stex_uris_path_seq }
    { \l__stex_uris_name_str }
    { \l__stex_uris_lang_str }
    {}
  }
}

\cs_new_protected:Nn \__stex_uris_split_file:N {
  \seq_set_eq:NN \l__stex_uris_path_seq #1
  \seq_pop_right:NN \l__stex_uris_path_seq \l__stex_uris_name_str
  \seq_set_split:NnV \l_tmpa_seq . \l__stex_uris_name_str
  \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_lang_str % .tex?
  \seq_if_empty:NTF \l_tmpa_seq {
    \str_set_eq:NN \l__stex_uris_name_str \l__stex_uris_lang_str
    \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
  }\__stex_uris_split_file_i:
}

\cs_new_protected:Nn \__stex_uris_split_file_i: {
  \cs_if_eq:NNTF \l__stex_uris_lang_str \q_no_value {
    \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
  }{
    \prop_if_in:NoF \c_stex_languages_prop \l__stex_uris_lang_str {
      \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_lang_str % actual language maybe?
      \cs_if_eq:NNTF \l__stex_uris_lang_str \q_no_value {
          \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
      }{
        \prop_if_in:NoF \c_stex_languages_prop \l__stex_uris_lang_str {
          \seq_put_right:No \l_tmpa_seq \l__stex_uris_lang_str
          \str_set:Nn \l__stex_uris_lang_str {en}
        }
      }
    }
```

```
  }
  \str_set:Ne \l__stex_uris_name_str {\seq_use:Nn \l_tmpa_seq .}
}

\cs_new_protected:Nn \__stex_uris_split_file_ii: {

}
```

(*End of definition for* `\stex_document_uri_from_archive_file:Nn`*. This function is documented on page 135.*)

sfunction

`\c_stex_main_document_uri`
`\l_stex_current_document_uri`

```
\tl_if_exist:NTF \l_stex_current_archive {
  \str_set:Ne \l_tmpa_str { \stex_archive_path:N \l_stex_current_archive }
  \seq_set_split:NnV \l_tmpa_seq / \l_tmpa_str
  \seq_set_eq:NN \l_tmpb_seq \c_stex_main_file
  \seq_map_inline:Nn \l_tmpa_seq {
    \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl
  }
  \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl
  \__stex_uris_doc_from_archive_file:NN \l_stex_current_document_uri \l_tmpb_seq
}{
  \__stex_uris_doc_from_archive_file:NN \l_stex_current_document_uri \c_stex_main_file
}

\tl_set_eq:NN \c_stex_main_document_uri \l_stex_current_document_uri
\str_set:Ne \l_stex_current_language_str { \stex_document_uri_language:N \l_stex_current_do

\stex_debug:nn{URIs}{Current~document:~ \stex_use_document_uri:N \c_stex_main_document_uri}
```

(*End of definition for* `\c_stex_main_document_uri` *and* `\l_stex_current_document_uri`*. These variables are documented on page 135.*)

29.2 Module URIs

`\stex_use_module_uri:N`
`\stex_use_module_uri:n`

```
\cs_new:Nn \stex_use_module_uri:N {
  \exp_after:wN \__stex_uris_module:nnn #1
}
\cs_new:Nn \stex_use_module_uri:n {
  \__stex_uris_module:nnn #1
}
\cs_new:Ne \__stex_uris_module:nnn {
  \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
  \exp_not:N \tl_if_empty:nF{#2}{
    \c_ampersand_str\tl_to_str:n{p=}#2
  }
  \c_ampersand_str\tl_to_str:n{m=}#3
}
```

(*End of definition for* `\stex_use_module_uri:N` *and* `\stex_use_module_uri:n`*. These functions are documented on page 135.*)

sfunction

\stex_module_uri_archive:N

```
\cs_new:Nn \stex_module_uri_archive:N {
  \exp_after:wN \use_i:nnn #1
}
```

(*End of definition for* `\stex_module_uri_archive:N`*. This function is documented on page 135.*)

sfunction

\stex_module_uri_path:N
\stex_module_uri_path:n

```
\cs_new:Nn \stex_module_uri_path:N {
  \exp_after:wN \use_ii:nnn #1
}
\cs_new:Nn \stex_module_uri_path:n {
  \use_ii:nnn #1
}
```

(*End of definition for* `\stex_module_uri_path:N` *and* `\stex_module_uri_path:n`*. These functions are documented on page 135.*)

sfunction

\stex_module_uri_name:N
\stex_module_uri_name:n

```
\cs_new:Nn \stex_module_uri_name:N {
  \exp_after:wN \use_iii:nnn #1
}

\cs_new:Nn \stex_module_uri_name:n {
  \use_iii:nnn #1
}
```

(*End of definition for* `\stex_module_uri_name:N` *and* `\stex_module_uri_name:n`*. These functions are documented on page 136.*)

sfunction

\stex_module_uri_split_name:NNN
\stex_module_uri_split_name:NNn

```
\cs_new_protected:Npn \stex_module_uri_split_name:NNN #1 {
  \exp_after:wN \__stex_uris_parent:NNnnn \exp_after:wN #1 \exp_after:wN
}

\cs_new_protected:Nn \stex_module_uri_split_name:NNn {
  \__stex_uris_parent:NNnnn #1 #2 #3
}

\cs_new_protected:Nn \__stex_uris_parent:NNnnn {
  \seq_set_split:Nnn #1 / {#5}
  \seq_pop_right:NN #1 #2
  \seq_if_empty:NTF #1 {
    \tl_set:Ne #1 { {#3} {#4} {#3}}
    \tl_clear:N #2
  }{
    \tl_set:Ne #1 { {#3} {#4} {\seq_use:Nn #1 /} }
  }
}
```

(*End of definition for* `\stex_module_uri_split_name:NNN` *and* `\stex_module_uri_split_name:NNn`*. These functions are documented on page 136.*)

sfunction

`\stex_module_uri_as_qm:n`

```
\cs_new:Nn \stex_module_uri_as_qm:n {
  \__stex_uris_as_qm:nnn #1
}
\cs_new:Nn \__stex_uris_as_qm:nnn {
  #1 / #2 ? #3
}
```

(*End of definition for* `\stex_module_uri_as_qm:n`*. This function is documented on page 136.*)

sfunction

`\stex_new_module_uri:n`

```
\cs_new:Npn \stex_new_module_uri:n {
  \stex_if_in_module:TF {
    \exp_after:wN \__stex_uris_new_nested_mod:nnnn \l_stex_current_module_uri
  }{
    \exp_after:wN \__stex_uris_new_mod:nnnnnn \l_stex_current_document_uri
  }
}

\cs_new:Nn \__stex_uris_new_mod:nnnnnn {
  {#1}
  \str_if_eq:nnTF{#3}{#6}{
    {#2}{#3}
  }{
    {#2 \tl_if_empty:nF{#2}/ #3}{ \tl_to_str:n{ #6 } }
  }
}
\cs_new:Nn \__stex_uris_new_nested_mod:nnnn {
  {#1}
  {#2}
  {#3 / \tl_to_str:n{#4}}
}
```

(*End of definition for* `\stex_new_module_uri:n`*. This function is documented on page 136.*)

sfunction

`\stex_uri_from_pair:Nnn`

```
\bool_new:N \l_stex_relative_import_bool
\cs_new_protected:Nn \stex_uri_from_pair:Nnn {
  \bool_set_false:N \l_stex_relative_import_bool
  \stex_debug:nn{uri}{resolving~<#2,#3>~in~\stex_use_document_uri:N \l_stex_current_documen
  \seq_set_split:Nne \l_tmpa_seq ? { \tl_to_str:n {#3} }
  \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_name_str
  \str_clear:N \l__stex_uris_path_str
  \seq_if_empty:NF \l_tmpa_seq {
    \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_path_str
  }
  \tl_if_empty:nTF { #2 }{
    \str_if_empty:NTF \l__stex_uris_path_str
```

```
      \__stex_uris_maybe_relative:N \__stex_uris_absolute:N
    #1
  }{
    \__stex_uris_pair_in_archive:Nn #1 { #2 }
  }
}
%\cs_generate_variant:Nn \stex_uri_from_pair:Nnn {Nee}

\cs_new_protected:Nn \__stex_uris_maybe_relative:N {
  \tl_set:Ne \l_tmpb_tl {
    \stex_document_uri_path:N \l_stex_current_document_uri
  }
  \tl_set:Ne \l_tmpa_tl {
    {
      \stex_document_uri_archive:N \l_stex_current_document_uri
    }
    {
      \l_tmpb_tl
      \exp_args:NNo \exp_args:Nne \str_if_eq:nnF \l__stex_uris_name_str {
        \stex_document_uri_name:N \l_stex_current_document_uri
      }{
        \str_if_empty:NF \l_tmpb_tl / \stex_document_uri_name:N \l_stex_current_document_ur
      }
    }
    { \l__stex_uris_name_str }
  }
  \stex_if_module_exists:NTF \l_tmpa_tl {
    \tl_set_eq:NN #1 \l_tmpa_tl
    \bool_set_true:N \l_stex_relative_import_bool
    \stex_debug:nn{uri}{returning~relative~\stex_use_module_uri:N #1}
  }{
    \__stex_uris_absolute:N #1
  }
}

\cs_new_protected:Nn \__stex_uris_absolute:N {
  \tl_if_exist:NTF \l_stex_current_archive {
    \tl_set:Ne #1 {
      { \stex_archive_id:N \l_stex_current_archive }
      { \l__stex_uris_path_str }
      { \l__stex_uris_name_str }
    }
    \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
  }{
    \__stex_uris_pair_no_archive:N #1
  }
}

\cs_new_protected:Nn \__stex_uris_pair_no_archive:N {
  \stex_file_resolve:Ne \l_tmpa_seq {
    \stex_file_use:N \g_stex_current_file / .. / \l__stex_uris_path_str
  }
  \tl_set:Ne #1 {
    { \c_stex_no_archive_str }
```

```
    { \stex_file_use:N \l_tmpa_seq }
    { \l__stex_uris_name_str }
  }
  \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
}

\cs_new_protected:Nn \__stex_uris_pair_in_archive:Nn {
 \tl_set:Ne #1 {
    { \tl_to_str:n{ #2 } }
    { \l__stex_uris_path_str }
    { \l__stex_uris_name_str }
 }
 \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
}
```

(*End of definition for* `\stex_uri_from_pair:Nnn`*. This function is documented on page 136.*)

sfunction

`\stex_uri_from_pair:Nn`

```
\cs_new_protected:Nn \stex_uri_from_pair:Nn {
  \peek_charcode:NTF [ {
    \__stex_uris_parse_i:Nw #1
  }{
    \__stex_uris_parse_ii:Nw #1
  } #2 \__stex_uris_end:
}

\cs_new_protected:Npn \__stex_uris_parse_i:Nw #1 [#2] #3 \__stex_uris_end: {
  \stex_uri_from_pair:Nnn #1 {#2} {#3}
}

\cs_new_protected:Npn \__stex_uris_parse_ii:Nw #1 #2 \__stex_uris_end: {
  \stex_uri_from_pair:Nnn #1 {} {#2}
}
```

(*End of definition for* `\stex_uri_from_pair:Nn`*. This function is documented on page 136.*)

sfunction

29.3 Symbol URIs

`\stex_use_symbol_uri:N`
`\stex_use_symbol_uri:n`

```
\cs_new:Nn \stex_use_symbol_uri:N {
  \exp_after:wN \__stex_uris_symbol:nnnn #1
}
\cs_new:Nn \stex_use_symbol_uri:n {
  \__stex_uris_symbol:nnnn #1
}
\cs_new:Ne \__stex_uris_symbol:nnnn {
  \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
  \exp_not:N \tl_if_empty:nF{#2}{
    \c_ampersand_str\tl_to_str:n{p=}#2
  }
  \c_ampersand_str\tl_to_str:n{m=}#3
```

```
  \c_ampersand_str\tl_to_str:n{s=}#4
}
```

(*End of definition for* `\stex_use_symbol_uri:N` *and* `\stex_use_symbol_uri:n`*. These functions are documented on page 136.*)

sfunction

`\stex_new_symbol_uri:n`

```
\cs_new:Nn \stex_new_symbol_uri:n {
  \l_stex_current_module_uri { \tl_to_str:n{ #1 } }
}
```

(*End of definition for* `\stex_new_symbol_uri:n`*. This function is documented on page 136.*)

sfunction

`\stex_new_symbol_uri:nn`

```
\cs_new:Nn \stex_new_symbol_uri:nn {
  #1 { #2 }
}
```

(*End of definition for* `\stex_new_symbol_uri:nn`*. This function is documented on page 136.*)

sfunction

`\stex_symbol_uri_archive:N`

```
\cs_new:Nn \stex_symbol_uri_archive:N {
  \exp_after:wN \use_i:nnnn #1
}
```

(*End of definition for* `\stex_symbol_uri_archive:N`*. This function is documented on page 136.*)

sfunction

`\stex_symbol_uri_path:N`

```
\cs_new:Nn \stex_symbol_uri_path:N {
  \exp_after:wN \use_ii:nnnn #1
}
```

(*End of definition for* `\stex_symbol_uri_path:N`*. This function is documented on page 136.*)

sfunction

`\stex_symbol_uri_module:N`
`\stex_symbol_uri_module:n`

```
\cs_new:Nn \stex_symbol_uri_module:N {
  \exp_after:wN \__stex_uris_first_three:nnnn #1
}
\cs_new:Nn \stex_symbol_uri_module:n {
  \__stex_uris_first_three:nnnn #1
}

\cs_new:Nn \__stex_uris_first_three:nnnn {
  {#1}{#2}{#3}
}
```

(*End of definition for* `\stex_symbol_uri_module:N` *and* `\stex_symbol_uri_module:n`*. These functions are documented on page 137.*)

sfunction

\stex_symbol_uri_name:N
\stex_symbol_uri_name:n

```
\cs_new:Nn \stex_symbol_uri_name:N {
  \exp_after:wN \use_iv:nnnn #1
}
\cs_new:Nn \stex_symbol_uri_name:n {
  \use_iv:nnnn #1
}
```

(*End of definition for* `\stex_symbol_uri_name:N` *and* `\stex_symbol_uri_name:n`*. These functions are documented on page 137.*)

sfunction

Chapter 30

Documents

```
⟨@@=stex_doc⟩
```

\STEXftmllink
\STEXlinkftml
\STEXsetlinkftml

```
\cs_new_protected:Npn \STEXftmllink {
  \exp_args:Ne\url{
    \stex_use_document_uri:N \l_stex_current_document_uri
  }
}

\cs_new_protected:Npn \STEXsetlinkftml #1 {
  \tl_gset:Nn \g__stex_doc_ftml_link_text_tl { #1 }
}

\tl_gset:Nn \g__stex_doc_ftml_link_text_tl{
  The~
  \stex_get_symbol:nF{FTML}{}
  \tl_if_empty:NTF \l_stex_get_symbol_uri {FTML}{\sn{FTML}}
  {}~ version ~ of ~ this ~ document ~ can ~ be ~ found ~ at\\
  \STEXftmllink
}

\NewDocumentCommand \STEXlinkftml { O{} } {
  \ifstexhtml\else
    \begin{center}
      \emph{
        \tl_if_empty:nTF{#1}\g__stex_doc_ftml_link_text_tl{#1}
      }
    \end{center}
  \fi
}
```

(*End of definition for* `\STEXftmllink`, `\STEXlinkftml`, *and* `\STEXsetlinkftml`. *These functions are documented on page 138.*)

sfunction

\stexdoctitle

Initial definition (before `\begin{document}`):

```
\tl_new:N \g__stex_doc_title_tl

\str_new:N \g_stex_document_kind_str
```

```
\tl_new:N \g_stex_document_kind_parameters_tl
\str_set:Nn \g_stex_document_kind_str{article}
\stex_if_html_backend:T{
  \AtBeginDocument{
    \exp_args:Nne\stex_html_node:nnn{span}{
      data-ftml-document-kind=\g_stex_document_kind_str
      \g_stex_document_kind_parameters_tl
    }{}
  }
}

\cs_new_protected:Npn \stexdoctitle #1 {
  \tl_gset:Nn \g__stex_doc_title_tl { #1 }
  \global\def\stexdoctitle##1{}
}
```

At begin document, we switch to:

```
\cs_new_protected:Nn \__stex_doc_set_title:n {
  \stex_if_smsmode:F{
    \gdef\stexdoctitle##1{}
    \stex_debug:nn{title}{Setting~title~to:~\tl_to_str:n{#1}}
    \tl_gset:Nn \g__stex_doc_title_tl { #1 }
    \__stex_doc_title_html:
  }
}

\cs_new_protected:Nn \__stex_doc_title_html: {
  \stex_if_do_html:T{
    \stex_annotate_invisible:nn{data-ftml-doctitle={}}{ \hbox{\g__stex_doc_title_tl} }
  }
}

\AtBeginDocument {
  \tl_if_empty:NTF \g__stex_doc_title_tl {
    \cs_set_eq:NN \stexdoctitle \__stex_doc_set_title:n
  }{
    \gdef\stexdoctitle#1{}
    \__stex_doc_title_html:
  }

  \cs_set_eq:NN \__stex_doc_maketitle: \maketitle
  \protected\gdef\maketitle{
    \tl_if_empty:NF \@title {
      \exp_args:No \stexdoctitle \@title
    }
    \__stex_doc_maketitle:
  }
}
```

(*End of definition for* `\stexdoctitle`*. This function is documented on page 138.*)

sfunction

30.1 Sections

\l_stex_current_section_level_int
\l_stex_current_section_level_str

```
\int_new:N \l_stex_current_section_level_int
\str_set:Nn \l_stex_current_section_level_str {document}
```

(*End of definition for* `\l_stex_current_section_level_int` *and* `\l_stex_current_section_level_str`. *These variables are documented on page 139.*)

\currentsectionlevel
\Currentsectionlevel

```
\cs_set_protected:Npn \currentsectionlevel {
  \stex_if_do_html:TF{
    \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
    \stex_annotate:nn{data-ftml-currentsectionlevel={},data-ftml-capitalize=false}{}
  }{
   \l_stex_current_section_level_str
  }
  \tl_if_exist:NT\xspace\xspace
}

\cs_set_protected:Npn \Currentsectionlevel {
  \stex_if_do_html:TF{
    \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
    \stex_annotate:nn{data-ftml-currentsectionlevel={},data-ftml-capitalize=true}{}
  }{
    \exp_after:wN \str_uppercase:n \l_stex_current_section_level_str
  }
  \tl_if_exist:NT\xspace\xspace
}
```

(*End of definition for* `\currentsectionlevel` *and* `\Currentsectionlevel`. *These functions are documented on page 139.*)

sfunction

sfragment (*env.*)

```
\stex_keys_define:nnnn{ sfragment }{
  \tl_clear:N \l_stex_key_short_tl
}{
  short   .tl_set:N   = \l_stex_key_short_tl
}{id}

\NewDocumentEnvironment{sfragment}{ O{} m}{
  \stex_keys_set:nn{sfragment}{#1}
  \__stex_doc_do_section:n{#2}
  \_stex_do_id: % TODO
}{
  \_sfragment_end:
}

\cs_new_protected:Npn \__stex_doc_do_section:n {
  \int_case:nnF \l_stex_current_section_level_int {
    {0}{\cs_if_exist:NTF \thepart {\_sfragment_do_level:nn{part}}{
      \int_incr:N \l_stex_current_section_level_int
      \__stex_doc_do_section:n
```

```
    }}
    {1}{\cs_if_exist:NTF \thechapter {\_sfragment_do_level:nn{chapter}}{
      \int_incr:N \l_stex_current_section_level_int
      \__stex_doc_do_section:n
    }}
    {2}{\_sfragment_do_level:nn{section}}
    {3}{\_sfragment_do_level:nn{subsection}}
    {4}{\_sfragment_do_level:nn{subsubsection}}
    {5}{\_sfragment_do_level:nn{paragraph}}
  }{\_sfragment_do_level:nn{subparagraph}}
}

\stex_if_html_backend:TF {
  \cs_new_protected:Nn \_sfragment_do_level:nn {
    \stexdoctitle{#2}
    \par
    \begin{stex_env_node}{section}{data-ftml-section={\int_use:N \l_stex_current_section_le
      \noindent\stex_html_node:nnn{h1}{data-ftml-title={}}{
        \_stex_annotate_force_break:n{#2}
      }\par
  }
  \cs_new_protected:Nn \_sfragment_end: {
    \end{stex_env_node}
  }
}{
  \cs_new_protected:Nn \_sfragment_do_level:nn {
    \stexdoctitle{#2}
    \tl_if_empty:NTF \l_stex_key_short_tl {
      \use:c{#1}
    }{
      \exp_args:Nne \use:nn{\use:c{#1}}{[\exp_args:No \exp_not:n \l_stex_key_short_tl]}
    }{#2}

    \int_compare:nNnF \l_stex_current_section_level_int = 6{
      \int_incr:N \l_stex_current_section_level_int
    }
    \tl_set:Nn\l_stex_current_section_level_str{#1}
  }
  \cs_new_protected:Nn \_sfragment_end: {}
}
```

`titlefragment` (*env.*)

```
\bool_new:N \l__stex_doc_titlefragment_bool

\NewDocumentEnvironment{titlefragment}{ O{} m}{
  \stex_keys_set:nn{sfragment}{#1}
  \cs_if_exist:NTF \maketitle {
    \bool_set_true:N \l__stex_doc_titlefragment_bool
    \title{#2}
    \tl_if_empty:NF \l_stex_key_short_tl {
      \cs_if_exist:NT \shorttitle {
        \exp_args:No \shorttitle \l_stex_key_short_tl
      }
    }
```

```
    \stexdoctitle{#2}
    \maketitle
  }{
    \bool_set_false:N \l__stex_doc_titlefragment_bool
    \__stex_doc_do_section:n{#2}
    \_stex_do_id:
  }
}{
    \bool_if:NF \l__stex_doc_titlefragment_bool \_sfragment_end:
}
```

blindfragment (*env.*)

```
\cs_new_protected:Nn \__stex_doc_skip_section_i: {
  \int_case:nn \l_stex_current_section_level_int {
    {0}{\cs_if_exist:NF \thepart {
      \int_incr:N \l_stex_current_section_level_int \__stex_doc_skip_section_i:
    }}
    {1}{\cs_if_exist:NF \thechapter {
      \int_incr:N \l_stex_current_section_level_int \__stex_doc_skip_section_i:
    }}
  }
  \int_compare:nNnF \l_stex_current_section_level_int = 6{
    \int_incr:N \l_stex_current_section_level_int
  }
}

\stex_if_html_backend:TF {
  \cs_new_protected:Nn \__stex_doc_skip_section: {
    \__stex_doc_skip_section_i:
    \begin{stex_annotate_env}{data-ftml-skipsection={\int_use:N \l_stex_current_section_lev
    \_stex_annotate_force_break:n{}
  }
}{
  \cs_set_eq:NN \__stex_doc_skip_section: \__stex_doc_skip_section_i:
}

\NewDocumentEnvironment{blindfragment}{}{
  \__stex_doc_skip_section:
}{
  \stex_if_html_backend:T{
    \stex_annotate_invisible:n{~}
    \end{stex_annotate_env}
  }
}
```

\skipfragment

```
\cs_new_protected:Npn \skipfragment {
  \int_case:nnF \l_stex_current_section_level_int {
    {0}{\cs_if_exist:NTF \thepart {\stepcounter{part}}{
      \int_incr:N \l_stex_current_section_level_int
      \skipfragment
    }}
    {1}{\cs_if_exist:NTF \thechapter {\stepcounter{chapter}}{
      \int_incr:N \l_stex_current_section_level_int
```

```
      \skipfragment
    }}
    {2}{\stepcounter{section}}
    {3}{\stepcounter{subsection}}
    {4}{\stepcounter{subsubsection}}
    {5}{\stepcounter{paragraph}}
  }{\stepcounter{subparagraph}}
}
```

(*End of definition for* `\skipfragment`. *This function is documented on page 139.*)

sfunction

`\setsectionlevel`

```
\cs_new_protected:Npn \setsectionlevel #1 {
  \str_case:nnF{#1}{
    {part}{\int_set:Nn \l_stex_current_section_level_int 0}
    {chapter}{\int_set:Nn \l_stex_current_section_level_int 1}
    {section}{\int_set:Nn \l_stex_current_section_level_int 2}
    {subsection}{\int_set:Nn \l_stex_current_section_level_int 3}
    {subsubsection}{\int_set:Nn \l_stex_current_section_level_int 4}
    {paragraph}{\int_set:Nn \l_stex_current_section_level_int 5}
  }{
    \int_set:Nn \l_stex_current_section_level_int 6
  }
  \stex_if_do_html:T{
    \cs_if_eq:NNTF\@onlypreamble\@notprerr{
      \stex_html_literal:n{<span~ data-ftml-sectionlevel="\int_use:N\l_stex_current_section
    }{
      \AtBeginDocument{
        \stex_html_literal:n{<span~ data-ftml-sectionlevel="\int_use:N\l_stex_current_secti
      }
    }
  }
}
```

In HTML mode, we make sure to record the top section level:

```
\stex_if_html_backend:T{
  \cs_new_protected:Nn \__stex_doc_check_topsect: {
    \int_case:nnF \l_stex_current_section_level_int {
      {0}{\cs_if_exist:NTF \thepart {
        \stex_annotate_invisible:nn{data-ftml-sectionlevel=0}{}
      }{
        \int_incr:N \l_stex_current_section_level_int
        \__stex_doc_check_topsect:
      }}
      {1}{\cs_if_exist:NTF \thechapter {
        \stex_annotate_invisible:nn{data-ftml-sectionlevel=1}{}
      }{
        \int_incr:N \l_stex_current_section_level_int
        \__stex_doc_check_topsect:
      }}
    }{
      \stex_annotate_invisible:nn{data-ftml-sectionlevel={\int_use:N\l_stex_current_section
    }
  }
```

```
  \AtBeginDocument{\__stex_doc_check_topsect:
    \cs_if_exist:NTF \thechapter {
      \int_case:nnT \l_stex_current_section_level_int{
        {0}{}
        {1}{}
      }{
        \stex_css_literal:n {
          .ftml-section {
            &~.ftml-title-section { &::before {
              content:~counter(ftml-chapter)~"."~counter(ftml-section)~"~";
            } }
          }
          .ftml-subsection {
            &~.ftml-title-subsection { &::before {
              content:~counter(ftml-chapter)~"."~counter(ftml-section)~"."~counter(ftml-sub
            } }
          }
          .ftml-subsubsection {
            &~.ftml-title-subsubsection { &::before {
              content:~counter(ftml-chapter)~"."~counter(ftml-section)~"."~counter(ftml-sub
            } }
          }
        }
      }
    }
  }
}
```

(*End of definition for* `\setsectionlevel`*. This function is documented on page 139.*)

sfunction

We make sure `\frontmatter` and `\backmatter` are always defined. TODO: this seems like a hack and probably shouldn't be here

```
\bool_if:NF \c_stex_no_frontmatter_bool {
  \cs_new_protected:Nn \__stex_doc_x_matter: {
    \cs_if_exist:NTF\frontmatter{
      \let\__stex_doc_orig_frontmatter\frontmatter
      \let\frontmatter\relax
    }{
      \tl_set:Nn\__stex_doc_orig_frontmatter{
        \clearpage
        %\@mainmatterfalse
        \pagenumbering{roman}
      }
    }
    \cs_if_exist:NTF\backmatter{
      \let\__stex_doc_orig_backmatter\backmatter
      \let\backmatter\relax
    }{
      \tl_set:Nn\__stex_doc_orig_backmatter{
        \clearpage
        %\@mainmatterfalse
        \pagenumbering{roman}
      }
    }
```

```
    \newenvironment{frontmatter}{
      \__stex_doc_orig_frontmatter
    }{
      \cs_if_exist:NTF\mainmatter{
        \mainmatter
      }{
        \clearpage
        %\@mainmattertrue
        \pagenumbering{arabic}
      }
    }
    \newenvironment{backmatter}{
      \__stex_doc_orig_backmatter
    }{
      \cs_if_exist:NTF\mainmatter{
        \mainmatter
      }{
        \clearpage
        %\@mainmattertrue
        \pagenumbering{arabic}
      }
    }
  }
  \AtBeginDocument \__stex_doc_x_matter:
}
```

30.2 Inter-Document References

```
⟨@@=stex_refs⟩
```

The .sref-file:

```
\iow_new:N \c__stex_refs_iow
\AtBeginDocument{ \iow_open:Nn \c__stex_refs_iow {\jobname.sref} }
\AtEndDocument{\iow_close:N \c__stex_refs_iow}
\str_set:Ne \c__stex_refs_iow_str { \stex_file_use:N \c_stex_pwd_file /  }
```

Keys:

```
\stex_keys_define:nnnn{sref / 1}{
  \tl_clear:N \l_stex_key_pre_tl
  \tl_clear:N \l_stex_key_post_tl
  \tl_clear:N \l_stex_key_fallback_tl
}{
  % TODO get rid of this
  fallback  .code:n = {},
  pre       .code:n = {},
  post      .code:n = {}
}{archive file}
\stex_keys_define:nnnn{sref / 2}{}{}{archive file, title}
```

Target declarations:

\sreflabel
\stex_ref_new_doc_target:n

```
\cs_new_protected:Nn \stex_ref_new_doc_target:n {
  \stex_if_smsmode:F{
    \tl_set:Ne \l__stex_refs_new_tl { \stex_new_document_element_uri:n { #1 } }
```

```
    \exp_args:Ne \label { \stex_use_document_uri:N \l__stex_refs_new_tl }
    \exp_args:Nne \STeXInternalNewSRefLabel{#1}{ \stex_use_document_uri:N \l__stex_refs_new
    \iow_now:Ne \@auxout {
      \STeXInternalNewSRefLabel{#1}{ \stex_use_document_uri:N \l__stex_refs_new_tl }
    }
    \iow_now:Ne \c__stex_refs_iow {
      \exp_not:N \STeXInternalSRefLabel
        { #1 }
        { \stex_use_document_uri:N \l__stex_refs_new_tl }
        { \exp_not:o \@currentHref}
        { \exp_not:o \@currentlabelname }
    }
  }
}

\NewDocumentCommand \sreflabel {m} { \stex_ref_new_doc_target:n{#1} }
```

(*End of definition for* `\sreflabel` *and* `\stex_ref_new_doc_target:n`*. These functions are documented on page 86.*)

sfunction

`\stex_ref_new_sym_target:n`

```
\cs_new_protected:Nn \stex_ref_new_sym_target:n {
  \stex_if_smsmode:F{
    \cs_if_exist:cTF{r@\tl_to_str:n{#1}}{
      \cs_if_exist:cT{__stex_sref_aux_sym_ \tl_to_str:n{#1}}{
        \cs_undefine:c{__stex_sref_aux_sym_ \tl_to_str:n{#1}}
        \exp_args:Ne\label { \tl_to_str:n{#1} }
      }
    }{ \exp_args:Ne\label { \tl_to_str:n{#1} } }
  }
}
```

(*End of definition for* `\stex_ref_new_sym_target:n`*. This function is documented on page* **??**.)

sfunction

Inserting References:

`\sref`

```
\NewDocumentCommand \sref { O{} m O{}}{
  \stex_keys_set:nn { sref / 1 }{ #1 }
  \tl_clear:N \l__stex_refs_tmp
  \str_if_empty:NTF \l_stex_key_file_str {
    \__stex_refs_local:n
  }{
    \__stex_refs_remote:nn{#3}
  }{#2}
}

\cs_new_protected:Nn \__stex_refs_local:n {
  \stex_debug:nn{sref}{Checking~#1}
  \tl_if_exist:cTF{ g_stex_sref_label_ #1 }{
    %% TODO INSERT FTML ANNOTATION
    \exp_args:Ne \__stex_refs_local_ref:n {\use:c{ g_stex_sref_label_ #1 }}
  }{
```

```
    \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
  }
}

\cs_new_protected:Nn \__stex_refs_local_ref:n {
  \cs_if_exist:cTF{autoref}{
    \l_stex_key_pre_tl \autoref{#1} \l_stex_key_post_tl
  }{
    \message{^^J^^J
      sTeX~Warning: \c_backslash_str sref ~ with ~ local ~ target ~ used ~ without ~ hyperr
    ^^J^^J}
    \l_stex_key_pre_tl \ref{#1} \l_stex_key_post_tl
  }
}

\cs_new_protected:Nn \__stex_refs_remote:nn {
  \str_set:Nn \l__stex_refs_key { #2 }
  \tl_set:Nn \l__stex_refs_in { #1 }
  \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
    \exp_args:NNo \stex_document_uri_from_archive_file:Nn \l__stex_refs_uri {\l_stex_key_fi
    \stex_debug:nn{sref}{Checking~#2~from~\stex_use_document_uri:N \l__stex_refs_uri}
    \tl_if_eq:NNTF \l__stex_refs_uri \c_stex_main_document_uri {
      \stex_debug:nn{sref}{From~current~document;~delegating~to~\string\autoref}
      \__stex_refs_local:n{ #2 }
    }{
      \str_set:Ne \l__stex_refs_uri { \stex_use_document_uri:N \l__stex_refs_uri }
      \str_set:Ne \l__stex_refs_file { \exp_args:No \stex_source_path:n {\l_stex_key_file_s
      \exp_args:No \IfFileExists \l__stex_refs_file {
        \__stex_refs_find:
      }{}
      \tl_if_empty:NTF \l__stex_refs_tmp {
        \stex_debug:nn{sref}{Not~found}
        \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
      }{
        \str_set_eq:NN \l__stex_refs_uri \l__stex_refs_tmp
        \stex_debug:nn{sref}{Found~ \l__stex_refs_uri}
        \__stex_refs_maybe_in:
      }
    }
  }
}

\cs_new_protected:Nn \__stex_refs_find: {
  \setbox0\vbox\bgroup
    \cs_set_eq:NN \STeXInternalSRefLabel \__stex_refs_check_i:nnnn
    \use:c{@ @ input}{\l__stex_refs_file}
    \exp_args:NNe \use:nn
  \egroup {
    \tl_set:Nn \exp_not:N \l__stex_refs_tmp { \l__stex_refs_tmp }
  }
}

\cs_new_protected:Nn \__stex_refs_check_i:nnnn {
  \exp_args:No \str_if_eq:nnT{ \l__stex_refs_key }{#1}{
```

```
    \exp_args:Nno \stex_str_if_starts_with:nnT{#2}{\l__stex_refs_uri}{
      \str_set:Nn \l__stex_refs_tmp {#2}
      \endinput
    }
  }
}

\cs_new_protected:Nn \__stex_refs_maybe_in: {
  \tl_if_empty:NTF\l__stex_refs_in {
    \cs_if_exist:cTF{r@\l__stex_refs_uri}{
      \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
    }{
      \tl_if_empty:NTF \l__stex_refs_default_file {
        \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
      }{
        \tl_set_eq:NN \l_stex_key_title_tl \l__stex_refs_default_title
        \str_set_eq:NN \l__stex_refs_file \l__stex_refs_default_file
        \str_if_eq:NNTF \l__stex_refs_default_file \c__stex_refs_iow_str {
          \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
        }{
          \stex_debug:nn{sref}{in~title=\exp_args:No\exp_not:n\l_stex_key_title_tl,file=\l_
          \__stex_refs_in:
        }
      }
    }
  }{
    \stex_debug:nn{sref}{in~\exp_args:No\exp_not:n\l__stex_refs_in}
    \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
      \str_set:Ne \l__stex_refs_file { \exp_args:No \stex_source_path:n {\l_stex_key_file_s
    }
    \exp_args:Nno \stex_keys_set:nn{ sref / 2 }{ \l__stex_refs_in }
    \__stex_refs_in:
  }
}

\cs_new_protected:Nn \__stex_refs_in: {
  \tl_clear:N \l__stex_refs_tmp
  \exp_args:No \IfFileExists \l__stex_refs_file {
      \__stex_refs_find_in:
  }{
    \stex_debug:nn{sref}{not~found:~\l__stex_refs_file}
  }
  \tl_if_empty:NTF\l__stex_refs_tmp{
    \stex_debug:nn{sref}{Not~found}
    \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
  }{
    \stex_debug:nn{sref}{found:~\meaning\l__stex_refs_tmp}
    \tl_set:Ne \l__stex_refs_tmp {
      \exp_after:wN \__stex_refs_in_text:nn \l__stex_refs_tmp
    }
    \cs_if_exist:NT \href {\exp_args:No \href \l__stex_refs_uri } \l__stex_refs_tmp
  }
}
```

```
\cs_new:Nn \__stex_refs_in_text:nn {
  \__stex_refs_in_text:w #1 \__stex_refs_stop:
  \tl_if_empty:nF{#2}{~(\exp_not:n{#2})}
  \tl_if_empty:NF \l_stex_key_title_tl {
    {}~in~\exp_args:No \exp_not:n \l_stex_key_title_tl
  }
}

\cs_new:Npn \__stex_refs_in_text:w #1.#2 \__stex_refs_stop: {
  \__stex_refs_uppercase:n #1~#2~
}

\cs_new:Nn \__stex_refs_uppercase:n { \uppercase{#1}}

\cs_new_protected:Nn \__stex_refs_find_in: {
  \stex_debug:nn{sref}{finding~\l__stex_refs_uri;~in~\l__stex_refs_file...?}
  \setbox0\vbox\bgroup
    \catcode`^^J=9\relax
    \cs_set_eq:NN \STeXInternalSRefLabel \__stex_refs_check_in:nnnn
    \use:c{@ @ input}{\l__stex_refs_file}
    \exp_args:NNe \use:nn
  \egroup {
    \tl_set:Nn \exp_not:N \l__stex_refs_tmp { \exp_args:No \exp_not:n \l__stex_refs_tmp }
  }
}

\cs_new_protected:Nn \__stex_refs_check_in:nnnn {
  \exp_args:No \str_if_eq:nnT{ \l__stex_refs_uri }{#2}{
    \tl_set:Nn \l__stex_refs_tmp { {#3}{#4} }
    \endinput
  }
}
```

(*End of definition for* \sref*. This function is documented on page 86.*)

sfunction

TODO

```
%\int_new:N \l__stex_refs_unnamed_counter_int




\cs_new:Npn \STeXInternalSRefLabel #1 #2 #3 #4 {}

\cs_new_protected:Npn \STeXInternalNewSRefLabel #1 #2 {
  \tl_gset:ce{ g_stex_sref_label_ #1 }{ \tl_to_str:n{ #2 } }
}

\tl_new:N \l__stex_refs_default_title
\str_new:N \l__stex_refs_default_file
\str_new:N \l__stex_refs_default_archive

\newcommand\srefsetin[3][]{
  \str_set:Ne \l__stex_refs_default_file { #2 }
  \tl_if_empty:nTF{#1}{
```

```
    \str_set:Ne \l__stex_refs_default_archive { \stex_archive_id:N \c_stex_main_archive }
  }{
    \str_set:Nn \l__stex_refs_default_archive { #1 }
  }
  \exp_args:No \stex_in_archive:nn \l__stex_refs_default_archive {
    \str_set:Ne \l__stex_refs_default_file { \exp_args:No \stex_source_path:n {\l_stex_key_
  }
  \str_set:Nn \l__stex_refs_default_title { #3 }
}
\NewDocumentCommand \extref { O{} m m}{???}

\cs_new_protected:Nn \stex_ref_new_symbol:n {}


\NewDocumentCommand \srefsym { m m }{
  \stex_get_symbol:n { #1 }
  \exp_args:Ne \srefsymuri { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }{ #2 }
}

\cs_new_protected:Npn \srefsymuri #1 {
  \cs_if_exist:cTF{r@\tl_to_str:n{#1}}{
    \cs_if_exist:NT \hyperlink { \hyperlink{#1} }
  }{
    \cs_if_exist:NT \href {\href{#1} }
  }
}
```

30.3 Inputting From MathHub Resources

```
⟨@@=stex_inputs⟩
```

\inputref

```
\NewDocumentCommand \inputref { O{} m}{
  \stex_in_archive:nn{#1}{
    \stex_document_uri_from_archive_file:Nn \l__stex_inputs_uri {#2}
    \__stex_inputs_ref:n{#2}
  }
}

\stex_if_html_backend:TF{
  \str_set:Nn \c__stex_inputs_ref_pre_str {<span~data-ftml-inputref="}
  \str_set:Nn \c__stex_inputs_ref_post_str {"~><!----->}
  \cs_new_protected:Nn \__stex_inputs_ref_i:n {
    \IfFileExists{\stex_source_path:n {#1}}{
      %\relax
      %\noindent\vbox{
        \exp_args:Ne\stex_html_literal:n{
          \c__stex_inputs_ref_pre_str
          \stex_use_document_uri:N\l__stex_inputs_uri
          \c__stex_inputs_ref_post_str
        }
      %}
    }{
```

```
      \stex_input_with_hooks:Ne\l__stex_inputs_uri {\stex_source_path:n {#1}}
    }
  }
  \cs_new_protected:Nn \__stex_inputs_ref:n {
    \ifnum\@listdepth>0
      \noindent\vbox{\vskip-26pt\relax\__stex_inputs_ref_i:n{#1}}\par
      %\begin{stex_env_node}{div}{style:foo=bar}\vskip-700pt\end{stex_env_node}
    \else
      \__stex_inputs_ref_i:n{#1}
    \fi
  }

}{
  \cs_new_protected:Nn \__stex_inputs_ref:n {
    \begingroup
      \inputreftrue
      \let \l_stex_metatheory_uri \c_stex_default_metatheory
      %\seq_clear:N \l_stex_all_modules_seq
      \stex_undefine:N \l_stex_current_module_uri
      \stex_debug:nn{inputref}{Inputrefing~document~\stex_use_document_uri:N \l__stex_input
      \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#1}}
    \endgroup\medskip
  }
}
```

(*End of definition for* `\inputref`*. This function is documented on page 140.*)

sfunction

`\mhinput`

```
\NewDocumentCommand \mhinput { O{} m}{
  \stex_in_archive:nn {#1} {
    \stex_document_uri_from_archive_file:Nn \l__stex_inputs_uri {#2}
    \ifinputref
      \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#2}}
    \else
      \inputreftrue
      \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#2}}
      \inputreffalse
    \fi
  }
}
```

(*End of definition for* `\mhinput`*. This function is documented on page 140.*)

sfunction

`\ifinputref`

```
\newif \ifinputref \inputreffalse
```

(*End of definition for* `\ifinputref`*. This function is documented on page 140.*)

sfunction

`\IfInputref`

```
\stex_if_html_backend:TF{
  \newcommand \IfInputref[2]{
```

```
    \stex_annotate:nn{data-ftml-ifinputref=true}{#1}
    \stex_annotate:nn{data-ftml-ifinputref=false}{#2}
  }
}{
  \newcommand \IfInputref[2]{
    \ifinputref #1 \else #2 \fi
  }
}
```

(*End of definition for* `\IfInputref`*. This function is documented on page 140.*)

sfunction

`\libinput`

```
\newcommand \libinput [2][] {
  \stex_in_archive:nn{#1}{
    \__stex_inputs_up_archive:nn{#2}{tex}
    \stex_debug:nn{lib}{Result:~\seq_use:Nn \l__stex_inputs_libinput_files_seq ,}
    \seq_if_empty:NTF \l__stex_inputs_libinput_files_seq {
      \msg_error:nnnn{stex}{error/nofile}{\libinput}{#2.tex}
    }{
      \seq_map_function:NN \l__stex_inputs_libinput_files_seq \input
    }
  }
}
```

(*End of definition for* `\libinput`*. This function is documented on page 140.*)

sfunction

`\libusepackage`

```
\newcommand\libusepackage[2][]{
  \__stex_inputs_up_archive:nn{#2}{sty}
  \int_compare:nNnTF {\seq_count:N \l__stex_inputs_libinput_files_seq} = 1 {
    \str_set:Ne \l__stex_inputs_tmp_str {\seq_item:Nn \l__stex_inputs_libinput_files_seq 1}
    \exp_args:Nne \use:n {\usepackage[#1]} {
      \str_range:Nnn\l__stex_inputs_tmp_str 1 {-5}
    }
  }{
    \msg_fatal:nnnn{stex}{error/nofile}{\libusepackage}{#2.sty}
  }
}
```

(*End of definition for* `\libusepackage`*. This function is documented on page 140.*)

sfunction

`\libusetikzlibrary`

```
\newcommand \libusetikzlibrary [2][] {
  \cs_if_exist:NF \usetikzlibrary {
    \msg_error:nnx{stex}{error/notikz}{\tl_to_str:n{\libusetikzlibrary}}
  }
  \tl_if_empty:nTF{#1}{
    \__stex_inputs_usetikzlibrary:n{#2}
  }{
    \stex_in_archive:nn{#1}{\__stex_inputs_usetikzlibrary:n{#2}}
  }
```

```
}

\cs_new_protected:Nn \__stex_inputs_usetikzlibrary:n{
  \__stex_inputs_up_archive:nn{tikzlibrary#1}{code.tex}
  \int_compare:nNnTF {\seq_count:N \l__stex_inputs_libinput_files_seq} = 1 {
    \exp_args:Nne \__stex_inputs_usetikzlibrary_i:nn{#1}{ \seq_item:Nn \l__stex_inputs_libi
  }{
    \msg_fatal:nnnn{stex}{error/nofile}{\libusetikzlibrary}{tikzlibrary#1.code.tex}
  }
}

\cs_new_protected:Nn \__stex_inputs_usetikzlibrary_i:nn {
  \pgfkeys@spdef\pgf@temp{#1}
  \expandafter\ifx\csname tikz@library@\pgf@temp @loaded\endcsname\relax%
  \expandafter\global\expandafter\let\csname tikz@library@\pgf@temp @loaded\endcsname=\pgfu
  \expandafter\edef\csname tikz@library@#1@atcode\endcsname{\the\catcode`\@}
  \expandafter\edef\csname tikz@library@#1@barcode\endcsname{\the\catcode`\|}
  \expandafter\edef\csname tikz@library@#1@dollarcode\endcsname{\the\catcode`\$}
  \catcode`\@=11
  \catcode`\|=12
  \catcode`\$=3
  \pgfutil@InputIfFileExists{#2}{}{}
  \catcode`\@=\csname tikz@library@#1@atcode\endcsname
  \catcode`\|=\csname tikz@library@#1@barcode\endcsname
  \catcode`\$=\csname tikz@library@#1@dollarcode\endcsname
}
```

(*End of definition for* `\libusetikzlibrary`. *This function is documented on page 140.*)

sfunction

`\addmhbibresource`

```
\newcommand \addmhbibresource [2][] {
  \stex_in_archive:nn{#1}{
    \__stex_inputs_up_archive:nn{#2}{bib}
    \stex_debug:nn{lib}{Result:~\seq_use:Nn \l__stex_inputs_libinput_files_seq ,}
    \seq_if_empty:NTF \l__stex_inputs_libinput_files_seq {
      \msg_error:nnnn{stex}{error/nofile}{\addmhbibresource}{#2.bib}
    }{
      \seq_map_function:NN \l__stex_inputs_libinput_files_seq \addbibresource
    }
  }
}
```

(*End of definition for* `\addmhbibresource`. *This function is documented on page 141.*)

sfunction

`\__stex_inputs_up_archive:nn` Iterates up the current archive's directory in search for `lib` files with name `#1.#2`

```
\cs_new_protected:Nn \__stex_inputs_up_archive:nn {
  \tl_if_exist:NF \l_stex_current_archive {
    \msg_error:nnn{stex}{error/notinarchive}\libinput
  }
  \seq_clear:N \l__stex_inputs_libinput_files_seq
  \seq_set_split:Nne \l__stex_inputs_path_seq / {\stex_archive_path:N \l_stex_current_archi
  \seq_set_split:Nne \l__stex_inputs_id_seq / {\stex_archive_id:N \l_stex_current_archive}
```

```
  \seq_map_inline:Nn \l__stex_inputs_id_seq {
    \seq_pop_right:NN \l__stex_inputs_path_seq \l_tmpa_tl
  }
  \stex_debug:nn{lib}{Checking~#1~in~\seq_use:Nn \l__stex_inputs_path_seq /}

  \bool_while_do:nn { ! \seq_if_empty_p:N \l__stex_inputs_id_seq }{
    \str_set:Ne \l__stex_inputs_path_str {\stex_file_use:N \l__stex_inputs_path_seq / meta-
    \stex_debug:nn{lib}{Checking~\l__stex_inputs_path_str}
    \IfFileExists{ \l__stex_inputs_path_str }{
      \exp_args:NNo \seq_if_in:NnF \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_
        \seq_put_right:No \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
      }
    }{}
    \seq_pop_left:NN \l__stex_inputs_id_seq \l__stex_inputs_path_str
    \seq_put_right:No \l__stex_inputs_path_seq \l__stex_inputs_path_str
  }

  \str_set:Ne \l__stex_inputs_path_str {\stex_file_use:N \l__stex_inputs_path_seq / lib / #
  \stex_debug:nn{lib}{Checking~\l__stex_inputs_path_str}
  \IfFileExists{ \l__stex_inputs_path_str }{
    \exp_args:NNo \seq_if_in:NnF \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_st
      \seq_put_right:No \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
    }
  }{}
}
```

(*End of definition for* `\__stex_inputs_up_archive:nn`.)

`\mhgraphics`
`\cmhgraphics`

```
\ltx@ifpackageloaded{graphicx}{\use:n}{\AtEndOfPackageFile{graphicx}}{
  \define@key{Gin}{archive}{
    \stex_in_archive:nn{#1}{}
    \str_set:Ne \Gin@mhrepos {
      \stex_source_path:nn{#1}{}
    }
  }
  \providecommand\mhgraphics[2][]{
    \str_set:Ne \Gin@mhrepos {
      \stex_source_path:n{}
    }
    \setkeys{Gin}{#1}
    \includegraphics[#1]{ \Gin@mhrepos / #2 }
  }
  \providecommand\cmhgraphics[2][]{\begin{center}\mhgraphics[#1]{#2}\end{center}}
}
```

(*End of definition for* `\mhgraphics` *and* `\cmhgraphics`. *These functions are documented on page 141.*)

sfunction

`\lstinputmhlisting`
`\clstinputmhlisting`

```
\ltx@ifpackageloaded{listings}{\use:n}{\AtEndOfPackageFile{listings}}{
  \define@key{lst}{archive}{
    \stex_in_archive:nn{#1}{}
    \str_set:Ne \lst@mhrepos{
```

```
      \stex_source_path:nn{#1}{}
    }
  }
  \newcommand\lstinputmhlisting[2][]{%
    \str_set:Ne \lst@mhrepos{
      \stex_source_path:n{}
    }
    \setkeys{lst}{#1}%
    \lstinputlisting[#1]{\lst@mhrepos / #2}}
  \newcommand\clstinputmhlisting[2][]{\begin{center}\lstinputmhlisting[#1]{#2}\end{center}}
}
```

(*End of definition for* `\lstinputmhlisting` *and* `\clstinputmhlisting`*. These functions are documented on page 141.*)

sfunction

`\mhtikzinput`
`\cmhtikzinput`

```
\ltx@ifpackageloaded{tikzinput}{\use:n}{\AtEndOfPackageFile{tikzinput}}{
  \define@key{Gin}{archive}{
    \stex_in_archive:nn{#1}{}
    \str_set:Ne \Gin@mhrepos{
      \stex_source_path:nn{#1}{}
    }
    \str_set:Ne \l__stex_inputs_gin_repo_str {#1}
  }
  \newcommand\mhtikzinput[2][]{%
    \str_clear:N \l__stex_inputs_gin_repo_str
    \str_set:Ne \Gin@mhrepos{
      \stex_source_path:n{}
    }
    \setkeys{Gin}{#1}%
    \exp_args:No \stex_in_archive:nn \l__stex_inputs_gin_repo_str {
      \tikzinput[#1]{\Gin@mhrepos / #2}
    }
  }
  \newcommand\cmhtikzinput[2][]{\begin{center}\mhtikzinput[#1]{#2}\end{center}}
}
```

(*End of definition for* `\mhtikzinput` *and* `\cmhtikzinput`*. These functions are documented on page 141.*)

sfunction

Chapter 31

Modules

```
⟨@@=stex_modules⟩
```

A module consists of four separate macros, storing its *symbols*, *incoming morphisms* (e.g. `\importmodules`), *notations*, and "initialization code", respectively:

```
\cs_new:Nn \__stex_modules_macro_short:nnn {
  #1 ? #2 ? #3
}
\cs_new:Nn \_stex_symbol_macro:N {
  c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _symbols_prop
}
\cs_new:Nn \_stex_symbol_macro:n {
  c_stex_module_ \__stex_modules_macro_short:nnn #1 _symbols_prop
}
\cs_new:Nn \_stex_morphisms_macro:N {
  c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _morphisms_prop
}
\cs_new:Nn \_stex_morphisms_macro:n {
  c_stex_module_ \__stex_modules_macro_short:nnn #1 _morphisms_prop
}
\cs_new:Nn \_stex_notations_macro:N {
  c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _notations_prop
}
\cs_new:Nn \_stex_notations_macro:n {
  c_stex_module_ \__stex_modules_macro_short:nnn #1 _notations_prop
}
\cs_new:Nn \_stex_module_code_macro:N {
  c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _code
}
\cs_new:Nn \_stex_module_code_macro:n {
  c_stex_module_ \__stex_modules_macro_short:nnn #1 _code
}
```

`\l_stex_current_module_uri` initially undefined.

(*End of definition for* `\l_stex_current_module_uri`. *This variable is documented on page 142.*)

`\l_stex_all_modules_seq`

```
\seq_new:N \l_stex_all_modules_seq
```

(*End of definition for* `\l_stex_all_modules_seq`. *This variable is documented on page 142.*)

smodule (*env.*)

```
\tl_new:N \l_stex_metatheory_uri

\stex_keys_define:nnnn{smodule}{
  \str_clear:N \l_stex_key_sig_str
}{
  meta          .code:n       = {
    \tl_if_empty:nTF {#1}{
      \tl_clear:N \l_stex_metatheory_uri
    }{
      \stex_uri_from_pair:Nn \l_stex_metatheory_uri { #1 }
    }
  },
  %ns            .code:n       = {
  %  \stex_uri_resolve:Ne \l_stex_current_ns_uri { #1 }
  %} ,
  %lang          .code:n       = {
  %  \stex_set_language:n { #1 }
  %} ,
  sig           .str_set_x:N  = \l_stex_key_sig_str ,
  %creators      .code:n       = {} , % todo ?
  %contributors  .code:n       = {} , % todo ?
}{id, title, style, deprecate}

\stex_if_html_backend:T {
  \cs_new_protected:Nn \__stex_modules_html_annots: {
    \exp_args:Ne \stex_annotate_env {
      data-ftml-module={\stex_module_uri_name:N \l_stex_current_module_uri}
      %data-ftml-language={ \l_stex_current_language_str},
      \str_if_empty:NF\l_stex_key_sig_str {, data-ftml-signature={\l_stex_key_sig_str}}
      \tl_if_empty:NF \l_stex_metatheory_uri {,
        data-ftml-metatheory={\stex_use_module_uri:N \l_stex_metatheory_uri}
      }
    }
    \stex_annotate_invisible:n{}
  }
}

\stex_new_stylable_env:nnnnnnn {module} {O{} m}{
  \stex_keys_set:nn { smodule }{ #1 }
  \tl_set_eq:NN \thistitle \l_stex_key_title_tl
  \tl_if_empty:NF \thistitle {
    \exp_args:No \stexdoctitle \thistitle
  }
  \stex_module_setup:n { #2 }

  \stex_if_do_html:T \__stex_modules_html_annots:
  \stex_if_smsmode:F {
    \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_current_module_uri }
    \str_set:Nn \thismodulename {#2}
    \stex_style_apply:
  }
  \stex_smsmode_do:
}{
```

```
  \stex_close_module:
  \stex_if_smsmode:F \stex_style_apply:
  \stex_if_do_html:T{ \endstex_annotate_env }
}{}{}{s}
```

\stex_module_setup:n

```
\cs_new_protected:Npn \stex_module_setup:n {
  \stex_if_in_module:TF \__stex_modules_nested:n \__stex_modules_top:n
}

\cs_new_protected:Nn \__stex_modules_top:n {
  \str_if_empty:NTF \l_stex_key_sig_str
    \_stex_module_setup_top_nosig:n \__stex_modules_top_sig:n {#1}
  \g_stex_every_module_tl
  \tl_if_empty:NF \l_stex_metatheory_uri {
    \stex_debug:nn{modules}{loading~metatheory~\stex_use_module_uri:N \l_stex_metatheory_ur
    \__stex_modules_load_meta:
  }
}
```

(*End of definition for* `\stex_module_setup:n`*. This function is documented on page 142.*)

sfunction

\stex_module_setup_top_nosig:n

```
\cs_new_protected:Nn \_stex_module_setup_top_nosig:n {
  \stex_metagroup_new:
  \tl_set:Ne \l__stex_modules_uri { \stex_new_module_uri:n {#1} }
  \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
  \exp_args:No \stex_if_module_exists:NTF \l__stex_modules_uri {
    \stex_debug:nn{modules}{(already exists)}
  }{
    \tl_gclear:c{ \_stex_module_code_macro:N \l__stex_modules_uri }
    \prop_gclear:c{ \_stex_morphisms_macro:N \l__stex_modules_uri }
    \prop_gclear:c{ \_stex_symbol_macro:N \l__stex_modules_uri }
    \prop_gclear:c{ \_stex_notations_macro:N \l__stex_modules_uri }
  }
  \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
  \seq_put_right:No \l_stex_all_modules_seq \l__stex_modules_uri
}
```

(*End of definition for* `\stex_module_setup_top_nosig:n`*. This function is documented on page 142.*)

sfunction

__stex_modules_load_meta: loads the metatheory:

```
\bool_new:N \l_stex_in_meta_bool

\cs_new_protected:Nn \__stex_modules_load_meta: {
  \exp_after:wN \stex_execute_in_module:n \exp_after:wN {
    \exp_after:wN \__stex_modules_load_meta_i:n \exp_after:wN { \l_stex_metatheory_uri }
  }
}

\cs_new_protected:Nn \__stex_modules_load_meta_i:n {
  \bool_if:NTF \l_stex_in_meta_bool {
```

```
    \stex_activate_module:n {#1}
  }{
    \bool_set_true:N \l_stex_in_meta_bool
    \stex_activate_module:n {#1}
    \bool_set_false:N \l_stex_in_meta_bool
  }
}
```

(*End of definition for* `\__stex_modules_load_meta:.`)

`\__stex_modules_top_sig:n` sets up a new module with a signature:

```
\cs_new_protected:Nn \__stex_modules_top_sig:n {
  \stex_debug:nn{modules}{Signature:\l_stex_key_sig_str}
  \stex_metagroup_new:
  \tl_set:Ne \l__stex_modules_uri { \stex_new_module_uri:n {#1} }
  \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
  \stex_if_module_exists:NTF \l__stex_modules_uri {
    \stex_debug:nn{modules}{(already exists)}
    \stex_activate_module:N \l__stex_modules_uri
  }{
    \stex_debug:nn{modules}{(needs loading)}
    \__stex_modules_load_sig:
  }
  \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
}

\cs_new_protected:Nn \__stex_modules_load_sig: {
  \stex_file_split_off_lang:NN \l__stex_modules_sigfile \g_stex_current_file
  \tl_set:Ne \l__stex_modules_sig {
    \exp_args:NNo \stex_document_uri_with_language:Nn
      \l_stex_current_document_uri \l_stex_key_sig_str
  }
  \stex_debug:nn{modules}{Loading~signature~\stex_use_document_uri:N \l__stex_modules_sig;~
  \exp_args:NNe \stex_file_in_smsmode:Nn \l__stex_modules_sig {
    \stex_file_use:N \l__stex_modules_sigfile . \l_stex_key_sig_str . tex
  }
  \stex_activate_module:N \l__stex_modules_uri
}
```

(*End of definition for* `\__stex_modules_top_sig:n.`)

`\__stex_modules_nested:n` sets up a new nested module:

```
\cs_new_protected:Nn \__stex_modules_nested:n {
  \stex_metagroup_new:
  \stex_debug:nn{module}{Nested~module~#1~in~\stex_use_module_uri:N \l_stex_current_module_
  \tl_set:Ne \l__stex_modules_uri { \stex_new_module_uri:n{#1} }
  \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
  \exp_args:No \stex_if_module_exists:NTF \l__stex_modules_uri {
    \stex_debug:nn{modules}{(already exists)}
  }{
    \tl_gclear:c{ \_stex_module_code_macro:N \l__stex_modules_uri }
    \prop_gclear:c{ \_stex_morphisms_macro:N \l__stex_modules_uri }
    \prop_gclear:c{ \_stex_symbol_macro:N \l__stex_modules_uri }
    \prop_gclear:c{ \_stex_notations_macro:N \l__stex_modules_uri }
  }
```

```
  \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
  \seq_put_left:No \l_stex_all_modules_seq \l__stex_modules_uri
}
```

(*End of definition for* `\__stex_modules_nested:n`.)

`\stex_close_module:`

```
\cs_new_protected:Nn \stex_close_module: {
  \bool_if:NT \c_stex_persist_write_mode_bool \__stex_modules_persist_module:
  \stex_debug:nn{module}{
    Closing~module~\stex_use_module_uri:N \l_stex_current_module_uri^^J
    Code:~\cs_meaning:c{ \_stex_module_code_macro:N \l_stex_current_module_uri }^^J
    Imports:~ \exp_after:wN \prop_to_keyval:N \cs:w \_stex_morphisms_macro:N \l_stex_curren
    Declarations:~ \exp_after:wN \prop_to_keyval:N \cs:w \_stex_symbol_macro:N \l_stex_curr
    Notations:~ \exp_after:wN \prop_to_keyval:N \cs:w \_stex_notations_macro:N \l_stex_curr
  }
}
```

Persist and restore modules in the `.sms`-file:

```
\cs_new_protected:Nn \__stex_modules_persist_module: {
  \stex_persist:e {
    \__stex_modules_restore_module:nnnn {\l_stex_current_module_uri}{
      \exp_after:wN \prop_to_keyval:N \cs:w
        \_stex_morphisms_macro:N \l_stex_current_module_uri
      \cs_end:
    }{
      \exp_after:wN \prop_to_keyval:N \cs:w
        \_stex_symbol_macro:N \l_stex_current_module_uri
      \cs_end:
    }{
      \exp_after:wN \exp_after:wN \exp_after:wN \exp_not:n
      \exp_after:wN \exp_after:wN \exp_after:wN
      { \cs:w \_stex_module_code_macro:N \l_stex_current_module_uri \cs_end: }
    }{}
    \prop_map_function:cN{\_stex_notations_macro:N \l_stex_current_module_uri}
      \__stex_modules_persist_nots_i:nn
    \exp_not:N \STEXRestoreNotsEnd {}
  }
}

\cs_new:Nn \__stex_modules_persist_nots_i:nn {
  \exp_not:n{#2}
}


\cs_new:Nn \__stex_modules_stringify_uri:nnn {
  {\tl_to_str:n{#1}}
  {\tl_to_str:n{#2}}
  {\tl_to_str:n{#3}}
}
\cs_new:Nn \__stex_modules_stringify_uri:nnnn {
  {\tl_to_str:n{#1}}
  {\tl_to_str:n{#2}}
  {\tl_to_str:n{#3}}
  {\tl_to_str:n{#4}}
```

```
}

\cs_new_protected:Nn \__stex_modules_restore_module:nnnn {
  \tl_set:Ne \l__stex_modules_uri { \__stex_modules_stringify_uri:nnn #1 }
  \stex_debug:nn{persist}{restoring~\stex_use_module_uri:N \l__stex_modules_uri}
  \prop_gset_from_keyval:cn{ \_stex_morphisms_macro:N \l__stex_modules_uri }{#2}
  \prop_gset_from_keyval:cn{\_stex_symbol_macro:N \l__stex_modules_uri}{#3}
  \prop_map_inline:cn{\_stex_symbol_macro:N \l__stex_modules_uri}{
    \stex_ref_new_symbol:n{#1?##1}
  }
  \def \l_tmpa_tl { #4 }
  \exp_args:Nno \tl_gset:cn{\_stex_module_code_macro:N \l__stex_modules_uri}\l_tmpa_tl
  \prop_gclear:c{\_stex_notations_macro:N \l__stex_modules_uri}
  \group_begin:
    \catcode`_=8\relax
    \catcode`:=12\relax
    \__stex_modules_restore_nots:n
}

\quark_new:N \STEXRestoreNotsEnd

\cs_new_protected:Nn \__stex_modules_restore_nots:n {
  \__stex_modules_restore_nots_i:n
}

\cs_new_protected:Nn \__stex_modules_restore_nots_i:n {
  \tl_if_eq:nnTF{#1}{\STEXRestoreNotsEnd}{
    \group_end:
  }{
    \__stex_modules_restore_nots_ii:nnnnn {#1}
  }
}

\cs_new_protected:Nn \__stex_modules_restore_nots_ii:nnnnn {
  \tl_set:Nn \l__stex_modules_tl {{#4}{#5}}
  % eliminate ## => #
  \exp_after:wN \def \exp_after:wN \l__stex_modules_tl \exp_after:wN {\l__stex_modules_tl}
  \exp_args:NNe\use:nn\prop_gput:cnn{
    {\_stex_notations_macro:N \l__stex_modules_uri}
    {\stex_use_symbol_uri:n{#1}\tl_to_str:n{?#2}}{
      {\__stex_modules_stringify_uri:nnnn #1}{\tl_to_str:n{#2}}{#3}
      \exp_args:No \exp_not:n \l__stex_modules_tl
    }
  }
  \__stex_modules_restore_nots_i:n
}
```

(*End of definition for* `\stex_close_module:`. *This function is documented on page 142.*)

sfunction

`\stex_every_module:n`

```
\tl_new:N \g_stex_every_module_tl
\cs_new_protected:Nn \stex_every_module:n {
  \tl_gput_right:Nn \g_stex_every_module_tl { #1 }
}
```

(*End of definition for* `\stex_every_module:n`. *This function is documented on page 142.*)

sfunction

`\stex_do_up_to_module:n`
`\stex_do_up_to_module:e`

```
\cs_new_protected:Nn \stex_do_up_to_module:n {
  \stex_metagroup_do_in:n {#1}
}
\cs_generate_variant:Nn \stex_do_up_to_module:n {e}
```

(*End of definition for* `\stex_do_up_to_module:n`. *This function is documented on page 142.*)

sfunction

`\stex_execute_in_module:n`
`\stex_execute_in_module:e`

```
\cs_new_protected:Nn \stex_execute_in_module:n { \stex_if_in_module:TF {
  \tl_gput_right:cn { \_stex_module_code_macro:N \l_stex_current_module_uri } { #1 }
  \stex_debug:nn{modules}{extending~activation~for~\stex_use_module_uri:N \l_stex_current_m
  \stex_do_up_to_module:n { #1 }
}{ #1 }}
\cs_generate_variant:Nn \stex_execute_in_module:n {e}
```

(*End of definition for* `\stex_execute_in_module:n`. *This function is documented on page 142.*)

sfunction

`\stex_if_in_module_p:`
`\stex_if_in_module:`*TF*

```
\prg_new_conditional:Nnn \stex_if_in_module: {p, T, F, TF} {
  \tl_if_exist:NTF \l_stex_current_module_uri
    \prg_return_true: \prg_return_false:
}
```

(*End of definition for* `\stex_if_in_module:TF`. *This function is documented on page 143.*)

sfunction

`\stex_if_module_exists_p:N`
`\stex_if_module_exists:N`*TF*
`\stex_if_module_exists_p:n`
`\stex_if_module_exists:n`*TF*

```
\prg_new_conditional:Nnn \stex_if_module_exists:N {p, T, F, TF} {
  \tl_if_exist:cTF { \_stex_module_code_macro:N #1 }
    \prg_return_true: \prg_return_false:
}
\prg_new_conditional:Nnn \stex_if_module_exists:n {p, T, F, TF} {
  \tl_if_exist:cTF { \_stex_module_code_macro:n {#1} }
    \prg_return_true: \prg_return_false:
}
```

(*End of definition for* `\stex_if_module_exists:NTF` *and* `\stex_if_module_exists:nTF`. *These functions are documented on page 143.*)

sfunction

`\stex_activate_module:N`
`\stex_activate_module:n`

```
\cs_new_protected:Nn \stex_activate_module:N {
  \exp_args:No \stex_activate_module:n #1
}

\cs_new_protected:Nn \stex_activate_module:n {
  \seq_if_in:NnF \l_stex_all_modules_seq {#1} {
    \stex_debug:nn{modules}{Activating~module~\stex_use_module_uri:n{#1}}
```

```
    \seq_put_right:Nn \l_stex_all_modules_seq {#1}
    \stex_pseudogroup:nn{
      \tl_set:Nn \l_stex_current_module_uri {#1}
      \tl_if_exist:cF { \_stex_module_code_macro:N \l_stex_current_module_uri }{
        \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:n {#1} }
      }
      \_stex_activate_symbols:
      \_stex_activate_notations:
      \stex_debug:nn{modules}{Executing:~\expandafter\meaning\csname\_stex_module_code_macr
      \use:c{ \_stex_module_code_macro:N \l_stex_current_module_uri }
    }{
      \stex_pseudogroup_restore:N \l_stex_current_module_uri
    }
    \stex_debug:nn{modules}{Activated~module~\stex_use_module_uri:n {#1} }
  }
}
```

(*End of definition for* `\stex_activate_module:N` *and* `\stex_activate_module:n`*. These functions are documented on page 143.*)

sfunction

`\setmetatheory`

```
\NewDocumentCommand \setmetatheory {O{} m}{
  \stex_debug:nn{metatheory}{Setting~metatheory~[#1]#2}
  \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
  \stex_debug:nn{metatheory}{Here:\stex_use_module_uri:N \l_stex_import_uri
  }
  \tl_set_eq:NN \l_stex_metatheory_uri \l_stex_import_uri
  \begingroup
    \stex_require_module:N \l_stex_metatheory_uri
  \endgroup
  \stex_smsmode_do:
}
```

(*End of definition for* `\setmetatheory`*. This function is documented on page 143.*)

sfunction

`\STEXexport`

```
\cs_new_protected:Npn \STEXexport {
  \ExplSyntaxOn
  \__stex_modules_export:n
}
\cs_new_protected:Nn \__stex_modules_export:n {
  \stex_ignore_spaces_and_pars:#1\ExplSyntaxOff
  \tl_gput_right:cn { \_stex_module_code_macro:N \l_stex_current_module_uri } {
    \stex_ignore_spaces_and_pars: #1
  }
  \stex_smsmode_do:
}

\stex_deactivate_macro:Nn \STEXexport {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \STEXexport}
```

(*End of definition for* `\STEXexport`*. This function is documented on page 143.*)

sfunction

\stex_structural_feature_module:nn
\stex_structural_feature_module_end:

```
\cs_new_protected:Nn \stex_structural_feature_module:nn {
  \stex_if_do_html:TF {
    \exp_args:Nne \begin{stex_annotate_env} {
      data-ftml-feature-#2={#1}
      \str_if_empty:NF \l_stex_macroname_str {,
        data-ftml-macroname={\l_stex_macroname_str}
      }
    }
    \stex_annotate_invisible:n{}
  }\group_begin:
  \stex_module_setup:n {#1}
}

\cs_new_protected:Nn \stex_structural_feature_module_end: {
  \tl_gset_eq:NN \g_stex_last_feature_str \l_stex_current_module_str
  \stex_close_module:
  \stex_if_do_html:TF{
    \end{stex_annotate_env}
  }\group_end:
}
```

(*End of definition for* `\stex_structural_feature_module:nn` *and* `\stex_structural_feature_module_end:`*. These functions are documented on page 143.*)

sfunction

31.1 SMS Mode

```
⟨@@=stex_smsmode⟩
```

\stex_sms_allow:N

```
\tl_new:N \g__stex_smsmode_allowed_tl

\cs_new_protected:Nn \stex_sms_allow:N {
  \tl_gput_right:Nn \g__stex_smsmode_allowed_tl {#1}
}
```

(*End of definition for* `\stex_sms_allow:N`*. This function is documented on page 144.*)

sfunction

\stex_sms_allow_escape:N

```
\tl_new:N \g__stex_smsmode_allowed_escape_tl

\cs_new_protected:Nn \stex_sms_allow_escape:N {
  \tl_gput_right:Nn \g__stex_smsmode_allowed_escape_tl {#1}
}
```

(*End of definition for* `\stex_sms_allow_escape:N`*. This function is documented on page 144.*)

sfunction

\stex_sms_allow_env:n

```
\seq_new:N \g__stex_smsmode_allowedenvs_seq

\cs_new_protected:Nn \stex_sms_allow_env:n {
```

```
  \seq_gput_right:Ne \g__stex_smsmode_allowedenvs_seq {\tl_to_str:n{#1}}
}
```

(*End of definition for* `\stex_sms_allow_env:n`. *This function is documented on page 144.*)

sfunction

Some initial allowed macros:

```
\stex_sms_allow:N \makeatletter
\stex_sms_allow:N \makeatother
\stex_sms_allow:N \ExplSyntaxOn
\stex_sms_allow:N \ExplSyntaxOff
\stex_sms_allow:N \rustexBREAK
\stex_sms_allow_env:n{smodule}
\stex_sms_allow_escape:N \setmetatheory
\stex_sms_allow_escape:N \STEXexport
```

`\stex_sms_allow_import:Nn`

```
\tl_new:N \g__stex_smsmode_allowed_import_tl
\tl_new:N \g__stex_smsmode_import_setup_tl

\cs_new_protected:Nn \stex_sms_allow_import:Nn {
  \tl_gput_right:Nn \g__stex_smsmode_allowed_import_tl {#1}
  \tl_gput_right:Nn \g__stex_smsmode_import_setup_tl {#2}
}
```

(*End of definition for* `\stex_sms_allow_import:Nn`. *This function is documented on page 144.*)

sfunction

`\stex_sms_allow_import_env:nn`

```
\seq_new:N \g__stex_smsmode_allowed_import_env_seq

\cs_new_protected:Nn \stex_sms_allow_import_env:nn {
  \seq_gput_right:Ne \g__stex_smsmode_allowed_import_env_seq {\tl_to_str:n{#1}}
  \tl_gput_right:Nn \g__stex_smsmode_import_setup_tl {#2}
}

%\stex_sms_allow_import_env:nn{smodule}{}
```

(*End of definition for* `\stex_sms_allow_import_env:nn`. *This function is documented on page 144.*)

sfunction

`\g_stex_sms_import_code`

```
\tl_new:N \g_stex_sms_import_code
```

(*End of definition for* `\g_stex_sms_import_code`. *This variable is documented on page 144.*)

`\stex_if_smsmode_p:`
`\stex_if_smsmode:TF`

```
\bool_new:N \g__stex_smsmode_bool

\prg_new_conditional:Nnn \stex_if_smsmode: { p, T, F, TF } {
  \bool_if:NTF \g__stex_smsmode_bool \prg_return_true: \prg_return_false:
}
```

(*End of definition for* `\stex_if_smsmode:TF`. *This function is documented on page 144.*)

sfunction

\stex_file_in_smsmode:Nn

```

\seq_new:N \l__stex_smsmode_cycles_seq

\cs_new_protected:Nn \stex_file_in_smsmode:Nn {
  \seq_gclear:N \l__stex_smsmode_importmodules_seq
  \seq_gclear:N \l__stex_smsmode_sigmodules_seq
  \tl_clear:N \g_stex_sms_import_code
  \seq_if_in:NnF \l__stex_smsmode_cycles_seq {#2} {
  \group_begin:
    \seq_push:Nn \l__stex_smsmode_cycles_seq {#2}
    \let \l_stex_metatheory_uri \c_stex_default_metatheory
    \seq_clear:N \l_stex_all_modules_seq
    \stex_undefine:N \l_stex_current_module_uri
    \cs_set:Npn \stex_check_term:nn ##1 ##2 {}
    \stex_debug:nn{sms}{Loading~#2~in~\stex_use_document_uri:N #1}
    \stex_with_file_hooks:Nnn #1 {#2}{
      \__stex_smsmode_in_smsmode:n {
        \group_begin:
          \let \__stex_smsmode_do_aux_curr:N \__stex_smsmode_do_aux_imports:N
          \g__stex_smsmode_import_setup_tl
          \__stex_smsmode_start_smsmode:n{#2}
        \group_end:
        %{
          \g_stex_sms_import_code
        %}
        %\group_begin:
          \let \__stex_smsmode_do_aux_curr:N \__stex_smsmode_do_aux_normal:N
          \__stex_smsmode_start_smsmode:n{#2}
        %\group_end:
      }
    }
  \group_end:
  }
}

\cs_new_protected:Nn \__stex_smsmode_in_smsmode:n {
  \stex_suppress_html:n {
    \vbox_set:Nn \l_tmpa_box {
      \bool_set_true:N \g__stex_smsmode_bool
      #1
    }
  }
}

\quark_new:N \q__stex_smsmode_break

\cs_new_protected:Nn \__stex_smsmode_start_smsmode:n {
  \everyeof{\q__stex_smsmode_break\exp_not:N}
  \let\stex_smsmode_do:\__stex_smsmode_smsmode_do:
  \exp_after:wN \exp_after:wN \exp_after:wN
  \stex_smsmode_do:
  \cs:w @ @ input\cs_end: "#1" \relax
}
```

(*End of definition for* `\stex_file_in_smsmode:Nn`. *This function is documented on page 144.*)

sfunction

`\stex_smsmode_do:`

```
\let\stex_smsmode_do:\relax

\cs_new_protected:Nn \__stex_smsmode_smsmode_do: { \__stex_smsmode_do:w }

\cs_new_protected:Npn \__stex_smsmode_do:w #1 {
  \exp_args:Ne \tl_if_empty:nTF { \tl_tail:n{ #1 }}{
    %\__stex_smsmode_check_cs:NNn \__stex_smsmode_do_aux:N \__stex_smsmode_do:w { #1 }
    \__stex_smsmode_check_cs:N {#1}
  }{
    \__stex_smsmode_do:w #1
  }
}

\cs_new_protected:Nn \__stex_smsmode_check_cs:N {
  %\stex_debug:nn{temp}{Checking \exp_not:N#1 \relax}
  \exp_after:wN\if\exp_after:wN\relax\noexpand #1 ~
  \exp_after:wN \__stex_smsmode_do_aux:N \exp_after:wN#1\else
  \exp_after:wN \__stex_smsmode_do:w \fi
}

%\cs_new_protected:Nn \__stex_smsmode_check_cs:NNn {
%  \message{^^JHERE: \tl_to_str:n{#1~and~#2~and~#3}}
%  \exp_after:wN\if\exp_after:wN\relax\exp_not:N#3
%  \exp_after:wN#1\exp_after:wN#3\else
%  \exp_after:wN#2\fi
%}

\cs_new_protected:Nn \__stex_smsmode_do_aux:N {
  \cs_if_eq:NNF #1 \q__stex_smsmode_break {
    \__stex_smsmode_do_aux_curr:N #1
  }
}

\cs_new_protected:Nn \__stex_smsmode_do_aux_normal:N {
  \tl_if_in:NnTF \g__stex_smsmode_allowed_tl {#1} {
    \stex_debug:nn{sms}{Executing~\tl_to_str:n{#1}}
    #1\__stex_smsmode_do:w
  }{
    \tl_if_in:NnTF \g__stex_smsmode_allowed_escape_tl {#1} {
      \stex_debug:nn{sms}{Executing~escaped~\tl_to_str:n{#1}}
      #1
    }{
      \cs_if_eq:NNTF \begin #1 {
        \__stex_smsmode_check_begin:Nn \g__stex_smsmode_allowedenvs_seq
      }{
        \cs_if_eq:NNTF \end #1 {
          \__stex_smsmode_check_end:Nn \g__stex_smsmode_allowedenvs_seq
        }{
          \__stex_smsmode_do:w
        }
```

```
      }
    }
  }
}

\cs_new_protected:Nn \__stex_smsmode_do_aux_imports:N {
  \tl_if_in:NnTF \g__stex_smsmode_allowed_import_tl {#1} {
    \stex_debug:nn{sms}{Executing~\tl_to_str:n{#1}~in~import}
    #1
  }{
    \cs_if_eq:NNTF \begin #1 {
      \__stex_smsmode_check_begin:Nn \g__stex_smsmode_allowed_import_env_seq
    }{
      \cs_if_eq:NNTF \end #1 {
        \__stex_smsmode_check_end:Nn \g__stex_smsmode_allowed_import_env_seq
      }{
        \__stex_smsmode_do:w
      }
    }
  }
}

\cs_new_protected:Nn \__stex_smsmode_check_begin:Nn {
  \seq_if_in:NeTF #1 { \tl_to_str:n{#2} }{
    \stex_debug:nn{sms}{Environment~#2}
    \begin{#2}
  }{
    \__stex_smsmode_do:w
  }
}
\cs_new_protected:Nn \__stex_smsmode_check_end:Nn {
  \seq_if_in:NeTF #1 { \tl_to_str:n{#2} }{
    \stex_debug:nn{sms}{End~Environment~#2}
    \end{#2}\__stex_smsmode_do:w
  }{
    \__stex_smsmode_do:w
  }
}
```

(*End of definition for* `\stex_smsmode_do:`. *This function is documented on page 144.*)

sfunction

31.2 Inheritance and Morphisms

```
⟨@@=stex_import⟩
```

`\stex_require_module:N`
`\stex_require_module_noerr:N`
`\stex_require_module_noerr:n`

```
\cs_new_protected:Nn \stex_require_module:N {
  \stex_if_module_exists:NF #1 {
    \__stex_import_load_module:NF #1 {
      \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:N #1}
    }
  }
  \stex_activate_module:N #1
```

```
}

\cs_new_protected:Nn \stex_require_module_noerr:N {
  \stex_if_module_exists:NF #1 {
    \__stex_import_load_module:NF #1 {}
    \str_if_empty:NF \l__stex_import_file_str {
      \stex_activate_module:N #1
    }
  }
}

\cs_new_protected:Nn \stex_require_module_noerr:n {
  \tl_set:Nn \l__stex_import_uri { #1 }
  \stex_require_module_noerr:N \l__stex_import_uri
}

\cs_new_protected:Npn \__stex_import_load_module:NF #1 #2 {
    \tl_set_eq:NN \l__stex_import_uri #1
    \tl_set:Ne \l__stex_import_archive_str { \stex_module_uri_archive:N #1 }
    \tl_set:Ne \l__stex_import_path_str { \stex_module_uri_path:N #1 }
    \tl_set:Ne \l__stex_import_name_str { \stex_module_uri_name:N #1 }
    \str_if_eq:NNTF \l__stex_import_archive_str \c_stex_no_archive_str {
      \str_set_eq:NN \l__stex_import_pre_str \l__stex_import_path_str
      \__stex_import_load_check:n {#2}
    }{
      \exp_args:No \stex_in_archive:nn \l__stex_import_archive_str {
        \str_set:Ne \l__stex_import_pre_str {
          \exp_args:Ne \stex_source_path:n{ \l__stex_import_path_str }
        }
        \exp_args:No \__stex_import_load_check:n {#2}
      }
    }
}

\cs_new_protected:Nn \__stex_import_load_check:n {
  \str_clear:N \l__stex_import_file_str
  \str_set_eq:NN \l__stex_import_language_str \l_stex_current_language_str
  \__stex_import_check_file:nnn{ / \l__stex_import_name_str .tex }{}{
    \__stex_import_check_file:nnn{/ \l__stex_import_name_str . \l_stex_current_language_str
      \__stex_import_check_file:nnn{/ \l__stex_import_name_str .en.tex}{
        \str_set:Nn \l__stex_import_language_str {en}
      }{
        \__stex_import_load_check_i:n{#1}
      }
    }
  }
  \__stex_import_load_check_ii:
}

\cs_new_protected:Nn \__stex_import_load_check_i:n {
  \exp_args:NNNo \seq_set_split:Nnn \l_tmpa_seq / \l__stex_import_path_str
  \seq_pop_right:NN \l_tmpa_seq \l__stex_import_name_str
  \str_set:Ne \l__stex_import_path_str { \seq_use:Nn \l_tmpa_seq /}
  \__stex_import_check_file:nnn{.tex}{}{
```

```
    \__stex_import_check_file:nnn{. \l_stex_current_language_str .tex}{}{
      \__stex_import_check_file:nnn{.en.tex}{
        \str_set:Nn \l__stex_import_language_str {en}
      }{
        #1
      }
    }
  }
}

\cs_new_protected:Nn \__stex_import_load_check_ii: {
  \str_if_empty:NF \l__stex_import_file_str {
    \stex_if_smsmode:TF{
      \exp_args:NNo \exp_args:Nne \str_if_eq:nnTF{\l__stex_import_file_str}{\stex_file_use:
        \stex_debug:nn{imports}{Skipping~current~file}
      }{
        \IfFileExists{ \l__stex_import_file_str }{
          \__stex_import_load_file:
        }{}
      }
    }{
      \IfFileExists{ \l__stex_import_file_str }{
        \__stex_import_load_file:
      }{}
    }
  }
}

\cs_new_protected:Npn \__stex_import_check_file:nnn #1 #2 {
  \stex_debug:nn{imports}{Checking~ \l__stex_import_pre_str #1}
  \IfFileExists{ \l__stex_import_pre_str #1 }{
    \stex_debug:nn{imports}{Success}
    \str_set:Ne \l__stex_import_file_str { \l__stex_import_pre_str #1 }
    #2
  }
}

\cs_new_protected:Nn \__stex_import_load_file: {
  \tl_set:Ne \l__stex_import_doc_uri {
    { \l__stex_import_archive_str }
    { \l__stex_import_path_str }
    { \l__stex_import_name_str }
    { \l__stex_import_language_str }
    {}
  }
  \exp_args:NNo \stex_file_in_smsmode:Nn \l__stex_import_doc_uri \l__stex_import_file_str
  \stex_if_module_exists:NF \l__stex_import_uri {
    \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:N \l__stex_import_uri}
  }
}
```

(*End of definition for* `\stex_require_module:N`, `\stex_require_module_noerr:N`, *and* `\stex_require_module_noerr:n`. *These functions are documented on page 145.*)

sfunction

\usemodule

```
\stex_new_stylable_cmd:nnnn {usemodule} { O{} m } {
  \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
  \stex_require_module:N \l_stex_import_uri
  \__stex_import_usemodule:
}{}%{\aftergroup\ignorespaces}

\cs_new_protected:Nn \__stex_import_usemodule: {
  \stex_debug:nn{usemodule}{Done.}
  \stex_if_do_html:T {
    \hbox{\stex_annotate_invisible:nn
      {data-ftml-usemodule=\stex_use_module_uri:N \l_stex_import_uri } {}}
  }
  \stex_if_smsmode:F{
    \group_begin:
    \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
    \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
    \tl_clear:N \thisstyle
    \stex_style_apply:
    \group_end:
  }
}
```

(*End of definition for* \usemodule*. This function is documented on page 145.*)

sfunction

\importmodule

```
\stex_new_stylable_cmd:nnnn{importmodule} { O{} m } {
  \__stex_import_import_module:nn {#1}{#2}
  \stex_smsmode_do:
}{\aftergroup\ignorespaces}

\stex_deactivate_macro:Nn \importmodule {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \importmodule}
\stex_sms_allow_escape:N \importmodule

\cs_new_protected:Nn \__stex_import_import_module:nn {
  \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
  \stex_require_module:N \l_stex_import_uri
  \__stex_import_import_module_i:n {#1}
}

\cs_new_protected:Nn \__stex_import_import_module_i:n {
  \stex_execute_in_module:e{
    \stex_activate_module:n { \l_stex_import_uri }
  }
  \stex_add_morphism:nonn
    {}{\l_stex_import_uri}{import}{}
  \stex_if_do_html:T {
    \stex_annotate_invisible:nn
      {data-ftml-import=\stex_use_module_uri:N \l_stex_import_uri } {}
  }
  \stex_if_smsmode:F{
    \group_begin:
```

```
    \tl_set:Nn \thisarchive {#1}
    \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
    \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
    \tl_clear:N \thisstyle
    \stex_style_apply:
    \group_end:
  }
}
```

In sms mode, we change the definition:

```
\cs_new_protected:Nn \__stex_import_import_module_presms:nn {
  \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
  \bool_if:NF \l_stex_relative_import_bool {
    \tl_gput_right:Ne \g_stex_sms_import_code {
      \stex_require_module_noerr:n { \l_stex_import_uri }
    }
  }
}

\stex_sms_allow_import:Nn \importmodule {
  \stex_reactivate_macro:N \importmodule
  \let \__stex_import_import_module:nn \__stex_import_import_module_presms:nn
}
```

(*End of definition for* `\importmodule`*. This function is documented on page 145.*)

sfunction

`\requiremodule`

```
\stex_new_stylable_cmd:nnnn {requiremodule} { O{} m } {
  \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
  \stex_require_module:N \l_stex_import_uri
  \__stex_import_requiremodule:
}{}%{\aftergroup\ignorespaces}
\stex_deactivate_macro:Nn \requiremodule {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \requiremodule}

\cs_new_protected:Nn \__stex_import_requiremodule: {
  \stex_debug:nn{requiremodule}{Done.}
  \stex_if_do_html:T {
    \hbox{\stex_annotate_invisible:nn
      {data-ftml-import=\stex_use_module_uri:N \l_stex_import_uri } {}}
  }
  \stex_if_smsmode:F{
    \group_begin:
    \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
    \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
    \tl_clear:N \thisstyle
    \stex_style_apply:
    \group_end:
  }
}
```

(*End of definition for* `\requiremodule`*. This function is documented on page 145.*)

sfunction

\stex_add_morphism:nnnn

```
\cs_new_protected:Nn \stex_add_morphism:nnnn {
  \exp_args:Nne \prop_gput:cnn{ \_stex_morphisms_macro:N \l_stex_current_module_uri }{
    \tl_if_empty:nTF{#1}{[\stex_use_module_uri:n {#2}]}{#1}
  }{{#1}{#2}{#3}{#4}}
}
\cs_generate_variant:Nn \stex_add_morphism:nnnn {nonn,oooe}
```

(*End of definition for* `\stex_add_morphism:nnnn`. *This function is documented on page 145.*)

sfunction

31.3 Theory Morphisms

```
⟨@@=stex_morphisms⟩
```

\stex_structural_feature_morphism:nnnnn
ructural_feature_morphism_with_macros:nnnnn
\stex_structural_feature_morphism_end:

```
\tl_new:N \l_stex_current_domain_uri
\bool_new:N \l__stex_morphisms_with_macros_bool

\cs_new_protected:Npn \stex_structural_feature_morphism_with_macros:nnnnn {
  \bool_set_true:N \l__stex_morphisms_with_macros_bool
  \__stex_morphisms_morphism:nnnnn
}
\cs_new_protected:Npn \stex_structural_feature_morphism:nnnnn {
  \bool_set_false:N \l__stex_morphisms_with_macros_bool
  \__stex_morphisms_morphism:nnnnn
}

\cs_new_protected:Nn \__stex_morphisms_morphism:nnnnn {
  \tl_clear:N \l_stex_current_domain_uri
  \stex_debug:nn{morphisms}{new~morphism:{#1}{#2}{#3}{#4}{#5}}
  \tl_if_empty:nT{#3}{
    \stex_get_mathstructure:n{#4}
    \tl_set_eq:NN \l_stex_current_domain_uri \l_stex_get_structure_module_uri
  }
  \tl_if_empty:NT \l_stex_current_domain_uri {
    \stex_uri_from_pair:Nnn \l_stex_import_uri { #3 }{ #4 }
    \group_begin:
    \stex_require_module:N \l_stex_import_uri
    \exp_args:Nne \use:nn \group_end: {
      \tl_set:Nn \exp_not:N\l_stex_current_domain_uri {\l_stex_import_uri}
    }
  }
  \tl_if_empty:nT{#1}{
    \msg_error:nn{stex}{error/morphism-needs-name}
  }
  \str_set:Nn \l_tmpa_str {#1}

  \stex_debug:nn{morphisms}{#2:~\l_tmpa_str;~for~\stex_use_module_uri:N \l_stex_current_dom

  \str_set:Nn \l__stex_morphisms_feature_str {#2}
  \str_set_eq:NN \l_stex_feature_name_str \l_tmpa_str

```

```
  \stex_if_do_html:TF {
    \begin{stex_annotate_env} {
      data-ftml-feature-#2={\l_tmpa_str},
      data-ftml-domain={\stex_use_module_uri:N \l_stex_current_domain_uri}
      #5
    }
    \stex_annotate_invisible:n{}
  }\group_begin:
  \__stex_morphisms_setup:
  \__stex_morphisms_reactivate:
  \stex_metagroup_new:
}

\cs_new_protected:Nn \__stex_morphisms_setup: {
  \seq_clear:N \l_stex_morphism_symbols_seq
  \seq_clear:N \l_stex_morphism_renames_seq
  \seq_clear:N \l_stex_morphism_morphisms_seq
  \seq_clear:N \l_stex_morphism_assigns_seq
  \__stex_morphisms_do_decls:
  \exp_args:No \__stex_morphisms_do:n \l_stex_current_domain_uri
}

\cs_new_protected:Nn \__stex_morphisms_do_decls: {
  \exp_args:No \stex_iterate_symbols:nn \l_stex_current_domain_uri {
    \exp_args:NNe \seq_put_right:Nn \l_stex_morphism_symbols_seq {
      { \stex_new_symbol_uri:nn{##1}{##3} }
      { {##2}{##4}{##5}{##6}{##7}{##8}##9 }
    }
  }
}


\cs_new_protected:Nn \__stex_morphisms_do:n {
  %\seq_clear:N \l__stex_morphisms_dones_seq
    \prop_map_inline:cn {\_stex_morphisms_macro:n{#1}}{
      %\seq_if_in:NnF \l__stex_morphisms_dones_seq {##2}{
      %  \seq_push:Nn \l__stex_morphisms_dones_seq {##2}
        \stex_debug:nn{morphisms}{Doing ~ \tl_to_str:n{##2}}
        \__stex_morphisms_do_morph:nnnn ##2
     % }
    }
}

\cs_new_protected:Nn \__stex_morphisms_do_morph:nnnn {
    \tl_if_empty:nF{#3}{
      \seq_put_right:Nn \l_stex_morphism_morphisms_seq {{#1}{#2}{#3}}
    }
    \__stex_morphisms_do:n{#2}
}


\cs_new_protected:Nn \stex_structural_feature_morphism_end: {
  \tl_gset_eq:NN \l_stex_current_domain_uri \l_stex_current_domain_uri
  \seq_gset_eq:NN \l_stex_morphism_symbols_seq \l_stex_morphism_symbols_seq
```

```
  \seq_gset_eq:NN \l_stex_morphism_renames_seq \l_stex_morphism_renames_seq
  \seq_gset_eq:NN \l_stex_morphism_assigns_seq \l_stex_morphism_assigns_seq
  \seq_gset_eq:NN \l_stex_morphism_morphisms_seq \l_stex_morphism_morphisms_seq
  \bool_gset_eq:NN \l__stex_morphisms_with_macros_bool \l__stex_morphisms_with_macros_bool
  \stex_if_do_html:TF{
    \end{stex_annotate_env}
  }\group_end:
  \__stex_morphisms_do_elaboration:
}

\cs_new_protected:Nn \__stex_morphisms_do_elaboration: {
  \stex_debug:nn{morphisms}{
    Elaborating:^^J\meaning \l_stex_morphism_symbols_seq
    ^^J^^J
    Renamings:^^J
    \meaning \l_stex_morphism_renames_seq
    ^^J^^J
    Assignments:^^J
    \meaning \l_stex_morphism_assigns_seq
  }

  \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
    \__stex_morphisms_elab:nn ##1
  }

  \stex_add_morphism:oooe
    \l_stex_feature_name_str
    \l_stex_current_domain_uri
    \l__stex_morphisms_feature_str
    {\seq_map_function:NN \l_stex_morphism_renames_seq \__stex_morphisms_rename:n}
}

\cs_new:Nn \__stex_morphisms_rename:n{
  \__stex_morphisms_rename:nn #1
}

\cs_new:Nn \__stex_morphisms_rename:nn{
  {#1}{\use_ii:nn#2}
}

\cs_new_protected:Nn \__stex_morphisms_elab:nn {
  \tl_clear:N \l__stex_morphisms_tmp
  \seq_map_inline:Nn \l_stex_morphism_renames_seq {
    \__stex_morphisms_elab_check_rename:nnn{#1}##1
  }
  \tl_if_empty:NTF \l__stex_morphisms_tmp {
    \exp_args:Ne \__stex_morphisms_elab_i:nnn {\stex_symbol_uri_name:n {#1}}{#1}
  }{
    \stex_debug:nn{morphisms}{Generating~1~\l__stex_morphisms_tmp}
    \use_i:nn{ \exp_args:Nno \use:nn {\__stex_morphisms_add_symbol:nnnnnnnnN {#1}} \l__stex
  }#2

  \exp_args:No\stex_iterate_notations:nn\l_stex_current_domain_uri {
    \str_if_eq:nnT{#1}{##1}{
```

```
      \exp_args:Ne \stex_add_notation:nnnnn {
        \exp_args:Ne \stex_new_symbol_uri:n {\exp_after:wN \use_ii:nn \l__stex_morphisms_tm
      }{##2}{##3}{##4}{##5}
    }
  }
}

\cs_new_protected:Nn \__stex_morphisms_elab_i:nnn {
  \tl_set:Ne \l__stex_morphisms_tmp {{}{\l_stex_feature_name_str / #1}}
  \stex_debug:nn{morphisms}{Generating~2~\l_stex_feature_name_str / #1}
  \bool_if:NTF \l__stex_morphisms_with_macros_bool {
    \exp_args:Nnno \__stex_morphisms_add_symbol:nnnnnnnnN {#2} {#3}
  }{
    \exp_args:Nnno \__stex_morphisms_add_symbol:nnnnnnnnN {#2} {}
  }
  {\l_stex_feature_name_str / #1}
}

\cs_new_protected:Npn \__stex_morphisms_add_symbol:nnnnnnnnN #1 {
  \tl_clear:N \l__stex_morphisms_tmp_b
  \seq_map_inline:Nn \l_stex_morphism_assigns_seq {
    \__stex_morphisms_elab_check_assign:nnnnn{#1}##1
  }
  \tl_if_empty:NTF \l__stex_morphisms_tmp_b
  \stex_add_symbol:nnnnnnnN
  \__stex_morphisms_add_symbol_i:nnnnnnnN
}

\cs_new_protected:Npn \__stex_morphisms_add_symbol_i:nnnnnnnN #1 #2 #3 #4 #5 {
  \stex_add_symbol:nnnnnnnN {#1}{#2}{#3}{#4}{assigned}
}

\cs_new_protected:Nn \__stex_morphisms_elab_check_assign:nnnnn {
  \tl_if_eq:nnT{#1}{{#2}{#3}{#4}{#5}}{
    \seq_map_break:n{
      \tl_set:Nn \l__stex_morphisms_tmp_b {assigned}
    }
  }
}

\cs_new_protected:Nn \__stex_morphisms_elab_check_rename:nnn {
  \tl_if_eq:nnT{#1}{#2}{
    \seq_map_break:n{
      \tl_set:Nn \l__stex_morphisms_tmp {#3}
    }
  }
}

\cs_new_protected:Nn \__stex_morphisms_do_for_list: {
  \seq_clear:N \l_stex_fors_seq
  \clist_map_inline:Nn \l_stex_key_for_clist {
    \exp_args:Ne\stex_get_in_morphism:n{\tl_to_str:n{##1}}
    \seq_put_right:No \l_stex_fors_seq
      {\l_stex_get_symbol_uri}
```

```
  }
}


\cs_new_protected:Nn \__stex_morphisms_definiens_impl:nn {
  \tl_if_empty:nF {#1} {
    \stex_get_in_morphism:n { #1 }
    \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
  }
  \tl_if_empty:NT \l_stex_current_def_uri {
    \msg_error:nn{stex}{error/definiensfor}
  }
  \stex_debug:nn{definiens}{Checking~\stex_use_symbol_uri:N \l_stex_current_def_uri }
  \stex_if_do_html:TF{
    \exp_after:wN \stex_metagroup_do_in:n \exp_after:wN {
      \exp_after:wN \seq_put_right:Nn \exp_after:wN  \l_stex_morphism_assigns_seq
      \exp_after:wN { \l_stex_current_def_uri }
    }
    \mode_leave_vertical: \stex_annotate:nn{data-ftml-assign={\stex_use_symbol_uri:N  \l_st
      \_stex_annotate_force_break:n{
        \stex_annotate:nn{data-ftml-definiens={}}{#2}
      }
    }
  }{
    \exp_args:No \__stex_morphisms_add_definiens:nn \l_stex_current_def_uri {#2}
    \stex_if_smsmode:F{#2}
  }
}

\cs_new_protected:Nn \__stex_morphisms_add_definiens:nn {
  \tl_set:Nn \l_stex_get_symbol_uri {#1}
  \stex_debug:nn{morphisms}{Adding~definiens~\tl_to_str:n{#1~#2}}
  %\__stex_morphisms_set_definiens_macros: #1\__stex_morphisms_break:
  \_stex_assign_do:n{#2}
}

%\cs_new_protected:Npn \__stex_morphisms_set_definiens_macros: #1?#2?#3\__stex_morphisms_br
%  \str_set:Nn \l_stex_get_symbol_uri { #1?#2} % TODO
%  \str_set:Nn \l_stex_get_symbol_name_str {#3}
%  \exp_args:Nne\use:nn{\__stex_morphisms_set_definiens_macros_i:nnnnnnn}{
%    \prop_item:Nn \l_stex_morphism_symbols_prop {[#1?#2]/[#3]}
%  }
%}

%\cs_new_protected:Nn \__stex_morphisms_set_definiens_macros_i:nnnnnnn {
%  \tl_set:Nn \l_stex_get_symbol_def_tl{#4}
%}


\cs_new_protected:Nn \__stex_morphisms_rename_all: {
    % TODO
}
```

```
\cs_new_protected:Nn \stex_structural_feature_morphism_check_total: {
  \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
    \__stex_morphisms_total_check:nn ##1
  }
}

\cs_new_protected:Nn \__stex_morphisms_total_check:nn {
  \__stex_morphisms_total_check:nnnnnnnN {#1} #2
}

\cs_new_protected:Nn \__stex_morphisms_total_check:nnnnnnnN {
  \tl_if_empty:nT{#5}{
    \seq_if_in:NnF \l_stex_morphism_assigns_seq {#1} {
      \msg_error:nnxx{stex}{error/needsdefiniens}{\stex_use_symbol_uri:n {#1}}{total~morphi
    }
  }
}

\cs_new:Npn \__stex_morphisms_split_qm:w #1 ? #2 ? #3 { #3 }



\cs_new_protected:Npn \__stex_morphisms_reactivate: {
  \stex_deactivate_macro:Nn \symdecl {module~environments}
  \stex_deactivate_macro:Nn \textsymdecl {module~environments}
  \stex_deactivate_macro:Nn \symdef {module~environments}
  \stex_deactivate_macro:Nn \notation {module~environments}
  \stex_deactivate_macro:Nn \importmodule {module~environments}
  \stex_deactivate_macro:Nn \requiremodule {module~environments}
  \stex_deactivate_macro:Nn \smodule {outside~of~morphisms}
  \stex_reactivate_macro:N \assign
  \stex_reactivate_macro:N \assignMorphism
  \stex_reactivate_macro:N \renamedecl
  \cs_set_eq:NN \_stex_do_for_list: \__stex_morphisms_do_for_list:
  \cs_set_eq:NN \_stex_add_definiens:nn \__stex_morphisms_add_definiens:nn
  \cs_set_eq:NN \_stex_definiens_impl:nn \__stex_morphisms_definiens_impl:nn
}
```

(*End of definition for* `\stex_structural_feature_morphism:nnnnn`, `\stex_structural_feature_morphism_with_macros:nnnnn`, *and* `\stex_structural_feature_morphism_end:`. *These functions are documented on page 145.*)

sfunction

`\stex_iterate_morphisms:nn`

```
\cs_new_protected:Nn \stex_iterate_morphisms:nn {
  \seq_clear:N \l__stex_morphisms_mods_seq
  \bool_set_true:N \l__stex_morphisms_continue_bool
  \cs_set:Npn \__stex_morphisms_cs:nnnn ##1 ##2 ##3 ##4 ##5 {
    #2
    \bool_if:NT \l__stex_morphisms_continue_bool {
      \str_if_eq:nnTF{##1}{[##2]}{
        \tl_put_right:Nn \l__stex_morphisms_todo_tl {
          \__stex_morphisms_iterate:nn{##5}{##2}
        }
      }{
```

```
        \tl_put_right:Nn \l__stex_morphisms_todo_tl {
          \__stex_morphisms_iterate:nn{##5 / ##1}{##2}
        }
      }
    }
  }
  \cs_set:Npn \stex_iterate_break:n ##1 {
    \bool_set_false:N \l__stex_morphisms_continue_bool
    \prop_map_break:n{##1}
  }
  \__stex_morphisms_iterate:nn{}{#1}
}
\cs_new_protected:Nn \__stex_morphisms_iterate:nn {
  \tl_clear:N \l__stex_morphisms_todo_tl
  \seq_if_in:NnF \l__stex_morphisms_mods_seq {#1 #2}{
    \seq_put_right:Nn \l__stex_morphisms_mods_seq {#1 #2}
    \prop_map_inline:cn{\_stex_morphisms_macro:n{#2}}{
      \__stex_morphisms_cs:nnnn ##2 {#1}
      % TODO
      % ##1: name or [mpath]
      % ##2 = {####1}{####2}{####3}{#####4}
      % ####1 = name
      % ####2 = mpath
      % ####3 = type
      % #####4 = {origname}{newname}*
    }
    \bool_if:NT \l__stex_morphisms_continue_bool \l__stex_morphisms_todo_tl
  }
}
```

(*End of definition for* `\stex_iterate_morphisms:nn`*. This function is documented on page 145.*)

sfunction

`\stex_get_in_morphism:n`

```
\cs_new_protected:Nn \stex_get_in_morphism:n {
  \stex_debug:nn{morphisms}{Getting~in~morphism:~#1}
  \tl_clear:N \l_stex_get_symbol_uri
  \str_set:Nn \l__stex_morphisms_get_str {#1}
  \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
    \__stex_morphisms_get_check:nn ##1
  }
  \tl_if_empty:NT \l_stex_get_symbol_uri {
    \seq_map_inline:Nn \l_stex_morphism_renames_seq {
      \__stex_morphisms_renamed_check:nn##1
    }
    \tl_if_empty:NT \l_stex_get_symbol_uri {
      \msg_error:nnxx{stex}{error/unknownsymbolin}{#1}{
        morphism~\l_stex_feature_name_str
      }
    }
  }
}
```

```
\cs_new_protected:Nn \__stex_morphisms_get_check:nn {
  \__stex_morphisms_check_name:nnnn #1 #2
}

\cs_new_protected:Nn \__stex_morphisms_check_name:nnnn {
  % TODO module name, path, archive, macroname ?
  \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str {#4} {
    \tl_set:Nn \l_stex_get_symbol_uri {{#1}{#2}{#3}{#4}}
    \__stex_morphisms_set:nnnnnnN
  }{
    \__stex_morphisms_macro:nnnnnnnN{{#1}{#2}{#3}{#4}}
  }
}

\cs_new_protected:Nn \__stex_morphisms_set:nnnnnnN {
  \seq_map_break:n{
    \str_set:Nn \l_stex_get_symbol_macro_str{#1}
    \int_set:Nn \l_stex_get_symbol_arity_int {#2}
    \tl_set:Nn \l_stex_get_symbol_args_tl {#3}
    \tl_set:Nn \l_stex_get_symbol_def_tl {#4}
    \tl_set:Nn \l_stex_get_symbol_type_tl {#5}
    \tl_set:Nn \l_stex_get_symbol_return_tl {#6}
    \tl_set:Nn \l_stex_get_symbol_invoke_cs {#7}
  }
}

\cs_new_protected:Npn \__stex_morphisms_macro:nnnnnnnN #1 #2 {
  \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str {#2}{
    \tl_set:Nn \l_stex_get_symbol_uri{#1}
    \__stex_morphisms_set:nnnnnnN
  }\__stex_morphisms_gobble:nnnnnnN
  {#2}
}

\cs_new_protected:Nn \__stex_morphisms_gobble:nnnnnnN {}

\cs_new_protected:Nn \__stex_morphisms_renamed_check:nn {
  \stex_debug:nn{here}{\tl_to_str:n{{#1}{#2}}:~\l__stex_morphisms_get_str}
  \__stex_morphisms_renamed_check:nnnnnn #1 #2
}

\cs_new_protected:Nn \__stex_morphisms_renamed_check:nnnnnn {
  \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str{#5}{
    \seq_map_break:n{
      \str_set:Nn \l__stex_morphisms_get_str {#4}
      \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
        \__stex_morphisms_get_check:nn ##1
      }
    }
  }{
    \exp_args:No \str_if_eq:nnT \l__stex_morphisms_get_str{#6}{
      \seq_map_break:n{
        \str_set:Nn \l__stex_morphisms_get_str {#4}
        \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
```

```
          \__stex_morphisms_get_check:nn ##1
        }
      }
    }
  }
}
```

(*End of definition for* `\stex_get_in_morphism:n`. *This function is documented on page 145.*)

sfunction

`copymodule` (*env.*)

```
\stex_new_stylable_env:nnnnnnn {copymodule}{m O{} m}{
  \__stex_morphisms_begin_copy:Nnnn\stex_structural_feature_morphism:nnnnn {#1}{#2}{#3}
}{
  \stex_if_smsmode:F {
    \stex_style_apply:
  }
  \stex_structural_feature_morphism_end:
}{}{}{}

\stex_new_stylable_env:nnnnnnn {copymodule*}{m O{} m}{
  \cs_set:Npn \stex_style_apply: {
    \_stex_apply_patch_begin:n{copymodule}
  }
  \__stex_morphisms_begin_copy:Nnnn\stex_structural_feature_morphism_with_macros:nnnnn {#1}
}{
  \cs_set:Npn \stex_style_apply: {
    \_stex_apply_patch_end:n{copymodule}
  }
  \stex_if_smsmode:F {
    \stex_style_apply:
  }
  \stex_structural_feature_morphism_end:
}{}{}{}

\cs_new_protected:Nn \__stex_morphisms_begin_copy:Nnnn {
  #1{#2}{morphism}{#3}{#4}{,data-ftml-total=false}
  \stex_if_smsmode:F {
    \tl_set:Nn \thiscopyname { #2 }
    \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
    \stex_style_apply:
  }
  \stex_smsmode_do:
}


\stex_deactivate_macro:Nn \copymodule {module~environments}
\stex_every_module:n {
  \stex_reactivate_macro:N \copymodule
}
\stex_sms_allow_env:n{copymodule}

\exp_after:wN \stex_deactivate_macro:Nn \csname copymodule*\endcsname {module~environments}
\stex_every_module:n {
```

```
  \exp_after:wN\stex_reactivate_macro:N \csname copymodule*\endcsname
}
\stex_sms_allow_env:n{copymodule*}

\stex_new_stylable_cmd:nnnn{copymod}{s m O{} m m}{
  \IfBooleanTF#1\stex_structural_feature_morphism_with_macros:nnnnn
  \stex_structural_feature_morphism:nnnnn{#2}{morphism}{#3}{#4}{,data-ftml-total={false}}
  \clist_map_function:nN{#5}\__stex_morphisms_parse_assign:n
  \stex_if_smsmode:F {
    \tl_set:Nn \thiscopyname { #2 }
    \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
    \stex_style_apply:
  }
  \stex_structural_feature_morphism_end:
  \stex_smsmode_do:
}{}

\stex_deactivate_macro:Nn \copymod {module~environments}
\stex_every_module:n {
  \stex_reactivate_macro:N \copymod
}
\stex_sms_allow_escape:N\copymod
```

`interpretmodule` (*env.*)

```
\stex_new_stylable_env:nnnnnnn {interpretmodule}{m O{} m}{
  \__stex_morphisms_begin_interpret:Nnnn\stex_structural_feature_morphism:nnnnn {#1}{#2}{#3
}{
  \stex_structural_feature_morphism_check_total:
  \stex_if_smsmode:F {
    \stex_style_apply:
  }
  \stex_structural_feature_morphism_end:
}{}{}{}

\stex_new_stylable_env:nnnnnnn {interpretmodule*}{m O{} m}{
  \cs_set:Npn \stex_style_apply: {
    \_stex_apply_patch_begin:n{interpretmodule}
  }
  \__stex_morphisms_begin_interpret:Nnnn\stex_structural_feature_morphism_with_macros:nnnnn
}{
  \cs_set:Npn \stex_style_apply: {
    \_stex_apply_patch_end:n{interpretmodule}
  }
  \stex_structural_feature_morphism_check_total:
  \stex_if_smsmode:F {
    \stex_style_apply:
  }
  \stex_structural_feature_morphism_end:
}{}{}{}

\cs_new_protected:Nn \__stex_morphisms_begin_interpret:Nnnn {
  #1{#2}{morphism}{#3}{#4}{,data-ftml-total=true}
  \stex_if_smsmode:F {
    \tl_set:Nn \thiscopyname { #2 }
```

```
    \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
    \stex_style_apply:
  }
  \stex_smsmode_do:
}

\stex_deactivate_macro:Nn \interpretmodule {module~environments}
\stex_every_module:n {
  \stex_reactivate_macro:N \interpretmodule
}
\stex_sms_allow_env:n{interpretmodule}

\exp_after:wN \stex_deactivate_macro:Nn \csname interpretmodule*\endcsname {module~environm
\stex_every_module:n {
  \exp_after:wN \stex_reactivate_macro:N \csname interpretmodule*\endcsname
}
\stex_sms_allow_env:n{interpretmodule*}

\stex_new_stylable_cmd:nnnn{interpretmod}{s m O{} m m}{
  \IfBooleanTF#1\stex_structural_feature_morphism_with_macros:nnnnn
  \stex_structural_feature_morphism:nnnnn{#2}{morphism}{#3}{#4}{,data-ftml-total=true}
  \clist_map_function:nN{#5}\__stex_morphisms_parse_assign:n
  \stex_if_smsmode:F {
    \tl_set:Nn \thiscopyname { #2 }
    \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
    \stex_style_apply:
  }
  \stex_structural_feature_morphism_check_total:
  \stex_structural_feature_morphism_end:
  \stex_smsmode_do:
}{}

\stex_deactivate_macro:Nn \interpretmod {module~environments}
\stex_every_module:n {
  \stex_reactivate_macro:N \interpretmod
}
\stex_sms_allow_escape:N\interpretmod
```

\assign

```
\stex_new_stylable_cmd:nnnn {assign} { m m }{
  \stex_get_in_morphism:n{#1}
  \stex_if_do_html:TF{
    \stex_annotate_invisible:n{\hbox{$\_stex_assign_do:n{#2}$}}
  }{
    \_stex_assign_do:n{#2}
  }
  \stex_smsmode_do:
}{}
\stex_sms_allow_escape:N\assign
\stex_deactivate_macro:Nn \assign {morphism~environments}

\cs_new_protected:Nn \_stex_assign_do:n{
  \stex_debug:nn{assign}{Assigning~\stex_use_symbol_uri:N \l_stex_get_symbol_uri~to~\tl_to_
  \stex_check_term:nn{assignment}{#1}
```

```
  \stex_if_do_html:T{
    \stex_annotate_invisible:nn{data-ftml-assign={\stex_use_symbol_uri:N \l_stex_get_symbol
      \_stex_annotate_force_break:n{
        \mode_if_math:TF{\stex_annotate:nn{data-ftml-definiens={}}{#1}}{\hbox{$\stex_annota
      }
    }
  }
  \exp_after:wN \stex_metagroup_do_in:n \exp_after:wN {
    \exp_after:wN \seq_put_right:Nn \exp_after:wN  \l_stex_morphism_assigns_seq
    \exp_after:wN { \l_stex_get_symbol_uri }
  }
}

\cs_new_protected:Nn \__stex_morphisms_parse_assign:n {
  \str_clear:N \l__stex_morphisms_name_str
  \str_clear:N \l__stex_morphisms_newname_str
  \tl_clear:N \l__stex_morphisms_ass_tl
  \stex_debug:nn{morphisms}{Parsing~#1}
  \exp_args:NNe \seq_set_split:Nnn \l__stex_morphisms_seq {\tl_to_str:n{@}} {#1}
  \int_compare:nNnTF {\seq_count:N \l__stex_morphisms_seq} = 1 {
    \stex_debug:nn{morphisms}{No~@}
    \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_next_tl
  }{
    \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_name_str
    \stex_debug:nn{morphisms}{Name:~\l__stex_morphisms_name_str}
    \exp_args:NNo \str_set:Nn \l__stex_morphisms_name_str \l__stex_morphisms_name_str
    \tl_set:Ne \l__stex_morphisms_next_tl {\seq_use:Nn \l__stex_morphisms_seq @}
  }
  \exp_args:NNNo \seq_set_split:Nnn \l__stex_morphisms_seq = \l__stex_morphisms_next_tl
  \str_if_empty:NTF \l__stex_morphisms_name_str {
    \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_name_str
    \exp_args:NNo \str_set:Nn \l__stex_morphisms_name_str \l__stex_morphisms_name_str
    \tl_set:Ne \l__stex_morphisms_ass_tl {\seq_use:Nn \l__stex_morphisms_seq =}
  }{
    \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_newname_str
    \exp_args:NNo \str_set:Nn \l__stex_morphisms_newname_str \l__stex_morphisms_newname_str
    \tl_set:Ne \l__stex_morphisms_ass_tl {\seq_use:Nn \l__stex_morphisms_seq =}
  }
  \__stex_morphisms_do_parsed_assign:
}

\cs_new_protected:Nn \__stex_morphisms_do_parsed_assign: {
  \exp_args:No \stex_get_in_morphism:n \l__stex_morphisms_name_str
  \str_if_empty:NF \l__stex_morphisms_newname_str {
    \stex_debug:nn{morphisms}{renaming~\l__stex_morphisms_name_str;~to~\l__stex_morphisms_n
    \exp_after:wN \__stex_morphisms_do_parsed_newname: \l__stex_morphisms_newname_str \__st
  }
  \tl_if_empty:NF \l__stex_morphisms_ass_tl {
    \stex_debug:nn{morphisms}{assigning~\l__stex_morphisms_name_str;~to~\exp_args:No \tl_to
    \exp_args:No \_stex_assign_do:n \l__stex_morphisms_ass_tl
  }
}

\cs_new_protected:Nn \__stex_morphisms_do_parsed_newname: {
```

```
  \peek_charcode:NTF [ {
    \__stex_morphisms_do_parsed_newname:w
  }{
    \__stex_morphisms_do_parsed_newname:w []
  }
}

\cs_new_protected:Npn \__stex_morphisms_do_parsed_newname:w [#1] #2 \__stex_morphisms_end:
  \_stex_renamedecl_do:nn{#1}{#2}
}
```

(*End of definition for* \assign. *This function is documented on page 146.*)

sfunction

\renamedecl

```
\stex_new_stylable_cmd:nnnn {renamedecl} { m O{} m }{
  \stex_get_in_morphism:n{#1}
  \_stex_renamedecl_do:nn{#2}{#3}
  \stex_smsmode_do:
}{}
\stex_sms_allow_escape:N\renamedecl
\stex_deactivate_macro:Nn \renamedecl {morphism~environments}

\cs_new_protected:Nn \_stex_renamedecl_do:nn {
  \stex_debug:nn{renamedecl}{Renaming~\stex_use_symbol_uri:N \l_stex_get_symbol_uri~to~[#1]
  \stex_if_do_html:T{
    \exp_args:Ne \stex_annotate_invisible:nn{
      data-ftml-rename={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
      data-ftml-macroname={#2}
      \str_if_empty:nF{#1}{ ,data-ftml-to={#1} }
    }{}
  }
  \stex_metagroup_do_in:e {
    \seq_push:Nn \exp_not:N \l_stex_morphism_renames_seq
    {{\l_stex_get_symbol_uri}{{#2}{
      \tl_if_empty:nTF{#1}{
        \l_stex_feature_name_str/\stex_symbol_uri_name:N \l_stex_get_symbol_uri
      }{#1}
    }}}
  }
}
```

(*End of definition for* \renamedecl. *This function is documented on page 146.*)

sfunction

\assignMorphism

```
\stex_new_stylable_cmd:nnnn {assignMorphism} { m m }{
  \str_clear:N \l__stex_morphisms_morphism_dom_str
  \exp_args:No \stex_iterate_morphisms:nn\l_stex_current_domain_uri{
    \stex_debug:nn{assignMorphism}{
      Checking:~#1~vs:^^J##1^^J##2^^J##3^^J##4
    }
    \str_if_eq:nnTF{#1}{##1}{
      \__stex_morphisms_do_morph_assign:nnn{##1}{##2}{##4}
```

```
    }{
      \stex_str_if_ends_with:nnT{##2}{#1}{
        \__stex_morphisms_do_morph_assign:nnn{##1}{##2}{##4}
      }
    }
  }
  \str_if_empty:NT \l__stex_morphisms_morphism_dom_str {
    \msg_error:nnn{stex}{error/nomorphism}{#1}
  }
  \bool_set_false:N \l_tmpa_bool
  \stex_iterate_morphisms:nn \l_stex_current_module_str {
    \stex_debug:nn{assignMorphism}{
      Checking:~#2~vs:^^J##1^^J##2^^J##3^^J##4
    }
    \str_if_eq:nnTF{#2}{##1}{
      \stex_debug:nn{assignMorphism}{match!}
      \stex_iterate_break:n{
        \stex_annotate_invisible:nn{
          data-ftml-assignmorphismfrom={\l__stex_morphisms_morphism_dom_str}
          data-ftml-assignmorphismto={\l_stex_current_module_str?##1}
        }{}
        \bool_set_true:N \l_tmpa_bool
      }
    }{
      \stex_str_if_ends_with:nnT{##2}{#2}{
        \stex_debug:nn{assignMorphism}{match!}
        \stex_iterate_break:n{
          \stex_annotate_invisible:nn{
            data-ftml-assignmorphismfrom={\l__stex_morphisms_morphism_dom_str},
            data-ftml-assignmorphismto={\l_stex_current_module_str?##1}
          }{}
          \bool_set_true:N \l_tmpa_bool
        }
      }
    }
  }
  \bool_if:NF \l_tmpa_bool {
    \msg_error:nnn{stex}{error/nomorphism}{#2}
  }
}{}
\stex_sms_allow_escape:N \assignMorphism
\stex_deactivate_macro:Nn \assignMorphism {morphism~environments}

\cs_new_protected:Nn \__stex_morphisms_do_morph_assign:nnn {
  \stex_iterate_break:n{
    \str_set:Ne \l__stex_morphisms_morphism_dom_str { \l_stex_current_domain_str ? #1 }
    \stex_debug:nn{assignMorphism}{match!}
    \stex_iterate_symbols:nn{#2}{
      \stex_debug:nn{assignMorphism}{removing~##1?##3}
      % TODO: non-trivial assignments
      \prop_remove:Nn \l_stex_morphism_symbols_prop {
        [##1]/[##3]
      }
    }
```

```
    }
}
```

(*End of definition for* `\assignMorphism`*. This function is documented on page 146.*)

sfunction

Chapter 32

Symbols

```
⟨@@=stex_syms⟩
```

32.1 Declarations

Keys used in various symbol declarations:

```
\stex_keys_define:nnnn{symargs}{
  \str_clear:N \l_stex_key_args_str
  \str_clear:N \l_stex_key_role_str
  \str_clear:N \l_stex_key_reorder_str
  \str_clear:N \l_stex_key_assoc_str
}{
  args      .str_set:N  = \l_stex_key_args_str ,
  reorder   .str_set:N  = \l_stex_key_reorder_str ,
  assoc     .choices:nn = {bin,binl,binr,pre,conj,pwconj}
    {\str_set:Ne \l_stex_key_assoc_str \l_keys_choice_tl},
  role      .str_set:N  = \l_stex_key_role_str
}{}

\stex_keys_define:nnnn{decl}{
  \str_clear:N \l_stex_key_name_str
  \str_clear:N \l_stex_key_args_str
  \tl_clear:N \l_stex_key_type_tl
  \tl_clear:N \l_stex_key_def_tl
  \tl_clear:N \l_stex_key_return_tl
  \str_clear:N \l_stex_key_wikidata_str
  \clist_clear:N \l_stex_key_argtypes_clist
}{
  name      .str_set:N    = \l_stex_key_name_str ,
  return    .tl_set:N     = \l_stex_key_return_tl ,
  argtypes  .clist_set:N  = \l_stex_key_argtypes_clist ,
  wikidata  .str_set:N    = \l_stex_key_wikidata_str ,
  type      .tl_set:N     = \l_stex_key_type_tl  ,
  def       .tl_set:N     = \l_stex_key_def_tl   ,
  align     .code:n       = {},
  gf        .code:n       = {}
}{style,deprecate,symargs}
```

\symdecl

```
\str_new:N \l_stex_macroname_str

\stex_new_stylable_cmd:nnnn {symdecl} { s m O{}} {
  \stex_keys_set:nn{decl}{#3}
  \IfBooleanTF #1 {
    \str_clear:N \l_stex_macroname_str
  }{
    \str_set:Ne \l_stex_macroname_str { #2 }
  }
  \__stex_syms_top:n{#2}
  \stex_if_smsmode:F \__stex_syms_style:
  \stex_smsmode_do:
}{}

\stex_deactivate_macro:Nn \symdecl {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \symdecl}
\stex_sms_allow_escape:N \symdecl

\cs_new_protected:Nn \__stex_syms_style: {
  \group_begin:
    \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
    \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
    \tl_set_eq:NN \thistype \l_stex_key_type_tl
    \tl_set_eq:NN \thisdefiniens \l_stex_key_def_tl
    \tl_set_eq:NN \thisargs \l_stex_key_args_str
    \tl_clear:N \thisstyle
    \stex_style_apply:
  \group_end:
}
```

(*End of definition for* `\symdecl`. *This function is documented on page 147.*)

sfunction

`\symdef`

```
\stex_new_stylable_cmd:nnnn {symdef} { m O{} m} {
  \stex_keys_set:nn{symdef}{#2}
  \str_set:Ne \l_stex_macroname_str { #1 }
  \__stex_syms_top:n{#1}
  \stex_debug:nn{symdef}{Doing~\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
  \str_set_eq:NN \l_stex_get_symbol_uri \l_stex_current_symbol_uri
  \stex_notation_parse:n{#3}
  \stex_if_check_terms:T{ \_stex_notation_check: }
  \_stex_notation_add:
  \stex_if_do_html:T{
    \_stex_notation_do_html:n{\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
  }
  \stex_if_smsmode:F \__stex_syms_def_style:
  \stex_smsmode_do:
}{}

\stex_deactivate_macro:Nn \symdef {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \symdef}
\stex_sms_allow_escape:N \symdef

```

```
\cs_new_protected:Nn \__stex_syms_def_style: {
  \group_begin:
  \tl_set:Nx \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
  \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
  \tl_set_eq:NN \thistype \l_stex_key_type_tl
  \tl_set_eq:NN \thisdefiniens \l_stex_key_def_tl
  \tl_set_eq:NN \thisargs \l_stex_key_args_str
  \tl_clear:N \thisstyle
  \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
  \def\thisnotation{
    \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs{}}{
      \_stex_notation_make_args:
    }
  }
  \stex_style_apply:
  \group_end:
}
```

(the symdef keyset is defined after notations as `\stex_keys_define:nnnn{symdef}{}{}{decl,notation}`)

(*End of definition for* `\symdef`*. This function is documented on page 147.*)

sfunction

`\textsymdecl`

```
\stex_keys_define:nnnn{textsymdecl}{
  \str_clear:N \l_stex_key_name_str
  \tl_clear:N \l_stex_key_type_tl
  \tl_clear:N \l_stex_key_def_tl
}{
  name      .str_set:N  = \l_stex_key_name_str ,
  type      .tl_set:N     = \l_stex_key_type_tl ,
  def       .tl_set:N     = \l_stex_key_def_tl,
  gf        .code:n       = {}
}{style,deprecate}

\stex_new_stylable_cmd:nnnn {textsymdecl} {m O{} m} {
  \stex_keys_set:nn{symdef}{}
  \stex_keys_set:nn{textsymdecl}{#2}
  \str_set:Ne \l_stex_macroname_str { #1 }
  \str_if_empty:NT \l_stex_key_name_str {
    \str_set:Nn \l_stex_key_name_str {#1}
  }%{
  %  \str_set:Ne \l_stex_key_name_str {\l_stex_key_name_str-sym}
  %}
  \str_set:Nn \l_stex_key_role_str {textsymdecl}
  \tl_set:Ne \l_stex_current_symbol_uri {
    \exp_args:No \stex_new_symbol_uri:n \l_stex_key_name_str
  }
  \stex_symdecl_do:
  \_stex_check_terms:
  \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnnN}{
    {\l_stex_macroname_str}
    {\l_stex_key_name_str}
    {0}{}
```

```
    {\tl_if_empty:NF \l_stex_key_def_tl{DEFED} }
    {}% type
    {\use:c{#1name_nospace}}% return
    \stex_invoke_text_symbol:
  }
  \exp_args:Ne \stex_ref_new_symbol:n
    { \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
  \stex_if_do_html:T {
    \_stex_symdecl_html:
  }

  \int_set:Nn \l_stex_get_symbol_arity_int 0
  \tl_clear:N \l_stex_key_op_tl
  \str_clear:N \l_stex_key_intent_str
  \str_clear:N \l_stex_key_prec_str
  \tl_set_eq:NN \l_stex_get_symbol_uri \l_stex_current_symbol_uri
  \stex_notation_parse:n{\hbox{#3}}
  \_stex_notation_add:
  \stex_if_do_html:T {
    \def\comp{\_comp}
    \_stex_notation_do_html:n{ \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
  }
  \stex_execute_in_module:e{
    \__stex_syms_set_textsymdecl_macro:nnn{#1}{ \stex_use_symbol_uri:N \l_stex_current_symb
    \exp_not:n{{#3}}
  }

  \stex_if_smsmode:F{
    \group_begin:
    \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
    \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
    \tl_clear:N \thisstyle
    \stex_style_apply:
    \group_end:
  }
  \stex_smsmode_do:
}{}

\stex_deactivate_macro:Nn \textsymdecl {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \textsymdecl}
\stex_sms_allow_escape:N \textsymdecl

\cs_new_protected:Nn \__stex_syms_set_textsymdecl_macro:nnn {
  \cs_set_protected:cpn{#1name_nospace}{#3}
  \cs_set_protected:cpn{#1name}{
    \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
    \mode_if_math:T\hbox{\let\xspace\relax #3}
    \mode_if_math:F{\cs_if_exist:NT\xspace\xspace}
  }
}
```

(*End of definition for* `\textsymdecl`. *This function is documented on page 147.*)

sfunction

`\__stex_syms_top:n` shared behaviour for `\symdecl` and `\symdef`; calls `\stex_symdecl_do:`, optionally checks

the term components, adds the symbol to the current module and produces the FTML for the declaration.

```
\cs_new_protected:Nn \__stex_syms_top:n {
  \str_if_empty:NT \l_stex_key_name_str {
    \str_set:Ne \l_stex_key_name_str { #1 }
  }
  \tl_set:Ne \l_stex_current_symbol_uri {
    \exp_args:No \stex_new_symbol_uri:n \l_stex_key_name_str
  }

  \stex_symdecl_do:
  \_stex_check_terms:
  \__stex_syms_add_decl:
  \stex_if_do_html:T \_stex_symdecl_html:
}
```

(*End of definition for* `\__stex_syms_top:n`.)

Adds the symbol to the current module:

```
\cs_new_protected:Nn \__stex_syms_add_decl: {
  \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnnN}{
    {\l_stex_macroname_str}
    {\l_stex_key_name_str}
    {\int_use:N \l_stex_get_symbol_arity_int}
    {\l_stex_get_symbol_args_tl}
    {\tl_if_empty:NF \l_stex_key_def_tl{DEFED} }
    {}
    {\exp_args:No \exp_not:n \l_stex_key_return_tl}
    \stex_invoke_symbol:
  }
  \exp_args:Ne \stex_ref_new_symbol:n
    {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
}
```

`\stex_symdecl_do:`

```
\cs_new_protected:Nn \stex_symdecl_do: {
  \_stex_do_deprecation:n \l_stex_key_name_str
  \__stex_syms_parse_arity:
  \__stex_syms_do_args:
}

\cs_new:Nn \_stex_return_args:nn {
  {\svar{ARGUMENT_#1}\_stex_eat_exclamation_point:}
}

\cs_new_protected:Nn \__stex_syms_do_args: {
  \tl_clear:N \l_stex_get_symbol_args_tl
  \int_step_inline:nn \l_stex_get_symbol_arity_int {
    \tl_put_right:Nn \l_stex_get_symbol_args_tl {##1}
    \tl_put_right:Ne \l_stex_get_symbol_args_tl {
      \str_item:Nn \l_stex_key_args_str {##1}
    }
  }
}
```

Constructs the arity string of the symbol:

```
\int_new:N \l_stex_assoc_args_count

\cs_new_protected:Nn \__stex_syms_parse_arity: {
  \int_zero:N \l_stex_get_symbol_arity_int
  \int_zero:N \l_stex_assoc_args_count
  \str_map_inline:Nn \l_stex_key_args_str {
    \str_case:nnF ##1 {
      0 { \str_map_break: }
      1 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {1}
        \str_set:Nn \l_stex_key_args_str {i}
      } }
      2 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {2}
        \str_set:Nn \l_stex_key_args_str {ii}
      } }
      3 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {3}
        \str_set:Nn \l_stex_key_args_str {iii}
      } }
      4 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {4}
        \str_set:Nn \l_stex_key_args_str {iiii}
      } }
      5 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {5}
        \str_set:Nn \l_stex_key_args_str {iiiii}
      } }
      6 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {6}
        \str_set:Nn \l_stex_key_args_str {iiiiii}
      } }
      7 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {7}
        \str_set:Nn \l_stex_key_args_str {iiiiiii}
      } }
      8 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {8}
        \str_set:Nn \l_stex_key_args_str {iiiiiiii}
      } }
      9 { \str_map_break:n{
        \int_set:Nn \l_stex_get_symbol_arity_int {9}
        \str_set:Nn \l_stex_key_args_str {iiiiiiiii}
      } }
      i {\int_incr:N \l_stex_get_symbol_arity_int}
      b {\int_incr:N \l_stex_get_symbol_arity_int}
      a {\int_incr:N \l_stex_get_symbol_arity_int \int_incr:N \l_stex_assoc_args_count}
      B {\int_incr:N \l_stex_get_symbol_arity_int \int_incr:N \l_stex_assoc_args_count}
    }{
      \msg_error:nnxx{stex}{error/wrongargs}{}{##1}
    }
  }
}
```

(*End of definition for* `\stex_symdecl_do:`. *This function is documented on page 147.*)

sfunction

`\_stex_symdecl_html:`

```
\cs_new_protected:Nn \_stex_symdecl_html: {
  \stex_annotate_invisible:n{
    \hbox{\exp_args:Ne \stex_annotate:nn {
      data-ftml-symdecl = { \l_stex_key_name_str},
      data-ftml-args = {\l_stex_key_args_str}
      \str_if_empty:NF \l_stex_macroname_str {,
        data-ftml-macroname={\l_stex_macroname_str}
      }
      \str_if_empty:NF \l_stex_key_wikidata_str {,
        data-ftml-wikidata={\l_stex_key_wikidata_str}
      }
      \str_if_empty:NF \l_stex_key_assoc_str {,
        data-ftml-assoctype={\l_stex_key_assoc_str}
      }
      \str_if_empty:NF \l_stex_key_reorder_str {,
        data-ftml-reorderargs={\l_stex_key_reorder_str}
      }
      \str_if_empty:NF \l_stex_key_role_str {,
        data-ftml-role={\l_stex_key_role_str}
      }
    }{\_stex_annotate_force_break:n{
      \bool_set_true:N \stex_in_invisible_html_bool
      \tl_if_empty:NF \l_stex_key_type_tl {
        $\stex_annotate:nn{data-ftml-type={}}{\l_stex_key_type_tl}$
      }
      \tl_if_empty:NF \l_stex_key_def_tl {
        $\stex_annotate:nn{data-ftml-definiens={}}{\l_stex_key_def_tl}$
      }
      \tl_if_empty:NF \l_stex_key_return_tl{
        \exp_args:Nno \use:n{
        \cs_generate_from_arg_count:NNnn \l__stex_syms_cs
        \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
        \tl_set:Ne \l__stex_syms_args_tl {\_stex_map_args:N \_stex_return_args:nn}
        $\stex_annotate:nn{data-ftml-returntype={}}{
          \exp_after:wN \l__stex_syms_cs \l__stex_syms_args_tl!
        }$
      }
      \clist_if_empty:NF \l_stex_key_argtypes_clist {
        \stex_annotate:nn{data-ftml-argtypes={}}{\_stex_annotate_force_break:n{
          \clist_map_inline:Nn \l_stex_key_argtypes_clist {
              $\stex_annotate:nn{data-ftml-type={}}{##1}$
          }
        }}
      }
    }}
  }}
}
```

(*End of definition for* `\_stex_symdecl_html:`. *This function is documented on page 147.*)

sfunction

\stex_add_symbol:nnnnnnN

```
\cs_new_protected:Nn \stex_add_symbol:nnnnnnnN {
  \stex_debug:nn{declaration}{New~declaration:~\stex_use_module_uri:N \l_stex_current_modul
    Macro:#1^^JArity:#3~(#4)^^J
    Def:~\tl_to_str:n{#5}^^J
    Type:~\tl_to_str:n{#6}^^J
    Returns:~\tl_to_str:n{#7}^^J
    Invokes:~\tl_to_str:n{#8}
  }
  \prop_gput:cnn{ \_stex_symbol_macro:N \l_stex_current_module_uri }
  {#2}{{#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}}
  \tl_if_empty:nF{#1}{
    \stex_do_up_to_module:n {
      \__stex_syms_activate_i:nnnnnnnn{#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}
    }
  }
}
```

Generate semantic macros etc.:

```
\cs_new_protected:Nn \_stex_activate_symbols: {
  \stex_debug:nn{activating}{All~symbols:~\cs_meaning:c{ \_stex_symbol_macro:N \l_stex_curr
  \prop_map_inline:cn{ \_stex_symbol_macro:N \l_stex_current_module_uri }{
    \__stex_syms_maybe_activate:nnnnnnnn ##2
  }
}

\cs_new_protected:Nn \__stex_syms_maybe_activate:nnnnnnnn {
  \str_if_empty:nF{#1}{
    \__stex_syms_activate_i:nnnnnnnn {#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}
  }
}

\cs_new_protected:Nn \__stex_syms_activate_i:nnnnnnnn {
  \stex_debug:nn{activating}{macro~#1:\stex_use_module_uri:N \l_stex_current_module_uri ? #
    \tl_to_str:n{{#3}{#4}{#5}{#6}{#7}#8}
  }
  \cs_set:cpe{#1} {
    \_stex_invoke_symbol:nnnnnnN
    {\exp_args:No \stex_new_symbol_uri:nn {\l_stex_current_module_uri} {#2} }
    \exp_not:n{{#3}{#4}{#5}{#6}{#7}{#8}}
  }
}
```

(*End of definition for* `\stex_add_symbol:nnnnnnN`. *This function is documented on page 147.*)

sfunction

\stex_has_definiens_p:N
\stex_has_definiens:N*TF*
\stex_has_definiens_p:n
\stex_has_definiens:n*TF*

```
\prg_new_conditional:Nnn \stex_has_definiens:N { p, T, F, TF } {
  \exp_args:No \stex_has_definiens:nTF #1 \prg_return_true: \prg_return_false:
}
\prg_new_conditional:Nnn \stex_has_definiens:n { p, T, F, TF } {
  \exp_args:Nne \use:nn{\__stex_syms_has_definiens:nnnnnnnN}{\exp_args:Nne \prop_item:cn{
    \exp_args:Ne \_stex_symbol_macro:n {\stex_symbol_uri_module:n{#1}}
  }{
```

```
    \stex_symbol_uri_name:n {#1}
  }}
}

\cs_new:Nn \__stex_syms_has_definiens:nnnnnnnN {
  \tl_if_empty:nTF{#5}\prg_return_false:\prg_return_true:
}
```

(*End of definition for* `\stex_has_definiens:NTF` *and* `\stex_has_definiens:nTF`*. These functions are documented on page 147.*)

sfunction

32.2 Retrieval

`\stex_iterate_symbols:n`
`\stex_iterate_break:`
`\stex_iterate_break:n`

```
\cs_new_protected:Nn \stex_iterate_symbols:n {
  \stex_pseudogroup_with:nn{\__stex_syms_sym_cs:nnnnnnnnN\stex_iterate_break:\stex_iterate_
    \cs_set:Npn \__stex_syms_sym_cs:nnnnnnnnN
    ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #1 }
    \cs_set:Npn \stex_iterate_break: {
      \prop_map_break:n{\seq_map_break:}
    }
    \cs_set:Npn \stex_iterate_break:n ##1 {
      \prop_map_break:n{\seq_map_break:n{##1}}
    }
    \seq_map_inline:Nn \l_stex_all_modules_seq {
      \prop_map_inline:cn{\_stex_symbol_macro:n {##1}}{
        \__stex_syms_sym_cs:nnnnnnnnN {##1} ####2
      }
    }
  }
}
```

(*End of definition for* `\stex_iterate_symbols:n`*,* `\stex_iterate_break:`*, and* `\stex_iterate_break:n`*. These functions are documented on page 148.*)

sfunction

`\stex_iterate_symbols:nn`

```
\cs_new_protected:Nn \stex_iterate_symbols:nn {
  \seq_clear:N \l__stex_syms_mods_seq
  \stex_pseudogroup_with:nn{\__stex_syms_sym_cs:nnnnnnnnN}{
    \cs_set:Npn \__stex_syms_sym_cs:nnnnnnnnN
    ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #2 }
    \clist_map_function:nN {#1} \__stex_syms_it_decl_i:n
  }
}

\cs_new_protected:Nn \__stex_syms_it_decl_i:n {
  \seq_if_in:NnF \l__stex_syms_mods_seq {#1} {
    \seq_put_left:Nn \l__stex_syms_mods_seq {#1}
    \prop_map_inline:cn{\_stex_morphisms_macro:n {#1} }{
      \__stex_syms_it_decl_check:nnnn ##2
    }
```

```
    \prop_map_inline:cn{\_stex_symbol_macro:n {#1} }{
      \__stex_syms_sym_cs:nnnnnnnnN {#1} ##2
    }
  }
}

\cs_new_protected:Nn \__stex_syms_it_decl_check:nnnn {
  \tl_if_empty:nT{#1}{
    \__stex_syms_it_decl_i:n {#2}
  }
}
```

(*End of definition for* `\stex_iterate_symbols:nn`. *This function is documented on page 148.*)

sfunction

`\stex_get_symbol:n`
`\stex_get_symbol:nF`

```
\int_new:N \l_stex_get_symbol_arity_int

\cs_new_protected:Nn \stex_get_symbol:n {
  \stex_get_symbol:nF{ #1 }{
    \msg_error:nnn{stex}{error/unknownsymbol}{#1}
  }
}

\cs_new_protected:Npn \stex_get_symbol:nF #1 #2 {
  \tl_clear:N \l_stex_get_symbol_uri
  \cs_if_exist:cTF { #1 }{
    \cs_set_eq:Nc \l__stex_syms_cs { #1 }
    % command name
    \exp_args:Ne \tl_if_empty:nTF { \cs_argument_spec:N \l__stex_syms_cs }{
      % ...that takes no arguments
      \exp_args:Ne \cs_if_eq:NNTF {\tl_head:N \l__stex_syms_cs}
        \_stex_invoke_symbol:nnnnnnN
        \__stex_syms_get_symbol_from_cs:
        {\__stex_syms_get_symbol_from_string:n { #1 }}
    }{
      \__stex_syms_get_symbol_from_string:n { #1 }
    }
  }{
    \__stex_syms_get_symbol_from_string:n { #1 }
  }
  \tl_if_empty:NT \l_stex_get_symbol_uri { #2 }
}

\cs_new_protected:Nn \__stex_syms_get_symbol_from_cs: {
  \stex_pseudogroup_with:nn{\_stex_invoke_symbol:nnnnnnN}{
    \cs_set:Npn \_stex_invoke_symbol:nnnnnnN ##1 ##2 ##3 ##4 ##5 ##6 ##7 {
      \tl_set:Nn \l_stex_get_symbol_uri { ##1 }
      \int_set:Nn \l_stex_get_symbol_arity_int {##2}
      \tl_set:Nn \l_stex_get_symbol_args_tl {##3}
      \tl_set:Nn \l_stex_get_symbol_def_tl {##4}
      \tl_set:Nn \l_stex_get_symbol_type_tl {##5}
      \tl_set:Nn \l_stex_get_symbol_return_tl {##6}
      \tl_set:Nn \l_stex_get_symbol_invoke_cs {##7}
```

```
    }
    \l__stex_syms_cs
  }
}

\cs_new_protected:Nn \__stex_syms_get_symbol_from_string:n {
  \stex_debug:nn{symbols}{Getting~from~string~#1...}
  \seq_set_split:Nnn \l__stex_syms_seq ? {#1}
  \seq_pop_right:NN \l__stex_syms_seq \l__stex_syms_name
  \seq_if_empty:NTF \l__stex_syms_seq {
    \exp_args:No \__stex_syms_get_from_one_string:n {#1}
  }{
    \exp_args:NNe \exp_args:Nno \__stex_syms_get_symbol_from_modules:nn {
      \seq_use:Nn \l__stex_syms_seq ?
    } \l__stex_syms_name
  }
}

\cs_new_protected:Nn \__stex_syms_sym_from_str_i:nnnn {
  \bool_lazy_any:nTF{
    {\str_if_eq_p:nn{#2}{#3}}
    {\str_if_eq_p:nn{#2}{#4}}
    {\stex_str_if_ends_with_p:nn{#4}{/#2}}
    }{
      \__stex_syms_sym_i_finish:nnnnnnnN{#1}{#4}
    }{
      \__stex_syms_sym_i_gobble:nnnnnn
    }
}
\cs_new_protected:Nn \__stex_syms_sym_i_gobble:nnnnnn {}

\cs_new_protected:Nn \__stex_syms_sym_i_finish:nnnnnnnN {
  \prop_map_break:n{\seq_map_break:n{
    \tl_set:Ne \l_stex_get_symbol_uri { \stex_new_symbol_uri:nn {#1} {#2}}
    \int_set:Nn \l_stex_get_symbol_arity_int {#3}
    \tl_set:Nn \l_stex_get_symbol_args_tl {#4}
    \tl_set:Nn \l_stex_get_symbol_def_tl {#5}
    \tl_set:Nn \l_stex_get_symbol_type_tl {#6}
    \tl_set:Nn \l_stex_get_symbol_return_tl {#7}
    \tl_set:Nn \l_stex_get_symbol_invoke_cs {#8}
  }}
}

\cs_new_protected:Nn \__stex_syms_get_symbol_from_modules:nn {
  %\seq_set_split:Nnn \l_tmpa_seq ? {#1}
  %\seq_pop_right:NN \l_tmpa_seq \l__stex_syms_name_str
  %\str_set:Ne \l__stex_syms_path_str {\seq_use:Nn \l_tmpa_seq ?}
  \stex_debug:nn{symbols}{Getting~#2~in~#1...}
  \seq_map_inline:Nn \l_stex_all_modules_seq {
    %\stex_debug:nn{symbols}{comparing~\stex_module_uri_as_qm:n{##1}~and~#1}
    \exp_args:Ne \stex_str_if_ends_with:nnT{\stex_module_uri_as_qm:n{##1}}{#1}{
      \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
        \__stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
      }
```

```
    }

    %\str_if_empty:NTF \l__stex_syms_path_str {
    %  \exp_args:NNe \exp_args:Nno \stex_str_if_ends_with:nnT {\stex_module_uri_name:n{##1}
    %}{
    %  \exp_args:NNNe \exp_args:No \str_if_eq:nnT\l__stex_syms_name_str{
    %    \stex_module_uri_name:n {##1}
    %  }
    %}{
    %  \str_if_empty:NTF \l__stex_syms_path_str {
    %    \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
    %      \__stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
    %    }
    %  }{
    %    \stex_debug:nn{symbols}{Comparing~\l__stex_syms_path_str;~with~\tl_to_str:n{##1}}
    %    \exp_args:NNe \exp_args:Nno \stex_str_if_ends_with:nnT{\stex_module_uri_path:n {##
    %      \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
    %        \__stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
    %      }
    %    }
    %  }
    %}
  }
}

\prg_new_conditional:Nnn \__stex_syms_uri_match:n {T,F,TF} {
  \str_if_eq:nnT
}


\cs_new_protected:Nn \__stex_syms_get_from_one_string:n {
  \stex_debug:nn{symbols}{Getting~#1~anywhere...}
  \stex_iterate_symbols:n{
    %\stex_debug:nn{symbols}{>#1==##2~|~#1==##3<...}
    \bool_lazy_any:nT{
      {\str_if_eq_p:nn{#1}{##2}}
      {\str_if_eq_p:nn{#1}{##3}}
      {\stex_str_if_ends_with_p:nn{##3}{/#1}}
      }{
        \stex_iterate_break:n{
          \tl_set:Ne \l_stex_get_symbol_uri { \stex_new_symbol_uri:nn {##1} {##3}}
          \int_set:Nn \l_stex_get_symbol_arity_int {##4}
          \tl_set:Nn \l_stex_get_symbol_args_tl {##5}
          \tl_set:Nn \l_stex_get_symbol_def_tl {##6}
          \tl_set:Nn \l_stex_get_symbol_type_tl {##7}
          \tl_set:Nn \l_stex_get_symbol_return_tl {##8}
          \tl_set:Nn \l_stex_get_symbol_invoke_cs {##9}
        }
    }
  }
}
```

(*End of definition for* `\stex_get_symbol:n` *and* `\stex_get_symbol:nF`*. These functions are documented on page 148.*)

sfunction

32.3 Variables

```
⟨@@=stex_vars⟩
```

\l_stex_variables_prop

```
\prop_new:N \l_stex_variables_prop
```

(*End of definition for* `\l_stex_variables_prop`. *This variable is documented on page 148.*)

\vardef

```
\bool_new:N \l__stex_vars_bind_bool
\cs_new_protected:Nn \_stex_variable:nnnnnnnN {}

\stex_new_stylable_cmd:nnnn {vardef} { m O{} m} {
  \stex_keys_set:nn{vardef}{#2}
  \str_set:Ne \l_stex_macroname_str { #1 }
  \str_if_empty:NT \l_stex_key_name_str {
    \str_set:Ne \l_stex_key_name_str { #1 }
  }

  \stex_symdecl_do:
  \stex_if_check_terms:T \_stex_check_terms:
  \__stex_vars_add:
  \__stex_vars_macro:
  \stex_if_do_html:T \__stex_vars_html:

  \int_set:Nn \l_stex_get_symbol_arity_int {\l_stex_get_symbol_arity_int}
  \stex_debug:nn{vardef}{Doing~\l_stex_key_name_str}
  \tl_set_eq:NN \l_stex_get_symbol_return_tl \l_stex_key_return_tl
  \stex_notation_parse:n{#3}
  \stex_if_check_terms:T{ \_stex_notation_check: }
  \_stex_var_notation_macro:
  \stex_if_do_html:T {
    \def\comp{\_varcomp}
    \_stex_notation_do_html:n \l_stex_key_name_str
  }
  \group_begin:
  \tl_set_eq:NN \thisvarname \l_stex_key_name_str
  \tl_clear:N \thisstyle
  \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
  \def\thisnotation{
    \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs{}}{
      \_stex_notation_make_args:
    }
  }
  \stex_style_apply:
  \group_end:\ignorespaces
}{}
```

(*End of definition for* `\vardef`. *This function is documented on page 148.*)

sfunction

Adds a new variable to the current scope:

```
\cs_new_protected:Nn \__stex_vars_add: {
  \exp_args:NNNo \exp_args:NNne
  \prop_put:Nnn \l_stex_variables_prop \l_stex_key_name_str {
```

```
    {\l_stex_macroname_str}
    {\l_stex_key_name_str}
    {\int_use:N \l_stex_get_symbol_arity_int}
    {\l_stex_get_symbol_args_tl}
    {\exp_args:No \exp_not:n \l_stex_key_def_tl}
    {\exp_args:No \exp_not:n \l_stex_key_type_tl}
    {\exp_args:No \exp_not:n \l_stex_key_return_tl}
    \stex_invoke_symbol:
  }
}
```

Defines the macro for a variable:

```
\cs_new_protected:Nn \__stex_vars_macro: {
  \tl_set:ce{\l_stex_macroname_str}{
    \_stex_invoke_variable:nnnnnnN
      {\l_stex_key_name_str}
      {\int_use:N \l_stex_get_symbol_arity_int}
      {\l_stex_get_symbol_args_tl}
      {\exp_args:No \exp_not:n \l_stex_key_def_tl}
      {\exp_args:No \exp_not:n \l_stex_key_type_tl}
      {\exp_args:No \exp_not:n \l_stex_key_return_tl}
      \stex_invoke_symbol:
  }
}
```

Insert the HTML for a variable declaration:

```

\cs_new_protected:Nn \__stex_vars_html: {
  \stex_if_do_html:T {
    \hbox\bgroup\exp_args:Ne \stex_annotate_invisible:nn {
      data-ftml-vardef = {\l_stex_key_name_str},
      data-ftml-args = {\l_stex_key_args_str}
      \str_if_empty:NF \l_stex_macroname_str {,
        data-ftml-macroname={\l_stex_macroname_str}
      }
      \str_if_empty:NF \l_stex_key_assoc_str {,
        data-ftml-assoctype={\l_stex_key_assoc_str}
      }
      \str_if_empty:NF \l_stex_key_role_str {,
        data-ftml-role={\l_stex_key_role_str}
      }
      \str_if_empty:NF \l_stex_key_reorder_str {,
        data-ftml-reorderargs={\l_stex_key_reorder_str}
      }
      \bool_if:NT \l__stex_vars_bind_bool {,
        data-ftml-bind={true}
      }
    }{
      \_stex_annotate_force_break:n{
        \bool_set_true:N \stex_in_invisible_html_bool
        \tl_if_empty:NF \l_stex_key_type_tl {
          \stex_annotate:nn{data-ftml-type={}}{$\l_stex_key_type_tl$}
        }
        \tl_if_empty:NF \l_stex_key_def_tl {
          \stex_annotate:nn{data-ftml-definiens={}}{$\l_stex_key_def_tl$}
```

```
        }
        \tl_if_empty:NF \l_stex_key_return_tl{
          \exp_args:Nno \use:n{
          \cs_generate_from_arg_count:NNnn \l__stex_vars_cs
          \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
          \tl_set:Ne \l__stex_vars_args_tl {\_stex_map_args:N \_stex_return_args:nn}
          $\stex_annotate:nn{data-ftml-returntype={}}{
            \exp_after:wN \l__stex_vars_cs \l__stex_vars_args_tl!}$
        }
        \tl_if_empty:NF \l_stex_key_argtypes_clist {
          \stex_annotate:nn{data-ftml-argtypes={}}{
            \_stex_annotate_force_break:n{
              \clist_map_inline:Nn \l_stex_key_argtypes_clist {
                $\stex_annotate:nn{data-ftml-type={}}{##1}$
              }
            }
          }
        }
      }
    }\egroup
  }
}
```

\stex_get_var:n

```
\cs_new_protected:Nn \stex_get_var:n {
  \str_clear:N \l_stex_get_variable_str
  \__stex_vars_get_var:n{#1}
  \str_if_empty:NT \l_stex_get_variable_str {
    \msg_error:nnn{stex}{error/unknownsymbol}{#1}
  }
}

\cs_new_protected:Nn \__stex_vars_get_var:n {
  \prop_map_inline:Nn \l_stex_variables_prop {
    \__stex_vars_check_var:nnnnnnnnN {#1} ##2
  }
}

\cs_new_protected:Nn \__stex_vars_check_var:nnnnnnnnN {
  \str_if_eq:nnTF{#1}{#2}{
    \prop_map_break:n{\__stex_vars_set_vars:nnnnnnN {#3}{#4}{#5}{#6}{#7}{#8}#9}
  }{
    \str_if_eq:nnT{#1}{#3}{
      \prop_map_break:n{\__stex_vars_set_vars:nnnnnnN {#3}{#4}{#5}{#6}{#7}{#8}#9}
    }
  }
}

\cs_new_protected:Nn \__stex_vars_set_vars:nnnnnnN {
  \stex_debug:nn{symbols}{Variable~#1~found}
  \cs_set:Npn \_stex_variable:nnnnnnnN ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 {}
  \str_set:Nn \l_stex_get_variable_str {#1}
  \int_set:Nn \l_stex_get_symbol_arity_int {#2}
  \tl_set:Nn \l_stex_get_symbol_args_tl {#3}
```

```
  \tl_set:Nn \l_stex_get_symbol_def_tl {#4}
  \tl_set:Nn \l_stex_get_symbol_type_tl {#5}
  \tl_set:Nn \l_stex_get_symbol_return_tl {#6}
  \tl_set:Nn \l_stex_get_symbol_invoke_cs {#7}
}
```

(*End of definition for* `\stex_get_var:n`. *This function is documented on page 148.*)

sfunction

32.3.1 Variable Sequences

```
⟨@@=stex_seqs⟩
```

`\varseq`

```
\stex_new_stylable_cmd:nnnn {varseq}{m O{} m m} {
  \stex_keys_set:nn{symdef}{#2}
  \str_set:Ne \l_stex_macroname_str { #1 }
  \str_if_empty:NT \l_stex_key_name_str {
    \str_set:Ne \l_stex_key_name_str { #1 }
  }
  \str_if_empty:NT \l_stex_key_args_str {
    \str_set:Nn \l_stex_key_args_str {1}
  }
  \stex_symdecl_do:

  \tl_set_eq:NN \l_stex_get_symbol_return_tl \l_stex_key_return_tl
  \clist_set:Nn \l__stex_seqs_range_clist {#3}
  \tl_if_empty:NTF \l_stex_key_op_tl {
    \stex_notation_parse:n{#4}
    \tl_clear:N \l_stex_key_op_tl
  }{
    \stex_notation_parse:n{#4}
  }
  \stex_if_do_html:T \__stex_seqs_html:
  \stex_if_check_terms:T \_stex_check_terms:
  \__stex_seqs_add:
  \__stex_seqs_macro:
  \stex_if_check_terms:T \_stex_notation_check:
  \_stex_var_notation_macro:
  \group_begin:
  \tl_set_eq:NN \thisvarname \l_stex_key_name_str
  \tl_clear:N \thisstyle
  \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
  \def\thisnotation{
    \l_stex_notation_macrocode_cs{}!
  }
  \stex_style_apply:
  \group_end:\ignorespaces
}{}

\cs_new_protected:Nn \__stex_seqs_add: {
  \exp_args:NNNo \exp_args:NNne
  \prop_put:Nnn \l_stex_variables_prop \l_stex_key_name_str {
    {\l_stex_macroname_str}
```

```
    {\l_stex_key_name_str}
    {\int_use:N \l_stex_get_symbol_arity_int}
    {\l_stex_get_symbol_args_tl}
    {\exp_args:No \exp_not:n \l_stex_key_def_tl}
    {\exp_args:No \exp_not:n \l__stex_seqs_range_clist}
    {\exp_args:No \exp_not:n \l_stex_key_return_tl}
    \stex_invoke_sequence:
  }
}

\cs_new_protected:Nn \__stex_seqs_macro: {
  \tl_set:ce{\l_stex_macroname_str}{
    \_stex_invoke_variable:nnnnnnN
      {\l_stex_key_name_str}
      {\int_use:N \l_stex_get_symbol_arity_int}
      {\l_stex_get_symbol_args_tl}
      {\exp_args:No \exp_not:n \l_stex_key_def_tl}
      {\exp_args:No \exp_not:n \l__stex_seqs_range_clist}
      {\exp_args:No \exp_not:n \l_stex_key_return_tl}
      \stex_invoke_sequence:
  }
}

\cs_new_protected:Nn \__stex_seqs_html: {
    \exp_args:Ne \stex_annotate_invisible:nn {
      data-ftml-varseq = {\l_stex_key_name_str},
      data-ftml-args = {\l_stex_key_args_str}
      \str_if_empty:NF \l_stex_macroname_str {,
        data-ftml-macroname={\l_stex_macroname_str}
      }
      \str_if_empty:NF \l_stex_key_assoc_str {,
        data-ftml-assoctype={\l_stex_key_assoc_str}
      }
      \str_if_empty:NF \l_stex_key_role_str {,
        data-ftml-role={\l_stex_key_role_str}
      }
      \str_if_empty:NF \l_stex_key_reorder_str {,
        data-ftml-reorderargs={\l_stex_key_reorder_str}
      }
    }{\hbox\bgroup
      \_stex_annotate_force_break:n{
        \tl_if_empty:NF \l_stex_key_type_tl {
          \stex_annotate:nn{data-ftml-type={}}{$\l_stex_key_type_tl$}
        }
        \tl_if_empty:NF \l_stex_key_def_tl {
          \stex_annotate:nn{data-ftml-definiens={}}{$\l_stex_key_def_tl$}
        }
        \tl_if_empty:NF \l_stex_key_return_tl{
          \exp_args:Nno \use:n{
          \cs_generate_from_arg_count:NNnn \l__stex_seqs_cs
          \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
          \tl_set:Nx \l__stex_seqs_args_tl {\_stex_map_args:N \_stex_return_args:nn}
          \stex_annotate:nn{data-ftml-returntype={}}{
            $\exp_after:wN \l__stex_seqs_cs \l__stex_seqs_args_tl!$}
```

```
        }
        \tl_if_empty:NF \l_stex_key_argtypes_clist {
          \stex_annotate:nn{data-ftml-argtypes={}}{
            \_stex_annotate_force_break:n{
              \clist_map_inline:Nn \l_stex_key_argtypes_clist {
                \stex_annotate:nn{data-ftml-type={}}{$##1$}
              }
            }
          }
        }
      }
    \egroup}
}
```

(*End of definition for* \varseq. *This function is documented on page 149.*)

sfunction

32.4 Expressions

```
⟨@@=stex_expr⟩
```

\symuse

```
\cs_new_protected:Npn \symuse #1 {
  \stex_get_symbol:n{#1}
  \exp_args:Nno \use:n {\_stex_invoke_symbol:NeoooooN
  \l_stex_get_symbol_uri
  {\int_use:N \l_stex_get_symbol_arity_int}
  \l_stex_get_symbol_args_tl
  \l_stex_get_symbol_def_tl
  \l_stex_get_symbol_type_tl
  \l_stex_get_symbol_return_tl}
  \l_stex_get_symbol_invoke_cs
}
```

(*End of definition for* \symuse. *This function is documented on page 149.*)

sfunction

_stex_next_symbol:n

```
\tl_new:N \l__stex_expr_reset_tl

\cs_new_protected:Nn \_stex_next_symbol:n {
  \tl_set:Ne \l_stex_every_symbol_tl {
    \tl_gset:Nn \exp_not:N \l__stex_expr_reset_tl {
      \tl_set:Nn \exp_not:N \l_stex_every_symbol_tl  {
        \exp_args:No \exp_not:n \l_stex_every_symbol_tl
      }
      \tl_gset:Nn \exp_not:N \l__stex_expr_reset_tl {
        \exp_args:No \exp_not:n \l__stex_expr_reset_tl
      }
    }
    \tl_set:Nn \exp_not:N \l_stex_every_symbol_tl  {
      \exp_args:No \exp_not:n \l_stex_every_symbol_tl
    }
    \exp_not:n{ \aftergroup \l__stex_expr_reset_tl }
```

```
    \exp_not:N \l_stex_every_symbol_tl
    \exp_not:n{ #1 }
  }
}
\cs_generate_variant:Nn \_stex_next_symbol:n {e}
```

(*End of definition for* `\_stex_next_symbol:n`. *This function is documented on page 149.*)

sfunction

`\_stex_invoke_symbol:NnnnnnN`
`\_stex_invoke_symbol:NeooooN`
`\_stex_invoke_symbol:nnnnnnN`

- `\l_stex_current_symbol_uri`,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- definiens (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invokation macro (is called after setup).

```
\cs_new_protected:Nn \_stex_invoke_symbol:nnnnnnN {
  \bool_if:NTF \l_stex_allow_semantic_bool{
    \group_begin:
    \tl_set:Nn \l_stex_current_symbol_uri {#1}
    \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_current_symbol_uri
    \tl_set:Nn \l_stex_current_display_tl {
      \stex_symbol_uri_name:N \l_stex_current_symbol_uri
    }
    \__stex_expr_invoke:nnnnN{#2}{#3}{#5}{#6}{#7}
  }{
    \msg_error:nnxx{stex}{error/notallowed}{
      \stex_use_symbol_uri:n{#1}
    }{
      \l_stex_current_full_tl
    }
  }
}

\cs_new_protected:Npn \_stex_invoke_symbol:NnnnnnN {
  \exp_args:No \_stex_invoke_symbol:nnnnnnN
}
\cs_generate_variant:Nn \_stex_invoke_symbol:NnnnnnN {NeooooN}
```

Whether semantic macros are allowed currently. This is false e.g. in the body of a semantic macro outside of one of its arguments:

```
\bool_new:N \l_stex_allow_semantic_bool
\bool_set_true:N \l_stex_allow_semantic_bool
```

Code to execute every time a semantic macro is invoked:

```
\tl_set:Nn \l_stex_every_symbol_tl {
  \bool_set_false:N \l_stex_allow_semantic_bool
}
```

The current top-level term; may be undefined:

```
\tl_new:N \l_stex_current_term_tl
```

Keeps track of all set up so it can be reexecuted; this may be necessary to e.g. typeset `\this` in a struct field:

```
\tl_new:N \l_stex_current_redo_tl
```

Setup for symbols; takes: arity, args, type, return, invoke. Calls invoke at the end.

In HTML mode, we make sure we enter horizontal mode *before* any annotations are inserted

```
\cs_new_protected:Nn \__stex_expr_invoke:nnnnN {
  \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
  \__stex_expr_setup:nnnnn{\_comp}{#1}{#2}{#4}{#3}
  \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
  \tl_put_right:Nn \l_stex_current_redo_tl{
    \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
  }
  #5
}
```

general setup; takes: comp, arity, args, return, type

```
\cs_new_protected:Nn \__stex_expr_setup:nnnnn {
  \tl_clear:N \l_stex_return_notation_tl
  \tl_set:Nn \l_stex_current_redo_tl {
    \let \this \stex_current_this:
    \def\comp{#1}
    \def\maincomp{\comp}
    \str_set:Nn \l_stex_current_arity_str{ #2 }
    \tl_set:Nn \l_stex_current_args_tl{ #3 }
    \tl_set:Nn \l_stex_current_return_tl{ #4 }
    \tl_set:Nn \l_stex_current_type_tl{ #5 }
    \tl_clear:N \l_stex_current_term_tl
  }
  \tl_put_right:No \l_stex_current_redo_tl \l_stex_every_symbol_tl
  \tl_put_left:Ne \l_stex_current_redo_tl {
    \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
  }
  \l_stex_current_redo_tl
}
```

(*End of definition for* `\_stex_invoke_symbol:NnnnnnN` *and* `\_stex_invoke_symbol:nnnnnnN`*. These functions are documented on page 149.*)

sfunction

`\stex_invoke_symbol:` Math or text mode?

```
\cs_new_protected:Nn \stex_invoke_symbol: {
  \stex_debug:nn{expressions}{Invoking~ \l_stex_current_full_tl}
  \mode_if_math:TF \__stex_expr_math: \__stex_expr_text:
}
```

(*End of definition for* `\stex_invoke_symbol:`*. This function is documented on page 149.*)

sfunction

`\stex_invoke_text_symbol:`

```
\cs_new_protected:Nn \stex_invoke_text_symbol: {
  \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
  \_stex_term_oms_or_omv:nn{}{\maincomp{\let\xspace\relax\l_stex_current_return_tl}}
  \group_end:\mode_if_math:F{\cs_if_exist:NT\xspace\xspace}
}
```

(*End of definition for* `\stex_invoke_text_symbol:`. *This function is documented on page 149.*)

sfunction

`\stex_invoke_sequence:`

```
\cs_new_protected:Nn \stex_invoke_sequence: {
  \peek_charcode_remove:NTF ! {
    \peek_charcode:NTF [ \__stex_expr_seq_op:w { \__stex_expr_seq_op:w [] }
  }\__stex_expr_seq_first:
}

\cs_new_protected:Npn \__stex_expr_seq_op:w [#1] {
  \stex_use_op_notation:nnTF \l_stex_current_full_tl{#1}{
    %\stex_debug:nn{vars}{Here: \meaning\l_stex_notation_cs}
    \_stex_maybe_brackets:nn{\neginfprec}{
      \_stex_term_oms_or_omv:nn{#1}
      {\l_stex_notation_cs}\group_end:
    }
  }{
    \__stex_expr_get_index_notation:n{#1}
    \peek_charcode:NTF [ \__stex_expr_range:w { \__stex_expr_range:w[] }
  }
}

\cs_new_protected:Nn \__stex_expr_get_index_notation:n {
  \stex_use_notation:nnTF \l_stex_current_full_tl{#1}{}{
    \stex_do_default_notation:
    \cs_set_eq:NN \l_stex_notation_cs \l_stex_default_notation
  }
}

\cs_new_protected:Npn \__stex_expr_range:w [#1] {
  \bool_set_true:N \l_stex_allow_semantic_bool
  \clist_clear:N \l__stex_expr_clist
  \clist_map_function:NN \l_stex_current_type_tl \__stex_expr_seq_arg:n
    \stex_annotate:nn{
      data-ftml-term=OMV,
      data-ftml-head={\l_stex_current_full_tl},
      data-ftml-notationid={}
    }{
      \l__stex_expr_clist
    }
    \group_end:
}

\cs_new_protected:Nn \__stex_expr_seq_arg:n {
  \tl_if_eq:nnTF{#1}{\ellipses}{
    \clist_put_right:Nn \l__stex_expr_clist {
        \ellipses
    }
  }{
    \clist_put_right:Nn \l__stex_expr_clist {
      \exp_args:No \str_if_eq:nnTF \l_stex_current_arity_str {1}{
        \group_begin:
          \l_stex_notation_cs \group_end: {#1}
```

```
      }{
        \group_begin:
          \l_stex_notation_cs \group_end: #1
      }
    }
  }
}

\cs_new_protected:Nn \__stex_expr_seq_first: {
  \exp_args:Nne \use:nn{
  \cs_generate_from_arg_count:NNnn \l_stex_notation_cs \cs_set:Npn
  \l_stex_current_arity_str} {{
    \tl_set:Nn \exp_not:N \l__stex_expr_first_args_tl {
      \int_step_function:nN \l_stex_current_arity_str \__stex_expr_do_first_arg:n
    }
    \exp_not:N \__stex_expr_do_first_next:
  }}
  \l_stex_notation_cs
}

\cs_new:Nn \__stex_expr_do_first_arg:n {{\exp_not:n{## #1}}}

\cs_new_protected:Nn \__stex_expr_do_first_next: {
  \peek_charcode_remove:NTF ! {
    \peek_charcode:NTF [ \__stex_expr_do_one:w {\__stex_expr_do_one:w []}
  }{
    \peek_charcode:NTF [ \__stex_expr_do_all:w {\__stex_expr_do_all:w []}
  }
}

\cs_new_protected:Npn \__stex_expr_do_one:w [#1] {
  \__stex_expr_get_index_notation:n{#1}
  \exp_args:Nno\use:nn{\l_stex_notation_cs\group_end:}\l__stex_expr_first_args_tl
}

\cs_new_protected:Npn \__stex_expr_do_all:w [#1] {
  \exp_args:Nno\use:nn{\__stex_expr_notation:w [#1]}\l__stex_expr_first_args_tl
}
```

(*End of definition for* `\stex_invoke_sequence:`. *This function is documented on page 149.*)

sfunction

`\stex_invoke_structure:`

```
\cs_new_protected:Nn \stex_invoke_structure: {
  \tl_set:Nn \l__stex_expr_set_comp_tl {\__stex_expr_set_thiscomp:}
  \__stex_expr_struct_top:n {}
}

\cs_new_protected:Nn \__stex_expr_set_thiscomp: {
  \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_thiscomp {
    \edef\maincomp {\_thiscomp{\comp}}
  }
}
```

```
\cs_new_protected:Nn \__stex_expr_struct_top:n {
  \stex_debug:nn{structure}{
    invoking~structure~{\l_stex_current_type_tl}<\tl_to_str:n{#1}>
  }
  \peek_charcode:NTF [ {
    \__stex_expr_merge:nw{#1}
  }{
    \__stex_expr_struct_type:n {#1}
    \tl_set:Nn \l_stex_struct_this_tl {}
    \peek_charcode_remove:NTF ! {
      \peek_charcode:NTF [ {
        \__stex_expr_maybe_notation:w
      }{
        \__stex_expr_maybe_notation:w []
      }
    }{
      \__stex_expr_invoke_this:n
    }
  }
}

\cs_new_protected:Npn \__stex_expr_merge:nw #1 [ #2 ] {
  \exp_args:Ne \stex_str_if_starts_with:nnTF {\tl_to_str:n{#2}}{comp=}{
    \__stex_expr_set_customcomp: #2 \__stex_expr_end:
    \__stex_expr_struct_top:n{#1}
  }{
    \exp_args:Ne \stex_str_if_starts_with:nnTF {\tl_to_str:n{#2}}{this=}{
      \__stex_expr_set_thisnotation: #2 \__stex_expr_end:
      \__stex_expr_struct_top:n{#1}
    }{
      \tl_if_empty:nTF{#1}{
        \__stex_expr_struct_top:n{#2}
      }{
        \tl_if_empty:nTF{#2}{
          \__stex_expr_struct_top:n{#1}
        }{
          \__stex_expr_struct_top:n{#1,#2}
        }
      }
    }
  }
}

\cs_new_protected:Npn \__stex_expr_set_thisnotation: this= #1 \__stex_expr_end: {
  \tl_set:Nn \l_stex_return_notation_tl { \comp{#1} }
  \tl_set:Nn \l__stex_expr_set_comp_tl {}
}

\cs_new_protected:Npn \__stex_expr_set_customcomp: comp= #1 \__stex_expr_end: {
  \tl_set:Nn \l__stex_expr_set_comp_tl {
    \__stex_expr_set_custom_comp:n{#1}
  }
  \tl_set:Nn \l_stex_return_notation_tl { \comp{} }
```

```
}
```

The structure type:

```
\cs_new_protected:Nn \__stex_expr_struct_type:n {
  \__stex_expr_do_assign_list:n{#1}
  \clist_if_empty:NTF \l__stex_expr_fields_clist {
    \int_compare:nNnTF {\clist_count:N \l_stex_current_type_tl}
      = 1 {
        \tl_set:Ne \l__stex_expr_current_type_tl {
          \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
          \exp_args:No \exp_not:n \l_stex_current_redo_tl
          \_stex_term_oms_or_omv:nn{}{}
        }
      }{
        \exp_args:No \__stex_expr_make_type:n \l_stex_current_type_tl
    }
  }{
    \int_compare:nNnTF {\clist_count:N \l_stex_current_type_tl}
      = 1 {
        \__stex_expr_make_type:n {}
      }{
        \exp_args:No \__stex_expr_make_type:n \l_stex_current_type_tl
    }
  }
}

\cs_new_protected:Nn \__stex_expr_do_assign_list:n {
  \clist_clear:N \l__stex_expr_fields_clist
  \tl_if_empty:nF {#1} {
    \keyval_parse:NNn\TODO\__stex_expr_do_assign:nn{#1}
  }
}

\cs_new_protected:Nn \__stex_expr_do_assign:nn {
  \clist_put_right:Nn \l__stex_expr_fields_clist {{#1}{#2}}
}

\cs_new_protected:Nn \__stex_expr_make_type:n {
  \tl_if_empty:nTF{#1}{
    \seq_clear:N \l_tmpa_seq
  }{
    \seq_set_split:Nnn \l_tmpa_seq ,{#1}
    \seq_pop_right:NN \l_tmpa_seq \l_tmpa_tl
    \seq_reverse:N \l_tmpa_seq
  }
  \tl_set:Ne \l__stex_expr_current_type_tl {
    \symuse{Metatheory?module~type~merge}{
      {
        \exp_args:No \exp_not:n \l_stex_current_redo_tl
        \_stex_term_oms_or_omv:nn{}{}
      }
      \seq_map_function:NN \l_tmpa_seq \__stex_expr_make_mod:n
      \clist_if_empty:NF \l__stex_expr_fields_clist {
        ,\symuse{Metatheory?anonymous~record}{
          \exp_args:Ne \tl_tail:n{
```

```
            \clist_map_function:NN \l__stex_expr_fields_clist \__stex_expr_make_oml:n
          }
        }
      }
    }
  }
}

\cs_new:Nn \__stex_expr_make_mod:n {
  ,\symuse{Metatheory?module~type}{
    \exp_args:Ne \stex_annotate:nn{data-ftml-term=OMMOD,data-ftml-head={\stex_use_module_ur
  }
}


\cs_new:Nn \__stex_expr_make_oml:n {
  \__stex_expr_make_oml:nn #1
}
\cs_new:Nn \__stex_expr_make_oml:nn {
  ,\stex_annotate:nn{
    data-ftml-term=OML,
    data-ftml-head={#1}
  }{
    \_stex_annotate_force_break:n{
      \stex_annotate:nn{data-ftml-definiens={}}{\exp_not:n{#2!}}
    }
  }
}
```

Insert the structure type as a term:

```
\cs_new:Nn \__stex_expr_current_type: {
  %\exp_args:No \exp_not:n \l_stex_current_redo_tl
  \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
    \exp_args:No\exp_not:n\l__stex_expr_current_type_tl
  }
  \_stex_term_oms_or_omv:nn{}{}
}
```

The structure type itself:

```
\cs_new_protected:Npn \__stex_expr_maybe_notation:w [ #1 ] {
  \tl_set_eq:NN \l_stex_current_term_tl \l__stex_expr_current_type_tl
  \stex_use_notation:nnTF \l_stex_current_full_tl {#1}{
    \l_stex_notation_cs\group_end:
  }{
    \__stex_expr_make_prop:
    \__stex_expr_make_prop_assign:
    \__stex_expr_present_i:w [#1]
  }
}

\cs_new_protected:Nn \__stex_expr_present: {
  \peek_charcode:NTF [ {
    \__stex_expr_present_i:w
  }{
    \__stex_expr_present:nn{}{}
```

```
  }
}

\cs_new_protected:Npn \__stex_expr_present_i:w [#1] {
  \int_compare:nNnTF{\clist_count:n{#1}} = 1 {
    \__stex_expr_present:nn{}{#1}
  }{
    \peek_charcode:NTF [ {
      \__stex_expr_present_ii:nw{#1}
    }{
      \__stex_expr_present:nn{#1}{}
    }
  }
}

%First: clist, second:notation-id
\cs_new_protected:Npn \__stex_expr_present_ii:nw #1 [#2] {
  \__stex_expr_present:nn{#1}{#2}
}

\cs_new_protected:Nn \__stex_expr_present:nn {
  \clist_clear:N \l__stex_expr_clist
  \tl_if_empty:nTF{#1}{
    \cs_set:Npn \l__stex_expr_cs ##1 ##2 ##3 {
      \tl_if_empty:nF{##2}{
        \__stex_expr_present_entry:nn {##1}{##3}
      }
    }
  }{
    \cs_set:Npn \l__stex_expr_cs ##1 ##2 ##3 {
      \exp_args:Ne \clist_if_in:nnT{\tl_to_str:n{#1}}{##1}{
        \__stex_expr_present_entry:nn {##1}{##3}
      }
    }
  }
  \prop_map_inline:Nn \l__stex_expr_prop {
    \l__stex_expr_cs {##1} ##2
  }
  \_stex_term_oms_or_omv:nn{}{
    \exp_args:Nno \use:n{
      \bool_set_true:N \l_stex_allow_semantic_bool
      \symuse{Metatheory?mathematical~structure}[#2]
    }{\l__stex_expr_clist}
  }\group_end:
}

\cs_new_protected:Nn \__stex_expr_present_entry:nn {
  \seq_if_in:NnTF \l__stex_expr_assigned_seq {#1}{
    \clist_put_right:Nn \l__stex_expr_clist {#2!}
  }{
    \exp_args:NNe \clist_put_right:Nn \l__stex_expr_clist {
      \_stex_next_symbol:n {
        \exp_args:No \exp_not:n \l__stex_expr_set_comp_tl
        \tl_set:Nn \exp_not:N \l_stex_struct_this_tl {
```

```
          \exp_args:No \exp_not:n \l_stex_struct_this_tl
        }
        \exp_not:n {
          \tl_set_eq:NN \this \l_stex_struct_this_tl
        }
        \tl_set:Nn \exp_not:N \l_stex_return_notation_tl {
          \exp_args:No \exp_not:n \l_stex_return_notation_tl
        }
      }
      \exp_not:n{#2!}
    }
  }
}

%\cs_new_protected:Nn \__stex_expr_set_custom_comp:n {
%  \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_customthiscomp {
%    \cs_set_protected:Npx \_customthiscomp ##1 {
%      \group_begin:
%        \bool_set_true:N \l_stex_allow_semantic_bool
%        \exp_not:n{
%          \cs_set:Npn \l__stex_expr_comp_cs ##1  {
%            #1
%          }
%          \def\maincomp
%        }{\comp}
%        \exp_not:N\l__stex_expr_comp_cs{\comp{##1}}
%      \group_end:
%    }
%    \def\maincomp {\_customthiscomp}
%  }
%}

\cs_new_protected:Nn \__stex_expr_set_custom_comp:n {
  \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_customthiscomp {
    \stex_debug:nn{structs}{Setting~custom~comp:~\tl_to_str:n{#1}}
    \exp_args:Nne \__stex_expr_set_custom_i:nn{#1}{\comp}
    \def\maincomp {\_customthiscomp}
  }
}

\cs_new_protected:Nn \__stex_expr_set_custom_i:nn {
  \cs_set_protected:Npn \_customthiscomp ##1 {
    \group_begin:
      \bool_set_true:N \l_stex_allow_semantic_bool
      \cs_set:Npn \l__stex_expr_comp_cs ####1 { #1 }
      \def \maincomp {#2}
      \l__stex_expr_comp_cs{#2{##1}}
    \group_end:
  }
}
```

`this` (of type `structure`):

```
\cs_new_protected:Nn \__stex_expr_invoke_this:n {
  \peek_charcode_remove:NTF ! {
    \exp_args:Nne\use:nn{
```

```
      \group_end:\symuse{Metatheory?of~type}[invisible]{
        \tl_if_empty:nTF{#1}{\_stex_annotate_force_break:n{}}{#1}
      }
    }{
      {
        \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
        \__stex_expr_current_type:
      }
    }
  }{
    \__stex_expr_invoke_maybe_field:nn{#1}
  }
}

\cs_new_protected:Nn \__stex_expr_invoke_maybe_field:nn {
  \__stex_expr_make_prop:
  \__stex_expr_set_this:n{#1}
  \tl_if_empty:nTF{#2}{
    \__stex_expr_make_prop_assign:
    \__stex_expr_present:
  }{
    \__stex_expr_invoke_field:n{#2}
  }
}

\cs_new_protected:Nn \__stex_expr_set_this:n {
  \tl_if_empty:nTF{#1}{
    %\tl_put_right:Nn \l_stex_current_redo_tl {
    %  \tl_clear:N \l_stex_struct_this_tl
    %}
  }{
    \tl_set:Ne \l_stex_struct_this_tl {{
      \bool_set_true:N \l_stex_allow_semantic_bool
      \bool_set_true:N \l_stex_in_this_bool
      \tl_set:Nn \exp_not:N \this {
        \exp_args:No \exp_not:n \this
      }
      \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
      \exp_not:n{#1}
    }}
    \tl_set_eq:NN \this \l_stex_struct_this_tl
    % \l_stex_return_notation_tl
  }
}

\cs_new_protected:Nn \__stex_expr_get_field_name:n {
  \str_set:Ne \l__stex_expr_field_name_str {
    \exp_args:Nne \use:n {\exp_after:wN \use_i:nn \use:n}
    {\prop_item:Nn \l__stex_expr_prop {#1}}
  }
  \str_if_empty:NT \l__stex_expr_field_name_str {
    \str_set:Nn \l__stex_expr_field_name_str {#1}
  }
}
```

```
\cs_new_protected:Nn \__stex_expr_invoke_field:n {
  \prop_if_in:NnTF \l__stex_expr_prop {#1}{
    \__stex_expr_get_field_name:n{#1}
    \tl_clear:N \l__stex_expr_more_nextsymbol_tl
    %\exp_args:NNe \seq_if_in:NnF \l__stex_expr_assigned_seq {\tl_to_str:n{#1}}{
      \tl_set:Ne \l__stex_expr_more_nextsymbol_tl {
        \tl_set:Nn \exp_not:N \l_stex_struct_this_tl {
          \exp_args:No \exp_not:n \l_stex_struct_this_tl
        }
        \exp_not:n {
          \tl_set_eq:NN \this \l_stex_struct_this_tl
        }
        \tl_set:Nn \exp_not:N \l_stex_return_notation_tl {
          \exp_args:No \exp_not:n \l_stex_return_notation_tl
        }
        \exp_args:No \exp_not:n \l__stex_expr_set_comp_tl
      }
    %}
    \exp_args:NNe \use:nn \group_end: {
      \_stex_next_symbol:n {
        \exp_args:No \exp_not:n \l__stex_expr_redo_tl
        \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
          \symuse{Metatheory?record~field}{
            \symuse{Metatheory?of~type}{
              \exp_args:No\tl_if_empty:nTF \l_stex_struct_this_tl {\_stex_annotate_force_br
            }{
              \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
              \__stex_expr_current_type:
            }
          }{
            \stex_annotate:nn{data-ftml-term=OML,data-ftml-head={\l__stex_expr_field_name_s
          }
        }
        \exp_args:No \exp_not:n \l__stex_expr_more_nextsymbol_tl
      }
      \exp_not:N \use_ii:nn
      \prop_item:Nn \l__stex_expr_prop {#1}
    }
  }{
    \msg_error:nnn{stex}{error/unknownfield}{#1}
  }
}

\cs_new_protected:Nn \__stex_expr_make_prop: {
  \prop_clear:N \l__stex_expr_prop
  \seq_clear:N \l__stex_expr_seq
  \seq_clear:N \l__stex_expr_assigned_seq
  \tl_clear:N \l__stex_expr_redo_tl
  \__stex_expr_prop_do_decls:
}

\cs_new_protected:Nn \__stex_expr_prop_do_decls: {
  \group_begin:
```

```
    \stex_debug:nn{expressions}{constructing~record}
    \seq_clear:N \l_stex_all_module_seq
    \exp_args:Ne \stex_activate_module:n {\clist_item:Nn \l_stex_current_type_tl 1}
    \exp_args:No \stex_iterate_symbols:nn \l_stex_current_type_tl {
      % uri, macro name, name, arity, args, type, def, return, macro
      \tl_if_empty:nTF{##2}{
        \exp_args:Nne\__stex_expr_do_decl_nomacro:nnnnnnnnn{##3}
      }{
        \exp_args:Nne\__stex_expr_do_decl:nnnnnnnnn{##2}
      }
      {\stex_new_symbol_uri:nn{##1}{##3}}{##3}{##4}{##5}{##6}{##7}{##8}##9
    }
    \exp_after:wN \tl_set:Nn \exp_after:wN \l__stex_expr_after_tl \exp_after:wN {
      \exp_after:wN \tl_set:Nn \exp_after:wN \l__stex_expr_prop \exp_after:wN { \l__stex_ex
    }
    \__stex_expr_prop_do_notations:
    \stex_debug:nn{expressions}{record:~\meaning\l__stex_expr_after_tl}
    \tl_put_right:Ne \l__stex_expr_after_tl {\tl_set:Nn \exp_not:N \l__stex_expr_redo_tl {\
  \exp_after:wN \group_end: \l__stex_expr_after_tl
}

% id, uri, name, arity, args, type, def, return, macro
\cs_new_protected:Nn \__stex_expr_do_decl_nomacro:nnnnnnnnn {
  \prop_if_in:NnF \l__stex_expr_prop {#1} {
    \seq_put_left:Ne \l__stex_expr_seq {\tl_to_str:n{#2}}
    \prop_put:Nnn \l__stex_expr_prop {#1}{
      {}{
        \_stex_invoke_symbol:nnnnnnN
        {#2}
        {#4}
        {#5}{#6}{#7}{#8}#9
      }
    }
  }
}

\cs_new_protected:Nn \__stex_expr_do_decl:nnnnnnnnn {
  \prop_if_in:NnF \l__stex_expr_prop {#1} {
    \seq_put_left:Ne \l__stex_expr_seq {\tl_to_str:n{#2}}
    \prop_put:Nnn \l__stex_expr_prop {#1}{
      {#3}{
        \_stex_invoke_symbol:nnnnnnN
        {#2}
        {#4}
        {#5}{#6}{#7}{#8}#9
      }
    }
  }
}

\cs_new_protected:Nn \__stex_expr_make_prop_assign: {
  \clist_if_empty:NF \l__stex_expr_fields_clist {
    \clist_map_inline:Nn \l__stex_expr_fields_clist {
      \__stex_expr_make_prop_assign:nn ##1
```

```
    }
  }
}

\cs_new_protected:Nn \__stex_expr_make_prop_assign:nn {
  \prop_if_in:NnTF \l__stex_expr_prop {#1}{
    \exp_args:NNe \seq_put_right:Nn \l__stex_expr_assigned_seq {\tl_to_str:n{#1}}
    \exp_args:Nne \use:nn {\__stex_expr_make_prop_assign_replace:nnnn {#1}{#2}}
    {\prop_item:Nn \l__stex_expr_prop {#1}}
  }{
    \msg_error:nnn{stex}{error/unknownfieldass}{#1}
  }
}
\cs_new_protected:Nn \__stex_expr_make_prop_assign_replace:nnnn {
  \prop_put:Nnn \l__stex_expr_prop {#1}{{#3}{#2}}
  \tl_if_empty:nF{#3}{
    \tl_set:cn{#1}{ #2 }
    \tl_put_right:Nn \l__stex_expr_redo_tl {
      \tl_set:cn{#1}{ #2 }
    }
  }
}

\cs_new_protected:Nn \__stex_expr_prop_do_notations: {
  \exp_args:No \stex_iterate_notations:nn \l_stex_current_type_tl {
    \exp_args:NNe \seq_if_in:NnT \l__stex_expr_seq {\tl_to_str:n{##1}}{
      \tl_set:Nn \l_tmpa_tl {
        \cs_if_exist:cF{l_stex_notation_ \stex_use_symbol_uri:n{##1}?##2_cs}{
          \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_cs}{##4}
        }
        \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?_cs}{
          \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?_cs}{##4}
        }
      }
      \exp_args:NNo \tl_put_right:Nn \l__stex_expr_redo_tl \l_tmpa_tl
      \exp_args:NNo \tl_put_right:Nn \l__stex_expr_after_tl \l_tmpa_tl
      \tl_if_empty:nF{##5}{
        \tl_set:Nn \l_tmpa_tl {
          \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_op_cs}{
            \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?##2_op_cs}{##5}
          }
          \cs_if_exist:cF{l_stex_notation_\stex_use_symbol_uri:n{##1}?_op_cs}{
            \tl_set:cn{l_stex_notation_\stex_use_symbol_uri:n{##1}?_op_cs}{##5}
          }
        }
        \exp_args:NNo \tl_put_right:Nn \l__stex_expr_redo_tl \l_tmpa_tl
        \exp_args:NNo \tl_put_right:Nn \l__stex_expr_after_tl \l_tmpa_tl
      }
    }
  }
}
```

(*End of definition for* `\stex_invoke_structure:`. *This function is documented on page 150.*)

sfunction

_stex_invoke_variable:nnnnnn

```
\cs_new_protected:Nn \_stex_invoke_variable:nnnnnnN {
  \bool_if:NTF \l_stex_allow_semantic_bool{
    \group_begin:
    \tl_set:Nn \l_stex_current_full_tl { #1 }
    \tl_set:Nn \l_stex_current_display_tl { #1 }
    \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
    \__stex_expr_setup:nnnnn{\_varcomp}{#2}{#3}{#6}{#5}
    \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_omv:nn
    \tl_put_right:Nn \l_stex_current_redo_tl {
      \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_omv:nn
    }
    #7
  }{
    \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
  }
}
```

(*End of definition for* `\_stex_invoke_variable:nnnnnn`. *This function is documented on page 150.*)

sfunction

\svar

```
\NewDocumentCommand \svar {O{} m}{
  \group_begin:
    \tl_if_empty:nTF{#1}{
      \tl_set:Nn \l_stex_current_full_tl { #2 }
      \tl_set:Nn \l_stex_current_display_tl { #2 }
    }{
      \tl_set:Nn \l_stex_current_full_tl { #1 }
      \tl_set:Nn \l_stex_current_display_tl { #1 }
    }
    \bool_if:NTF \l_stex_allow_semantic_bool{
        \tl_clear:N \l_stex_current_term_tl
        \_stex_term_omv:nn{}{\_varcomp{#2}}
    }{
      \msg_error:nnxx{stex}{error/notallowed}{Variable}{\l_stex_current_full_tl}
    }
  \group_end:
}
```

(*End of definition for* `\svar`. *This function is documented on page 150.*)

sfunction

Now for the brunt of the work:

Math

Modifiers (`!` or `*`)?

```
\cs_new_protected:Nn \__stex_expr_math: {
  \stex_debug:nn{expressions}{math~mode}
  \peek_charcode_remove:NTF ! {
    % operator
    \peek_charcode_remove:NTF * \__stex_expr_op_custom:n {
      % op notation
      \peek_charcode:NTF [ \__stex_expr_op_notation:w {
```

```
        \__stex_expr_op_notation:w []
      }
    }
  }{
    \peek_charcode_remove:NTF * \__stex_expr_custom:n {
      % normal
      \peek_charcode:NTF [ \__stex_expr_notation:w {
        \__stex_expr_notation:w []
      }
    }
  }
}
```

Text

Operator?

```
\cs_new_protected:Nn \__stex_expr_text: {
  \stex_debug:nn{expressions}{text~mode}
  \peek_charcode_remove:NTF ! \__stex_expr_op_custom:n \__stex_expr_custom:n
}
```

Notation

Operator?

```
\cs_new_protected:Npn \__stex_expr_notation:w [ #1 ] {
  \peek_charcode_remove:NTF ! {
    \__stex_expr_op_notation:w [#1]
  }{
    \__stex_expr_full_notation:n {#1}
  }
}
```

(Possibly) complex notation taking arguments with optional particular identifier:

```
\cs_new_protected:Nn \__stex_expr_full_notation:n {
  \stex_use_notation:nnTF \l_stex_current_full_tl {#1}{
    \stex_debug:nn{expressions}{using~notation~"#1"~for~\l_stex_current_full_tl}
    \tl_if_empty:NTF \l_stex_current_return_tl {
      \stex_debug:nn{expressions}{return~empty}
      \l_stex_notation_cs{\group_end:\_stex_eat_exclamation_point:}
    }{
      \stex_debug:nn{expressions}{return?}
      \exp_after:wN \__stex_expr_maybe_return:n \exp_after:wN {
        \l_stex_notation_cs{}
      }
    }
  }{
    \stex_debug:nn{expressions}{notation~"#1"~for~\l_stex_current_full_tl{}~does~not~exist;
    \stex_do_default_notation:
    \tl_if_empty:NTF \l_stex_current_return_tl {
      \l_stex_default_notation{\group_end:\_stex_eat_exclamation_point:}
    }{
      \exp_after:wN
      \__stex_expr_maybe_return:n
      \exp_after:wN
      {\l_stex_default_notation {}}
```

```
    }
  }
}
```

Operator notation:

```
\cs_new_protected:Npn \__stex_expr_op_notation:w [#1] {
  \stex_debug:nn{expressions}{op~notation~for~\l_stex_current_full_tl}
  \stex_use_op_notation:nnTF \l_stex_current_full_tl {#1}{
    \_stex_maybe_brackets:nn{\neginfprec}{
      \_stex_term_oms_or_omv:nn{#1}
      {\l_stex_notation_cs}
    }
    \group_end:
  }{
    \int_compare:nNnTF \l_stex_current_arity_str = 0 {
      \tl_clear:N \l_stex_current_return_tl
      \__stex_expr_notation:w [#1]
    }{
      \stex_do_default_notation_op:
      \_stex_maybe_brackets:nn{\neginfprec}{
        \_stex_term_oms_or_omv:nn{#1}
        {\l_stex_default_notation}
      }
      \group_end:
    }
  }
}
```

Custom Notations

Operator:

```
\cs_new_protected:Nn \__stex_expr_op_custom:n {
  \stex_debug:nn{expressions}{custom~op}
  \bool_set_true:N \l_stex_allow_semantic_bool
  \_stex_term_oms_or_omv:nn{}{\maincomp{#1}}
  \group_end:
}
```

Complex:

```
\int_new:N \l__stex_expr_arg_counter_int

\cs_new_protected:Nn \__stex_expr_custom:n {
  \stex_debug:nn{custom}{custom~notation~for~\l_stex_current_full_tl}
  \stex_pseudogroup:nn{
    \bool_set_true:N \l_stex_allow_semantic_bool
    \prop_gclear:N \l__stex_expr_customs_prop
    \seq_gclear:N \l__stex_expr_customs_seq
    \int_gzero:N \l__stex_expr_arg_counter_int
    \tl_if_empty:NF \l_stex_current_args_tl {
      \exp_after:wN \__stex_expr_add_prop_arg:nnw \l_stex_current_args_tl \_stex_args_end:
      \cs_set_eq:NN \arg \__stex_expr_arg:n
    }
    \tl_set_eq:NN \l_stex_get_symbol_args_tl \l_stex_current_args_tl
    \cs_set_eq:NN \__stex_expr_do_ab_next:nn \_stex_term_oma:nn
    \_stex_map_args:N \__stex_expr_check_b:nn
```

```
    \__stex_expr_do_ab_next:nn{}{#1}
  }{
    \prop_if_exist:NT \l__stex_expr_customs_prop {
      \prop_gset_from_keyval:Nn \exp_not:N \l__stex_expr_customs_prop {
        \prop_to_keyval:N \l__stex_expr_customs_prop
      }
    }
    \int_gset:Nn \l__stex_expr_arg_counter_int { \int_use:N \l__stex_expr_arg_counter_int}
    \seq_if_exist:NT \l__stex_expr_customs_seq {
      \seq_gset_split:Nnn \exp_not:N \l__stex_expr_customs_seq , {
        \seq_use:Nn \l__stex_expr_customs_seq ,
      }
    }
  }
  % TODO check that all arguments are present
  \group_end:
}

\cs_new_protected:Npn \__stex_expr_add_prop_arg:nnw #1 #2 #3\_stex_args_end: {
  \prop_gput:Nnn \l__stex_expr_customs_prop {#1} {}
  \seq_gput_right:Nn \l__stex_expr_customs_seq {#2}
  \tl_if_empty:nF{#3}{\__stex_expr_add_prop_arg:nnw #3 \_stex_args_end:}
}

\cs_new:Nn \__stex_expr_check_b:nn {
  \str_case:nn #2 {
    b {\cs_set_eq:NN \__stex_expr_do_ab_next:nn \_stex_term_omb:nn}
    B {\cs_set_eq:NN \__stex_expr_do_ab_next:nn \_stex_term_omb:nn}
  }
}
```

Arguments:

```
\NewDocumentCommand \__stex_expr_arg:n {s O{} m} {
  \IfBooleanTF #1 {
    \stex_annotate_invisible:n{
      \__stex_expr_arg_inner:nn{#2}{#3}
    }
  }{
    \__stex_expr_arg_inner:nn{#2}{#3}
  }
}

\cs_new_protected:Nn \__stex_expr_arg_inner:nn {
  \tl_if_empty:nTF{#1}{
    \int_gincr:N \l__stex_expr_arg_counter_int
    \exp_args:Ne \__stex_expr_check:nTF{ \int_use:N \l__stex_expr_arg_counter_int }{
      \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl
    }{
      \__stex_expr_arg_inner:nn{}
    }{ #2 }
  }{
    \__stex_expr_check:nTF {#1}{
      \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl { #2 }
    }{
      \exp_args:No \str_case:nnTF \l_tmpb_tl {
```

```
        {a}{
          \exp_args:NNne \prop_gput:Nnn \l__stex_expr_customs_prop {#1}{
            \l_tmpa_tl X
          }
          \tl_set:Nx \l_tmpa_tl { #1 \int_eval:n {\tl_count:N \l_tmpa_tl + 1} }
        }
        {B}{
          \exp_args:NNne \prop_gput:Nnn \l__stex_expr_customs_prop {#1}{
            \l_tmpa_tl X
          }
          \tl_set:Ne \l_tmpa_tl { #1 \int_eval:n {\tl_count:N \l_tmpa_tl + 1} }
        }
      }{
        \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl { #2 }
      }{
        \msg_error:nnxx{stex}{error/invalidarg}{#1}{\l_stex_current_full_tl}
      }
    }
  }
}

\prg_new_conditional:Nnn \__stex_expr_check:n {TF} {
  \exp_args:NNe \prop_get:NnNTF \l__stex_expr_customs_prop {#1} \l_tmpa_tl {
    \tl_set:Ne \l_tmpb_tl {\seq_item:Nn \l__stex_expr_customs_seq {#1} }
    \tl_if_empty:NTF \l_tmpa_tl {
      \exp_args:NNe \prop_gput:Nnn \l__stex_expr_customs_prop
        { #1 }{X}
      \exp_args:No \str_case:nnF \l_tmpb_tl {
        {a}{
          \tl_set:Ne \l_tmpa_tl{ #1 1 }
        }
        {B}{
          \tl_set:Ne \l_tmpa_tl{ #1 1 }
        }
      }{
        \tl_set:Ne \l_tmpa_tl{ #1 }
      }
      \prg_return_true:
    }{
      \prg_return_false:
    }
  }{
    \msg_error:nnxx{stex}{error/invalidarg}{#1}{\l_stex_current_full_tl}
    \prg_return_false:
  }
}

% #1 argnum #2 argmode #3 code
\cs_new_protected:Nn \__stex_expr_arg_do:nnn {
  \stex_debug:nn{custom}{Doing~argument~#1~of~mode~#2:~\tl_to_str:n{#3}}
  \group_begin:
    \bool_set_true:N \l_stex_allow_semantic_bool
    \_stex_term_arg:nnn {#2}{#1}{#3}
  \group_end:
```

```
}
\cs_generate_variant:Nn \__stex_expr_arg_do:nnn {oon}
```

Return code

```
\cs_new_protected:Nn \__stex_expr_maybe_return:n {
  \tl_set:Nn \l__stex_expr_return_this_tl {#1}
  \tl_clear:N \l__stex_expr_return_args_tl
  \exp_args:Nne \use:n {
  \cs_generate_from_arg_count:NNnn \__stex_expr_ret_cs:
    \cs_set:Npn \l_stex_current_arity_str } {
      \int_step_function:nN \l_stex_current_arity_str \__stex_expr_return_arg:n
      \__stex_expr_return_next:
    }
  \__stex_expr_ret_cs:
}

\cs_new:Nn \__stex_expr_return_arg:n {
  \tl_put_right:Nn \exp_not:N \l__stex_expr_return_args_tl {{## #1}}
}

\cs_new_protected:Nn \__stex_expr_return_next: {
  \peek_charcode_remove:NTF ! {
    \exp_after:wN \l__stex_expr_return_this_tl \l__stex_expr_return_args_tl \group_end:
  }\__stex_expr_return:
}

\cs_new_protected:Nn \__stex_expr_return: {
  \tl_set:Ne \l__stex_expr_return_this_tl {
    \tl_if_empty:NTF \l_stex_return_notation_tl {
      \exp_after:wN \exp_after:wN \exp_after:wN
      \exp_not:n \exp_after:wN \exp_after:wN \exp_after:wN {
        \exp_after:wN \l__stex_expr_return_this_tl \l__stex_expr_return_args_tl
      }
    }{
      \l_stex_return_notation_tl
    }
  }
  \stex_debug:nn{return}{Notation:~\meaning\l__stex_expr_return_this_tl}

  \exp_after:wN
  \tl_put_left:Nn \exp_after:wN \l__stex_expr_return_this_tl \exp_after:wN {
    \exp_after:wN \group_begin: \l_stex_current_redo_tl
  }
  \exp_args:Nne \use:n {
  \cs_generate_from_arg_count:NNnn \__stex_expr_ret_cs:
    \cs_set:Npn \l_stex_current_arity_str } {
      \exp_args:No \exp_not:n \l_stex_current_return_tl
    }
  \stex_debug:nn{return}{
    \meaning\__stex_expr_ret_cs:^^J
    \meaning\l__stex_expr_return_this_tl^^J
    \exp_args:No \exp_not:n \l__stex_expr_return_args_tl^^J
  }
```

```
  \exp_args:Nne \use:nn {
    \exp_after:wN \group_end: \__stex_expr_ret_cs:
  }{
    \exp_args:No \exp_not:n \l__stex_expr_return_args_tl
    {
      \exp_args:No \exp_not:n \l__stex_expr_return_this_tl
      \group_end:
    }
  }
}
```

32.4.1 Term Annotations

_stex_eat_exclamation_point:

```
\cs_new_protected:Nn \_stex_eat_exclamation_point: {
  \peek_charcode_remove:NT ! \_stex_eat_exclamation_point:
}
```

(*End of definition for* `\_stex_eat_exclamation_point:`. *This function is documented on page 150.*)

sfunction

We occasionally allow for semantic macros where they otherwise shouldn't be, if we are in "invisible" parts:

```
\bool_new:N \stex_in_invisible_html_bool
```

`\_stex_term_oms:nn`

```
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \_stex_term_oms:nn {
    \bool_if:NTF\l_stex_in_this_bool{#2}{
      \__stex_expr_check_comp:
      \tl_if_empty:NTF \l_stex_current_term_tl {
        \__stex_expr_annotate:nnn{OMID}{#1}{#2}
      }{
        \__stex_expr_do_headterm:nn{#1}{#2}
      }
    }
  }
}{
  \cs_new_protected:Nn \_stex_term_oms:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_chec
}

\cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
```

(*End of definition for* `\_stex_term_oms:nn`. *This function is documented on page 150.*)

sfunction

`\_stex_term_omv:nn`

```
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \_stex_term_omv:nn {
    \bool_if:NTF\l_stex_in_this_bool{#2}{
      \__stex_expr_check_comp:
      \tl_if_empty:NTF \l_stex_current_term_tl {
        \__stex_expr_annotate:nnn{OMV}{#1}{#2}
      }{
```

```
        \__stex_expr_do_headterm:nn{#1}{#2}
      }
    }
  }
}{
  \cs_new_protected:Nn \_stex_term_omv:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_chec
}
```

(*End of definition for* `\_stex_term_omv:nn`. *This function is documented on page 150.*)

sfunction

In the case where the "symbol" (OMS or OMV) is the field of some structure or otherwise a complex expression, we have to insert the full term representing the operator:

```
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \__stex_expr_do_headterm:nn {
    \bool_if:NTF \stex_in_invisible_html_bool {
      {\bool_set_true:N \l_stex_allow_semantic_bool
        \ensuremath{\l_stex_current_term_tl}
      }
    }{
      \__stex_expr_annotate:nnn{complex}{#1}{
        \stex_annotate_invisible:nn{data-ftml-headterm={}}{
          {\bool_set_true:N \l_stex_allow_semantic_bool
            \ensuremath{\l_stex_current_term_tl}
          }
        }
        #2
      }
    }
  }
}{
  \cs_new_protected:Nn \__stex_expr_do_headterm:nn { #2 }
}
```

`\_stex_term_oma:nn`

```
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \_stex_term_oma:nn {
    \bool_if:NTF\l_stex_in_this_bool{#2}{
      \__stex_expr_check_comp:
      \__stex_expr_annotate:nnn{OMA}{#1}{
        \tl_if_empty:NF \l_stex_current_term_tl {
          \stex_annotate_invisible:nn{data-ftml-headterm={}}{{
            \bool_set_true:N \l_stex_allow_semantic_bool
            \l_stex_current_term_tl
          }}
        }
        #2
      }
    }
  }
}{
  \cs_new_protected:Nn \_stex_term_oma:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_chec
}
```

(*End of definition for* `\_stex_term_oma:nn`. *This function is documented on page 150.*)

sfunction

_stex_term_omb:nn

```
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \_stex_term_omb:nn {
    \bool_if:NTF\l_stex_in_this_bool{#2}{
      \__stex_expr_check_comp:
      \__stex_expr_annotate:nnn{OMBIND}{#1}{
        \tl_if_empty:NF \l_stex_current_term_tl {
          \stex_annotate_invisible:nn{data-ftml-headterm={}}{{
            \bool_set_true:N \l_stex_allow_semantic_bool
            \l_stex_current_term_tl
          }}
        }
        #2
      }
    }
  }
}{
  \cs_new_protected:Nn \_stex_term_omb:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_chec
}
```

(*End of definition for* _stex_term_omb:nn. *This function is documented on page 150.*)

sfunction

Does the actual annotating: TODO: This shouldn't use \l_stex_current_symbol_uri!

```
% kind, notation id, body
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \__stex_expr_annotate:nnn {
    \exp_args:Ne \stex_annotate:nn{
      data-ftml-term=#1,
      data-ftml-head={\l_stex_current_full_tl},
      data-ftml-notationid={#2},
    }{
      \_stex_annotate_force_break:n{#3}
    }
  }
}{
  \cs_new_protected:Nn \__stex_expr_annotate:nnn { #3 }
}
```

32.4.2 Symbol Arguments

_stex_term_arg:nnnnn

#1 : argument number
#2 : argument mode
#3 : precedence
#4 : argument name (WiP)
#5 : code / body

```
\cs_new_protected:Nn \_stex_term_arg:nnnnn {
  \group_begin:
    \tl_clear:N \l_stex_current_symbol_uri
    \tl_clear:N \l_stex_current_full_tl
    \tl_clear:N \l_stex_current_display_tl
    \tl_clear:N \l_stex_current_term_tl
```

```
    \int_set:Nn \l_stex_notation_downprec { #3 }
    \bool_set_true:N \l_stex_allow_semantic_bool
    \_stex_term_arg:nnn {#2}{#1}{#5}
  \group_end:
}
```

(*End of definition for* `\_stex_term_arg:nnnnn`. *This function is documented on page 151.*)

sfunction

`\_stex_term_arg:nnn`

```
\cs_new_protected:Nn \__stex_expr_check_comp: {
  \bool_if:NT \g_stex_in_comp_bool {
    \msg_error:nn{stex}{error/argincomp}
  }
}
\stex_if_html_backend:TF {
  \cs_new_protected:Nn \_stex_term_arg:nnn {
    \stex_if_do_html:TF{
      \bool_if:NTF\l_stex_in_this_bool{#3}{
        \_stex_annotate_force_break:n{
          \stex_annotate:nn{
            data-ftml-arg={#2}, data-ftml-argmode={#1}
          }{
            \_stex_annotate_force_break:n{}
            \__stex_expr_check_comp:
            #3
          }
        }
      }
    }{ #3 }
  }
}{
  \cs_new_protected:Nn \_stex_term_arg:nnn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_che
}
```

(*End of definition for* `\_stex_term_arg:nnn`. *This function is documented on page 151.*)

sfunction

`\_stex_term_arg_aB:nnnnn` The following macros fill up `\l_stex_aB_args_seq` and ultimately call `\_stex_term_do_aB_clist:`. By default, this does:

```
\cs_new:Nn \__stex_expr_do_aB_clist: {
  \_stex_annotate_force_break:n{
    \seq_use:Nn \l_stex_aB_args_seq {
      \mathpunct{\comp{,}}
    }
  }
}

\cs_set_eq:NN \_stex_term_do_aB_clist: \__stex_expr_do_aB_clist:

\cs_new_protected:Nn \_stex_is_sequentialized:n {
  \group_begin: #1 \group_end:
}
```

Setup:

```
\int_new:N \l__stex_expr_count_int
\cs_new_protected:Nn \_stex_term_arg_aB:nnnnn {
  \stex_debug:nn{sequences}{Processing~argument:~ \tl_to_str:n{#5}}
  \tl_if_empty:nTF{#5}{
    \_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{}
  }{
    \seq_clear:N \l_stex_aB_args_seq
    \int_zero:N \l__stex_expr_count_int
    \clist_map_inline:nn{#5}{
      \__stex_expr_aB_arg:nnnnn{##1}{#1}{#2}{#3}{#4}
    }
    \_stex_term_do_aB_clist:
  }
}
```

We "pattern match" the sequence argument - is it a comma-separated list? A sequence variable macro? `\argmap`? `\seqmap`?

```
% 1: code 2: argnum 3: argmode 4: precedence 5: argname
\cs_new_protected:Npn \__stex_expr_aB_arg:nnnnn #1 #2 #3 {
  \int_incr:N \l__stex_expr_count_int
  \stex_debug:nn{expressions}{matching~argument~ #2 #3:~ \tl_to_str:n{#1}}
  \__stex_expr_is_varseq:nTF{#1}{
    \stex_debug:nn{expressions}{=sequence~variable}
    \exp_after:wN \exp_after:wN \exp_after:wN
    \__stex_expr_assoc_seq:nnnnnnn
    \exp_after:wN
    \__stex_expr_gobble:nnnnnnnn #1 \__stex_expr_end:
  }{
    \__stex_expr_is_seqmap:nTF{#1}{
      \stex_debug:nn{expressions}{=seqmap}
      \exp_args:NNe \use:nn \__stex_expr_do_seqmap:nnnnnn {\tl_tail:n{#1}}
    }{
      \stex_debug:nn{expressions}{=simple}
      \__stex_expr_aB_simple_arg:nnnnn{#1}
    }
  }
  {#2}{#3}
}

\cs_new:Npn \__stex_expr_gobble:nnnnnnnn #1 #2 #3 #4 #5 #6 #7 #8 #9 \__stex_expr_end: {
  {#2} #3 {#6}
}
```

Is it a varseq?

```
\prg_new_conditional:Nnn \__stex_expr_is_varseq:n {TF} {
  \int_compare:nNnTF {\tl_count:n{#1}} = 1 {
    \exp_args:Ne \cs_if_eq:NNTF {\exp_args:No \tl_head:n{#1}}
    \_stex_invoke_variable:nnnnnnN {
      \exp_args:Ne \cs_if_eq:NNTF {\exp_args:No\tl_item:nn{#1}{8}}
        \stex_invoke_sequence:
          \prg_return_true:\prg_return_false:
    }\prg_return_false:
  }\prg_return_false:
}
```

Is it a seqmap?

```
\prg_new_conditional:Nnn \__stex_expr_is_seqmap:n {TF} {
  \int_compare:nNnTF {\tl_count:n{#1}} = 3 {
    \exp_args:Ne \tl_if_eq:nnTF {\tl_head:n{#1}} {\seqmap}
      \prg_return_true:\prg_return_false:
  }\prg_return_false:
}
```

Is it an argsep?

```
\prg_new_conditional:Nnn \__stex_expr_is_argsep:n {TF} {
  \int_compare:nNnTF {\tl_count:n{#1}} = 3 {
    \exp_args:Ne \tl_if_eq:nnTF {\tl_head:n{#1}} {\argsep}
      \prg_return_true:\prg_return_false:
  }\prg_return_false:
}
```

Simple argument:

```
\cs_new_protected:Nn \__stex_expr_aB_simple_arg:nnnnn{
  \seq_put_right:Ne \l_stex_aB_args_seq {
    \_stex_term_arg:nnnnn{#2\int_use:N\l__stex_expr_count_int}{#3}{#4}{#5}{
      \exp_not:n{
        \tl_set_eq:NN \_stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
        #1
      }
    }
  }
}
```

Sequence variable:

```
% 1: name 2: arity 3: clist 4: argnum 5: argmode 6: precedence 7: argname
\cs_new_protected:Nn \__stex_expr_assoc_seq:nnnnnnn {
  \group_begin:
    \seq_clear:N \l_stex_aB_args_seq
    \tl_set:Nn \l_stex_current_full_tl{#1}
    \tl_set:Nn \l_stex_current_display_tl{#1}
    \__stex_expr_assoc_make_seq:nnn{#1}{#3}{#2}
  \exp_args:NNe \use:nn \group_end: {
    \seq_put_right:Nn \exp_not:N \l_stex_aB_args_seq {
      \_stex_is_sequentialized:n{
        \_stex_term_arg:nnnnn{#4\int_use:N\l__stex_expr_count_int}{#5}{#6}{#7}{
          \bool_set_true:N \l_stex_allow_semantic_bool
          \tl_if_empty:NF \l_stex_current_term_tl {
            \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
              \exp_args:No \exp_not:n \l_stex_current_term_tl
            }
          }
          \stex_annotate:nn{
            data-ftml-term=OMV,
            data-ftml-head={#1},
            data-ftml-notationid={}
          }{
            \_stex_annotate_force_break:n{
              \_stex_term_do_aB_clist:
            }
          }
```

```
        }
      }
    }
  }
}

% #1: name, #2: clist, #3:arity
\cs_new_protected:Nn \__stex_expr_assoc_make_seq:nnn {
  \stex_use_notation:nnTF {#1}{}{
    \cs_set_eq:NN \l__stex_expr_cs \l_stex_notation_cs
  }{
    \stex_do_default_notation:
    \cs_set_eq:NN \l__stex_expr_cs \l_stex_default_notation
  }
  \clist_map_inline:nn{#2}{
    \tl_if_eq:nnTF{##1}{\ellipses}{
      \seq_put_right:Nn \l_stex_aB_args_seq { ##1 }
    }{
      \int_compare:nNnTF {#3} = 1 {
        \tl_set:Nn \l__stex_expr_iarg_tl { {##1} }
      }{
        \tl_set:Nn \l__stex_expr_iarg_tl { ##1 }
      }
      \seq_put_right:Ne \l_stex_aB_args_seq {
        \group_begin:
        \exp_not:n {
          \tl_set_eq:NN \_stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
          \def\comp{\_varcomp}
          \tl_set:Nn \l_stex_current_full_tl{#1}
        }
        \exp_after:wN \exp_after:wN \exp_after:wN \exp_not:n
        \exp_after:wN \exp_after:wN \exp_after:wN {
          \exp_after:wN \l__stex_expr_cs \exp_after:wN \group_end: \l__stex_expr_iarg_tl
        }
      }
    }
  }
}
```

seqmap:

```
% 1: fun 2: clist 3: argnum 4: argmode 5: precedence 6: argname
\cs_new_protected:Nn \__stex_expr_do_seqmap:nnnnnn {
  \group_begin:
    \cs_set:Npn \l_tmpa_cs ##1 {#1}
    \seq_clear:N \l_stex_aB_args_seq
    \clist_map_inline:nn{#2}{
      \__stex_expr_is_varseq:nTF{##1}{
        \exp_after:wN
        \__stex_expr_varseq_in_map:nnnnnnnn ##1
      }{
        \seq_put_right:Nn \l_stex_aB_args_seq {##1}
      }
    }
    \seq_clear:N \l__stex_expr_old_seq
    \seq_map_inline:Nn \l_stex_aB_args_seq {
```

```
      \tl_if_eq:nnTF{##1}{\ellipses}{
        \seq_put_right:Nn \l__stex_expr_old_seq {##1}
      }
      % TODO \_stex_is_sequentialized:n
      {
        \exp_args:NNo \seq_put_right:Nn \l__stex_expr_old_seq {
          \l_tmpa_cs {##1}
        }
      }
    }
    \seq_set_eq:NN \l_stex_aB_args_seq \l__stex_expr_old_seq
  \tl_set:Ne \l_tmpa_tl {
    \symuse{Metatheory?bind}{
      \svar[MAP_DUMMY]{\exp_not:N\mathrm{MAPDUMMY}}
    }{
      \exp_args:No\exp_not:n{\l_tmpa_cs{\svar[MAP_DUMMY]{\mathrm{MAPDUMMY}}}}
    }
  }
  \exp_args:NNe \use:nn \group_end: {
    \seq_put_right:Nn \exp_not:N \l_stex_aB_args_seq {
      \_stex_is_sequentialized:n{
        \_stex_term_arg:nnnnn{#3\int_use:N\l__stex_expr_count_int}{#4}{#5}{#6}{
          \bool_set_true:N \l_stex_allow_semantic_bool
          \tl_set:Nn \exp_not:N \l_stex_current_full_tl
            {\l_stex_current_full_tl}
          \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
            \exp_not:N \symuse{Metatheory?sequence~map}
            {\exp_not:n{#2}}
            {\exp_args:No \exp_not:n \l_tmpa_tl}
          }
          \__stex_expr_do_headterm:nn{}{
            \_stex_term_do_aB_clist:
          }
        }
      }
    }
  }
}

\cs_new_protected:Nn \__stex_expr_varseq_in_map:nnnnnnnn {
  \__stex_expr_assoc_make_seq:nnn{#2}{#6}{#3}
}
```

(*End of definition for* `\_stex_term_arg_aB:nnnnn`. *This function is documented on page 151.*)

sfunction

32.4.3 Notation components

```
⟨@@=stex_comps⟩
```

\comp
\compemph@uri
\compemph

```
\cs_set_protected:Npn \comp {}

\cs_new_protected:Npn \compemph@uri #1 #2 {
```

```
    \compemph{ #1 }
}

\cs_new_protected:Npn \compemph #1 { #1 }

\cs_new_protected:Npn \_comp {
  \_do_comp:nnNn {comp}{}\compemph@uri
}

\cs_new_protected:Npn \_thiscomp #1 #2 {
  {\tl_set:cn{this}{{}}#1{#2}\c_math_subscript_token{
    \group_begin:
    \bool_set_true:N \l_stex_allow_semantic_bool
    \bool_set_true:N \l_stex_in_this_bool
    \l_stex_struct_this_tl
    \group_end:
  }
  }
}

\def\maincomp{\comp}
```

(*End of definition for* `\comp`, `\compemph@uri`, *and* `\compemph`. *These functions are documented on page 151.*)

sfunction

`\varemph@uri`
`\varemph`

```
\cs_new_protected:Npn \varemph@uri #1 #2 {
    \varemph{#1}
}

\cs_new_protected:Npn \varemph #1 { #1 }

\cs_new_protected:Npn \_varcomp {
  \_do_comp:nnNn {comp}{}\varemph@uri
}
```

(*End of definition for* `\varemph@uri` *and* `\varemph`. *These functions are documented on page 151.*)

sfunction

`\defemph@uri`
`\defemph`

```
\cs_new_protected:Npn \defemph@uri #1 #2 {
    \defemph{#1}
}

\cs_new_protected:Npn \defemph #1 {
    \ifmmode\else\expandafter\textbf\fi{#1}
}

\cs_new_protected:Npn \_defcomp {
  \_do_comp:nnNn {defcomp}{}\defemph@uri
}
```

(*End of definition for* `\defemph@uri` *and* `\defemph`. *These functions are documented on page 151.*)

sfunction

\symrefemph@uri
\symrefemph

```
\cs_new_protected:Npn \symrefemph@uri #1 #2 {
    \symrefemph{#1}
}

\cs_new_protected:Npn \symrefemph #1 { \emph{#1} }
```

(*End of definition for* `\symrefemph@uri` *and* `\symrefemph`*. These functions are documented on page 151.*)

sfunction

The following commands do the actual attributes:

_do_comp:nnNn

```
\bool_new:N \g_stex_in_comp_bool

\cs_new_protected:Nn \_do_comp:nnNn {
  \stex_pseudogroup_with:nn{\comp}{
    \def\comp##1{##1}
    \bool_set_true:N \g_stex_in_comp_bool
    \stex_if_html_backend:TF {
      \stex_annotate:nn { data-ftml-#1 = {#2}}{ #4 }
    }{
      \tl_if_empty:NTF \l_stex_current_full_tl {
        #4
      }{
        #3 { #4 } \l_stex_current_full_tl

      }
    }
    \bool_set_false:N \g_stex_in_comp_bool
  }
}
```

(*End of definition for* `\_do_comp:nnNn`*. This function is documented on page 151.*)

sfunction

32.4.4 Symbol References

Keys:

```
\stex_keys_define:nnnn{symname}{
  \tl_clear:N \l_stex_key_pre_tl
  \tl_clear:N \l_stex_key_post_tl
  %\tl_clear:N \l_stex_key_root_tl
}{
  pre    .tl_set:N   = \l_stex_key_pre_tl ,
  post   .tl_set:N   = \l_stex_key_post_tl ,
  %root   .code:n     = {}%.tl_set:N   = \l_stex_key_root_tl
}{}

\stex_keys_define:nnnn{symref}{
  %\tl_clear:N \l_stex_key_root_tl
}{
  root   .code:n     = {}%.tl_set:N   = \l_stex_key_root_tl
}{}
```

\symref
\sr

```
\NewDocumentCommand \symref { O{} m m} {
  \group_begin:
  \stex_keys_set:nn{symname}{#1}
  \stex_get_symbol:n{#2}
  \__stex_comps_do_ref:nNn{#3}\symrefemph@uri\_stex_term_oms:nn
}
\let\sr\symref
```

(*End of definition for* \symref *and* \sr*. These functions are documented on page 152.*)

sfunction

\symname
\sn
\sns
\Symname
\Sn\Sns

```
\cs_new:Nn \__stex_comps_split_slash:n {
  \__stex_comps_split_slash_i:nw #1//
}

\cs_new:Npn \__stex_comps_split_slash_i:nw #1/#2/ {
  \tl_if_empty:nTF{#2}{#1}{
    \__stex_comps_split_slash_i:nw #2/
  }
}

\NewDocumentCommand \symname { O{} m} {
  \group_begin:
  \stex_keys_set:nn{symname}{#1}
  \stex_get_symbol:n{#2}
  \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
  \tl_set:Ne \l_stex_current_display_tl {
    \exp_args:Ne \__stex_comps_split_slash:n {\stex_symbol_uri_name:N \l_stex_get_symbol_ur
  }
  \__stex_comps_do_ref:nNn{
    \l_stex_key_pre_tl \l_stex_current_display_tl \l_stex_key_post_tl
  }\symrefemph@uri\_stex_term_oms:nn
}
\let\sn\symname
\protected\def\sns{\symname[post=s]}

\cs_new:Nn \__stex_comps_uppercase:n { \uppercase{#1}}

\NewDocumentCommand \Symname { O{} m} {
  \group_begin:
  \stex_keys_set:nn{symname}{#1}
  \stex_get_symbol:n{#2}
  \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
  \tl_set:Ne \l_stex_current_display_tl {
    \exp_args:Ne \__stex_comps_split_slash:n {\stex_symbol_uri_name:N \l_stex_get_symbol_ur
  }
  \__stex_comps_do_ref:nNn{
    \l_stex_key_pre_tl \exp_args:NNe \use:nn \__stex_comps_uppercase:n \l_stex_current_disp
  }\symrefemph@uri\_stex_term_oms:nn
}
\let\Sn\Symname
\protected\def\Sns{\Symname[post=s]}
```

(*End of definition for* \symname *and others. These functions are documented on page 152.*)

sfunction

\varref
\varname
\Varname

```
\NewDocumentCommand \varref { O{} m m} {
  \group_begin:
  \stex_keys_set:nn{symname}{#1}
  \stex_get_var:n{#2}
  \__stex_comps_do_ref:nNn{#3}\varemph@uri{
    \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
    \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
    \def\comp{\_varcomp}
    \_stex_term_omv:nn
  }
}

\NewDocumentCommand \varname { O{} m} {
  \group_begin:
  \stex_keys_set:nn{symname}{#1}
  \stex_get_var:n{#2}
  \__stex_comps_do_ref:nNn{
    \l_stex_key_pre_tl\l_stex_get_variable_str\l_stex_key_post_tl
  }\varemph@uri{
    \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
    \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
    \def\comp{\_varcomp}
    \_stex_term_omv:nn
  }
}

\NewDocumentCommand \Varname { O{} m} {
  \group_begin:
  \stex_keys_set:nn{symname}{#1}
  \stex_get_var:n{#2}
  \__stex_comps_do_ref:nNn{
    \l_stex_key_pre_tl\exp_after:wN\__stex_comps_uppercase:n\l_stex_get_variable_str\l_stex
  }\varemph@uri{
    \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
    \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
    \def\comp{\_varcomp}
    \_stex_term_omv:nn
  }
}
```

(*End of definition for* \varref*,* \varname*, and* \Varname*. These functions are documented on page 152.*)

sfunction

\definiendum
\definame
\Definame
\defnotation

```
\stex_keys_define:nnnn{defname}{}{
  gf     .code:n       = {},
  root   .code:n       = {}
}{symname}

\stex_keys_define:nnnn{definiendum}{}{
```

```
  gf     .code:n       = {},
  root   .code:n       = {}
}{}

\NewDocumentCommand \definiendum { O{} m m} {
  \stex_keys_set:nn{definiendum}{ #1 }
  \__stex_comps_do_defref:nn{#2}{#3}
}
\stex_deactivate_macro:Nn \definiendum {definition~environments}

\NewDocumentCommand \definame { O{} m } {
  \stex_keys_set:nn{defname}{#1}
  \__stex_comps_do_defref:nn{#2}{
    \l_stex_key_pre_tl\l_stex_current_display_tl\l_stex_key_post_tl
  }
}
\protected\def\definames{\definame[post=s]}
\stex_deactivate_macro:Nn \definame {definition~environments}


\NewDocumentCommand \Definame { O{} m } {
  \stex_keys_set:nn{defname}{#1}
  \__stex_comps_do_defref:nn{#2}{
    \l_stex_key_pre_tl \exp_args:NNe \use:nn \__stex_comps_uppercase:n \l_stex_current_disp
  }
}
\protected\def\Definames{\Definame[post=s]}
\stex_deactivate_macro:Nn \Definame {definition~environments}


\NewDocumentCommand \defnotation{ m } {
  \_stex_next_symbol:n { \def\comp{\_defcomp}}#1
}
\stex_deactivate_macro:Nn \defnotation {definition~environments}
```

(*End of definition for* `\definiendum` *and others. These functions are documented on page 152.*)

sfunction

Does the actual referencing:

```
\cs_new_protected:Nn \__stex_comps_do_ref:nNn {
  \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
  \bool_if:NTF \l_stex_allow_semantic_bool{
    \tl_set_eq:NN \l_stex_current_symbol_uri \l_stex_get_symbol_uri
    \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_current_symbol_uri
    %\str_set:Ne \l_stex_current_symbol_name_str { \stex_symbol_uri_name:N \l_stex_current_
    %\str_if_in:NnT \l_stex_current_symbol_name_str / {
    %  \str_set:Ne \l_stex_current_symbol_name_str {
    %    \exp_after:wN \__stex_comps_slash:w \l_stex_current_symbol_name_str
    %    /\_stex_args_end:
    %  }
    %}
    \tl_clear:N \l_stex_current_term_tl
    \def\comp{\_comp}
    \let\compemph@uri#2
    #3{}{\comp{#1}}
```

```
  }{
    \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
  }
  \group_end:
}
```

\definame et al:

```
\cs_new_protected:Nn \__stex_comps_do_defref:nn {
  \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
  \group_begin:
  \stex_get_symbol:n{#1}
  \bool_if:NTF \l_stex_allow_semantic_bool{
    \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
    \tl_set:Nn \l_stex_current_display_tl {
      \stex_symbol_uri_name:N \l_stex_get_symbol_uri
    }
    %\str_if_in:NnT \l_stex_get_svariable_str / {
    %  \str_set:Ne \l_stex_get_variable_str {
    %    \exp_after:wN \__stex_comps_slash:w \l_stex_get_variable_str
    %    /\_stex_args_end:
    %  }
    %}
    \exp_args:Ne \stex_ref_new_sym_target:n \l_stex_current_full_tl
    \_do_comp:nnNn {definiendum}{\l_stex_current_full_tl}\defemph@uri{#2}
  }{
    \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
  }
  \group_end:
}

\cs_new:Npn \__stex_comps_slash:w #1/#2/#3\_stex_args_end: {
  \tl_if_empty:nTF{#3}{
  #2
  }{
    \__stex_comps_slash:w #2 / #3 \_stex_args_end:
  }
}
```

\foldexpr

```
\stex_if_html_backend:TF {
  \NewDocumentCommand \foldexpr { O{} mm } {
    \stex_annotate:nn{
      data-ftml-fold-expression={\exp_args:Ne \str_if_eq:nnT{#1}{show}{show}}
    }{
      \stex_annotate:nn{
        data-ftml-fold-short={}
      }{#2}
      #3
    }
  }
}{
  \NewDocumentCommand \foldexpr { O{} mm } {
    \exp_args:Ne \str_if_eq:nnTF{#1}{hide}{
      \underbrace{...}{#2}
```

```
    }{
      \underbrace{#3}{#2}
    }
  }
}
```

(*End of definition for* `\foldexpr`*. This function is documented on page 152.*)

sfunction

32.5 Checking

`\stex_if_check_terms_p:`
`\stex_if_check_terms:`*TF*

```
⟨@@=stex_check⟩
\stex_if_html_backend:TF {
  \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
    \prg_return_false:
  }
}{
  \stex_get_env:Nn\__stex_check_env_str{STEX_CHECKTERMS}
  \str_if_empty:NF\__stex_check_env_str{
    \exp_args:No \str_if_eq:nnF \__stex_check_env_str{false}{
      \bool_set_true:N \c_stex_check_terms_bool
    }
  }
  \bool_if:NTF \c_stex_check_terms_bool {
    \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
      \prg_return_true:
    }
  }{
    \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
      \prg_return_false:
    }
  }
}
```

(*End of definition for* `\stex_if_check_terms:TF`*. This function is documented on page 152.*)

sfunction

`\stex_check_term:nn`

```
\stex_if_check_terms:TF{
  \cs_new_protected:Nn \stex_check_term:nn {
    \marginpar {\tiny Checking~#1:~
      \group_begin:
        $#2$
      \group_end:
    }
  }
}{
  \cs_new_protected:Nn \stex_check_term:nn {}
}
```

(*End of definition for* `\stex_check_term:nn`*. This function is documented on page 152.*)

sfunction

_stex_check_terms:

```
\stex_if_check_terms:TF{
  \cs_new_protected:Nn \_stex_check_terms: {
    \stex_debug:nn{check_terms}{Checking~type...}
    \tl_if_empty:NF \l_stex_key_type_tl {
      \stex_check_term:nn{type}\l_stex_key_type_tl
    }
    \stex_debug:nn{check_terms}{Checking~definiens...}
    \tl_if_empty:NF \l_stex_key_def_tl {
      \stex_check_term:nn{definiens}\l_stex_key_def_tl
    }
    \stex_debug:nn{check_terms}{Checking~return...}
    \tl_if_empty:NF \l_stex_key_return_tl {
      \stex_check_term:nn{return}{\l_stex_key_return_tl!}
    }
    \stex_debug:nn{check_terms}{Checking~argument~types...}
    \group_begin:\l_stex_key_argtypes_clist\group_end:
    \tl_if_empty:NF \l_stex_key_argtypes_clist {
      \stex_check_term:nn{argument~types}{\l_stex_key_argtypes_clist}
    }
  }
}{
  \cs_new_protected:Nn \_stex_check_terms: {}
}
```

(*End of definition for* `\_stex_check_terms:`. *This function is documented on page 152.*)

sfunction

Chapter 33

Notations

```
⟨@@=stex_notations⟩
```

keys:

```
\stex_keys_define:nnnn{notation}{
  \str_clear:N \l_stex_key_variant_str
  \str_clear:N \l_stex_key_prec_str
  \str_clear:N \l_stex_key_op_tl
  \str_clear:N \l_stex_key_intent_str
  \clist_clear:N \l_stex_key_intent_args_clist
}{
  variant    .str_set_x:N  = \l_stex_key_variant_str ,
  prec       .str_set_x:N  = \l_stex_key_prec_str ,
  op         .tl_set:N     = \l_stex_key_op_tl ,
  intent     .str_set:N    = \l_stex_key_intent_str ,
  argnames   .clist_set:N  = \l_stex_key_intent_args_clist ,
  unknown    .code:n       = {
    \str_if_empty:NTF \l_keys_key_str {
        \str_set:Ne \l_stex_key_variant_str {\l_keys_key_tl}
    }{
        \str_set_eq:NN \l_stex_key_variant_str \l_keys_key_str
    }
  }
}{style}

\stex_keys_define:nnnn{symdef}{}{}{decl,notation}

\stex_keys_define:nnnn{vardef}{
  \bool_set_false:N \l__stex_notations_bind_bool
}{
  bind  .bool_set:N  = \l__stex_notations_bind_bool
}{symdef}

```

\notation

```
\stex_new_stylable_cmd:nnnn {notation} { s m O{} m} {
  \stex_keys_set:nn{notation}{#3}
  \stex_get_symbol:n{#2}
  \stex_notation_parse:n{#4}
  \stex_if_check_terms:T{ \_stex_notation_check: }
```

```
  \_stex_notation_add:
  \stex_if_do_html:T {
    \def\comp{\_comp}
    \_stex_notation_do_html:n{\stex_use_symbol_uri:N \l_stex_get_symbol_uri}
  }
  \IfBooleanTF#1{
    \_stex_notation_set_default:n{
      \l_stex_get_symbol_uri
    }
  }{}
  \stex_if_smsmode:F{
    \group_begin:
    \__stex_notations_styledefs:
    \stex_style_apply:
    \group_end:
  }
  \stex_smsmode_do:
}{}

\stex_deactivate_macro:Nn \notation {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \notation}
\stex_sms_allow_escape:N \notation

\cs_new_protected:Nn \__stex_notations_styledefs: {
  \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
  \str_set:Nn \thisdeclname {\stex_symbol_uri_name:N \l_stex_get_symbol_uri}
  \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_get_symbol_uri}
  \def\thisnotation{
    \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs{}}{
      \_stex_notation_make_args:
    }
  }
}
```

(*End of definition for* `\notation`. *This function is documented on page 153.*)

sfunction

`\varnotation`

```
\stex_new_stylable_cmd:nnnn {varnotation} { s m O{} m} {
  \stex_keys_set:nn{notation}{#3}
  \stex_get_var:n{#2}
  \str_set_eq:NN \l_stex_key_name_str \l_stex_get_variable_str
  \stex_notation_parse:n{#4}
  \stex_if_check_terms:T{ \_stex_notation_check: }
  \_stex_var_notation_macro:
  \IfBooleanTF#1{
    \_stex_notation_set_default:n{\l_stex_get_variable_str}
  }{}
  \group_begin:
  \tl_set_eq:NN \thisvarname \l_stex_get_variable_str
  \tl_clear:N \thisstyle
  \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
  \def\thisnotation{
    \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs{}}{
```

```
      \_stex_notation_make_args:
    }
  }
  \stex_style_apply:
  \group_end:
}{}
```

(*End of definition for* `\varnotation`. *This function is documented on page 153.*)

sfunction

`\stex_notation_parse:n`

```
\cs_new_protected:Nn \stex_notation_parse:n {
  \tl_if_empty:NF \l_stex_key_op_tl {
    \tl_set:Ne \l_stex_key_op_tl { \exp_not:N\maincomp {
      \exp_args:No \exp_not:n \l_stex_key_op_tl
    } }
  }
  \seq_clear:N \l__stex_notations_precs_seq
  \tl_clear:N \l_stex_notation_args_tl
  \int_compare:nNnTF \l_stex_get_symbol_arity_int = 0 {
    \__stex_notations_const_precs:
    \tl_if_empty:NT \l_stex_key_op_tl {
      \tl_set:Nn \l_stex_key_op_tl { \maincomp{#1} }
    }
  }{
    \__stex_notations_fun_precs:
    \__stex_notations_do_missing_args:n{#1}
    \tl_if_empty:NT \l_stex_key_op_tl {
      \hbox_set:Nn \l_tmpa_box {
        \tl_clear:N \l_stex_current_full_tl
        \tl_clear:N \l_stex_current_display_tl
        \cs_set:Npn \l_tmpa_cs ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #1 }
        \cs_set:Npn \maincomp ##1 {
          \tl_gset:Nn \l_stex_key_op_tl { \maincomp{##1} }
          ##1
        }
        \cs_set:Npn \argsep ##1 ##2 {##1 ##2}
        \cs_set:Npn \argmap ##1 ##2 ##3 {##1 ##3}
        \cs_set:Npn \argarraymap ##1 ##2 ##3 ##4 {
          ##1 ##2
        }
        \stex_suppress_html:n{$\l_tmpa_cs abcdefghj$}
      }
    }
  }
  \exp_args:NNe
  \tl_set:Nn \l_stex_notation_macrocode_cs {
    \STEXInternalNotation
      { \l_stex_key_variant_str }
      { \l__stex_notations_opprec_tl }
      { \l_stex_key_intent_str }
      { \l_stex_notation_args_tl }
      {
        \int_compare:nNnTF \l_stex_get_symbol_arity_int = 0
```

```
        { \exp_not:n { \maincomp{ #1 } } }
        { \exp_not:n { #1 } \l__stex_notations_missing_tl }
      }
  }
  \stex_debug:nn{notation}{Notation:~\meaning\l_stex_notation_macrocode_cs}
}
```

precedences:

```
\cs_new_protected:Nn \__stex_notations_const_precs: {
  \str_if_empty:NTF \l_stex_key_prec_str {
    \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
  }{
    \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
      \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
    }{
      \tl_set_eq:NN \l__stex_notations_opprec_tl \l_stex_key_prec_str
    }
  }
}

\cs_new_protected:Nn \__stex_notations_fun_precs: {
  \str_if_empty:NTF \l_stex_key_prec_str {
    \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
  }{
    \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
      \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
    }{
      \tl_set_eq:NN \l__stex_notations_opprec_tl \l_stex_key_prec_str
    }
  }
  \str_if_empty:NTF \l_stex_key_prec_str {
    \tl_set:Nn \l__stex_notations_opprec_tl { 0 }
    \int_step_inline:nn \l_stex_get_symbol_arity_int {
      \seq_put_right:Nn \l__stex_notations_precs_seq {0}
    }
  }{
    \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
      \stex_debug:nn{notation}{No~brackets}
      \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
      \int_step_inline:nn \l_stex_get_symbol_arity_int {
        \exp_args:NNo \seq_put_right:Nn \l__stex_notations_precs_seq \infprec
      }
    }\__stex_notations_parse_precs:
  }
  \__stex_notations_do_argnames:
}

\cs_new_protected:Nn \__stex_notations_parse_precs: {
  \stex_debug:nn{notation}{parsing~precedence~\l_stex_key_prec_str}
  \seq_set_split:NnV \l__stex_notations_seq ; \l_stex_key_prec_str
  \seq_pop_left:NNTF \l__stex_notations_seq \l__stex_notations_str {
    \tl_set_eq:NN \l__stex_notations_opprec_tl \l__stex_notations_str
    \seq_pop_left:NNT \l__stex_notations_seq \l__stex_notations_str {
      \exp_args:NNo \seq_set_split:NnV \l__stex_notations_seq
        {\tl_to_str:n{x}} \l__stex_notations_str
```

```
    }
  }{
    \tl_set:No \l__stex_notations_opprec_tl { 0 }
  }
  \int_step_inline:nn \l_stex_get_symbol_arity_int {
    \seq_pop_left:NNTF \l__stex_notations_seq \l__stex_notations_str {
      \seq_put_right:No \l__stex_notations_precs_seq \l__stex_notations_str
    }{
      \seq_put_right:No \l__stex_notations_precs_seq \l__stex_notations_opprec_tl
    }
  }
}
```

Experimental; named arguments:

```
\cs_new_protected:Nn \__stex_notations_do_argnames: {
  \tl_clear:N \l_stex_notation_args_tl
  \_stex_map_args:N \__stex_notations_do_argname:nn
}

\cs_new_protected:Nn \__stex_notations_do_argname:nn {
  \clist_if_empty:NTF \l_stex_key_intent_args_clist {
    \tl_put_right:Ne \l_stex_notation_args_tl {
      #1#2{\seq_item:Nn \l__stex_notations_precs_seq #1}{
        \str_if_empty:NF \l_stex_key_intent_str {#1}
      }
    }
  }{
    \tl_put_right:Ne \l_stex_notation_args_tl {
      #1#2{\seq_item:Nn \l__stex_notations_precs_seq #1}
      {\c_dollar_str\clist_item:Nn \l_stex_key_intent_args_clist 1}
    }
    \clist_pop:NN \l_stex_key_intent_args_clist \l_tmpa_tl
  }
}
```

inserts hidden arguments that don't occur in the body of the notation:

```
\cs_new:Nn \__stex_notations_do_missing_args:n {
  \str_set:Nn \l__stex_notations_missing_str {#1}
  \exp_args:NNe \seq_set_split:NnV \l_tmpa_seq {\c_hash_str\c_hash_str\c_hash_str\c_hash_st
  \str_clear:N \l__stex_notations_missing_str
  \seq_map_inline:Nn \l_tmpa_seq {
    \tl_put_right:Nn \l__stex_notations_missing_str { ##1 }
  }
  \tl_clear:N \l__stex_notations_missing_tl
  \_stex_map_args:N \__stex_notations_add_missing_args:nn
}

\cs_new:Nn \__stex_notations_add_missing_args:nn {
  % TODO this skips arguments if e.g. ##1 occurs rather than #1!!
  \exp_args:NNe \str_if_in:NnF \l__stex_notations_missing_str {\c_hash_str\c_hash_str#1}{
    \tl_put_right:Nn \l__stex_notations_missing_tl{\STEXinvisible{## #1}}
  }
}
```

(*End of definition for* `\stex_notation_parse:n`. *This function is documented on page 153.*)

sfunction

_stex_notation_add:

```
\cs_new_protected:Nn \_stex_notation_add: {
  \stex_add_notation:ooeoo
    {\l_stex_get_symbol_uri}
    \l_stex_key_variant_str
    {\int_use:N \l_stex_get_symbol_arity_int}
    \l_stex_notation_macrocode_cs
    \l_stex_key_op_tl
}
```

(*End of definition for* _stex_notation_add:. *This function is documented on page 153.*)

sfunction

\stex_add_notation:nnnnn
\stex_add_notation:ooeoo

```
\cs_new:Nn \__stex_notations_macro:nn {
  l_stex_notation_ #1?#2 _cs
}
\cs_new:Nn \__stex_notations_op_macro:nn {
  l_stex_notation_ #1?#2 _op_cs
}

\cs_new_protected:Nn \stex_add_notation:nnnnn {
  \exp_args:Nne \stex_debug:nn{notations}{Adding~notation:^^J
    #1~\c_hash_str#2~#3^^J
    \tl_to_str:n{#4}^^J\tl_to_str:n{#5}^^J
    to~\stex_use_module_uri:N \l_stex_current_module_uri
  }
  \prop_gput:cen{ \_stex_notations_macro:N \l_stex_current_module_uri }
  {\stex_use_symbol_uri:n{#1}?#2}{{#1}{#2}{#3}{#4}{#5}}
  \stex_do_up_to_module:n {
    \__stex_notations_set_macro:nnnnn{#1}{#2}{#3}{#4}{#5}
    %\__stex_notations_activate_not:nn{#1}{#2}
  }
}
\cs_generate_variant:Nn \stex_add_notation:nnnnn {ooeoo}

\cs_new_protected:Nn \_stex_activate_notations: {
  \stex_debug:nn{activating}{All~notations:~\cs_meaning:c{ \_stex_notations_macro:N \l_stex
  \prop_map_inline:cn{ \_stex_notations_macro:N \l_stex_current_module_uri }{
    \__stex_notations_set_macro:nnnnn ##2
  }
}

\cs_new_protected:Nn \__stex_notations_activate_not:nn {
  \bool_set_false:N \l_tmpa_bool
  \stex_debug:nn{notations}{activating~\stex_use_symbol_uri:n{#1}?#2}
  \prop_map_inline:cn{ \_stex_notations_macro:N \l_stex_current_module_uri }{
    %\stex_debug:nn{notations}{##1?}
    \exp_args:Ne \str_if_eq:nnT{\stex_use_symbol_uri:n{#1}?#2}{##1}{
      \prop_map_break:n{
        %\stex_debug:nn{notations}{Found:~\tl_to_str:n{##2}}
        \bool_set_true:N \l_tmpa_bool
```

```
        \__stex_notations_set_macro:nnnnn ##2
      }
    }
  }
  \bool_if:NF \l_tmpa_bool {
    \errmessage{Woop}
  }
}

\cs_new_protected:Nn \__stex_notations_set_macro:nnnnn {
  \tl_set:cn {\__stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{#2}}{#4}
  \cs_if_exist:cF{\__stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{
    \tl_set:cn {\__stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{#4}
  }
  \tl_if_empty:nF{#5}{
    \tl_set:cn{\__stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{#2}}{#5}
    \cs_if_exist:cF{\__stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{
      \tl_set:cn{\__stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{#5}
    }
  }
}
```

(*End of definition for* `\stex_add_notation:nnnnn`*. This function is documented on page 153.*)

sfunction

`\_stex_map_args:N`
`\_stex_map_notation_args:N`

```
\cs_new:Nn \_stex_map_args:N {
  \tl_if_empty:NF \l_stex_get_symbol_args_tl {
    \exp_after:wN \__stex_notations_map_args_i:w \exp_after:wN
    #1 \l_stex_get_symbol_args_tl \__stex_notations_args_end:
  }
}
\cs_new:Npn \__stex_notations_map_args_i:w #1 #2 #3 #4 \__stex_notations_args_end: {
  #1 {#2} {#3}
  \tl_if_empty:nF{#4}{
    \__stex_notations_map_args_i:w #1 #4 \__stex_notations_args_end:
  }
}

\cs_new:Nn \_stex_map_notation_args:N {
  \tl_if_empty:NF \l_stex_notation_args_tl {
    \exp_after:wN \__stex_notations_map_args_ii:w \exp_after:wN
    #1 \l_stex_notation_args_tl \__stex_notations_args_end:
  }
}
\cs_new:Npn \__stex_notations_map_args_ii:w #1 #2 #3 #4 #5 #6 \__stex_notations_args_end: {
  #1 {#2} {#3} {#4} {#5}
  \tl_if_empty:nF{#6}{
    \__stex_notations_map_args_ii:w #1 #6 \__stex_notations_args_end:
  }
}
```

(*End of definition for* `\_stex_map_args:N` *and* `\_stex_map_notation_args:N`*. These functions are documented on page 153.*)

sfunction

_stex_var_notation_macro:

```
\cs_new_protected:Nn \_stex_var_notation_macro: {
  \tl_set_eq:cN {\__stex_notations_macro:nn{\l_stex_key_name_str}{\l_stex_key_variant_str}}
  \cs_if_exist:cF {\__stex_notations_macro:nn{\l_stex_key_name_str}{}}{
    \tl_set_eq:cN{\__stex_notations_macro:nn{\l_stex_key_name_str}{}}
      \l_stex_notation_macrocode_cs
  }
  \tl_if_empty:NF \l_stex_key_op_tl {
    \tl_set_eq:cN {\__stex_notations_op_macro:nn{\l_stex_key_name_str}{\l_stex_key_variant_
    \cs_if_exist:cF{\__stex_notations_op_macro:nn{\l_stex_key_name_str}{}}{
      \cs_set_eq:cN{\__stex_notations_op_macro:nn{\l_stex_key_name_str}{}}
      \l_stex_key_op_tl
    }
  }
}
```

(*End of definition for* _stex_var_notation_macro:. *This function is documented on page 153.*)

sfunction

\setnotation

_stex_notation_set_default:n

```
\cs_new_protected:Nn \_stex_notation_set_default:n{
  \stex_if_in_module:TF{
    \stex_add_notation:ooeoo{#1}{}
      {\int_use:N \l_stex_get_symbol_arity_int}
      \l_stex_notation_macrocode_cs
      \l_stex_key_op_tl
  }{
    \cs_set_eq:cN {\__stex_notations_macro:nn {\stex_use_symbol_uri:N \l_stex_get_symbol_ur
      \l_stex_notation_macrocode_cs
    \tl_if_empty:NF \l_stex_key_op_tl {
      \cs_set_eq:cN{\__stex_notations_op_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol
        \l_stex_key_op_tl
    }
  }
}

\cs_new_protected:Npn \setnotation #1 #2 {
  \stex_get_symbol:n{#1}
  \cs_if_exist:cTF{\__stex_notations_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol_uri
    \tl_set_eq:Nc \l_stex_notation_macrocode_cs {\__stex_notations_macro:nn{\stex_use_symbo
    \cs_if_exist:cTF{\__stex_notations_op_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbo
      \tl_set_eq:Nc \l_stex_key_op_tl {\__stex_notations_op_macro:nn{\stex_use_symbol_uri:N
    }{
      \tl_clear:N \l_stex_key_op_tl
    }
    \_stex_notation_set_default:n{
      \l_stex_get_symbol_uri
    }
  }{
    \msg_error:nnxx{stex}{error/unknownnotation}{#2}{
      \stex_use_symbol_uri:N \l_stex_get_symbol_uri
    }
  }
}
```

(*End of definition for* `\setnotation` *and* `\_stex_notation_set_default:n`*. These functions are documented on page 153.*)

sfunction

`\stex_iterate_notations:nn`

```
\cs_new_protected:Nn \stex_iterate_notations:nn {
  \seq_clear:N \l__stex_notations_mods_seq
  \stex_pseudogroup_with:nn{\__stex_notations_not_cs:nnnnn}{
    \cs_set:Npn \__stex_notations_not_cs:nnnnn
    ##1 ##2 ##3 ##4 ##5 { #2 }
    \clist_map_function:nN {#1} \__stex_notations_it_not_i:n
  }
}

\cs_new_protected:Nn \__stex_notations_it_not_i:n {
  \seq_if_in:NnF \l__stex_notations_mods_seq {#1} {
    \seq_put_left:Nn \l__stex_notations_mods_seq {#1}
    \prop_map_inline:cn{\_stex_notations_macro:n{#1}}{
      \__stex_notations_not_cs:nnnnn ##2
    }
    \prop_map_inline:cn{\_stex_morphisms_macro:n{#1}}{
      \__stex_notations_it_not_check:nnnn ##2
    }
  }
}
\cs_new_protected:Nn \__stex_notations_it_not_check:nnnn {
  \tl_if_empty:nT{#1}{
    \__stex_notations_it_not_i:n {#2}
  }
}
```

(*End of definition for* `\stex_iterate_notations:nn`*. This function is documented on page 154.*)

sfunction

`\stex_use_notation:nnTF`
`\stex_use_op_notation:nnTF`

```
\cs_new_protected:Npn \stex_use_notation:nnTF #1 #2 #3 #4 {
  \cs_if_exist:cTF{\__stex_notations_macro:nn{#1}{#2}}{
    \cs_set_eq:Nc \l_stex_notation_cs {\__stex_notations_macro:nn{#1}{#2}}
    #3
  }{#4}
}
\cs_new_protected:Npn \stex_use_op_notation:nnTF #1 #2 #3 #4 {
  \cs_if_exist:cTF{\__stex_notations_op_macro:nn{#1}{#2}}{
    \cs_set_eq:Nc \l_stex_notation_cs {\__stex_notations_op_macro:nn{#1}{#2}}
    #3
  }{#4}
}
```

(*End of definition for* `\stex_use_notation:nnTF` *and* `\stex_use_op_notation:nnTF`*. These functions are documented on page 154.*)

sfunction

`\_stex_notation_check:`

```
\cs_new_protected:Nn \_stex_notation_check: {
```

```
  \stex_check_term:nn{notation}{
    \cs_set:Npn \comp ##1 {##1}
    \stex_debug:nn{check}{Checking~notation:~\cs_meaning:N \l_stex_notation_macrocode_cs ^^
    \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs{}}{
      \_stex_notation_make_args:
    }
  }
}
```

(*End of definition for* `\_stex_notation_check:`. *This function is documented on page 154.*)

sfunction

`\_stex_notation_do_html:n`

```
\cs_new_protected:Nn \_stex_notation_do_html:n {
  \stex_annotate_invisible:n{\_stex_annotate_force_break:n{\hbox{\stex_annotate:nn {
    data-ftml-notation={#1},
    data-ftml-notationfragment={\l_stex_key_variant_str},
    data-ftml-precedence={\l__stex_notations_opprec_tl},
    data-ftml-argprecs={\seq_use:Nn \l__stex_notations_precs_seq ,}
  }{
    \cs_set_protected:Npn \argsep ##1 ##2 {
      \stex_annotate:nn{data-ftml-argsep={}}{
        ##1 \tl_if_empty:nTF{##2}{\!\,}{##2}
      }
    }
    \cs_set_protected:Npn \argmap ##1 ##2 ##3 {
      \cs_set:Npn \__stex_notations_map_cs: ####1 { ##2 }
      \stex_annotate:nn{data-ftml-argmap={}}{
        \__stex_notations_map_cs:{##1} \stex_annotate:nn{data-ftml-argmap-sep={}}{##3}
      }
    }
    \cs_set_protected:Npn \maincomp {
      \_do_comp:nnNn {maincomp}{}\compemph@uri
    }
    \stex_debug:nn{notation}{Doing~notation~HTML:\meaning\l_stex_get_symbol_args_tl}
    $
      \tl_clear:N \l_stex_current_full_tl
      \stex_annotate:nn{data-ftml-notationcomp={}}{
        \exp_args:Nne \use:nn {
          \l_stex_notation_macrocode_cs {}
        }{
          \_stex_map_args:N \__stex_notations_make_arg_html:nn
        }
      }
    $
    \tl_if_empty:NF \l_stex_key_op_tl {
      $
      \tl_clear:N \l_stex_current_full_tl
      \stex_annotate:nn{data-ftml-notationopcomp={}}{
        \_stex_term_oms:nn{\l_stex_key_variant_str}{\l_stex_key_op_tl}
      }
      $
    }
  }}
```

```
  }}
}

\cs_new:Nn \__stex_notations_make_arg_html:nn {
  {\stex_annotate:nn{data-ftml-argnum=#1}{x}}
}
```

(*End of definition for* `\_stex_notation_do_html:n`. *This function is documented on page 154.*)

sfunction

`\_stex_notation_make_args:`

```
\cs_new:Nn \_stex_notation_make_args: {
  \_stex_map_notation_args:N \__stex_notations_make_arg:nnnn
}

\cs_new:Nn \__stex_notations_make_arg:nnnn  {
  \str_case:nnF #2 {
    a {{
        a\c_math_subscript_token{#1,1},
        a\c_math_subscript_token{#1,2}
      }}
    B {{
        B\c_math_subscript_token{#1,1},
        B\c_math_subscript_token{#1,2}
      }}
  }{{
    \_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}
      {{#2}\c_math_subscript_token{#1}}
  }}
}
```

(*End of definition for* `\_stex_notation_make_args:`. *This function is documented on page 154.*)

sfunction

`\stex_do_default_notation:`

`\stex_do_default_notation_op:`

```
\cs_new_protected:Nn \stex_do_default_notation: {
  \stex_do_default_notation_op:
  \tl_if_empty:NTF \l_stex_current_args_tl {
    \tl_clear:N \l__stex_notations_args_tl
  }\__stex_notations_default_args:
  \tl_set:Ne \l_stex_default_notation {\STEXInternalNotation{}{0}{}{\l__stex_notations_args
    \exp_args:No \exp_not:n \l_stex_default_notation
  }}
}

\cs_new_protected:Nn \stex_do_default_notation_op: {
  \__stex_notations_make_name:
  \tl_set:Ne \l_stex_default_notation {\exp_not:N \maincomp{ \exp_not:N \mathrm {\l__stex_n
}

\cs_new_protected:Nn \__stex_notations_default_args: {
  \__stex_notations_make_name:
  \tl_set_eq:NN \l_stex_get_symbol_args_tl \l_stex_current_args_tl
  \tl_set:Ne \l__stex_notations_args_tl {
```

```
    \_stex_map_args:N \__stex_notations_augment_arg:nn
  }
  \tl_put_right:Nn \l_stex_default_notation {\comp(}
  \seq_clear:N \l_tmpa_seq
  \int_step_inline:nn \l_stex_current_arity_str {
    \seq_put_right:Nn \l_tmpa_seq {#### ##1}
  }
  \tl_put_right:Ne \l_stex_default_notation {
    \seq_use:Nn \l_tmpa_seq {\mathpunct{\comp{,}}}
  }
  \tl_put_right:Nn \l_stex_default_notation {\comp)}
}

\cs_new:Nn \__stex_notations_augment_arg:nn {
  #1#2{0}{}
}

\cs_new_protected:Nn \__stex_notations_make_name: {
  \exp_args:NNNe \seq_set_split:Nnn \l_tmpa_seq / \l_stex_current_display_tl
  \seq_pop_right:NN \l_tmpa_seq \l__stex_notations_name_str
}
```

(*End of definition for* `\stex_do_default_notation:` *and* `\stex_do_default_notation_op:`*. These functions are documented on page 154.*)

sfunction

`\STEXInternalNotation`

```
\cs_new_protected:Npn \STEXInternalNotation #1 #2 #3 #4 #5 #6 {
  \__stex_notations_process:nnnnnn{#1}{#2}{#3}{#4}{#5}{
    \l__stex_notations_code_tl
    #6
  }
}

\cs_new_protected:Npn \__stex_notations_process:nnnnnn #1 #2 #3 #4 {
  \tl_if_empty:nTF{#4}{
    \__stex_notations_simple:nnnnn{#1}{#2}{#3}
  }{
    \__stex_notations_complex:nnnnnn{#1}{#2}{#3}{#4}
  }
}

\cs_new_protected:Nn \__stex_notations_simple:nnnnn {
  \stex_debug:nn{Notation~code}{\tl_to_str:n{#4}}
  \tl_set:Nn \l__stex_notations_code_tl {
    \cs_set:Npn \l__stex_notations_cs {
      \_stex_maybe_brackets:nn{#2}{
        \_stex_term_oms_or_omv:nn{#1}{#4}
      }
    }
    \l__stex_notations_cs
  }
  #5
}
```

```
\cs_new_protected:Nn \__stex_notations_complex:nnnnnn {
  \stex_debug:nn{Notation~code}{\tl_to_str:n{#5}}
  \int_zero:N \l_tmpa_int
  \tl_set:Nn \l__stex_notations_pre_tl {\cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term
  \tl_set:Nn \l__stex_notations_code_tl {
    \cs_generate_from_arg_count:NNnn \l__stex_notations_cs \cs_set:Npn \l_tmpa_int
    {
      \_stex_maybe_brackets:nn{#2}{
        \_stex_term_oma_or_omb:nn{#1}{
          \bool_set_false:N \l_stex_brackets_dones_bool
          #5
        }
      }
    }
    \l__stex_notations_cs
  }
  \tl_set:Nn \l__stex_notations_after_tl{
    \exp_args:NNo
    \tl_put_left:Nn \l__stex_notations_code_tl \l__stex_notations_pre_tl
    \tl_put_left:Ne \l__stex_notations_code_tl {
      \int_set:Nn \l_tmpa_int {\int_use:N \l_tmpa_int}
    }
    #6
  }
  \__stex_notations_parse_args:nnnnw #4 \__stex_notations_args_end:
}

\cs_new_protected:Npn \__stex_notations_parse_args:nnnnw #1 #2 #3 #4 #5 \__stex_notations_a
  \tl_if_empty:nTF{#5}{
    \__stex_notations_add_last:nnnnn{#1}{#2}{#3}{#4}
  }{
    \__stex_notations_add_next:nnnnnn{#1}{#2}{#3}{#4}{#5}
  }
}

\cs_new_protected:Nn \__stex_notations_add_next:nnnnnn {
  \__stex_notations_add:nnnnn{#1}{#2}{#3}{#4}{#6}
  \__stex_notations_parse_args:nnnnw #5 \__stex_notations_args_end:
}

\cs_new_protected:Nn \__stex_notations_add_last:nnnnn {
  \__stex_notations_add:nnnnn{#1}{#2}{#3}{#4}{#5}
  \stex_debug:nn{sequences}{Executing~notation:~\meaning\l__stex_notations_code_tl}
  \l__stex_notations_after_tl
}

\cs_new_protected:Nn \__stex_notations_add:nnnnn {
  \int_incr:N \l_tmpa_int
  \str_case:nn{#2}{
    i {
      \tl_put_right:Nn \l__stex_notations_code_tl {
        {\_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}}
      }
```

```
    }
    b {
      \tl_set:Nn \l__stex_notations_pre_tl {
        \cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term_omb:nn
      }
      \tl_put_right:Nn \l__stex_notations_code_tl {
        {\_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}}
      }
    }
    a {
      \tl_put_right:Nn \l__stex_notations_code_tl {
        {\_stex_term_arg_aB:nnnnn{#1}{#2}{#3}{#4}{#5}}
      }
    }
    B  {
      \tl_set:Nn \l__stex_notations_pre_tl {
        \cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term_omb:nn
      }
      \tl_put_right:Nn \l__stex_notations_code_tl {
        {\_stex_term_arg_aB:nnnnn{#1}{#2}{#3}{#4}{#5}}
      }
    }
  }
}
```

(*End of definition for* `\STEXInternalNotation`. *This function is documented on page 154.*)

sfunction

`\argsep`

```
\cs_new_protected:Npn \argsep #1 #2 {
  \__stex_notations_check_aB_arg:Nn\argsep{#1}
  \stex_pseudogroup_with:nn{\_stex_term_do_aB_clist:}{
    \tl_set:Nn \_stex_term_do_aB_clist: {
      \seq_use:Nn \l_stex_aB_args_seq {#2}
    }
    #1
  }
}

\cs_new_protected:Nn \__stex_notations_check_aB_arg:Nn {
  \exp_args:Ne \cs_if_eq:NNF {\tl_head:n{#2}}
    \_stex_term_arg_aB:nnnnn {
    \msg_error:nnx{stex}{error/assocarg}{\tl_to_str:n{#1}}
  }
}
```

(*End of definition for* `\argsep`. *This function is documented on page 154.*)

sfunction

`\seqmap` implemented in notations via "pattern matching" the argument

```
\cs_new_protected:Npn \seqmap #1 #2 {
  \symuse{Metatheory?sequence~expression}{\seqmap{#1}{#2}}%\l_tmpa_tl {#2}
}
```

(*End of definition for* \seqmap. *This function is documented on page 154.*)

sfunction

\argmap

```
\cs_new_protected:Npn \argmap #1 #2 #3 {
  \__stex_notations_check_aB_arg:Nn\argmap{#1}
  \stex_pseudogroup_with:nn{
    \_stex_term_do_aB_clist:
    \__stex_notations_map_cs:
  }{
    \cs_set:Npn \__stex_notations_map_cs: ##1 { #2 }
    \tl_set:Nn \_stex_term_do_aB_clist: {
      \seq_clear:N \l_tmpa_seq
      \seq_map_inline:Nn \l_stex_aB_args_seq {
        \tl_if_eq:nnTF{##1}{\ellipses}{
          \seq_put_right:Nn \l_tmpa_seq \ellipses
        }{
          \seq_put_right:Nx \l_tmpa_seq {
            \exp_after:wN \exp_not:n \exp_after:wN { \__stex_notations_map_cs: {##1} }
          }
        }
      }
      \seq_set_eq:NN \l_stex_aB_args_seq \l_tmpa_seq
      \seq_use:Nn \l_stex_aB_args_seq {#3}
    }
    #1
  }
}
```

(*End of definition for* \argmap. *This function is documented on page 154.*)

sfunction

\argarraymap

```
\int_new:N \l__stex_notations_clist_count_int
\cs_new_protected:Npn \argarraymap #1 #2 #3 #4 {
  \__stex_notations_check_aB_arg:Nn\argarraymap{#1}
  \stex_pseudogroup_with:nn{
    \_stex_term_do_aB_clist:
    \__stex_notations_map_cs:
  }{
    \cs_set:Npn \__stex_notations_map_cs: ##1 { #3 }
    \int_set:Nn \l__stex_notations_clist_count_int {\exp_args:No\clist_count:n{\tl_to_str:n
    \tl_set:Nn \_stex_term_do_aB_clist: {
      \tl_clear:N \l_tmpa_tl
      \int_zero:N \l_tmpa_int
      \seq_map_inline:Nn \l_stex_aB_args_seq {
        \int_incr:N \l_tmpa_int
        \int_compare:nNnT \l_tmpa_int > \l__stex_notations_clist_count_int {
          \int_set:Nn \l_tmpa_int 1
        }
        \tl_put_right:Ne \l_tmpa_tl {
          \exp_after:wN \exp_not:n \exp_after:wN { \__stex_notations_map_cs: {##1} }
          \clist_item:nn{#4}\l_tmpa_int
        }
```

```
      }
      \seq_set_eq:NN \l_stex_aB_args_seq \l_tmpa_seq
      \begin{array}{#2}
          \l_tmpa_tl
      \end{array}
    }
    #1
  }
}
```

(*End of definition for* `\argarraymap`*. This function is documented on page 154.*)

sfunction

33.1 Automated Bracketing

Variables for current (downwards) precedence, set of parentheses etc.:

```
\int_new:N \l_stex_notation_downprec
\tl_set:Nn \l__stex_notations_left_bracket_str (
\tl_set:Nn \l__stex_notations_right_bracket_str )
\bool_new:N \l_stex_brackets_dones_bool
```

`\infprec`
`\neginfprec`

```
\tl_const:Ne \infprec {\int_use:N \c_max_int}
\tl_const:Ne \neginfprec {-\int_use:N \c_max_int}

\int_set:Nn \l_stex_notation_downprec \infprec
```

(*End of definition for* `\infprec` *and* `\neginfprec`*. These variables are documented on page 155.*)

`\_stex_maybe_brackets:nn`

```
\cs_new_protected:Nn \_stex_maybe_brackets:nn {
  \bool_if:NTF \l_stex_brackets_dones_bool {
    \bool_set_false:N \l_stex_brackets_dones_bool
    #2
  } {
    \stex_debug:nn{brackets}{#1>\int_eval:n \l_stex_notation_downprec?}
    \int_compare:nNnTF { #1 } > \l_stex_notation_downprec {
      %\bool_if:NTF \l_stex_inparray_bool { #2 }{
        \dobrackets {
          #2
        }
      %}
    }{
      #2
    }
  }
}
```

(*End of definition for* `\_stex_maybe_brackets:nn`*. This function is documented on page 155.*)

sfunction

\dobrackets

```
\cs_new_protected:Npn \dobrackets #1 {
  \group_begin:
    \bool_set_true:N \l_stex_brackets_dones_bool
    \mathopen{\tl_if_exist:NT\l_stex_current_symbol_uri\comp
      \l__stex_notations_left_bracket_str
    }
    #1
  \group_end:
  \mathclose{\tl_if_exist:NT\l_stex_current_symbol_uri\comp
    \l__stex_notations_right_bracket_str
  }
}
```

(*End of definition for* \dobrackets*. This function is documented on page 155.*)

sfunction

\withbrackets

```
\cs_new_protected:Npn \withbrackets #1 #2 #3 {
  \stex_pseudogroup:nn{
    \tl_set:Nn \l__stex_notations_left_bracket_str { #1 }
    \tl_set:Nn \l__stex_notations_right_bracket_str { #2 }
    #3
  }{
    \stex_pseudogroup_restore:N \l__stex_notations_left_bracket_str
    \stex_pseudogroup_restore:N \l__stex_notations_right_bracket_str
  }
}
```

(*End of definition for* \withbrackets*. This function is documented on page 155.*)

sfunction

\dowithbrackets

```
\cs_new_protected:Npn \dowithbrackets #1 #2 #3 {
  \withbrackets{#1}{#2}{\dobrackets{#3}}
}
```

(*End of definition for* \dowithbrackets*. This function is documented on page 155.*)

sfunction

Chapter 34

Structures

```
⟨@@=stex_structures⟩
```

Keys:

```
\stex_keys_define:nnnn{mathstructure}{
  \tl_clear:N \l_stex_current_this_tl
  \str_clear:N \l__stex_structures_name_str
}{
  this    .tl_set:N = \l_stex_current_this_tl ,
  unknown .code:n   = {
    \str_if_empty:NTF \l_keys_key_str {
      \str_set:Ne \l__stex_structures_name_str {\l_keys_key_tl}
    }{
      \str_set_eq:NN \l__stex_structures_name_str \l_keys_key_str
    }
  }
}{}
```

`mathstructure` (*env.*)

```
\stex_new_stylable_env:nnnnnnn {mathstructure}{m O{}}{
  \__stex_structures_begin:nn{#1}{#2}
  \stex_smsmode_do:
}{
  \stex_structural_feature_module_end:
  \__stex_structures_do_externals:
}{}{}{}

\stex_sms_allow_env:n{mathstructure}
\stex_deactivate_macro:Nn \mathstructure {module~environments}
\stex_every_module:n {\stex_reactivate_macro:N \mathstructure}
```

Shared code for `mathstructure` and `extstructure`:

```
\cs_new_protected:Nn \__stex_structures_begin:nn {
  \stex_keys_set:nn {mathstructure}{#2}
  \str_if_empty:NT \l__stex_structures_name_str {
    \str_set:Nn \l__stex_structures_name_str {#1}
  }
  \def\comp{\_comp}

  \tl_set:Ne \l__stex_structures_struct_uri {
```

```
    \exp_args:No \stex_new_module_uri:n \l__stex_structures_name_str
  }

  \exp_args:Nne \use:nn { \stex_add_symbol:nnnnnnnN }
    { {#1}{\l__stex_structures_name_str}{0}{}{defed}{\l__stex_structures_struct_uri}}
    {}\stex_invoke_structure:
  \str_set:Ne \l_stex_macroname_str {#1}

  \exp_args:No \stex_structural_feature_module:nn
    {\l__stex_structures_name_str}{structure}
}

\cs_new_protected:Nn \__stex_structures_do_externals: {
  \tl_set:Nn \l__stex_structures_replace_this_tl {####1}
}

```

`extstructure` (*env.*)

```
\stex_new_stylable_env:nnnnnnn {extstructure}{m O{} m}{
  \seq_clear:N \l__stex_structures_imports_seq
  \clist_map_inline:nn{#3}{
    \stex_get_mathstructure:n{##1}
    \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
      \stex_if_module_exists:nT{####1}{
        \seq_put_right:Nn \l__stex_structures_imports_seq{####1}
      }
    }
  }
  \__stex_structures_begin:nn{#1}{#2}
  \seq_map_inline:Nn\l__stex_structures_imports_seq{
    \stex_if_do_html:T {
      \hbox{\stex_annotate_invisible:nn
        {data-ftml-import={\stex_use_module_uri:n{##1}}} {}}
    }
    \stex_add_morphism:nonn
      {}{##1}{import}{}
    \stex_execute_in_module:e{
      \stex_activate_module:n {##1}
    }
  }
  \stex_smsmode_do:
}{
  \stex_structural_feature_module_end:
  \__stex_structures_do_externals:
}{}{}{}

\stex_sms_allow_env:n{extstructure}
\stex_deactivate_macro:Nn \extstructure {module~environments}
\stex_every_module:n {
  \stex_reactivate_macro:N \extstructure
}

\stex_new_stylable_env:nnnnnnn {extstructure*}{m}{
  \__stex_structures_new_extstruct_name:
```

```
  \seq_clear:N \l__stex_structures_imports_seq
  \stex_get_mathstructure:n{#1}

  \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
    \stex_if_module_exists:nT{##1}{
      \seq_put_right:Nn \l__stex_structures_imports_seq{##1}
    }
  }

  \stex_module_uri_split_name:NNN \l__stex_structures_parent_uri \l__stex_structures_last_n

  \stex_execute_in_module:e{
    \__stex_structures_extend_structure:nnnn{\l__stex_structures_parent_uri}{\l__stex_struc
  }

  \exp_args:No \__stex_structures_begin:nn\l__stex_structures_exstruct_name_str{}

  \seq_map_inline:Nn \l__stex_structures_imports_seq {
    \stex_if_do_html:T {
      \stex_annotate_invisible:nn
        {data-ftml-import= {\stex_use_module_uri:n{##1}}} {}
    }
    %\stex_add_morphism:nonn
    %  {}{##1}{import}{}
    %\stex_execute_in_module:e{
    %  \stex_activate_module:n {##1}
    %}
    \stex_activate_module:n {##1}
  }

  \stex_smsmode_do:
}{
  \prop_map_inline:cn{\_stex_symbol_macro:N \l_stex_current_module_uri }{
    \__stex_structures_check_def:nnnnnnnn ##2
  }
  \stex_structural_feature_module_end:
  %\exp_args:Ne \stex_do_up_to_module:n {
  %  \stex_activate_module:n {\exp_args:No \stex_new_module_uri:n {\l__stex_structures_exst
  %}
  \__stex_structures_do_externals:
}{}{}{}

\stex_sms_allow_env:n{extstructure*}
\exp_after:wN \stex_deactivate_macro:Nn
  \cs:w extstructure*\cs_end: {module~environments}
\stex_every_module:n {
  \exp_after:wN \stex_reactivate_macro:N \cs:w extstructure*\cs_end:
}

\cs_new_protected:Nn \__stex_structures_extend_structure:nnnn {
  \stex_debug:nn{ext}{Extending~\stex_use_module_uri:n {#1} / #2~by~\stex_use_module_uri:n
  \exp_args:Nne \tl_put_right:cn{\_stex_module_code_macro:n{#4}}{
    \stex_activate_module:n{#3}
  }
```

```
  \exp_args:Nne \prop_put:cnn{\_stex_morphisms_macro:n{#4}}{\stex_use_module_uri:n{#3}}{
    {}{#3}{extstruct}{}
  }
  \exp_args:Nne \use:nn {\__stex_structures_extend_ii:nnnn}{
    \exp_args:NNe \use:nn \__stex_structures_extend_i:nnnnnnnNn {
      \prop_item:cn{\_stex_symbol_macro:n {#1} } { #2 }
    } { #3 }
  }{#1}{#2}
}

\cs_new_protected:Nn \__stex_structures_extend_ii:nnnn {
  \prop_put:cnn{\_stex_symbol_macro:n {#3} } { #4 }{#2}
  \tl_set:ce{#1}{
    \_stex_invoke_symbol:nnnnnnN {\stex_new_symbol_uri:nn {#3}{#4}}
    \use_none:nn #2
  }
}

\cs_new:Nn \__stex_structures_extend_i:nnnnnnnNn {
  {#1}{{#1}{#2}{#3}{#4}{#5}{#6,#9}{#7}#8}
}

\cs_new_protected:Nn \__stex_structures_check_def:nnnnnnnn {
  \tl_if_empty:nT{#5}{
    \msg_error:nnnn{stex}{error/needsdefiniens}{#2}{extstructure*}
  }
}

\cs_new_protected:Nn \__stex_structures_new_extstruct_name: {
  \stex_do_up_to_module:n {
    \str_set:Ne \l__stex_structures_extname_count {\int_eval:n{\l__stex_structures_extname_
  }
  \str_set:Ne \l__stex_structures_exstruct_name_str {EXTSTRUCT_\l__stex_structures_extname_
}

\stex_every_module:n{ \str_set:Nn \l__stex_structures_extname_count 0}
```

\this

```
\bool_new:N \l_stex_in_this_bool

\cs_new_protected:Npn \stex_current_this: {
  { \bool_set_true:N \l_stex_allow_semantic_bool
    \tl_if_empty:NTF \l_stex_current_this_tl {{}}{
      \tl_set:Nn \l_stex_current_full_tl {
        \exp_args:Ne \stex_use_symbol_uri:n {
          \exp_args:No \stex_new_symbol_uri:n \l__stex_structures_name_str
        }
      }
      \str_set_eq:NN \l_stex_current_display_tl \l__stex_structures_name_str
      \bool_set_true:N \l_stex_in_this_bool
      \maincomp{\l_stex_current_this_tl}
      \bool_set_false:N \l_stex_in_this_bool
    }
  }
```

```
}

\let \this \stex_current_this:
```

(*End of definition for* `\this`. *This function is documented on page 156.*)

sfunction

`\stex_get_mathstructure:n`

```
\cs_new_protected:Nn \stex_get_mathstructure:n {
  \tl_clear:N \l_stex_get_structure_module_uri
  \stex_get_symbol:nF{#1}{
    \msg_error:nnn{stex}{error/unknownstructure}{#1}
  }
  \exp_args:No \tl_if_eq:NNTF \l_stex_get_symbol_invoke_cs \stex_invoke_structure: {
    \tl_set:Ne \l_stex_get_structure_module_uri { \exp_after:wN \__stex_structures_get:w \l
  }{
    \msg_error:nnn{stex}{error/unknownstructure}{#1}
  }
}

\cs_new:Npn \__stex_structures_get:w #1 , #2 \stex_end: { #1 }
```

(*End of definition for* `\stex_get_mathstructure:n`. *This function is documented on page 156.*)

sfunction

`\usestructure`

```
\cs_new_protected:Npn \usestructure #1 {
  \stex_get_mathstructure:n{ #1 }
  \stex_debug:nn{usestructure}{Using~structure~#1:~\stex_use_module_uri:N \l_stex_get_struc
  \seq_clear:N \l__stex_structures_imports_seq
  \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
    \stex_if_module_exists:nT{##1}{
      \seq_put_right:Nn \l__stex_structures_imports_seq{##1}
    }
  }
  \seq_map_inline:Nn \l__stex_structures_imports_seq {
    \stex_if_do_html:T {
      \hbox{\stex_annotate_invisible:nn
        {data-ftml-usemodule=\stex_use_module_uri:n {##1}} {}}
    }
    \stex_activate_module:n {##1}
  }
}
```

(*End of definition for* `\usestructure`. *This function is documented on page 156.*)

sfunction

Chapter 35

Statements

```
⟨@@=stex_statements⟩
```

Keys:

```
\stex_keys_define:nnnn{statement}{
  \str_clear:N \l_stex_key_name_str
  \str_clear:N \l_stex_key_macroname_str
  \clist_clear:N \l_stex_key_for_clist
  \str_clear:N \l_stex_key_args_str
  \tl_clear:N \l_stex_key_type_tl
  \tl_clear:N \l_stex_key_def_tl
  \tl_clear:N \l_stex_key_return_tl
  \clist_clear:N \l_stex_key_argtypes_clist
}{
  name     .str_set:N = \l_stex_key_name_str ,
  for      .clist_set:N = \l_stex_key_for_clist ,
  macro    .str_set:N = \l_stex_key_macroname_str ,
  % start  .str_set:N = \l_stex_key_title_str , % TODO remove
  type     .tl_set:N = \l_stex_key_type_tl ,
  judgment .code:n = {},
  from     .code:n= {}, % TODO remove
  to       .code:n={} % TODO remove
}{id,title,style,symargs}
```

`\_stex_do_for_list:`

```
\cs_new_protected:Npn \_stex_do_for_list: {
  \seq_clear:N \l_stex_fors_seq
  \clist_map_inline:Nn \l_stex_key_for_clist {
    \exp_args:Ne\stex_get_symbol:n{\tl_to_str:n{##1}}
    \seq_put_right:No \l_stex_fors_seq
      {\l_stex_get_symbol_uri}
  }
}
```

(*End of definition for* `\_stex_do_for_list:`. *This function is documented on page 157.*)

sfunction

`\stex_new_statement:nnn`

```
\cs_new_protected:Nn \stex_new_statement:nnn {
  \stex_new_stylable_env:nnnnnnn {#1}{O{}}{
```

```
    \stex_keys_set:nn{statement}{##1}
    #3

    \stex_if_smsmode:F {
      \exp_args:Nne \begin{stex_annotate_env}{
        \__stex_statements_html_keyvals:nn{#1}{false}
      }
      \tl_set_eq:NN \thistitle \l_stex_key_title_tl
      \str_set_eq:NN \thisname \l_stex_key_name_str
      \clist_set_eq:NN \thisfor \l_stex_key_for_str
      \stex_if_html_backend:TF {
        \noindent
        \stex_annotate:nn{data-ftml-title={}, style:display={none}}{\_stex_annotate_force_b
          \tl_if_empty:NF \l_stex_key_title_tl{~}
          \l_stex_key_title_tl
        }}
      }
      \stex_style_apply:
    }
    \_stex_do_id:
    \stex_smsmode_do:
  }{
    \stex_if_smsmode:F {
      \stex_if_html_backend:F \stex_style_apply:
      \end{stex_annotate_env}
    }
  }{}{}{s}
  \stex_sms_allow_env:n{s#1}

  \tl_if_empty:nF{#2}{\__stex_statements_make_macro:nnn{#1}{#2}{#3}}
}

\cs_new_protected:Nn \__stex_statements_make_macro:nnn {
  \exp_after:wN \NewDocumentCommand \cs:w inline#2\cs_end: { O{} m}{
    \group_begin:
      \stex_keys_set:nn{statement}{##1}
      #3
      \_stex_do_id:
      \stex_if_smsmode:F{
        \exp_args:Ne \stex_annotate:nn{\__stex_statements_html_keyvals:nn{#1}{true}}{
          \_stex_annotate_force_break:n{##2}
        }
      }
    \group_end:
    %\stex_if_smsmode:TF \stex_smsmode_do: \stex_ignore_spaces_and_pars:
    \stex_smsmode_do:
  }
  \exp_after:wN \stex_sms_allow_escape:N\cs:w inline#2\cs_end:
}

\cs_new:Nn \__stex_statements_html_keyvals:nn {
  data-ftml-#1={},
  data-ftml-inline={#2},
  \seq_if_empty:NF \l_stex_fors_seq {,
```

```
    data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn }
  }
  \str_if_empty:NF \l_stex_key_id_str {,
    data-ftml-id={\l_stex_key_id_str}%{\stex_uri_use:N \l_stex_current_doc_uri ? \l_stex_ke
  }
  \clist_if_empty:NF \l_stex_key_style_clist {,
    data-ftml-styles={\l_stex_key_style_clist}
  }
}

\cs_new:Nn \_stex_comma_sep_uri:nn {
  \int_compare:nNnF{#1}=1 {,}
  \stex_use_symbol_uri:n{#2}
}
```

(*End of definition for* `\stex_new_statement:nnn`. *This function is documented on page 157.*)

sfunction

`\__stex_statements_setup:n` sets up a statement that may declare a new symbol with the given `role` by processing the optional arguments, calling `\stex_symdecl_do:`, etc.

```
\cs_new_protected:Nn \__stex_statements_setup:n {
  \str_if_empty:NF \l_stex_key_macroname_str {
    \str_if_empty:NT \l_stex_key_name_str {
      \str_set_eq:NN \l_stex_key_name_str \l_stex_key_macroname_str
    }
  }
  \_stex_do_for_list:
  \tl_clear:N \l_stex_current_def_uri

  \str_if_empty:NTF \l_stex_key_name_str \__stex_statements_setup_noname: {
    \__stex_statements_setup_named:n{#1}
  }
}

\cs_new_protected:Nn \__stex_statements_setup_named:n {
  \__stex_statements_force_id:
  \seq_put_right:Ne \l_stex_fors_seq {
    \l_stex_current_module_uri {\l_stex_key_name_str}
  }
  \str_set_eq:NN \l_stex_macroname_str \l_stex_key_macroname_str
  \str_set:Nn \l_stex_key_role_str {#1}
  \stex_symdecl_do:
  \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnnN}{
    {\l_stex_key_macroname_str}{\l_stex_key_name_str}
    {\int_use:N \l_stex_get_symbol_arity_int}
    {\l_stex_get_symbol_args_tl}
    {#1}{}{}\stex_invoke_symbol:
  }
  \stex_if_do_html:T \_stex_symdecl_html:
  \tl_set:Ne \l_stex_current_def_uri {\exp_args:No \stex_new_symbol_uri:nn \l_stex_current_
  \stex_debug:nn{statement}{name:~\stex_use_symbol_uri:N \l_stex_current_def_uri}
}

\cs_new_protected:Nn \__stex_statements_setup_noname: {
```

```
  \stex_debug:nn{statement}{no~name}
  \int_compare:nNnTF {\seq_count:N \l_stex_fors_seq} = 1 {
    \tl_set:Ne \l_stex_current_def_uri {\seq_item:Nn \l_stex_fors_seq 1}
    \stex_debug:nn{statement}{for:~\stex_use_symbol_uri:N \l_stex_current_def_uri}
  }{
    \stex_debug:nn{statement}{no~for}
  }
}

\cs_new_protected:Nn \__stex_statements_force_id: {
  %\str_if_empty:NT \l_stex_key_id_str {
  %  \_stex_ref_new_id:n{}
  %  \str_set_eq:NN \l_stex_key_id_str \l__stex_refs_str
  %}
}
```

(*End of definition for* `\__stex_statements_setup:n`.)

`\__stex_statements_setup_def:` Sets up all the macros allowed in definition-like environments:

```
\cs_new_protected:Nn \__stex_statements_setup_def: {
  \stex_if_smsmode:F{
    \seq_map_inline:Nn \l_stex_fors_seq {
      \exp_args:Ne \stex_ref_new_sym_target:n{\stex_use_symbol_uri:n{##1}}
    }
  }
  \stex_reactivate_macro:N \definiendum
  \stex_reactivate_macro:N \defnotation
  \stex_reactivate_macro:N \definame
  \stex_reactivate_macro:N \Definame
  \stex_reactivate_macro:N \varbind
  \stex_reactivate_macro:N \definiens
}
```

(*End of definition for* `\__stex_statements_setup_def:`.)

`sdefinition` (*env.*)

```
\stex_new_statement:nnn{definition}{def}{
  \__stex_statements_force_id:
  \__stex_statements_setup:n{}
  \__stex_statements_setup_def:
}
```

`sassertion` (*env.*)

```
\stex_new_statement:nnn{assertion}{ass}{
  \__stex_statements_setup:n{assertion}
  \stex_if_smsmode:F{
    \seq_map_inline:Nn \l_stex_fors_seq {
      \exp_args:Ne \stex_ref_new_sym_target:n{\stex_use_symbol_uri:n{##1}}
    }
  }
  \stex_reactivate_macro:N \varbind
  \stex_reactivate_macro:N \conclusion
  \stex_reactivate_macro:N \premise
  \stex_reactivate_macro:N \definiendum
```

```
  \stex_reactivate_macro:N \defnotation
  \stex_reactivate_macro:N \definame
  \stex_reactivate_macro:N \Definame
}
```

sexample (*env.*)

```
\stex_new_statement:nnn{example}{ex}{
  \stex_if_smsmode:F {\__stex_statements_setup:n{example}}
}
```

sparagraph (*env.*)

```
\stex_new_statement:nnn{paragraph}{}{
  \clist_if_in:NnTF \l_stex_key_style_clist {symdoc}{
    \__stex_statements_force_id:
    \__stex_statements_setup:n{}
    \__stex_statements_setup_def:
  }{
    \__stex_statements_setup:n{}
  }
}
```

definiens

```
\NewDocumentCommand \definiens { O{} m }{
  \group_begin:
  \_stex_definiens_impl:nn{#1}{#2}
  \group_end:
  \stex_smsmode_do:
}

\cs_new_protected:Nn \_stex_definiens_impl:nn {
  \tl_if_empty:nF {#1} {
    \stex_get_symbol:n { #1 }
    \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
  }
  \tl_if_empty:NT \l_stex_current_def_uri {
    \msg_error:nn{stex}{error/definiensfor}
  }
  \stex_debug:nn{definiens}{Checking~\stex_use_symbol_uri:N \l_stex_current_def_uri }

  \exp_args:No \_stex_add_definiens:nn \l_stex_current_def_uri {#2}
}

\stex_deactivate_macro:Nn \definiens {definition~environments}
\stex_sms_allow_escape:N \definiens
```

(*End of definition for* definiens*. This function is documented on page 157.*)

sfunction

_stex_add_definiens:nn

```
\cs_new_protected:Nn \_stex_add_definiens:nn {
  \stex_if_do_html:TF {
    \stex_annotate:nn{data-ftml-definiens={\stex_use_symbol_uri:n{#1}}}{
      #2 %\_stex_annotate_force_break:n{ #2 }
    }
```

```
  }{ \stex_if_smsmode:F{#2} }
  \stex_if_in_module:T {
    \exp_args:Ne \str_if_eq:noT {\stex_symbol_uri_module:n{#1}}\l_stex_current_module_uri{
      \__stex_statements_add_definiens:n{#1}
    }
  }
}

\cs_new_protected:Nn \__stex_statements_add_definiens:n {
  \stex_debug:nn{definiens}{Adding~definiens~to~\stex_use_symbol_uri:n{#1}}
  \exp_args:Nne \prop_gput:cne{\exp_args:Ne \_stex_symbol_macro:n {\stex_symbol_uri_module:
    {\stex_symbol_uri_name:n {#1}} {
      \exp_args:Nne \use:nn { \__stex_statements_definiens_inner:nnnnnnnN }{ \exp_args:Nne
  }
}

\cs_new:Nn \__stex_statements_definiens_inner:nnnnnnnN {
  {#1}{#2}{#3}{#4}{defed}{#6}{#7}{#8}
}
```

(*End of definition for* `\_stex_add_definiens:nn`. *This function is documented on page 157.*)

sfunction

`\varbind`

```
\NewDocumentCommand \varbind {m} {
  \clist_map_inline:nn {#1} {
    \stex_get_var:n {##1}
    \stex_if_do_html:T {
      \stex_annotate_invisible:nn {data-ftml-bind=\l_stex_get_variable_str}{}
    }
  }
}
\stex_deactivate_macro:Nn \varbind {definition~or~assertion~environments}
```

(*End of definition for* `\varbind`. *This function is documented on page 157.*)

sfunction

`\conclusion`

```
\NewDocumentCommand \conclusion { O{} m} {
  \group_begin:
  \tl_if_empty:nF {#1} {
    \stex_get_symbol:n { #1 }
    \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
  }
  \tl_if_empty:NT \l_stex_current_def_uri {
    \msg_error:nn{stex}{error/conclusionfor}
  }
  \stex_annotate:nn{ data-ftml-conclusion={\stex_use_symbol_uri:N \l_stex_current_def_uri}}
    #2 %\_stex_annotate_force_break:n{ #2 }
  }
  \group_end:
}
\stex_deactivate_macro:Nn \conclusion {assertion~environments}
```

(*End of definition for* `\conclusion`*. This function is documented on page 157.*)

sfunction

`\premise`

```
\NewDocumentCommand \premise {O{} m} {
  \tl_if_empty:nF {#1} {
    \stex_debug:nn{Here:}{Variable~#1}
    \exp_args:Nne\use:nn{\vardef}{{v#1}[name=#1]{#1}}
  }
  \stex_annotate:nn{data-ftml-premise={#1}}{#2}
}
\stex_deactivate_macro:Nn \premise {assertion~environments}
```

(*End of definition for* `\premise`*. This function is documented on page 157.*)

sfunction

Chapter 36

Proofs

⟨@@=stex_proof⟩

Keys:

```
\stex_keys_define:nnnn{ spfcommon }{
  \tl_clear:N \l_stex_key_for_clist
  \tl_clear:N \l_stex_key_term_tl
  \tl_clear:N \l_stex_key_method_tl
  \tl_clear:N \l_stex_key_just_tl
}{
  for         .clist_set:N  = \l_stex_key_for_clist ,
  term        .tl_set:N     = \l_stex_key_term_tl,
  method      .tl_set:N     = \l_stex_key_method_tl,
  just        .tl_set:N     = \l_stex_key_just_tl,
}{id,style,title}

\bool_set_false:N \l_stex_key_showname_bool

\stex_keys_define:nnnn{ spfnamed }{
  \str_clear:N \l_stex_key_name_str
  \str_clear:N \l_stex_key_numname_str
  \bool_set_false:N \l_stex_key_showname_bool
}{
  name        .str_set_x:N  = \l_stex_key_name_str,
  numname     .str_set_x:N  = \l_stex_key_numname_str,
  showname    .bool_set:N   = \l_stex_key_showname_bool
}{spfcommon}

\cs_new_protected:Nn \__stex_proof_do_spf_name: {
  \str_if_empty:NTF \l_stex_key_numname_str {
    \str_if_empty:NTF \l_stex_key_name_str {
      \bool_set_false:N \l_stex_key_showname_bool
    } {
      \exp_args:Nne\use:nn{\vardef}{{v\l_stex_key_name_str}[name=\l_stex_key_name_str]{\l_s
    }
  }{
    \bool_set_false:N \l_stex_key_showname_bool
    \__stex_proof_number_as_string:N \l__stex_proof_counter_str
    \exp_args:Nne \use:nn {\vardef}{{\l_stex_key_numname_str}[name=\l__stex_proof_counter_s
```

```
  }
}

\cs_new_protected:Nn \__stex_proof_do_spf_name_i: {
  \str_if_empty:NTF \l__stex_proof_key_numname_str {
    %\str_if_empty:NF \l__stex_proof_key_name_str {
      \exp_args:Nne\use:nn{\vardef}{{v\l__stex_proof_key_name_str}[name=\l__stex_proof_key_
    %}
  }{
    \exp_args:Nne \use:nn {\vardef}{{\l__stex_proof_key_numname_str}[name=\l__stex_proof_co
  }
}

\cs_new_protected:Nn \__stex_proof_do_spf_varname: {
  \bool_if:NT \l_stex_key_showname_bool {
    ~($\csname v\l_stex_key_name_str\endcsname$)~
  }
}

\stex_keys_define:nnnn{ spftop }{
  \tl_set_eq:NN \l_stex_key_proofend_tl \__stex_proof_proof_box_tl
  \bool_set_false:N \l_stex_key_hide_bool
  \tl_clear:N \l_stex_key_continues_tl
  \tl_clear:N \l_stex_key_functions_tl
  \tl_clear:N \l_stex_key_from_tl
}{
  proofend    .tl_set:N     = \l_stex_key_proofend_tl,
  hide        .bool_set:N   = \l_stex_key_hide_bool,
  continues   .tl_set:N     = \l_stex_key_continues_tl, % <- unused
  functions   .tl_set:N     = \l_stex_key_functions_tl, % <- unused
  from        .tl_set:N     = \l_stex_key_from_tl % <- unused
}{spfcommon}

\stex_keys_define:nnnn{ subproof }{}{}{spftop,spfnamed}

\bool_set_true:N \l__stex_proof_inc_counter_bool
```

For proofs, we will have to have deeply nested structures of enumerated list-like environments. However, LaTeX only allows `enumerate` environments up to nesting depth 4 and general list environments up to listing depth 6. This is not enough for us. Therefore we have decided to go along the route proposed by Leslie Lamport to use a single top-level list with dotted sequences of numbers to identify the position in the proof tree. Unfortunately, we could not use his `pf.sty` package directly, since it does not do automatic numbering, and we have to add keyword arguments all over the place, to accomodate semantic information.

```
\intarray_new:Nn\l__stex_proof_counter_intarray{50}

\cs_new_protected:Npn \__stex_proof_insert_number: {
  \int_set:Nn \l_tmpa_int {1}
  \bool_while_do:nn {
    \int_compare_p:nNn {
      \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
    } > 0
  }{
```

```
    \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int .
    \int_incr:N \l_tmpa_int
  }
}
\cs_new_protected:Nn \__stex_proof_number_as_string:N {
  \str_clear:N #1
  \int_set:Nn \l_tmpa_int {1}
  \bool_while_do:nn {
    \int_compare_p:nNn {
      \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
    } > 0
  }{
    \str_put_right:Ne #1 {\intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int .}
    \int_incr:N \l_tmpa_int
  }
}

\cs_new_protected:Npn \__stex_proof_inc_counter: {
  \int_set:Nn \l_tmpa_int {1}
  \bool_while_do:nn {
    \int_compare_p:nNn {
      \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
    } > 0
  }{
    \int_incr:N \l_tmpa_int
  }
  \int_compare:nNnF \l_tmpa_int = 1 {
    \int_decr:N \l_tmpa_int
  }
  \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int {
    \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int + 1
  }
}

\cs_new_protected:Npn \__stex_proof_add_counter: {
  \int_set:Nn \l_tmpa_int {1}
  \bool_while_do:nn {
    \int_compare_p:nNn {
      \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
    } > 0
  }{
    \int_incr:N \l_tmpa_int
  }
  \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int { 1 }
}

\cs_new_protected:Npn \__stex_proof_remove_counter: {
  \int_set:Nn \l_tmpa_int {1}
  \bool_while_do:nn {
    \int_compare_p:nNn {
      \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
    } > 0
  }{
    \int_incr:N \l_tmpa_int
```

```
  }
  \int_decr:N \l_tmpa_int
  \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int { 0 }
}
```

sproof (*env.*)

```
\bool_set_false:N \l__stex_proof_in_spfblock_bool

\cs_new_protected:Nn \__stex_proof_begin_proof:nn {\par
  \intarray_gzero:N \l__stex_proof_counter_intarray
  \intarray_gset:Nnn \l__stex_proof_counter_intarray 1 1
  \stex_keys_set:nn{spftop}{#1}
  \_stex_do_for_list:
  \stex_if_do_html:T {
    \__stex_proof_html_env_proof:
  }
  \seq_map_inline:Nn \l_stex_fors_seq {
    \stex_debug:nn{definiens}{Adding~definiens~to~##1}
    \stex_if_do_html:TF{
      \STEXinvisible{\hbox{\_stex_add_definiens:nn {##1}{proven}}}
    }{
      \_stex_add_definiens:nn {##1}{\STEXinvisible{proven}}
    }
  }
  \stex_if_do_html:F\stex_style_apply:
  %\_stex_do_id:
  \stex_reactivate_macro:N \subproof
  \stex_reactivate_macro:N \spfstep
  \stex_reactivate_macro:N \conclude
  \stex_reactivate_macro:N \assumption
  \stex_reactivate_macro:N \eqstep
  \stex_reactivate_macro:N \yield
  \stex_reactivate_macro:N \spfescape
  \stex_reactivate_macro:N \spfblock
  \stex_reactivate_macro:N \spfjust
  \stex_reactivate_macro:N \spfby
  \stex_reactivate_macro:N \spfarg
  \noindent\stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{#2}}\par
  \stex_if_do_html:TF{
    \use:c{stex_annotate_env}{data-ftml-proofbody={}}
    \_stex_annotate_force_break:n{}
  }{
    \bool_if:NT \l_stex_key_hide_bool{\setbox0\vbox\bgroup}
  }
}

\cs_new_protected:Nn \__stex_proof_html_env_proof: {
  \exp_args:Nne \use:c{stex_annotate_env}{
    data-ftml-proof={}
    \seq_if_empty:NF \l_stex_fors_seq {,
      data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn
    }
    \bool_if:NT \l_stex_key_hide_bool {,
      data-ftml-proofhide=true
```

```
    }
  }
  \__stex_proof_html:
}

\cs_new_protected:Nn \__stex_proof_html_env_subproof: {
  \str_if_empty:NF \l_stex_key_numname_str {
    \__stex_proof_number_as_string:N \l__stex_proof_counter_str
  }
  \exp_args:Nne \use:c{stex_annotate_env}{
    data-ftml-subproof={}
    \seq_if_empty:NF \l_stex_fors_seq {,
      data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn
    }
    \bool_if:NT \l_stex_key_hide_bool {,
      data-ftml-proofhide=true
    }
    \str_if_empty:NTF \l_stex_key_numname_str {
      \str_if_empty:NF \l_stex_key_name_str {,
        data-ftml-stepname={\l_stex_key_name_str}
      }
    }{
      , data-ftml-stepname={\l__stex_proof_counter_str}
    }
  }
  \__stex_proof_html:
}

\cs_new_protected:Nn \__stex_proof_html: {
  \_stex_annotate_force_break:n{}
  \tl_set:Ne\l_tmpa_tl {\l_stex_key_term_tl\l_stex_key_method_tl\l_stex_key_just_tl}
  \tl_if_empty:NF\l_tmpa_tl{
    \stex_annotate_invisible:n{\hbox{
      \tl_if_empty:NF \l_stex_key_term_tl {
        $\stex_annotate:nn{data-ftml-proofterm={}}{\l_stex_key_term_tl}$
      }
      \tl_if_empty:NF \l_stex_key_just_tl {
        $\stex_annotate:nn{data-ftml-spfjust={}}{\l_stex_key_just_tl}$
      }
      \tl_if_empty:NF \l_stex_key_method_tl {
        \stex_annotate:nn{data-ftml-proofmethod={}}{\l_stex_key_method_tl}
      }
    }}
  }
}

\cs_new_protected:Nn \__stex_proof_start_comment: {
  \bool_if:NT \l__stex_proof_in_spfblock_bool {
    \par
    \group_begin:\stexcommentfont
  }
}
\cs_new_protected:Nn \__stex_proof_end_comment: {
  \bool_if:NT \l__stex_proof_in_spfblock_bool {
```

```
    \par
    \group_end:
  }
}
\cs_new_protected:Npn \spfescape #1{
  \__stex_proof_end_comment: #1 \__stex_proof_start_comment:
}
\stex_deactivate_macro:Nn \spfescape {sproof~environments}

\stex_new_stylable_env:nnnnnnn{proof}{O{} m}{
  \__stex_proof_begin_proof:nn{#1}{#2}
  \bool_set_true:N\l__stex_proof_in_spfblock_bool
  %\__stex_proof_start_list:n{}
  \__stex_proof_start_comment:
}{
  \stex_if_do_html:TF{
    %\__stex_proof_end_list:
    \use:c{endstex_annotate_env}\use:c{endstex_annotate_env}
  }\stex_style_apply:
}{
  \emph{\sproofautorefname :}~
}{
  \sproofend
}{s}

\AddToHook{env/sproof/end}{
  \__stex_proof_end_comment:
  \stex_if_do_html:F{\bool_if:NT \l_stex_key_hide_bool\egroup}
}


\stex_new_stylable_env:nnnnnnn{proof*}{O{}}{
  \__stex_proof_begin_proof:nn{#1}{}
  \bool_set_false:N\l__stex_proof_in_spfblock_bool
}{
  \stex_if_do_html:TF{\use:c{endstex_annotate_env}\use:c{endstex_annotate_env}}\stex_style_
}{
  \emph{Proof:}~
}{
  \sproofend
}{s}

\cs_new_protected:Nn \__stex_proof_start_list:n {
  %\par
  \group_begin:\list{}{
    \setlength\topsep{0pt}
    \setlength\parsep{0pt}
    \setlength\rightmargin{0pt}
  }\item[#1]
}

\cs_new_protected:Nn \__stex_proof_end_list: {
  %\par
  \endlist\group_end:
```

```
}
```

subproof (*env.*)

```
\str_set_eq:NN \subproofautorefname \spfstepautorefname

\cs_new_protected:Nn \__stex_proof_do_after_subgroup:N {
  \bgroup
    \expandafter \__stex_proof_do_after_subgroup_i:nn #1
  \egroup
  \__stex_proof_do_spf_name_i:
}
\cs_new_protected:Nn \__stex_proof_do_after_subgroup_i:nn {
    \exp_args:Nno \stex_keys_set:nn{subproof}{#1}
    \str_gset_eq:NN \l__stex_proof_key_name_str \l_stex_key_name_str
    \str_gset_eq:NN \l__stex_proof_key_numname_str \l_stex_key_numname_str
    \bool_gset_eq:NN \l__stex_proof_key_showname_bool \l_stex_key_showname_bool
    \str_gset:Nn \l__stex_proof_counter_str {#2}
}
\cs_new_protected:Npn \__stex_proof_set_after_subgroup:n {
  \str_if_empty:NTF \l_stex_key_numname_str {
    \str_if_empty:NTF \l_stex_key_name_str \use_none:n \__stex_proof_set_after_subgroup_i:n
  }\__stex_proof_set_after_subgroup_i:n
}
\cs_new_protected:Npn \__stex_proof_set_after_subgroup_with_number:n {
  \str_if_empty:NTF \l_stex_key_numname_str {
    \str_if_empty:NTF \l_stex_key_name_str \use_none:n \__stex_proof_set_after_subgroup_wit
  }\__stex_proof_set_after_subgroup_with_number_i:n
}
\cs_new_protected:Nn \__stex_proof_set_after_subgroup_i:n {
  \tl_gset:cn{subproof_arg_ \int_use:N \currentgrouplevel}{{#1}{}}

  \__stex_proof_set_aftergroup: % \bool_if:NTF \l__stex_proof_in_spfblock_bool \__stex_proo
}
\cs_new_protected:Nn \__stex_proof_set_after_subgroup_with_number_i:n {
  \__stex_proof_number_as_string:N \l__stex_proof_counter_str
  \tl_gset:ce{subproof_arg_ \int_use:N \currentgrouplevel}{{\exp_not:n{#1}}{\l__stex_proof_
  \__stex_proof_set_aftergroup:
}
\cs_new_protected:Nn \__stex_proof_set_aftergroup: {
    \aftergroup \__stex_proof_do_after_subgroup:N
    \expandafter \aftergroup \csname subproof_arg_ \int_use:N \currentgrouplevel\endcsname
}

\stex_new_stylable_env:nnnnnnn{subproof}{s O{} m}{
  \stex_keys_set:nn{subproof}{#2}
  \_stex_do_for_list:
  \stex_if_do_html:T {
    \__stex_proof_html_env_subproof:
  }
  \seq_map_inline:Nn \l_stex_fors_seq {
    \stex_debug:nn{definiens}{Adding~definiens~to~##1}
    \stex_if_do_html:TF{
      \STEXinvisible{\hbox{\_stex_add_definiens:nn {##1}{proven}}}
    }{
```

```
      \_stex_add_definiens:nn {##1}{\STEXinvisible{proven}}
    }
  }

  \IfBooleanTF #1 {
    \bool_set_false:N \l__stex_proof_in_spfblock_bool
    \__stex_proof_set_after_subgroup:n{#2}
    \stex_if_do_html:F \stex_style_apply:
    \str_if_empty:NF \l_stex_key_id_str {
      \__stex_proof_number_as_string:N \@currentlabel
      %\str_set:Ne \@currentHref{subproof.\@currentlabel}
      %\_stex_do_id:
    }


    \stex_annotate:nn{data-ftml-title={}}{#3}
  }{
    \bool_if:NTF \l__stex_proof_in_spfblock_bool {

      \__stex_proof_set_after_subgroup_with_number:n{#2}

      %\str_if_empty:NF \l_stex_key_id_str {
        %\__stex_proof_number_as_string:N \@currentlabel
        %\str_set:Ne \@currentHref{subproof.\@currentlabel}
        %\_stex_do_id:
      %}
      \use:c{stex_annotate_env}{data-ftml-title={}} \__stex_proof_start_list:n{\__stex_proo
        \__stex_proof_add_counter:
        \stex_if_do_html:F \stex_style_apply:
    }{
      \noindent \stex_annotate:nn{data-ftml-title={}}{#3} \par
      \stex_if_do_html:F \stex_style_apply:

      \__stex_proof_set_after_subgroup:n{#2}

      %\_stex_do_id:
    }
  }
  \stex_if_do_html:TF{
    \use:c{stex_annotate_env}{data-ftml-proofbody={}}
    \_stex_annotate_force_break:n{}
  }{\bool_if:NT \l_stex_key_hide_bool{\setbox0\vbox\bgroup}}
  \__stex_proof_start_comment:
}{
  \bool_if:NT\l__stex_proof_in_spfblock_bool {
    \__stex_proof_remove_counter:
    \__stex_proof_inc_counter:
  }

  \stex_if_do_html:TF{
    \use:c{endstex_annotate_env} %proofbody
    \bool_if:NT\l__stex_proof_in_spfblock_bool \__stex_proof_end_list:
    \use:c{endstex_annotate_env} % subproof
  }{
```

```
    \stex_style_apply:
    \bool_if:NT \l__stex_proof_in_spfblock_bool {\__stex_proof_inc_counter:\__stex_proof_en
  }
}{}{}{}

\AddToHook{env/subproof/before}{
  \__stex_proof_end_comment:
}

\AddToHook{env/subproof/after}{
  \__stex_proof_start_comment:
}

\AddToHook{env/subproof/end}{
  \__stex_proof_end_comment:
  \stex_if_do_html:F{\bool_if:NT \l_stex_key_hide_bool\egroup}
}

\stex_deactivate_macro:Nn \subproof {sproof~environments}

```

`spfsketchenv` (*env.*) TODO:

```
\newenvironment{spfsketchenv}{}{}
```

`\spfsketch`

```
\stex_new_stylable_cmd:nnnn{spfsketch}{O{} m}{\par
  \begin{spfsketchenv}
  \stex_keys_set:nn{spftop}{#1}
  \_stex_do_for_list:
  \_stex_do_id:
  \par
  \exp_args:Nne \use:c{stex_annotate_env}{
    data-ftml-proofsketch={
      \seq_if_empty:NF \l_stex_fors_seq {
        \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
      }
    }
  }
    \stex_style_apply:
    #2\par
    \use:c{endstex_annotate_env}
  \end{spfsketchenv}
}{
  \noindent\emph{\spfsketchenvautorefname :}~
}
```

(*End of definition for* `\spfsketch`*. This function is documented on page 158.*)

sfunction

Keys for proof step macros:

```
\stex_keys_define:nnnn { spfsteps } {
  %\str_clear:N \l_stex_key_name_str
}{
  %name        .str_set_x:N  = \l_stex_key_name_str
}{spfcommon,spfnamed}
```

Common setup for all the proof step macros:

```
\cs_new_protected:Nn \__stex_proof_make_step_macro:Nnnnn {
  \NewDocumentCommand #1 {s O{} +m} {
    \__stex_proof_end_comment:
    \stex_keys_set:nn{spfsteps}{##2}
    %\str_if_empty:NF \l_stex_key_name_str {
    %  \exp_args:Nne\use:nn{\vardef}{{v\l_stex_key_name_str}[name=\l_stex_key_name_str]{\l_
    %}
    \__stex_proof_do_spf_name:

    \spfstepenv
    \str_if_empty:NF \l_stex_key_id_str {
      %\__stex_proof_number_as_string:N \@currentlabel
      %\str_set:Ne \@currentHref{spfstep.\@currentlabel}
      %\_stex_do_id:
    }

    \bool_if:NTF \l__stex_proof_in_spfblock_bool {
      \IfBooleanTF ##1 {
        \__stex_proof_step_html_i:nn{#2}{##3}
      }{
        \__stex_proof_step_html:nn{#2}{\__stex_proof_start_list:n{#3} ##3 \__stex_proof_end
        #5
      }
      \endspfstepenv
      \__stex_proof_start_comment:
    }{
      \__stex_proof_step_html_i:nn{#2}{##3}
      \endspfstepenv
    }
  }
  \stex_deactivate_macro:Nn #1 {sproof~environments}
}

\cs_new_protected:Nn \__stex_proof_step_html:nn {
  \str_if_empty:NF \l_stex_key_numname_str {
    \__stex_proof_number_as_string:N \l__stex_proof_counter_str
  }
  \stex_if_do_html:TF{
    %\group_begin:
    \exp_args:Nne \use:c{stex_annotate_env}{
      data-ftml-spf#1={
        \seq_if_empty:NF \l_stex_fors_seq {
          \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
        }
      }
      \str_if_empty:NTF \l_stex_key_numname_str {
        \str_if_empty:NF \l_stex_key_name_str {,
          data-ftml-stepname={\l_stex_key_name_str}
        }
      }{
        , data-ftml-stepname={\l__stex_proof_counter_str}
      }
    }
```

```
    \__stex_proof_html:
    #2
    \par
    \use:c{endstex_annotate_env}
    %\group_end:
  }{ #2 }
}
\cs_new_protected:Nn \__stex_proof_step_html_i:nn {
  \stex_if_do_html:TF{
    \noindent
    %\group_begin:
    \exp_args:Ne \stex_annotate:nn{
      data-ftml-spf#1={
        \seq_if_empty:NF \l_stex_fors_seq {
          \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
        }
      }
      \str_if_empty:NTF \l_stex_key_numname_str {
        \str_if_empty:NF \l_stex_key_name_str {,
          data-ftml-stepname={\l_stex_key_name_str}
        }
      }{
        , data-ftml-stepname={\l__stex_proof_counter_str}
      }
    }{
      \__stex_proof_html:
      #2
    }
    %\group_end:
  }{ #2 }
}
```

`spfstepenv` (*env.*) TODO:

```
\newenvironment{spfstepenv}{
  \str_set_eq:NN \spfstepenvautorefname \spfstepautorefname
}{}
```

`spfblock` (*env.*)

```
\NewDocumentEnvironment{spfblock}{}{
  \bool_set_false:N \l__stex_proof_in_spfblock_bool
}{
  \aftergroup\__stex_proof_start_comment:
}

\stex_deactivate_macro:Nn \spfblock {sproof~environments}
\AddToHook{env/spfblock/before}{
  \__stex_proof_end_comment:
}
```

`\spfstep`

```
\__stex_proof_make_step_macro:Nnnnn \spfstep {step} {\__stex_proof_insert_number:\__stex_pr
```

(*End of definition for* `\spfstep`*. This function is documented on page 158.*)

sfunction

\assumption

```
\__stex_proof_make_step_macro:Nnnnn \assumption {assumption} {\__stex_proof_insert_number:\
```

(*End of definition for* \assumption*. This function is documented on page 158.*)

sfunction

\conclude

```
\__stex_proof_make_step_macro:Nnnnn \conclude {conclusion} {$\Rightarrow$} {} {}
```

(*End of definition for* \conclude*. This function is documented on page 158.*)

sfunction

\eqstep

```
\NewDocumentCommand \eqstep {s m}{
  \__stex_proof_end_comment:
  \spfstepenv

  \bool_if:NTF \l__stex_proof_in_spfblock_bool {
    \IfBooleanTF #1 {
      \__stex_proof_step_html_i:nn{eqstep}{$= #2$}
    }{
      \__stex_proof_step_html:nn{eqstep}{\__stex_proof_start_list:n{$=$} $#2$ \__stex_proof
    }
    \endspfstepenv
    \__stex_proof_start_comment:
  }{
    \__stex_proof_step_html_i:nn{eqstep}{$= #2$}
    \endspfstepenv
  }
}
\stex_deactivate_macro:Nn \eqstep {sproof~environments}
```

(*End of definition for* \eqstep*. This function is documented on page 158.*)

sfunction

\yield

```
\NewDocumentCommand \yield {+m}{
  \stex_annotate:nn{data-ftml-proofterm={}}{ #1 }
}
\stex_deactivate_macro:Nn \yield {sproof~environments}
```

(*End of definition for* \yield*. This function is documented on page 158.*)

sfunction

\spfjust

```
\newcommand\spfjust[1]{
  \stex_annotate:nn{data-ftml-spfjust={}}{ #1 }
}
\stex_deactivate_macro:Nn \spfjust {sproof~environments}
```

(*End of definition for* \spfjust*. This function is documented on page 158.*)

sfunction

\spfby

```
\newcommand\spfby[1]{
  \stex_annotate:nn{data-ftml-proofmethod={}}{ #1 }
}
\stex_deactivate_macro:Nn \spfby {sproof~environments}
```

(*End of definition for* \spfby*. This function is documented on page 158.*)

sfunction

\spfarg

```
\newcommand\spfarg[2][]{
  \stex_annotate:nn{data-ftml-spfarg={#1}}{ #2 }
}
\stex_deactivate_macro:Nn \spfarg {sproof~environments}
```

(*End of definition for* \spfarg*. This function is documented on page 159.*)

sfunction

\sproofend

```
\tl_set:Nn \__stex_proof_proof_box_tl {
  \ltx@ifpackageloaded{amssymb}{$\square$}{
    \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
  }
}

\tl_set:Nn \sproofend {
  \tl_if_empty:NF \l_stex_key_proofend_tl {
    \hfil\null\nobreak\hfill\l_stex_key_proofend_tl\par\smallskip
  }
}
```

(*End of definition for* \sproofend*. This function is documented on page 159.*)

sfunction

\stexcommentfont

```
\cs_new_protected:Npn \stexcommentfont {
  \small\itshape
}
```

(*End of definition for* \stexcommentfont*. This function is documented on page 159.*)

sfunction

Chapter 37

Metatheory

```
⟨@@=stex_meta⟩
```

Setup:

```
\group_begin:
\cs_set:Npn \__stex_modules_persist_module: {}
\cs_set:Npn \stex_check_term:nn #1 #2 {}
\cs_set:Npn \_stex_sref_do_aux:n #1 { #1 }
\bool_set_false:N \c_stex_check_terms_bool
\bool_set_false:N \l__stex_annotate_do_output_bool
\tl_if_exist:cF {c_stex_mathhub_FTML/meta_archive}{
  \tl_gset:ce {c_stex_mathhub_FTML/meta_archive}{
    {\tl_to_str:n{FTML/meta}}
    {\tl_to_str:n{http://mathhub.info}}
    {}
  }
}
\tl_set:Ne \l_stex_current_document_uri {
  {\tl_to_str:n{FTML/meta}}{}
  {\tl_to_str:n{Metatheory}}{\tl_to_str:n{en}}{}
}
\_stex_module_setup_top_nosig:n{Metatheory}
\g_stex_every_module_tl
\symdef{of~type}[args=ii,invisible]{#1}
\notation{of~type}[colon]{#1 \mathbin{\comp{:}} #2}

\symdef{apply}[args=ia,prec=0;\infprec x\infprec]{#1\mathopen{\comp(} #2 \mathclose{\comp)}}
\notation{apply}[lambda]{#1\; \argsep{#2}{\;}}
\notation{apply}[infixop]{\argsep{#2}{\mathbin{#1}}\STEXinvisible{#1}}
\notation{apply}[infixrel]{\argsep{#2}{\mathrel{#1}}\STEXinvisible{#1}}

\symdef{autoprove}[name=prove automatically]{\mathrm{automatically\ proved}}

% structures
\symdef{module~type}[args=i,op=\mathtt{MOD}]
  {\mathopen{\comp{\mathtt{MOD}(}}#1\mathclose{\comp{)}}}}
\symdef{module~type~merge}[args=a,op=\oplus]
  {\argsep{#1}{\mathbin{\comp{\oplus}}}}
\symdef{anonymous~record}[args=a]
```

```
  {\mathopen{\comp{[[}}#1\mathclose{\comp{]]}}}}
\symdef{record~field}[args=2]{#1\comp{.}#2}
\symdecl*{record~type}

\symdecl{mathstruct}[name=mathematical~structure,args=a] % TODO
\notation{mathstruct}[angle,prec=nobrackets]
  {\mathopen{\comp\langle} #1 \mathclose{\comp\rangle}}
\notation{mathstruct}[parens,prec=nobrackets]
  {\mathopen{\comp(} #1 \mathclose{\comp)}}

% sequences
\symdef{ellipses}[ldots]{\ldots}
\symdef{sequence~expression}[comma,args=a]{#1}
\symdef{sequence~type}[args=1]{#1^{\comp\ast}}
\symdef{sequence~map}[args=ai]{
  \comp{\mathrm{map}}\mathopen{\comp{(}}#1\mathpunct{\comp{,}}
  #2\mathclose{\comp{)}}
}

\symdecl{bind}[args=Bi,assoc=pre]
\notation{bind}[depfun,prec=nobrackets,op=(\cdot)\;\to\;\cdot]
  {\mathopen{\comp(} #1 \mathclose{\comp{)}}\mathbin{\comp{\to}}} #2}
\notation{bind}[forall]{\comp\forall #1.\;#2}
\notation{bind}[Pi]{\mathop{\comp\prod}\c_math_subscript_token{#1}#2}

\symdef{implicit~bind}[args=Bi,assoc=pre]{\mathopen{\comp\{} #1 \mathclose{\comp{\}}\c_math_

\symdecl{arrow}[args=ai,assoc=pre]
\notation{arrow}[single]{
  \argsep{#1}{\mathbin{\comp\times}}
  \mathrel{\maincomp\to} #2
}
\notation{arrow}[double]{
  \argsep{#1}{\mathbin{\comp\times}}
  \mathrel{\maincomp\Rightarrow} #2
}

\symdef{intsecttp}[intersection type,args=a,sqcap,prec=0;-10]{ \argsep{#1}{\maincomp{\mathr

\symdef{bindin}[args=Bi]{ \mathop{\maincomp{\amalg}}_{#1}#2 }

\symdecl*{integer~literal}
\notation{integer~literal}{\mathbb Z}

\symdecl*{ordinal}
\notation{ordinal}{\mathtt{Ord}}

% propositions
\symdef{prop}[name=proposition]{\mathtt{Prop}}
\symdef{judgment~holds}[args=i,role=judgment]{\comp\vdash\;#1}

% any object
\symdef{object}{\mathtt{Obj}}

```

```
\symdef{letin}[name=let in,args=Bi,role=let]{\maincomp{\mathrm{let}}\ #1\ \comp{\mathrm{in}}

% TODO DELETE
\symdef{aseqdots}[args=a,prec=nobrackets]
  {#1\comp{,\ldots}}%{##1\comp,##2}
\symdef{aseqfromto}[args=ai,prec=nobrackets]
  {#1\comp{,\ldots,}#2}%{##1\comp,##2}
\symdef{aseqfromtovia}[args=aii,prec=nobrackets]
  {#1\comp{,\ldots,}#2\comp{,\ldots,}#3}%{##1\comp,##2}
```

Closing:

```
\global \let \l_stex_metatheory_uri \l_stex_current_module_uri
\global \let \c_stex_default_metatheory \l_stex_metatheory_uri
\stex_close_module:
\group_end:
```

Chapter 38

Others

```
⟨@@=stex_others⟩

\cs_new_protected:Npn \MSC #1 {}

\_stex_persist_read_now:
\cs_new_protected:Nn \__stex_others_newlabel:n {
  \tl_gset:cn{__stex_sref_aux_sym_ \tl_to_str:n{#1}}{X}
  \exp_args:Ne\__stex_others_old_newlabel:{\tl_to_str:n{#1}}
}
\AtBeginDocument{
  \iow_now:Nn \@auxout {
    \ExplSyntaxOn
    \let\__stex_others_old_newlabel:\newlabel
    \let\newlabel\__stex_others_newlabel:n
    \ExplSyntaxOff
  }
}
```

Inference Rules

```
\NewDocumentCommand \FTMLrule {m m}{
  \tl_if_empty:nTF{#2}{
    \seq_clear:N \l_tmpa_seq
  }{
    \seq_set_split:Nnn \l_tmpa_seq , {#2}
  }
  \int_zero:N \l_tmpa_int
  \stex_annotate_invisible:n{\hbox{
    $
    \stex_annotate:nn{data-ftml-inferencerule={#1}}{
      \_stex_annotate_force_break:n{~
        \seq_if_empty:NF \l_tmpa_seq {
          \seq_map_inline:Nn \l_tmpa_seq {
            \int_incr:N \l_tmpa_int
            \stex_annotate:nn{
              data-ftml-argmode=i,
              data-ftml-arg={\int_use:N \l_tmpa_int}
            }{ ##1 }
          }
        }
```

```
        ~}
      }$
   }}
}
\stex_deactivate_macro:Nn \FTMLrule {module~environments}
\stex_every_module:n{\stex_reactivate_macro:N \FTMLrule}
```

Chapter 39

Additional Packages

39.1 Implementation: The notesslides Package

39.1.1 Class and Package Options

We define some Package Options and switches for the `notesslides` class and activate them by passing them on to `beamer.cls` and `omdoc.cls` and the `notesslides` package. We pass the `nontheorem` option to the `statements` package when we are not in notes mode, since the `beamer` package has its own (overlay-aware) theorem environments.

```
⟨*cls⟩
⟨@@=notesslides⟩
\ProvidesExplClass{notesslides}{2025/11/11}{4.0.0}{notesslides Class}
\RequirePackage{l3keys2e}

\str_const:Nn \c__notesslides_class_str {article}

\keys_define:nn{notesslides / cls}{
  class   .str_set_x:N = \c__notesslides_class_str,
  notes   .bool_set:N  = \c_notesslides_notes_bool ,
  slides  .code:n      = { \bool_set_false:N \c_notesslides_notes_bool },
  %docopt  .str_set_x:N = \c__notesslides_docopt_str,
  unknown .code:n      = {
    \PassOptionsToClass{\CurrentOption}{beamer}
    \PassOptionsToClass{\CurrentOption}{\c__notesslides_class_str}
    \PassOptionsToPackage{\CurrentOption}{notesslides}
    \PassOptionsToPackage{\CurrentOption}{stex}
  }
}
\ProcessKeysOptions{ notesslides / cls }

\RequirePackage{stex}
\stex_if_html_backend:T {
  \bool_set_true:N\c_notesslides_notes_bool
}

\bool_if:NTF \c_notesslides_notes_bool {
  \PassOptionsToPackage{notes=true}{notesslides}
  \message{notesslides.cls:~Formatting~document~in~notes~mode}
```

```
}{
  \PassOptionsToPackage{notes=false}{notesslides}
  \message{notesslides.cls:~Formatting~document~in~slides~mode}
}

\bool_if:NTF \c_notesslides_notes_bool {
  \LoadClass{\c__notesslides_class_str}
}{
  \LoadClass[10pt,notheorems,xcolor={dvipsnames,svgnames}]{beamer}
  %\newcounter{Item}
  %\newcounter{paragraph}
  %\newcounter{subparagraph}
  %\newcounter{Hfootnote}
}
\RequirePackage{notesslides}
⟨/cls⟩
```

now we do the same for the `notesslides` package.

```
⟨*package⟩
\ProvidesExplPackage{notesslides}{2025/11/11}{4.0.0}{notesslides Package}
\RequirePackage{l3keys2e}

\keys_define:nn{notesslides / pkg}{
  notes           .bool_set:N   = \c_notesslides_notes_bool ,
  slides          .code:n       = { \bool_set_false:N \c_notesslides_notes_bool },
  sectocframes    .bool_set:N   = \c_notesslides_sectocframes_bool ,
  topsect         .str_set_x:N  = \c_notesslides_topsect_str,
  unknown         .code:n       = {
    \PassOptionsToPackage{\CurrentOption}{stex}
    \PassOptionsToPackage{\CurrentOption}{tikzinput}
  }
}
\ProcessKeysOptions{ notesslides / pkg }

\RequirePackage{stex}
\stex_if_html_backend:T {
  \bool_set_true:N\c_notesslides_notes_bool
}

\cs_set:Npn \sectiontitleemph #1 {
  \textbf{\Large #1}
}

\newif\ifnotes
\bool_if:NTF \c_notesslides_notes_bool {
  \notestrue
  \PassOptionsToPackage{dvipsnames,svgnames}{xcolor}
  \RequirePackage[noamsthm,hyperref]{beamerarticle}
  \RequirePackage{mdframed}
  \str_if_empty:NTF \c_notesslides_topsect_str{
    %\setsectionlevel{section}
  } {
    \exp_args:No \setsectionlevel \c_notesslides_topsect_str
  }
```

```
}{
  \notesfalse

  \cs_new_protected:Nn \__notesslides_do_sectocframes: {
    \cs_set_protected:Nn \__notesslides_do_label:n {
      \str_case:nnF{##1}{
        {part}    {
          \tl_set:Nx\l__notesslides_num{\thepart}
          \tl_set:cx{@ @ label}{
            \cs_if_exist:NTF\parttitlename{\exp_not:N\parttitlename}{\exp_not:N\partname}{}
        }
        {chapter} {
          \tl_set:Nx\l__notesslides_num{\thechapter}
          \tl_set:cx{@ @ label}{
            \cs_if_exist:NTF\chaptertitlename{\exp_not:N\chaptertitlename}{\exp_not:N\chapt
        }
        {section} {
          \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection
          \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
        }
        {subsection} {
          \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection
          \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
        }
        {subsubsection} {
          \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection
          \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
        }
        {paragraph} {
          \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection
          \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
        }
      }{
        \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection.\
        \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
      }
    }
    \cs_set_protected:Nn \_sfragment_do_level:nn {
      \tl_if_exist:cT{c@##1}{\stepcounter{##1}}
      \addcontentsline{toc}{##1}{\protect\numberline{\use:c{the##1}}##2}
      \__notesslides_do_label:n{##1}
      \pdfbookmark[\int_use:N \l_stex_current_section_level_int]{\l__notesslides_num\ ##2}{
      \begin{frame}[noframenumbering]
        \vfill\centering
        \sectiontitleemph{
          \use:c{@ @ label} ##2
        }
      \end{frame}
      \int_incr:N \l_stex_current_section_level_int
      \str_set:Nn \l_stex_current_section_level_str{##1}
    }
  }

  \cs_set_protected:Nn \_stex_titlefragment: {
```

```
    \begin{frame}[noframenumbering]\maketitle\end{frame}
    \let\maketitle\relax
  }

  \AtBeginDocument{
    \str_if_empty:NTF \c_notesslides_topsect_str {
      \setsectionlevel{section}
    } {
      \exp_args:No \setsectionlevel \c_notesslides_topsect_str
      \exp_args:No \str_if_eq:nnTF \c_notesslides_topsect_str {chapter} {
        \__notesslides_define_chapter:
      }{
        \exp_args:No \str_if_eq:nnT \c_notesslides_topsect_str {part} {
          \__notesslides_define_chapter:
          \__notesslides_define_part:
        }
      }
    }
  }

  \bool_if:NT \c_notesslides_sectocframes_bool {
    \__notesslides_do_sectocframes:
  }
}

\cs_new_protected:Nn \__notesslides_define_chapter: {
  \cs_if_exist:NF \chaptername {
    \cs_set_protected:Npn \chaptername {Chapter}
  }
  \cs_if_exist:NF \chapter {
    \cs_set_protected:Npn \chapter {INVALID}
  }
  \cs_if_exist:NF \c@chapter {
    \newcounter{chapter}\counterwithin*{section}{chapter}
  }
}

\cs_new_protected:Nn \__notesslides_define_part: {
  \cs_if_exist:NF \partname {
    \cs_set_protected:Npn \partname {Part}
  }
  \cs_if_exist:NF \part {
    \cs_set_protected:Npn \part {INVALID}
  }
  \cs_if_exist:NF \c@part {
    \newcounter{part}\counterwithin*{chapter}{part}
  }
}
```

`\prematurestop` We initialize `\afterprematurestop`, and provide `\prematurestop@endsfragment` which looks up `\sfragment@level` and recursively ends enough `{sfragment}`s.

```
\def \c__notesslides_document_str{document}
\newcommand\afterprematurestop{}
\def\prematurestop@endsfragment{
```

```
  \unless\ifx\@currenvir\c__notesslides_document_str
    \expandafter\expandafter\expandafter\end\expandafter\expandafter\expandafter{\expandaft
    \expandafter\prematurestop@endsfragment
  \fi
}
\providecommand\prematurestop{
  \stex_if_html_backend:F{
  \message{Stopping~sTeX~processing~prematurely}
  \prematurestop@endsfragment
  \afterprematurestop
  \end{document}
  }
}
```

(*End of definition for* `\prematurestop`. *This function is documented on page 111.*)

39.1.2 Notes and Slides

For the notes case, we also provide the `\usetheme` macro that would otherwise come from the the `beamer` class.

```
\bool_if:NT \c_notesslides_notes_bool {
  \renewcommand\usetheme[2][]{\usepackage[#1]{beamertheme#2}}
}
\NewDocumentCommand \libusetheme {O{} m} {
  \libusepackage[#1]{beamertheme#2}
}
```

We define the sizes of slides in the notes. Somehow, we cannot get by with the same here.

```
\newlength{\slidewidth}\setlength{\slidewidth}{13.5cm}
\newlength{\slideheight}\setlength{\slideheight}{9cm}
```

We first set up the slide boxes in `notes` mode. We set up sizes and provide a box register for the frames and a counter for the slides.

```
\ifnotes

\newlength{\slideframewidth}
\setlength{\slideframewidth}{1.5pt}
```

`frame` (*env.*) We first define the keys.

```
\cs_new_protected:Nn \__notesslides_do_yes_param:Nn {
  \exp_args:Nx \str_if_eq:nnTF { \str_uppercase:n{ #2 } }{ yes }{
    \bool_set_true:N #1
  }{
    \bool_set_false:N #1
  }
}

\stex_keys_define:nnnn{notesslides / frame}{
  \str_clear:N \l__notesslides_frame_label_str
  \bool_set_true:N \l__notesslides_frame_allowframebreaks_bool
  \bool_set_true:N \l__notesslides_frame_allowdisplaybreaks_bool
  \bool_set_true:N \l__notesslides_frame_fragile_bool
  \bool_set_true:N \l__notesslides_frame_shrink_bool
```

```
  \bool_set_true:N \l__notesslides_frame_squeeze_bool
  \bool_set_true:N \l__notesslides_frame_t_bool
}{
  label               .str_set_x:N  = \l__notesslides_frame_label_str,
  allowframebreaks    .code:n       = {
    \__notesslides_do_yes_param:Nn \l__notesslides_frame_allowframebreaks_bool { #1 }
  },
  allowdisplaybreaks  .code:n       = {
    \__notesslides_do_yes_param:Nn \l__notesslides_frame_allowdisplaybreaks_bool { #1 }
  },
  fragile             .code:n       = {
    \__notesslides_do_yes_param:Nn \l__notesslides_frame_fragile_bool { #1 }
  },
  shrink              .code:n       = {
    \__notesslides_do_yes_param:Nn \l__notesslides_frame_shrink_bool { #1 }
  },
  squeeze             .code:n       = {
    \__notesslides_do_yes_param:Nn \l__notesslides_frame_squeeze_bool { #1 }
  },
  t                   .code:n       = {
    \__notesslides_do_yes_param:Nn \l__notesslides_frame_t_bool { #1 }
  },
  unknown   .code:n       = {}
}{}
```

We redefine the `itemize` environment so that it looks more like the one in `beamer`.

```
\cs_new_protected:Nn \__notesslides_setup_itemize: {
  \def\itemize@level{outer}
  \def\itemize@outer{outer}
  \def\itemize@inner{inner}
  %\newcommand\metakeys@show@keys[2]{\marginnote{{\scriptsize ##2}}}
  \renewenvironment{itemize}{
    \ifx\itemize@level\itemize@outer
      \def\itemize@label{$\rhd$}
    \fi
    \ifx\itemize@level\itemize@inner
      \def\itemize@label{$\scriptstyle\rhd$}
    \fi
    \begin{list}
    {\itemize@label}
    {\setlength{\labelsep}{.3em}
      \setlength{\labelwidth}{.5em}
      \setlength{\leftmargin}{1.5em}
    }
    \edef\itemize@level{\itemize@inner}
  }{
    \end{list}
  }
}
```

We create the box with the `mdframed` environment from the equinymous package.

```
\stex_if_html_backend:TF {
  %\stex_css_literal:n{
  %  .stex-frame {
```

```
  %    background-color:#fff;
  %    margin:5px ~ 0;
  %    border:2px ~ solid ~ #000;
  %    border-radius:5px;
  %    padding:5px;
  %    padding-right:10px;
  %    width: var(--rustex-curr-width);
  %    display:block;
  %  }
  %}

  \cs_new_protected:Nn \__notesslides_frame_box_begin: {
    \vbox\bgroup
    \begin{stex_annotate_env}{data-ftml-slide={}}%{class:stex-frame={}}
      \mdf@patchamsthm\notesslidesfont
  }
  \cs_new_protected:Nn \__notesslides_frame_box_end: {
    %^^A \notesslides@slidelabel
    \medskip\par\hbox to \textwidth{\tiny\notesslidesfooter}
    \end{stex_annotate_env}\egroup
  }
}{
  \cs_new_protected:Nn \__notesslides_frame_box_begin: {
    \begin{mdframed}[
      linewidth=\slideframewidth,
      skipabove=1ex,
      skipbelow=1ex,
      userdefinedwidth=\slidewidth,
      align=center,
      %usetwoside=false
    ]\notesslidesfont
  }
  \cs_new_protected:Nn \__notesslides_frame_box_end: {
    \medskip\par\noindent\tiny\notesslidesfooter%^^A\notesslides@slidelabel
    \end{mdframed}
  }
}
```

We define the environment, read them, and construct the slide number and label.

```
\renewenvironment{frame}[1][]{
  \stex_keys_set:nn{notesslides / frame}{#1}
  \stepcounter{framenumber}
  \renewcommand\newpage{\addtocounter{framenumber}{1}}
  \def\@currentlabel{\theframenumber}
  \str_if_empty:NF \l__notesslides_frame_label_str {
    \label{\l__notesslides_frame_label_str}
  }
  \__notesslides_setup_itemize:
  \__notesslides_frame_box_begin:
}{
  \__notesslides_frame_box_end:
}
```

Now, we need to redefine the frametitle (we are still in course notes mode).

\frametitle

```
\renewcommand{\frametitle}[1]{
  \stexdoctitle { #1 }
  \stex_if_html_backend:TF{
    \notesslidestitleemph{\noindent\stex_annotate:nn{data-ftml-title={}}{#1}}\par
  }{
    \notesslidestitleemph{#1}
  }
  \medskip
}
```

(*End of definition for* \frametitle*. This function is documented on page* ??.)

\pause

```
\newcommand\pause{}
```

(*End of definition for* \pause*. This function is documented on page* ??.)

We redefine the columns and column environments:

```
\renewenvironment{columns}[1][]{
  \par\noindent
  \begin{minipage}
  \slidewidth\centering\leavevmode
%  \stex_if_html_backend:T{
%    \cs_if_exist:NT \rustex_if:T {
%      \rustex_if:T {\par
%        \rustex_direct_HTML:n{<table><tr><td>}
%      }
%    }
%  }
}{
%  \stex_if_html_backend:T{
%    \cs_if_exist:NT \rustex_if:T {
%      \rustex_if:T {\par
%        \rustex_direct_HTML:n{</td></tr></table>}
%      }
%    }
%  }
  \end{minipage}\par\noindent
}
\newsavebox\columnbox
\renewenvironment<>{column}[2][]{
  \begin{lrbox}{\columnbox}
%    \stex_if_html_backend:T{
%      \cs_if_exist:NT \rustex_if:T {
%        \rustex_if:T {\par
%          \rustex_direct_HTML:n{</td><td>}
%        }
%      }
%    }
    \begin{minipage}{#2}
}{
  \end{minipage}
%  \stex_if_html_backend:T{
%    \cs_if_exist:NT \rustex_if:T {
```

```
%       \rustex_if:T {\par
%         \rustex_direct_HTML:n{</td><td>}
%       }
%     }
%  }
  \end{lrbox}\usebox\columnbox
}
\fi
```

39.1.3 Environment and Macro Patches

The `note` environment is used to leave out text in the `slides` mode. It does not have a counterpart in OMDoc. So for course notes, we define the `note` environment to be a no-operation otherwise we declare the `note` environment to produce no output.

```
\bool_if:NTF \c_notesslides_notes_bool {
  \renewenvironment{note}{\ignorespaces}{}
}{
  \renewenvironment{note}{\setbox \l_tmpa_box\vbox\bgroup}{\egroup}
}
```

For other environments we introduce variants prefixed with `n`, which are excluded in `slides` mode.

```
\cs_new_protected:Nn \__notesslides_notes_env:nnnn {
  \bool_if:NTF \c_notesslides_notes_bool {
    \newenvironment{#1}#2{#3}{#4}
  }{
    \newenvironment{#1}#2{
      \cs_set:Npn \__notesslides_eat: ####1 \end ####2 {
        \str_if_eq:nnTF{#1}{####2}{
          \end{#1}
        }{
          \__notesslides_eat:
        }
      }
      \__notesslides_eat:
      %\setbox\l_tmpa_box\vbox\bgroup#3
    }{
      %#4\egroup
    }
  }
}

\__notesslides_notes_env:nnnn{nparagraph}{[1][]}{\begin{sparagraph}[#1]}{\end{sparagraph}}
\__notesslides_notes_env:nnnn{nfragment}{[2][]}{\begin{sfragment}[#1]{#2}}{\end{sfragment}}
\__notesslides_notes_env:nnnn{ndefinition}{[1][]}{\begin{sdefinition}[#1]}{\end{sdefinition
\__notesslides_notes_env:nnnn{nassertion}{[1][]}{\begin{sassertion}[#1]}{\end{sassertion}}
\__notesslides_notes_env:nnnn{nproof}{[2][]}{\begin{sproof}[#1]{#2}}{\end{sproof}}
\__notesslides_notes_env:nnnn{nexample}{[1][]}{\begin{sexample}[#1]}{\end{sexample}}

\RequirePackage{graphicx}

\NewDocumentCommand\frameimage{s O{} m}{
  \IfBooleanTF #1 {
```

```
    \begin{frame}[plain]
  }{
    \begin{frame}
  }
    \bool_if:NTF \c_notesslides_notes_bool {
      \slidewidth=\dimexpr\slidewidth-(2\slideframewidth)\relax
    }{
      \slidewidth=\textwidth\relax
    }
    \def\Gin@ewidth{}\setkeys{Gin}{#2}
    \tl_if_empty:NTF \Gin@ewidth {
      \mhgraphics[width=\slidewidth,#2]{#3}
    }{
      \mhgraphics[#2]{#3}
    }
  \end{frame}
}
```

hacking inputref:

\inputref*

```
\cs_set_eq:NN\__notesslides_inputref:\inputref
\cs_set_protected:Npn\inputref{\@ifstar\ninputref\__notesslides_inputref:}
\bool_if:NTF \c_notesslides_notes_bool {
  \newcommand\ninputref[2][]{
    \__notesslides_inputref:[#1]{#2}
  }
}{
  \newcommand\ninputref[2][]{}
}
```

(*End of definition for* \inputref*. *This function is documented on page 110.*)

39.1.4 Styling Across Notes/Slides

```
\def\notesslidestitleemph#1{
  {\Large\bf\sf#1}
  \vskip0.1\baselineskip
  \leaders\vrule width \textwidth
  \vskip0.4pt%
  \nointerlineskip
}

\def\notesslidesfooter{}

\let\notesslidesfont\sffamily
```

39.1.5 Beamer Compatibility

All of this should be removed and made part of a template

```

\stex_if_do_html:TF{
  \NewDocumentEnvironment{slideshow}{}{
    \begin{stex_annotate_env}{data-ftml-slideshow={}}
    \cs_set_protected:Npn \nextslide ##1 {
```

```
      \begin{stex_annotate_env}{data-ftml-slideshow-slide={}} ##1
      \end{stex_annotate_env}
    }
    \cs_set_protected:Npn \lastslide ##1 {
      \begin{stex_annotate_env}{data-ftml-slideshow-slide={}} ##1
      \end{stex_annotate_env}
    }
  }{
    \end{stex_annotate_env}
  }
}{
  \int_new:N \l__notesslides_slideshow_counter_int
  \NewDocumentEnvironment{slideshow}{}{
    \int_zero:N \l__notesslides_slideshow_counter_int
    \cs_set_protected:Npn \nextslide ##1 {
      \int_incr:N \l__notesslides_slideshow_counter_int
      \only<\int_use:N \l__notesslides_slideshow_counter_int>{##1}
    }
    \cs_set_protected:Npn \lastslide ##1 {
      \int_incr:N \l__notesslides_slideshow_counter_int
      \only<\int_use:N \l__notesslides_slideshow_counter_int ->{##1}
    }
  }{

  }
}

\bool_if:NT \c_notesslides_notes_bool {
  \def\author{\@dblarg\ns@author}
  \long\def\ns@author[#1]#2{%
    \tl_if_empty:nTF{#1}{
      \def\beamer@shortauthor{#2}
    }{
      \def\beamer@shortauthor{#1}
    }
    \def\@author{#2}
  }
  \def\title{\@dblarg\ns@title}
  \long\def\ns@title[#1]#2{%
    \tl_if_empty:nTF{#1}{
      \def\beamer@shorttitle{#2}
    }{
      \def\beamer@shorttitle{#1}
    }
    \def\@title{#2}
    \stexdoctitle{#2}
  }
  \def\insertshortauthor{
    \hbox\bgroup\def\\{}\cs_if_exist:NT\beamer@shortauthor\beamer@shortauthor\egroup
  }
  \def\insertshorttitle{
    \hbox\bgroup\def\\{}\cs_if_exist:NT\beamer@shorttitle\beamer@shorttitle\egroup
  }
  \stex_if_html_backend:TF{
```

```
    \def\insertframenumber{\stex_annotate:nn{data-ftml-slide-number={}}{}}
  }{
    \def\insertframenumber{\@arabic\c@framenumber}
  }
  \def\insertshortdate{\today}
}
```

39.1.6 TODO Excursions

\excursion The excursion macros are very simple, we define a new internal macro \excursionref and use it in \excursion, which is just an \inputref that checks if the new macro is defined before formatting the file in the argument.

```
\gdef\printexcursions{}



\stex_if_html_backend:TF{
  \newcommand\excursionref[4][]{% repos, label, path, text
    \begin{sparagraph}[title=Excursion]
      #4~
      \stex_in_archive:nn{#1}{
        \stex_document_uri_from_archive_file:Nn \l__notesslides_uri {#3}
        \exp_args:Ne\href{\stex_use_document_uri:N\l__notesslides_uri}{here}
      }
    \end{sparagraph}
  }
  \def\excursion{\excursionref}
}{
  \newcommand\excursionref[4][]{% repos, label, path, text
    \bool_if:NT \c_notesslides_notes_bool {
      \begin{sparagraph}[title=Excursion]
        #4~\sref[fallback=the appendix]{#2}.
      \end{sparagraph}
    }
  }
  \newcommand\activate@excursion[2][]{
    \tl_gput_right:Nn\printexcursions{\inputref[#1]{#2}}
  }
  \newcommand\excursion[4][]{% repos, label, path, text
    \bool_if:NT \c_notesslides_notes_bool {
      \activate@excursion[#1]{#3}
      \excursionref[#1]{#2}{#3}{#4}
    }
  }

}
```

(*End of definition for* \excursion*. This function is documented on page 112.*)

\excursiongroup

```
\keys_define:nn{notesslides / excursiongroup }{
  id        .str_set_x:N  = \l__notesslides_excursion_id_str,
  intro     .tl_set:N     = \l__notesslides_excursion_intro_tl,
  archive   .str_set_x:N  = \l__notesslides_excursion_mhrepos_str
```

```
}
\cs_new_protected:Nn \__notesslides_excursion_args:n {
  \tl_clear:N \l__notesslides_excursion_intro_tl
  \str_clear:N \l__notesslides_excursion_id_str
  \str_clear:N \l__notesslides_excursion_mhrepos_str
  \keys_set:nn {notesslides / excursiongroup }{ #1 }
}


\stex_if_html_backend:TF{
  \newcommand\excursiongroup[1][]{}
}{
  \newcommand\excursiongroup[1][]{
    \__notesslides_excursion_args:n{ #1 }
    \tl_if_empty:NF\printexcursions
    {\IfInputref{}{\begin{note}
      \begin{sfragment}{Excursions}% TODO pass on id
        \ifdefempty\l__notesslides_excursion_intro_tl{}{
          \exp_args:NNe \use:nn \inputref{[\l__notesslides_excursion_mhrepos_str]{
            \l__notesslides_excursion_intro_tl
          }}
        }
        \printexcursions%
      \end{sfragment}
    \end{note}}}
  }
}

\ifcsname beameritemnestingprefix\endcsname\else\def\beameritemnestingprefix{}\fi
```

(*End of definition for* `\excursiongroup`*. This function is documented on page 112.*)

```
\prop_new:N \g__notesslides_variables_prop
\cs_set_protected:Npn \setSGvar #1 #2 {
  \prop_gput:Nnn \g__notesslides_variables_prop {#1}{#2}
}
\cs_set_protected:Npn \useSGvar #1 {
  \prop_item:Nn \g__notesslides_variables_prop {#1}
}
\cs_set_protected:Npn \ifSGvar #1 #2 #3 {
  \prop_get:NnNF \g__notesslides_variables_prop {#1} \l__notesslides_tmp {
    \PackageError{document-structure}
    {The sTeX Global variable #1 is undefined}
    {set it with \protect\setSGvar}\TODO better error
  }
  \tl_if_eq:NnT \l__notesslides_tmp {#2}{ #3 }
}


⟨/package⟩
```

39.2 Implementation: The problem Package

39.2.1 Package Options

The first step is to declare (a few) package options that handle whether certain information is printed or not. They all come with their own conditionals that are set by the options.

```
⟨*package⟩
⟨@@=problems⟩
\ProvidesExplPackage{problem}{2025/11/11}{4.0.0}{Semantic Markup for Problems}
\RequirePackage{l3keys2e}

\keys_define:nn { problem / pkg }{
  notes     .default:n    = { true },
  notes     .bool_set:N   = \c__problems_notes_bool,
  gnotes    .default:n    = { true },
  gnotes    .bool_set:N   = \c__problems_gnotes_bool,
  hints     .default:n    = { true },
  hints     .bool_set:N   = \c__problems_hints_bool,
  solutions .default:n    = { true },
  solutions .bool_set:N   = \c__problems_solutions_bool,
  pts       .default:n    = { true },
  pts       .bool_set:N   = \c__problems_pts_bool,
  min       .default:n    = { true },
  min       .bool_set:N   = \c__problems_min_bool,
  %boxed     .default:n    = { true },
  %boxed     .bool_set:N   = \c__problems_boxed_bool,
  test     .default:n    = { true },
  test     .bool_set:N   = \c__problems_test_bool,
  unknown     .code:n       = {
    \PassOptionsToPackage{\CurrentOption}{stex}
  }
}
\newif\ifsolutions

\ProcessKeysOptions{ problem / pkg }
\bool_if:NTF \c__problems_solutions_bool {
  \solutionstrue
}{
  \solutionsfalse
}
\newif\ifintest
\bool_if:NTF \c__problems_test_bool {
  \intesttrue
}{
  \intestfalse
}

\RequirePackage{stex}

\newcommand\precondition[2]{
  \stex_get_symbol:n{#2}
  \tl_if_empty:NF \l_stex_get_symbol_uri {
    \str_case:nnTF {#1}{
```

```
      {remember}{}
      {understand}{}
      {analyze}{}
      {evaluate}{}
      {apply}{}
      {create}{}
    }{
      \ifstexhtml\else\bool_if:NT\c_stex_metadata_bool {
        \marginpar{\tiny \textbf{Precondition:}~#1~
          \exp_args:Ne\symrefemph@uri{\stex_symbol_uri_name:N \l_stex_get_symbol_uri}{\l_st
        }
      }\fi
      \stex_annotate_invisible:nn{
        data-ftml-preconditionsymbol={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
        data-ftml-preconditiondimension={#1}
      }{}
    }{\errmessage{Unknown~cognitive~dimension~#1}}
  }
}
\newcommand\objective[2]{
  \stex_get_symbol:n{#2}
  \tl_if_empty:NF \l_stex_get_symbol_uri {
    \str_case:nnTF {#1}{
      {remember}{}
      {understand}{}
      {analyze}{}
      {evaluate}{}
      {apply}{}
      {create}{}
    }{
      \ifstexhtml\else\bool_if:NT\c_stex_metadata_bool {
        \marginpar{\tiny \textbf{Objective:}~#1~
        \exp_args:Ne\symrefemph@uri{\stex_symbol_uri_name:N \l_stex_get_symbol_uri}{\l_stex
        }
      }\fi
      \stex_annotate_invisible:nn{
        data-ftml-objectivesymbol={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
        data-ftml-objectivedimension={#1}
      }{}
    }{\errmessage{Unknown~cognitive~dimension~#1}}
  }
}
```

`\problem@kw@*` For multilinguality, we define internal macros for keywords that can be specialized in `*.ldf` files.

```
\AddToHook{begindocument}{
  \ExplSyntaxOn\makeatletter
  \input{problem-english.ldf}
  \ltx@ifpackageloaded{babel}{
   \clist_set:Nx \l_tmpa_clist {\exp_args:No \tl_to_str:n \bbl@loaded}
      \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{ngerman}}{
        \input{problem-ngerman.ldf}
      }
      \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{finnish}}{
```

```
        \input{problem-finnish.ldf}
      }
      \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{french}}{
        \input{problem-french.ldf}
      }
      \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{russian}}{
        \input{problem-russian.ldf}
      }
  }{}
  \makeatother\ExplSyntaxOff
}
```

(*End of definition for* `\problem@kw@*`. *This function is documented on page* **??**.)

39.2.2 Problems and Solutions

We now prepare the KeyVal support for problems. The key macros just set appropriate internal macros.

```
\bool_new:N \l_stex_key_autogradable_bool
\stex_keys_define:nnnn{ problem }{
  \tl_clear:N \l_stex_key_pts_tl
  \tl_set:Nn \l_stex_key_min_tl 0
  \str_clear:N \l_stex_key_name_str
  \str_clear:N \l_stex_key_mhrepos_str
  \bool_set_false:N \l_stex_key_autogradable_bool
}{
  pts     .tl_set:N     = \l_stex_key_pts_tl,
  min     .tl_set:N     = \l_stex_key_min_tl,
  name    .str_set:N    = \l_stex_key_name_str,
  autogradable .bool_set:N = \l_stex_key_autogradable_bool,
  archive  .code:n = {},
  %archive .str_set:N    = \l_stex_key_mhrepos_str,
  %creators .code:n = {}
  %imports .tl_set:N     = \l__problems_prob_imports_tl,
  %refnum  .int_set:N    = \l__problems_prob_refnum_int,
}{id,title,style,uses}
```

Then we set up a counter for problems.

`\numberproblemsin`

```
\newcounter{sproblem}[section]
\newcommand\numberproblemsin[1]{
  \@addtoreset{sproblem}{#1}
  \def\thesproblem{\arabic{#1}.\arabic{sproblem}}
}
\numberproblemsin{section}
%\def\theplainsproblem{\arabic{sproblem}}
%\def\thesproblem{\thesection.\theplainsproblem}
```

(*End of definition for* `\numberproblemsin`. *This function is documented on page* **??**.)

`sproblem` (*env.*)

```
\cs_new:Nn \__problems_activate_macros: {
  \stex_reactivate_macro:N \solution
  \stex_reactivate_macro:N \mcb
```

```
  \stex_reactivate_macro:N \scb
  \stex_reactivate_macro:N \fillinsol
  \stex_reactivate_macro:N \hint
  \stex_reactivate_macro:N \exnote
  \stex_reactivate_macro:N \gnote
}

\fp_new:N \g_problem_total_pts_fp
\fp_new:N \g_problem_total_min_fp

\bool_new:N \l__problems_in_problem_bool
\bool_new:N \l__problems_has_pts_bool
\bool_new:N \l__problems_has_min_bool
\bool_set_false:N \l__problems_in_problem_bool
\stex_new_stylable_env:nnnnnnn {problem} {O{}}{
  \bool_if:NT \l__problems_in_problem_bool {
    \msg_error:nn{stex}{error/nestedproblem}
  }
  \cs_if_exist:NTF \l_problem_inputproblem_keys_tl {
    \tl_put_left:Nn \l_problem_inputproblem_keys_tl {#1,}
    \exp_args:Nno \stex_keys_set:nn{problem}{
      \l_problem_inputproblem_keys_tl
    }
  }{
    \stex_keys_set:nn{problem}{#1}
  }
  \refstepcounter{sproblem}

  \stex_if_do_html:T {
    \str_if_empty:NT \l_stex_key_name_str {
      \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
      \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
    }
    \exp_args:Nne \begin{stex_annotate_env} {
      data-ftml-problem={\l_stex_key_name_str},
      %data-ftml-language={ \l_stex_current_language_str},
      data-ftml-autogradable={\bool_if:NTF \l_stex_key_autogradable_bool {true}{false}}
      \tl_if_empty:NF \l_stex_key_pts_tl{,data-ftml-problempoints={\l_stex_key_pts_tl}}
    }
    \noindent \stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{\l_stex_k
    \tl_if_empty:NF \l_stex_key_title_tl {
      \exp_args:No \stexdoctitle \l_stex_key_title_tl
    }
    \_stex_annotate_force_break:n{}
  }
  \tl_set_eq:NN \thistitle \l_stex_key_title_tl
  \tl_if_empty:NT \l_stex_key_pts_tl { \tl_set:Nn \l_stex_key_pts_tl{0} }


  \bool_set_true:N \l__problems_in_problem_bool
  \tl_set_eq:NN \l__problems_pts_tl \l_stex_key_pts_tl
  \tl_set_eq:NN \l__problems_min_tl \l_stex_key_min_tl
  \tl_if_eq:NnTF \l__problems_pts_tl {0}
    {\bool_set_false:N \l__problems_has_pts_bool}
```

```
    {\bool_set_true:N \l__problems_has_pts_bool}
  \tl_if_eq:NnTF \l__problems_min_tl {0}
    {\bool_set_false:N \l__problems_has_min_bool}
    {\bool_set_true:N \l__problems_has_min_bool}
  \int_gzero:N \g__problems_subproblem_int

  \stex_if_do_html:F\stex_style_apply:
  \_stex_do_id:
  \__problems_activate_macros:
}{
  \fp_gadd:Nn \g_problem_total_pts_fp { \l__problems_pts_tl}
  \fp_gadd:Nn \g_problem_total_min_fp { \l__problems_min_tl}
  \__problems_record_problem:
  \stex_if_do_html:F\stex_style_apply:
  \stex_if_do_html:T{ \end{stex_annotate_env} }
}{
  \par\noindent\problemheader
  \stex_if_do_html:F{
    \bool_if:NT \c__problems_pts_bool {
      \tl_if_eq:NnF \l__problems_pts_tl {0}{
        \marginpar{\l__problems_pts_tl{}~\problem@kw@pts\smallskip}
      }
    }
    \bool_if:NT \c__problems_min_bool {
      \tl_if_eq:NnF \l__problems_min_tl {0} {
        \marginpar{\l__problems_min_tl{}~\problem@kw@minutes\smallskip}
      }
    }
  }
  \par
  \stex_ignore_spaces_and_pars:
}{
  \par\bigskip
%  \bool_if:NT \c__problems_test_bool \pagebreak
}{s}

\tl_set:Nn \problemheader {
  \stex_if_do_html:TF{
    \tl_if_empty:NF \thistitle {
      \stex_annotate:nn{data-ftml-title={}}{\textbf{\thistitle}}
    }
  }{
    \textbf{\sproblemautorefname{~}\thesproblem
      \tl_if_empty:NF \thistitle {
        {~}(\thistitle)
      }
    }
  }
}

\cs_new_protected:Nn \__problems_record_problem: {
  \exp_args:NNe \iow_now:Nn \@auxout {
    \problem@restore {\thesproblem}{\l__problems_pts_tl}{\l__problems_min_tl}
  }
```

```
}

\cs_new_protected:Npn \problem@restore #1 #2 #3 {}
```

subproblem (*env.*)

```
\int_new:N \g__problems_subproblem_int

\stex_new_stylable_env:nnnnnnn {subproblem} {O{}}{
  \stex_keys_set:nn{problem}{#1}
  \bool_if:NF \l__problems_in_problem_bool{
    \ifstexhtml\else
      \par{\bfseries WARNING~subproblem~to~be~used~in~some~problem}\par
    \fi
    \__problems_activate_macros:
    \bool_set_true:N \l__problems_in_problem_bool
    \tl_set:Nn \l__problems_pts_tl{0}
    \tl_set:Nn \l__problems_min_tl{0}
  }
  \str_if_empty:NT \l_stex_key_name_str {
    \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
    \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
  }
  \stex_if_do_html:T {
    \str_if_empty:NT \l_stex_key_name_str {
      \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
      \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
    }
    \exp_args:Nne \begin{stex_annotate_env} {
      data-ftml-subproblem={\l_stex_key_name_str},
      %data-ftml-language={ \l_stex_current_language_str},
      data-ftml-autogradable={\bool_if:NTF \l_stex_key_autogradable_bool {true}{false}}
      \tl_if_empty:NF \l_stex_key_pts_tl{,data-ftml-problempoints={\l_stex_key_pts_tl}}
    }
    \noindent \stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{\l_stex_k
    \tl_if_empty:NF \l_stex_key_title_tl {
      \exp_args:No \stexdoctitle \l_stex_key_title_tl
    }
    \tl_if_empty:NF \l_stex_key_title_tl {
      \exp_args:No \stexdoctitle \l_stex_key_title_tl
    }
    \_stex_annotate_force_break:n{}
  }
  \tl_if_empty:NT \l_stex_key_pts_tl { \tl_set:Nn \l_stex_key_pts_tl{0} }
  \int_gincr:N \g__problems_subproblem_int
  \bool_if:NF \l__problems_has_pts_bool {
    \tl_gset:Ne \l__problems_pts_tl {\fp_to_decimal:n {\l__problems_pts_tl + \l_stex_key_pt
  }
  \bool_if:NF \l__problems_has_min_bool {
    \tl_gset:Ne \l__problems_min_tl {\fp_to_decimal:n {\l__problems_min_tl + \l_stex_key_mi
  }
  \stex_if_smsmode:F {\stex_if_do_html:F \stex_style_apply: }
}{
  \stex_if_do_html:TF{ \end{stex_annotate_env} }{\stex_if_smsmode:F\stex_style_apply:}
}{
```

```
  \begin{list}{}{
    \setlength\topsep{0pt}
    \setlength\parsep{0pt}
    \setlength\rightmargin{0pt}
  }\item[\int_use:N \g__problems_subproblem_int .]
  \bool_if:NT \c__problems_pts_bool {
    \bool_if:NF \l__problems_has_pts_bool {
      \marginpar{\smallskip\l_stex_key_pts_tl{}~\problem@kw@pts}
    }
  }
  \bool_if:NT \c__problems_min_bool {
    \bool_if:NF \l__problems_has_min_bool{
      \marginpar{\smallskip\l_stex_key_min_tl{}~\problem@kw@minutes}
    }
  }
  \ignorespaces
}{
  \end{list}
}{}
```

\includeproblem

```
\stex_keys_define:nnnn{ includeproblem }{
  \str_clear:N \l_stex_key_mhrepos_str
}{
  archive .str_set:N    = \l_stex_key_mhrepos_str,
  unknown .code:n = {}
}{}

\NewDocumentCommand\includeproblem{O{} m}{
  \group_begin:
  \tl_set:Nn \l_problem_inputproblem_keys_tl {#1}
  \stex_keys_set:nn{includeproblem}{#1}
  \exp_args:Nno \use:nn{\inputref[}\l_stex_key_mhrepos_str]{#2}
  \group_end:
}

```

(*End of definition for* \includeproblem. *This function is documented on page 117.*)

solution (*env.*)

```
\int_new:N \g_problem_id_counter
\dim_new:N \l_stex_key_testspace_dim
\stex_keys_define:nnnn{ solution }{
  \str_clear:N \l_stex_key_answerclass_str
  \dim_zero:N \l_stex_key_testspace_dim
}{
  testspace   .dim_set:N = \l_stex_key_testspace_dim,
  answerclass .str_set:N = \l_stex_key_answerclass_str
}{id,title,style}

\cs_new_protected:Nn \__problems_solution_start:n {
  \stex_keys_set:nn{ solution }{#1}
  \str_if_empty:NT \l_stex_key_id_str {
    \int_gincr:N \g_problem_id_counter
```

```
    \str_set:Nx \l_stex_key_id_str {
      SOLUTION_\int_use:N \g_problem_id_counter
    }
  }
  \stex_if_do_html:T{
    \begin{stex_annotate_env}{
      data-ftml-solution=\l_stex_key_id_str,
      data-ftml-answerclass={\l_stex_key_answerclass_str}
    }
  }
}

\stex_new_stylable_env:nnnnnnn { solution }{ O{} }{
  \stex_if_do_html:TF{
    \__problems_solution_start:n{#1}
  }{
    \ifsolutions
      \__problems_solution_start:n{#1}
      \stex_style_apply:
    \else
      \stex_keys_set:nn{ solution }{#1}
      \testspace{\l_stex_key_testspace_dim}
      \setbox\l_tmpa_box\vbox\bgroup
    \fi
  }
}{
  \stex_if_do_html:TF{
    \end{stex_annotate_env}
  }{
    \ifsolutions
      \stex_style_apply:
      \stex_if_do_html:T{
        \end{stex_annotate_env}
      }
    \else
      \egroup
    \fi
  }
}{
  \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
  \noindent\emph{\problem@kw@solution\tl_if_empty:NF \l_stex_key_title_tl{
    {~}\l_stex_key_title_tl \str_if_empty:NF \l_stex_key_answerclass_str {
      (Answer Class: \l_stex_key_answerclass_str)
    }
  } :~}
}{
  \par\rule[.3em]{\linewidth}{0.4pt}\newline
}{}

\stex_deactivate_macro:Nn \solution {sproblem~environments}
```

\startsolutions
\stopsolutions

```
\cs_new_protected:Npn \startsolutions{
  \global\solutionstrue
```

```
}
\cs_new_protected:Npn \stopsolutions{
  \global\solutionsfalse
}
```

(*End of definition for* `\startsolutions` *and* `\stopsolutions`*. These functions are documented on page 114.*)

hint (*env.*)

```

\stex_keys_define:nnnn{ problemenv }{}{}{id,title,style}

\cs_new_protected:Nn \__problems_hint_start:n {
  \stex_keys_set:nn{ problemenv }{#1}
  \str_if_empty:NT \l_stex_key_id_str {
    \int_gincr:N \g_problem_id_counter
    \str_set:Nx \l_stex_key_id_str {
      HINT_\int_use:N \g_problem_id_counter
    }
  }
  \stex_if_do_html:T{
    \begin{stex_annotate_env}{
      data-ftml-problemhint=\l_stex_key_id_str
    }
  }
}

\stex_new_stylable_env:nnnnnnn { hint }{ O{} }{
  \stex_if_do_html:TF{
    \__problems_hint_start:n{#1}
  }{
    \bool_if:NTF \c__problems_hints_bool {
      \__problems_hint_start:n{#1}
      \stex_style_apply:
    }{
      \setbox\l_tmpa_box\vbox\bgroup
    }
  }
}{
  \stex_if_do_html:TF{
    \end{stex_annotate_env}
  }{
    \bool_if:NTF \c__problems_hints_bool {
      \stex_style_apply:
      \stex_if_do_html:T{
        \end{stex_annotate_env}
      }
    }{
      \egroup
    }
  }
}{
  \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
  \noindent\emph{\problem@kw@hint\tl_if_empty:NF \l_stex_key_title_tl{
```

```
    {~}\l_stex_key_title_tl
  } :~}
}{
  \par\rule[.3em]{\linewidth}{0.4pt}\newline
}{}
\stex_deactivate_macro:Nn \hint {sproblem~environments}
```

exnote (*env.*)

```
\cs_new_protected:Nn \__problems_exnote_start:n {
  \stex_keys_set:nn{ problemenv }{#1}
  \str_if_empty:NT \l_stex_key_id_str {
    \int_gincr:N \g_problem_id_counter
    \str_set:Nx \l_stex_key_id_str {
      EXNOTE_\int_use:N \g_problem_id_counter
    }
  }
  \stex_if_do_html:T{
    \begin{stex_annotate_env}{
      data-ftml-problemnote=\l_stex_key_id_str
    }
  }
}

\stex_new_stylable_env:nnnnnnn { exnote }{ O{} }{
  \stex_if_do_html:TF{
    \__problems_exnote_start:n{#1}
  }{
    \bool_if:NTF \c__problems_notes_bool {
      \__problems_exnote_start:n{#1}
      \stex_style_apply:
    }{
      \setbox\l_tmpa_box\vbox\bgroup
    }
  }
}{
  \stex_if_do_html:TF{
    \end{stex_annotate_env}
  }{
    \bool_if:NTF \c__problems_notes_bool {
      \stex_style_apply:
      \stex_if_do_html:T{
        \end{stex_annotate_env}
      }
    }{
      \egroup
    }
  }
}{
  \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
  \noindent\emph{\problem@kw@note\tl_if_empty:NF \l_stex_key_title_tl{
    {~}\l_stex_key_title_tl
  } :~}
}{
  \par\rule[.3em]{\linewidth}{0.4pt}\newline
```

```
}{}
\stex_deactivate_macro:Nn \exnote {sproblem~environments}
```

gnote (*env.*)

```
\int_new:N \l__problems_anscls_int

\cs_new_protected:Nn \__problems_gnote_start:n {
  \stex_keys_set:nn{ problemenv }{#1}
  \str_if_empty:NT \l_stex_key_id_str {
    \int_gincr:N \g_problem_id_counter
    \str_set:Nx \l_stex_key_id_str {
      GNOTE_\int_use:N \g_problem_id_counter
    }
  }

  \stex_if_do_html:TF{
    \begin{stex_annotate_env}{
      data-ftml-problemgnote=\l_stex_key_id_str
    }
  }
  \stex_style_apply:
}

\stex_new_stylable_env:nnnnnnn { gnote }{ O{} }{
  \stex_if_do_html:TF{
    \__problems_gnote_start:n{#1}
  }{
    \bool_if:NTF \c__problems_gnotes_bool {
      \__problems_gnote_start:n{#1}
    }{
      \setbox\l_tmpa_box\vbox\bgroup
    }
  }
  \stex_reactivate_macro:N \anscls
}{
  \stex_if_do_html:TF{
    \end{stex_annotate_env}
  }{
    \bool_if:NTF \c__problems_gnotes_bool {
      \stex_style_apply:
      \stex_if_do_html:T{
        \end{stex_annotate_env}
      }
    }{
      \egroup
    }
  }
}{
  \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
  \noindent\emph{\problem@kw@grading\str_if_empty:NF \l_stex_key_title_tl{
    {~}\l_stex_key_title_tl
  } :~}
}{
  \par\rule[.3em]{\linewidth}{0.4pt}\newline
```

```
}{}
\stex_deactivate_macro:Nn \gnote {sproblem~environments}


\stex_keys_define:nnnn{ anscls }{
  \str_clear:N \l_stex_key_pts_str
  \tl_clear:N \l_stex_key_feedback_tl
}{
  pts     .str_set:N = \l_stex_key_pts_str,
  feedback .tl_set:N = \l_stex_key_feedback_tl,
  update  .code:n    = {}
}{id}
\newcommand \anscls [2][] {
  \stex_keys_set:nn{ anscls }{#1}
  \str_if_empty:NT \l_stex_key_id_str {
    \int_incr:N \l__problems_anscls_int
    \str_set:Nx \l_stex_key_id_str {
      AC\int_use:N \l__problems_anscls_int
    }
  }
  \stex_if_do_html:TF{
    \begin{list}{}{
      \setlength\topsep{0pt}
      \setlength\parsep{0pt}
      \setlength\rightmargin{0pt}
    }\item[] \exp_args:Ne \stex_annotate:nn{
      data-ftml-answerclass={\l_stex_key_id_str}
      \str_if_empty:NF \l_stex_key_pts_str{
        ,data-ftml-answerclass-pts={\l_stex_key_pts_str}
      }
    }{
      #2~
      \tl_if_empty:NF \l_stex_key_feedback_tl{
        \stex_annotate:nn{
          data-ftml-answerclass-feedback={}
        }{\l_stex_key_feedback_tl}
      }
    }
    \end{list}
  }{
    \begin{list}{}{
      \setlength\topsep{0pt}
      \setlength\parsep{0pt}
      \setlength\rightmargin{0pt}
    }\item[\l_stex_key_id_str] #2
      \str_if_empty:NF \l_stex_key_pts_str {\par
        ~ \problem@kw@points :~\l_stex_key_pts_str
      }
      \tl_if_empty:NF \l_stex_key_feedback_tl {\par
        ~ \problem@kw@feedback :~\l_stex_key_feedback_tl
      }
    \end{list}
  }
}
```

```
\stex_deactivate_macro:Nn \anscls {gnote~environments}
```

The margin pars are reader-visible, so we need to translate

```
\def\pts#1{
  \bool_if:NT \c__problems_pts_bool {
    \stex_annotate:nn{data-ftml-problempoints={#1}}{\marginpar{#1~\problem@kw@pts}}
  }\hbox_unpack:N\c_empty_box
}
\def\min#1{
  \bool_if:NT \c__problems_min_bool {
    \stex_annotate:nn{data-ftml-problemminutes={}}{\marginpar{#1~\problem@kw@minutes}}
  }\hbox_unpack:N\c_empty_box
}
```

mcb (*env.*)

```
\stex_new_stylable_env:nnnnnnn{mcb}{O{}}{
  \stex_keys_set:nn{style}{#1}
  \cs_set:Nn \__problems_mccline:n{
    \begin{list}{}{
      \setlength\topsep{0pt}
      \setlength\parsep{0pt}
      \setlength\rightmargin{0pt}
    }\item[\problem_mcc_box_tl] ##1 \end{list}
  }

  \stex_if_do_html:T{
    \tl_set:Nn\problem_mcc_box_tl{}
    \__problems_maybe_inline:n{
      \exp_args:Nne \begin{stex_annotate_env}{
        data-ftml-multiple-choice-block={}
        \clist_if_empty:NF \l_stex_key_style_clist {,
          data-ftml-styles={\l_stex_key_style_clist}
        }
      }
    }
    \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
      \cs_set:Nn \__problems_mccline:n {##1}
    }{
      \clist_if_in:NnT \l_stex_key_style_clist {dropdown} {
      \cs_set:Nn \__problems_mccline:n {##1}
    }
    }
  }
  \stex_deactivate_macro:Nn \mcb {sproblem~environments}
  \stex_deactivate_macro:Nn \scb {sproblem~environments}
  \stex_deactivate_macro:Nn \solution {sproblem~environments}
  \stex_deactivate_macro:Nn \hint {sproblem~environments}
  \stex_deactivate_macro:Nn \exnote {sproblem~environments}
  \stex_deactivate_macro:Nn \gnote {sproblem~environments}
  \stex_reactivate_macro:N \mcc
  \stex_if_do_html:F \stex_style_apply:
}{
  \stex_if_do_html:TF{
    \__problems_maybe_inline:n{\end{stex_annotate_env}}
```

```
  }\stex_style_apply:
}{\par}{}{}

\stexstylemcb[inline]{
  {\Large[}\cs_set:Nn \__problems_mccline:n{\problem_mcc_box_tl{~} #1}
}{{\Large]}}

\stexstylemcb[dropdown]{
  {\Large[}\cs_set:Nn \__problems_mccline:n{\problem_mcc_box_tl{~} #1}
}{{\Large]}}

\stex_deactivate_macro:Nn \mcb {sproblem~environments}
```

we define the keys for the `mcc` macro

```
\cs_new_protected:Nn \__problems_do_yes_param:Nn {
  \exp_args:Nx \str_if_eq:nnTF { \str_lowercase:n{ #2 } }{ yes }{
    \bool_set_true:N #1
  }{
    \bool_set_false:N #1
  }
}
\stex_keys_define:nnnn{mcc}{
  \tl_clear:N \l_stex_key_feedback_tl
  \bool_set_false:N \l_stex_key_T_bool
  \tl_clear:N \l_stex_key_Ttext_tl
  \tl_clear:N \l_stex_key_Ftext_tl
}{
  feedback  .tl_set:N     = \l_stex_key_feedback_tl ,
  T         .code:n       = {\bool_set_true:N \l_stex_key_T_bool} ,
  F         .code:n       = {\bool_set_false:N \l_stex_key_T_bool} ,
  Ttext     .tl_set:N     = \l_stex_key_Ttext_tl ,
  Ftext     .tl_set:N     = \l_stex_key_Ftext_tl ,
}{id}

```

`\mcc`

```

\cs_set:Nn \__problems_maybe_newline: { \\ }

\stex_if_html_backend:TF{
  \tl_set:Nn \problem_mcc_box_tl {
    \ltx@ifpackageloaded{amssymb}{$\square$}{
      \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
    }
  }

  \newcommand\mcc[2][]{
    \stex_keys_set:nn{mcc}{#1}
    \__problems_mccline:n{
      \stex_if_do_html:T{
        \stex_annotate:nn{data-ftml-problem-choice={
          \bool_if:NTF \l_stex_key_T_bool {true}{false}
        }}{
          #2~
```

```
          \stex_annotate:nn{data-ftml-problem-choice-verdict={}}{
            \bool_if:NTF \l_stex_key_T_bool {
              \tl_if_empty:NTF \l_stex_key_Ttext_tl \problem@kw@correct \l_stex_key_Ttext_t
            }{
              \tl_if_empty:NTF \l_stex_key_Ftext_tl \problem@kw@wrong \l_stex_key_Ftext_tl
            }
          }
          \tl_if_empty:NF \l_stex_key_feedback_tl {
            \stex_annotate:nn{data-ftml-problem-choice-feedback={}}{
              \l_stex_key_feedback_tl
            }
          }
        }
      }
    }
  }
}{
  \tl_set:Nn \problem_mcc_box_default_tl {
    \ltx@ifpackageloaded{amssymb}{$\square$}{
      \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
    }
  }
  \tl_set:Nn \problem_mcc_box_tl {
    \ifsolutions
      \bool_if:NTF \l_stex_key_T_bool {
        \makebox[0pt][l]{\problem_mcc_box_default_tl}
          \hspace{0.1em}$\cs_if_exist:NTF\checkmark{
            \raisebox{.15ex}{\checkmark}
          } X$
      }{\problem_mcc_box_default_tl}
    \else
      \problem_mcc_box_default_tl
    \fi
  }
  \newcommand\mcc[2][]{
    \stex_keys_set:nn{mcc}{#1}
    \ifsolutions
      \tl_set:Nx \l_tmpa_tl{
        \bool_if:NTF \l_stex_key_T_bool {
          \tl_if_empty:NF \l_stex_key_Ttext_tl {\exp_not:N \l_stex_key_Ttext_tl}
        }{
          \tl_if_empty:NF \l_stex_key_Ftext_tl {\exp_not:N \l_stex_key_Ftext_tl}
        }
        \tl_if_empty:NF \l_stex_key_feedback_tl {
          \exp_not:n{\\\emph{\l_stex_key_feedback_tl}}
        }
      }
    \fi

    \__problems_mccline:n{ #2
      \ifsolutions
        \tl_if_empty:NF \l_tmpa_tl { \footnote{\l_tmpa_tl} }
      \fi
    }
```

```
  }
}
\stex_deactivate_macro:Nn \mcc {mcb~environments}
```

(*End of definition for* \mcc. *This function is documented on page 115.*)

scb (*env.*)

```

\cs_new_protected:Nn \__problems_maybe_inline:n {
  \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
    \let\__problems_oldpar:\stex_par:
    \cs_set:Nn\stex_par:{~}
    \cs_set:Nn\__problems_maybe_newline:{}
    #1
    \let\stex_par:\__problems_oldpar:
  }{
    \clist_if_in:NnTF \l_stex_key_style_clist {dropdown} {
      \let\__problems_oldpar:\stex_par:
      \cs_set:Nn\stex_par:{~}
      \cs_set:Nn\__problems_maybe_newline:{}
      #1
      \let\stex_par:\__problems_oldpar:
    }{\par #1}
  }
}

\stex_new_stylable_env:nnnnnnn{scb}{O{}}{
  \stex_keys_set:nn{style}{#1}
  \cs_set:Nn \__problems_sccline:n{
    \begin{list}{}{
      \setlength\topsep{0pt}
      \setlength\parsep{0pt}
      \setlength\rightmargin{0pt}
    }\item[\problem_scc_box_tl] ##1 \end{list}
  }

  \stex_if_do_html:T{
    \__problems_maybe_inline:n{
      \exp_args:Ne\stex_annotate_env{
        data-ftml-single-choice-block={}
        \clist_if_empty:NF \l_stex_key_style_clist {,
          data-ftml-styles={\l_stex_key_style_clist}
        }
      }
    }
    \tl_set:Nn\problem_scc_box_tl{}
    \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
      \cs_set:Nn \__problems_sccline:n {##1}
    }{
      \clist_if_in:NnT \l_stex_key_style_clist {dropdown} {
        \cs_set:Nn \__problems_sccline:n {##1}
      }
    }
  }
```

```
  \stex_deactivate_macro:Nn \mcb {sproblem~environments}
  \stex_deactivate_macro:Nn \scb {sproblem~environments}
  \stex_deactivate_macro:Nn \solution {sproblem~environments}
  \stex_deactivate_macro:Nn \hint {sproblem~environments}
  \stex_deactivate_macro:Nn \exnote {sproblem~environments}
  \stex_deactivate_macro:Nn \gnote {sproblem~environments}
  \stex_reactivate_macro:N \scc
  \stex_if_do_html:F \stex_style_apply:
}{
  \stex_if_do_html:TF{
    \__problems_maybe_inline:n { \endstex_annotate_env }
  }\stex_style_apply:
}{\par}{}{}

\stexstylescb[inline]{
  {\Large[}\cs_set:Nn \__problems_sccline:n{\problem_scc_box_tl{~} #1}
}{{\Large]}}

\stexstylescb[dropdown]{
  {\Large[}\cs_set:Nn \__problems_sccline:n{\problem_scc_box_tl{~} #1}
}{{\Large]}}

\stex_deactivate_macro:Nn \scb {sproblem~environments}
```

\scc

```

\stex_if_html_backend:TF{
  \tl_set:Nn\problem_scc_box_tl{$\bigcirc$}
  \newcommand\scc[2][]{
    \stex_keys_set:nn{mcc}{#1}
    \__problems_sccline:n{
      \stex_if_do_html:T{
        \stex_annotate:nn{data-ftml-problem-choice={
          \bool_if:NTF \l_stex_key_T_bool {true}{false}
        }}{
          #2~
          \stex_annotate:nn{data-ftml-problem-choice-verdict={}}{
            \bool_if:NTF \l_stex_key_T_bool {
              \tl_if_empty:NTF \l_stex_key_Ttext_tl \problem@kw@correct \l_stex_key_Ttext_t
            }{
              \tl_if_empty:NTF \l_stex_key_Ftext_tl \problem@kw@wrong \l_stex_key_Ftext_tl
            }
          }
          \tl_if_empty:NF \l_stex_key_feedback_tl {
            \stex_annotate:nn{data-ftml-problem-choice-feedback={}}{
              \l_stex_key_feedback_tl
            }
          }
        }
      }
    }
  }
}{
  \tl_set:Nn\problem_scc_box_default_tl{$\bigcirc$}
```

```
  \tl_set:Nn\problem_scc_box_tl{
    \ifsolutions
      \bool_if:NTF \l_stex_key_T_bool {
        \makebox[0pt][l]{\problem_scc_box_default_tl}
          \hspace{0.1em}$\cs_if_exist:NTF\checkmark{
            \raisebox{.15ex}{\checkmark}
          } X$
      }{\problem_scc_box_default_tl}
    \else
      \problem_scc_box_default_tl
    \fi
  }
  \newcommand\scc[2][]{
    \stex_keys_set:nn{mcc}{#1}
    \ifsolutions
      \tl_set:Nx \l_tmpa_tl{
        \bool_if:NTF \l_stex_key_T_bool {
          \tl_if_empty:NF \l_stex_key_Ttext_tl {\exp_not:N \l_stex_key_Ttext_tl}
        }{
          \tl_if_empty:NF \l_stex_key_Ftext_tl {\exp_not:N \l_stex_key_Ftext_tl}
        }
        \tl_if_empty:NF \l_stex_key_feedback_tl {
          \exp_not:n{ \\\emph{\l_stex_key_feedback_tl} }
        }
      }
    \fi

    \__problems_sccline:n{ #2
      \ifsolutions
        \tl_if_empty:NF \l_tmpa_tl { \footnote{\l_tmpa_tl} }
      \fi
    }
  }
}

\stex_deactivate_macro:Nn \scc {scb~environments}


\newcommand\yesTnoF{
  \begin{scb}[style=inline]
    \scc[T]{yes}~\scc[F]{no}
  \end{scb}
}
\newcommand\yesFnoT{
  \begin{scb}[style=inline]
    \scc[F]{yes}~\scc[T]{no}
  \end{scb}
}
\newcommand\trueTfalseF{
  \begin{scb}[style=inline]
    \scc[T]{true}~\scc[F]{false}
  \end{scb}
}
\newcommand\trueFfalseT{
```

```
  \begin{scb}[style=inline]
    \scc[F]{true}~\scc[T]{false}
  \end{scb}
}
```

(*End of definition for* \scc. *This function is documented on page* ??.)

\fillinsol

```
\stex_keys_define:nnnn{fillinsol}{
  \tl_clear:N \l__problems_fillin_solution_tl
  \dim_zero:N \l_stex_key_testspace_dim
}{
  testspace   .dim_set:N = \l_stex_key_testspace_dim,
  exact    .code:n = {\__problems_parse_fillin_arg:nnnn{exact}#1 },
  numrange .code:n = {\__problems_parse_fillin_arg:nnnn{numrange}#1 },
  regex    .code:n = {\__problems_parse_fillin_arg:nnnn{regex}#1 }
}{}

\tl_const:Nn \c__problems_true_tl{T}
\tl_const:Nn \c__problems_false_tl{F}

\msg_set:nnn{problem}{error/neither-true-nor-false}{
  Fillinsol~verdict~#1~is~neither~T~nor~F!
}

\stex_if_html_backend:TF{
  \cs_new:Nn \__problems_parse_fillin_arg:nnnn {
    \tl_set:Nn \l_tmpa_tl{#3}
    \tl_if_eq:NNF \l_tmpa_tl\c__problems_true_tl{
      \tl_if_eq:NNF \l_tmpa_tl\c__problems_false_tl {
        \msg_error:nnn{problem}{error/neither-true-nor-false}{#3}
      }
    }
    \tl_put_right:Ne \l__problems_fillin_solution_tl {
      \stex_annotate:nn{
        data-ftml-fillin-case={#1},
        data-ftml-fillin-case-value={\tl_to_str:n{#2}},
        data-ftml-fillin-case-verdict={
          \tl_if_eq:NNTF\l_tmpa_tl\c__problems_true_tl{true}{false}
        },
      }{\_stex_annotate_force_break:n{\exp_not:n{#4}}}
    }
  }
}{
  \cs_new:Nn \__problems_parse_fillin_arg:nnnn {
    \tl_put_right:Nn \l__problems_fillin_solution_tl {
      #1 & \tl_to_str:n{#2} & #3 & #4 \crcr
    }
  }
}


\newcommand\fillinsol[2][]{
  \stex_if_do_html:F \quad
```

```
  \mode_if_math:TF{
    \hbox{\__problems_fillinsol:nn{#1}{#2}}
  }{
    \__problems_fillinsol:nn{#1}{#2}
  }
  \stex_if_do_html:F \quad
}

\stex_if_html_backend:TF{
  \cs_new_protected:Nn \__problems_fillinsol:nn {
    \stex_keys_set:nn{fillinsol}{#1}
    \tl_if_empty:nF{#2}{
      \__problems_parse_fillin_arg:nnnn{exact}{#2}{T}{}
    }
    \hbox_set:Nn \l_tmpa_box{\fbox{\huge{\texttt{\tl_to_str:n{#2}}}}\hskip\l_stex_key_testsp
    \exp_args:Ne \stex_annotate:nn{
      data-ftml-fillinsol={},
      data-ftml-fillinsol-width={\dim_to_decimal:n{1.5 \box_wd:N \l_tmpa_box }}
    }{
      \_stex_annotate_force_break:n{
        \l__problems_fillin_solution_tl
        #2
      }
    }
  }
}{
  \cs_new_protected:Nn \__problems_fillinsol:nn {
    \stex_keys_set:nn{fillinsol}{#1}
    \ifsolutions

      \cs_if_exist:NT\textcolor{\textcolor{red}}{\fbox{\texttt{\tl_to_str:n{#2}}}}
      \tl_if_empty:NF \l__problems_fillin_solution_tl {
        \footnote{
          \halign{ ~##~\hfil & ~##~\hfil & ~##~\hfil & ~##~\hfil \cr
            \textbf{type}&\textbf{case}&\textbf{verdict}&\textbf{feedback}\cr
            \tl_if_empty:nF{#2}{
              exact & #2 & T & \cr
            }
            \l__problems_fillin_solution_tl
          }
        }
      }
    \else
      \fbox{\dim_compare:nNnTF\l_stex_key_testspace_dim={0pt}{
        \phantom{\huge{\tl_to_str:n{#2}}}
      }{
        \hspace{\l_stex_key_testspace_dim}\vphantom{\huge{A \tl_to_str:n{#2}}}
      }}
    \fi
  }
}

\stex_deactivate_macro:Nn \fillinsol {sproblem~environments}
```

(*End of definition for* `\fillinsol`. *This function is documented on page 117.*)

\testemptypage

```
\newcommand\testemptypage[1][]{%
 \bool_if:NT \c__problems_test_bool {\ \vfill\begin{center}\hwexam@kw@testemptypage\end{cen
}
```

(*End of definition for* \testemptypage. *This function is documented on page* ??.)

\testspace

```
\newcommand\testspace[1]{\bool_if:NT \c__problems_test_bool {\vspace*{#1}}}
\newcommand\testsmallspace{\testspace{1cm}}
\newcommand\testmedspace{\testspace{2cm}}
\newcommand\testbigspace{\testspace{3cm}}
```

(*End of definition for* \testspace. *This function is documented on page* ??.)

\testnewpage

```
\newcommand\testnewpage{\bool_if:NT \c__problems_test_bool {\newpage}}
```

(*End of definition for* \testnewpage. *This function is documented on page* ??.)

\testnewpage

```
\newcommand{\testnewpageInProblem}%
{\ifintest\vfill\begin{center}\emph{continued on next page}\end{center}\testnewpage\fi}%
```

(*End of definition for* \testnewpage. *This function is documented on page* ??.)

```
⟨/package⟩
```

39.3 Implementation: The hwexam Package

39.3.1 Package Options

The first step is to declare (a few) package options that handle whether certain information is printed or not. Some come with their own conditionals that are set by the options, the rest is just passed on to the `problems` package.

```
⟨*package⟩
\ProvidesExplPackage{hwexam}{2025/11/11}{4.0.0}{homework assignments and exams}
\RequirePackage{l3keys2e}

\keys_define:nn {hwexam / pkg}{
 multiple  .default:n   = { false },
 multiple .bool_set:N  = \c_hwexam_multiple_bool,
 qrcode  .default:n    = { false },
 qrcode  .bool_set:N  = \c_hwexam_qrcode_bool,
 unknown   .code:n    = {
  \PassOptionsToPackage{\CurrentOption}{problem}
 }
}
\ProcessKeysOptions{ hwexam /pkg }
\RequirePackage{problem}
```

\hwexam_kw_* For multilinguality, we define internal macros for keywords that can be specialized in *.ldf files.

```
\AddToHook{begindocument}{
 \ExplSyntaxOn\makeatletter
 \input{hwexam-english.ldf}
 \ltx@ifpackageloaded{babel}{
   \clist_set:Nx \l_tmpa_clist {\exp_args:No \tl_to_str:n \bbl@loaded}
   \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{ngerman}}{
    \input{hwexam-ngerman.ldf}
   }
   \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{finnish}}{
    \input{hwexam-finnish.ldf}
   }
   \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{french}}{
    \input{hwexam-french.ldf}
   }
   \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{russian}}{
    \input{hwexam-russian.ldf}
   }
 }{}
 \makeatother\ExplSyntaxOff
}
```

(*End of definition for* \hwexam_kw_*. *This function is documented on page* **??**.)

39.3.2 QR Codes

```
\group_begin:
  \escapechar=-1
  \xdef\_@@_qr_backslash{\string\\}
\group_end:

\bool_if:NT \c_hwexam_qrcode_bool {
 \RequirePackage{qrcode}
 \RequirePackage{marginnote}
 \str_new:N \g_@@_qr_json_str
 \bool_new:N \g_@@_qr_in_problems_json_bool
 \bool_set_false:N \g_@@_qr_in_problems_json_bool
 \bool_new:N \g_@@_qr_in_subproblems_json_bool
 \bool_set_false:N \g_@@_qr_in_subproblems_json_bool
 \bool_new:N \g_@@_qr_in_anscls_json_bool
 \bool_set_false:N \g_@@_qr_in_anscls_json_bool
 \bool_if:NTF \c__problems_gnotes_bool {
   \gdef \qrjson { \str_gput_right:Nx \g_@@_qr_json_str}
 }{
  \gdef \qrjson #1 {}
 }
 \qrjson{[}
 \AtEndDocument{\qrjson{]}}
 \def\_@@_qr_escape_char:n #1 {
  #1\exp_args:Nno\use:nn{\if_charcode:w#1}\_@@_qr_backslash#1\fi
 }

 \gdef\_@@_qr_escape:n #1{\exp_args:Ne\str_map_function:nN{\tl_to_str:n{#1}}\_@@_qr_escape_
```

```
\gdef \_@@_qr_escape:o #1{\exp_args:No\_@@_qr_escape:n#1}

\def\qrschema{TO:DO:\examnumber}

\bool_if:NTF \c__problems_test_bool {
 \def\doproblemqr{
  \ifstexhtml\else{
   \reversemarginpar\marginnote{\qrcode[height=1.5cm]{\qrschema}}
   \normalmarginpar
  }\fi
 }
 \def\insertexamnumber{
  \ifstexhtml\else
   \tl_if_exist:NTF \examnumber {
    {\Large\bfseries ID:~\examnumber}
   }{
    {\color{red}\Large\bfseries!!!~ WARNING:~NO~{\string\examnumber}~SET~!!!}\gdef\examnum
   }
   \global\def\insertexamnumber{}
  \fi
 }
}{
 \def\doproblemqr{}
 \def\insertexamnumber{}
}

\stexstyleproblem[noqr]{
 \par\noindent\problemheader \xdef\qrid{\thesproblem}
 \bool_if:NT \c__problems_pts_bool {
  \tl_if_eq:NnF \l__problems_pts_tl {0}{
   \marginpar{\l__problems_pts_tl{}~\problem@kw@points\smallskip}
  }
 }
 \bool_if:NT \c__problems_min_bool {
  \tl_if_eq:NnF \l__problems_min_tl {0} {
   \marginpar{\l__problems_min_tl{}~\problem@kw@minutes\smallskip}
  }
 }
 \par
 \stex_ignore_spaces_and_pars:
}{
 \par\bigskip
}

\stexstyleproblem{
 \par\noindent\problemheader \xdef\qrid{\thesproblem}
 \doproblemqr
 \bool_if:NT \c__problems_pts_bool {
  \tl_if_eq:NnF \l__problems_pts_tl {0}{
   \marginpar{\l__problems_pts_tl{}~\problem@kw@points\smallskip}
  }
 }
 \bool_if:NT \c__problems_min_bool {
  \tl_if_eq:NnF \l__problems_min_tl {0} {
```

```
    \marginpar{\l__problems_min_tl{}~\problem@kw@minutes\smallskip}
   }
  }

  \bool_if:NTF \g_@@_qr_in_problems_json_bool {
   \qrjson{,}
  }{
   \bool_gset_true:N \g_@@_qr_in_problems_json_bool
  }

  \qrjson {
   \c_left_brace_str
    "id":"\_@@_qr_escape:o\l_stex_key_id_str","title":"\_@@_qr_escape:o\l_stex_key_title_tl
    "number":"\thesproblem"
  }

  \tl_if_eq:NnF \l__problems_pts_tl {0}{
   \qrjson {
    ,"pts":\l__problems_pts_tl
   }
  }
  \par
  \stex_ignore_spaces_and_pars:
 }{
  \bool_if:NT \g_@@_qr_in_subproblems_json_bool {
   \qrjson {[]}
  }
  \bool_gset_false:N \g_@@_qr_in_subproblems_json_bool
  \qrjson {\c_right_brace_str}
  \par\bigskip
 }

 \stexstylesubproblem[noqr]{
  \begin{list}{}{
   \setlength\topsep{0pt}
   \setlength\parsep{0pt}
   \setlength\rightmargin{0pt}
  }\item[\int_use:N \g__problems_subproblem_int .]
  \xdef\qrid{\thesproblem.\int_use:N \g__problems_subproblem_int}
  \bool_if:NT \c__problems_pts_bool {
   \bool_if:NF \l__problems_has_pts_bool {
    \marginpar{\smallskip\l_stex_key_pts_tl{}~\problem@kw@points}
   }
  }
  \bool_if:NT \c__problems_min_bool {
   \bool_if:NF \l__problems_has_min_bool{
    \marginpar{\smallskip\l_stex_key_min_tl{}~\problem@kw@minutes}
   }
  }
 }{\end{list}}

 \stexstylesubproblem{
  \begin{list}{}{
   \setlength\topsep{0pt}
```

```
    \setlength\parsep{0pt}
    \setlength\rightmargin{0pt}
   }\item[\int_use:N \g__problems_subproblem_int .]
   \xdef\qrid{\thesproblem.\int_use:N \g__problems_subproblem_int}
   \doproblemqr
   \bool_if:NT \c__problems_pts_bool {
    \bool_if:NF \l__problems_has_pts_bool {
     \marginpar{\smallskip\l_stex_key_pts_tl{}~\problem@kw@points}
    }
   }
   \bool_if:NT \c__problems_min_bool {
    \bool_if:NF \l__problems_has_min_bool{
     \marginpar{\smallskip\l_stex_key_min_tl{}~\problem@kw@minutes}
    }
   }
   \bool_if:NTF \g_@@_qr_in_subproblems_json_bool {
    \qrjson {,}
   }{
    \qrjson {
     ,"subproblems":[
    }
    \bool_gset_true:N \g_@@_qr_in_subproblems_json_bool
   }
   \qrjson {
    \c_left_brace_str
     "id":"\_@@_qr_escape:o\l_stex_key_id_str","title":"\_@@_qr_escape:o\l_stex_key_title_tl
     "number":"\thesproblem.\int_use:N \g__problems_subproblem_int"
   }
   \bool_if:NF \l__problems_has_pts_bool {
    \qrjson {
     ,"pts":\l_stex_key_pts_tl
    }
   }
  }{
   \qrjson {\c_right_brace_str}
   \end{list}
  }

  \stexstylegnote{
   \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
   \noindent\emph{\problem@kw@grading\str_if_empty:NF \l_stex_key_title_tl{
    {~}\l_stex_key_title_tl
   } :~}
   \qrjson {
    ,"gnote": \c_left_brace_str
    "id":"\_@@_qr_escape:o\l_stex_key_id_str","title":"\_@@_qr_escape:o\l_stex_key_title_tl"
    "anscls":[
   }
  }{
   \qrjson {
    ]\c_right_brace_str
   }
   \par\rule[.3em]{\linewidth}{0.4pt}\newline
  }
```

```
\renewcommand \anscls [2][] {
 \stex_keys_set:nn{ anscls }{#1}
 \str_if_empty:NT \l_stex_key_id_str {
  \int_incr:N \l__problems_anscls_int
  \str_set:Nx \l_stex_key_id_str {
   AC\int_use:N \l__problems_anscls_int
  }
 }
 \bool_if:NTF \g_@@_qr_in_anscls_json_bool {
  \qrjson{,}
 }{
  \bool_set_true:N \g_@@_qr_in_anscls_json_bool
 }
 \qrjson{
  \c_left_brace_str
   "id":"\_@@_qr_escape:o\l_stex_key_id_str","description":"\_@@_qr_escape:n{#2}",
   "feedback":"\_@@_qr_escape:o\l_stex_key_feedback_tl",
   "pts":"\l_stex_key_pts_str"
  \c_right_brace_str
 }
 \begin{list}{}{
  \setlength\topsep{0pt}
  \setlength\parsep{0pt}
  \setlength\rightmargin{0pt}
 }\item[\l_stex_key_id_str]
  \stex_if_do_html:TF{
   \exp_args:Ne \stex_annotate:nn{
    data-ftml-answerclass={\l_stex_key_id_str}
    \str_if_empty:NF \l_stex_key_pts_str{
     ,data-ftml-answerclass-pts={\l_stex_key_pts_str}
    }
   }{
    #2
    \tl_if_empty:NF \l_stex_key_feedback_tl{
     \stex_annotate_invisible:nn{
      data-ftml-answerclass-feedback={true}
     }{\l_stex_key_feedback_tl}
    }
   }
  }{#2}
  \str_if_empty:NF \l_stex_key_pts_str {\par
   ~ \problem@kw@points :~\l_stex_key_pts_str
  }
  \str_if_empty:NF \l_stex_key_feedback_tl {\par
   ~ \problem@kw@feedback :~\l_stex_key_feedback_tl
  }
 \end{list}
}
\stex_deactivate_macro:Nn \anscls {gnote~environments}

\AtEndDocument{
 \message{^^J^^J\g_@@_qr_json_str^^J^^J}
 \bool_if:NT \c__problems_gnotes_bool {
```

```
    \iow_new:N \c_@@_qr_json_iow
    \iow_open:Nn \c_@@_qr_json_iow {\jobname-vollkorn.json}
    \iow_now:Nx \c_@@_qr_json_iow {\g_@@_qr_json_str}
    \iow_close:N \c_@@_qr_json_iow
   }
  }

  \renewcommand\testemptypage[1][]{%
   \bool_if:NT \c__problems_test_bool {\
   \xdef\qrid{P\thepage}
   \doproblemqr
   \vfill\begin{center}\hwexam@kw@testemptypage\end{center}\eject
   }
  }
 }
```

39.3.3 Assignments

Then we set up a counter for problems and make the problem counter inherited from `problem.sty` depend on it. Furthermore, we specialize the `\prob@label` macro to take the assignment counter into account.

`assignment` (*env.*)

```
 \stex_keys_define:nnnn{ assignment }{
   \tl_clear:N \l_stex_key_number_tl
  \tl_clear:N \l_stex_key_given_tl
  \tl_clear:N \l_stex_key_due_tl
 }{
   number  .tl_set:N     = \l_stex_key_number_tl,
   given   .tl_set:N     = \l_stex_key_given_tl,
   due     .tl_set:N     = \l_stex_key_due_tl,
  unknown .code:n = {}
 }{id,title,style}

 \newcounter{assignment}
 \stex_new_stylable_env:nnnnnnn {assignment}{O{}}{
   \cs_if_exist:NTF \l_hwexam_includeassignment_keys_tl {
     \tl_put_left:Nn \l_hwexam_includeassignment_keys_tl {#1,}
     \exp_args:Nno \stex_keys_set:nn{assignment}{
       \l_hwexam_includeassignment_keys_tl
     }
   }{
     \stex_keys_set:nn{assignment}{#1}
   }
  \tl_if_empty:NF \l_stex_key_number_tl {
   \global\setcounter{assignment}{\int_eval:n{\l_stex_key_number_tl-1}}
  }
  \global\refstepcounter{assignment}
  \setcounter{sproblem}{0}
  \def\thesproblem{\theassignment.\arabic{sproblem}}
  \stex_if_do_html:T{
   \tl_if_empty:NF \l_stex_key_title_tl {
    \stexdoctitle \l_stex_key_title_tl
   }
```

```
 }
 \stex_style_apply:
 \_stex_do_id:
}{
 \stex_style_apply:
}{
 \par\begin{center}
 \textbf{\Large\assignmentautorefname~\theassignment
  \tl_if_empty:NF \l_stex_key_title_tl {
   {~}---~\l_stex_key_title_tl
  }
 }\par\smallskip
 \textbf{
  \tl_if_empty:NF \l_stex_key_given_tl {
   \hwexam@kw@given :~\l_stex_key_given_tl\quad
  }
  \tl_if_empty:NF \l_stex_key_due_tl {
   \hwexam@kw@due :~\l_stex_key_due_tl\quad
  }
 }
 \end{center}
 \par\bigskip
}{
 \par\pagebreak
}{}
```

\includeassignment

```
\NewDocumentCommand\includeassignment{O{} m}{
 \group_begin:
 \tl_set:Nn \l_hwexam_includeassignment_keys_tl {#1}
 \stex_keys_set:nn{includeproblem}{#1}
 \exp_args:Nno \use:nn{\inputref[}\l_stex_key_mhrepos_str]{#2}
 \group_end:
}
```

(*End of definition for* \includeassignment. *This function is documented on page 76.*)

Restoring information about problems:

```
\prop_new:N \c_@@_problems_prop
\tl_set:Nn \c_@@_total_mins_tl {0pt}
\tl_set:Nn \c_@@_total_pts_tl {0pt}
\int_new:N \c_@@_total_problems_int
\cs_set_protected:Npn \problem@restore #1 #2 #3 {
 \int_gincr:N \c_@@_total_problems_int
 \prop_gput:Nnn \c_@@_problems_prop {#1}{{#2}{#3}}
 \tl_gset:Ne \c_@@_total_pts_tl { \dim_eval:n { \c_@@_total_pts_tl + #2pt }}
 \tl_gset:Ne \c_@@_total_mins_tl { \dim_eval:n { \c_@@_total_mins_tl + #2pt }}
}
```

\correction@table This macro generates the correction table

```
\newcommand\correction@table{
 \int_compare:nNnT \c_@@_total_problems_int = 0 {
  \int_incr:N \c_@@_total_problems_int
  \prop_put:Nnn \c_@@_problems_prop {~}{{~}{~}}
 }
```

```
 \tl_clear:N \l_tmpa_tl
 \tl_clear:N \l_tmpb_tl
 \tl_clear:N \l_tmpc_tl
 \prop_map_inline:Nn \c_@@_problems_prop {
  \tl_put_right:Nn \l_tmpa_tl { ##1 & }
  \tl_put_right:Nx \l_tmpb_tl { \use_i:nn ##2 & }
  \tl_put_right:Nn \l_tmpc_tl { & }
 }
 \resizebox{\textwidth}{!}{%
\exp_args:Nne \begin{tabular}{|l|*{\int_use:N \c_@@_total_problems_int}{c|}c||l|}\hline
&\exp_args:Ne \multicolumn{\int_eval:n{ \c_@@_total_problems_int + 1}}{c||}
{\footnotesize\hwexam@kw@forgrading} &\\\hline
\hwexam@kw@probs & \l_tmpa_tl \hwexam@kw@sum & \hwexam@kw@grade\\\hline
\hwexam@kw@pts & \l_tmpb_tl \dim_to_decimal:n\c_@@_total_pts_tl & \\\hline
\hwexam@kw@reached & \l_tmpc_tl & \\[.7cm]\hline
\end{tabular}}}
```

(*End of definition for* `\correction@table`. *This function is documented on page* **??**.)

`\testheading`

```
\def\hwexamheader{\input{hwexam-default.header}}

\def\hwexamminutes{
 \tl_if_empty:NTF \hwexam@duration {
  {\hwexam@min}~\hwexam@minutes@kw
 }{
  \hwexam@duration
 }
}

\stex_keys_define:nnnn{ hwexam / testheading }{
 \tl_clear:N \hwexam@min
 \tl_clear:N \hwexam@duration
 \tl_clear:N \hwexam@reqpts
 \tl_clear:N \hwexam@tools
}{
 min   .tl_set:N  = \hwexam@min,
 duration .tl_set:N  = \hwexam@duration,
 reqpts  .tl_set:N  = \hwexam@reqpts,
 tools  .tl_set:N  = \hwexam@tools
}{}

\newenvironment{testheading}[1][]{
 \stex_keys_set:nn { hwexam / testheading}{#1}

 \tl_set:Nx \hwexam@totalpts {\dim_to_decimal:n \c_@@_total_pts_tl}
 \tl_set:Nx \hwexam@totalmin {\dim_to_decimal:n \c_@@_total_mins_tl}
 \tl_set:Nx \hwexam@checktime {\dim_to_decimal:n { \hwexam@min pt - \hwexam@totalmin pt }}

 \newif\if@bonuspoints
 \tl_if_empty:NTF \hwexam@reqpts {
  \@bonuspointsfalse
 }{
  \tl_set:Nx \hwexam@bonuspts {
```

```
    \dim_to_decimal:n{\hwexam@totalpts pt - \hwexam@reqpts pt}
   }
   \@bonuspointstrue
  }

  \makeatletter\hwexamheader\makeatother
 }{
  \newpage
 }
```

(*End of definition for* \testheading. *This function is documented on page* **??**.)

examdata
quizdata
homeworkdata

```
\bool_new:N \g_@@_allow_data_bool
\bool_gset_true:N \g_@@_allow_data_bool
\AtBeginDocument{
 \bool_gset_false:N \g_@@_allow_data_bool
}

\msg_set:nnn{stex}{hwexam/doubledata}{
  Exam/Homework/Quiz~Data~already~set
}

\msg_set:nnn{stex}{hwexam/nodate}{
  Exam/Homework/Quiz~Data~missing~date~value
}

\msg_set:nnn{stex}{hwexam/nocourse}{
  Exam/Homework/Quiz~Data~missing~course~value
}

\stex_keys_define:nnnn{ hwexam / commondata }{
 % https://docs.rs/dateparser/latest/dateparser/#accepted-date-formats
 \str_clear:N \l_@@_key_exam_date_str
 \str_clear:N \l_@@_key_exam_course_id_str
 \str_clear:N \l_@@_key_exam_term_str
 \int_set:Nn \l_@@_key_exam_num_int {1}
}{
 date     .str_set:N = \l_@@_key_exam_date_str,
 course   .str_set:N = \l_@@_key_exam_course_id_str,
 term     .str_set:N = \l_@@_key_exam_term_str,
 num      .int_set:N = \l_@@_key_exam_num_int
}{}

\stex_keys_define:nnnn{ hwexam / examdata }{
 \bool_set_false:N \l_@@_key_exam_retake_bool
}{
 retake   .bool_set:N = \l_@@_key_exam_retake_bool
}{hwexam/commondata}

\cs_set_protected:Nn \_@@_do_metadata:nnnn {
 \bool_if:NF \g_@@_allow_data_bool {
  \msg_fatal:nn{stex}{hwexam/doubledata}
 }
```

```
 \bool_gset_false:N \g_@@_allow_data_bool
 \stex_keys_set:nn { hwexam / #1}{#2}
 \str_set:Nn \g_stex_document_kind_str{#4}
 \str_if_empty:NT\l_@@_key_exam_date_str{
  \str_set:Ne \l_@@_key_exam_date_str \today
  %\msg_fatal:nn{stex}{hwexam/nodate}
 }
 \str_if_empty:NT\l_@@_key_exam_course_id_str{
  \msg_fatal:nn{stex}{hwexam/nocourse}
 }
 \cs_if_exist:NT \date {
  \exp_args:No \date \l_@@_key_exam_date_str
 }

 \tl_set:Ne \g_stex_document_kind_parameters_tl{
  ,data-ftml-document-kind-date={\l_@@_key_exam_date_str}
  ,data-ftml-document-kind-num=\int_use:N \l_@@_key_exam_num_int
  ,data-ftml-document-kind-course=\l_@@_key_exam_course_id_str
  \str_if_empty:NF \l_@@_key_exam_term_str{
   ,data-ftml-document-kind-term=\l_@@_key_exam_term_str
  }
  #3
 }
}

\newcommand\examdata[1]{
 \_@@_do_metadata:nnnn{examdata}{#1}{
  ,data-ftml-document-kind-retake=\bool_if:NTF \l_@@_key_exam_retake_bool{true}{false}
 }{exam}
}

\newcommand\homeworkdata[1]{
 \_@@_do_metadata:nnnn{commondata}{#1}{}{homework}
}

\newcommand\quizdata[1]{
 \_@@_do_metadata:nnnn{commondata}{#1}{}{quiz}
}

```

(*End of definition for* `examdata`, `quizdata`, *and* `homeworkdata`. *These functions are documented on page* ??.)

⟨/package⟩

39.3.4 Leftovers

at some point, we may want to reactivate the logos font, then we use

```
here we define the logos that characterize the assignment
\font\bierfont=../assignments/bierglas
\font\denkerfont=../assignments/denker
\font\uhrfont=../assignments/uhr
\font\warnschildfont=../assignments/achtung
```

```
\newcommand\bierglas{{\bierfont\char65}}
\newcommand\denker{{\denkerfont\char65}}
\newcommand\uhr{{\uhrfont\char65}}
\newcommand\warnschild{{\warnschildfont\char 65}}
\newcommand\hardA{\warnschild}
\newcommand\longA{\uhr}
\newcommand\thinkA{\denker}
\newcommand\discussA{\bierglas}
```

39.4 Tikzinput Implementation

```
⟨@@=tikzinput⟩
⟨*package⟩

%%%%%%%%%%%%   tikzinput.dtx   %%%%%%%%%%%%

\ProvidesExplPackage{tikzinput}{2025/11/11}{4.0.0}{tikzinput package}
\RequirePackage{l3keys2e}

\keys_define:nn { tikzinput } {
  image   .bool_set:N   = \c_tikzinput_image_bool,
  image   .default:n    = false ,
  unknown   .code:n       = {}
}

\ProcessKeysOptions { tikzinput }

\bool_if:NTF \c_tikzinput_image_bool {
  \RequirePackage{graphicx}

  \providecommand\usetikzlibrary[]{}
  \newcommand\tikzinput[2][]{\includegraphics[#1]{#2}}
}{
  \RequirePackage{tikz}
  \RequirePackage{standalone}

  \newcommand \tikzinput [2] [] {
    \setkeys{Gin}{#1}
    \ifx \Gin@ewidth \Gin@exclamation
      \ifx \Gin@eheight \Gin@exclamation
        \input { #2 }
      \else
        \resizebox{!}{ \Gin@eheight }{
          \input { #2 }
        }
      \fi
    \else
      \ifx \Gin@eheight \Gin@exclamation
        \resizebox{ \Gin@ewidth }{!}{
          \input { #2 }
        }
      \else
```

```
        \resizebox{ \Gin@ewidth }{ \Gin@eheight }{
          \input { #2 }
        }
      \fi
    \fi
  }
}

\newcommand \ctikzinput [2] [] {
  \begin{center}
    \tikzinput [#1] {#2}
  \end{center}
}
```

⟨/package⟩

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols

\! 6603
\$ 2121, 2124, 2128
* commands:
  *:n *145*
\, ... 6603
\; 8042, 8074, 8076, 8103
@@ commands:
  \g_@@_allow_data_bool 10225, 10226, 10228, 10263, 10266
  _@@_do_metadata:nnnn 10262, 10292, 10298, 10302
  \l_@@_key_exam_course_id_str 10246, 10251, 10273, 10283
  \l_@@_key_exam_date_str . . 10245, 10250, 10269, 10270, 10277, 10281
  \l_@@_key_exam_num_int 10248, 10253, 10282
  \l_@@_key_exam_retake_bool 10257, 10259, 10293
  \l_@@_key_exam_term_str 10247, 10252, 10284, 10285
  \c_@@_problems_prop 10151, 10157, 10164, 10169
  _@@_qr_escape:n 9856, 9857, 9923, 9990, 10010, 10035, 10036
  _@@_qr_escape_char:n . . . 9852, 9856
  \g_@@_qr_in_anscls_json_bool 9843, 9844, 10028, 10031
  \g_@@_qr_in_problems_json_bool 9839, 9840, 9915, 9918
  \g_@@_qr_in_subproblems_json_bool 9841, 9842, 9935, 9938, 9980, 9986
  \c_@@_qr_json_iow 10073, 10074, 10075, 10076
  \g_@@_qr_json_str 9838, 9846, 10071, 10075
  \c_@@_total_mins_tl 10152, 10159, 10208
  \c_@@_total_problems_int . 10154, 10156, 10162, 10163, 10175, 10176
  \c_@@_total_pts_tl 10153, 10158, 10179, 10207
\\ 88, 115, 827, 1433, 8254, 8259, 8669, 8672, 9454, 9515, 9649, 9832, 10177, 10178, 10179, 10180
\{ 8079
\} 8079
\␣ 19, 79, 604, 741, 8046, 8108, 8286, 9785, 10081
_@@_qr_backslash 9832, 9853
_comp . . 3839, 4578, 5843, 6058, 6210, 6958
_customthiscomp 4938, 4939, 4951, 4956, 4959, 4964
_defcomp 5876, 6042
_thiscomp 4707, 4708, 5847
_varcomp 4280, 5195, 5216, 5766, 5865, 5973, 5987, 6001
\| 2120, 2123, 2127

A

\activateexcursion *64*, *112*
\addbibresource *82*, *141*, 2137
\addcontentsline 8284
\addmhbibresource *82*, *141*, 2130
\addtocounter 8479
\AddToHook 7689, 7829, 7833, 7837, 7964, 8855, 9810
\aftergroup *125, 126*, 2886, 2906, 2955, 4541, 7755, 7756, 7960
\afterprematurestop . . *63*, *111*, 8348, 8359
\amalg 8093
\anscls . . . 9296, 9329, 9371, 10020, 10068
\apply *90*
\arabic 8896, 8899, 10114
\arg *40*, *95*, 5316
\argarraymap *94*, *154*, 6290, 6848
\argmap *94*, *154*, *295*, 6289, 6606, 6824
\argsep *36*, *94*, *154*, 5699, 6288, 6601, 6805, 8042, 8043, 8044, 8052, 8083, 8087, 8091
\assign *58*, *146*, 3264, 3523
assignment (env.) *76*, *119*, 10088
\assignmentautorefname 10126
\assignMorphism *146*, 3265, 3628
\assumption *158*, 7585, 7968
\ast 8067
\AtBeginDocument 69, 77, 791, 799, 1452, 1479, 1660, 1685, 1757, 1761, 8130, 8303, 10227
\AtEndDocument . . . 612, 1762, 9851, 10070

\AtEndOfPackageFile 2173, 2189, 2204
\author 8649
\autoref *86*, 1826, 1842

B

\backmatter *210*, 1724, 1725, 1726
\baselineskip 8607
\beameritemnestingprefix 8747
\begin *98*, *128*, *139*, *142–144*, *157*, 1439, 1551, 1614, 2032, 2187, 2202, 2222, 2566, 2724, 2742, 2757, 3019, 6871, 7186, 7846, 8287, 8299, 8425, 8452, 8462, 8501, 8522, 8530, 8568, 8569, 8570, 8571, 8572, 8573, 8579, 8581, 8619, 8621, 8625, 8687, 8699, 8734, 8735, 8937, 9037, 9064, 9117, 9180, 9228, 9279, 9338, 9357, 9385, 9395, 9550, 9666, 9671, 9676, 9681, 9785, 9793, 9944, 9963, 10040, 10084, 10125, 10175, 10356
\begingroup 46, 2040, 2545
\bf 8606
\bfseries 9021, 9871, 9873
\bgroup 1863, 1944, 4323, 4486, 7598, 7722, 7811, 8451, 8546, 8561, 8669, 8672, 9134, 9194, 9242, 9293
\bigcirc 9600, 9626
\bigskip 8990, 9898, 9940, 10140
blindfragment (env.) *84*, *139*, 1597
bool commands:
  \bool_gset_eq:NN 3075, 7731
  \bool_gset_false:N 9938, 10228, 10266
  \bool_gset_true:N .. 9918, 9986, 10226
  \bool_if:NTF 473, 581, 587, 588, 594, 681, 709, 947, 956, 958, 1052, 1595, 1712, 2347, 2399, 2627, 2940, 3136, 3276, 3308, 3672, 4338, 4548, 5190, 5214, 5495, 5505, 5511, 5521, 5525, 5545, 5559, 5563, 5577, 5606, 5613, 5627, 6047, 6070, 6128, 6458, 6887, 6893, 7473, 7598, 7608, 7624, 7657, 7663, 7691, 7747, 7787, 7811, 7814, 7821, 7825, 7839, 7886, 7974, 8190, 8198, 8235, 8319, 8363, 8543, 8549, 8583, 8598, 8648, 8698, 8708, 8795, 8801, 8820, 8843, 8919, 8940, 8976, 8981, 8991, 9019, 9040, 9054, 9057, 9069, 9070, 9074, 9075, 9190, 9201, 9238, 9249, 9290, 9301, 9373, 9378, 9468, 9472, 9495, 9509, 9606, 9610, 9629, 9643, 9785, 9787, 9791, 9835, 9845, 9861, 9885, 9890, 9904, 9909, 9915, 9935, 9950, 9951, 9955, 9956, 9970, 9971, 9975, 9976, 9980, 9993, 10028, 10072, 10081, 10263, 10293, 10322
  \bool_if_exist:NTF 339, 353
  \bool_lazy_any:nTF 4169, 4238
  \bool_lazy_any_p:n 285
  \bool_new:N 27, 670, 1281, 1575, 2338, 2624, 2982, 4257, 4569, 5492, 5884, 6881, 7103, 8875, 8914, 8915, 8916, 9839, 9841, 9843, 10225
  \bool_not_p:n 284, 288
  \bool_set_false:N 455, 582, 678, 686, 944, 967, 1283, 1590, 2352, 2989, 3289, 3645, 4572, 5900, 6198, 6446, 6737, 6888, 7116, 7436, 7441, 7451, 7456, 7480, 7562, 7697, 7775, 7958, 8023, 8024, 8174, 8215, 8379, 8881, 8917, 8957, 8960, 9437, 9442, 9448, 9840, 9842, 9844, 10257
  \bool_set_true:N 341, 355, 463, 571, 577, 675, 690, 941, 1321, 1580, 2350, 2667, 2985, 3273, 3657, 3667, 3990, 4343, 4570, 4639, 4909, 4941, 4966, 5008, 5009, 5301, 5310, 5425, 5526, 5532, 5550, 5568, 5601, 5724, 5814, 5850, 5851, 5889, 6125, 6453, 6905, 7106, 7114, 7494, 7675, 8187, 8227, 8377, 8385, 8386, 8387, 8388, 8389, 8390, 8953, 8958, 8961, 9024, 9435, 9447, 10031
  \bool_while_do:Nn 942
  \bool_while_do:nn 283, 2153, 7499, 7511, 7523, 7540, 7552
  \l_tmpa_bool 3645, 3657, 3667, 3672, 6446, 6453, 6458
box commands:
  \box_wd:N 9748
  \c_empty_box 1499, 1509, 3865, 9375, 9380
  \l_tmpa_box 2666, 6280, 8546, 8561, 9134, 9194, 9242, 9293, 9745, 9748
boxed *113*

C

\catcode 19, 20, 46, 79, 88, 604, 741, 742, 827, 1945, 2119, 2120, 2121, 2122, 2123, 2124, 2126, 2127, 2128, 2458, 2459
\cdot 8074
\centering 8288, 8502
\chapter *26*, *84*, 8328, 8329
\chaptername 8259, 8325, 8326
\chaptertitlename 8259
\checkmark 9497, 9498, 9631, 9632
\child *125*

\circ *47*
\clearpage 1719, 1729, 1740, 1751
clist commands:
\clist_clear:N . . . 125, 399, 3713, 4640, 4790, 4890, 6179, 7156, 7161
\clist_count:N 4769, 4780
\clist_count:n 4873, 6856
\clist_get:NN 508, 546
\clist_if_empty:NTF 454, 504, 542, 4006, 4768, 4815, 5135, 6377, 7241, 9397, 9561
\clist_if_in:NnTF 104, 107, 131, 7336, 8860, 8863, 8866, 8869, 9402, 9405, 9530, 9537, 9567, 9570, 9815, 9818, 9821, 9824
\clist_if_in:nnTF 4899
\clist_item:Nn 5086, 6386
\clist_item:nn 6867
\clist_map_function:NN . . . 4641, 4818
\clist_map_function:nN 3441, 3507, 4091, 6553
\clist_map_inline:Nn 126, 135, 457, 3176, 4008, 4361, 4505, 5136, 6981, 7016, 7139, 7174
\clist_map_inline:nn 363, 412, 5650, 5753, 5782, 6979, 7391
\clist_pop:NN 6388
\clist_put_right:Nn 127, 4654, 4658, 4797, 4917, 4919
\clist_set:Nn 42, 123, 4419, 8859, 9814
\clist_set_eq:NN 121, 7191
\l_tmpa_clist 8859, 8860, 8863, 8866, 8869, 9814, 9815, 9818, 9821, 9824
\clstinputmhlisting *85*, *141*, 2189
\cmhgraphics *85*, *141*, 2173
\cmhtikzinput *121*, *141*, 2204
\color 9873
\columnbox 8520, 8522, 8540
\comp *31–34*, *40*, *92*, *95*, *105*, *151*, 3839, 4280, 4589, 4590, 4708, 4756, 4764, 4947, 4948, 4958, 5632, 5766, 5835, 5887, 5888, 5973, 5987, 6001, 6042, 6058, 6060, 6210, 6587, 6681, 6687, 6689, 6906, 6911, 6958, 8039, 8041, 8050, 8052, 8054, 8055, 8060, 8062, 8067, 8069, 8070, 8075, 8076, 8077, 8079, 8083, 8087, 8103, 8108, 8112, 8114, 8116
\comp-macro *151*
\compemph *105*, *151*, 5835
\conclude *158*, 7584, 7969
\conclusion . . *44*, *101*, *103*, *157*, 7325, 7399
\copymod *55–57*, *145*, 3451, 3453, 3455
copymodule (env.) *145*, 3391
\copymodule 3426, 3428
\counterwithin 8332, 8344
\cr 9764, 9765, 9767
\crcr 9723
cs commands:
\cs:w 483, 486, 517, 520, 605, 2403, 2404, 2405, 2411, 2415, 2421, 2680, 7057, 7059, 7215, 7229
\cs_argument_spec:N 4125
\cs_end: 483, 486, 517, 520, 605, 2403, 2404, 2405, 2413, 2417, 2421, 2680, 7057, 7059, 7215, 7229
\cs_generate_from_arg_count:NNnn 3999, 4352, 4496, 4673, 5434, 5470, 6733
\cs_generate_variant:Nn 56, 146, 176, 248, 331, 717, 1299, 2497, 2503, 2979, 4546, 4568, 5429, 6436
\cs_if_eq:NNTF . . 65, 73, 787, 795, 1169, 1174, 1657, 2710, 2724, 2727, 2742, 2745, 4127, 5683, 5685, 6816
\cs_if_exist:NTF 660, 1532, 1536, 1579, 1583, 1599, 1602, 1631, 1635, 1669, 1675, 1686, 1714, 1724, 1737, 1748, 1796, 1797, 1825, 1883, 1924, 1998, 1999, 2001, 2096, 3867, 4122, 4610, 5165, 5168, 5176, 5179, 6465, 6470, 6502, 6508, 6532, 6534, 6574, 6580, 8254, 8259, 8262, 8266, 8270, 8274, 8278, 8325, 8328, 8331, 8337, 8340, 8343, 8504, 8512, 8524, 8534, 8669, 8672, 8922, 9497, 9631, 9761, 10101, 10276
\cs_meaning:N . 2402, 4033, 6439, 6588
\cs_new:Nn 26, 34, 205, 306, 634, 748, 756, 873, 880, 1067, 1074, 1086, 1089, 1092, 1095, 1106, 1109, 1112, 1115, 1118, 1124, 1130, 1207, 1210, 1213, 1220, 1223, 1226, 1229, 1233, 1254, 1257, 1268, 1276, 1376, 1379, 1382, 1390, 1393, 1396, 1399, 1402, 1405, 1409, 1412, 1415, 1928, 1940, 2225, 2228, 2231, 2234, 2237, 2240, 2243, 2246, 2249, 2429, 2434, 2439, 3104, 3108, 3903, 4066, 4683, 4826, 4833, 4836, 4846, 5345, 5442, 5629, 5925, 5950, 6391, 6402, 6416, 6419, 6475, 6488, 6638, 6641, 6645, 6692, 7085, 7232, 7246, 7387, 8901, 9703, 9721
\cs_new:Npn 711, 712, 720, 721, 866, 869, 876, 975, 1121, 1127, 1260, 1936, 1965, 3252, 5678, 5929, 6089, 6481, 6494, 7134

\cs_new_nopar:Nn 359, 374
\cs_new_protected:Nn 3, 4, 42, 45, 53, 67, 73, 77, 87, 88, 97, 98, 103, 113, 141, 153, 169, 177, 192, 208, 215, 245, 249, 313, 324, 333, 346, 482, 495, 503, 516, 541, 556, 586, 598, 610, 615, 639, 684, 714, 725, 726, 764, 810, 819, 883, 905, 913, 926, 937, 954, 964, 979, 1133, 1139, 1157, 1168, 1187, 1240, 1244, 1282, 1301, 1328, 1341, 1353, 1361, 1464, 1473, 1548, 1556, 1560, 1573, 1597, 1612, 1667, 1713, 1775, 1794, 1814, 1824, 1835, 1862, 1872, 1881, 1909, 1942, 1954, 1989, 2015, 2029, 2039, 2106, 2115, 2141, 2277, 2314, 2323, 2340, 2346, 2355, 2370, 2382, 2398, 2408, 2446, 2465, 2469, 2477, 2491, 2494, 2498, 2516, 2520, 2554, 2564, 2577, 2587, 2592, 2597, 2611, 2617, 2632, 2664, 2675, 2684, 2695, 2702, 2709, 2715, 2737, 2754, 2762, 2771, 2780, 2789, 2812, 2827, 2842, 2869, 2888, 2912, 2918, 2938, 2959, 2974, 2993, 3031, 3040, 3050, 3061, 3069, 3082, 3112, 3133, 3158, 3166, 3174, 3184, 3209, 3224, 3229, 3234, 3240, 3244, 3271, 3294, 3311, 3331, 3335, 3345, 3365, 3367, 3372, 3415, 3482, 3535, 3551, 3579, 3591, 3610, 3679, 3742, 3773, 3862, 3870, 3883, 3897, 3907, 3918, 3969, 4016, 4032, 4039, 4045, 4069, 4086, 4095, 4107, 4114, 4140, 4155, 4168, 4179, 4181, 4193, 4234, 4258, 4295, 4308, 4321, 4371, 4379, 4385, 4395, 4443, 4457, 4470, 4528, 4547, 4576, 4585, 4603, 4607, 4612, 4631, 4652, 4671, 4685, 4701, 4706, 4712, 4766, 4789, 4796, 4800, 4864, 4889, 4915, 4937, 4955, 4963, 4973, 4990, 5001, 5021, 5031, 5074, 5082, 5106, 5120, 5134, 5142, 5151, 5161, 5189, 5222, 5241, 5252, 5299, 5307, 5361, 5422, 5430, 5446, 5452, 5489, 5494, 5505, 5510, 5521, 5524, 5541, 5544, 5559, 5562, 5577, 5581, 5591, 5594, 5605, 5611, 5627, 5639, 5643, 5703, 5714, 5746, 5778, 5831, 5886, 6045, 6066, 6139, 6147, 6150, 6170, 6231, 6263, 6312, 6324, 6351, 6371, 6376, 6408, 6423, 6438, 6445, 6463, 6500, 6514, 6548, 6557, 6568, 6585, 6594, 6660, 6670, 6675, 6696, 6715, 6728, 6763, 6768, 6774, 6815, 6886, 6953, 6973, 7062, 7077, 7089, 7095, 7122, 7180, 7214, 7250, 7264, 7283, 7293, 7299, 7351, 7366, 7379, 7448, 7462, 7472, 7508, 7564, 7602, 7615, 7638, 7656, 7662, 7706, 7715, 7721, 7727, 7744, 7749, 7754, 7870, 7903, 7930, 8126, 8248, 8324, 8336, 8375, 8413, 8450, 8455, 8461, 8471, 8548, 8720, 9008, 9108, 9171, 9219, 9269, 9433, 9529, 9740, 9757
\cs_new_protected:Npn 16, 148, 303, 327, 422, 694, 698, 701, 705, 706, 738, 1236, 1369, 1373, 1419, 1425, 1460, 1530, 1629, 1645, 1967, 1997, 2310, 2550, 2686, 2794, 2860, 2984, 2988, 3144, 3154, 3216, 3256, 3357, 3599, 4120, 4515, 4565, 4618, 4638, 4693, 4698, 4733, 4755, 4760, 4853, 4872, 4885, 5245, 5277, 5339, 5657, 5837, 5841, 5843, 5847, 5859, 5863, 5865, 5868, 5872, 5876, 5879, 5883, 6530, 6573, 6579, 6700, 6707, 6755, 6805, 6821, 6824, 6849, 6903, 6915, 6925, 7105, 7135, 7172, 7497, 7521, 7538, 7550, 7668, 7734, 7739, 8015, 8123, 9014, 9162, 9165
\cs_set:Nn 9384, 9403, 9406, 9425, 9429, 9454, 9532, 9533, 9539, 9540, 9549, 9568, 9571, 9590, 9594
\cs_set:Npe 4049
\cs_set:Npn 487, 524, 530, 601, 602, 885, 898, 904, 2642, 3274, 3288, 3401, 3406, 3467, 3472, 4000, 4071, 4073, 4076, 4089, 4142, 4353, 4397, 4497, 4673, 4892, 4898, 4943, 4967, 5435, 5471, 5780, 6283, 6284, 6288, 6289, 6290, 6551, 6587, 6607, 6718, 6733, 6830, 6855, 8020, 8021, 8022, 8230, 8553
\cs_set_eq:NN 54, 74, 315, 319, 1481, 1487, 1618, 1864, 1946, 3267, 3268, 3269, 4123, 4579, 4581, 4634, 5196, 5198, 5316, 5319, 5347, 5348, 5508, 5637, 5748, 5751, 6509, 6521, 6524, 6575, 6581, 6731, 6784, 6797, 8596
\cs_set_protected:Nn 1009, 8249, 8282, 8298, 10262
\cs_set_protected:Npn 69, 1497, 1507, 3863, 3864, 4964, 5835, 6601, 6606, 6612, 8326, 8329, 8338, 8341, 8597, 8620, 8624, 8635, 8639, 8749, 8752, 8755, 10155
\cs_set_protected:Npx 4939
\cs_undefine:N 351, 1040, 1798

\l_tmpa_cs 5780, 5798, 5807, 6283, 6293
\csname ... 349, 648, 2117, 2118, 2119, 2120, 2121, 2126, 2127, 2128, 2531, 3432, 3434, 3498, 3500, 7474, 7756
\ctikzinput ... *121*, 10355
\currentgrouplevel ... 325, 328, 334, 335, 337, 339, 341, 347, 349, 351, 353, 355, 7745, 7751, 7756
\CurrentOption ... 10, 8177, 8178, 8179, 8180, 8219, 8220, 8789, 9805
\Currentsectionlevel ... *84*, *139*, 1497
\currentsectionlevel ... *84*, *139*, 1497

D

\date ... 10276, 10277
\DeclareOption ... 10
\def ... *93*, 654, 658, 661, 663, 1462, 2454, 2480, 3782, 3839, 4280, 4287, 4436, 4589, 4590, 4946, 4951, 4959, 4968, 5766, 5858, 5888, 5948, 5965, 5973, 5987, 6001, 6027, 6037, 6042, 6058, 6210, 6235, 6255, 6958, 8347, 8349, 8414, 8415, 8416, 8420, 8423, 8480, 8588, 8605, 8613, 8649, 8650, 8652, 8654, 8656, 8658, 8659, 8661, 8663, 8665, 8668, 8669, 8671, 8672, 8675, 8677, 8679, 8695, 8747, 8896, 8899, 8900, 9372, 9377, 9852, 9859, 9862, 9868, 9875, 9879, 9880, 10114, 10182, 10184
\defemph ... *105*, *151*, 5868
\Definame ... *102*, *152*, 6005, 7308, 7330
\definame ... *23*, *33*, *39*, *101*, *102*, *105*, *152*, *304*, 6005, 7307, 7329
\Definames ... 6037
\definames ... 6027
\definiendum ... *23*, *33*, *39*, *101*, *102*, *105*, *152*, 6005, 7305, 7327
\definiens ... *41*, *50*, *58*, *101–103*, 7310, 7344, 7364, 7365
definiens ... *157*, 7344
\defnotation ... *33*, *39*, *102*, *152*, 6005, 7306, 7328
\detokenize ... 8860, 8863, 8866, 8869, 9815, 9818, 9821, 9824
dim commands:
  \dim_compare:nNnTF ... 9774
  \dim_eval:n ... 10158, 10159
  \dim_new:N ... 9099
  \dim_to_decimal:n ... 9748, 10179, 10207, 10208, 10209, 10216
  \dim_zero:N ... 9102, 9687
\dimexpr ... 8584
do commands:
  _do_comp:nNn ... *151*
  _do_comp:nnNn ... *151*, 5844, 5866, 5877, 5884, 5886, 6082, 6613
\dobracket ... *94*
\dobrackets ... *155*, 6894, 6903, 6926
\document-URI ... *125*, *144*
\doproblemqr 9862, 9879, 9903, 9969, 10083
\dowithbrackets ... *155*, 6925
\due ... *76*, *119*
\duration ... *77*, *119*

E

\edef ... 2119, 2120, 2121, 4708, 8431
\egroup ... 1867, 1949, 4368, 4512, 7691, 7724, 7839, 8458, 8546, 8563, 8669, 8672, 9147, 9207, 9255, 9307
\eject ... 9785, 10084
\ellipses ... *98*, 4653, 4655, 5754, 5792, 6834, 6835
\else ... 647, 659, 1438, 2033, 2055, 2070, 2698, 2705, 5873, 8747, 8820, 8843, 9020, 9131, 9146, 9501, 9635, 9773, 9863, 9869, 10336, 10341, 10346
\emph ... *17, 18*, *104*, 1440, 5883, 7684, 7701, 7863, 9152, 9212, 9260, 9312, 9515, 9649, 9793, 10005
\end ... *128*, *144*, 1443, 1557, 1626, 2032, 2187, 2202, 2222, 2581, 2727, 2745, 2765, 3077, 6873, 7206, 7861, 8292, 8299, 8351, 8360, 8433, 8458, 8473, 8518, 8532, 8540, 8553, 8555, 8568, 8569, 8570, 8571, 8572, 8573, 8594, 8622, 8626, 8629, 8693, 8701, 8742, 8743, 8972, 9062, 9081, 9139, 9144, 9199, 9204, 9247, 9252, 9299, 9304, 9355, 9368, 9389, 9420, 9554, 9668, 9673, 9678, 9683, 9785, 9793, 9960, 10000, 10066, 10084, 10139, 10181, 10358
\endcsname ... 349, 647, 648, 2117, 2118, 2119, 2120, 2121, 2126, 2127, 2128, 2531, 3432, 3434, 3498, 3500, 7474, 7756, 8747
\endgroup ... 48, 50, 2047, 2547
\endinput ... 1876, 1957
\endlist ... 7717
\endspfstepenv ... 7893, 7897, 7980, 7984
endstex commands:
  \endstex_annotate_env ... 2308, 9585
\ensuremath ... 5527, 5533
environments:
  assignment ... *76*, *119*, 10088
  blindfragment ... *84*, *139*, 1597

copymodule *145*, 3391
exnote 1, 9219
extstructure *98*, *156*, 6977
extstructure* *98*
frame 1, 1, 8375
gnote 1, 9267
hint 1, 9168
interpretmodule *145*, 3456
mathstructure *98*, *156*, 6942
mcb 1, 9382
nassertion 1, 1
ndefinition 1, 1
nexample 1, 1
note 1, 1
nparagraph 1, 1, 1, 1
nsproof 1, 1
problem 1
sassertion *101*, *157*, 7317
scb 9528
sdefinition *101*, *157*, 7312
sexample *101*, *157*, 7332
sfragment *84*, *139*, 1516
smodule *88*, *142*, 2253
solution 1, 9098
sparagraph *101*, *157*, 7335
spfblock *158*, 7957
spfsketchenv *158*, 7844
spfstepenv *158*, 7954
sproblem 8901
sproof *103*, *158*, 7562
stex_annotate_env *128*, 694
stex_env_node *128*, 694
subproblem 9015
subproof *158*, 7719
testheading *77*, *119*
titlefragment *139*, 1575
\envname *144*
\eq *31*
\eqstep *158*, 7586, 7970
\equal *30*
\errmessage 6459, 8829, 8852
\escapechar 88, 827, 9831
\everyeof 2676
\examdata 10291
examdata 10225
\examnumber 9859, 9870, 9871, 9873
\excursion *64*, *112*, 8681
\excursiongroup *64*, *112*, 8715
\excursionref *64*, *112*, 8686, 8695, 8697, 8710
exnote (env.) 1, 9219
\exnote 8907, 9266, 9414, 9579
exp commands:
\exp_after:wN 158, 483, 486, 517, 520, 867, 870, 877, 975, 999, 1090, 1107, 1110, 1113, 1116, 1119, 1122, 1128, 1208, 1221, 1224, 1230, 1237, 1262, 1264, 1377, 1397, 1400, 1403, 1413, 1512, 1922, 2229, 2235, 2241, 2247, 2341, 2342, 2403, 2404, 2405, 2411, 2415, 2419, 2420, 2480, 2678, 2697, 2698, 2699, 2704, 2705, 2706, 3127, 3194, 3195, 3196, 3432, 3434, 3498, 3500, 3545, 3546, 3547, 3583, 4003, 4356, 4500, 5023, 5096, 5097, 5102, 5260, 5270, 5272, 5315, 5448, 5455, 5456, 5457, 5465, 5466, 5467, 5480, 5662, 5664, 5769, 5770, 5771, 5784, 5997, 6053, 6077, 6477, 6490, 6838, 6866, 7056, 7059, 7128, 7215, 7229
\exp_args:Ne 58, 63, 69, 77, 81, 157, 265, 266, 274, 791, 799, 803, 888, 931, 1420, 1778, 1799, 1801, 1818, 1994, 2019, 2278, 2687, 2805, 3118, 3126, 3127, 3177, 3613, 3825, 3894, 3971, 4060, 4125, 4127, 4200, 4323, 4471, 4707, 4734, 4738, 4817, 4828, 4899, 4938, 4956, 5086, 5364, 5582, 5683, 5685, 5693, 5699, 5941, 5958, 6081, 6099, 6109, 6450, 6816, 7049, 7109, 7175, 7221, 7302, 7321, 7373, 7381, 7383, 7934, 8128, 8691, 8822, 8845, 9342, 9559, 9746, 9856, 10046, 10176
\exp_args:NNe 89, 474, 477, 864, 874, 881, 1866, 1948, 2377, 2481, 3042, 3556, 4162, 4207, 4219, 4919, 5035, 5049, 5144, 5163, 5396, 5399, 5669, 5720, 5810, 5961, 6034, 6297, 6393, 6404, 7071, 8737, 9009
\exp_args:Nne 304, 1034, 1311, 1453, 1565, 1779, 2088, 2109, 2566, 2845, 2975, 3004, 3219, 3783, 3816, 3884, 4059, 4288, 4672, 4958, 4975, 5023, 5090, 5092, 5145, 5433, 5469, 5479, 6236, 6256, 6424, 6589, 6619, 6964, 7064, 7067, 7070, 7186, 7272, 7381, 7383, 7417, 7453, 7458, 7465, 7468, 7603, 7619, 7851, 7875, 7909, 8937, 9037, 9395, 10175
\exp_args:NNNe 4209, 6697
\exp_args:NNne 4296, 4444, 5375, 5381
\exp_args:NNNo 986, 2828, 3566, 4296, 4444
\exp_args:NNno 143
\exp_args:Nnno 3137, 3139
\exp_args:NNNx 986
\exp_args:NNo 13, 42, 63, 83, 104, 107, 127,

131, 156, 222, 348, 735, 785, 989, 1311, 1839, 2157, 2168, 2373, 2845, 2877, 3563, 3569, 3573, 5172, 5173, 5183, 5184, 5797, 6344, 6357, 6745
\exp_args:Nno 335, 337, 372, 1874, 1904, 2455, 3121, 3998, 4162, 4207, 4219, 4351, 4495, 4517, 4695, 4699, 4908, 7728, 8924, 9094, 9853, 10103, 10148
\exp_args:NNx 90, 93, 426, 439, 8860, 8863, 8866, 8869, 9815, 9818, 9821, 9824
\exp_args:No 48, 257, 308, 348, 570, 576, 642, 653, 770, 899, 998, 1490, 1565, 1584, 1838, 1846, 1847, 1873, 1884, 1887, 1892, 1894, 1900, 1901, 1902, 1911, 1924, 1932, 1950, 1955, 1982, 1983, 2218, 2294, 2327, 2387, 2485, 2517, 2803, 2807, 3037, 3041, 3124, 3204, 3337, 3358, 3373, 3381, 3580, 3586, 3587, 3630, 3812, 3875, 3891, 4051, 4056, 4160, 4209, 4302, 4303, 4304, 4314, 4315, 4316, 4450, 4451, 4452, 4463, 4464, 4465, 4532, 4535, 4539, 4566, 4659, 4773, 4777, 4784, 4811, 4847, 4849, 4921, 4923, 4929, 5011, 5038, 5044, 5046, 5051, 5055, 5064, 5087, 5101, 5162, 5373, 5401, 5472, 5477, 5482, 5484, 5683, 5685, 5727, 5807, 5820, 6124, 6266, 6666, 6856, 6961, 6969, 7025, 7028, 7050, 7110, 7127, 7279, 7361, 8243, 8307, 8308, 8311, 8859, 8945, 9045, 9048, 9814, 9857, 10277
\exp_args:Nx 8376, 9434
\exp_not:N 94, 257, 308, 310, 898, 1049, 1050, 1055, 1056, 1087, 1096, 1097, 1102, 1214, 1215, 1383, 1384, 1784, 1868, 1950, 2425, 2676, 2696, 2704, 3005, 3620, 4530, 4531, 4534, 4538, 4542, 4599, 4675, 4678, 4772, 4848, 4922, 4928, 4948, 4981, 5010, 5013, 5037, 5043, 5052, 5057, 5066, 5101, 5324, 5330, 5443, 5721, 5726, 5805, 5811, 5815, 5817, 5818, 6265, 6672, 8254, 8259, 9510, 9512, 9644, 9646
\exp_not:n 257, 308, 899, 1565, 1787, 1788, 1894, 1900, 1930, 1932, 1950, 2419, 2430, 2485, 3844, 3891, 4052, 4302, 4303, 4304, 4314, 4315, 4316, 4450, 4451, 4452, 4463, 4464, 4465, 4532, 4535, 4539, 4541, 4543, 4683, 4773, 4811, 4842, 4847, 4849, 4921, 4923, 4925, 4929, 4932, 4942, 5011, 5014, 5038, 5040, 5044, 5046, 5051, 5055, 5064, 5101, 5456, 5472, 5477, 5482, 5484, 5706, 5727, 5764, 5769, 5807, 5819, 5820, 6266, 6306, 6307, 6666, 6838, 6866, 7751, 9515, 9649, 9717
\expandafter 48, 648, 2117, 2118, 2119, 2120, 2121, 2531, 5873, 7723, 7756, 8351, 8352
\ExplSyntaxOff 2555, 2603, 8135, 8873, 9828
\ExplSyntaxOn *143*, 2551, 2602, 8132, 8856, 9811
\extref *87*, 1987
extstructure (env.) *98*, *156*, 6977
\extstructure 7006, 7008
extstructure* (env.) *98*

F

\fbox 9745, 9761, 9774
\fi 650, 665, 1444, 2035, 2059, 2070, 2699, 2706, 4577, 4608, 5194, 5873, 6046, 6067, 8353, 8421, 8424, 8542, 8747, 8824, 8847, 9022, 9135, 9148, 9503, 9518, 9523, 9637, 9652, 9657, 9779, 9793, 9853, 9866, 9876, 10340, 10350, 10351
fiboxed *60*, *108*
file commands:
\file_if_exist:nTF 599, 616, 966
\fillinsol *117*, 8905, 9685
\fn *47*
\foldexpr *152*, 6096
\foo *16*, *47*, *90*
\fooname *16*
\footnote 9522, 9656, 9763
\footnotesize 10177
\foral *40*
\forall *40*, 8076
fp commands:
\fp_gadd:Nn 8968, 8969
\fp_new:N 8911, 8912
\fp_to_decimal:n 9055, 9058
frame (env.) 1, 1, 8375
\frameimage *62*, *111*, 8577
frameimages *60*, *108*
\frametitle 8489
\frontmatter *210*, 1714, 1715, 1716
\FTMLrule 8138, 8162, 8163
\fun *39*
\funspace *34*

G

\gdef 1466, 1483, 1488, 8681, 9846, 9848, 9856, 9857, 9873

\given *76*, *119*
\global 1462, 2118, 8117, 8118, 9163, 9166, 9875, 10110, 10112
gnote (env.) 1, 9267
\gnote 8908, 9318, 9415, 9580
gnotes *76*, *113*, *118*
group commands:
\group_begin: 18, 78, 87, 600, 740, 826, 2457, 2573, 2637, 2646, 2654, 2895, 2929, 2966, 3002, 3025, 3743, 3774, 3848, 4283, 4432, 4549, 4660, 4664, 4940, 4965, 5083, 5191, 5206, 5424, 5467, 5595, 5640, 5715, 5763, 5779, 5849, 5919, 5936, 5953, 5967, 5979, 5993, 6068, 6141, 6164, 6219, 6251, 6904, 7216, 7345, 7400, 7659, 7708, 7908, 7933, 8019, 9091, 9830, 10145
\group_end: 82, 85, 93, 606, 807, 830, 2471, 2582, 2650, 2657, 2660, 2900, 2935, 2971, 3004, 3078, 3751, 3788, 3853, 4293, 4440, 4610, 4623, 4649, 4661, 4665, 4695, 4856, 4912, 4949, 4970, 4976, 5049, 5102, 5220, 5257, 5268, 5284, 5295, 5303, 5336, 5427, 5448, 5480, 5485, 5603, 5640, 5720, 5771, 5810, 5853, 6064, 6086, 6143, 6164, 6222, 6261, 6910, 7225, 7347, 7411, 7665, 7717, 7927, 7951, 8120, 9095, 9833, 10149
\group_insert_after:N 340, 354

H

\halign 9764
\hbox 1475, 2891, 2962, 3526, 3541, 3836, 3866, 3971, 4323, 4486, 6595, 6990, 7146, 7575, 7642, 7768, 8006, 8145, 8457, 8669, 8672, 9459, 9490, 9732
hbox commands:
\hbox_set:Nn 6280, 9745
\hbox_unpack:N 1499, 1509, 3865, 9375, 9380
\HCode 660
\hfil 8012, 9764
\hfill 8012
hint (env.) 1, 9168
\hint 8906, 9218, 9413, 9578
hints *76*, *113*, *118*
\hline . . 10175, 10177, 10178, 10179, 10180
\homeworkdata 10297
homeworkdata 10225
\href 1924, 2001, 8691
\hrule 8006, 9459, 9490
\hskip 9745
\hspace 9497, 9631, 9777
\HTML *16*, *25*
\huge 9745, 9775, 9777
hwexam commands:
\l_hwexam_includeassignment_keys_tl . 10101, 10102, 10104, 10146
\hwexam_kw_* 9810
\c_hwexam_multiple_bool 9801
\c_hwexam_qrcode_bool . . . 9803, 9835
\hwexamheader 10182, 10221
\hwexamminutes 10184
\hyperlink 1999

I

\if 2697, 2704
if commands:
\if_charcode:w 9853
\IfBooleanTF . . 3439, 3505, 3728, 5352, 6213, 6248, 7774, 7887, 7975, 8578
\ifcsname 647, 8747
\ifdefempty 8736
\IfFileExists 6, 21, 1847, 1911, 2016, 2156, 2167, 2848, 2853, 2862
\IfInputref . . . *26, 27*, *85*, *140*, 2063, 8734
\ifinputref . . *26*, *85*, *140*, 2053, 2062, 2070
\ifintest 8800, 9793
\ifmmode 5873
\ifnotes *61*, *109*, 8234, 8371
\ifnum 2030
\ifSGvar *63*, *112*, 8755
\ifsolutions *114*, 8792, 9128, 9141, 9494, 9507, 9521, 9628, 9641, 9655, 9759
\ifstexhtml *27*, *83*, *128*, 670, 1438, 8820, 8843, 9020, 9863, 9869
\ifvmode 4577, 4608, 5194, 6046, 6067
\ifx 2117, 8350, 8419, 8422, 10333, 10334, 10342
\ignorespaces 2886, 2906, 2955, 4293, 4440, 8544, 9079
image *120*
\importmodule *35*, *43*, *48*, *96*, *97*, *124*, *128*, *143*, *145*, *222*, 2903, 3261
\includeassignment *76*, 10144
\includegraphics . . . *85*, *141*, 2185, 10326
\includeproblem *75*, *117*, 9083
\indent 4577, 4608, 5194, 6046, 6067
\infprec *92, 93*, *155*, 6344, 6882, 8041
\inline *157*
\inline* *102*
\inlineass *44*, *48*, *51*, *102*, *157*
\inlinedef *44*, *102*, *157*
\inlineex *102*, *157*
\input . . *26*, *82*, *125*, *143*, *182*, 44, 246, 669, 2080, 8857, 8861, 8864, 8867,

8870, 9812, 9816, 9819, 9822, 9825, 10182, 10335, 10338, 10344, 10348
\inputassignment *119*
\inputref *26, 27, 84, 85, 104, 124, 140*, 2005, 8596, 8597, 8705, 8737, 9094, 10148
\inputref* *61, 110*, 8596
\inputreffalse 2058, 2062
\inputreftrue 2041, 2056
\insertexamnumber 9868, 9875, 9880
\insertframenumber 8675, 8677
\insertshortauthor 8668
\insertshortdate 8679
\insertshorttitle 8671
\inset *39*
int commands:
\int_case:nn 1598
\int_case:nnTF 1531, 1630, 1668, 1687
\int_compare:nNnTF 328, 1568, 1606, 2086, 2108, 3557, 4769, 4780, 4873, 5286, 5682, 5692, 5698, 5757, 6271, 6305, 6862, 6892, 7247, 7285, 7530, 10162
\int_compare_p:nNn 7500, 7512, 7524, 7541, 7553
\int_decr:N 7531, 7559
\int_eval:n 347, 349, 351, 5378, 5384, 6891, 7097, 10110, 10176
\int_gincr:N 5363, 9053, 9111, 9174, 9222, 9272, 10156
\int_gset:Nn 5328
\int_gzero:N 5313, 8962
\int_incr:N 1533, 1537, 1569, 1600, 1603, 1607, 1632, 1636, 1672, 1678, 3960, 3961, 3962, 3963, 5658, 6775, 6861, 7505, 7517, 7528, 7545, 7557, 8151, 8293, 8636, 8640, 9332, 10023, 10163
\int_new:N 323, 1495, 1960, 3916, 4112, 5305, 5642, 6848, 6878, 8632, 9015, 9098, 9267, 10154
\int_set:Nn 1647, 1648, 1649, 1650, 1651, 1652, 1654, 3348, 3831, 3925, 3929, 3933, 3937, 3941, 3945, 3949, 3953, 3957, 4144, 4184, 4245, 4273, 4399, 5600, 6748, 6856, 6863, 6885, 7498, 7510, 7522, 7539, 7551, 10248
\int_set_eq:NN 325
\int_step_function:nN . . . 4676, 5436
\int_step_inline:nn 3909, 6336, 6343, 6363, 6683
\int_use:N 1551, 1614, 1658, 1661, 1682, 3887, 4300, 4312, 4448, 4461, 4519, 5328, 5364, 5705, 5723, 5813, 6412, 6517, 6748, 6882, 6883, 7274, 7745, 7751, 7756, 8154, 8286, 8637, 8641, 9068, 9113, 9176, 9224, 9274, 9334, 9948, 9949, 9967, 9968, 9991, 10025, 10175, 10282
\int_zero:N 3919, 3920, 5649, 6730, 6859, 8144, 8634
\c_max_int 6882, 6883
\l_tmpa_int 6730, 6733, 6748, 6775, 6859, 6861, 6862, 6863, 6867, 7498, 7501, 7504, 7505, 7510, 7513, 7516, 7517, 7522, 7525, 7528, 7530, 7531, 7533, 7534, 7539, 7542, 7545, 7547, 7551, 7554, 7557, 7559, 7560, 8144, 8151, 8154
intarray commands:
\intarray_gset:Nnn 7533, 7547, 7560, 7566
\intarray_gzero:N 7565
\intarray_item:Nn 7501, 7504, 7513, 7516, 7525, 7534, 7542, 7554
\intarray_new:Nn 7495
\interpretmod *145*, 3518, 3520, 3522
interpretmodule (env.) *145*, 3456
\interpretmodule 3492, 3494
\intestfalse 8804
\intesttrue 8802
ior commands:
\ior_close:N 623, 629, 831, 997
\ior_map_inline:Nn 985
\ior_new:N 971
\ior_open:Nn 617, 625, 980
\ior_open:NnTF 825
\ior_str_get:NN 828
\ior_str_map_inline:Nn 619, 626
\g_tmpa_ior 617, 619, 623, 625, 626, 629, 825, 828, 831
iow commands:
\iow_close:N . . . 612, 622, 1762, 10076
\iow_new:N 585, 1760, 10073
\iow_now:Nn 620, 627, 715, 1780, 1783, 8131, 9009, 10075
\iow_open:Nn . . . 611, 618, 1761, 10074
\g_tmpa_iow 618, 620, 622
\isassociative *43*
\iscommutative *43*
\item 7712, 9068, 9342, 9361, 9389, 9554, 9948, 9967, 10044
\itshape 8016

J

\jobname *79*, 235, 591, 611, 616, 617, 618, 625, 630, 1761, 10074
\join *56*

K

keys commands:
\l_keys_choice_tl 3702
\keys_define:nn 29, 372, 8171, 8213, 8715, 8771, 9799, 10314
\l_keys_key_str 6187, 6190, 6935, 6938
\l_keys_key_tl . . . 13, 735, 6188, 6936
\keys_set:nn *126*, 376, 8724
keyval commands:
\keyval_parse:NNn 4792

L

\label *86*, 1778, 1799, 1801, 8482
\labelsep 8427
\labelwidth 8428
\langle 8060
\Large 8231, 8606, 9425, 9426, 9429, 9430, 9590, 9591, 9594, 9595, 9871, 9873, 10126
\lastslide 8624, 8639
\LaTeX *23*
\latex *23*
\ldots 8065, 8112, 8114, 8116
\leaders 8608
\leavevmode 8502
\leftmargin 8429
\let 1715, 1716, 1725, 1726, 2042, 2118, 2639, 2647, 2655, 2677, 2682, 2949, 3866, 4588, 4609, 5924, 5947, 5964, 6059, 7121, 8117, 8118, 8133, 8134, 8300, 8615, 9531, 9535, 9538, 9542
\libinput *19*, *82*, *140*, 2073, 2143
\libusepackage . . . *82, 83*, *140*, 2084, 8367
\libusetheme *61*, 8366
\libusetikzlibrary *82*, *121*, *140*, 2095
\linewidth . . . 9151, 9158, 9211, 9216, 9259, 9264, 9311, 9316, 10004, 10017
\list 7708
\LoadClass 16, 8199, 8201
\long 8650, 8659
\lstinputlisting *85*, *141*, 2201
\lstinputmhlisting *85*, *141*, 2189

M

\macro *123, 124*, *126*, *135*, *144*
\macroname *31*, *127*
\magma *46*
\maincomp *31–34*, *47*, *92*, *100*, *105*, 4590, 4609, 4707, 4708, 4938, 4946, 4951, 4956, 4959, 4968, 5302, 5858, 6265, 6274, 6284, 6285, 6306, 6612, 6672, 7115, 8084, 8088, 8091, 8093, 8108
\mainmatter 1737, 1738, 1748, 1749
\makeatletter 2600, 8856, 9811, 10221
\makeatother 2601, 8873, 9828, 10221
\makebox 9496, 9630
\maketitle *139*, 1487, 1488, 1579, 1588, 8299, 8300
\marginnote 8417, 9864
\marginpar *152*, 6140, 8821, 8844, 8978, 8983, 9071, 9076, 9374, 9379, 9887, 9892, 9906, 9911, 9952, 9957, 9972, 9977
\mathbb 8096
\mathbin *32*, 8039, 8043, 8052, 8075, 8083, 8087
\mathclose *32*, 6911, 8041, 8050, 8054, 8060, 8062, 8070, 8075, 8079
\mathhub *80*, *132*, 32, 822
\mathop *32*, 8077, 8093
\mathopen *32*, 6906, 8041, 8050, 8054, 8060, 8062, 8069, 8075, 8079
\mathord *32*
\mathpunct *32*, 5632, 6687, 8069
\mathrel . . . *31, 32*, 8044, 8084, 8088, 8091
\mathrm . 5805, 5807, 6672, 8046, 8069, 8108
mathstructure (env.) *98*, *156*, 6942
\mathstructure 6951, 6952
\mathtt 8049, 8050, 8099, 8102, 8106
mcb (env.) 1, 9382
\mcb 8903, 9410, 9432, 9575
\mcc *70*, *115*, 9416, 9453
\meaning 1920, 2531, 3084, 3087, 3090, 4620, 5100, 5463, 5475, 5476, 6310, 6615, 6770
\medskip 2047, 8457, 8472, 8496
\meet *56*
\message 28, 1828, 2703, 8192, 8195, 8357, 10071
\mhframeimage *62*, *111*
\mhgraphics *85*, *141*, 2173, 8590, 8592
\mhinput *85*, *140*, 2050
\mhtikzinput *121*, *141*, 2204
\min *77*, *119*
\min 9377
min *76*, *113*, *118*
\mmlarg *129*, 697
\mmlintent *129*, 697
\mname *89*, *98*
mode commands:
\mode_if_math:TF 3541, 3866, 3867, 4605, 4610, 9731
\mode_if_vertical:TF 1499, 1509, 3865
\mode_leave_vertical: 3198
\MSC 8123
msg commands:
\msg_error:nn 3009, 3190, 5607, 7357, 7406, 8920

\msg_error:nnn 58, 469, 780, 2097, 2143, 2527, 2774, 2879, 3643, 3673, 4116, 4375, 5070, 5148, 6818, 7125, 7130, 9707
\msg_error:nnnn . . 70, 2078, 2135, 3247, 3323, 3965, 4557, 5202, 5218, 5389, 5416, 6062, 6084, 6543, 7091
\msg_fatal:nn 916, 10264, 10271, 10274
\msg_fatal:nnnn 920, 2092, 2111
\msg_none:nn 117
\msg_redirect_module:nnn 132
\msg_redirect_name:nnn 136
\msg_set:nnn 114, 9698, 10231, 10235, 10239
\msg_warning:nn 844
\msg_warning:nnnn 636
\mult *37*, *93*, *94*
\multicolumn 10176
\multiple *76*, *118*

N

nassertion (env.) 1, 1
\Nat *34*
ndefinition (env.) 1, 1
\neginfprec *37*, *92*, *93*, *155*, 4621, 5280, 5291, 6314, 6317, 6326, 6329, 6342, 6882
\newcommand 483, 517, 520, 1975, 2064, 2069, 2073, 2084, 2095, 2130, 2196, 2202, 2212, 2222, 7992, 7996, 8000, 8348, 8417, 8498, 8599, 8603, 8686, 8697, 8704, 8707, 8729, 8731, 8809, 8832, 8894, 9329, 9463, 9505, 9601, 9639, 9665, 9670, 9675, 9680, 9729, 9784, 9787, 9788, 9789, 9790, 9791, 9792, 10161, 10291, 10297, 10301, 10326, 10331, 10355
\newcounter 8202, 8203, 8204, 8205, 8332, 8344, 8893, 10099
\NewDocumentCommand *127*, 486, 1437, 1793, 1804, 1987, 1992, 2005, 2050, 2539, 5205, 5351, 5918, 5935, 5952, 5966, 5978, 5992, 6015, 6021, 6031, 6041, 6097, 6108, 7215, 7344, 7390, 7399, 7414, 7871, 7970, 7988, 8138, 8366, 8577, 9090, 10144
\NewDocumentEnironment *127*
\NewDocumentEnvironment 523, 1522, 1577, 1621, 7957, 8618, 8633
\newenvironment 1734, 1745, 7844, 7954, 8550, 8552, 10204
\newif 648, 672, 2062, 8234, 8792, 8800, 10211
\newlabel 8133, 8134
\newlength 8369, 8370, 8373
\newline 9151, 9158, 9211, 9216, 9259, 9264, 9311, 9316, 10004, 10017
\newpage 8479, 9791, 10223
\newsavebox 8520
nexample (env.) 1, 1
\nextslide 8620, 8635
\ninputref 8597, 8599, 8603
\nobreak 8012
\noexpand 2697
\noindent 1552, 2018, 2031, 7193, 7593, 7800, 7863, 7932, 8472, 8492, 8500, 8518, 8943, 8974, 9043, 9152, 9212, 9260, 9312, 9884, 9902, 10005
\nointerlineskip 8610
\normalmarginpar 9865
\notation *31*, *32*, *37*, *90*, *92*, *97*, *143*, *153*, 3260, 6203, 8039, 8042, 8043, 8044, 8059, 8061, 8074, 8076, 8077, 8082, 8086, 8096, 8099
note (env.) 1, 1
notes *60*, *76*, *108*, *113*, *118*
\notesfalse 8246
notesslides commands:
\c_notesslides_notes_bool 8173, 8174, 8187, 8190, 8198, 8214, 8215, 8227, 8235, 8363, 8543, 8549, 8583, 8598, 8648, 8698, 8708
\c_notesslides_sectocframes_bool 8216, 8319
\c_notesslides_topsect_str 8217, 8240, 8243, 8304, 8307, 8308, 8311
notesslides internal commands:
\c__notesslides_class_str 8169, 8172, 8178, 8199
__notesslides_define_chapter: 8309, 8312, 8324
__notesslides_define_part: 8313, 8336
__notesslides_do_label:n 8249, 8285
__notesslides_do_sectocframes: 8248, 8320
__notesslides_do_yes_param:Nn 8375, 8394, 8397, 8400, 8403, 8406, 8409
\c__notesslides_docopt_str . . . 8175
\c__notesslides_document_str 8347, 8350
__notesslides_eat: . 8553, 8557, 8560
__notesslides_excursion_args:n 8720, 8732
\l__notesslides_excursion_id_str 8716, 8722
\l__notesslides_excursion_intro_tl 8717, 8721, 8736, 8738

\l__notesslides_excursion_mhrepos_str 8718, 8723, 8737
\l__notesslides_frame_allowdisplaybreaks_bool 8386, 8397
\l__notesslides_frame_allowframebreaks_bool 8385, 8394
__notesslides_frame_box_begin: 8450, 8461, 8485
__notesslides_frame_box_end: 8455, 8471, 8487
\l__notesslides_frame_fragile_bool 8387, 8400
\l__notesslides_frame_label_str 8384, 8392, 8481, 8482
\l__notesslides_frame_shrink_bool 8388, 8403
\l__notesslides_frame_squeeze_bool 8389, 8406
\l__notesslides_frame_t_bool 8390, 8409
__notesslides_inputref: 8596, 8597, 8600
__notesslides_notes_env:nnnn 8548, 8568, 8569, 8570, 8571, 8572, 8573
\l__notesslides_num 8252, 8254, 8257, 8259, 8262, 8263, 8266, 8267, 8270, 8271, 8274, 8275, 8278, 8279, 8286
__notesslides_setup_itemize: 8413, 8484
\l__notesslides_slideshow_counter_int 8632, 8634, 8636, 8637, 8640, 8641
\l__notesslides_tmp 8756, 8761
\l__notesslides_uri 8690, 8691
\g__notesslides_variables_prop 8748, 8750, 8753, 8756
\notesslidesfont 8453, 8469, 8615
\notesslidesfooter 8457, 8472, 8613
\notesslidestitleemph .. 8492, 8494, 8605
\notestrue 8236
nparagraph (env.) 1, 1, 1, 1
nsproof (env.) 1, 1
\null 8012
\number *76*, *119*
\numberline 8284
\numberproblemsin 8893

O

\objective 8832
\OMDoc *25*
\omdoc *16*
\only 8637, 8641
\oplus 8051, 8052

P

\PackageError 8757
\pagebreak 8991, 10142
\pagenumbering ... 1721, 1731, 1742, 1753
\par 47, 1550, 1554, 2031, 7564, 7593, 7658, 7664, 7707, 7716, 7796, 7800, 7845, 7850, 7859, 7925, 8012, 8457, 8472, 8492, 8500, 8505, 8513, 8518, 8525, 8535, 8974, 8987, 8990, 9021, 9151, 9158, 9211, 9216, 9259, 9264, 9311, 9316, 9362, 9365, 9422, 9543, 9587, 9884, 9895, 9898, 9902, 9932, 9940, 10004, 10017, 10060, 10063, 10125, 10130, 10140, 10142
\paragraph *26*, *84*
\parent *125*
\parsep 7710, 9066, 9340, 9359, 9387, 9552, 9946, 9965, 10042
\part *26*, *84*, 8340, 8341
\partname 8254, 8337, 8338
\parttitlename 8254
\PassOptionsToClass 8177, 8178
\PassOptionsToPackage 10, 8179, 8180, 8191, 8194, 8219, 8220, 8237, 8789, 9805
\pause 8498
\PDF *16*, *25*
\pdfbookmark 8286
peek commands:
\peek_charcode:NTF 1362, 3592, 4614, 4627, 4687, 4689, 4716, 4722, 4865, 4876, 5228, 5235
\peek_charcode_remove:NTF 4613, 4686, 4721, 4974, 5224, 5226, 5233, 5243, 5246, 5447, 5490
\phantom 9775
\plus *35–38*, *92–94*
\precondition 8809
\prematurestop *63*, *111*, 8347
\premise *103*, *157*, 7326, 7414
prg commands:
\prg_new_conditional:Nnn 57, 62, 263, 272, 680, 2504, 2508, 2512, 2626, 4055, 4058, 4229, 5395, 5681, 5691, 5697, 6118, 6129, 6133
\prg_new_protected_conditional:Nnn 279
\prg_return_false: 60, 65, 264, 267, 273, 275, 295, 299, 682, 2506, 2510, 2514, 2627, 4056, 4067, 5413, 5417, 5687, 5688, 5689, 5694, 5695, 5700, 5701, 6119, 6134

\prg_return_true: 60, 65, 265, 267, 275, 299, 682, 2506, 2510, 2514, 2627, 4056, 4067, 5411, 5687, 5694, 5700, 6130
\printexcursion *64*, *112*
\printexcursions . 8681, 8705, 8733, 8741
problem (env.) 1
problem commands:
\g_problem_id_counter 9098, 9111, 9113, 9174, 9176, 9222, 9224, 9272, 9274
\l_problem_inputproblem_keys_tl 8922, 8923, 8925, 9092
\problem_mcc_box_default_tl 9488, 9496, 9500, 9502
\problem_mcc_box_tl 9389, 9393, 9425, 9429, 9457, 9493
\problem_scc_box_default_tl 9626, 9630, 9634, 9636
\problem_scc_box_tl 9554, 9566, 9590, 9594, 9600, 9627
\g_problem_total_min_fp . . 8912, 8969
\g_problem_total_pts_fp . . 8911, 8968
\problemheader . . . 8974, 8994, 9884, 9902
problems internal commands:
__problems_activate_macros: 8901, 8966, 9023
\l__problems_anscls_int 9267, 9332, 9334, 10023, 10025
\c__problems_boxed_bool 8785
__problems_do_yes_param:Nn . . 9433
__problems_exnote_start:n 9219, 9236, 9239
\c__problems_false_tl . . . 9696, 9706
\l__problems_fillin_solution_tl . . . 9686, 9710, 9722, 9751, 9762, 9769
__problems_fillinsol:nn 9732, 9734, 9740, 9757
__problems_gnote_start:n 9269, 9288, 9291
\c__problems_gnotes_bool 8775, 9290, 9301, 9845, 10072
\l__problems_has_min_bool . 8916, 8960, 8961, 9057, 9075, 9956, 9976
\l__problems_has_pts_bool 8915, 8957, 8958, 9054, 9070, 9951, 9971, 9993
__problems_hint_start:n 9171, 9188, 9191
\c__problems_hints_bool 8777, 9190, 9201
\l__problems_in_problem_bool 8914, 8917, 8919, 8953, 9019, 9024
__problems_maybe_inline:n 9394, 9420, 9529, 9558, 9585
__problems_maybe_newline: 9454, 9533, 9540
__problems_mccline:n 9384, 9403, 9406, 9425, 9429, 9465, 9520
\c__problems_min_bool 8783, 8981, 9074, 9378, 9890, 9909, 9955, 9975
\l__problems_min_tl 8955, 8959, 8969, 8982, 8983, 9010, 9026, 9058, 9891, 9892, 9910, 9911
\c__problems_notes_bool 8773, 9238, 9249
__problems_oldpar: 9531, 9535, 9538, 9542
__problems_parse_fillin_-arg:nnnn 9690, 9691, 9692, 9703, 9721, 9743
\l__problems_path_seq 8934, 8935, 9029, 9030, 9034, 9035
\l__problems_prob_imports_tl . 8890
\l__problems_prob_refnum_int . 8891
\c__problems_pts_bool 8781, 8976, 9069, 9373, 9885, 9904, 9950, 9970
\l__problems_pts_tl . . 8954, 8956, 8968, 8977, 8978, 9010, 9025, 9055, 9886, 9887, 9905, 9906, 9927, 9929
__problems_record_problem: 8970, 9008
__problems_sccline:n 9549, 9568, 9571, 9590, 9594, 9603, 9654
__problems_solution_start:n 9108, 9126, 9129
\c__problems_solutions_bool 8779, 8795
\g__problems_subproblem_int 8962, 9015, 9053, 9068, 9948, 9949, 9967, 9968, 9991
\c__problems_test_bool 8787, 8801, 8991, 9785, 9787, 9791, 9861, 10081
\c__problems_true_tl 9695, 9705, 9715
\ProcessKeysOptions 43, 8183, 8223, 8794, 9808, 10320
\ProcessOptions 11
\prod 8077
prop commands:
\prop_clear:N 5075
\prop_const_from_keyval:Nn . 426, 439
\prop_gclear:N 2331, 2332, 2333, 2391, 2392, 2393, 2456, 5311
\prop_get:NnNTF 465, 5396, 8756
\prop_gput:Nnn 2481, 2975, 4024, 5340, 5375, 5381, 5399, 6429, 7381, 8750, 10157

\prop_gset_from_keyval:Nn 2449, 2450, 5324
\prop_if_exist:NTF ... 914, 1141, 5323
\prop_if_in:NnTF 222, 1172, 1177, 5032, 5107, 5121, 5143
\prop_item:Nn 3220, 4059, 5024, 5067, 5146, 7072, 7383, 8753
\prop_map_break:n 3290, 4074, 4077, 4182, 4387, 4390, 6451
\prop_map_function:NN 2423
\prop_map_inline:Nn 2451, 3052, 3298, 4034, 4080, 4098, 4101, 4201, 4214, 4220, 4380, 4904, 6440, 6448, 6560, 6563, 7045, 10169
\prop_new:N 4256, 8748, 10151
\prop_put:Nnn 4297, 4445, 5109, 5123, 5152, 7067, 7078, 10164
\prop_remove:Nn 3686
\prop_set_eq:NN 1043
\prop_to_keyval:N 2403, 2404, 2405, 2411, 2415, 5325
\protect 8284, 8759
\protected .. 1488, 5948, 5965, 6027, 6037
\providecommand .. 2180, 2187, 8355, 10325
\ProvidesExplClass 5, 8166
\ProvidesExplPackage 20, 8210, 8768, 9796, 10311
\pts 9372
pts *76*, *113*, *118*

Q

\qrcode 9864
\qrid 9884, 9902, 9949, 9968, 10082
\qrjson 9846, 9848, 9850, 9851, 9916, 9921, 9928, 9936, 9939, 9981, 9983, 9988, 9994, 9999, 10008, 10014, 10029, 10033
\qrschema 9859, 9864
\quad 8263, 8267, 8271, 8275, 8279, 9730, 9736, 10133, 10136
quark commands:
\q_no_value 1169, 1174
\quark_new:N 2463, 2673
quark internal commands:
\q__stex_smsmode_break 2673, 2676, 2710
\quizdata 10301
quizdata 10225

R

\raisebox 9498, 9632
\rangle 8060
\realize *57, 58*
\ref *86*, 1831
\refstepcounter 8930, 10112
\relax 19, 20, 46, 604, 605, 741, 742, 1716, 1726, 1945, 2017, 2031, 2117, 2458, 2459, 2680, 2682, 2696, 2697, 2704, 3866, 4577, 4608, 4609, 5194, 6046, 6067, 8300, 8584, 8586
\renamedecl *58*, *146*, 3266, 3602
\renewcommand 8364, 8479, 8489, 10020, 10080
\renewenvironment 8418, 8476, 8499, 8521, 8544, 8546
\reqpts *77*, *119*
\requiremodule ... *95, 96*, *145*, 2951, 3262
\RequirePackage 4, 13, 19, 24, 25, 474, 477, 8167, 8185, 8207, 8211, 8225, 8238, 8239, 8575, 8769, 8807, 9797, 9809, 9836, 9837, 10312, 10323, 10328, 10329
\resizebox ... 10174, 10337, 10343, 10347
\reversemarginpar 9864
\rhd 8420, 8423
\Rightarrow 7969, 8088
\rightmargin 7711, 9067, 9341, 9360, 9388, 9553, 9947, 9966, 10043
\rule 9151, 9158, 9211, 9216, 9259, 9264, 9311, 9316, 10004, 10017
rustex commands:
\rustex_direct_HTML:n 8506, 8514, 8526, 8536
\rustex_if:TF 8504, 8505, 8512, 8513, 8524, 8525, 8534, 8535
\rustexBREAK 2604

S

sassertion (env.) *101*, *157*, 7317
scb (env.) 9528
\scb 8904, 9411, 9576, 9597
\scc 9581, 9598
\scriptsize 8417
\scriptstyle 8423
sdefinition (env.) *101*, *157*, 7312
\section *25, 26*, *84*, *139*
\sectiontitleemph 8230, 8289
sectocframes *60*, *108*
seq commands:
\seq_clear:N 142, 180, 456, 846, 849, 2043, 2145, 2640, 3032, 3033, 3034, 3035, 3051, 3175, 3272, 4087, 4802, 5076, 5077, 5085, 5648, 5716, 5781, 5790, 6269, 6549, 6682, 6832, 6978, 7013, 7138, 7173, 8140
\seq_count:N .. 2086, 2108, 3557, 7285
\seq_gclear:N 2633, 2634, 5312
\seq_get_right:NN ... 8935, 9030, 9035
\seq_gpush:Nn 64, 786

\seq_gput_right:Nn .. 2598, 2618, 5341
\seq_gset_eq:NN 244, 252, 3071, 3072, 3073, 3074
\seq_gset_split:Nnn 5330
\seq_if_empty:NTF 178, 196, 264, 273, 299, 915, 1162, 1247, 1288, 2077, 2134, 4159, 7235, 7605, 7621, 7853, 7911, 7936, 8149
\seq_if_empty_p:N 286, 287, 2153
\seq_if_exist:NTF 5329
\seq_if_in:NnTF 63, 785, 2157, 2168, 2521, 2636, 2755, 2763, 3053, 3246, 3296, 4096, 4916, 5035, 5163, 6558
\seq_item:Nn 265, 266, 274, 2087, 2109, 5397, 6379, 6385, 7286
\seq_map_break: 929, 1061, 4074
\seq_map_break:n 3160, 3168, 3346, 3374, 3382, 4077, 4182
\seq_map_function:NN 182, 186, 2080, 2137, 3101, 4814, 7854, 7912, 7937
\seq_map_indexed_function:NN 7236, 7606, 7622
\seq_map_inline:Nn 851, 927, 1029, 1033, 1194, 2148, 3093, 3114, 3146, 3235, 3315, 3319, 3376, 3384, 4079, 4198, 5791, 6395, 6833, 6860, 6988, 7030, 7144, 7301, 7320, 7572, 7765, 8150
\seq_new:N 24, 746, 2252, 2595, 2615, 2630
\seq_pop:NN 179
\seq_pop_left:NN 163, 292, 293, 1033, 1195, 1197, 2161, 3559, 3561, 3568, 3572
\seq_pop_left:NNTF 988, 6354, 6356, 6364
\seq_pop_right:NN 197, 210, 212, 217, 219, 221, 948, 1159, 1161, 1173, 1246, 1286, 1289, 2149, 2829, 4158, 4195, 4805, 6698
\seq_push:Nn 185, 847, 2638, 3054, 3620
\seq_put_left:Nn 164, 183, 2396, 4097, 5108, 5122, 6559
\seq_put_right:Nn 200, 213, 223, 226, 235, 467, 858, 1178, 2158, 2162, 2169, 2336, 2523, 3042, 3063, 3178, 3195, 3297, 3546, 5144, 5704, 5721, 5755, 5762, 5787, 5793, 5797, 5811, 6337, 6344, 6365, 6367, 6684, 6835, 6837, 6983, 7018, 7141, 7176, 7266
\seq_reverse:N 4806
\seq_set_eq:NN 188, 209, 216, 234, 280, 281, 1032, 1158, 1193, 5802, 6842, 6870
\seq_set_split:Nnn 143, 211, 218, 850, 939, 940, 987, 1030, 1160, 1192, 1245, 1285, 2146, 2147, 2828, 3556, 3566, 4157, 4194, 4804, 6353, 6357, 6393, 6697, 8142
\seq_use:Nn 206, 213, 226, 472, 475, 478, 864, 921, 990, 1184, 1251, 2076, 2133, 2151, 2830, 3564, 3570, 3574, 4163, 4196, 5331, 5631, 6599, 6687, 6809, 6843
\l_tmpa_seq 456, 467, 472, 475, 478, 845, 847, 850, 851, 939, 943, 1030, 1031, 1033, 1160, 1161, 1162, 1173, 1178, 1184, 1192, 1194, 1285, 1286, 1288, 1289, 1342, 1347, 2828, 2829, 2830, 4194, 4195, 4196, 4802, 4804, 4805, 4806, 4814, 6393, 6395, 6682, 6684, 6687, 6697, 6698, 6832, 6835, 6837, 6842, 6870, 8140, 8142, 8149, 8150
\l_tmpb_seq 1032, 1033, 1034, 1193, 1195, 1197, 1198
\seqmap *98*, *154*, *295*, 5693, 6821
\setbox ... *144*, 1863, 1944, 7598, 7811, 8546, 8561, 9134, 9194, 9242, 9293
\setcounter 10110, 10113
\setkeys ... 2184, 2200, 2217, 8588, 10332
\setlength 7709, 7710, 7711, 8369, 8370, 8374, 8427, 8428, 8429, 9065, 9066, 9067, 9339, 9340, 9341, 9358, 9359, 9360, 9386, 9387, 9388, 9551, 9552, 9553, 9945, 9946, 9947, 9964, 9965, 9966, 10041, 10042, 10043
\setlicensing *62*, *110*
\setmetatheory *143*, 2539, 2606
\setnotation *32*, *93*, *153*, 6514
\setsectionlevel *26*, *84*, *139*, 1645, 8241, 8243, 8305, 8307
\setSGvar *63*, *112*, 8749, 8759
\setslidelogo *62*, *110*
\setsource *62*, *110*
sexample (env.) *101*, *157*, 7332
\sf 8606
\sffamily 8615
sfragment (env.) *84*, *139*, 1516
sfragment commands:
_sfragment_do_level:nn 1532, 1536, 1540, 1541, 1542, 1543, 1544, 1548, 1560, 8282
_sfragment_end: 1527, 1556, 1573, 1595
\shorttitle 1583, 1584
\skipfragment *84*, *139*, 1629
\slideframewidth . 8373, 8374, 8463, 8584
\slideheight 8370

slides *60*, *108*
\slidewidth 8369, 8466, 8502, 8584, 8586, 8590
\smacro *94, 95*
\small 8016
\smallskip 8012, 8978, 8983, 9071, 9076, 9151, 9211, 9259, 9311, 9887, 9892, 9906, 9911, 9952, 9957, 9972, 9977, 10004, 10130
smodule (env.) *88*, *142*, 2253
\smodule 3263
\Sn *19*, *91*, 5964
\sn *19*, *23*, *91*, *152*, 1432, 5925
\Sn\Sns *152*, 5925
\Sns *19*, *91*, 5965
\sns *19*, *91*, *152*, 5925
\solution *68*
solution (env.) 1, 9098
\solution 8902, 9161, 9412, 9577
solutions *76*, *113*, *118*
\solutionsfalse 8798, 9166
\solutionstrue 8796, 9163
\source *124*
sparagraph (env.) *101*, *157*, 7335
\spfarg *159*, 7592, 8000
spfblock (env.) *158*, 7957
\spfblock 7589, 7963
\spfby *158*, 7591, 7996
\spfescape 7588, 7668, 7671
\spfjust *158*, 7590, 7992
\spfsketch *158*, 7845
spfsketchenv (env.) *158*, 7844
\spfsketchenvautorefname 7863
\spfstep *158*, 7583, 7967
\spfstepautorefname 7719, 7955
spfstepenv (env.) *158*, 7954
\spfstepenv 7879, 7972
\spfstepenvautorefname 7955
sproblem (env.) 8901
\sproblemautorefname 9000
sproof (env.) *103*, *158*, 7562
\sproofautorefname 7684
\sproofend *159*, 7686, 7703, 8004
\sqcap 8091
\square 8005, 9458, 9489
\sr *19*, *23*, *90*, *152*, 5918
\sref *86, 87*, *101*, 1804, 8700
\sreflabel *86*, *88*, 1775
\srefsetin *86*, 1975
\srefsym *91*, 1992
\srefsymuri *91*, 1994, 1997
\startsolutions *114*, 9162
\stepcounter 1631, 1635, 1639, 1640, 1641, 1642, 1643, 8283, 8478
\sTeX *14*, *16*, *83*
\stex *16*, *23*, *83*
stex commands:
\l_stex_aB_args_seq *294*, 5631, 5648, 5704, 5716, 5721, 5755, 5762, 5781, 5787, 5791, 5802, 5811, 6809, 6833, 6842, 6843, 6860, 6870
\stex_activate_module:N *143*, 2362, 2380, 2516, 2516, 2777, 2784
\stex_activate_module:n *143*, 2348, 2351, 2516, 2517, 2520, 2920, 5086, 6996, 7038, 7040, 7050, 7065, 7149
_stex_activate_notations: 2530, 6438
_stex_activate_symbols: . 2529, 4032
_stex_add_definiens:nn *157*, 3268, 7361, 7366, 7366, 7575, 7577, 7768, 7770
\stex_add_morphism:nnnn *145*, 2922, 2974, 2974, 2979, 3097, 6993, 7035
\stex_add_notation:nnnnn *153*, 3126, 6409, 6416, 6423, 6436, 6516
\stex_add_symbol:nnnnnnN . . *147*, 4016
\stex_add_symbol:nnnnnnnN *148*, 3150, 3155, 3816, 3884, 4016, 6964, 7272
\l_stex_all_module_seq 5085
\l_stex_all_modules_seq *142*, 2043, 2252, 2336, 2396, 2521, 2523, 2640, 4079, 4198
\l_stex_allow_semantic_bool *151*, 4548, 4569, 4570, 4572, 4639, 4909, 4941, 4966, 5008, 5190, 5214, 5301, 5310, 5425, 5526, 5532, 5550, 5568, 5601, 5724, 5814, 5850, 6047, 6070, 7106
\stex_annotate:nn *128, 129*, 694, 699, 702, 1500, 1510, 2065, 2066, 3198, 3200, 3541, 3971, 3992, 3995, 4002, 4007, 4009, 4345, 4348, 4355, 4359, 4362, 4489, 4492, 4499, 4503, 4506, 4642, 4828, 4837, 4842, 5061, 5582, 5615, 5730, 5891, 6098, 6101, 6595, 6602, 6608, 6609, 6618, 6629, 6639, 7194, 7221, 7368, 7408, 7419, 7593, 7644, 7647, 7650, 7785, 7800, 7934, 7989, 7993, 7997, 8001, 8147, 8152, 8492, 8675, 8943, 8997, 9043, 9342, 9350, 9374, 9379, 9467, 9471, 9479, 9605, 9609, 9617, 9711, 9746, 10046
\stex_annotate_env 2278, 9559
_stex_annotate_force_break:n *129*, 697, 1553, 1615, 3199, 3540, 3989, 4007, 4342, 4360, 4487, 4504,

4841, 4977, 5055, 5587, 5614, 5618, 5630, 5735, 6595, 7194, 7222, 7369, 7409, 7593, 7596, 7639, 7810, 8148, 8943, 8947, 9043, 9050, 9717, 9750
\stex_annotate_invisible:n . *129*, 694, 695, 1625, 2286, 2572, 3024, 3526, 3970, 5353, 6595, 7642, 8145
\stex_annotate_invisible:nn *129*, 694, 1475, 1670, 1676, 1682, 2891, 2925, 2962, 3539, 3613, 3653, 3663, 4323, 4471, 5531, 5549, 5567, 6990, 7032, 7146, 7394, 8825, 8848, 10054
_stex_apply_patch_begin:n 525, 541, 3402, 3468
_stex_apply_patch_end:n 531, 556, 3407, 3473
\stex_archive_base:N *132*, 869, 869, 1048
\stex_archive_base:n *132*, 869, 873, 1087, 1096, 1214, 1383
\stex_archive_id:N *132*, 866, 866, 888, 1041, 1045, 1048, 1049, 1054, 1055, 1143, 1331, 1978, 2147
\stex_archive_path:N *132*, 876, 876, 1069, 1191, 2146
\stex_archive_path:n *132*, 876, 880, 1079
_stex_args_end: 5315, 5339, 5342, 6054, 6078, 6089, 6093
_stex_assign_do:n 3213, 3526, 3528, 3535, 3587
\l_stex_assoc_args_count 3916, 3920, 3962, 3963
\l_stex_brackets_dones_bool 6737, 6881, 6887, 6888, 6905
\stex_check_term:nn *152*, 2642, 3537, 6138, 6139, 6147, 6153, 6157, 6161, 6166, 6586, 8021
_stex_check_terms: . . . *152*, 3815, 3879, 4268, 4427, 6149, 6150, 6170
\c_stex_check_terms_bool 36, 6125, 6128, 8023
\stex_close_module: *142*, 2306, 2398, 2398, 2579, 8119
_stex_comma_sep_uri:nn 7236, 7246, 7606, 7622
\stex_css_link:n *129*, 697
\stex_css_literal:n *129*, 81, 697, 803, 1691, 8437
\l_stex_current_* *149, 150*
\l_stex_current_archive *132, 133*, 887, 888, 897, 908, 1029, 1068, 1069, 1141, 1143, 1190, 1191, 1329, 1331, 2142, 2146, 2147
\l_stex_current_args_tl 4592, 5314, 5315, 5318, 6662, 6677
\l_stex_current_arity_str 4591, 4659, 4674, 4676, 5286, 5435, 5436, 5471, 6683
\l_stex_current_def_uri 3187, 3189, 3192, 3196, 3198, 3204, 7257, 7279, 7280, 7286, 7287, 7354, 7356, 7359, 7361, 7403, 7405, 7408
\l_stex_current_display_tl *149*, 4552, 5193, 5209, 5212, 5598, 5718, 5940, 5944, 5957, 5961, 5972, 5986, 6000, 6024, 6034, 6072, 6282, 6697, 7113
\l_stex_current_doc_uri 7239
\l_stex_current_document_uri *125*, *135*, *144*, 253, 254, 259, 1128, 1190, 1264, 1284, 1303, 1307, 1312, 1314, 1421, 2374, 8032
\l_stex_current_domain_str . . . 3681
\l_stex_current_domain_uri 2981, 2994, 2998, 3000, 3005, 3013, 3021, 3037, 3041, 3070, 3099, 3124, 3419, 3444, 3486, 3510, 3630
\g_stex_current_file *125*, 244, 252, 257, 1071, 1081, 1343, 2371, 2845, 8934, 9029, 9034
\l_stex_current_full_tl *149*, 4551, 4560, 4599, 4604, 4619, 4632, 4644, 4772, 4855, 4981, 5013, 5057, 5192, 5202, 5208, 5211, 5218, 5253, 5254, 5265, 5278, 5279, 5308, 5389, 5416, 5584, 5597, 5717, 5767, 5815, 5816, 5893, 5896, 5939, 5956, 5971, 5985, 5999, 6049, 6062, 6071, 6081, 6082, 6084, 6281, 6617, 6628, 7108
\l_stex_current_language_str *125*, *127*, 254, 258, 453, 458, 459, 1164, 1170, 1175, 1204, 2280, 2814, 2816, 2832, 8939, 9039
\l_stex_current_module_str 2578, 3646, 3655, 3665
\l_stex_current_module_uri *142*, 1262, 1391, 2044, 2252, 2279, 2300, 2335, 2367, 2384, 2395, 2401, 2402, 2403, 2404, 2405, 2410, 2412, 2416, 2421, 2423, 2499, 2500, 2505, 2525, 2526, 2531, 2532, 2534, 2556, 2641, 2975, 4017, 4024, 4033, 4034, 4046, 4051, 6427, 6429, 6439, 6440, 6448, 7045, 7267, 7279, 7373, 8117
\l_stex_current_ns_uri 2266
\l_stex_current_redo_tl 4575, 4580, 4587, 4597, 4598, 4601,

4773, 4811, 4847, 5003, 5197, 5467
\l_stex_current_return_tl 4593, 4609, 5255, 5267, 5287, 5472
\l_stex_current_section_level_int *139*, 1495, 1531, 1533, 1537, 1551, 1568, 1569, 1598, 1600, 1603, 1606, 1607, 1614, 1630, 1632, 1636, 1647, 1648, 1649, 1650, 1651, 1652, 1654, 1658, 1661, 1668, 1672, 1678, 1682, 1687, 8286, 8293
\l_stex_current_section_level_str *139*, 1495, 1502, 1512, 1571, 8294
\l_stex_current_symbol_args_tl *149, 150*, *272*
\l_stex_current_symbol_arity_int *149, 150*, *272*
\l_stex_current_symbol_invoke_cs *149*
\l_stex_current_symbol_name_str 6050, 6051, 6052, 6053
\l_stex_current_symbol_return_tl *149, 150*, *272*
\l_stex_current_symbol_type_tl *149, 150*, *272*
\l_stex_current_symbol_uri *149, 150*, *272*, *293*, 3744, 3757, 3758, 3763, 3775, 3811, 3826, 3835, 3840, 3843, 3849, 3874, 3895, 4550, 4551, 4553, 5596, 6048, 6049, 6050, 6906, 6911
\l_stex_current_term_tl 4574, 4595, 4848, 4854, 5052, 5215, 5497, 5513, 5527, 5533, 5548, 5551, 5566, 5569, 5599, 5725, 5726, 5727, 5817, 6057
\stex_current_this: . 4588, 7105, 7121
\l_stex_current_this_tl 6930, 6933, 7107, 7115
\l_stex_current_type_tl 4594, 4641, 4714, 4769, 4777, 4780, 4784, 5086, 5087, 5162
\stex_deactivate_macro:Nn *123*, 67, 67, 2562, 2908, 2956, 3257, 3258, 3259, 3260, 3261, 3262, 3263, 3426, 3432, 3451, 3492, 3498, 3518, 3533, 3608, 3677, 3738, 3769, 3858, 6019, 6028, 6038, 6044, 6227, 6951, 7006, 7056, 7364, 7398, 7413, 7421, 7671, 7842, 7900, 7963, 7987, 7991, 7995, 7999, 8003, 8162, 9161, 9218, 9266, 9318, 9371, 9410, 9411, 9412, 9413, 9414, 9415, 9432, 9527, 9575, 9576, 9577, 9578, 9579, 9580, 9597, 9662, 9783, 10068
\stex_debug:nn *124*, 103, 103, 133, 137, 166, 173, 237, 472, 547, 603, 832, 837, 865, 907, 918, 965, 1002, 1036, 1044, 1065, 1135, 1206, 1284, 1322, 1335, 1350, 1359, 1467, 1815, 1840, 1842, 1851, 1855, 1894, 1900, 1914, 1917, 1920, 1943, 2045, 2076, 2133, 2151, 2155, 2166, 2319, 2326, 2328, 2356, 2359, 2361, 2364, 2376, 2384, 2386, 2388, 2400, 2448, 2500, 2522, 2531, 2536, 2540, 2542, 2643, 2696, 2717, 2721, 2739, 2756, 2764, 2846, 2861, 2863, 2889, 2960, 2995, 3013, 3055, 3083, 3120, 3135, 3192, 3211, 3312, 3368, 3536, 3555, 3558, 3562, 3582, 3586, 3611, 3631, 3647, 3651, 3661, 3682, 3684, 3757, 4017, 4033, 4046, 4156, 4197, 4199, 4218, 4235, 4237, 4274, 4396, 4604, 4620, 4713, 4957, 5084, 5100, 5223, 5242, 5254, 5256, 5259, 5265, 5278, 5300, 5308, 5423, 5463, 5474, 5644, 5659, 5661, 5668, 5671, 6151, 6155, 6159, 6163, 6310, 6341, 6352, 6424, 6439, 6447, 6449, 6452, 6588, 6615, 6716, 6729, 6770, 6891, 7063, 7137, 7280, 7284, 7287, 7289, 7359, 7380, 7416, 7573, 7766
\c_stex_debug_clist 30, 42, 104, 107, 121, 125, 127, 131, 135
\c_stex_default_metatheory 2042, 2639, 8118
\l_stex_default_notation 4634, 5268, 5273, 5293, 5751, 6665, 6666, 6672, 6681, 6686, 6689
_stex_definiens_impl:nn 3269, 7346, 7351
\stex_do_default_notation: *154*, 4633, 5266, 5750, 6660, 6660
\stex_do_default_notation_op: *154*, 5290, 6660, 6661, 6670
_stex_do_deprecation:n 634, 634, 3898
_stex_do_for_list: ... *157*, 3267, 7172, 7172, 7256, 7568, 7761, 7848
_stex_do_id: 639, 639, 1525, 1592, 7201, 7219, 7581, 7781, 7794, 7805, 7849, 7883, 8965, 10121
\stex_do_up_to_module:n *142*, 2494, 2494, 2497, 2501, 4027, 6431, 7049, 7096
\g_stex_document_kind_parameters_tl 1449, 1455, 10280
\g_stex_document_kind_str 14, 1448, 1450, 1454, 10268
\stex_document_uri:n *134*

\stex_document_uri_archive:N *134*, 1106, 1106, 1307
\stex_document_uri_element:N *135*, 1118, 1118
\stex_document_uri_from_archive_-file:Nn ... *135*, 1133, 1133, 1839, 2007, 2052, 8690
\stex_document_uri_language:N *135*, 254, 1115, 1115, 1204
\stex_document_uri_name:N *135*, 1112, 1112, 1312, 1314
\stex_document_uri_path:N *135*, 1109, 1109, 1303
\stex_document_uri_with_language:Nn ... *135*, 1121, 1121, 2373
_stex_eat_exclamation_point: ... *150*, 3904, 5257, 5268, 5489, 5489, 5490
_stex_end: ... 414, 422
\stex_end: ... 7128, 7134
\stex_every_module:n *142*, 2490, 2491, 2563, 2909, 2957, 3427, 3433, 3452, 3493, 3499, 3519, 3739, 3770, 3859, 6228, 6952, 7007, 7058, 7102, 8163
\g_stex_every_module_tl 2317, 2490, 2492, 8037
\l_stex_every_symbol_tl 4529, 4531, 4532, 4538, 4539, 4542, 4571, 4597
\stex_execute_in_module:n *142*, 2341, 2498, 2498, 2503, 2919, 3842, 6995, 7024, 7037
\l_stex_feature_name_str .. 3016, 3098, 3134, 3135, 3141, 3324, 3623
\stex_file_in_smsmode:Nn *144*, 2377, 2629, 2632, 2877
\stex_file_resolve:Nn *124*, 147, 153, 169, 176, 233, 243, 251, 852, 854, 1342
\stex_file_set:Nn ... *124*, 141, 141, 146, 160, 171, 845, 1134
\stex_file_split_off_ext:NN *124*, 208, 208
\stex_file_split_off_lang:NN ... *124*, 215, 215, 2371, 8934, 9029, 9034
\stex_file_use:N ... *124*, 166, 173, 205, 205, 237, 825, 845, 847, 855, 859, 931, 965, 966, 968, 981, 1034, 1071, 1081, 1135, 1150, 1343, 1347, 1763, 2154, 2165, 2376, 2378, 2845
\l_stex_fors_seq ... *157*, 3175, 3178, 7173, 7176, 7235, 7236, 7266, 7285, 7286, 7301, 7320, 7572, 7605, 7606, 7621, 7622, 7765, 7853, 7854, 7911, 7912, 7936, 7937
\stex_get_env:Nn *124*, 86, 87, 98, 119, 229, 231, 239, 241, 568, 574, 652, 823, 6122
\stex_get_in_morphism:n *145*, 3177, 3186, 3311, 3311, 3524, 3580, 3603
\stex_get_mathstructure:n ... *156*, 2997, 6980, 7014, 7122, 7122, 7136
\l_stex_get_structure_module_uri *156*, 2998, 7022, 7025, 7123, 7128, 7137
\l_stex_get_svariable_str 6075
\stex_get_symbol:n *148*, 1993, 4112, 4114, 4516, 5921, 5938, 5955, 6069, 6205, 6531, 7175, 7353, 7402, 8810, 8833
\stex_get_symbol:nTF *148*, 1431, 4112, 4115, 4120, 7124
\l_stex_get_symbol_args_tl *148*, 3349, 3888, 3908, 3910, 3911, 4145, 4185, 4246, 4301, 4313, 4400, 4449, 4462, 4520, 5318, 6476, 6478, 6615, 6677, 7275
\l_stex_get_symbol_arity_int *148*, 3348, 3831, 3887, 3909, 3919, 3925, 3929, 3933, 3937, 3941, 3945, 3949, 3953, 3957, 3960, 3961, 3962, 3963, 4000, 4112, 4144, 4184, 4245, 4273, 4300, 4312, 4353, 4399, 4448, 4461, 4497, 4519, 6271, 6305, 6336, 6343, 6363, 6412, 6517, 7274
\l_stex_get_symbol_def_tl *148*, 3225, 3350, 4146, 4186, 4247, 4401, 4521
\l_stex_get_symbol_invoke_cs *148*, 3353, 4149, 4189, 4250, 4404, 4524, 7127
\l_stex_get_symbol_macro_str . 3347
\l_stex_get_symbol_name_str .. 3218
\l_stex_get_symbol_return_tl *148*, 3352, 4148, 4188, 4249, 4275, 4403, 4418, 4523
\l_stex_get_symbol_type_tl *148*, 3351, 4147, 4187, 4248, 4402, 4522, 6981, 7016, 7128, 7139
\l_stex_get_symbol_uri . *148*, 1432, 1994, 3179, 3187, 3210, 3217, 3313, 3318, 3322, 3338, 3359, 3536, 3539, 3547, 3611, 3614, 3621, 3623, 3758, 3835, 4121, 4137, 4143, 4183, 4244, 4518, 5939, 5941, 5956, 5958, 6048, 6071, 6073, 6211, 6215, 6233, 6234, 6410, 6521, 6524, 6532, 6533, 6534, 6535, 6540, 6544, 7177, 7354, 7403, 8811, 8822, 8826, 8834, 8845, 8849

\stex_get_var:n *148*, 4371, 4371, 5969, 5981, 5995, 6243, 7392
\l_stex_get_variable_str 4372, 4374, 4398, 5971, 5972, 5983, 5985, 5986, 5997, 5999, 6000, 6076, 6077, 6244, 6249, 6252, 7394
\stex_has_definiens:N 4055
\stex_has_definiens:n 4058
\stex_has_definiens:NTF ... *147*, 4055
\stex_has_definiens:nTF *147*, 4055, 4056
\stex_has_definiens_p:N ... *147*, 4055
\stex_has_definiens_p:n ... *147*, 4055
\c_stex_home_file .. *125*, 238, 825, 845
\stex_html_literal:n 27, 35, 749, 757, 1658, 1661, 2019
\stex_html_node:nnn *128, 129*, 694, 1453, 1552
\stex_if_check_terms: 6118, 6129, 6133
\stex_if_check_terms:TF *152*, 3760, 4268, 4277, 4427, 4430, 6116, 6138, 6149, 6207, 6246
\stex_if_check_terms_p: ... *152*, 6116
\stex_if_do_html: 680
\stex_if_do_html:TF ... *128*, 680, 689, 1474, 1498, 1508, 1656, 2298, 2308, 2565, 2580, 2890, 2924, 2961, 3018, 3076, 3193, 3525, 3538, 3612, 3762, 3827, 3838, 3881, 4271, 4279, 4322, 4426, 5612, 6209, 6989, 7031, 7145, 7278, 7367, 7393, 7569, 7574, 7580, 7594, 7679, 7691, 7699, 7762, 7767, 7777, 7798, 7801, 7808, 7819, 7839, 7907, 7931, 8617, 8932, 8964, 8971, 8972, 8975, 8995, 9032, 9060, 9062, 9116, 9125, 9138, 9143, 9179, 9187, 9198, 9203, 9227, 9235, 9246, 9251, 9278, 9287, 9298, 9303, 9337, 9392, 9417, 9419, 9466, 9557, 9582, 9584, 9604, 9730, 9736, 10045, 10115
\stex_if_do_html_p: *128*, 680
\stex_if_file_absolute:N ... 263, 272
\stex_if_file_absolute:NTF *125*, 262, 853
\stex_if_file_absolute_p:N . *125*, 262
\stex_if_file_starts_with:NN *125*, 279
\stex_if_file_starts_with:NNTF .. *125*, 279, 1031
\stex_if_html_backend 518
\stex_if_html_backend:TF ... *128*, *179*, 2, 529, 670, 673, 697, 710, 724, 1451, 1547, 1611, 1624, 1666, 2012, 2063, 2276, 4577, 4608, 5194, 5493, 5509, 5523, 5543, 5561, 5580, 5610, 5890, 6046, 6067, 6096, 6117, 7192, 7205, 8186, 8226, 8356, 8436, 8491, 8503, 8511, 8523, 8533, 8674, 8685, 8728, 9456, 9599, 9702, 9739
\stex_if_html_backend_p: ... *128*, 670
\stex_if_in_module: 2504
\stex_if_in_module:TF *143*, 1261, 2311, 2498, 2504, 6515, 7372
\stex_if_in_module_p: *143*, 2504
\stex_if_module_exists:N 2508
\stex_if_module_exists:n 2512
\stex_if_module_exists:NTF *143*, 1319, 2327, 2360, 2387, 2508, 2772, 2781, 2878
\stex_if_module_exists:nTF *143*, 2508, 6982, 7017, 7140
\stex_if_module_exists_p:N *143*, 2508
\stex_if_module_exists_p:n *143*, 2508
\stex_if_smsmode: 2626
\stex_if_smsmode:TF *144*, 640, 1465, 1776, 1795, 2299, 2307, 2624, 2844, 2894, 2928, 2965, 3205, 3394, 3409, 3417, 3442, 3460, 3476, 3484, 3508, 3734, 3765, 3847, 6218, 7185, 7204, 7220, 7226, 7300, 7319, 7333, 7371, 9060, 9062
\stex_if_smsmode_p: *144*, 2624
\stex_ignore_spaces_and_pars: ... *123*, 45, 45, 48, 2555, 2557, 7226, 8988, 9896, 9933
\l_stex_import_uri 2541, 2542, 2544, 2883, 2884, 2892, 2896, 2897, 2913, 2914, 2920, 2923, 2926, 2931, 2932, 2939, 2942, 2952, 2953, 2963, 2967, 2968, 3001, 3003, 3005
\stex_in_archive:nn *132*, 883, 883, 1838, 1901, 1982, 2006, 2051, 2074, 2102, 2131, 2175, 2191, 2206, 2218, 2803, 8689
\g_stex_in_comp_bool 5606, 5884, 5889, 5900
\stex_in_invisible_html_bool ... 3990, 4343, 5492, 5525
\l_stex_in_meta_bool 2338, 2347, 2350, 2352
\l_stex_in_this_bool 5009, 5495, 5505, 5511, 5521, 5545, 5559, 5563, 5577, 5613, 5627, 5851, 7103, 7114, 7116
\l_stex_inparray_bool 6893
\stex_input_with_hooks:Nn .. *125*, 245, 245, 248, 2026, 2046, 2054, 2057
\stex_invoke_sequence: *149*, 4453, 4466, 4612, 4612, 5686

\stex_invoke_structure: *150*, 4701, 4701, 6966, 7127
\stex_invoke_symbol: *149*, 3892, 4305, 4317, 4603, 4603, 7276
_stex_invoke_symbol:NnnnnnN *149*, 4517, 4547, 4565, 4568
_stex_invoke_symbol:nnnnnnN *149*, 4050, 4128, 4141, 4142, 4547, 4547, 4566, 5111, 5125, 7080
\stex_invoke_text_symbol: *149*, 3823, 4607, 4607
_stex_invoke_variable:nnnnnn *150*, 5189
_stex_invoke_variable:nnnnnnN 4310, 4459, 5189, 5684
_stex_is_sequentialized:n 5639, 5722, 5795, 5812
\stex_iterate_break: *148*, 4069, 4070, 4073
\stex_iterate_break:n *148*, 3288, 3652, 3662, 3680, 4069, 4070, 4076, 4243
\stex_iterate_morphisms:nn *145*, 3271, 3271, 3630, 3646
\stex_iterate_notations:nn *154*, 3124, 5162, 6548, 6548
\stex_iterate_symbols:n *148*, 4069, 4069, 4236
\stex_iterate_symbols:nn *148*, 3041, 3683, 4086, 4086, 5087
\l_stex_key_* *147*
\l_stex_key_answerclass_str 9101, 9105, 9119, 9153, 9154
\l_stex_key_archive_str 379, 382, 1838, 1901
\l_stex_key_args_str 3694, 3699, 3708, 3748, 3779, 3912, 3921, 3926, 3930, 3934, 3938, 3942, 3946, 3950, 3954, 3958, 3973, 4325, 4413, 4414, 4473, 7157
\l_stex_key_argtypes_clist 3713, 3717, 4006, 4008, 4358, 4361, 4502, 4505, 6164, 6165, 6166, 7161
\l_stex_key_assoc_str 3697, 3702, 3980, 3981, 4329, 4330, 4477, 4478
\l_stex_key_autogradable_bool 8875, 8881, 8886, 8940, 9040
\l_stex_key_continues_tl . 7481, 7487
\l_stex_key_def_tl 3710, 3720, 3747, 3778, 3793, 3797, 3820, 3889, 3994, 3995, 4302, 4314, 4347, 4348, 4450, 4463, 4491, 4492, 6156, 6157, 7159
\stex_key_def_tl *152*
\l_stex_key_deprecate_str 405, 407, 635, 636
\l_stex_key_due_tl 10091, 10095, 10135, 10136
\l_stex_key_fallback_tl 1767, 1820, 1852, 1918
\l_stex_key_feedback_tl 9323, 9326, 9349, 9352, 9365, 9366, 9441, 9446, 9478, 9480, 9514, 9515, 9616, 9618, 9648, 9649, 10036, 10053, 10056, 10063, 10064
\l_stex_key_file_str 380, 383, 1807, 1839, 1846, 1902, 1983
\l_stex_key_for_clist *157*, 3176, 7156, 7164, 7174, 7425, 7430
\l_stex_key_for_str 7191
\l_stex_key_from_tl 7483, 7489
\l_stex_key_Ftext_tl 9444, 9450, 9475, 9512, 9613, 9646
\l_stex_key_functions_tl . 7482, 7488
\l_stex_key_given_tl 10090, 10094, 10132, 10133
\l_stex_key_hide_bool 7480, 7486, 7598, 7608, 7624, 7691, 7811, 7839
\l_stex_key_id_str 387, 389, 641, 642, 7238, 7239, 7294, 7296, 7778, 7791, 7880, 9110, 9112, 9118, 9173, 9175, 9181, 9221, 9223, 9229, 9271, 9273, 9280, 9331, 9333, 9343, 9361, 9923, 9990, 10010, 10022, 10024, 10035, 10044, 10047
\l_stex_key_intent_args_clist 6179, 6185, 6377, 6386, 6388
\l_stex_key_intent_str 3833, 6178, 6184, 6302, 6380
\l_stex_key_just_tl 7428, 7433, 7640, 7646, 7647
\l_stex_key_macroname_str 7155, 7165, 7251, 7253, 7269, 7273
\l_stex_key_method_tl 7427, 7432, 7640, 7649, 7650
\l_stex_key_mhrepos_str 8880, 8888, 9084, 9086, 9094, 10148
\l_stex_key_min_tl 8878, 8884, 8955, 9058, 9076, 9957, 9977
\l_stex_key_name_str 3707, 3715, 3745, 3776, 3791, 3795, 3805, 3806, 3808, 3812, 3818, 3850, 3871, 3872, 3875, 3886, 3898, 3972, 4263, 4264, 4274, 4281, 4284, 4297, 4299, 4311, 4324, 4410, 4411, 4433, 4445, 4447, 4460, 4472, 6244, 6501, 6502, 6503, 6507, 6508, 6509, 7154, 7163, 7190, 7252, 7253, 7259, 7267,

7273, 7279, 7439, 7443, 7450, 7453, 7474, 7628, 7629, 7729, 7736, 7741, 7866, 7868, 7874, 7875, 7916, 7917, 7941, 7942, 8879, 8885, 8933, 8935, 8938, 9028, 9030, 9033, 9035, 9038
\l_stex_key_number_tl 10089, 10093, 10109, 10110
\l_stex_key_numname_str 7440, 7444, 7449, 7458, 7616, 7627, 7730, 7735, 7740, 7904, 7915, 7940
\l_stex_key_op_tl 3832, 4420, 4422, 6177, 6183, 6264, 6265, 6266, 6273, 6274, 6279, 6285, 6414, 6506, 6507, 6510, 6519, 6523, 6525, 6535, 6537, 6626, 6630
\l_stex_key_post_tl 1766, 1826, 1831, 5905, 5909, 5944, 5961, 5983, 5997, 6024, 6034
\l_stex_key_pre_tl 1765, 1826, 1831, 5904, 5908, 5944, 5961, 5983, 5997, 6024, 6034
\l_stex_key_prec_str 3834, 6176, 6182, 6313, 6316, 6319, 6325, 6328, 6331, 6334, 6340, 6352, 6353
\l_stex_key_proofend_tl 7479, 7485, 8011, 8012
\l_stex_key_pts_str 9322, 9325, 9344, 9345, 9362, 9363, 10037, 10048, 10049, 10060, 10061
\l_stex_key_pts_tl 8877, 8883, 8941, 8950, 8954, 9041, 9052, 9055, 9071, 9952, 9972, 9995
\l_stex_key_reorder_str 3696, 3700, 3983, 3984, 4335, 4336, 4483, 4484
\l_stex_key_return_tl 3711, 3716, 3891, 3997, 4000, 4275, 4304, 4316, 4350, 4353, 4418, 4452, 4465, 4494, 4497, 6160, 6161, 7160
\stex_key_return_tl *152*
\l_stex_key_role_str 3695, 3703, 3810, 3986, 3987, 4332, 4333, 4480, 4481, 7270
\l_stex_key_root_tl 5906, 5910, 5914, 5916
\l_stex_key_short_tl 1517, 1519, 1562, 1565, 1582, 1584
\l_stex_key_showname_bool . 7436, 7441, 7445, 7451, 7456, 7473, 7731
\l_stex_key_sig_str .. 2256, 2271, 2281, 2315, 2356, 2374, 2376, 2378
\l_stex_key_style_clist 399, 401, 504, 508, 542, 546, 7241, 7242, 7336, 9397, 9398, 9402, 9405, 9530, 9537, 9561, 9562, 9567, 9570
\l_stex_key_T_bool 9442, 9447, 9448, 9468, 9472, 9495, 9509, 9606, 9610, 9629, 9643
\l_stex_key_term_tl 7426, 7431, 7640, 7643, 7644
\l_stex_key_testspace_dim 9099, 9102, 9104, 9133, 9687, 9689, 9745, 9774, 9777
\l_stex_key_title_str 7166
\l_stex_key_title_tl ... 393, 395, 1889, 1894, 1931, 1932, 2292, 7189, 7195, 7196, 8943, 8944, 8945, 8949, 9043, 9044, 9045, 9047, 9048, 9152, 9153, 9212, 9213, 9260, 9261, 9312, 9313, 9923, 9990, 10005, 10006, 10010, 10116, 10117, 10127, 10128
\l_stex_key_Ttext_tl 9443, 9449, 9473, 9510, 9611, 9644
\l_stex_key_type_tl 3709, 3719, 3746, 3777, 3792, 3796, 3991, 3992, 4303, 4315, 4344, 4345, 4488, 4489, 6152, 6153, 7158, 7167
\stex_key_type_tl *152*
\l_stex_key_variant_str 3781, 4286, 4435, 6175, 6181, 6188, 6190, 6232, 6254, 6300, 6411, 6501, 6507, 6597, 6630
\l_stex_key_wikidata_str 3712, 3718, 3977, 3978
\l_stex_keys_cls_id 6, 13, 36, 43, 44, 46, 81, 728, 735, 758, 765, 766, 768, 803
\l_stex_keys_counter_id 7, 10, 28, 37, 61, 62, 63, 64, 729, 732, 750, 759, 783, 784, 785, 786
\l_stex_keys_counter_parent 8, 11, 29, 48, 50, 51, 52, 53, 54, 55, 56, 58, 730, 733, 751, 770, 772, 773, 774, 775, 776, 777, 778, 780
\stex_keys_define:nnnn *126*, 5, 359, 359, 378, 386, 392, 398, 404, 410, 727, 1516, 1764, 1774, 2255, 3693, 3706, 3790, 5903, 5913, 6005, 6010, 6174, 6195, 6197, 6929, 7153, 7424, 7438, 7478, 7492, 7865, 8383, 8876, 9083, 9100, 9169, 9321, 9440, 9685, 10088, 10192, 10243, 10256
\stex_keys_set:nn *126*, 17, 374, 374, 739, 1523, 1578, 1805, 1904, 2291, 3727, 3754, 3802, 3803, 4261, 4408, 5920, 5937, 5954, 5968, 5980, 5994, 6016, 6022, 6032, 6204, 6242, 6954, 7182, 7217, 7567, 7728, 7760, 7847, 7873, 8477, 8924, 8928, 9018, 9093, 9109, 9132, 9172,

9220, 9270, 9330, 9383, 9464, 9506, 9548, 9602, 9640, 9741, 9758, 10021, 10103, 10107, 10147, 10205, 10267
\stex_kpsewhich:Nn *123*, 77, 77, 89, 99
\c_stex_language_abbrevs_prop *127*, 426
\c_stex_languages_clist . 31, 454, 457
\c_stex_languages_prop *127*, 222, 426, 465, 1172, 1177
\g_stex_last_feature_str 2578
\l_stex_macroname_str *147*, 2568, 2569, 3724, 3729, 3731, 3755, 3804, 3817, 3885, 3974, 3975, 4262, 4298, 4309, 4326, 4327, 4409, 4446, 4458, 4474, 4475, 6967, 7269
\c_stex_main_archive *133*, 1029, 1978
\c_stex_main_document_uri *135*, 1190, 1841
\c_stex_main_file *124*, 228, 244, 1193, 1200
_stex_map_args:N *153*, 4001, 4354, 4498, 5320, 6373, 6399, 6475, 6475, 6622, 6679
_stex_map_notation_args:N *153*, 6475, 6488, 6642
\c_stex_mathhub_<id>_archive . . *132*
\c_stex_mathhub_file 931
\c_stex_mathhub_files *132*, 822, 915, 921, 927, 1029
\c_stex_mathhub_main_archive 1039, 1040
_stex_maybe_brackets:nn *155*, 4621, 5280, 5291, 6719, 6735, 6886, 6886
\c_stex_metadata_bool 37, 8820, 8843
\stex_metagroup_do_in:n *125, 126*, 327, 327, 331, 348, 2495, 3194, 3545, 3619
\stex_metagroup_new: *126*, 323, 324, 2324, 2357, 2383, 3028
\l_stex_metatheory_uri 2042, 2253, 2260, 2262, 2282, 2283, 2318, 2319, 2342, 2544, 2546, 2639, 8117, 8118
_stex_module_code_macro:N 2246, 2330, 2390, 2402, 2421, 2455, 2499, 2509, 2526, 2531, 2532, 2556
_stex_module_code_macro:n 2249, 2513, 7064
\stex_module_setup:n *142*, 2296, 2310, 2310, 2574
_stex_module_setup_top_nosig:n 2316, 2323, 8036
\stex_module_setup_top_nosig:n *142*, 2323
\stex_module_uri:n *135*
\stex_module_uri_archive:N *135*, 1220, 1220, 2796
\stex_module_uri_as_qm:n *136*, 1254, 1254, 4199, 4200
\stex_module_uri_name:N *136*, 1229, 1229, 2279, 2798, 2897, 2932, 2968
\stex_module_uri_name:n *136*, 1229, 1233, 4207, 4210
\stex_module_uri_path:N *135*, 1223, 1223, 2797
\stex_module_uri_path:n *135*, 1223, 1226, 4219
\stex_module_uri_split_name:NNN *136*, 1236, 1236, 7022
\stex_module_uri_split_name:NNn *136*, 1236, 1240
\l_stex_morphism_assigns_seq 3035, 3073, 3090, 3146, 3195, 3246, 3546
\l_stex_morphism_morphisms_seq 3034, 3063, 3074
\l_stex_morphism_renames_seq 3033, 3072, 3087, 3101, 3114, 3319, 3620
\l_stex_morphism_symbols_prop 3220, 3686
\l_stex_morphism_symbols_seq 3032, 3042, 3071, 3084, 3093, 3235, 3315, 3376, 3384
_stex_morphisms_macro:N . . 2234, 2331, 2391, 2403, 2412, 2449, 2975
_stex_morphisms_macro:n 2237, 3052, 3298, 4098, 6563, 7067
\stex_new_document_element_uri:n *135*, 1127, 1127, 1777
\stex_new_module_uri:n *136*, 1260, 1260, 2325, 2358, 2385, 6961, 7025, 7050
\stex_new_statement:nnn *157*, 7180, 7180, 7312, 7317, 7332, 7335
\stex_new_stylable_cmd:nnnn *127*, 482, 482, 2882, 2903, 2951, 3438, 3504, 3523, 3602, 3628, 3726, 3753, 3801, 4260, 4407, 6203, 6241, 7845
\stex_new_stylable_env:nnnnnnn *127*, 516, 516, 2290, 3391, 3400, 3456, 3466, 6942, 6977, 7011, 7181, 7673, 7695, 7759, 8918, 9017, 9124, 9186, 9234, 9286, 9382, 9547, 10100
\stex_new_symbol_uri:n *136*, 1390, 1390, 3127, 3812, 3875, 7110
\stex_new_symbol_uri:nn *136*, 1393, 1393, 3043, 4051, 4183, 4244, 5094, 7080, 7279

_stex_next_symbol:n *149*, 4526, 4528, 4546, 4920, 5050, 6042
\c_stex_no_archive *133*, 1020
\c_stex_no_archive_str *133*, 1020, 1146, 1346, 2799
\c_stex_no_archive_uri ... 1022, 1028
\c_stex_no_frontmatter_bool 39, 1712
_stex_notation_add: *153*, 3761, 3837, 6208, 6408, 6408
\l_stex_notation_args_tl 6270, 6303, 6372, 6378, 6384, 6489, 6491, 6588
_stex_notation_check: . *154*, 3760, 4277, 4430, 6207, 6246, 6585, 6585
\l_stex_notation_cs *154*, 4620, 4623, 4634, 4661, 4665, 4673, 4680, 4695, 4856, 5257, 5261, 5282, 5748, 6575, 6581
_stex_notation_do_html:n *154*, 3763, 3840, 4281, 6211, 6594, 6594
\l_stex_notation_downprec 5600, 6878, 6885, 6891, 6892
\l_stex_notation_macrocode_cs *153*, 3783, 4288, 4437, 6236, 6256, 6298, 6310, 6413, 6501, 6504, 6518, 6522, 6533, 6588, 6589, 6620
_stex_notation_make_args: *154*, 3784, 4289, 6237, 6257, 6590, 6641, 6641
\stex_notation_parse:n *153*, 3759, 3836, 4276, 4421, 4424, 6206, 6245, 6263, 6263
_stex_notation_set_default:n *153*, 6214, 6249, 6514, 6514, 6539
_stex_notations_macro:N 2240, 2333, 2393, 2405, 2423, 2456, 2482, 6429, 6439, 6440, 6448
_stex_notations_macro:n . 2243, 6560
\stex_par: 9531, 9532, 9535, 9538, 9539, 9542
\stex_persist:n *127*, *184*, 586, 601, 602, 711, 712, 714, 717, 720, 721, 1004, 1047, 1054, 2409
\c_stex_persist_force_bool .. 35, 581
\c_stex_persist_mode_bool 33, 571, 582, 587
_stex_persist_read_now: 586, 1053, 8125
\c_stex_persist_write_mode_bool 34, 577, 588, 594, 709, 1052, 2399
\stex_pseudogroup:nn *125, 126*, 250, 303, 303, 685, 884, 2524, 5309, 6916
\stex_pseudogroup_restore:N *126*, 258, 259, 306, 306, 897, 2534, 6921, 6922
\stex_pseudogroup_with:nn *126*, 313, 313, 4070, 4088, 4141, 5887, 6550, 6807, 6826, 6851
\c_stex_pwd_file *124*, 228, 855, 1031, 1032, 1763
\stex_reactivate_macro:N *123*, 67, 73, 2563, 2909, 2948, 2957, 3264, 3265, 3266, 3428, 3434, 3453, 3494, 3500, 3520, 3739, 3770, 3859, 6228, 6952, 7008, 7059, 7305, 7306, 7307, 7308, 7309, 7310, 7324, 7325, 7326, 7327, 7328, 7329, 7330, 7582, 7583, 7584, 7585, 7586, 7587, 7588, 7589, 7590, 7591, 7592, 8163, 8902, 8903, 8904, 8905, 8906, 8907, 8908, 9296, 9416, 9581
\stex_ref_new_doc_target:n 642, 1775, 1775, 1793
_stex_ref_new_id:n 7295
\stex_ref_new_sym_target:n 1794, 1794, 6081, 7302, 7321
\stex_ref_new_symbol:n 1989, 2452, 3825, 3894
\l_stex_relative_import_bool 1281, 1283, 1321, 2940
_stex_renamedecl_do:nn 3600, 3604, 3610
\stex_require_module:N . *145*, 2546, 2771, 2771, 2884, 2914, 2953, 3003
\stex_require_module_noerr:N *145*, 2771, 2780, 2791
\stex_require_module_noerr:n *145*, 2771, 2789, 2942
_stex_return_args:nn 3903, 4001, 4354, 4498
\l_stex_return_notation_tl 4586, 4756, 4764, 4928, 4929, 5017, 5043, 5044, 5454, 5460
\stex_set_language:n 2269
\stex_sms_allow:N *144*, 2585, 2587, 2600, 2601, 2602, 2603, 2604
\stex_sms_allow_env:n *144*, 2595, 2597, 2605, 3430, 3436, 3496, 3502, 6950, 7005, 7055, 7209
\stex_sms_allow_escape:N *144*, 2590, 2592, 2606, 2607, 2910, 3455, 3522, 3532, 3607, 3676, 3740, 3771, 3860, 6229, 7229, 7365
\stex_sms_allow_import:Nn *144*, 2608, 2611, 2947
\stex_sms_allow_import_* *144*
\stex_sms_allow_import_env:nn *144*, 2615, 2617, 2622

\g_stex_sms_import_code *144*, 2623, 2635, 2652, 2941
\stex_smsmode_do: *144*, 2304, 2548, 2559, 2677, 2679, 2682, 2682, 2905, 3422, 3448, 3489, 3515, 3530, 3605, 3735, 3766, 3855, 6224, 6944, 6999, 7043, 7202, 7226, 7227, 7348
\stex_source_path:n *133*, 1067, 1067, 1076, 1846, 1902, 1983, 2016, 2026, 2045, 2046, 2054, 2057, 2182, 2198, 2215, 2805
\stex_source_path:nn *133*, 1067, 1074, 2177, 2193, 2208
_stex_sref_do_aux:n 8022
\stex_str_if_ends_with:nn .. *123*, 62
\stex_str_if_ends_with:nnTF *123*, 62, 3637, 3660, 4200, 4207, 4219
\stex_str_if_ends_with_p:nn *123*, 62, 4172, 4241
\stex_str_if_starts_with:nn *123*, 57
\stex_str_if_starts_with:nnTF *123*, 57, 413, 998, 1874, 4734, 4738
\stex_str_if_starts_with_p:nn *123*, 57
\l_stex_struct_this_tl 4720, 4922, 4923, 4926, 5004, 5007, 5016, 5037, 5038, 5041, 5055, 5852
\stex_structural_feature_-module:nn ... *143*, 2564, 2564, 6969
\stex_structural_feature_module_-end: *143*, 2564, 2577, 6946, 7001, 7048
\stex_structural_feature_-morphism:nnnnn *145*, 2981, 2988, 3392, 3440, 3457, 3506
\stex_structural_feature_-morphism_check_total: 3234, 3459, 3475, 3513
\stex_structural_feature_-morphism_end: . *145*, 2981, 3069, 3397, 3412, 3447, 3463, 3479, 3514
\stex_structural_feature_-morphism_with_macros:nnnnn .. *145*, 2981, 2984, 3404, 3439, 3470, 3505
\stex_style_apply: *127*, 3, 482, 487, 524, 530, 725, 2302, 2307, 2899, 2934, 2970, 3395, 3401, 3406, 3410, 3420, 3445, 3461, 3467, 3472, 3477, 3487, 3511, 3750, 3787, 3852, 4292, 4439, 6221, 6260, 7199, 7205, 7580, 7682, 7699, 7777, 7798, 7801, 7824, 7858, 8964, 8971, 9060, 9062, 9130, 9142, 9192, 9202, 9240, 9250, 9283, 9302, 9417, 9421, 9582, 9586, 10120, 10123
\stex_suppress_html:n *128*, 684, 684, 2665, 6293
_stex_symbol_macro:N 2228, 2332, 2392, 2404, 2416, 2450, 2451, 4024, 4033, 4034, 7045
_stex_symbol_macro:n 2231, 4060, 4080, 4101, 4201, 4214, 4220, 7072, 7078, 7381, 7383
\stex_symbol_uri:n *136*
\stex_symbol_uri_archive:N *136*, 1396, 1396
\stex_symbol_uri_module:N *137*, 1402, 1402
\stex_symbol_uri_module:n *137*, 1402, 1405, 4060, 7373, 7381, 7383
\stex_symbol_uri_name:N *137*, 1412, 1412, 3623, 4553, 5941, 5958, 6050, 6073, 6233, 8822, 8845
\stex_symbol_uri_name:n *137*, 1412, 1415, 3118, 4062, 7382, 7383
\stex_symbol_uri_path:N *136*, 1399, 1399
\stex_symdecl_do: *147*, *257*, *331*, 3814, 3878, 3897, 3897, 4267, 4416, 7271
_stex_symdecl_html: *147*, 3828, 3881, 3969, 3969, 7278
_stex_term_arg:nnn *151*, 5426, 5602, 5605, 5611, 5627
_stex_term_arg:nnnnn *151*, 5594, 5594, 5646, 5705, 5723, 5813, 6656, 6779, 6787
_stex_term_arg_aB:nnnnn *151*, 5629, 5643, 6792, 6800, 6817
_stex_term_do_aB_clist: ... *294*, 5637, 5653, 5707, 5736, 5765, 5823, 6807, 6808, 6827, 6831, 6852, 6857
_stex_term_oma:nn *150*, 5319, 5543, 5544, 5559, 6731
_stex_term_oma_or_omb:nn 6731, 6736, 6784, 6797
_stex_term_omb:nn *150*, 5347, 5348, 5561, 5562, 5577, 6784, 6797
_stex_term_oms:nn *150*, 4579, 4581, 5493, 5494, 5505, 5508, 5922, 5945, 5962, 6630
_stex_term_oms_or_omv:nn 4579, 4581, 4609, 4622, 4774, 4812, 4851, 4907, 5196, 5198, 5281, 5292, 5302, 5508, 6720
_stex_term_omv:nn *150*, 5196, 5198, 5216, 5509, 5510, 5521, 5974, 5988, 6002
_stex_titlefragment: 8298

\stex_undefine:N *123*, 53, 53, 56, 310, 320, 2044, 2641
\stex_uri_from_archive_file:Nn . *135*
\stex_uri_from_pair:Nn *136*, 1361, 1361, 2262
\stex_uri_from_pair:Nnn *136*, 1281, 1282, 1299, 1370, 1374, 2541, 2883, 2913, 2939, 2952, 3001
\stex_uri_resolve:Nn 2266
\stex_uri_use:N 7239
\stex_use_archive_uri:n *134*, 1086, 1086
\stex_use_document_uri:N *134*, 1089, 1089, 1206, 1284, 1421, 1778, 1779, 1781, 1786, 1840, 1845, 2021, 2045, 2376, 2643, 8691
\stex_use_document_uri:n *134*, 1089, 1092
\stex_use_module_uri:N *135*, 1207, 1207, 1322, 1335, 1350, 1359, 2283, 2300, 2319, 2326, 2359, 2384, 2386, 2401, 2448, 2500, 2542, 2774, 2879, 2892, 2896, 2926, 2931, 2963, 2967, 3013, 3021, 3419, 3444, 3486, 3510, 4017, 4046, 6427, 7137
\stex_use_module_uri:n *135*, 1207, 1210, 2522, 2527, 2536, 2976, 4828, 6991, 7033, 7063, 7067, 7147
\stex_use_notation:nn *154*
\stex_use_notation:nnTF *154*, 4632, 4855, 5253, 5747, 6573, 6573
\stex_use_op_notation:nnTF *154*, 4619, 5279, 6573, 6579
\c_stex_use_sref_bool 27
\stex_use_symbol_uri:N *136*, 1376, 1376, 1994, 3192, 3198, 3536, 3539, 3611, 3614, 3744, 3757, 3763, 3775, 3826, 3840, 3843, 3849, 3895, 4551, 5939, 5956, 6049, 6071, 6211, 6234, 6521, 6524, 6532, 6533, 6534, 6535, 6544, 7280, 7287, 7359, 7408, 7854, 7912, 7937, 8826, 8849
\stex_use_symbol_uri:n *136*, 1376, 1379, 2483, 3247, 4558, 5165, 5166, 5168, 5169, 5176, 5177, 5179, 5180, 6430, 6447, 6450, 6464, 6465, 6466, 6469, 6470, 6471, 7109, 7248, 7302, 7321, 7368, 7380
_stex_var_notation_macro: *153*, 4278, 4431, 6247, 6500, 6500
_stex_variable:nnnnnnnN . 4258, 4397
\l_stex_variables_prop *148*, 4256, 4297, 4380, 4445
\stex_with_file_hooks:Nnn *125*, 246, 249, 249, 2644

stex internal commands:

\l__stex_annotate_do_output_bool . . . 670, 675, 678, 681, 686, 690, 8024
__stex_annotate_env_str . . . 652, 653
__stex_check_env_str 6122, 6123, 6124
__stex_comps_do_defref:nn 6017, 6023, 6033, 6066
__stex_comps_do_ref:nNn . . 5922, 5943, 5960, 5970, 5982, 5996, 6045
__stex_comps_slash:w 6053, 6077, 6089, 6093
__stex_comps_split_slash:n 5925, 5941, 5958
__stex_comps_split_slash_i:nw 5926, 5929, 5931
__stex_comps_uppercase:n 5950, 5961, 5997, 6034
__stex_debug_:nn 105, 108, 113
\l__stex_debug_cl 121, 123, 126
__stex_debug_env_str . 119, 120, 123
__stex_doc_check_topsect: 1667, 1673, 1679, 1685
__stex_doc_do_section:n 1524, 1530, 1534, 1538, 1591
\g__stex_doc_ftml_link_text_tl 1426, 1429, 1441
__stex_doc_maketitle: . . . 1487, 1492
__stex_doc_orig_backmatter 1725, 1728, 1746
__stex_doc_orig_frontmatter 1715, 1718, 1735
__stex_doc_set_title:n . . 1464, 1481
__stex_doc_skip_section: 1612, 1618, 1622
__stex_doc_skip_section_i: 1597, 1600, 1603, 1613, 1618
__stex_doc_title_html: 1469, 1473, 1484
\g__stex_doc_title_tl 1446, 1461, 1468, 1475, 1480
\l__stex_doc_titlefragment_bool 1575, 1580, 1590, 1595
__stex_doc_x_matter: . . . 1713, 1757
__stex_expr_aB_arg:nnnnn 5651, 5657
__stex_expr_aB_simple_arg:nnnnn 5672, 5703
__stex_expr_add_prop_arg:nnw 5315, 5339, 5342
\l__stex_expr_after_tl 5096, 5100, 5101, 5102, 5173, 5184
__stex_expr_annotate:nnn . 5498, 5514, 5530, 5547, 5565, 5581, 5591

__stex_expr_arg:n 5316, 5351
\l__stex_expr_arg_counter_int 5305, 5313, 5328, 5363, 5364
__stex_expr_arg_do:nnn 5365, 5371, 5387, 5422, 5429
__stex_expr_arg_inner:nn 5354, 5357, 5361, 5367
\l__stex_expr_assigned_seq 4916, 5035, 5077, 5144
__stex_expr_assoc_make_seq:nnn 5719, 5746, 5832
__stex_expr_assoc_seq:nnnnnnn 5663, 5714
__stex_expr_check:n 5395
__stex_expr_check:nTF ... 5364, 5370
__stex_expr_check_b:nn .. 5320, 5345
__stex_expr_check_comp: 5496, 5505, 5512, 5521, 5546, 5559, 5564, 5577, 5605, 5619, 5627
\l__stex_expr_clist .. 4640, 4647, 4654, 4658, 4890, 4911, 4917, 4919
\l__stex_expr_comp_cs 4943, 4948, 4967, 4969
\l__stex_expr_count_int 5642, 5649, 5658, 5705, 5723, 5813
\l__stex_expr_cs 4892, 4898, 4905, 5748, 5751, 5771
__stex_expr_current_type: 4846, 4982, 5058
\l__stex_expr_current_type_tl 4771, 4808, 4849, 4854
__stex_expr_custom:n 5233, 5243, 5307
\l__stex_expr_customs_prop 5311, 5323, 5324, 5325, 5340, 5375, 5381, 5396, 5399
\l__stex_expr_customs_seq 5312, 5329, 5330, 5331, 5341, 5397
__stex_expr_do_aB_clist: 5629, 5637, 5707, 5765
__stex_expr_do_ab_next:nn 5319, 5321, 5347, 5348
__stex_expr_do_all:w ... 4689, 4698
__stex_expr_do_assign:nn 4792, 4796
__stex_expr_do_assign_list:n 4767, 4789
__stex_expr_do_decl:nnnnnnnnn 5092, 5120
__stex_expr_do_decl_nomacro:nnnnnnnnn 5090, 5106
__stex_expr_do_first_arg:n 4676, 4683
__stex_expr_do_first_next: 4678, 4685
__stex_expr_do_headterm:nn 5500, 5516, 5524, 5541, 5822
__stex_expr_do_one:w ... 4687, 4693
__stex_expr_do_seqmap:nnnnnn 5669, 5778
__stex_expr_end: 4735, 4739, 4755, 4760, 5665, 5678
\l__stex_expr_field_name_str 5022, 5026, 5027, 5061
\l__stex_expr_fields_clist 4768, 4790, 4797, 4815, 4818, 5135, 5136
\l__stex_expr_first_args_tl 4675, 4695, 4699
__stex_expr_full_notation:n 5249, 5252
__stex_expr_get_field_name:n 5021, 5033
__stex_expr_get_index_notation:n 4626, 4631, 4694
__stex_expr_gobble:nnnnnnnn 5665, 5678
\l__stex_expr_iarg_tl 5758, 5760, 5771
__stex_expr_invoke:nnnnN 4555, 4576
__stex_expr_invoke_field:n 4997, 5031
__stex_expr_invoke_maybe_-field:nn 4986, 4990
__stex_expr_invoke_this:n 4728, 4973
__stex_expr_is_argsep:n 5697
__stex_expr_is_seqmap:n 5691
__stex_expr_is_seqmap:nTF ... 5667
__stex_expr_is_varseq:n 5681
__stex_expr_is_varseq:nTF 5660, 5783
__stex_expr_make_mod:n .. 4814, 4826
__stex_expr_make_oml:n .. 4818, 4833
__stex_expr_make_oml:nn . 4834, 4836
__stex_expr_make_prop: 4858, 4991, 5074
__stex_expr_make_prop_assign: 4859, 4994, 5134
__stex_expr_make_prop_assign:nn 5137, 5142
__stex_expr_make_prop_assign_-replace:nnnn 5145, 5151
__stex_expr_make_type:n 4777, 4782, 4784, 4800
__stex_expr_math: 4605, 5222
__stex_expr_maybe_notation:w 4723, 4725, 4853
__stex_expr_maybe_return:n 5260, 5271, 5430
__stex_expr_merge:nw ... 4717, 4733
\l__stex_expr_more_nextsymbol_tl 5034, 5036, 5064

__stex_expr_notation:w 4699, 5235, 5236, 5245, 5288
\l__stex_expr_old_seq 5790, 5793, 5797, 5802
__stex_expr_op_custom:n 5226, 5243, 5299
__stex_expr_op_notation:w 5228, 5229, 5247, 5277
__stex_expr_present: ... 4864, 4995
__stex_expr_present:nn 4868, 4874, 4879, 4886, 4889
__stex_expr_present_entry:nn ... 4894, 4900, 4915
__stex_expr_present_i:w 4860, 4866, 4872
__stex_expr_present_ii:nw 4877, 4885
\l__stex_expr_prop 4904, 5024, 5032, 5067, 5075, 5097, 5107, 5109, 5121, 5123, 5143, 5146, 5152
__stex_expr_prop_do_decls: 5079, 5082
__stex_expr_prop_do_notations: . 5099, 5161
__stex_expr_range:w 4627, 4638
\l__stex_expr_redo_tl 5051, 5078, 5101, 5155, 5172, 5183
\l__stex_expr_reset_tl 4526, 4530, 4534, 4535, 4541
__stex_expr_ret_cs: 5434, 5439, 5470, 5475, 5480
__stex_expr_return: 5449, 5452
__stex_expr_return_arg:n 5436, 5442
\l__stex_expr_return_args_tl ... 5432, 5443, 5448, 5457, 5477, 5482
__stex_expr_return_next: 5437, 5446
\l__stex_expr_return_this_tl ... 5431, 5448, 5453, 5457, 5463, 5466, 5476, 5484
\l__stex_expr_seq 5076, 5108, 5122, 5163
__stex_expr_seq_arg:n ... 4641, 4652
__stex_expr_seq_first: .. 4615, 4671
__stex_expr_seq_op:w ... 4614, 4618
\l__stex_expr_set_comp_tl 4702, 4757, 4761, 4921, 5046
__stex_expr_set_custom_comp:n .. 4762, 4937, 4955
__stex_expr_set_custom_i:nn ... 4958, 4963
__stex_expr_set_customcomp: ... 4735, 4760
__stex_expr_set_this:n .. 4992, 5001
__stex_expr_set_thiscomp: 4702, 4706
__stex_expr_set_thisnotation: .. 4739, 4755
__stex_expr_setup:nnnnn 4578, 4585, 5195
__stex_expr_struct_top:n . 4703, 4712, 4736, 4740, 4743, 4746, 4748
__stex_expr_struct_type:n 4719, 4766
__stex_expr_text: 4605, 5241
__stex_expr_varseq_in_map:nnnnnnnn 5785, 5831
__stex_groups_do: 340, 346, 354
__stex_groups_do_in:n 329, 333
\l__stex_groups_lvl_int 323, 325, 328
\l__stex_import_archive_str 2796, 2799, 2803, 2871
__stex_import_check_file:nnn ... 2815, 2816, 2817, 2831, 2832, 2833, 2860
\l__stex_import_doc_uri .. 2870, 2877
\l__stex_import_file_str 2783, 2813, 2843, 2845, 2848, 2853, 2864, 2877
__stex_import_import_module:nn . 2904, 2912, 2949
__stex_import_import_module_i:n 2915, 2918
__stex_import_import_module_-presms:nn 2938, 2949
\l__stex_import_language_str ... 2814, 2818, 2834, 2874
__stex_import_load_check:n 2801, 2807, 2812
__stex_import_load_check_i:n ... 2820, 2827
__stex_import_load_check_ii: ... 2824, 2842
__stex_import_load_file: 2849, 2854, 2869
__stex_import_load_module:NTF .. 2773, 2782, 2794
\l__stex_import_name_str 2798, 2815, 2816, 2817, 2829, 2873
\l__stex_import_path_str 2797, 2800, 2805, 2828, 2830, 2872
\l__stex_import_pre_str 2800, 2804, 2861, 2862, 2864
__stex_import_requiremodule: ... 2954, 2959
\l__stex_import_uri 2790, 2791, 2795, 2878, 2879
__stex_import_usemodule: 2885, 2888
\l__stex_inputs_gin_repo_str ... 2210, 2213, 2218

\l__stex_inputs_id_seq 2147, 2148, 2153, 2161
\l__stex_inputs_libinput_files_seq 2076, 2077, 2080, 2086, 2087, 2108, 2109, 2133, 2134, 2137, 2145, 2157, 2158, 2168, 2169
\l__stex_inputs_path_seq 2146, 2149, 2151, 2154, 2162, 2165
\l__stex_inputs_path_str 2154, 2155, 2156, 2157, 2158, 2161, 2162, 2165, 2166, 2167, 2168, 2169
__stex_inputs_ref:n 2008, 2029, 2039
__stex_inputs_ref_i:n 2015, 2031, 2034
\c__stex_inputs_ref_post_str 2014, 2022
\c__stex_inputs_ref_pre_str 2013, 2020
\l__stex_inputs_tmp_str .. 2087, 2089
__stex_inputs_up_archive:nn 2075, 2085, 2107, 2132, 2141, 2141
\l__stex_inputs_uri .. 2007, 2021, 2026, 2045, 2046, 2052, 2054, 2057
__stex_inputs_usetikzlibrary:n 2100, 2102, 2106
__stex_inputs_usetikzlibrary_i:nn 2109, 2115
__stex_keys_split_at_bracket:w 414, 422
\l__stex_keys_tl 364, 365, 366, 368, 371, 372
\l__stex_lang_turkish_bool 455, 463, 473
__stex_mathhub: 975, 999
\l__stex_mathhub_base_str 983, 993, 998, 999, 1001, 1005
\l__stex_mathhub_bool 941, 942, 944, 947, 956, 958, 967
__stex_mathhub_check_manifest: 946, 954
__stex_mathhub_check_manifest:n 955, 957, 959, 964
\l__stex_mathhub_cs 885, 888, 890, 894, 898, 899, 904
__stex_mathhub_do_manifest:nn 919, 926, 1010
\l__stex_mathhub_file_str . 981, 1001
__stex_mathhub_find_manifest:n 931
__stex_mathhub_find_manifest:nn 928, 937, 1034
\l__stex_mathhub_id_str 982, 992, 1001, 1005
\l__stex_mathhub_key .. 988, 989, 991
\c__stex_mathhub_manifest_ior 971, 980, 985, 997
\l__stex_mathhub_manifest_str 929, 932, 938, 968, 980, 1035
__stex_mathhub_parse_manifest:n 933, 979, 1038
__stex_mathhub_replace_https:w 975, 999
__stex_mathhub_require:n .. 906, 913
__stex_mathhub_restore:nn 1005, 1009, 1048
\l__stex_mathhub_seq ... 940, 943, 948, 965, 966, 968, 981, 987, 988, 990
__stex_mathhub_set_current:n 893, 905
\l__stex_mathhub_str 823, 824, 828, 829, 834, 840, 843, 850, 852, 853, 854, 859
\l__stex_mathhub_tl 948
\l__stex_mathhub_val .. 990, 992, 993
__stex_modules_export:n . 2552, 2554
__stex_modules_html_annots: 2277, 2298
__stex_modules_load_meta: 2320, 2338, 2340
__stex_modules_load_meta_i:n 2342, 2346
__stex_modules_load_sig: 2365, 2370
__stex_modules_macro_short:nnn 2225, 2229, 2232, 2235, 2238, 2241, 2244, 2247, 2250
__stex_modules_nested:n 2311, 2382, 2382
__stex_modules_persist_module: 2399, 2408, 8020
__stex_modules_persist_nots_i:nn 2424, 2429
__stex_modules_restore_module:nnnn 2410, 2446
__stex_modules_restore_nots:n 2460, 2465
__stex_modules_restore_nots_i:n 2466, 2469, 2488
__stex_modules_restore_nots_ii:nnnnn 2473, 2477
\l__stex_modules_sig 2372, 2376, 2377
\l__stex_modules_sigfile 2371, 2376, 2378
__stex_modules_stringify_uri:nnn 2434, 2447
__stex_modules_stringify_uri:nnnn 2439, 2484
\l__stex_modules_tl . 2478, 2480, 2485
__stex_modules_top:n ... 2311, 2314

__stex_modules_top_sig:n 2316, 2355, 2355
\l__stex_modules_uri . 2325, 2326, 2327, 2330, 2331, 2332, 2333, 2335, 2336, 2358, 2359, 2360, 2362, 2367, 2380, 2385, 2386, 2387, 2390, 2391, 2392, 2393, 2395, 2396, 2447, 2448, 2449, 2450, 2451, 2455, 2456, 2482
__stex_morphisms_add_definiens:nn 3204, 3209, 3268
__stex_morphisms_add_symbol:nnnnnnnnN 3121, 3137, 3139, 3144
__stex_morphisms_add_symbol_-i:nnnnnnnN 3151, 3154
\l__stex_morphisms_ass_tl 3554, 3570, 3574, 3585, 3586, 3587
__stex_morphisms_begin_copy:Nnnn 3392, 3404, 3415
__stex_morphisms_begin_interpret:Nnnn 3457, 3470, 3482
__stex_morphisms_break: . 3212, 3216
__stex_morphisms_check_name:nnnn 3332, 3335
\l__stex_morphisms_continue_bool 3273, 3276, 3289, 3308
__stex_morphisms_cs:nnnn 3274, 3299
__stex_morphisms_definiens_-impl:nn 3184, 3269
__stex_morphisms_do:n 3037, 3050, 3065
__stex_morphisms_do_decls: 3036, 3040
__stex_morphisms_do_elaboration: 3079, 3082
__stex_morphisms_do_for_list: 3174, 3267
__stex_morphisms_do_morph:nnnn 3056, 3061
__stex_morphisms_do_morph_-assign:nnn 3635, 3638, 3679
__stex_morphisms_do_parsed_-assign: 3576, 3579
__stex_morphisms_do_parsed_-newname: 3583, 3591
__stex_morphisms_do_parsed_-newname:w 3593, 3595, 3599
\l__stex_morphisms_dones_seq 3051, 3053, 3054
__stex_morphisms_elab:nn 3094, 3112
__stex_morphisms_elab_check_-assign:nnnnn 3147, 3158
__stex_morphisms_elab_check_-rename:nnn 3115, 3166
__stex_morphisms_elab_i:nnn 3118, 3133
__stex_morphisms_end: ... 3583, 3599
\l__stex_morphisms_feature_str 3015, 3100
__stex_morphisms_get_check:nn 3316, 3331, 3377, 3385
\l__stex_morphisms_get_str 3314, 3337, 3358, 3368, 3373, 3375, 3381, 3383
__stex_morphisms_gobble:nnnnnnN 3361, 3365
__stex_morphisms_iterate:nn 3279, 3283, 3292, 3294
__stex_morphisms_macro:nnnnnnnN 3341, 3357
\l__stex_morphisms_mods_seq 3272, 3296, 3297
__stex_morphisms_morphism:nnnnn 2986, 2990, 2993
\l__stex_morphisms_morphism_dom_-str ... 3629, 3642, 3654, 3664, 3681
\l__stex_morphisms_name_str 3552, 3561, 3562, 3563, 3567, 3568, 3569, 3580, 3582, 3586
\l__stex_morphisms_newname_str 3553, 3572, 3573, 3581, 3582, 3583
\l__stex_morphisms_next_tl 3559, 3564, 3566
__stex_morphisms_parse_assign:n 3441, 3507, 3551
__stex_morphisms_reactivate: 3027, 3256
__stex_morphisms_rename:n 3101, 3104
__stex_morphisms_rename:nn 3105, 3108
__stex_morphisms_rename_all: . 3229
__stex_morphisms_renamed_-check:nn 3320, 3367
__stex_morphisms_renamed_-check:nnnnnn 3369, 3372
\l__stex_morphisms_seq 3556, 3557, 3559, 3561, 3564, 3566, 3568, 3570, 3572, 3574
__stex_morphisms_set:nnnnnnN 3339, 3345, 3360
__stex_morphisms_set_definiens_-macros: 3212, 3216
__stex_morphisms_set_definiens_-macros_i:nnnnnnn 3219, 3224
__stex_morphisms_setup: . 3026, 3031
__stex_morphisms_split_qm:w . 3252
\l__stex_morphisms_tmp 3113, 3117, 3120, 3121, 3127, 3134, 3169

\l__stex_morphisms_tmp_b 3145, 3149, 3161
\l__stex_morphisms_todo_tl 3278, 3282, 3295, 3308
__stex_morphisms_total_check:nn 3236, 3240
__stex_morphisms_total_check:nnnnnnnN 3241, 3244
\l__stex_morphisms_with_macros_bool ... 2982, 2985, 2989, 3075, 3136
__stex_notations_activate_not:nn 6433, 6445
__stex_notations_add:nnnnn 6764, 6769, 6774
__stex_notations_add_last:nnnnn 6757, 6768
__stex_notations_add_missing_args:nn 6399, 6402
__stex_notations_add_next:nnnnnn 6759, 6763
\l__stex_notations_after_tl 6744, 6771
__stex_notations_args_end: 6478, 6481, 6484, 6491, 6494, 6497, 6752, 6755, 6765
\l__stex_notations_args_tl 6663, 6665, 6678
__stex_notations_augment_arg:nn 6679, 6692
\l__stex_notations_bind_bool 6198, 6200
__stex_notations_check_aB_arg:Nn 6806, 6815, 6825, 6850
\l__stex_notations_clist_count_int 6848, 6856, 6862
\l__stex_notations_code_tl 6702, 6717, 6732, 6746, 6747, 6770, 6778, 6786, 6791, 6799
__stex_notations_complex:nnnnnn 6711, 6728
__stex_notations_const_precs: 6272, 6312
\l__stex_notations_cs 6718, 6723, 6733, 6742
__stex_notations_default_args: 6664, 6675
__stex_notations_do_argname:nn 6373, 6376
__stex_notations_do_argnames: 6348, 6371
__stex_notations_do_missing_args:n 6278, 6391
__stex_notations_fun_precs: 6277, 6324
__stex_notations_it_not_check:nnnn 6564, 6568
__stex_notations_it_not_i:n 6553, 6557, 6570
\l__stex_notations_left_bracket_str 6879, 6907, 6917, 6921
__stex_notations_macro:nn 6416, 6464, 6465, 6466, 6501, 6502, 6503, 6521, 6532, 6533, 6574, 6575
__stex_notations_make_arg:nnnn 6642, 6645
__stex_notations_make_arg_html:nn 6622, 6638
__stex_notations_make_name: 6671, 6676, 6696
__stex_notations_map_args_i:w 6477, 6481, 6484
__stex_notations_map_args_ii:w 6490, 6494, 6497
__stex_notations_map_cs: 6607, 6609, 6828, 6830, 6838, 6853, 6855, 6866
\l__stex_notations_missing_str 6392, 6393, 6394, 6396, 6404
\l__stex_notations_missing_tl 6307, 6398, 6405
\l__stex_notations_mods_seq 6549, 6558, 6559
\l__stex_notations_name_str 6672, 6698
__stex_notations_not_cs:nnnnn 6550, 6551, 6561
__stex_notations_op_macro:nn ... 6419, 6469, 6470, 6471, 6507, 6508, 6509, 6524, 6534, 6535, 6580, 6581
\l__stex_notations_opprec_tl 6301, 6314, 6317, 6319, 6326, 6329, 6331, 6335, 6342, 6355, 6361, 6367, 6598
__stex_notations_parse_args:nnnnw 6752, 6755, 6765
__stex_notations_parse_precs: 6346, 6351
\l__stex_notations_pre_tl 6731, 6746, 6783, 6796
\l__stex_notations_precs_seq 6269, 6337, 6344, 6365, 6367, 6379, 6385, 6599
__stex_notations_process:nnnnnn 6701, 6707
\l__stex_notations_right_bracket_str . 6880, 6912, 6918, 6922
\l__stex_notations_seq 6353, 6354, 6356, 6357, 6364

__stex_notations_set_macro:nnnnn 6432, 6441, 6454, 6463
__stex_notations_simple:nnnnn 6709, 6715
\l__stex_notations_str 6354, 6355, 6356, 6358, 6364, 6365
__stex_notations_styledefs: 6220, 6231
__stex_others_newlabel:n 8126, 8134
__stex_others_old_newlabel: 8128, 8133
__stex_path_: 148, 158
\l__stex_path_a_seq . . . 280, 286, 292
\l__stex_path_a_tl 292, 294
\l__stex_path_b_seq 281, 287, 293, 299
\l__stex_path_b_tl 293, 294
\l__stex_path_can_seq 180, 183, 188, 196, 197, 200
\l__stex_path_can_str 179, 181, 185, 197
__stex_path_canonicalize:N 161, 172, 177
__stex_path_dodots:n . 182, 186, 192
\l__stex_path_return_tl 282, 288, 295, 298, 300
\l__stex_path_seq 211, 212, 213, 218, 219, 221, 223, 226, 251, 252
\l__stex_path_str 149, 150, 151, 154, 156, 157, 158, 160, 163, 170, 171, 210, 211, 212, 217, 218, 219, 221, 222, 223, 229, 231, 233, 239, 241, 243
\l__stex_path_win_drive 150, 155, 162, 164
__stex_path_win_take:w 148, 158
__stex_persist_env_str 568, 569, 570, 574, 575, 576
__stex_persist_load_file:n 591, 598, 630
__stex_persist_read_and_write: 589, 615
\c__stex_persist_sms_iow 585, 611, 612, 627, 715
__stex_persist_write_only: 594, 610, 624, 631
__stex_proof_add_counter: 7538, 7797
__stex_proof_begin_proof:nn 7564, 7674, 7696
\l__stex_proof_counter_intarray 7495, 7501, 7504, 7513, 7516, 7525, 7533, 7534, 7542, 7547, 7554, 7560, 7565, 7566
\l__stex_proof_counter_str 7457, 7458, 7468, 7617, 7632, 7732, 7750, 7751, 7905, 7920, 7945
__stex_proof_do_after_subgroup:N 7721, 7755
__stex_proof_do_after_subgroup_-i:nn 7723, 7727
__stex_proof_do_spf_name: 7448, 7877
__stex_proof_do_spf_name_i: 7462, 7725
__stex_proof_do_spf_varname: 7472, 7796, 7967, 7968
__stex_proof_end_comment: 7662, 7669, 7690, 7830, 7838, 7872, 7965, 7971
__stex_proof_end_list: 7680, 7715, 7821, 7825, 7890, 7978
__stex_proof_html: 7612, 7635, 7638, 7923, 7948
__stex_proof_html_env_proof: 7570, 7602
__stex_proof_html_env_subproof: 7615, 7763
\l__stex_proof_in_spfblock_bool 7562, 7657, 7663, 7675, 7697, 7747, 7775, 7787, 7814, 7821, 7825, 7886, 7958, 7974
__stex_proof_inc_counter: 7521, 7816, 7825, 7967, 7968
\l__stex_proof_inc_counter_bool 7494
__stex_proof_insert_number: 7497, 7796, 7967, 7968
\l__stex_proof_key_name_str 7464, 7465, 7729
\l__stex_proof_key_numname_str 7463, 7468, 7730
\l__stex_proof_key_showname_bool 7731
__stex_proof_make_step_macro:Nnnnn 7870, 7967, 7968, 7969
__stex_proof_number_as_string:N 7457, 7508, 7617, 7750, 7779, 7792, 7881, 7905
__stex_proof_proof_box_tl 7479, 8004
__stex_proof_remove_counter: 7550, 7815
__stex_proof_set_after_subgroup:n 7734, 7776, 7803
__stex_proof_set_after_subgroup_-i:n 7736, 7737, 7744
__stex_proof_set_after_subgroup_-with_number:n 7739, 7789
__stex_proof_set_after_subgroup_-with_number_i:n . 7741, 7742, 7749
__stex_proof_set_aftergroup: 7747, 7752, 7754

__stex_proof_set_aftergroup_double: 7747
__stex_proof_start_comment: 7656, 7669, 7677, 7812, 7834, 7894, 7960, 7981
__stex_proof_start_list:n 7676, 7706, 7796, 7890, 7978
__stex_proof_step_html:nn 7890, 7903, 7978
__stex_proof_step_html_i:nn 7888, 7896, 7930, 7976, 7983
__stex_refs_check_i:nnnn 1864, 1872
__stex_refs_check_in:nnnn 1946, 1954
\l__stex_refs_default_archive 1973, 1978, 1980, 1982
\l__stex_refs_default_file 1886, 1890, 1891, 1894, 1972, 1976, 1983
\l__stex_refs_default_title 1889, 1971, 1985
\l__stex_refs_file 1846, 1847, 1865, 1890, 1902, 1911, 1914, 1943, 1947
__stex_refs_find: 1848, 1862
__stex_refs_find_in: . . . 1912, 1942
\l__stex_refs_in 1837, 1882, 1900, 1904
__stex_refs_in: 1895, 1905, 1909
__stex_refs_in_text:nn . . 1922, 1928
__stex_refs_in_text:w . . . 1929, 1936
\c__stex_refs_iow 1760, 1761, 1762, 1783
\c__stex_refs_iow_str . . . 1763, 1891
\l__stex_refs_key 1836, 1873
__stex_refs_local:n 1808, 1814, 1843
__stex_refs_local_ref:n 1818, 1824, 1884, 1887, 1892
__stex_refs_maybe_in: . . . 1856, 1881
\l__stex_refs_new_tl 1777, 1778, 1779, 1781, 1786
__stex_refs_remote:nn . . . 1810, 1835
__stex_refs_stop: 1929, 1936
\l__stex_refs_str 7296
\l__stex_refs_tmp 1806, 1850, 1854, 1868, 1875, 1910, 1916, 1920, 1921, 1922, 1924, 1950, 1956
\l__stex_refs_unnamed_counter_int 1960
__stex_refs_uppercase:n . 1937, 1940
\l__stex_refs_uri 1839, 1840, 1841, 1845, 1854, 1855, 1874, 1883, 1884, 1887, 1892, 1924, 1943, 1955
__stex_seqs_add: 4428, 4443
\l__stex_seqs_args_tl . . . 4498, 4500
\l__stex_seqs_cs 4496, 4500
__stex_seqs_html: 4426, 4470
__stex_seqs_macro: 4429, 4457
\l__stex_seqs_range_clist 4419, 4451, 4464
\g__stex_smsmode_allowed_escape_tl 2590, 2593, 2720
\g__stex_smsmode_allowed_import_env_seq 2615, 2618, 2743, 2746
\g__stex_smsmode_allowed_import_tl 2608, 2612, 2738
\g__stex_smsmode_allowed_tl 2585, 2588, 2716
\g__stex_smsmode_allowedenvs_seq 2595, 2598, 2725, 2728
\g__stex_smsmode_bool 2624, 2627, 2667
__stex_smsmode_check_begin:Nn 2725, 2743, 2754
__stex_smsmode_check_cs:N 2689, 2695
__stex_smsmode_check_cs:NNn 2688, 2702
__stex_smsmode_check_end:Nn 2728, 2746, 2762
\l__stex_smsmode_cycles_seq 2630, 2636, 2638
__stex_smsmode_do:w 2684, 2686, 2688, 2691, 2699, 2718, 2730, 2748, 2759, 2765, 2767
__stex_smsmode_do_aux:N 2688, 2698, 2709
__stex_smsmode_do_aux_curr:N 2647, 2655, 2711
__stex_smsmode_do_aux_imports:N 2647, 2737
__stex_smsmode_do_aux_normal:N 2655, 2715
\g__stex_smsmode_import_setup_tl 2609, 2613, 2619, 2648
\l__stex_smsmode_importmodules_seq 2633
__stex_smsmode_in_smsmode:n 2645, 2664
\l__stex_smsmode_sigmodules_seq 2634
__stex_smsmode_smsmode_do: 2677, 2684
__stex_smsmode_start_smsmode:n 2649, 2656, 2675
__stex_statements_add_definiens:n 7374, 7379
__stex_statements_definiens_inner:nnnnnnnN 7383, 7387
__stex_statements_force_id: 7265, 7293, 7313, 7337
__stex_statements_html_keyvals:nn 7187, 7221, 7232
__stex_statements_make_macro:nnn 7211, 7214

__stex_statements_setup:n 7250, 7250, 7314, 7318, 7333, 7338, 7341
__stex_statements_setup_def: ... 7299, 7299, 7315, 7339
__stex_statements_setup_named:n ... 7260, 7264
__stex_statements_setup_noname: ... 7259, 7283
__stex_structures_begin:nn ... 6943, 6953, 6987, 7028
__stex_structures_check_-def:nnnnnnnn ... 7046, 7089
__stex_structures_do_externals: ... 6947, 6973, 7002, 7052
\l__stex_structures_exstruct_-name_str ... 7025, 7028, 7050, 7099
__stex_structures_extend_-i:nnnnnnnNn ... 7071, 7085
__stex_structures_extend_-ii:nnnn ... 7070, 7077
__stex_structures_extend_-structure:nnnn ... 7025, 7062
\l__stex_structures_extname_-count ... 7097, 7099, 7102
__stex_structures_get:w . 7128, 7134
\l__stex_structures_imports_seq 6978, 6983, 6988, 7013, 7018, 7030, 7138, 7141, 7144
\l__stex_structures_last_name_-str ... 7022, 7025
\l__stex_structures_name_str ... 6931, 6936, 6938, 6955, 6956, 6961, 6965, 6970, 7110, 7113
__stex_structures_new_extstruct_-name: ... 7012, 7095
\l__stex_structures_parent_uri 7022, 7025
\l__stex_structures_replace_-this_tl ... 6974
\l__stex_structures_struct_uri 6960, 6965
__stex_styles_apply_patch:n 488, 503
\g__stex_styles_counters_seq ... 24, 63, 64, 746, 785, 786
__stex_styles_css_patch:nnnn ... 16, 97, 521, 738, 819
\l__stex_styles_first_str ... 60, 62, 81, 782, 784, 803
__stex_styles_patch:nnn ... 484, 495
__stex_styles_patch:nnnn ... 4, 88, 518, 726, 810
__stex_styles_patch_html: ... 34, 74, 77, 756, 796, 799
__stex_styles_patch_html_c: ... 26, 66, 69, 748, 788, 791
__stex_styles_patch_i:nnn ... 21, 42, 743, 764
__stex_syms_activate_i:nnnnnnnn ... 4028, 4041, 4045
__stex_syms_add_decl: ... 3880, 3883
\l__stex_syms_args_tl ... 4001, 4003
\l__stex_syms_cs 3999, 4003, 4123, 4125, 4127, 4151
__stex_syms_def_style: .. 3765, 3773
__stex_syms_do_args: ... 3900, 3907
__stex_syms_get_from_one_-string:n ... 4160, 4234
__stex_syms_get_symbol_from_cs: ... 4129, 4140
__stex_syms_get_symbol_from_-modules:nn ... 4162, 4193
__stex_syms_get_symbol_from_-string:n ... 4130, 4132, 4135, 4155
__stex_syms_has_definiens:nnnnnnnN ... 4059, 4066
__stex_syms_it_decl_check:nnnn 4099, 4107
__stex_syms_it_decl_i:n ... 4091, 4095, 4109
__stex_syms_maybe_activate:nnnnnnnn ... 4035, 4039
\l__stex_syms_mods_seq ... 4087, 4096, 4097
\l__stex_syms_name ... 4158, 4164
\l__stex_syms_name_str ... 4195, 4207, 4209
__stex_syms_parse_arity: 3899, 3918
\l__stex_syms_path_str ... 4196, 4206, 4213, 4218, 4219
\l__stex_syms_seq ... 4157, 4158, 4159, 4163
__stex_syms_set_textsymdecl_-macro:nnn ... 3843, 3862
__stex_syms_style: ... 3734, 3742
__stex_syms_sym_cs:nnnnnnnnN 4070, 4071, 4081, 4088, 4089, 4102
__stex_syms_sym_from_str_i:nnnn ... 4168, 4202, 4215, 4221
__stex_syms_sym_i_finish:nnnnnnnN ... 4174, 4181
__stex_syms_sym_i_gobble:nnnnnn ... 4176, 4179
__stex_syms_top:n ... 3733, 3756, 3870, 3870
__stex_syms_uri_match:n ... 4229
__stex_uris_absolute:N ... 1293, 1324, 1328

\l__stex_uris_archive 1142, 1146, 1149
__stex_uris_as_qm:nnn ... 1255, 1257
__stex_uris_doc:nnnnn 1090, 1093, 1095
__stex_uris_doc_from_archive_-file:NN 1136, 1139, 1198, 1200
__stex_uris_end: ... 1366, 1369, 1373
\l__stex_uris_file .. 1134, 1135, 1136
__stex_uris_first_three:nnnn 1403, 1406, 1409
\l__stex_uris_lang_str 1152, 1161, 1163, 1164, 1169, 1170, 1172, 1173, 1174, 1175, 1177, 1178, 1179
__stex_uris_maybe_relative:N 1293, 1301
__stex_uris_module:nnn 1208, 1211, 1213
\l__stex_uris_name_str 1151, 1159, 1160, 1163, 1184, 1286, 1311, 1317, 1333, 1348, 1357
__stex_uris_new_mod:nnnnnn 1264, 1268
__stex_uris_new_nested_mod:nnnn 1262, 1276
__stex_uris_pair_in_archive:Nn 1296, 1353
__stex_uris_pair_no_archive:N 1337, 1341
__stex_uris_parent:NNnnn 1237, 1241, 1244
__stex_uris_parse_i:Nw .. 1363, 1369
__stex_uris_parse_ii:Nw . 1365, 1373
\l__stex_uris_path_seq 1150, 1158, 1159
\l__stex_uris_path_str 1287, 1289, 1292, 1332, 1343, 1356
__stex_uris_split_file:N 1140, 1157
__stex_uris_split_file_i: 1165, 1168
__stex_uris_split_file_ii: .. 1187
__stex_uris_symbol:nnnn 1377, 1380, 1382
__stex_uris_with_elem:nnnnnn 1128, 1130
__stex_uris_with_language:nnnnnn 1122, 1124
__stex_vars_add: 4269, 4295
\l__stex_vars_args_tl ... 4354, 4356
\l__stex_vars_bind_bool .. 4257, 4338
__stex_vars_check_var:nnnnnnnnN 4381, 4385
\l__stex_vars_cs 4352, 4356
__stex_vars_get_var:n ... 4373, 4379
__stex_vars_html: 4271, 4321
__stex_vars_macro: 4270, 4308
__stex_vars_set_vars:nnnnnnN 4387, 4390, 4395
stex_annotate_env (env.) *128*, 694
stex_env_node (env.) *128*, 694
\stexcommentfont *159*, 7659, 8015
\stexdoctitle *138*, 1446, 1549, 1561, 1587, 2294, 8490, 8666, 8945, 9045, 9048, 10117
\STEXexport ... *43*, *88*, *89*, *143*, 2550, 2607
\STEXftmllink *138*, 1419
\stexhtmlfalse 677
\stexhtmltrue 674
\STeXInternalNewSRefLabel 1779, 1781, 1967
\STEXInternalNotation *154*, 6299, 6665, 6700
\STeXInternalSRefLabel 1784, 1864, 1946, 1965
\STEXinvisible ... *83*, *129*, 694, 6405, 7575, 7577, 7768, 7770, 8043, 8044
\STEXlinkftml *138*, 1419
\STEXRestoreNotsEnd 2425, 2463, 2470
\STEXsetlinkftml *138*, 1419
\stexstyle⟨*environmentname*⟩ *127*
\stexstyle⟨*macroname*⟩ *127*
\stexstyleassertion *106*
\stexstyleassign *106*
\stexstyleassignMorphism *106*
\stexstylecopymod *106*
\stexstylecopymodule *106*
\stexstyledefinition *106*
\stexstyleexample *106*
\stexstyleextstructure *106*
\stexstylegnote 10003
\stexstyleimportmodule *106*
\stexstyleinterpretmod *106*
\stexstyleinterpretmodule *106*
\stexstylemathstructure *106*
\stexstylemcb 9424, 9428
\stexstyleMMTinclude *106*
\stexstylemodule *106*
\stexstylenotation *106*
\stexstyleparagraph *106*, *127*
\stexstyleproblem 9883, 9901
\stexstyleproof *106*
\stexstylerealization *106*
\stexstylerealize *106*
\stexstylerenamedecl *106*
\stexstylerequiremodule *106*
\stexstylescb 9589, 9593
\stexstylespfsketch *106*
\stexstylesubproblem 9943, 9962
\stexstylesubproof *106*
\stexstylesymdecl *106*
\stexstylesymdef *106*
\stexstyletextsymdecl *106*

\stexstyleusemodule *106*
\stexstylevardef *106*
\stexstylevarnotation *106*
\stexstylevarseq *106*
\stopsolutions *114*, 9162
str commands:
\c_ampersand_str . 1098, 1100, 1101, 1103, 1216, 1218, 1385, 1387, 1388
\c_backslash_str 156, 1829
\c_colon_str 987
\c_dollar_str 6386
\c_hash_str 6393, 6404, 6425
\c_left_brace_str 9922, 9989, 10009, 10034
\c_percent_str 89, 90, 239
\c_right_brace_str 9939, 9999, 10015, 10038
\str_case:Nn 991
\str_case:nn 5346, 6776
\str_case:nnTF 48, 770, 1646, 3922, 5373, 5401, 6646, 8250, 8812, 8835
\str_clear:N 6, 7, 8, 91, 155, 379, 380, 387, 405, 728, 729, 730, 834, 938, 1287, 2213, 2256, 2813, 3552, 3553, 3629, 3694, 3695, 3696, 3697, 3707, 3708, 3712, 3729, 3791, 3833, 3834, 4372, 6175, 6176, 6177, 6178, 6394, 6931, 7154, 7155, 7157, 7439, 7440, 7509, 7866, 8384, 8722, 8723, 8879, 8880, 9084, 9101, 9322, 10245, 10246, 10247
\str_const:Nn 1020, 8169
\str_count:n 59, 64
\str_gput_right:Nn 9846
\str_gset:Nn 7732
\str_gset_eq:NN 829, 7729, 7730
\str_if_empty:NTF 43, 61, 120, 162, 181, 458, 569, 575, 635, 641, 765, 783, 822, 824, 843, 929, 932, 1035, 1292, 1314, 1807, 2281, 2315, 2568, 2783, 2843, 3567, 3581, 3642, 3805, 3871, 3974, 3977, 3980, 3983, 3986, 4206, 4213, 4263, 4326, 4329, 4332, 4335, 4374, 4410, 4413, 4474, 4477, 4480, 4483, 5026, 6123, 6187, 6313, 6325, 6334, 6380, 6935, 6955, 7238, 7251, 7252, 7259, 7294, 7449, 7450, 7463, 7464, 7616, 7627, 7628, 7735, 7736, 7740, 7741, 7778, 7791, 7874, 7880, 7904, 7915, 7916, 7940, 7941, 8240, 8304, 8481, 8933, 9028, 9033, 9110, 9153, 9173, 9221, 9271, 9312, 9331, 9344, 9362, 10005, 10022, 10048, 10060, 10063, 10269, 10273, 10284
\str_if_empty:nTF 89, 142, 193, 496, 811, 3616, 4040
\str_if_eq:NNTF 294, 1891, 2799
\str_if_eq:nnTF 58, 63, 90, 157, 194, 195, 266, 462, 570, 576, 653, 1003, 1270, 1311, 1873, 1955, 2845, 3125, 3277, 3337, 3358, 3373, 3381, 3634, 3650, 4209, 4230, 4386, 4389, 4659, 6099, 6109, 6124, 6316, 6328, 6340, 6450, 7373, 8308, 8311, 8376, 8554, 9434
\str_if_eq_p:nn 4170, 4171, 4239, 4240
\str_if_in:NnTF 6051, 6075, 6404
\str_item:Nn 157, 3912
\str_item:nn 266
\str_lowercase:n 9434
\str_map_break: 3923
\str_map_break:n . 3924, 3928, 3932, 3936, 3940, 3944, 3948, 3952, 3956
\str_map_function:nN 9856
\str_map_inline:Nn 3921
\str_new:N 453, 1448, 1972, 1973, 3724, 9838
\str_put_left:Nn 62, 784
\str_put_right:Nn 7516
\str_range:Nnn 2089
\str_range:nnn 59, 64
\str_replace_all:Nnn 156
\str_set:Nn 13, 14, 44, 46, 50, 51, 52, 53, 54, 55, 56, 60, 83, 94, 149, 150, 151, 154, 170, 254, 459, 461, 735, 766, 768, 772, 773, 774, 775, 776, 777, 778, 782, 864, 968, 973, 981, 982, 983, 989, 990, 999, 1142, 1179, 1184, 1191, 1204, 1450, 1496, 1763, 1836, 1845, 1846, 1875, 1902, 1976, 1978, 1980, 1983, 1985, 2013, 2014, 2087, 2154, 2165, 2176, 2181, 2192, 2197, 2207, 2210, 2214, 2300, 2301, 2804, 2818, 2830, 2834, 2864, 2896, 2897, 2931, 2932, 2967, 2968, 3011, 3015, 3217, 3218, 3314, 3347, 3375, 3383, 3563, 3569, 3573, 3681, 3702, 3731, 3755, 3804, 3806, 3808, 3810, 3872, 3926, 3930, 3934, 3938, 3942, 3946, 3950, 3954, 3958, 4196, 4262, 4264, 4398, 4409, 4411, 4414, 4591, 5022, 5027, 6050, 6052, 6076, 6188, 6233, 6392, 6936, 6956, 6967, 7097, 7099, 7102, 7270, 7780, 7793, 7882, 8294, 9112, 9175, 9223, 9273, 9333, 10024, 10268, 10270

\str_set_eq:NN 840, 992, 993, 1146, 1163, 1164, 1170, 1175, 1854, 1890, 2800, 2814, 3016, 3758, 3781, 4286, 4435, 6190, 6232, 6244, 6254, 6938, 7113, 7190, 7253, 7269, 7296, 7719, 7955
\str_uppercase:n 1512, 8376
\l_tmpa_str 461, 465, 466, 467, 469, 973, 975, 1191, 1192, 3011, 3013, 3016, 3020
\string 1842, 9832, 9873
\subparagraph *26*, *84*
subproblem (env.) 9015
subproof (env.) *158*, 7719
\subproof 7582, 7842
\subproofautorefname 7719
\subsection *25, 26*, *84*, *139*
\subsubsection *26*, *84*
\svar *97*, *150*, 3904, 5205, 5805, 5807
\symbol *91*
\symdecl *22*, *30, 31*, *36*, *41*, *89–91*, *101*, *106*, *143*, *147*, *149*, *257*, 3257, 3724, 8056, 8058, 8073, 8081, 8095, 8098
\symdef *31–33*, *37*, *90*, *97*, *143*, *147*, *149*, *257*, 3259, 3753, 8038, 8041, 8046, 8049, 8051, 8053, 8055, 8065, 8066, 8067, 8068, 8079, 8091, 8093, 8102, 8103, 8106, 8108, 8111, 8113, 8115
\Symname *91*, *102*, *152*, 5925
\symname *18, 19*, *23*, *56*, *91, 92*, *102*, *104*, *152*, 5925
\symref *18, 19*, *23*, *42*, *90, 91*, *95*, *102*, *104*, *152*, 5918
\symrefemph *104, 105*, *151*, 5879
\symuse *42*, *91*, *149*, 4515, 4809, 4816, 4827, 4910, 4976, 5053, 5054, 5804, 5818, 6822
sys commands:
\sys_get_shell:nnN 80
\sys_if_platform_windows:TF 86, 147, 228, 238, 262

T

\target *124*
\test *76*, *118*
test *113*
\testbigspace 9790
\testemptypage *75*, *118*
\testemptypage 9784, 10080
testheading (env.) *77*, *119*
\testheading 10182
\testmedspace 9789
\testnewpage *75*, *118*
\testnewpage 9791, 9792
\testnewpageInProblem 9792
\testsmallspace . *75*, *75*, *75*, *118*, *118*, *118*
\testsmallspace 9788
\testspace *75*, *118*
\testspace 9133, 9787
\TeX *23*
\tex *23*
TEX and LATEX 2ε commands:
\@ 2119, 2122, 2126
\@addtoreset 8895
\@arabic 8677
\@author 8656
\@auxout 1780, 8131, 9009
\@bonuspointsfalse 10213
\@bonuspointstrue 10218
\@currentHref . 1787, 7780, 7793, 7882
\@currentlabel 7779, 7780, 7792, 7793, 7881, 7882, 8480
\@currentlabelname 1788
\@currenvir 8350, 8351
\@dblarg 8649, 8658
\@gobble 48
\@ifnextchar 47
\@ifstar 8597
\@listdepth 2030
\@mainmatterfalse 1720, 1730
\@mainmattertrue 1741, 1752
\@notprerr 1657
\@onlypreamble 1657
\@rustexfalse 649
\@title 1489, 1490, 8665
\activate@excursion 8704, 8709
\bbl@loaded 8859, 9814
\beamer@shortauthor . 8652, 8654, 8669
\beamer@shorttitle . . 8661, 8663, 8672
\c@chapter 8331
\c@framenumber 8677
\c@part 8343
\compemph@uri *105*, *151*, 5835, 6059, 6613
\correction@table 10161
\defemph@uri *105*, *151*, 5868, 6082
\define@key 2174, 2190, 2205
\Gin@eheight 10334, 10337, 10342, 10347
\Gin@ewidth 8588, 8589, 10333, 10343, 10347
\Gin@exclamation . 10333, 10334, 10342
\Gin@mhrepos 2176, 2181, 2185, 2207, 2214, 2219
\hwexam@bonuspts 10215
\hwexam@checktime 10209
\hwexam@duration 10185, 10188, 10194, 10199
\hwexam@kw@due 10136
\hwexam@kw@forgrading 10177

\hwexam@kw@given 10133
\hwexam@kw@grade 10178
\hwexam@kw@probs 10178
\hwexam@kw@pts 10179
\hwexam@kw@reached 10180
\hwexam@kw@sum 10178
\hwexam@kw@testemptypage 9785, 10084
\hwexam@min 10186, 10193, 10198, 10209
\hwexam@minutes@kw 10186
\hwexam@reqpts 10195, 10200, 10212, 10216
\hwexam@tools 10196, 10201
\hwexam@totalmin 10208, 10209
\hwexam@totalpts 10207, 10216
\if@bonuspoints 10211
\if@rustex 657
\itemize@inner 8416, 8422, 8431
\itemize@label 8420, 8423, 8426
\itemize@level 8414, 8419, 8422, 8431
\itemize@outer 8415, 8419
\lst@mhrepos 2192, 2197, 2201
\ltx@ifpackageloaded . 2173, 2189, 2204, 8005, 8858, 9458, 9489, 9813
\mdf@patchamsthm 8453
\metakeys@show@keys 8417
\notesslides@slidelabel . . 8456, 8472
\ns@author 8649, 8650
\ns@title 8658, 8659
\pgf@temp 2116, 2117, 2118
\pgfkeys@spdef 2116
\pgfutil@empty 2118
\pgfutil@InputIfFileExists . . . 2125
\prematurestop@endsfragment 8349, 8352, 8358
\problem@kw@* 8855
\problem@kw@correct 9473, 9611
\problem@kw@feedback . . . 9366, 10064
\problem@kw@grading 9312, 10005
\problem@kw@hint 9212
\problem@kw@minutes 8983, 9076, 9379, 9892, 9911, 9957, 9977
\problem@kw@note 9260
\problem@kw@points 9363, 9887, 9906, 9952, 9972, 10061
\problem@kw@pts 8978, 9071, 9374
\problem@kw@solution 9152
\problem@kw@wrong 9475, 9613
\problem@restore . . . 9010, 9014, 10155
\stex@backend *128*, 647
\symrefemph@uri *104, 105*, *151*, 5879, 5922, 5945, 5962, 8822, 8845
\varemph@uri *105*, *151*, 5859, 5970, 5984, 5998
tex commands:
\tex_undefined:D 54
\texname *23*
\text *19*
\textbf *18*, 5873, 8231, 8821, 8844, 8997, 9000, 9765, 10126, 10131
\textcolor 9761
\textsymdecl . *23*, *90*, *147*, *149*, 3258, 3790
\texttt 9745, 9761
\textwarning *63*, *111*
\textwidth 8457, 8586, 8608, 10174
\the 334, 335, 337, 339, 341, 353, 355, 2119, 2120, 2121
\theassignment 10114, 10126
\thechapter 1536, 1602, 1635, 1675, 1686, 8257, 8262, 8266, 8270, 8274, 8278
\theframenumber 8480
\thepage 10082
\theparagraph 8274, 8278
\thepart 1532, 1599, 1631, 1669, 8252
\theplainsproblem 8899, 8900
\thesection 8262, 8266, 8270, 8274, 8278, 8900
\thesproblem 8896, 8900, 9000, 9010, 9884, 9902, 9924, 9949, 9968, 9991, 10114
\thesubparagraph 8278
\thesubsection . . . 8266, 8270, 8274, 8278
\thesubsubsection 8270, 8274, 8278
\this . . . *50–52*, *98*, *100*, *156*, *273*, 4588, 4926, 5010, 5011, 5016, 5041, 7103
\thisarchive 2930
\thisargs 3748, 3779
\thiscopyname 3418, 3443, 3485, 3509
\thisdeclname 3745, 3776, 3850, 6233
\thisdecluri 3744, 3775, 3849, 6234
\thisdefiniens 3747, 3778
\thisfor 7191
\thismodulename . . 2301, 2897, 2932, 2968
\thismoduleuri 2300, 2896, 2931, 2967, 3419, 3444, 3486, 3510
\thisname 7190
\thisnotation 3782, 4287, 4436, 6235, 6255
\thisnotationvariant 3781, 4286, 4435, 6232, 6254
\thisstyle 505, 508, 509, 510, 543, 546, 547, 548, 549, 557, 560, 561, 2898, 2933, 2969, 3749, 3780, 3851, 4285, 4434, 6253
\thistitle *105*, 2292, 2293, 2294, 7189, 8949, 8996, 8997, 9001, 9002
\thistype 3746, 3777
\thisvarname 4284, 4433, 6252
\throwaway *144*

\tikzinput *121*, *141*, 2219, 10326, 10331, 10357
tikzinput commands:
\c_tikzinput_image_bool 38, 10315, 10322
\times 8083, 8087
\tiny . . . *152*, 6140, 8457, 8472, 8821, 8844
\title *76*, *119*
\title 1581, 8658
titlefragment (env.) *139*, 1575
tl commands:
\tl_clear:N 282, 393, 505, 543, 1249, 1517, 1765, 1766, 1767, 1806, 1910, 2260, 2635, 2898, 2933, 2969, 2994, 3113, 3145, 3295, 3313, 3554, 3709, 3710, 3711, 3749, 3780, 3792, 3793, 3832, 3851, 3908, 4121, 4285, 4422, 4434, 4586, 4595, 5004, 5034, 5078, 5215, 5287, 5432, 5596, 5597, 5598, 5599, 5904, 5905, 5906, 5914, 6057, 6253, 6270, 6281, 6282, 6372, 6398, 6537, 6617, 6628, 6663, 6858, 6930, 7123, 7158, 7159, 7160, 7257, 7425, 7426, 7427, 7428, 7481, 7482, 7483, 8721, 8877, 9323, 9441, 9443, 9444, 9686, 10089, 10090, 10091, 10166, 10167, 10168, 10193, 10194, 10195, 10196
\tl_const:Nn 1022, 6882, 6883, 9695, 9696
\tl_count:N 5378, 5384
\tl_count:n 5682, 5692, 5698
\tl_gclear:N 2330, 2390
\tl_gput_left:Nn 365, 367, 368
\tl_gput_right:Nn 335, 2492, 2499, 2556, 2588, 2593, 2612, 2613, 2619, 2941, 8705
\tl_gset:Nn 257, 337, 360, 361, 1001, 1426, 1429, 1461, 1468, 1968, 2455, 4530, 4534, 6285, 7745, 7751, 8026, 8127, 9055, 9058, 10158, 10159
\tl_gset_eq:NN 81, 1049, 1050, 1055, 1056, 2578, 3070
\tl_head:N 4127, 4707, 4938, 4956
\tl_head:n 5683, 5693, 5699, 6816
\tl_if_empty:NTF 298, 557, 1432, 1480, 1489, 1562, 1582, 1820, 1850, 1852, 1882, 1886, 1916, 1918, 1931, 2282, 2293, 2318, 3000, 3117, 3149, 3189, 3318, 3322, 3585, 3820, 3889, 3991, 3994, 3997, 4137, 4344, 4347, 4350, 4358, 4420, 4488, 4491, 4494, 4502, 5255, 5267, 5314, 5398, 5454, 5497, 5513, 5548, 5566, 5725, 5893, 6152, 6156, 6160, 6165, 6264, 6273, 6279, 6476, 6489, 6506, 6523, 6626, 6662, 7107, 7195, 7356, 7405, 7641, 7643, 7646, 7649, 8011, 8589, 8733, 8811, 8834, 8941, 8944, 8950, 8996, 9001, 9041, 9044, 9047, 9052, 9152, 9212, 9260, 9349, 9365, 9473, 9475, 9478, 9510, 9512, 9514, 9522, 9611, 9613, 9616, 9644, 9646, 9648, 9656, 9762, 10053, 10109, 10116, 10127, 10132, 10135, 10185, 10212
\tl_if_empty:nTF . . . 265, 274, 362, 886, 1069, 1071, 1075, 1079, 1081, 1097, 1102, 1131, 1215, 1273, 1291, 1384, 1441, 1930, 1977, 2099, 2259, 2687, 2976, 2996, 3008, 3062, 3185, 3245, 3622, 4026, 4067, 4108, 4125, 4742, 4745, 4791, 4801, 4891, 4893, 4977, 4993, 5002, 5055, 5089, 5153, 5174, 5207, 5342, 5362, 5645, 5930, 6090, 6468, 6483, 6496, 6569, 6603, 6708, 6756, 7090, 7211, 7352, 7401, 7415, 8139, 8651, 8660, 9742, 9766
\tl_if_empty_p:N 288
\tl_if_eq:NNTF . . 943, 1841, 4707, 4938, 4956, 7127, 9705, 9706, 9715
\tl_if_eq:NnTF 8761, 8956, 8959, 8977, 8982, 9886, 9891, 9905, 9910, 9927
\tl_if_eq:nnTF . . 2470, 3159, 3167, 4653, 5693, 5699, 5754, 5792, 6834
\tl_if_exist:NTF 307, 334, 347, 509, 548, 560, 656, 887, 1011, 1064, 1068, 1078, 1190, 1329, 1504, 1514, 1816, 2142, 2505, 2509, 2513, 2526, 6906, 6911, 8025, 8283, 9870
\tl_if_in:NnTF 2716, 2720, 2738
\tl_item:nn 5685
\tl_map_inline:nn 314, 318
\tl_new:N 1446, 1449, 1971, 2253, 2490, 2585, 2590, 2608, 2609, 2623, 2981, 4526, 4574, 4575
\tl_put_left:Nn 4598, 5466, 6746, 6747, 8923, 10102
\tl_put_right:Nn 1053, 3278, 3282, 3910, 3911, 4580, 4597, 5003, 5101, 5155, 5172, 5173, 5183, 5184, 5197, 5443, 6378, 6384, 6396, 6405, 6681, 6686, 6689, 6778, 6786, 6791, 6799, 6865, 7064, 9710, 9722, 10170, 10171, 10172
\tl_set:Nn 90, 91, 93, 94, 295, 308, 492, 497, 499, 536, 537, 812, 813, 815, 816, 1012, 1148,

1248, 1251, 1302, 1305, 1330, 1345, 1354, 1571, 1718, 1728, 1777, 1837, 1868, 1921, 1950, 1956, 2325, 2358, 2372, 2385, 2447, 2478, 2525, 2790, 2796, 2797, 2798, 2870, 2930, 3005, 3134, 3161, 3169, 3210, 3225, 3338, 3349, 3350, 3351, 3352, 3353, 3359, 3418, 3419, 3443, 3444, 3485, 3486, 3509, 3510, 3564, 3570, 3574, 3744, 3775, 3811, 3849, 3874, 4001, 4143, 4145, 4146, 4147, 4148, 4149, 4183, 4185, 4186, 4187, 4188, 4189, 4244, 4246, 4247, 4248, 4249, 4250, 4309, 4354, 4400, 4401, 4402, 4403, 4404, 4458, 4498, 4529, 4531, 4538, 4550, 4551, 4552, 4571, 4587, 4592, 4593, 4594, 4599, 4675, 4702, 4720, 4756, 4757, 4761, 4764, 4771, 4772, 4808, 4848, 4922, 4928, 4981, 5007, 5010, 5013, 5036, 5037, 5043, 5052, 5057, 5096, 5097, 5101, 5154, 5156, 5164, 5166, 5169, 5175, 5177, 5180, 5192, 5193, 5208, 5209, 5211, 5212, 5378, 5384, 5397, 5403, 5406, 5409, 5431, 5453, 5717, 5718, 5726, 5758, 5760, 5767, 5803, 5815, 5817, 5848, 5939, 5940, 5956, 5957, 5971, 5972, 5985, 5986, 5999, 6000, 6049, 6071, 6072, 6234, 6265, 6274, 6298, 6314, 6317, 6326, 6329, 6335, 6342, 6361, 6464, 6466, 6469, 6471, 6665, 6672, 6678, 6717, 6731, 6732, 6744, 6783, 6796, 6808, 6831, 6857, 6879, 6880, 6917, 6918, 6960, 6974, 7079, 7108, 7128, 7279, 7286, 7640, 8004, 8010, 8032, 8252, 8253, 8257, 8258, 8262, 8263, 8266, 8267, 8270, 8271, 8274, 8275, 8278, 8279, 8878, 8950, 8994, 9025, 9026, 9052, 9092, 9393, 9457, 9488, 9493, 9508, 9566, 9600, 9626, 9627, 9642, 9704, 10146, 10152, 10153, 10207, 10208, 10209, 10215, 10280

`\tl_set_eq:NN` 68, 253, 364, 366, 371, 908, 1028, 1039, 1041, 1203, 1320, 1889, 2292, 2335, 2367, 2395, 2544, 2795, 2998, 3187, 3745, 3746, 3747, 3748, 3776, 3777, 3778, 3779, 3835, 3850, 4275, 4284, 4418, 4433, 4854, 4926, 5016, 5041, 5318, 5707, 5765, 6048, 6252, 6319, 6331, 6355, 6501, 6503, 6507, 6533, 6535, 6677, 7189, 7354, 7403, 7479, 8949, 8954, 8955

`\tl_set_rescan:Nnn` 466

`\tl_tail:n` 2687, 4817, 5669

`\tl_to_str:n` 68, 74, 104, 107, 128, 131, 143, 166, 173, 235, 315, 319, 320, 426, 439, 845, 987, 1013, 1014, 1024, 1087, 1096, 1098, 1100, 1101, 1103, 1214, 1216, 1218, 1273, 1279, 1285, 1355, 1383, 1385, 1387, 1388, 1391, 1467, 1796, 1797, 1798, 1799, 1801, 1968, 1998, 2097, 2435, 2436, 2437, 2440, 2441, 2442, 2443, 2483, 2484, 2500, 2598, 2618, 2703, 2717, 2721, 2739, 2755, 2763, 3055, 3177, 3211, 3368, 3536, 3556, 3586, 4019, 4020, 4021, 4022, 4047, 4218, 4714, 4734, 4738, 4899, 4957, 5035, 5108, 5122, 5144, 5163, 5423, 5644, 5659, 6358, 6426, 6452, 6716, 6729, 6818, 6856, 7175, 8027, 8028, 8033, 8034, 8127, 8128, 8859, 9713, 9723, 9745, 9761, 9775, 9777, 9814, 9856

`\tl_trim_spaces:N` 84

`\l_tmpa_tl` 80, 81, 83, 1033, 1195, 1197, 1305, 1319, 1320, 2149, 2454, 2455, 4805, 5164, 5172, 5173, 5175, 5183, 5184, 5365, 5371, 5376, 5378, 5382, 5384, 5387, 5396, 5398, 5403, 5406, 5409, 5803, 5820, 6388, 6822, 6858, 6865, 6872, 7640, 7641, 9508, 9522, 9642, 9656, 9704, 9705, 9706, 9715, 10166, 10170, 10178

`\l_tmpb_tl` 1302, 1310, 1314, 5365, 5371, 5373, 5387, 5397, 5401, 10167, 10171, 10179

`tmpc` commands:

`\l_tmpc_tl` 10168, 10172, 10180

`\to` 8074, 8075, 8084

`\today` 8679, 10270

`\TODO` 4792, 8759

`token` commands:

`\c_math_subscript_token` *43*, *89*, 5848, 6648, 6649, 6652, 6653, 6657, 8077, 8079

`topsect` *60*

`\topsep` 7709, 9065, 9339, 9358, 9386, 9551, 9945, 9964, 10041

`\trueFfalseT` 9680

`\trueTfalseF` 9675

`\type` *76*, *119*

U

`\underbrace` 6110, 6112

`\univ` *56*

`\unless` 8350

`\uppercase` 149, 1940, 5950

`\url` 1420

use commands:
\use:N 375, 506, 510, 512, 544, 549, 551, 558, 561, 563, 874, 881, 1002, 1563, 1565, 1818, 1865, 1947, 2532, 3822, 7595, 7603, 7619, 7681, 7699, 7796, 7809, 7820, 7822, 7851, 7860, 7909, 7926, 8284, 8290
\use:n 329, 2088, 2173, 2189, 2204, 3998, 4351, 4495, 4517, 4908, 5023, 5433, 5469
\use:nn 93, 304, 474, 477, 874, 881, 1565, 1866, 1948, 2481, 3004, 3121, 3219, 3783, 3816, 3884, 4059, 4288, 4672, 4695, 4699, 4975, 5049, 5145, 5479, 5669, 5720, 5810, 5961, 6034, 6236, 6256, 6589, 6619, 6964, 7070, 7071, 7272, 7383, 7417, 7453, 7458, 7465, 7468, 7875, 8737, 9094, 9853, 10148
\use_i:nn 3121, 5023, 10171
\use_i:nnn 867, 1221
\use_i:nnnn 1397
\use_i:nnnnn 1107
\use_ii:nn 3109, 3127, 5066
\use_ii:nnn 870, 874, 1224, 1227
\use_ii:nnnn 1400
\use_ii:nnnnn 1110
\use_iii:nnn 877, 881, 1230, 1234
\use_iii:nnnnn 1113
\use_iv:nnnn 1413, 1416
\use_iv:nnnnn 1116
\use_none:n 7736, 7741
\use_none:nn 7081
\use_v:nnnnn 1119
\usebox 8540
\usemodule . *16*, *21*, *22*, *25*, *35*, *43*, *95*, *96*, *98*, *124*, *128*, *145*, 416, 423, 2882
\usepackage 7, 22, 2088, 8364
\useSGvar *63*, *112*, 8752
\usestructure *98*, *156*, 7135
\usetheme 8364
\usetikzlibrary *82*, *140*, 2096, 10325

V

\varbind *41*, *97*, *102*, *157*, 7309, 7324, 7390
\vardef *33*, *41*, *97*, *98*, *148*, 4257, 7417, 7453, 7458, 7465, 7468, 7875
\varemph *105*, *151*, 5859
\Varname *152*, 5966
\varname *152*, 5966
\varnotation *97*, *153*, 6241
\varref *152*, 5966
\varseq *38*, *98*, *149*, 4407
\vbox *144*, 1863, 1944, 2018, 2031, 7598, 7811, 8006, 8451, 8546, 8561, 9134, 9194, 9242, 9293, 9459, 9490
vbox commands:
\vbox_set:Nn 2666
\vdash 8103
\vfill 8288, 9785, 9793, 10084
\vM *46*
\vMs *48*
\vop *39*, *41*
\vphantom 9777
\vrule 8006, 8608, 9459, 9490
\vskip 2031, 2032, 8006, 8607, 8609, 9459, 9490
\vspace 9787

W

\wff *19*
\withbrackets *94*, *155*, 6915, 6926

X

\xdef . . 9832, 9884, 9902, 9949, 9968, 10082
\xspace *139*, 1504, 1514, 3866, 3867, 4609, 4610

Y

\yesFnoT 9670
\yesTnoF 9665
\yield *158*, 7587, 7988